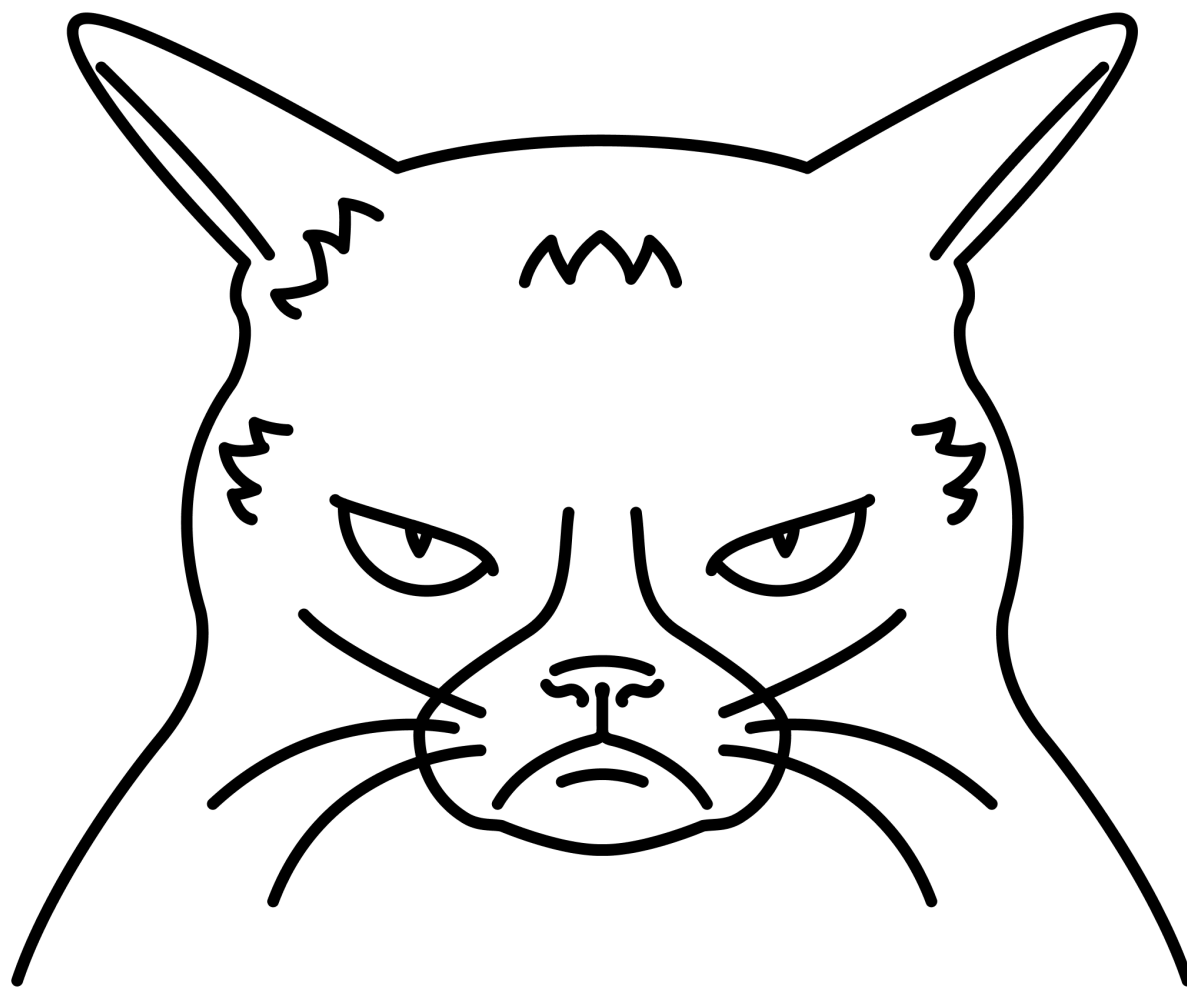


EARLY ACCESS

The Impatient Programmer's Guide to **Bevy and Rust**

Build a 2D Game From Scratch



Febin John James

The Impatient Programmer's Guide to Bevy and Rust : Build a 2D Game from Scratch

Febin John James

Table of Contents

Chapter 1: Let There Be a Player

Setup Instructions

Thinking in Systems

Setup System

Setting up a Camera

Registering our setup function

Setting up the Player

Implementing Player Movement

Registering the Movement System

Adding Sprite Graphics

Refactoring Code

Building the Player Module

Defining Player Directions

Animation System Components

Spawning the Player

Movement System

Animation Implementation

Creating the Player Plugin

Final Integration

Chapter 2: Let There Be a World

Procedural Generation

From Theory to Implementation

Setting Up Our Toolkit

Bevy Procedural Tilemaps

The Map Module

Creating SpawnableAsset

Loading Sprite Assets

Adding the Dirt Tile

Connecting the Tilemap to Asset Loading

Creating the TilemapHandles Struct

Loading the Atlas from Disk

Converting Sprite Names to Renderable Sprites

From Tiles to Models

How Models Expose Sockets

Building Models

The TerrainModelBuilder

Building the Foundation

Building the Dirt Layer

Generating Dirt

Adding the Grass Layer

Step 1: Adding Grass Sprites to the Tilemap

Step 2: Adding Grass Sockets

Step 3: Building the Grass Layer Rules

Step 4: Calling the Grass Layer Function

Step 5: Updating Grid Layers

Adding the Yellow Grass Layer

Step 1: Add Yellow Grass Sprites to Tilemap

Step 2: Define Yellow Grass Sockets

Step 3: Building the Yellow Grass Layer Rules

Step 4: Calling the Yellow Grass Layer Function

Step 5: Updating Grid Layers

Adding the Water Layer

Step 1: Add Water Sprites to Tilemap

Step 2: Define Water Sockets

Step 3: Building the Water Layer Rules

Step 4: Calling the Water Layer Function

Step 5: Updating Grid Layers

Adding the Props Layer

Step 1: Add Props Sprites to Tilemap

Step 2: Define Props Sockets

Step 3: Building the Props Layer Rules

Step 4: Calling the Props Layer Function

Step 5: Updating Grid Layers

Congratulations!

Wait, Just one more thing!

Chapter 3: Let The Data Flow

The Problem with Hardcoded Characters

Maintenance Nightmare

Data-Driven Design

Setting Up the Characters Module

Character Schema

Setting Up the Config

The Animation Engine

Direction Tracking

Tracking Animation State

Frame Calculation

Animating Characters

The Movement System

Reading Player Input

Calculating Movement Speed

The Player Marker

The Movement System

Handling Jump Completion

Spawning Characters

Resources for Character Management

Creating the Texture Atlas Layout

Spawning the Player Entity

Character Switching

Bringing It All Together

Adding the RON Asset Loader

Creating the Characters Plugin

Plugging Into the Game

Chapter 4: Let There Be Collisions

Systems Running When They Shouldn't

Systems That Run Only When Needed

State-Based Schedules

Implementing Game States

Defining Game States

Loading Screen

Pause Menu

State Plugin

Integrating StatePlugin

The State Pattern for Characters

Too Many Boolean Flags

The State Pattern Solution

Why This Is Better

Implementing Character States

The CharacterState Enum

Querying State with Methods

Animation Refactoring

Replace Animation System

Completing the State-Based Refactoring

Refactoring Player Input

State Machine Logic

The Main Input Handler

Handling Jump Completion

Wiring Up the New Systems

Building a Collision System

Defining Tile Types

The Collision Map

Circle Collision

Swept Collision

Building the Collision Map

Shore Conversion

Debug Visualization

Wiring It Together: The Collision Module

Integrating Collision with Map Assets

Updating Map Rules with Tile Types

Centralizing Configuration

Creating the Player Collider

Validating Movement Against Collision

Updating the Characters Module

Attaching Collider to the Player

The Illusion of Depth

Creating the Rendering Module

Integrating Depth Sorting

Adjusting Water Generation

What We Built

Chapter 5: Let There Be Pickups

Building the Pickup System

The Pickup System

Configuring Pickup Radius

Building the Inventory System

Defining Item Types

Display Names

The Pickable Component

The Inventory Resource

Adding Items to Inventory

Pickup Detection System

Getting the Player Position

Collecting Items to Process

Processing Collected Items

Wiring It Together

Integrating the Plugin

Making Items Pickable

Updating the Asset Spawner

Making the Spawner Attach Components

Marking Props as Pickable

Zooming In and Following the Player

Updating the Config

Updating Map Generation

Building the Camera System

Camera Module

Integrating the Camera

Chapter 6: Let There Be Particles

Let's Give Your Players Magic Powers

Understanding Particle Systems

Building the Particle System

Particle Components

Particle Emitter

Particle Configuration

Particle System Updates

The Shader

Custom Shader Material

Particles Module Plugin

Creating the Combat Module

Power Types

Player Combat Component

Combat Systems

Combat Module Plugin

Putting it all together

Chapter 7: Let There Be Enemies

What Makes an Enemy Different from a Player?

Thinking in Components

Creating the Enemy Module

Enemy Components

Making Enemies Follow You

Understanding A* Pathfinding

The EnemyPath Component

Extending CollisionMap for Pathfinding

Understanding the astar Function

Making Enemies Attack

Fixing Player Spawning

Updating the Plugin

Adding Enemy Configuration

Spawning Enemies

The Enemy Plugin

- Fixing Enemy Depth Sorting
- Preventing Entity Stacking
- Integrating the Enemy Module

Optimizing Debug Builds

Chapter 8: Let There Be Damage

The Health Component

Making Powers Deal Damage

- Creating Invisible Hitboxes
- Spawning Projectile Hitboxes
- Moving Projectiles Forward
- Understanding the Observer Pattern
- Creating Combat Events
- Detecting Projectile Hits
- Creating Observer Handlers
- Updating Health to Trigger Death Events

Health Bars

- Spawning Health Bars
- Updating Health Bars
- Adding Health to Characters
- Registering the Combat Systems

Adding Game Over and Restart

- Adding the GameOver State
- Creating the Game Over Screen
- Detecting Player Death
- Registering Game Over Systems

Chapter 9: Let The World Scale

Why WFC Struggles with Large Grids

The Multi-Chunk Strategy

Border Pre-Seeding

Making Chunks Overlap

The Need for Async Generation

- Adding Chunk Configuration
- Rewriting the Generator for Async
- Imports and Resources
- The Setup System
- The Polling System
- Background Generation Logic
- Generation Order and Backtracking
- Pre-Seeding Borders
- The Core Generation Function
- Skipping Duplicate Overlap Tiles

Removing the Plugin-Based Approach

State Management

The Loading Screen

Updating the Collision Map

Updating Character Depth Sorting

Chapter 10: Let There Be Memory

What We Need to Save

Facing Direction

Power Types

Character State

Tile Types

Item Types

Building the Save Data Structure

File Path Functions

Checksum Function

Building the Save/Load UI

Bevy UI

Spawning the UI

Handling Button Clicks

Implementing Save

Implementing Loading

Implementing Load Logic

Integrating the Save System

Adding Game States

Creating the Main Menu

Updating the Pause Menu

Updating Game Over

Wiring Up the State System

Final Integration

Chapter 11: Let There Be Sound

Building the Audio Module

Loading Audio Assets

Managing Background Music

Sound Effects with Observers

Audio Plugin

Triggering Sound Effects

The Player Jump

Casting Spells

Combat

Inventory

UI Buttons

Wiring It Up in Main

Packaging

Copy Assets at Build Time

Dynamic Asset Paths

Removing Debug Assertions

Chapter 12: Let There Be Networking

Thinking in Systems for Multiplayer

System 1: Player Identification (Identity & Auth)

System 2: Storing Player State (Persistence)

System 3: Real-Time Updates (State Synchronization)

System 4: Game Logic Validation (Server Authority)

SpacetimeDB

Installing SpacetimeDB

Installing the CLI

Upgrading to v2

Upgrading Rust

Starting a Local Server

Creating the Server Module

Building Blocks of SpacetimeDB

Defining the Player Table

Generating Human-Readable Usernames

Writing the Reducers

Module Startup

A Client Connects

A Client Disconnects

Choosing a Custom Name

Building the Module

Publishing to Your Local Server

Viewing the Logs

Testing with SQL

Testing Reducers from the CLI

Connecting the Bevy Client

Generate Client Bindings

Create the NetworkPlugin

Mini System 1: Connecting

Mini System 2: Processing Messages Every Frame

Mini System 3: Multiplayer Connection Screen

Mini System 4: Back to Main Menu

Mini System 5: Cleanup on Return to Menu

Wiring Up

Chapter 13: Let There Be a Shared World

Multiplayer Map Generation

The Problem

The Solution

Setup World Configuration

Rebuild and Republish

Regenerate Client Bindings

Create the WorldSeed Resource

Make WFC Deterministic

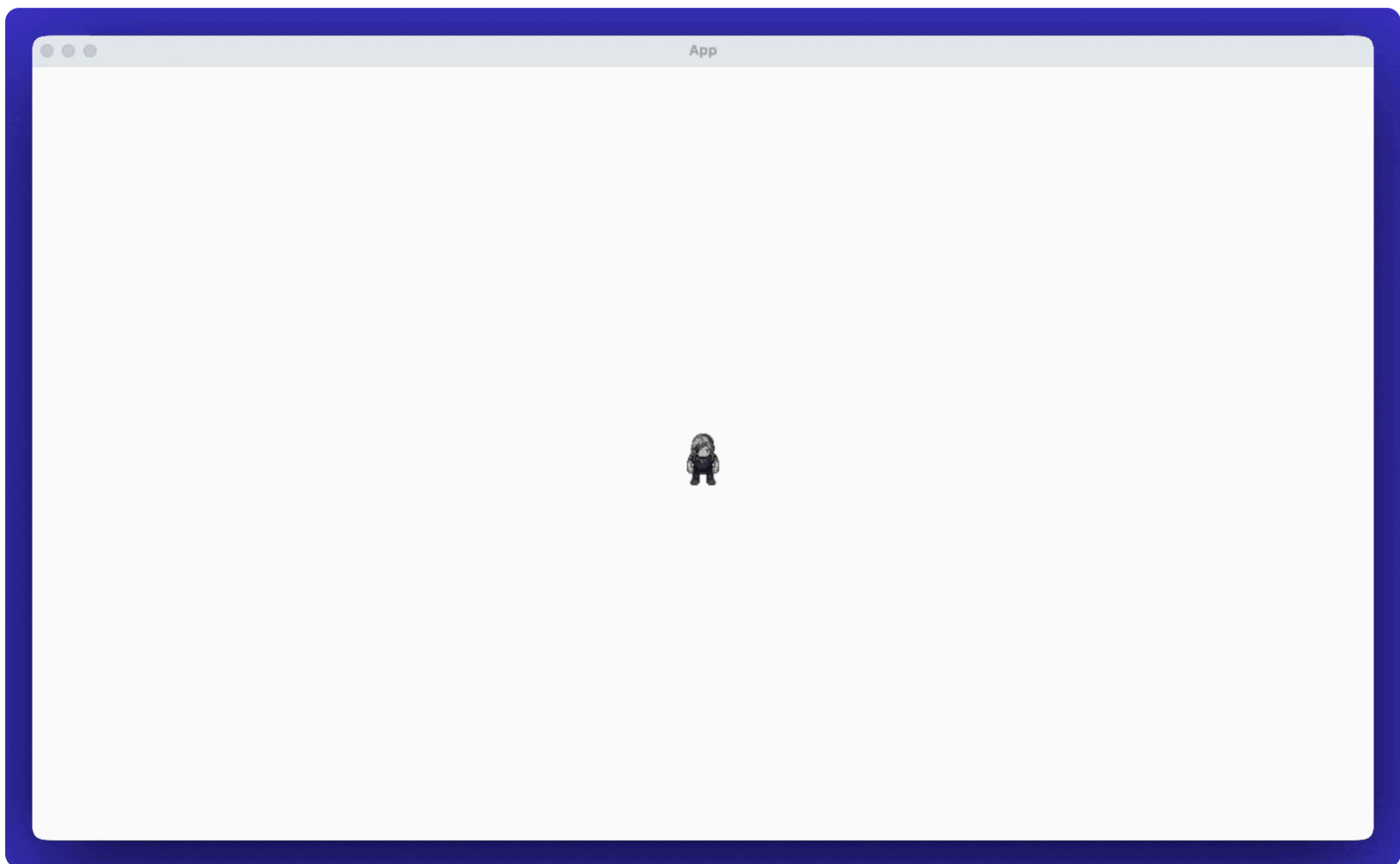
Fetching WorldConfig

Shared World Generation

The Impatient Programmer's Guide to Bevy and Rust: Chapter 1 - Let There Be a Player

This is chapter 1 of my rust-bevy tutorial series “The Impatient Programmer’s Guide to Bevy and Rust: Build a 2D Game from Scratch”. I will be going in-depth into bevy and game development in this series, also cover NPCs powered by AI Agents. [Join our community](#) to be updated on new releases on this series. Source code for this chapter is [available here](#).

Here’s what you will be able to achieve by the end of this tutorial.



Setup Instructions

If you haven’t setup rust yet, please follow the [official guide](#) to set it up.

1. Create a fresh project:

```
cargo new bevy_game
cd bevy_game
```

2. Open `Cargo.toml` and add Bevy:

```
[dependencies]
bevy = "0.19"
```

I am going to assume you have programming experience in any one language like javascript or python.

Thinking in Systems

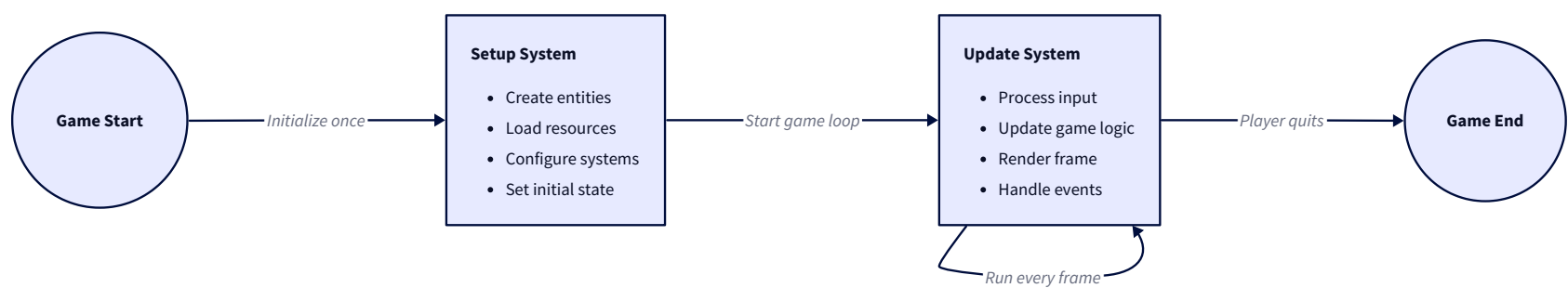
What do we need to build a simple game that allows player to move from keyboard input?

- **World System:** Create a game world where the player can move.
- **Input System:** Monitor keyboard input and translate it into player movement.

We see the above pattern very common in building games, the pattern of create/setup and the pattern of monitoring game state changes and updating the game world entities.

- **Setup:** Spawn enemy with different behaviors, load enemy sprites and animations, configure enemy health and damage values.
- **Update:** Move enemies toward the player, check if enemies are hit by bullets, remove dead enemies and spawn new enemies at intervals.

At the core of Bevy, we have this simple **setup and update system**. This foundational pattern is what makes Bevy both powerful and easy to understand - once you grasp this concept, building with bevy becomes easier.



Setup System

Imagine a function called setup, that gives you “commands” as an argument, and it give you spawn ability to create or spawn anything you want. Well, bevy gives you exactly that.

```
// Pseudo code, doesn't compile
fn setup(mut commands: Commands) {
    // Create anything you want
    commands.spawn()
}
```

What's mut?

`mut` is short for mutate. In Rust we need to explicitly mention that we are planning to change this value. By default, Rust treats declared values as read only. `mut` tells Rust we plan to change the value. Rust uses this knowledge to prevent a whole class of bugs.

Why do I need to tell Rust I'm going to change something?



Rust has rules to prevent you from making mistakes.



But other languages don't make me do this!

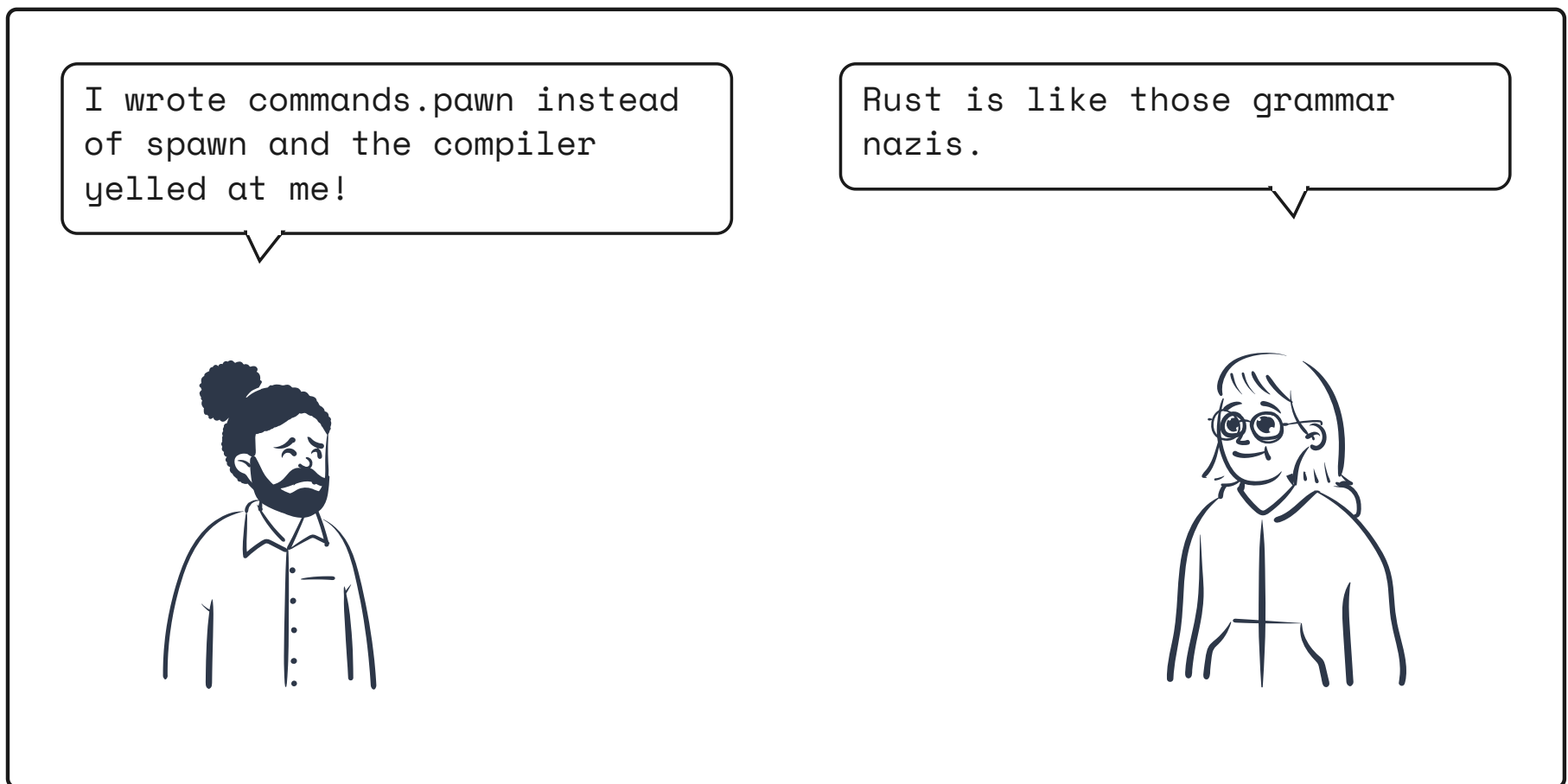


Exactly! That's why they let you shoot yourself in the foot!.



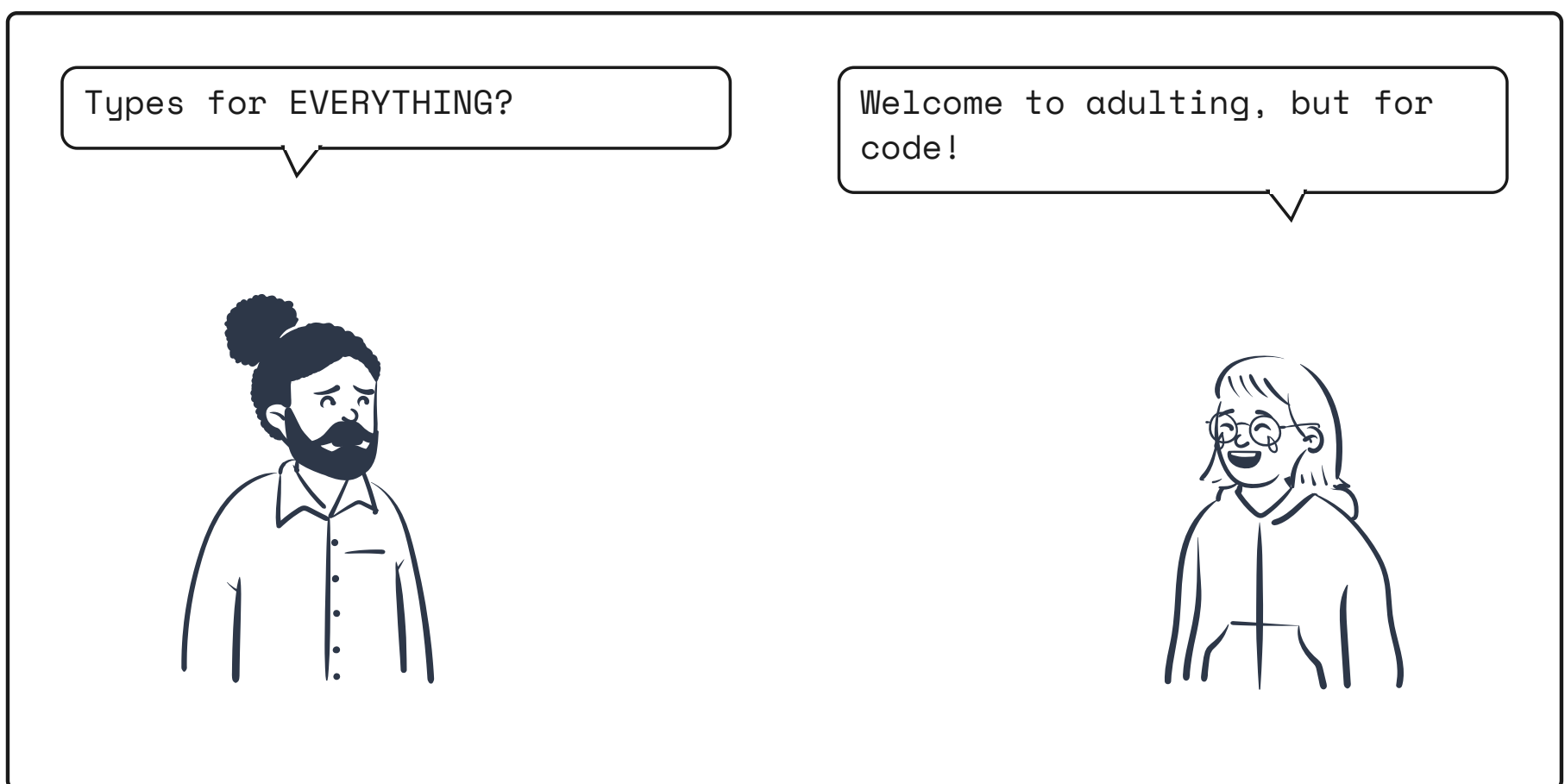
And what's this `mut commands: Commands`?

It's from bevy library, that allows you to add things to your game world. Rust likes explicit types, so `:Commands` lets the compiler know this parameter is Bevy's command interface. Skip the hint and Rust can't be sure what shows up, so it blocks the build.



What's a type?

A type tells Rust what kind of value you are handling—numbers, text, timers, Bevy commands, and so on. Once the compiler knows the type, it can check that every operation you perform on it actually makes sense.



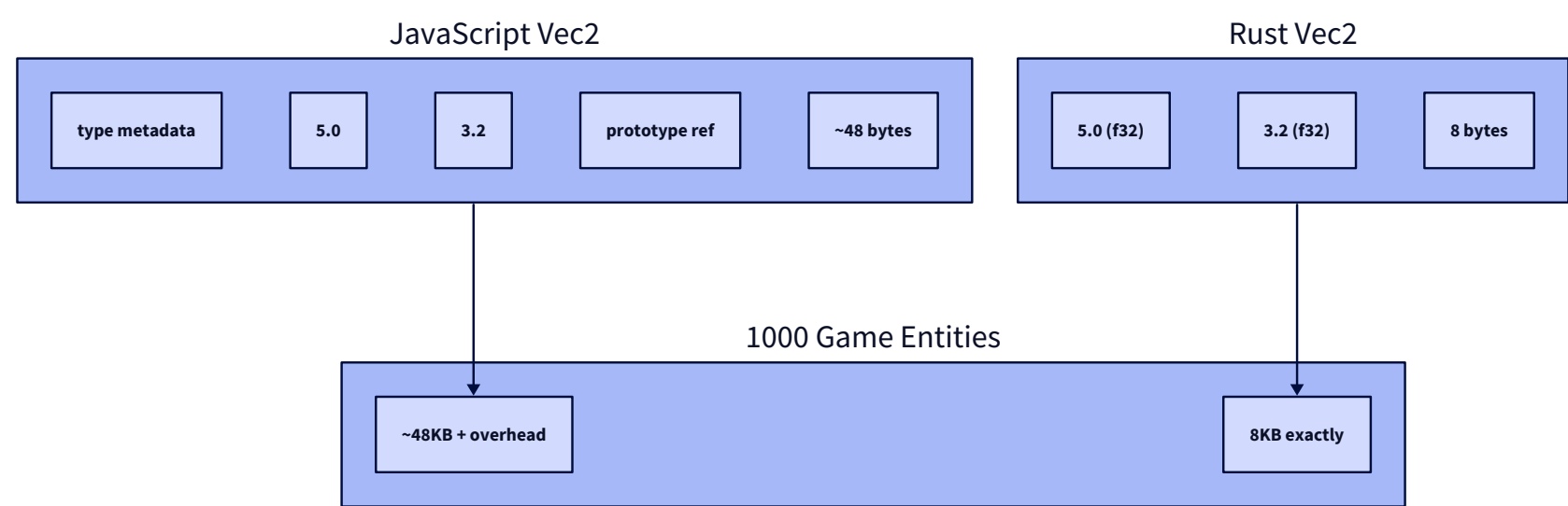
Isn't this too much work?

It prevents mistakes and helps you with performance, ex: If you try to add a string to a number, the compiler stops you before the game runs. On top of that, knowing the exact type lets Rust pack data tightly.

A `Vec2` is just two numbers, so Rust stores exactly two numbers, no surprise extra space, which keeps things fast when you have thousands of game entities. These helps your game to be memory efficient.

When you create a `Vec2` object in JavaScript or Python, you're not just storing two numbers. The runtime adds type metadata, property information, and prototype references - turning your simple

8-byte data structure into ~48 bytes of memory.

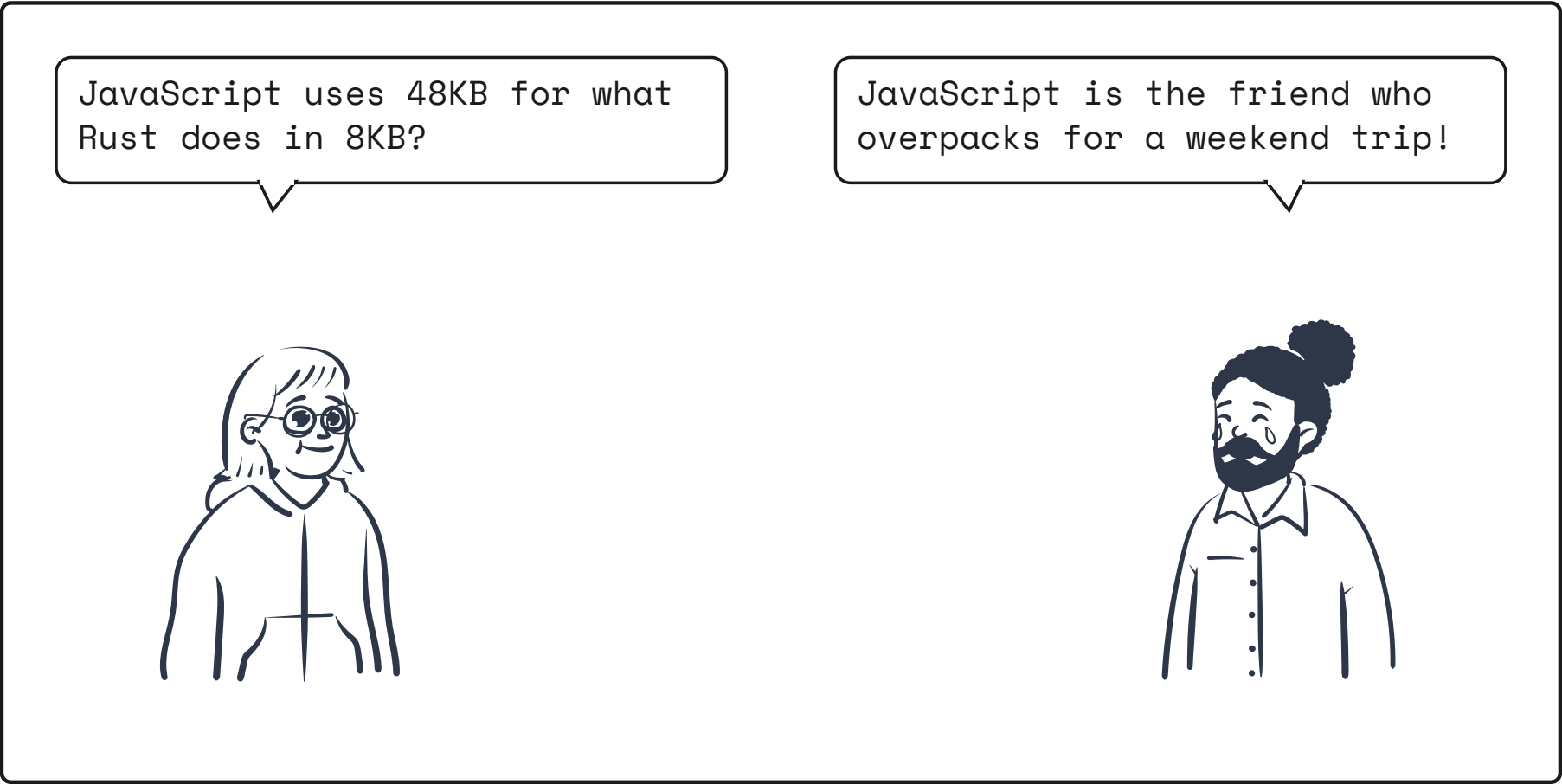


Rust’s type system works at compile time. When you declare a Vec2 struct with two f32 fields, that’s exactly what gets stored - just 8 bytes, no extra metadata. The compiler already knows the types, so no runtime type information is needed.

With 1000 game entities, this difference becomes dramatic:

Language	Memory Usage	Overhead
Rust	8KB	None
Dynamic language	~48KB+	6x overhead

This isn’t just about memory usage - it’s about performance. Smaller, predictable memory layouts mean better CPU cache utilization, which directly translates to faster frame rates in games where you’re processing thousands of entities every frame. The compiler catches type errors before your game runs, and the tight memory packing keeps it running fast.



Setting up a Camera

What should we setup first?

We need a camera because nothing shows on screen without one. The world can exist in data, but the camera decides what actually gets drawn.

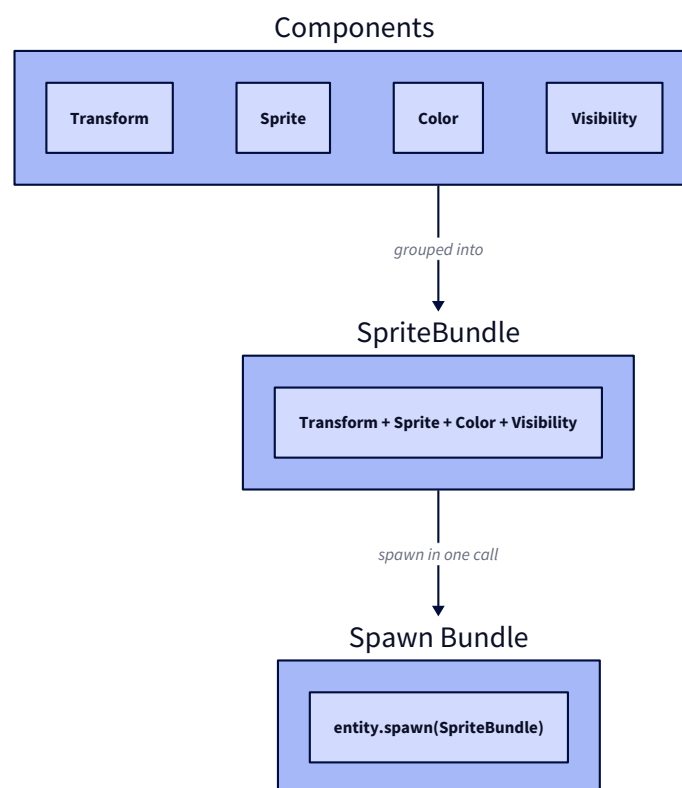
```
//Pseudo code, don't use this yet
fn setup(mut commands: Commands) {
    commands.spawn(Camera2d)
}
```

What's Camera2d?

`Camera2d` is Bevy's built-in 2D camera bundle. Spawn it and you get a ready-to-go view of your 2D scene.

What's a bundle?

A bundle is just a group of components you often spawn together. For example, Bevy ships a `SpriteBundle` that packs position, texture, color tint, and visibility; spawning that bundle gives a sprite entity in one call.



Registering our setup function

We need to register our setup function with bevy to be triggered on Startup.

Update your `src/main.rs` with the following code.

```
// Replace your main.rs with the following code.
use bevy::prelude::*;

fn main() {
    App::new()
        .add_plugins(DefaultPlugins)
        .add_systems(Startup, setup)
        .run();
}

fn setup(mut commands: Commands) {
```

```
        commands.spawn(Camera2d);  
    }
```

What's `bevy::prelude`?

`bevy::prelude` is like a starter kit, things you reach for constantly when building a game—`App`, `Commands`, components, math helpers, etc.

What's `App::new()`...

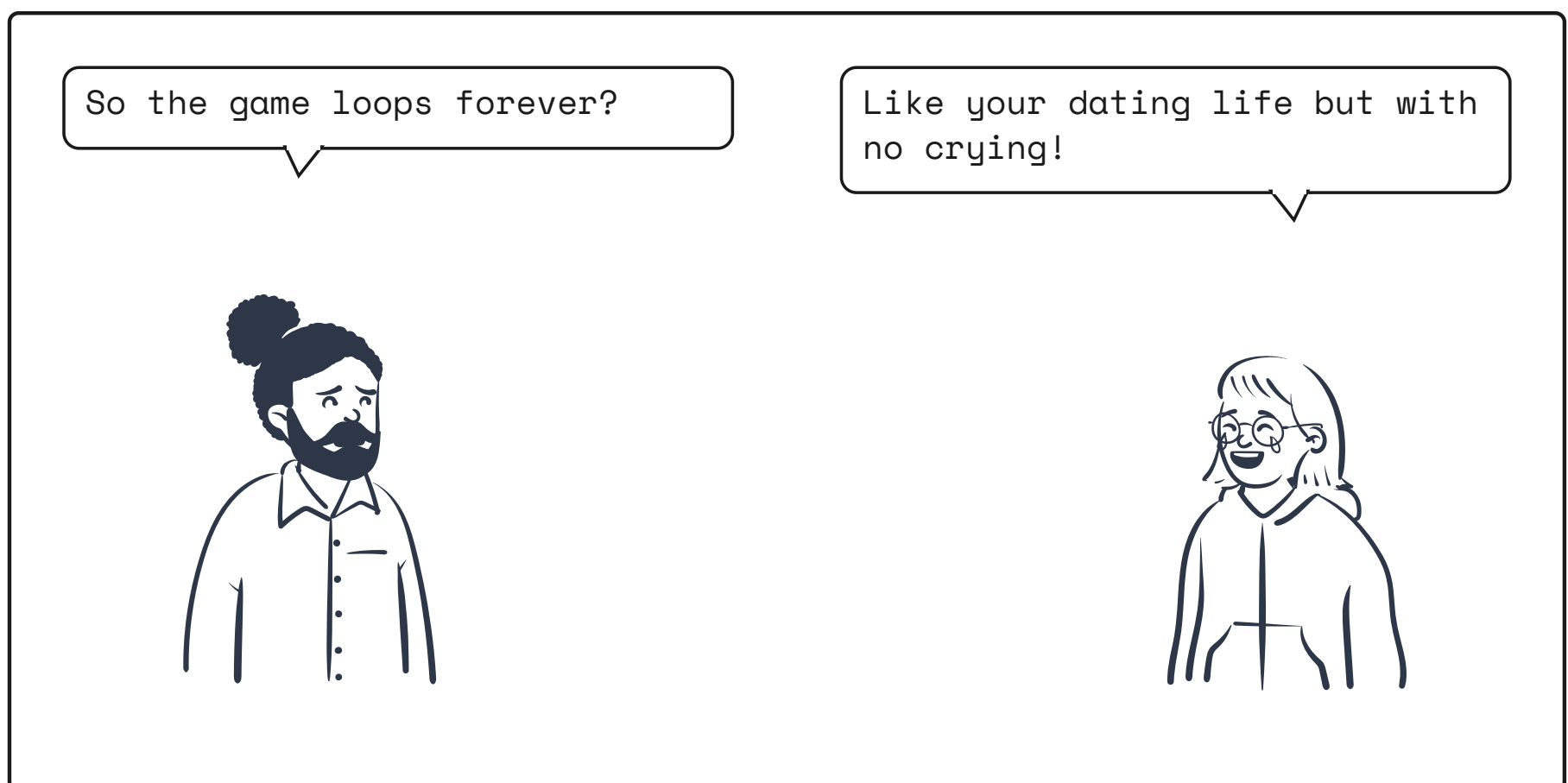
`App::new()` creates the game application, `add_plugins(DefaultPlugins)` loads rendering, input, audio, and other systems, `add_systems(Startup, setup)` registers our startup function, and `run()` hands control to Bevy's main loop.

Why register a startup function? What does it do?

Startup systems run once before the first frame. We use them to set up cameras, players, and other things that should exist as soon as the window opens.

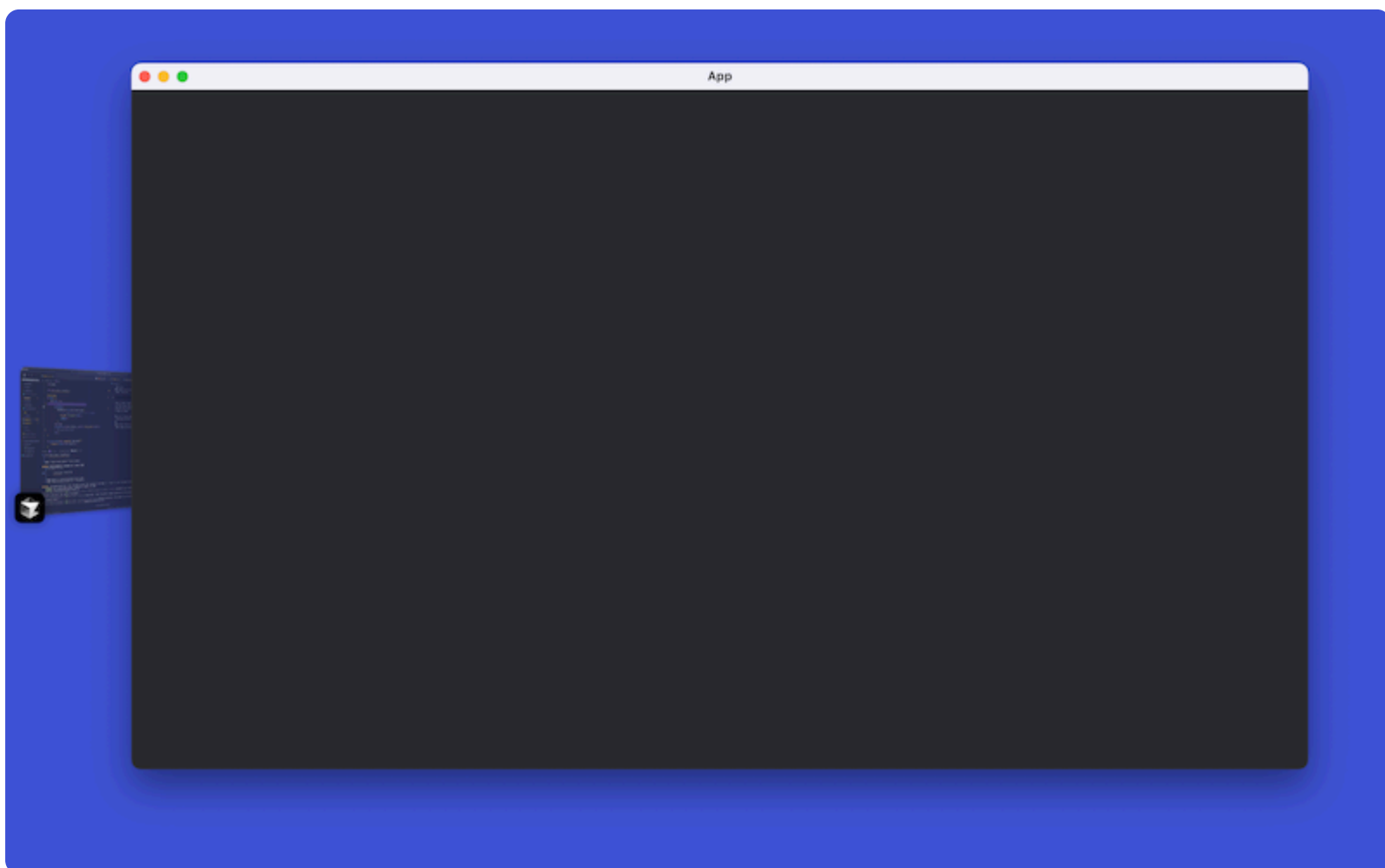
What's Bevy main loop?

After startup, Bevy enters its main loop: it polls input, runs your systems, updates the world, and renders a frame. That loop repeats until you quit the game.



Let's run it

```
cargo run
```

I coded for an hour and got a blank screen!



Achievement unlocked:
Professional developer
experience!



A blank screen? Yup, we have only setup the camera, now let's add our player.

Setting up the Player

Now let's create our player! Remember our **Setup System** and **Update System** from earlier? Well, in Bevy, everything is an **Entity** with **Components** that systems can work with.

Think of an entity as a unique ID (like a social security number) and components as the data attached to it. For our player, we need components like movement speed, health, or special abilities. In Rust, the perfect way to create these components is with a **struct**.

`struct` is one of the core building blocks of rust. It groups similar data together. Here we declare an empty `Player` struct so the type itself acts as a tag we can attach to the player entity. Later we can add things like player health.

```
// Place this before main function in main.rs
#[derive(Component)]
```

```
struct Player;
```

So the struct is empty... does that mean my player is invisible?



No, it means he exist in spirit. You've made a philosophical protagonist



Why tag?

Tag marks an entity for later lookup. Because `Player` is attached to only our hero, systems can ask Bevy, “give me the entity with the Player tag” and work with just that one.

What's this `#[derive(component)]`?

`derive` tells Rust to attach the component macro code to this struct. A macro is Rust's way of generating pre-defined template code for you. `#[derive(Component)]` automatically injects the boilerplate Bevy needs, so it can store and find `Player` entities, saving us from copying the same glue code everywhere. We'll take a closer look at macros later in the series. This is the moment our player type becomes a component.

What's a component, and why should the player be a component?

A component is a piece of data attached to an entity. Position, velocity, health, and even the idea of “this is the player” all live in components. By making `Player` a component we can query for that entity later, add more components (like health or inventory), and let Bevy's systems pick out exactly the entities they need to update.

For now we will represent our character on screen using the “@” symbol. Modify the setup function.

```
// Replace existing setup function in main.rs with the following code
fn setup(mut commands: Commands) {
    commands.spawn(Camera2d);

    // Code Update Alert
    // Append the following lines to your setup function.
    commands.spawn((
        Text2d::new("@",
            TextFont {
                font_size: FontSize::Px(12.0),
                font: default(),
                ..default()
            },
        TextColor(Color::WHITE),
        Transform::from_translation(Vec3::ZERO),
        Player,
```



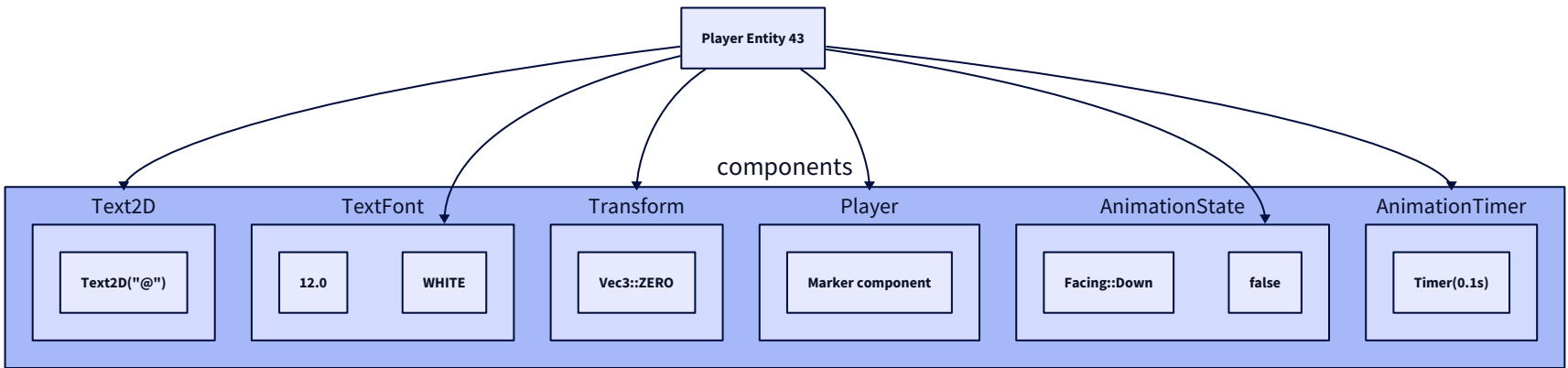
```
    ));  
}
```

`commands.spawn(( ... ))` takes that tuple and treats it as a bundle of components. This single call adds the text we want to show, its font settings, the color, the position, and the `Player` tag that identifies the entity.

What's a tuple?

A tuple is an ordered list of values written in parentheses. Rust keeps track of each position, so `(Text2d::new("@"), TextColor(Color::WHITE))` holds two values side by side without needing to create a struct.

Whats an entity?

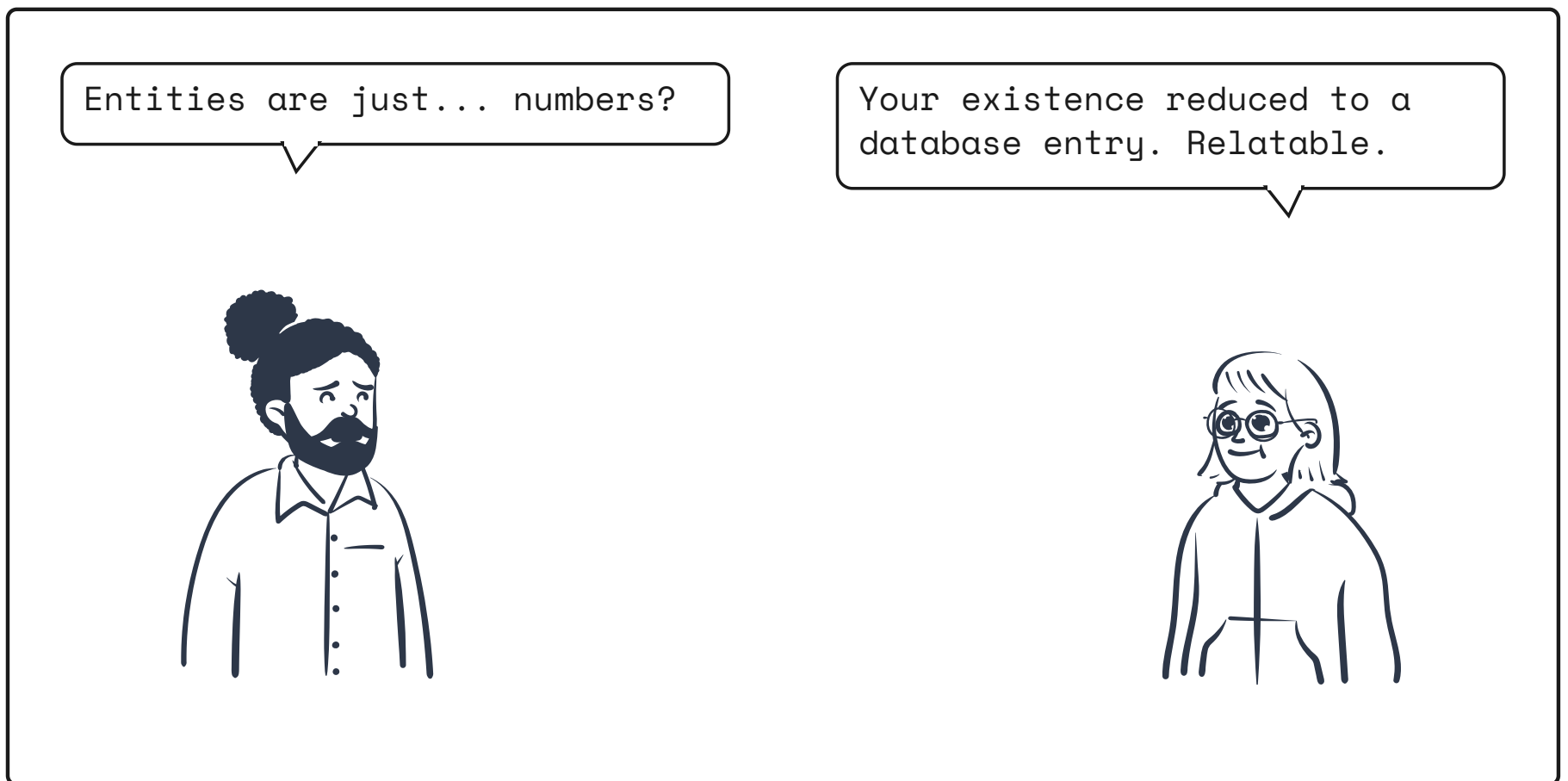


An entity is the unique ID Bevy uses to tie components together. By itself it holds no data, but once you attach components to it, that ID represents something in your game world.

Each bundle produces a new entity with a unique ID and the listed components, which you can picture like this:

Entity	Components it carries
#42	Camera2d
#43	Text2d("@"), TextFont, TextColor, Transform, Player

Once the queue flushes, those entities live in the world, ready for systems to discover them by the tags (components) they carry. We will be using this later when we want to do things like moving or animating them, or making them attack enemies, etc.



Implementing Player Movement

Now let's create our **Update System** for player movement! Think about what we need to move a player:

1. **Keyboard input** - to know which keys are pressed
2. **Time** - to make movement smooth regardless of frame rate
3. **Player position** - to actually move the player

In other game engines, you'd spend time manually connecting these systems together. But here's the magic of Bevy - you just ask for what you need in your function parameters, and Bevy automatically provides it!



Let's write our move player function.

```
// Append this code to main.rs
fn move_player(
```

```

// "Bevy, give me keyboard input"
input: Res<ButtonInput<KeyCode>>,
// "Bevy, give me the game timer"
time: Res<Time>,
// "Bevy, give me the player's position"
mut player_transform: Single<&mut Transform, With<Player>>,
) {
    let mut direction = Vec2::ZERO;
    if input.pressed(KeyCode::ArrowLeft) {
        direction.x -= 1.0;
    }
    if input.pressed(KeyCode::ArrowRight) {
        direction.x += 1.0;
    }
    if input.pressed(KeyCode::ArrowUp) {
        direction.y += 1.0;
    }
    if input.pressed(KeyCode::ArrowDown) {
        direction.y -= 1.0;
    }

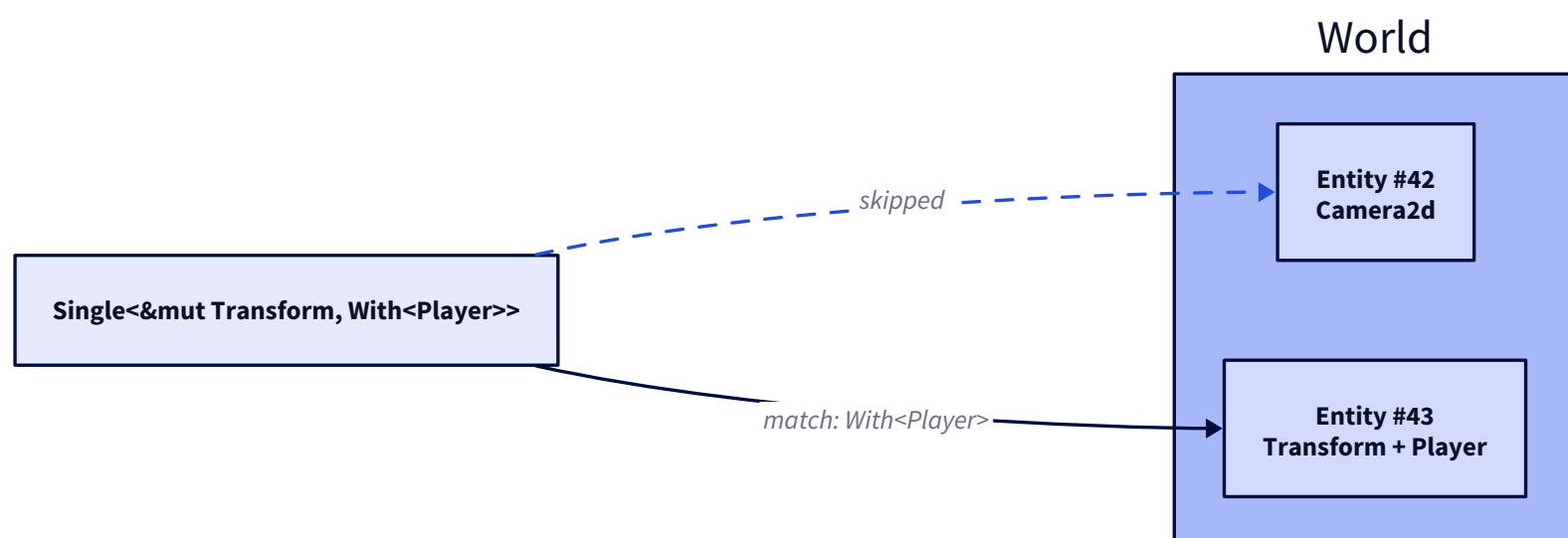
    if direction != Vec2::ZERO {
        let speed = 300.0; // pixels per second
        let delta = direction.normalize() * speed * time.delta_secs();
        player_transform.translation.x += delta.x;
        player_transform.translation.y += delta.y;
    }
}

```

What's Res?

Res or Resources are pieces of game-wide information that are not tied to any single entity. For example, `Res<Time>` gives you the game's master clock, so every system reads the same "time since last frame" value.

Explain `Single<&mut Transform, With<Player>>`



`Single<&mut Transform, With<Player>>` asks Bevy for exactly one entity that has a `Transform` component and also carries the `Player` tag. The `&mut Transform` part means we intend to modify that transform or player position (Remember, we added transform component in the setup function).

If more than one player existed, this extractor would complain, which is perfect for a single-hero game.

What's Vec2::ZERO?

`Vec2::ZERO` is a two-dimensional vector with both values set to zero: `Vec2 { x: 0.0, y: 0.0 }`. We use it as the starting direction before reading keyboard input.

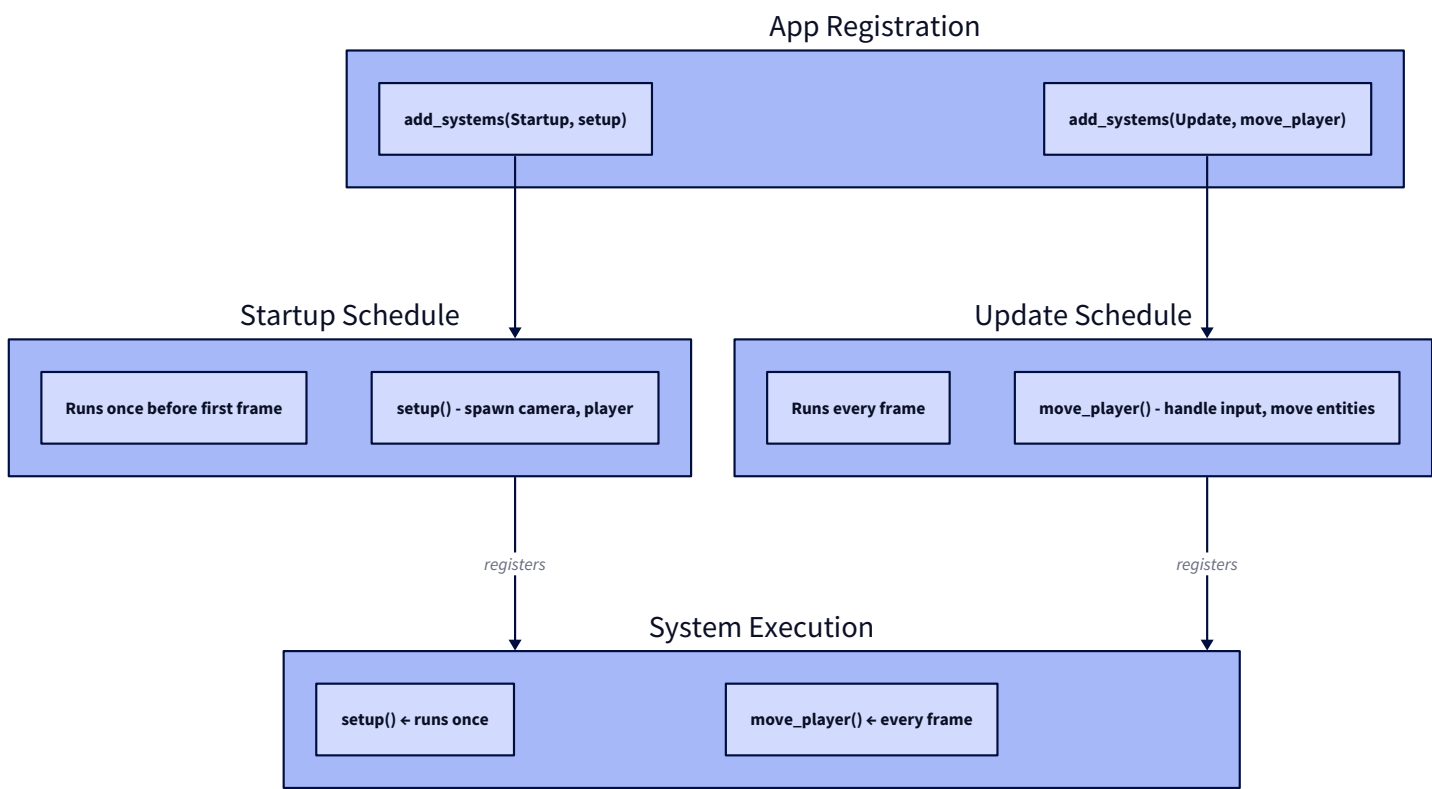
What's this KeyCode::... pattern?

`KeyCode::ArrowLeft`, `KeyCode::ArrowRight` ... are enums (will cover enums later) that represent specific keys on the keyboard. Checking `input.pressed(KeyCode::ArrowLeft)` simply asks Bevy whether that key is held down during the current frame.

We ignore zero direction so the player stands still when no keys are pressed. Once we have input, `normalize()` converts the vector to length 1 so diagonal movement isn't faster than straight movement. `speed` says how many pixels per second to move, and `time.delta_secs()` returns the frame time—the number of seconds since the previous frame—so multiplying them gives the distance we should travel this update. Finally we add that delta to the player's transform translation to move the sprite on screen.

Registering the Movement System

```
// Update main function
fn main() {
    App::new()
        .add_plugins(DefaultPlugins)
        .add_systems(Startup, setup)
        .add_systems(Update, move_player) // Line update alert
        .run();
}
```



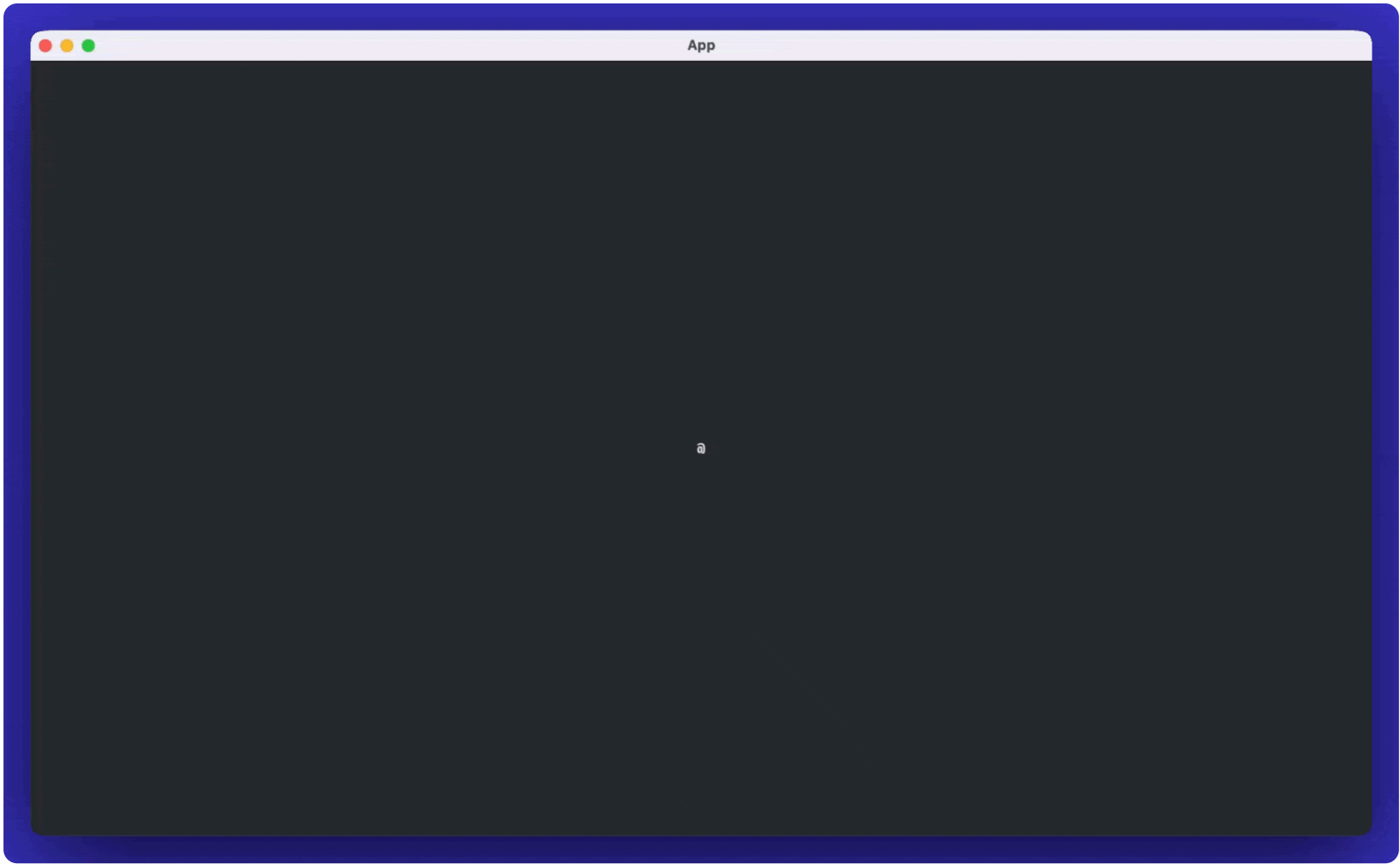
Why is move_player added as Update? What's Update?

`move_player` runs every frame, so we plug it into the `Update` schedule. `Update` is Bevy's built-in stage that fires once per game loop after startup is done.

So does systems mean functions we want bevy to execute at a particular time, like initial setup or on every game loop update?

Exactly. A system is just a Rust function you hand to Bevy with a sticky note that says “run me during Startup” or “execute me on every Update,” and the engine dutifully obeys.

Let’s run it.



Adding Sprite Graphics

We’ll use the Universal LPC SpriteSheet Generator to give our character some personality. You can remix body parts, clothes, and colors at [this link](#) and export a full spritesheet.

For this project the spritesheet is already included in the [repo](#). Drop the provided image files into `src/assets` so Bevy can find them when the game runs. You will need to create the `assets` directory inside the src folder.

Universal LPC Spritesheet Generator

Free/Libre pixel art from the [Liberated Pixel Cup](#)

You must [credit the authors of this artwork](#).

Download: Spritesheet single image (PNG) Credits (text) Credits (CSV)

Download image pack with credits and JSON (ZIP): Split by animation

Split by item Split by animation and item

Import/Export: Export to Clipboard (JSON) Import from Clipboard (JSON)

Edit: Reset all Replace Mask (pink) ☒ Match body color

View: Collapse All Expand Selected ☐ Compact Display

Search:

Filters

- Licenses: ▶
- Supported Animation ▶

Body

- Body type ▶
- Shadow ▶
- Body color ▶
- Special ▶
- Wounds ▶
- Prostheses ▶
- Wheelchair ▶
- Tails ▶
- Wings ▶
- Lizard ▶

Head

- Heads ▶
- Face ▶
- Eye Color ▶
- Other Eyes ▶
- Ears ▶
- Furry Ears ▶
- Nose ▶
- Eyebrows ▶
- Wrinkles ▶
- Beards ▶
- Mustaches ▶
- Hair ▶
- Appendages ▶
- Head Coverings ▶
- Hats ▶

Preview

Walk

1-2-3-4-5-6-7-8

Complete sprite sheet (click to zoom in, double click to zoom out):

Refactoring Code

We need to add more code and adding to `main.rs` will make hard to read and understand.

Update your `main.rs` file to the following, also create another file `player.rs`

`mod player;` pulls in the `player.rs` module, keeping our main file slim. Splitting code like this makes it easier to grow the project without one big file doing everything.

```
//main.rs
use bevy::prelude::*;

mod player;

fn main() {
    App::new()
        .insert_resource(ClearColor(Color::WHITE)) // We have updated the bg color to white
        .add_plugins(
            DefaultPlugins.set(AssetPlugin {
                // Our assets live in `src/assets` for this project
                file_path: "src/assets".into(),
                ..default()
            }),
        )
        .add_systems(Startup, setup_camera)
        .run();
}

fn setup_camera(mut commands: Commands) {
    commands.spawn(Camera2d);
}
```

`.insert_resource(ClearColor(Color::WHITE))` drops a global setting into the app so every frame starts with a white background.

`AssetPlugin` is Bevy's loader for textures, audio, and other assets. We tweak its `file_path` so it looks inside `src/assets`, which is where our sprites live.

Building the Player Module

Now that we have our basic app structure in place, it's time to bring our character to life! We'll create a dedicated `player.rs` module to handle everything related to our little adventurer. This keeps our code organized and makes it easier to add more features later.

This module will hold all the constants, components, and systems that make our player move, animate, and interact with the world.

Add following code in `player.rs`

```
// player.rs
use bevy::prelude::*;

// Atlas constants
const TILE_SIZE: u32 = 64; // 64x64 tiles
const WALK_FRAMES: usize = 9; // 9 columns per walking row
const MOVE_SPEED: f32 = 140.0; // pixels per second
const ANIM_DT: f32 = 0.1; // seconds per frame (~10 FPS)

#[derive(Component)]
struct Player; // Moved from main.rs to here
```


These constants give names to the numbers we reuse for the spritesheet and movement math: tile size, frames per row, walk speed, and how fast the animation advances. Moving the `Player` marker here keeps all player-specific types in one module.

Defining Player Directions

```
// Append these lines of code to player.rs
#[derive(Component, Debug, Clone, Copy, PartialEq, Eq)]
enum Facing {
    Up,
    Left,
    Down,
    Right,
}
```

What's an enum?

An enum lists a handful of allowed values. Our character only ever faces four directions, so the `Facing` enum captures those options in one place.

Why can't I use a struct here?

A struct would need a bunch of booleans like `facing_up:true`, `facing_left:false`, and you'd have to keep them in sync. That makes it complicated. Enum guarantees you only pick one direction.

When to decide between using enum and struct?

Use a struct when you need several fields, like a position with `x` and `y`, or player stats with health and stamina. Use an enum when you choose one option from a list, like which tool the player wields (sword, bow, wand) or which menu screen is active.

Why the purpose of adding Debug, Clone, Copy, PartialEq, Eq macros?

`Debug` lets us print the facing for logs, `Clone` and `Copy` make it trivial to duplicate the value, and `PartialEq`/`Eq` allow equality checks when we compare directions.

Why can't I copy through simple assignment, why should I add these macros?

By default Rust moves values instead of copying them, so the compiler makes you opt in. Deriving `Copy` (and `Clone` as a helper) says "it's cheap, go ahead and duplicate it." `PartialEq` and `Eq` let us compare two facings directly, which is how we detect when the player changes direction.

What do you mean by rust moves values, instead of copying?

When you assign most values in Rust, the old variable stops owning it or in other words dies. Adding `Copy` keeps both variables valid.

Why does the old variable stop owning or die when assigned?

Rust enforces that each value has a single owner so memory can be freed safely. We'll unpack the ownership rules and borrowing in later chapters.

This is a bit going over my head!

Yea, don't worry, we have many more chapters to go and as you move through them, we will tackle this. It's alright to not understand these concepts now.

Animation System Components

When we have a spritesheet with multiple frames (like our 9-frame walking animation), we need a way to control how fast those frames play. Without timing control, our character would either be frozen on one frame or cycling through frames so fast it looks like a blur!

Think of it like a flipbook - if you flip the pages too slowly, the animation looks choppy. If you flip too fast, you can't see what's happening. The `AnimationTimer` gives us precise control

over this timing, ensuring our character’s walking animation looks smooth and natural at just the right speed.

`AnimationTimer` wraps a Bevy `Timer` so each player entity knows when to advance to the next animation frame. Each tick represents one slice of time; once the timer hits its interval we move to the next sprite in the sheet.

```
// Append these lines of code to player.rs
#[derive(Component, Deref, DerefMut)]
struct AnimationTimer(Timer);

#[derive(Component)]
struct AnimationState {
    facing: Facing,
    moving: bool,
    was_moving: bool,
}
```

What’s `Deref`, `DerefMut` macros doing?

`Deref` lets our wrapper pretend to be the inner `Timer` when we read from it, and `DerefMut` does the same for writes. That means we can just call `timer.tick(time.delta())` on `AnimationTimer` without manually pulling out the inner value first.

So we are renaming the `Timer` to `AnimationTimer`?

We’re wrapping the timer, not renaming it. Think of `AnimationTimer` as a little box that holds a `Timer`, plus a label that says “this one belongs to the player animation.” When we spawn a player we create a fresh `Timer` and tuck it into that box, so each player could have its own timer if we needed multiple heroes.

So it’s an instance of `AnimationTime`?

Yes, `AnimationTimer` is a tuple struct that contains a `Timer`. We build one when we spawn the player so each entity can carry its own timer data. This pattern shows up whenever you want to attach extra meaning to an existing type without writing a brand-new API.

`AnimationState` remembers which way the player points, whether they are moving, and whether they just started or stopped. Systems read this to choose animation rows and reset frames when movement changes.

Spawning the Player

We load the spritesheet through the `AssetServer`, create a texture atlas layout so Bevy knows the grid, and pick the starting frame for a hero facing down. Then we spawn an entity with the sprite, transform at the origin, our marker components, and the timer that will drive animation.

```
// Append these lines of code to player.rs
fn spawn_player(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
) {
    // Load the spritesheet and build a grid layout: 64x64 tiles, 9 columns, 12 rows
    let texture = asset_server.load("male_spritesheet.png");
    let layout = atlas_layouts.add(TextureAtlasLayout::from_grid(
        UVec2::splat(TILE_SIZE),
        WALK_FRAMES as u32, // columns used for walking frames
        12,                // at least 12 rows available
        None,
        None,
    ));
}
```



```

// Start facing down (towards user), idle on first frame of that row
let facing = Facing::Down;
let start_index = atlas_index_for(facing, 0);

commands.spawn((
  Sprite::from_atlas_image(
    texture,
    TextureAtlas {
      layout,
      index: start_index,
    },
  ),
  Transform::from_translation(Vec3::ZERO),
  Player,
  AnimationState { facing, moving: false, was_moving: false },
  AnimationTimer(Timer::from_seconds(ANIM_DT, TimerMode::Repeating)),
));
}

```

`AnimationState { facing, moving: false, was_moving: false }` sets the starting direction and flags that the character is idle right now and was idle last frame.

`AnimationTimer(Timer::from_seconds(ANIM_DT, TimerMode::Repeating))` creates a repeating stopwatch that fires every `ANIM_DT` seconds to advance the spritesheet.

What's an `AssetServer`?

The `AssetServer` is Bevy's file loader and manager that handles loading and caching of game assets like images, sounds, and 3D models.

When you call `asset_server.load("path/to/sprite.png")`, it doesn't immediately load the file into memory. Instead, it returns a handle that you can use later. This is called "lazy loading" - the actual file loading happens in the background, and Bevy will notify you when it's ready.

This approach is efficient because:

- Multiple entities can share the same sprite without loading it multiple times
- Assets are only loaded when actually needed
- Bevy can optimize and batch-load assets for better performance
- If an asset fails to load, it won't crash your entire game

In our case, `asset_server.load("sprites/player.png")` requests the spritesheet and returns a handle to track its loading status.

It loads in the background?

Like your motivation.
Eventually shows up.



Movement System

Now that we have our player spawned with all the necessary components, we need to update our existing movement system to track which direction the player is facing. This is crucial for the animation system to know which row of the spritesheet to use - each direction (up, down, left, right) corresponds to a different row in our sprite atlas.

This updated system will detect the movement direction and update the `AnimationState` accordingly, so our animation system can pick the correct sprite row for walking animations in each direction.

```
// Append these lines of code to player.rs
fn move_player(
    input: Res<ButtonInput<KeyCode>>,
    time: Res<Time>,
    mut player: Query<(&mut Transform, &mut AnimationState), With<Player>>,
) {
    let Ok((mut transform, mut anim)) = player.single_mut() else {
        return;
    };

    let mut direction = Vec2::ZERO;
    if input.pressed(KeyCode::ArrowLeft) {
        direction.x -= 1.0;
    }
    if input.pressed(KeyCode::ArrowRight) {
        direction.x += 1.0;
    }
    if input.pressed(KeyCode::ArrowUp) {
        direction.y += 1.0;
    }
    if input.pressed(KeyCode::ArrowDown) {
        direction.y -= 1.0;
    }

    if direction != Vec2::ZERO {
        let delta = direction.normalize() * MOVE_SPEED * time.delta_secs();
        transform.translation.x += delta.x;
        transform.translation.y += delta.y;
        anim.moving = true;
    }

    // Update facing based on dominant direction
```

```

    if direction.x.abs() > direction.y.abs() {
        anim.facing = if direction.x > 0.0 { Facing::Right } else { Facing::Left };
    } else {
        anim.facing = if direction.y > 0.0 { Facing::Up } else { Facing::Down };
    }
} else {
    anim.moving = false;
}
}

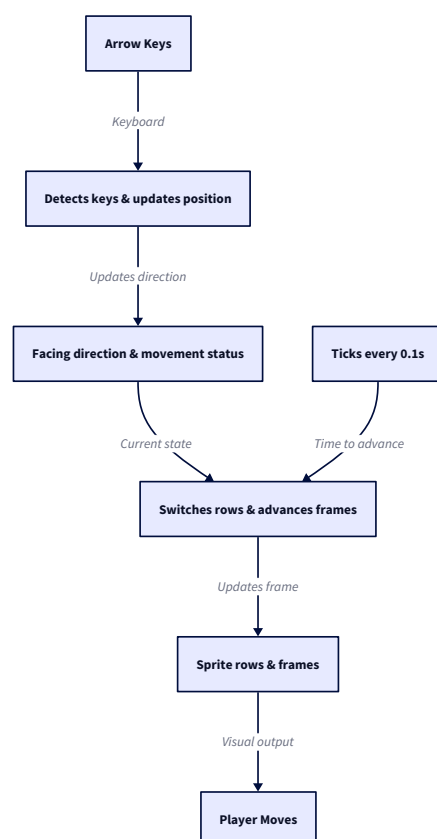
```

The query asks Bevy for the single entity tagged `Player`, giving us mutable access to its `Transform` and `AnimationState`. We build a direction vector from the pressed keys, normalize it so diagonal input isn't faster, and move the player by $\text{speed} \times \text{frame time}$. The facing logic compares horizontal vs vertical strength to decide which way the sprite should look. We record whether the player is moving now so later systems can detect when motion starts or stops.

Animation Implementation

Now we have all the pieces in place - our player can move in different directions, and we're tracking which way they're facing. The final piece is the animation system that actually updates the sprite frames to create the walking animation.

This system takes the direction information from our movement system and uses the animation timer to cycle through the sprite frames at the right speed. It handles the complex logic of switching between different animation rows when the player changes direction, and ensures smooth transitions between walking and idle states.



```

// Append these lines of code to player.rs
fn animate_player(
    time: Res<Time>,
    mut query: Query<(&mut AnimationState, &mut AnimationTimer, &mut Sprite), With<Player>>,
) {
    let Ok((mut anim, mut timer, mut sprite)) = query.single_mut() else {
        return;
    };

    let atlas = match sprite.texture_atlas.as_mut() {
        Some(a) => a,
        None => return,
    };

```

```

};

// Compute the target row and current position in the atlas (column/row within the 9-column row)
let target_row = row_zero_based(anim.facing);
let mut current_col = atlas.index % WALK_FRAMES;
let mut current_row = atlas.index / WALK_FRAMES;

// If the facing changed (or we weren't on a walking row), snap to the first frame of the target row
if current_row != target_row {
    atlas.index = row_start_index(anim.facing);
    current_col = 0;
    current_row = target_row;
    timer.reset();
}

let just_started = anim.moving && !anim.was_moving;
let just_stopped = !anim.moving && anim.was_moving;

if anim.moving {
    if just_started {
        // On tap or movement start, immediately advance one frame for visible feedback
        let row_start = row_start_index(anim.facing);
        let next_col = (current_col + 1) % WALK_FRAMES;
        atlas.index = row_start + next_col;
        // Restart the timer so the next advance uses a full interval
        timer.reset();
    } else {
        // Continuous movement: advance based on timer cadence
        timer.tick(time.delta());
        if timer.just_finished() {
            let row_start = row_start_index(anim.facing);
            let next_col = (current_col + 1) % WALK_FRAMES;
            atlas.index = row_start + next_col;
        }
    }
} else if just_stopped {
    // Not moving: keep current frame to avoid snap. Reset timer on transition to idle.
    timer.reset();
}

// Update previous movement state
anim.was_moving = anim.moving;
}

// Returns the starting atlas index for the given facing row
fn row_start_index(facing: Facing) -> usize {
    row_zero_based(facing) * WALK_FRAMES
}

fn atlas_index_for(facing: Facing, frame_in_row: usize) -> usize {
    row_start_index(facing) + frame_in_row.min(WALK_FRAMES - 1)
}

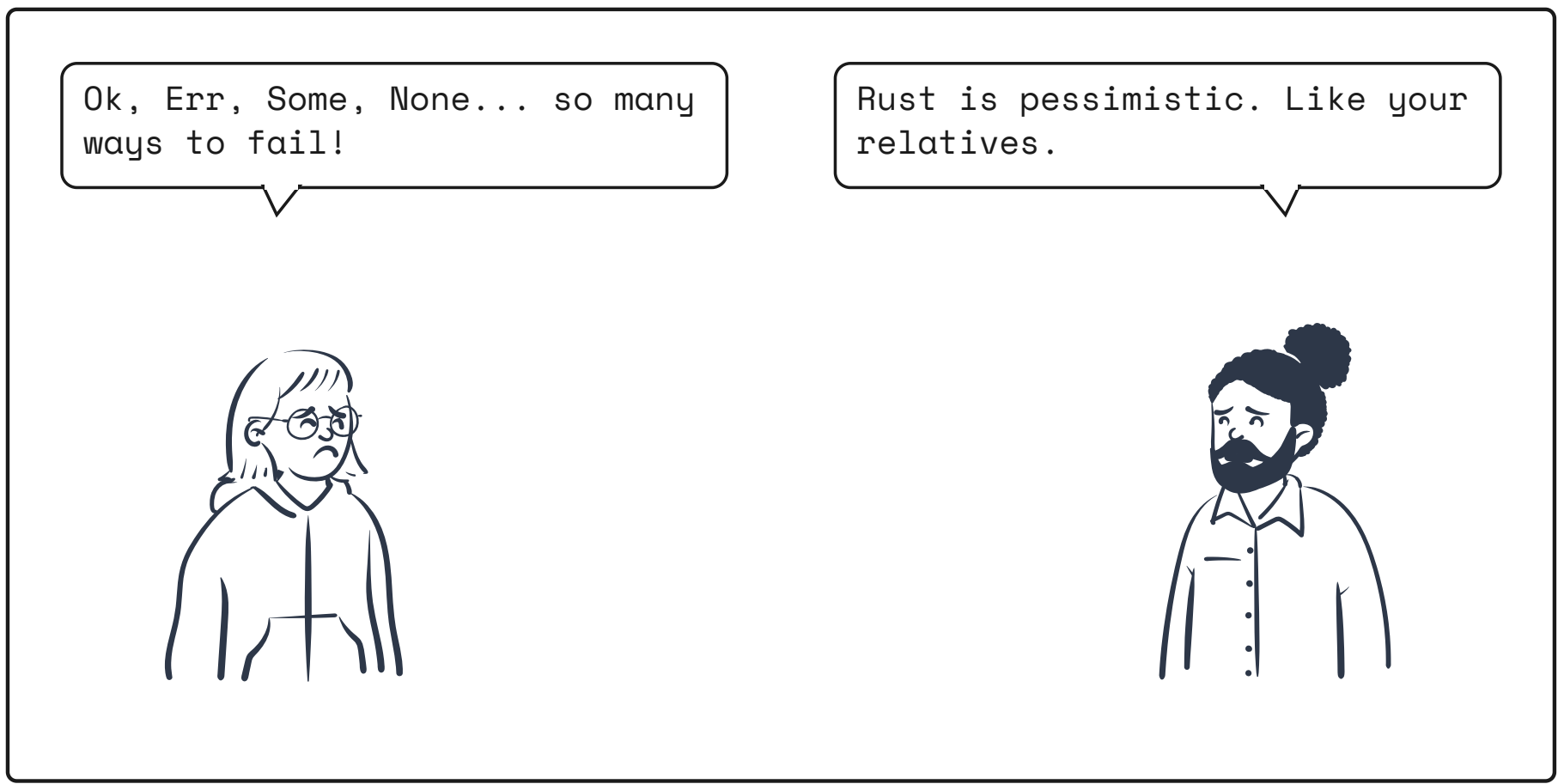
fn row_zero_based(facing: Facing) -> usize {
    match facing {
        Facing::Up => 8,
        Facing::Left => 9,
        Facing::Down => 10,
        Facing::Right => 11,
    }
}

```

```
let Ok((mut anim, mut timer, mut sprite)) = query.single_mut() else { return; };
```

 both checks the result and names the pieces we need. If the query succeeds, the code binds `anim`, `timer`, and `sprite` so we can use them later. If it fails (no player, or more than one), we hit the `else` branch and exit immediately. Rust uses the `Result` type for this: `Ok` means “query returned exactly one result,” `Err` means “something about that query didn’t match.”

After that we `match` on an `Option`, which is Rust’s “maybe there is a value” type. `Some(atlas)` means the texture atlas exists and we can tweak it; `None` means it hasn’t loaded yet, so we skip and let the next frame try again. It’s the same pattern you’d use when checking a map or cache: only use the value when the lookup returns something.



`animate_player` pulls the animation state, timer, and sprite handle for the player. It figures out which row of the atlas matches the current facing, snaps to that row when direction changes, and uses the timer to step through columns at a steady pace. When movement stops we reset the timer so the animation rests on the last frame shown. The helper functions map a facing to the correct row and frame index so the math stays readable.

Creating the Player Plugin

```
// Append these lines of code to player.rs
pub struct PlayerPlugin;

impl Plugin for PlayerPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(Startup, spawn_player)
            .add_systems(Update, (move_player, animate_player));
    }
}
```

`PlayerPlugin` is our “player module” in plug form. In Bevy, a plugin is just a struct that knows how to register systems, resources, and assets. By implementing `Plugin` for `PlayerPlugin`, we give Bevy a checklist: whenever this plugin is added to the app, run the code inside to set up everything the player feature needs. This keeps `main.rs` from becoming a tangle of player-specific calls.

The `build` method is the checklist. Bevy passes us a mutable `App`, and we bolt on the systems we care about. `spawn_player` is scheduled in `Startup` so the sprite appears as soon as the game launches. `move_player` and `animate_player` go into the `Update` schedule so they execute every frame

—handling input and animation in lockstep. With everything declared here, dropping `PlayerPlugin` into `App::new()` automatically wires up the entire player flow.

So this build function is an in-built structure, which I have to write?

Yes. The `Plugin` trait says “any plugin must provide a `build(&self, app: &mut App)` function.” We implement that trait, so Rust expects us to supply the body. Bevy calls this method when it loads the plugin, which is why we add all our systems inside it.

What’s a trait?

A trait is a contract describing what methods a type must provide. Bevy’s `Plugin` trait says “give me a `build` function so I can register your systems.” By implementing that trait for `PlayerPlugin`, we hook into Bevy’s startup process and inject our own setup code. Traits let different types share behavior, `PlayerPlugin` behaves like any other Bevy plugin, but it installs our player-specific systems.

Final Integration

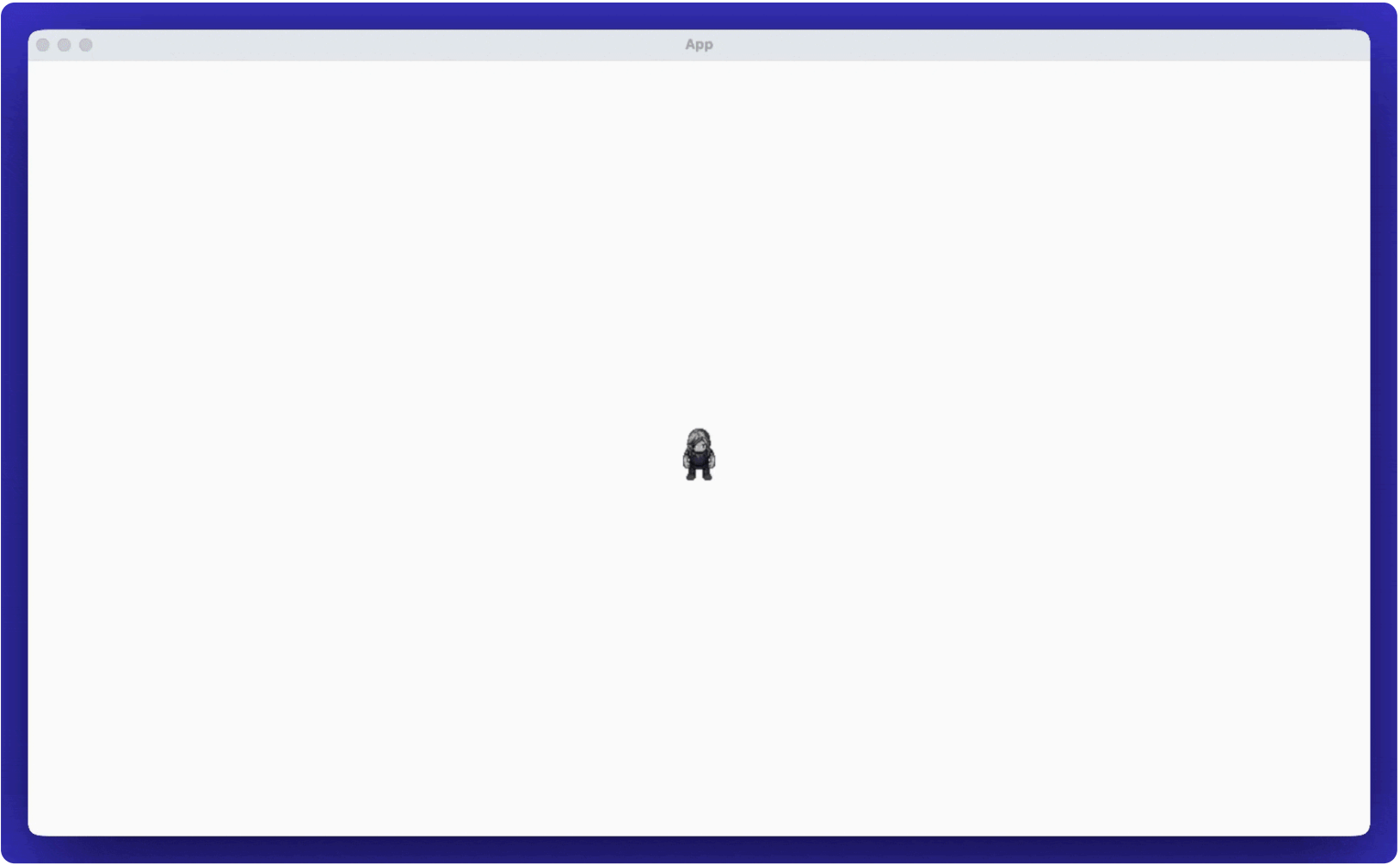
```
// Update the main function in main.rs

use crate::player::PlayerPlugin;

fn main() {
    App::new()
        .insert_resource(ClearColor(Color::WHITE))
        .add_plugins(
            DefaultPlugins.set(AssetPlugin {
                file_path: "src/assets".into(),
                ..default()
            }),
        )
        .add_systems(Startup, setup_camera)
        .add_plugins(PlayerPlugin) // Update this line
        .run();
}
```

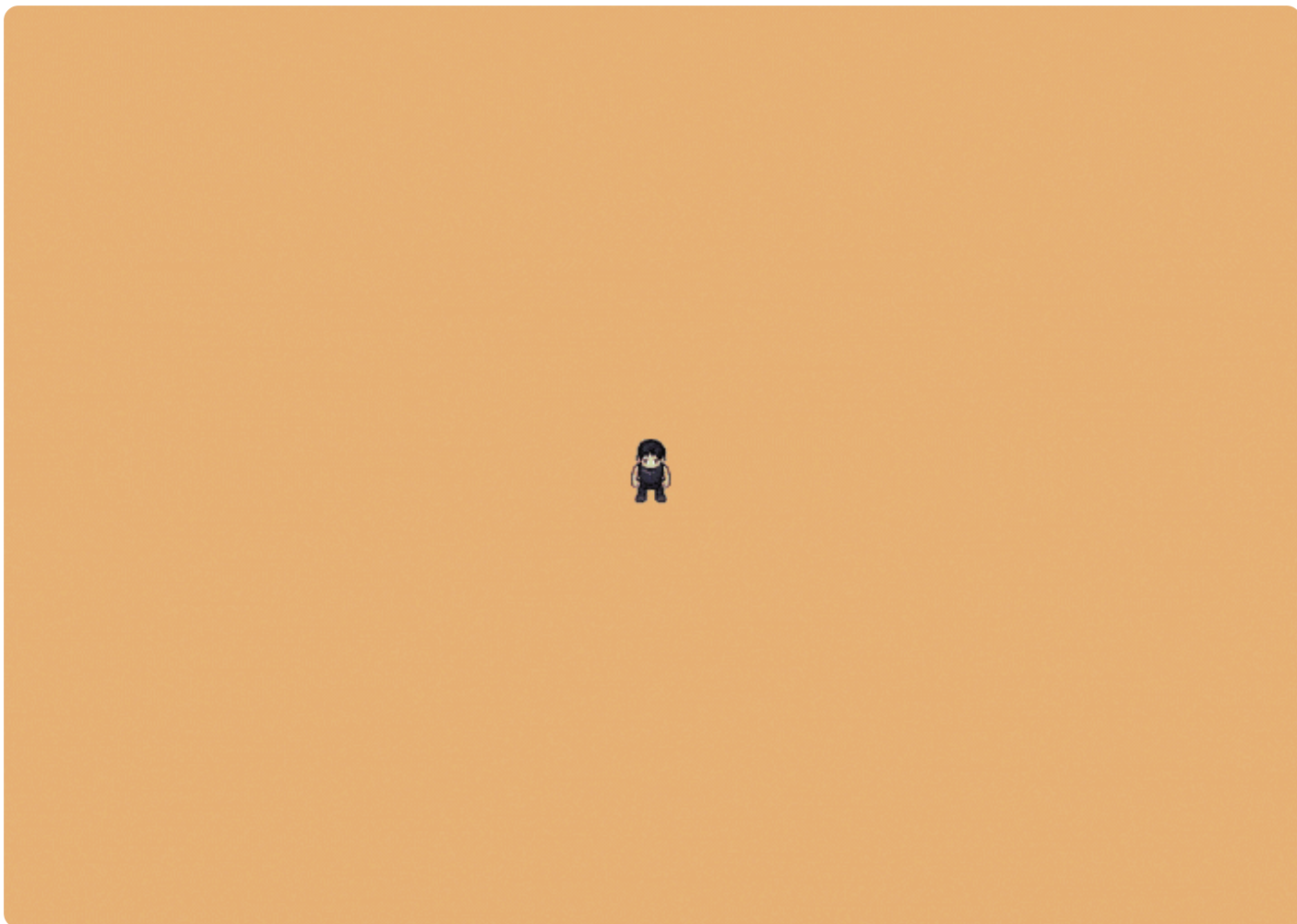
Let’s run it.

```
cargo run
```

The Impatient Programmer's Guide to Bevy and Rust: Chapter 2 - Let There Be a World

By the end of this tutorial, you'll build a procedurally generated game world with layered terrain, water bodies, and props.



Procedural Generation

I respect artists who hand craft tiles to build game worlds. But I belong to the impatient/lazy species.

I went on an exploration and came across procedural generation.

Little did I know the complexities involved. I was on the verge of giving up, however because of the comments and messages from readers of previous chapter, I kept going. And the enlightenment came three days ago, all the pieces fit together.

Basically it's about automatically fitting things together like a jigsaw puzzle. To solve this problem, let's again think in systems.

What do we need to generate the game world procedurally?

1. Tileset.
2. Sockets for tiles because only compatible tiles should fit.
3. Compatibility rules.
4. Magic algorithm that uses these components to generate a coherent world.

How does this magic algorithm work?

That "magic algorithm" has a name: Wave Function Collapse (WFC). The easiest way to see it is with a tiny Sudoku. Same idea: pick the cell with the fewest valid options, place a value, update neighbors, and repeat. If a choice leads to a dead end, undo that guess and try the next option.

Small 4×4 Sudoku

Let’s solve this step by step, focusing on the most constrained cells first.

Initial Puzzle: We need to fill in the empty cells (marked with dots) following Sudoku rules.

?	.	2	.
.	3	.	.
.	.	.	1
4	.	.	.

Step 1 – Finding the most constrained cell:
Let’s analyze the top-left 2×2 box:

- Row 1 already has: 2
- Column 1 already has: 4
- Top-left box already has: 3
- Available numbers: 1, 2, 3, 4
- Eliminating: 2 (in row), 4 (in column), 3 (in box)
- **Only 1 remains!**

1	.	2	.
.	3	.	.
.	.	.	1
4	.	.	.

Propagation Effect: Now that we placed 1, we can eliminate 1 from:

- Row 1: ✓ (already done)
- Column 1: ✓ (already done)
- Top-left 2×2 box: ✓ (already done)

This makes other cells more constrained!

Step 2 – Next most constrained cell:
Now let’s find the next cell with the fewest options.

1	?	2	.
.	3	.	.
.	.	.	1
4	.	.	.

Analysis for the position:

- Row 1 already has: 1, 2
- Column 2 already has: 3
- Top-left box already has: 1, 3
- Available numbers: 1, 2, 3, 4
- Eliminating: 1 (in row), 2 (in row), 3 (in column and box)
- **Only 4 remains!**

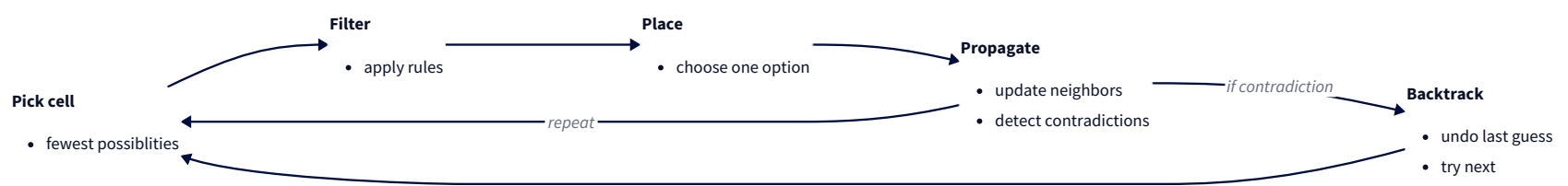
1	4	2	.
.	3	.	.
.	.	.	1
4	.	.	.

Key Insight

This is the essence of constraint propagation! Each placement immediately reduces the options for neighboring cells, making the puzzle progressively easier to solve.

We continue this process: pick the most constrained cell → place the only possible value → propagate constraints → repeat.

If any cell ends up with zero possibilities, we've hit a contradiction—in Sudoku, you backtrack and try a different value.



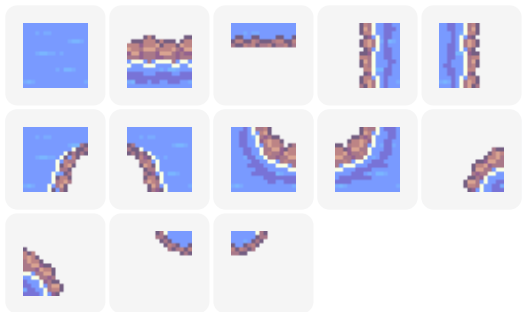
For our tile-based world: Imagine each grid cell as a Sudoku cell, but instead of numbers, we’re placing tiles.

Then we craft rules for valid connections:

- **Rule 1:** Water center tiles connect to other water tiles on all sides
- **Rule 2:** Water edge tiles have two types of sides - water-facing sides connect to other water tiles, land-facing sides connect to the shore

The algorithm uses these rules to ensure tiles fit together properly, creating coherent water bodies with natural-looking shorelines.

Let’s see this in action:

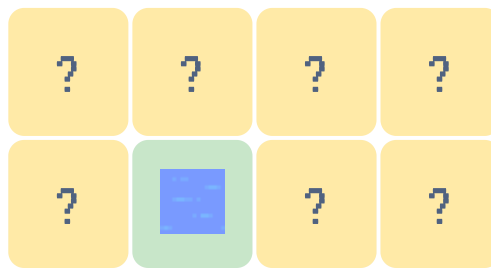


Step 1 - Initial Grid



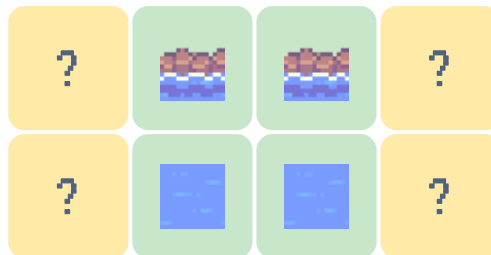
We start with an empty grid where every cell can potentially hold any tile. The ? symbols represent the “superposition” - each cell contains all possible tiles until we begin constraining them through the algorithm.

Step 2 - First Placement



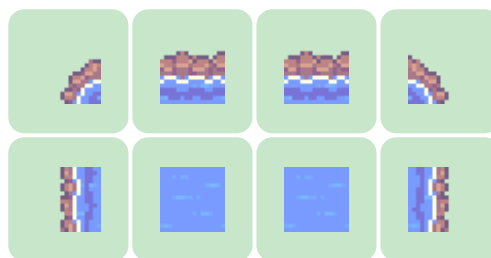
The algorithm starts by placing the initial water center tile. Following **Rule 1**, this center tile needs other water tiles on all sides. This immediately constrains the neighboring cells - they must be water tiles that can connect to the center.

Step 3 - Propagate Constraints



Constraint propagation kicks in! The algorithm expands the water area by placing more center tiles. The edge tiles at the top follow **Rule 2** - their bottom sides (water-facing) connect to the center tiles, while their top sides (land-facing) will connect to the shore.

Step 4 - Final Result



The algorithm completes by filling the edges with appropriate boundary tiles. Notice how our rules create perfect connections - center tiles (**Rule 1**) have water on all sides, while edge and corner tiles (**Rule 2**) have water-facing sides connecting inward and land-facing sides connecting to the shore, creating a coherent geography.

This demonstrates the core Wave Function Collapse algorithm in action:

1. **Find the most constrained cell** - the one with the fewest valid tiles that could fit
2. **Place a tile** whose sockets are compatible with its neighbors
3. **Propagate constraints** - this placement immediately reduces the valid options for surrounding cells
4. **Repeat** until the grid is complete

When we hit a dead end (no valid tiles for a cell), our implementation takes a simpler approach than Sudoku: instead of backtracking through previous choices, we restart with a fresh random seed (up to a retry limit) and run the entire process again until we generate a valid map.

What do you mean by fresh random seed?

A “random seed” is a starting number that controls which “random” sequence the algorithm will follow. Same seed = same tile placement order every time. When we hit a dead end, instead of backtracking, we generate a new random seed and start over—this gives us a completely different sequence of tile choices to try.

Can configuring this randomness help us customize maps?

Yes! The algorithm’s randomness comes from the order in which it picks cells and tiles, and we can control this to influence the final result. By adjusting the random seed or the selection strategy, we can:

- **Bias toward certain patterns** - Weight certain tiles more heavily to create specific landscape types.
- **Control size and complexity** - Influence whether we get small ponds or large lakes.
- **Create predictable variations** - Use the same seed for consistent results, or different seeds for variety.

The same tileset can generate endless variations of coherent landscapes, from simple ponds to complex branching river systems, all by tweaking the randomness probability configuration.

While Wave Function Collapse is powerful, it has its limitations.

- **No large-scale structure control** - WFC focuses on tile compatibility, so it won't automatically create big patterns like "one large lake" or "mountain ranges".
- **Can get stuck** - Complex rules might lead to impossible situations where no valid tiles remain, requiring restarts.
- **Performance depends on complexity** - More tile types and stricter rules increase computation time and failure rates.
- **Requires careful rule design** - Poorly designed compatibility rules can lead to unrealistic or broken landscapes.

We'll address these limitations in a later chapter. For now, we'll focus on building a functional section of our game world that will become the foundation for building larger game worlds.

From Theory to Implementation

Now that we understand **how** Wave Function Collapse works—the constraint propagation, socket compatibility, and tile placement logic. It's time to transform this knowledge into actual running code.

The reality of implementation:

Building a WFC algorithm from scratch is complex. You'd need to implement:

- Constraint propagation across the entire grid
- Backtracking when hitting dead ends
- Efficient data structures for tracking possibilities
- Grid coordinate management
- Random selection with proper probability weights

That's a lot of algorithmic complexity before we even get to the game-specific parts, like sprites, rules, and world design.

Our approach:

Instead of reinventing the wheel, we'll use a library that handles the WFC algorithm internals. This lets us focus on what makes our game unique: the tiles, the rules, the world aesthetics. We define **what** we want; the library figures out **how** to achieve it.

Setting Up Our Toolkit

Let's add the procedural generation library to our project. We'll be using the `bevy_procedural_tilemaps` crate, which I built by forking `ghx_proc_gen` library. I created this fork primarily to ensure compatibility with latest Bevy and to simplify this tutorial.

If you need advanced features, check out the original `ghx_proc_gen` crate by Gilles Henaux, which includes 3D capabilities and debugging tools.

Hope you are following the code from first chapter. Here's the [source code](#).

Update your `Cargo.toml` with the `bevy_procedural_tilemaps` crate.

```
[package]
name = "bevy_game"
version = "0.1.0"
edition = "2024"

[dependencies]
bevy = "0.19" // Line update alert
bevy_procedural_tilemaps = "0.4.0" // Line update alert
```

Bevy Procedural Tilemaps

The `bevy_procedural_tilemaps` library handles the complex logic of generating coherent, rule-based worlds.

What the library handles

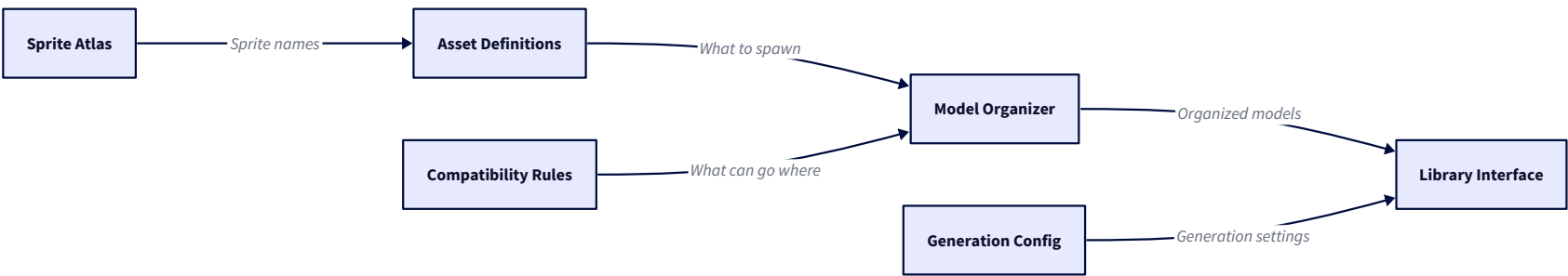
The library takes care of the **algorithmic complexity** of procedural generation:

- **Rule Processing:** Converts our game rules into the library’s internal format
- **Generator Creation:** Builds the procedural generation engine with our configuration
- **Constraint Solving:** Figures out which tiles can go where based on rules
- **Grid Management:** Handles the 2D grid system and coordinate transformations
- **Entity Spawning:** Creates Bevy entities and positions them correctly

What we need to provide

We need to give the library the **game-specific information** it needs:

- **Sprite Definitions:** What sprites to use for each tile type
- **Compatibility Rules:** Which tiles can be placed next to each other
- **Generation Configuration:** The patterns and constraints for our specific game world
- **Asset Data:** Sprite information, positioning, and custom components



Now that we understand how the procedural generation system works, let’s build our map module.

Important Note

Unlike Chapter 1 where you could see immediate results, procedural generation requires doing some ground work. You'll create asset loaders, tile spawning infrastructure before seeing your generated world.

The payoff comes when you finish the grass layer, at that point, you've learned the complete pattern. Adding water and props becomes straightforward repetition with different sprites and connection rules. Along the way, you'll understand Rust concepts (lifetimes, trait bounds, closures) that apply to any Rust project.

Don't give up if concepts feel unclear on first read. That's normal with procedural generation. Revisit confusing sections, experiment with the code, and understanding will emerge. Mastery comes through tinkering, not perfect comprehension on the first pass.

The Map Module

We'll create a dedicated `map` folder inside the `src` folder to house all our world generation logic.

Why create a separate folder for map generation?

The map system requires multiple components working together. World generation involves:

- **Asset management** - Loading and organizing hundreds of tile images.
- **Rule definitions** - Compatibility rules between different terrain types.
- **Grid setup** - Configuring map dimensions and coordinate systems.

Trying to fit all this logic into a single file would create a large file that can become difficult to navigate.

```
src/  
├─ main.rs  
├─ player.rs  
└─ map/  
    ├─ mod.rs  
    └─ assets.rs
```

What's `mod.rs`

The `mod.rs` file is Rust's way of declaring what modules exist in a folder. It's like the "table of contents" for our map module. Add the following line to your `mod.rs`.

```
// src/map/mod.rs  
pub mod assets; // Exposes assets.rs as a module
```

Why `mod.rs` specifically?

It's Rust convention, when you create a folder, Rust looks for `mod.rs` to understand the module structure.

Creating SpawnableAsset

Let's start by creating our `assets.rs` file inside the `map` folder. This will be the foundation that defines how we spawn sprites in our world.

The `bevy_procedural_tilemaps` library needs to know **what to actually place** at each generated location.

It requires the following details:

1. Which sprite to use from our tilemap atlas?
2. Where exactly to position it?
3. What components to add (collision, physics, etc.)?

The library expects us to provide this information in a very specific format. And doing this for every single tile type in your game - grass, dirt, trees, rocks, water, etc will result in redundant code.

This is where `SpawnableAsset` comes in. It's our **abstraction layer** to help you avoid unnecessary boilerplate.

```
// src/map/assets.rs

use bevy::{prelude::*, sprite::Anchor};
use bevy_procedural_tilemaps::prelude::*;

#[derive(Clone)]
pub struct SpawnableAsset {
    /// Name of the sprite inside our tilemap atlas
    sprite_name: &'static str,
    /// Offset in grid coordinates (for multi-tile objects)
    grid_offset: GridDelta,
    /// Offset in world coordinates (fine positioning)
    offset: Vec3,
    /// Function to add custom components (like collision, physics, etc.)
    components_spawner: fn(&mut EntityCommands),
}
```

SpawnableAsset Struct

The `SpawnableAsset` struct contains all the information needed to spawn a tile in our world. The `sprite_name` field gives a name to your sprite (like “grass”, “tree”, “rock”).

The `grid_offset` is used for objects that span multiple tiles - it's a positioning within the tile grid itself.

For example, the follow tree asset needs four tiles.



Grid Offset

Tree Part	Grid Offset	Description
Bottom-left	<code>(0, 0)</code>	Stays at original position
Bottom-right	<code>(1, 0)</code>	Moves one tile right
Top-left	<code>(0, 1)</code>	Moves one tile up
Top-right	<code>(1, 1)</code>	Moves one tile up and right

The `offset` field, on the other hand, is for fine-tuning the position within the tile - like

moving a rock slightly to the left or making sure a tree trunk is perfectly centered within its tile space.

Let’s see how `offset` works with rock positioning:



Offset

Rock	Offset	Description
Rock 1	<code>(0, 0)</code>	Centered in tile
Rock 2	<code>(-8, -6)</code>	Moved slightly left and up
Rock 3	<code>(6, 5)</code>	Moved slightly right and down

Finally, the `components_spawner` is a function that adds custom behavior like collision, physics, or other game mechanics.

Why is sprite name defined as `&'static str`?

Let’s break down `&'static str` piece by piece to understand why we use it for sprite names.

The `&` symbol means “reference to” - instead of making a new copy of the text, we just note where the original text is located.

The `'static` is a special lifetime annotation that tells Rust “this text will exist for the entire duration of your game.” When you write `"grass"` directly in your code, Rust bakes it into your game file when you build it. It’s always there, from game startup to shutdown.

What’s a lifetime and what has `'static` got to do with it?

A **lifetime** is Rust’s way of tracking how long data lives in memory. Rust needs to know when it’s safe to use data and when it might be deleted.

Most data has a limited lifetime. For example:

- Local variables live only while a function runs
- Function parameters live only while the function executes
- Data created in a loop might be deleted when the loop ends

But some data lives forever - like string literals embedded in your program. The `'static` lifetime means “this data lives for the entire duration of the program” - it never gets deleted.

This is perfect for our sprite names because they’re hardcoded in our source code (like `"grass"`, `"tree"`, `"rock"`) and will never change or be deleted while the program runs. Rust can safely let us use these references anywhere in our code because it knows the data will always be there.

What’s a string literal?

A string literal is text you write directly in quotes in your code: `"grass"`, `"dirt"`, `"tree"`.

Why does Rust need to know when it’s safe to use data? Other languages don’t seem to care about this.

Most languages (like C, C++, Java, Python) handle memory safety differently:

- **C/C++:** Don’t track lifetimes at all - you can accidentally use deleted data, leading to crashes or security vulnerabilities
- **Java/Python/C#:** Use garbage collection - the runtime automatically deletes unused data, but this adds overhead and unpredictable pauses
- **Rust:** Tracks lifetimes at compile time - prevents crashes without runtime overhead

The Problem Other Languages Have

```
// Psuedo code warning, don't use
// This would crash in C++ or cause undefined behavior
let sprite_name = {
    let temp = "grass";
    &temp // temp gets deleted here!
};
println!("{}", sprite_name); // CRASH! Using deleted data
```

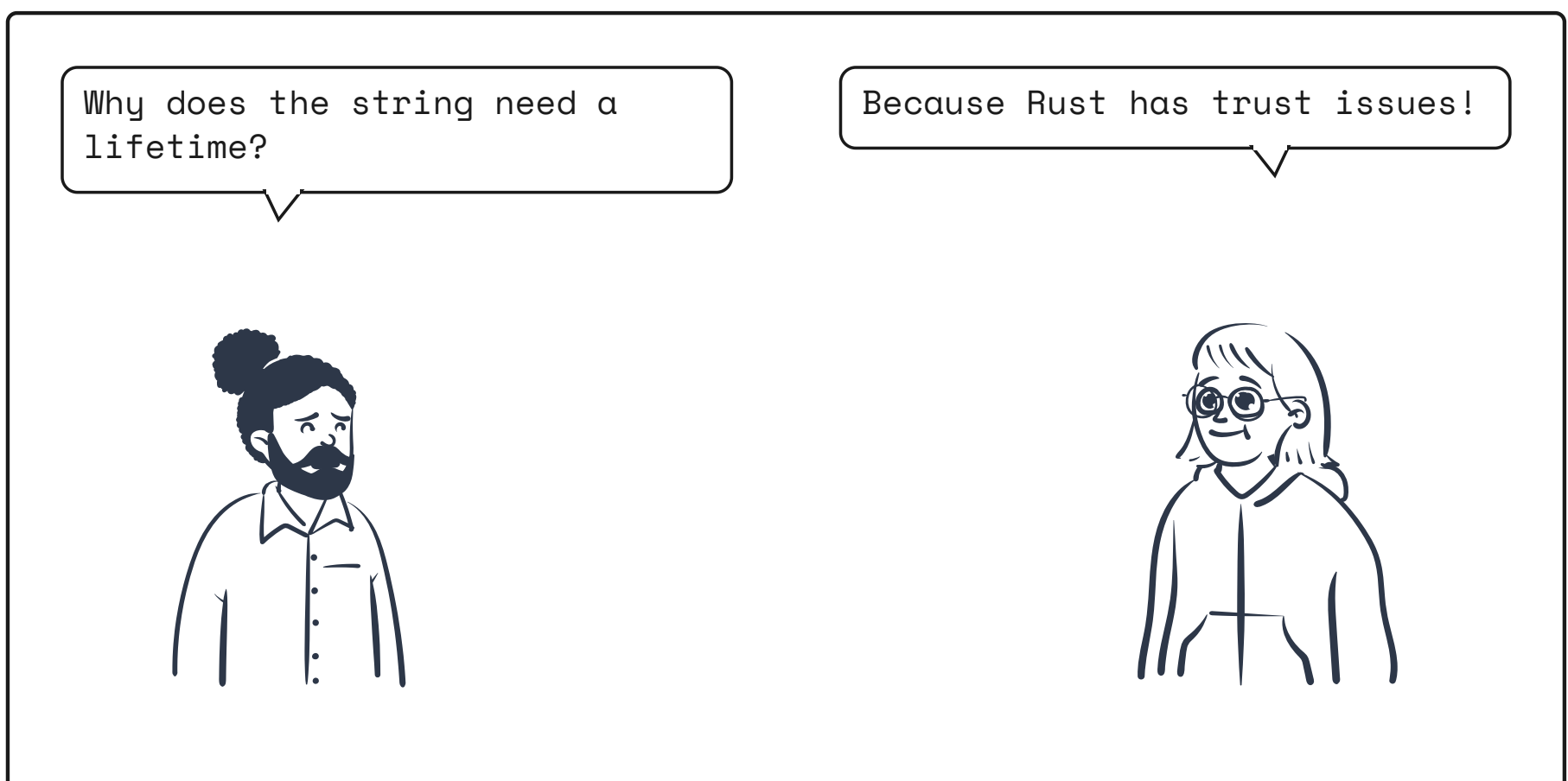
Rust Prevents This

Rust's compiler analyzes your code and says "Hey, you're trying to use data that might be deleted. I won't let you compile this unsafe code." This catches bugs before your game even runs.

Does `str` mean String data type?

Not quite. `str` represents text data, known as a **string slice**, but you can only use it through a reference like `&str` (a view of text stored somewhere else). `String` is text you own and can modify. Our sprite names like "grass" are baked into the program, so `&str` just points to that text without copying it - much more efficient than using `String`.

`&'static str` means "a reference (`&`), to a string slice (`str`) that lives for the entire program duration (`'static`).". This gives us the best of all worlds: memory efficiency (no copying), performance (direct access), and safety (Rust knows the data will always be valid).



What's `GridDelta`?

`GridDelta` is a struct that represents movement in grid coordinates. It specifies "how many tiles to move" in each direction. For example, `GridDelta::new(1, 0, 0)` means "move one tile to the right", while `GridDelta::new(0, 1, 0)` means "move one tile up". It's used for positioning multi-tile objects like the tree sprite with multiple tiles we mentioned earlier in grid offset.

Why's `components_spawner` defined as `fn(&mut EntityCommands)`?

This is a function pointer that takes a mutable reference to `EntityCommands` (Bevy's way of adding components to entities). Looking at the code in `assets.rs`, we can see it defaults to an empty function that does nothing.

The function pointer allows us to customize what components get added to each spawned entity. For example, a tree sprite might need collision components for physics, while a decorative flower

might only need basic rendering components. Each sprite can have its own custom set of components without affecting others.

Why do we need a mutable reference to EntityCommands?

Yes! In Rust, you need a mutable reference (`&mut`) when you want to modify something.

`EntityCommands` needs to be mutable because it's used to add, remove, or modify components on entities.

Now let's add some helpful methods to our `SpawnableAsset` struct to make it easier to create and configure sprite assets.

Append the following code to the same `assets.rs` file.

```
// src/map/assets.rs
impl SpawnableAsset {
    pub fn new(sprite_name: &'static str) -> Self {
        Self {
            sprite_name,
            grid_offset: GridDelta::new(0, 0, 0),
            offset: Vec3::ZERO,
            components_spawner: |_| {}, // Default: no extra components
        }
    }

    pub fn with_grid_offset(mut self, offset: GridDelta) -> Self {
        self.grid_offset = offset;
        self
    }
}
```

What's `-> Self`?

In Rust, you must specify the return type of functions (unlike some languages that can infer it). The `-> Self` tells the compiler exactly what type the function returns, which helps catch errors at compile time. `Self` means “the same type as the struct this method belongs to” - so `Self` refers to `SpawnableAsset` here.

What's `|_| {}`?

This is a closure (anonymous function) that does nothing. The `|_|` means “takes one parameter but ignores it” (the underscore means we don't use the parameter), and `{}` is an empty function body.

We need this because our `SpawnableAsset` struct requires a `components_spawner` field (as we saw in the struct definition), but for basic sprites we don't want to add any custom components. This empty closure serves as a “do nothing” default. We'll learn how to use this field to add custom components in later chapters, but for now it's just a placeholder that satisfies the struct's requirements.

What's a closure? What do you mean by anonymous function?

A closure is a function that can “capture” variables from its surrounding environment. An anonymous function means it doesn't have a name - you define it inline where you need it, rather than declaring it separately like `fn my_function()`.

Example usage

```
// Psuedo code warning, don't use
let mut player_health = 100;

// This closure captures 'player_health' by mutable reference
let mut take_damage = || {
    player_health -= 25; // Modifies the original variable
    player_health
};
```

```
// The closure can modify the original variable
take_damage(); // player_health is now 75
take_damage(); // player_health is now 50
```

Why use closures here?

In our `SpawnableAsset` struct closure can be used to allow each sprite to have custom behavior when spawned. For example, a tree might need collision components, while a decorative flower might need different components. The closure can capture game state and configuration to customize spawning behavior for each sprite type.

Why is semicolon missing in the last line of these functions?

In Rust, the last expression in a function is automatically returned without needing a `return` keyword or semicolon. This makes it easier to specify what value should be returned - you just write the expression you want to return, and Rust handles the rest. This is Rust's way of making code cleaner and more concise.

Why can't you manipulate or retrieve `grid_offset` directly?

The fields of `GridDelta` are private (they have no `pub` keyword), which means they can only be accessed from within the module where `GridDelta` is defined. This is called "encapsulation" - it prevents developers from making mistakes by modifying the struct's data directly, which could break the internal logic. We provide the public method `with_grid_offset()` to safely modify it while maintaining the struct's integrity.

Now that we understand how to define our sprites with `SpawnableAsset`, **how do we load and use these sprites in our game?**

Loading Sprite Assets

Our game uses a **sprite atlas** - a single large image containing all our sprites. Bevy needs to know where each sprite is located within this image, and we need to avoid reloading the same image multiple times.

Create a folder `tile_layers` in your `src/assets` folder and place `tilemap.png` inside it, you can get it from this [github repo](#).



The tilemap assets used in this example are based on [16x16 Game Assets](#) by George Bailey, available on OpenGameArt under CC-BY 4.0 license. **However, to follow this tutorial, please use `tilemap.png` provide from the chapter's [github repo](#).**

Now inside `src/map` folder create a file `tilemap.rs`. When you add a file inside map folder, ensure to register it in `mod.rs` by adding the line `pub mod tilemap`.

This is where our tilemap definition comes in - it acts as a “map” that tells Bevy the coordinates of every sprite in our atlas.

```
// src/map/tilemap.rs
use bevy::math::{URect, UVec2};

pub struct TilemapSprite {
    pub name: &'static str,
    pub pixel_x: u32,
    pub pixel_y: u32,
}

pub struct TilemapDefinition {
    pub tile_width: u32,
    pub tile_height: u32,
    pub atlas_width: u32,
    pub atlas_height: u32,
    pub sprites: &'static [TilemapSprite],
}
```

The `TilemapSprite` struct represents a single sprite within our atlas. It stores the sprite’s name (like “dirt” or “green_grass”) and its exact pixel coordinates within the atlas image.

The `TilemapDefinition` struct serves as the “blueprint” that Bevy uses to understand how to slice up our atlas image into individual sprites.

1. `tile_width` and `tile_height` - How big each individual sprite is (in our case, 32×32 pixels)
2. `atlas_width` and `atlas_height` - The total size of your entire sprite atlas image (the big image containing all sprites)
3. `sprites` - The list of all sprites in your atlas, each with its name and location

Though our tilemap stores sprite names and pixel coordinates, Bevy’s texture atlas system requires numeric indices and rectangular regions. These methods perform the necessary conversions.

Append the following code to your `tilemap.rs`.

```
// src/map/tilemap.rs

impl TilemapDefinition {
    pub const fn tile_size(&self) -> UVec2 {
        UVec2::new(self.tile_width, self.tile_height)
    }

    pub const fn atlas_size(&self) -> UVec2 {
        UVec2::new(self.atlas_width, self.atlas_height)
    }

    pub fn sprite_index(&self, name: &str) -> Option<usize> {
        self.sprites.iter().position(|sprite| sprite.name == name)
    }

    pub fn sprite_rect(&self, index: usize) -> URect {
        let sprite = &self.sprites[index];
        let min = UVec2::new(sprite.pixel_x, sprite.pixel_y);
        URect::from_corners(min, min + self.tile_size())
    }
}
```

The `tile_size()` method converts our tile dimensions into a `UVec2` (unsigned 2D vector), which Bevy uses for size calculations. Similarly, `atlas_size()` provides the total atlas dimensions as a `UVec2`, which Bevy uses to create the texture atlas layout.

The `sprite_index()` method helps in finding sprites by name. When we want to render a “dirt” tile, this method searches through our sprite array and returns the index position of that sprite.

Finally, `sprite_rect()` takes a sprite index and calculates the exact rectangular region within our atlas that contains that sprite. It uses `URect` (unsigned rectangle) to define the boundaries, which Bevy’s texture atlas system requires to know which part of the large image to display.

Now let’s put our tilemap definition to use by adding our first sprite - the dirt tile.

Adding the Dirt Tile

Let’s start with a simple dirt tile to test our tilemap system. The dirt tile sits at pixel coordinates (128, 0) in our 256x320 atlas image. We’ll add more sprites later as we build out our game world.

Append this code to `tilemap.rs`

```
// src/map/tilemap.rs
pub const TILEMAP: TilemapDefinition = TilemapDefinition {
    tile_width: 32,
    tile_height: 32,
    atlas_width: 256,
    atlas_height: 320,
    sprites: &[
        TilemapSprite {
            name: "dirt",
            pixel_x: 128,
            pixel_y: 0,
        },
    ],
};
```

Notice how we’re using a const definition - this means all this sprite metadata is determined at compile time.

Connecting the Tilemap to Asset Loading

Now that we’ve defined our tilemap and sprites in `tilemap.rs`, we need to connect this to our asset loading system in `assets.rs`.

Let’s update the imports in `assets.rs` to bring in our `TILEMAP` definition:

```
// src/map/assets.rs
use bevy::prelude::*;
use bevy_procedural_tilemaps::prelude::*;
use crate::map::tilemap::TILEMAP; // <--- line update alert
```

With the import in place, we can now build the three key functions that helps our procedural rendering system:

1. `TilemapHandles` - Container that holds our loaded atlas and layout data
2. `prepare_tilemap_handles` - Loads the atlas image from disk and creates the texture atlas layout defining each sprite’s rectangular region
3. `load_assets` - Converts sprite names into `Sprite` data structures ready for rendering

Let’s build these step by step.

Creating the TilemapHandles Struct

First, we need a way to hold references to both the atlas image and its layout. Go ahead and append this code into your `assets.rs`:

```
// src/map/assets.rs
#[derive(Clone)]
pub struct TilemapHandles {
    pub image: Handle<Image>,
    pub layout: Handle<TextureAtlasLayout>,
}

impl TilemapHandles {
    pub fn sprite(&self, atlas_index: usize) -> Sprite {
        Sprite::from_atlas_image(
            self.image.clone(),
            TextureAtlas::from(self.layout.clone()).with_index(atlas_index),
        )
    }
}
```

The `TilemapHandles` struct is a container for two handles: `image` points to our loaded sprite sheet file, while `layout` points to the atlas layout that tells Bevy how to slice that image into individual sprites.

The `sprite(atlas_index)` method is a convenience function that creates a ready-to-render `Sprite` by combining the image and layout with a specific index. For example, if the dirt tile is at index 0, calling `tilemap_handles.sprite(0)` gives us a `Sprite` configured to display just the dirt tile from our atlas.

Loading the Atlas from Disk

Now let's create the function that actually loads the atlas image file and sets up the layout. We will be using our `TILEMAP` definition from earlier.

```
// src/map/assets.rs
pub fn prepare_tilemap_handles(
    asset_server: &Res<AssetServer>,
    atlas_layouts: &mut ResMut<Assets<TextureAtlasLayout>>,
    assets_directory: &str,
    tilemap_file: &str,
) -> TilemapHandles {
    let image = asset_server.load:::<Image>(format!("{assets_directory}/{tilemap_file}"));
    let mut layout = TextureAtlasLayout::new_empty(TILEMAP.atlas_size());
    for index in 0..TILEMAP.sprites.len() {
        layout.add_texture(TILEMAP.sprite_rect(index));
    }
    let layout = atlas_layouts.add(layout);

    TilemapHandles { image, layout }
}
```

Breaking it down:

1. **Load the image:** `asset_server.load()` requests the atlas image file from disk
2. **Create empty layout:** `TextureAtlasLayout::new_empty(TILEMAP.atlas_size())` creates a layout matching our 256x320 atlas
3. **Register each sprite:** The loop iterates through all sprites in `TILEMAP`, using `TILEMAP.sprite_rect(index)` to get each sprite's coordinates and adding them to the layout
4. **Store and return:** The layout is added to Bevy's asset system, and we return a `TilemapHandles` containing both handles

This is where `TILEMAP.atlas_size()` and `TILEMAP.sprite_rect()` from our tilemap definition come into play - they tell Bevy exactly how to slice up our atlas image!

This function loads the atlas into memory and sets up the layout structure, but it doesn't actually generate the game world yet. We're just preparing the tools that the procedural generator will use later to create the map.

Converting Sprite Names to Renderable Sprites

Finally, we need a way to convert sprite names (like “dirt”) into `Sprite` objects that can be rendered.

```
// src/map/assets.rs
pub fn load_assets(
    tilemap_handles: &TilemapHandles,
    assets_definitions: Vec<Vec<SpawnableAsset>>,
) -> ModelsAssets<Sprite> {
    let mut models_assets = ModelsAssets::<Sprite>::new();
    for (model_index, assets) in assets_definitions.into_iter().enumerate() {
        for asset_def in assets {
            let SpawnableAsset {
                sprite_name,
                grid_offset,
                offset,
                components_spawner,
            } = asset_def;

            let Some(atlas_index) = TILEMAP.sprite_index(sprite_name) else {
                panic!("Unknown atlas sprite '{}'", sprite_name);
            };

            models_assets.add(
                model_index,
                ModelAsset {
                    assets_bundle: tilemap_handles.sprite(atlas_index),
                    grid_offset,
                    world_offset: offset,
                    spawn_commands: components_spawner,
                },
            )
        }
    }
    models_assets
}
```

Why the two loops?

Some tiles are simple and need just one sprite (like dirt). Others are complex and need multiple sprites (like a tree that needs 4 parts).

The outer loop says “for each type of tile,” and the inner loop says “for each sprite that tile needs.”

Let's walk through what happens when we load a dirt tile:

1. We have: `SpawnableAsset { sprite_name: "dirt", ... }`
2. The function asks TILEMAP: “Where is ‘dirt’?” → TILEMAP replies: “Index 0”
3. It then asks TilemapHandles: “Give me a Sprite for index 0” → Gets back a `Sprite` object
4. Finally, it packages everything together with the positioning info and stores it

What does the final data look like?

After `load_assets` completes, we have a collection of `ModelAsset` objects in memory. Here's what the data structure looks like for a few tiles:

Model	Field	Value	What It Means
Dirt	<code>assets_bundle</code>	<code>Sprite(atlas_index: 0)</code>	Points to dirt sprite in atlas
	<code>grid_offset</code>	<code>(0, 0, 0)</code>	No grid offset needed
	<code>world_offset</code>	<code>(0, 0, 0)</code>	No world offset needed
Tree (bottom)	<code>assets_bundle</code>	<code>Sprite(atlas_index: 31)</code>	Points to tree bottom sprite
	<code>grid_offset</code>	<code>(0, 0, 0)</code>	Placed at base position
	<code>world_offset</code>	<code>(0, 0, 0)</code>	Centered
Tree (top)	<code>assets_bundle</code>	<code>Sprite(atlas_index: 30)</code>	Points to tree top sprite
	<code>grid_offset</code>	<code>(0, 1, 0)</code>	One tile up from bottom
	<code>world_offset</code>	<code>(0, 0, 0)</code>	Centered

Important: These are just data structures in memory - nothing is drawn on screen yet! The actual rendering happens later when the procedural generator uses these prepared `ModelAsset` objects to spawn entities.

Great Progress! You've made it through the foundation layer - sprites, tilemaps, and asset loading. Now we have the visual pieces (assets), but how does the generator know which tiles can be placed next to each other? That's where models and sockets come in!

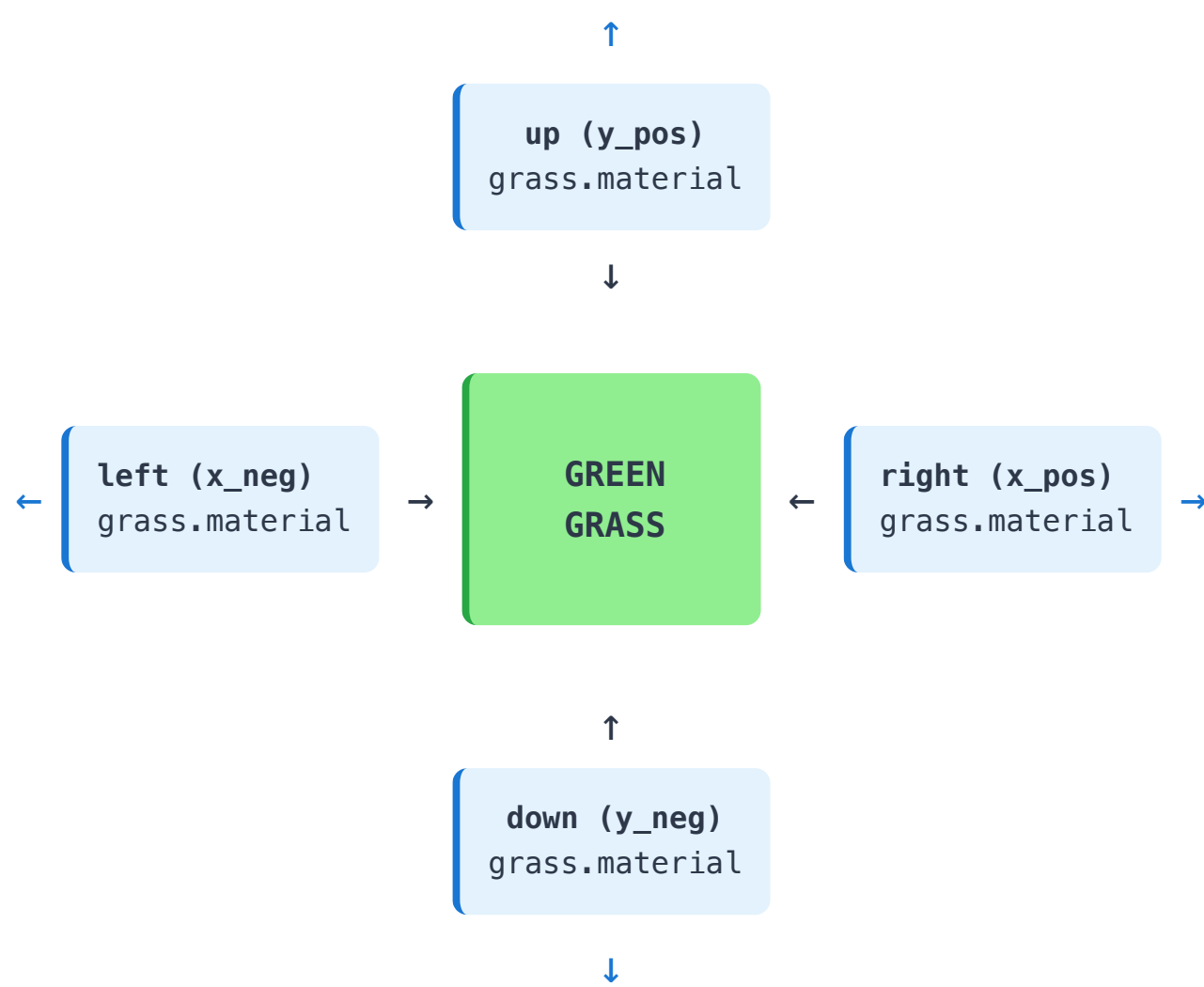
From Tiles to Models

You already understand **tiles** - the individual visual pieces like grass, dirt, and water. Now we need to build models by adding sockets to these tiles and define connection rules so the generator can figure out valid placements.

How Models Expose Sockets

Models expose **sockets** - labeled connection points on each edge. Let's look at a green grass model and see how it exposes sockets in different directions.

Horizontal Plane (x and y directions)



Vertical Axis (z direction)



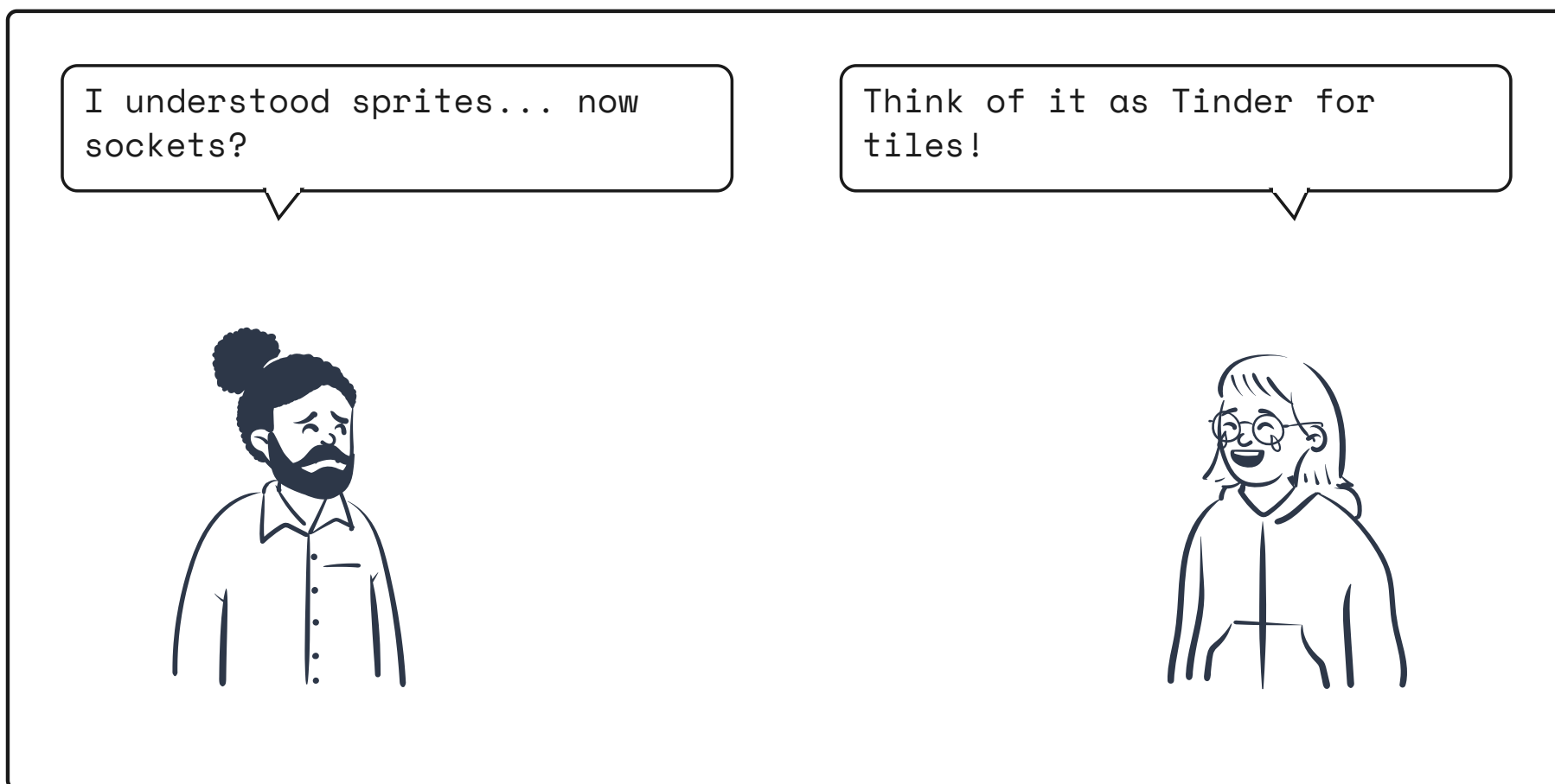
How does z-axis make sense in a 2D game?

Even though we're building a 2D game, the z-axis represents **layering** - Imagine stacking transparent sheets on top of each other. Here's how it works with our yellow grass example:

The Layering System:

- **Dirt tiles** form the base layer (ground level)
- **Green grass tiles** can sit on top of dirt (one layer up)

- **Yellow grass tiles** can sit on top of green grass (another layer up)



Building Models

Now that we understand how models expose sockets in all six directions, we need a way to create these models and link them to their visual sprites.

We'll use a helper called `TerrainModelBuilder` that keeps models and sprites paired correctly as we build our world.

The TerrainModelBuilder

Create a new file `models.rs` inside the `map` folder, and don't forget to add `pub mod models;` to your `mod.rs`.

```
// src/map/models.rs
use bevy_procedural_tilemaps::prelude::*;
use crate::map::assets::SpawnableAsset;

/// Utility wrapper that ensures model declarations and their asset bindings stay aligned.
pub struct TerrainModelBuilder {
    pub models: ModelCollection<Cartesian3D>,
    pub assets: Vec<Vec<SpawnableAsset>>,
}
```

The `TerrainModelBuilder` holds:

1. `models`: What the WFC algorithm uses
2. `assets`: The sprites for the respective model

Now let's add these methods to the builder.

```
// src/map/models.rs
impl TerrainModelBuilder {
    pub fn new() -> Self {
        Self {
            models: ModelCollection::new(),
            assets: Vec::new(),
        }
    }
}
```

```

    }

    pub fn create_model<T>(
        &mut self,
        template: T,
        assets: Vec<SpawnableAsset>,
    ) -> &mut Model<Cartesian3D>
    where
        T: Into<ModelTemplate<Cartesian3D>>,
    {
        let model_ref = self.models.create(template);
        self.assets.push(assets);
        model_ref
    }

    pub fn into_parts(self) -> (Vec<Vec<SpawnableAsset>>, ModelCollection<Cartesian3D>) {
        (self.assets, self.models)
    }
}

```

The `new()` method creates an empty builder to start with.

The `create_model()` method both a socket definition and the corresponding sprites, then adds them to their respective collections at the same index.

Finally, `into_parts()` splits the builder back into separate collections when you're done building, so the assets can go to the renderer and the models can go to the WFC generator.

What's `<T>` doing in `pub fn create_model<T>`?

The `<T>` is Rust's **generic type parameter** - it's a placeholder that gets filled in with the actual type when you call the function. In our case, we might pass in different types of socket definitions (simple single-socket tiles or complex multi-socket tiles), but we want to perform the same operation on all of them.

Generics let us write one function that works with multiple types, as long as they can all be converted into a `ModelTemplate`. This is powerful because it means we can add new socket definition types in the future without changing our `TerrainModelBuilder` code.

What's this `where T: Into<ModelTemplate<Cartesian3D>>`?

This is a **trait bound** that tells Rust what capabilities the generic type `T` must have. The `where` clause says "T must be able to convert itself into a `ModelTemplate<Cartesian3D>` (a 3D model template)."

`Into` is Rust's way of saying "this type knows how to transform itself into that type" - like how a string can be converted into a number, or how our socket definitions can be converted into model templates. This means we can pass in any type that knows how to become a `ModelTemplate` - whether it's simple single-socket tiles, complex multi-socket tiles, or even a custom socket type you create later.

This gives us flexibility while ensuring type safety. The compiler will catch any attempts to pass in a type that can't be converted, preventing runtime errors!

Building the Foundation

Now that we understand how to keep models and assets synchronized, let's start building our procedural world from the ground up. The dirt layer forms the foundation that everything else sits on.

Layers Make WFC Simpler

Without layers, we'd need to cram all our rules into a single layer: "water connects to water and grass", "grass connects to grass and dirt", "trees connect to grass", "dirt connects to dirt" - plus all the edge cases and special connections.

This creates a massive web of interdependencies that makes the WFC algorithm struggle to find valid solutions.

By using layers, we break this complexity into manageable pieces. Each layer only needs to worry about its own connections, making the WFC algorithm much more likely to find valid solutions quickly.

Let's create our dirt layer, make a new file `sockets.rs` inside the `map` folder, and don't forget to add `pub mod sockets;` to your `mod.rs`.

```
// src/map/sockets.rs
use bevy_procedural_tilemaps::prelude::*;

pub struct TerrainSockets {
    pub dirt: DirtLayerSockets,
}

pub struct DirtLayerSockets {
    pub layer_up: Socket,      // What can sit on top of dirt
    pub layer_down: Socket,    // What dirt can sit on
    pub material: Socket,      // What dirt connects to horizontally
}
```

The dirt layer needs three types of sockets.

1. `layer_up` - This socket handles what can be placed in the layer above dirt. Remember layers are to separate rule cram concerns (water can be above grass without touching it).
2. `layer_down` - It handles what layer the dirt itself can be placed on. For the base layer, this will connect to void (empty space).
3. `material` - This takes care of horizontal connections between dirt tiles, ensuring they connect properly to form continuous ground.

Initializing the Sockets

Now we need to actually create these socket instances. Append this function to `sockets.rs`:

```
// src/map/sockets.rs
pub fn create_sockets(socket_collection: &mut SocketCollection) -> TerrainSockets {
    let mut new_socket = || -> Socket { socket_collection.create() };

    let sockets = TerrainSockets {
        dirt: DirtLayerSockets {
            layer_up: new_socket(),
            material: new_socket(),
            layer_down: new_socket(),
        },
    };
    sockets
}
```

The `create_sockets` function takes a `SocketCollection` and creates all our socket instances. The `new_socket` closure is a helper that calls `socket_collection.create()` to generate unique socket IDs. Each socket gets a unique identifier that the WFC algorithm uses to track compatibility rules.

Building the Dirt Layer

Now that we have our socket system defined and initialized, we need to create the rules that tell the WFC algorithm how to use these sockets. This is where we define models and how they connect to each other.

Create a new file `rules.rs` inside the `map` folder, and don't forget to add `pub mod rules;` to your `mod.rs`.

```
// src/map/rules.rs
use crate::map::assets::SpawnableAsset;
use crate::map::models::TerrainModelBuilder;
use crate::map::sockets::*;
use bevy_procedural_tilemaps::prelude::*;

fn build_dirt_layer(
    terrain_model_builder: &mut TerrainModelBuilder,
    terrain_sockets: &TerrainSockets,
    socket_collection: &mut SocketCollection,
) {
    terrain_model_builder
        .create_model(
            SocketsCartesian3D::Simple {
                x_pos: terrain_sockets.dirt.material, // right
                x_neg: terrain_sockets.dirt.material, // left
                z_pos: terrain_sockets.dirt.layer_up, // top
                z_neg: terrain_sockets.dirt.layer_down, // bottom
                y_pos: terrain_sockets.dirt.material, // up
                y_neg: terrain_sockets.dirt.material, // down
            },
            vec![SpawnableAsset::new("dirt")],
        )
        .with_weight(20.);

    socket_collection.add_connections(vec![(
        terrain_sockets.dirt.material,
        vec![terrain_sockets.dirt.material],
    )]);
}
```

Understanding the Dirt Layer Rules:

1. **Creates a dirt model** - Defines a tile that **exposes** sockets on all six sides
2. **Exposes socket types** - Horizontal sides expose `dirt.material`, vertical sides expose layer sockets
3. **Assigns a sprite** - `SpawnableAsset::new("dirt")` tells the renderer which sprite to use
4. **Sets the weight** - `.with_weight(20.)` makes dirt tiles 20 times more likely to be placed
5. **Defines connection rules** - `add_connections` tells WFC that `dirt.material` can connect to other `dirt.material`

This creates a simple but effective foundation layer that can form continuous ground while supporting other layers on top!

Now let's append the `build_world` function that the generator will call to get all our dirt layer rules and models:

```
// src/map/rules.rs
pub fn build_world() -> (
    Vec<Vec<SpawnableAsset>>,
    ModelCollection<Cartesian3D>,
    SocketCollection,
) {
    let mut socket_collection = SocketCollection::new();
    let terrain_sockets = create_sockets(&mut socket_collection);

    let mut terrain_model_builder = TerrainModelBuilder::new();

    // Build dirt layer
    build_dirt_layer(
```



```

        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    let (assets, models) = terrain_model_builder.into_parts();

    (assets, models, socket_collection)
}

```

What This Function Does:

1. **Creates the socket collection** - This is where all our socket connection rules are stored
2. **Gets our socket definitions** - Calls `create_sockets()` to get all the socket types we defined
3. **Creates the model builder** - This keeps our models and assets synchronized
4. **Builds the dirt layer** - Calls our `build_dirt_layer` function to create all the dirt models and rules
5. **Returns the three collections** - Assets for rendering, models for WFC rules, and socket collection for connections

This function is what the generator calls to get all the rules and models needed to create our procedural world!

Generating Dirt

Now that we have all our components - assets, models, sockets, and rules - we need to configure the procedural generation engine.

Create a new file `generate.rs` inside the `map` folder, and don't forget to add `pub mod generate;` to your `mod.rs`.

```

// src/map/generate.rs
use bevy_procedural_tilemaps::prelude::*;
use bevy::prelude::*;

use crate::map::{
    assets::{load_assets, prepare_tilemap_handles},
    rules::build_world,
};

// ----- Configurable values -----
/// Modify these values to control the map size.
pub const GRID_X: u32 = 25;
pub const GRID_Y: u32 = 18;

// -----

const ASSETS_PATH: &str = "tile_layers";
const TILEMAP_FILE: &str = "tilemap.png";
/// Size of a block in world units (in Bevy 2d, 1 pixel is 1 world unit)
pub const TILE_SIZE: f32 = 32.;
/// Size of a grid node in world units
const NODE_SIZE: Vec3 = Vec3::new(TILE_SIZE, TILE_SIZE, 1.);

const ASSETS_SCALE: Vec3 = Vec3::ONE;
/// Number of z layers in the map, derived from the default terrain layers.
const GRID_Z: u32 = 1;

pub fn map_pixel_dimensions() -> Vec2 {
    Vec2::new(TILE_SIZE * GRID_X as f32, TILE_SIZE * GRID_Y as f32)
}

```


Understanding the Configuration Constants:

Let's break down what each of these constants controls:

1. **GRID_X** and **GRID_Y** - These define the size of our generated world in tiles. A 25×18 grid means 450 total tiles (25 × 18 = 450). You can adjust these to create larger or smaller worlds, though larger grids may cause the WFC algorithm to struggle - we'll address scaling issues in a later chapter.
2. **TILE_SIZE** - This is the size of each tile in world units. Since we're using 32×32 pixel sprites, each tile takes up 32 world units. This affects how big your world appears on screen.
3. **NODE_SIZE** - This tells the generator how much space each grid cell occupies in the 3D world. Equal values = perfect tile fit, smaller NODE_SIZE = overlapping sprites, larger NODE_SIZE = gaps between tiles.
4. **GRID_Z** - This defines how many layers our world has. We're currently using 1 layer for dirt, but we'll add more layers later to stack different terrain types on top of each other (dirt, grass, yellow grass, water, props).
5. **ASSETS_SCALE** - This controls the size multiplier for sprites. **Vec3::ONE** means sprites render at their original size.

Now let's append the **setup_generator** function that sets up our procedural generation engine:

```
// src/map/generate.rs
pub fn setup_generator(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
) {
    // 1. Rules Initialization - Get tile definitions and connection rules
    let (assets_definitions, models, socket_collection) = build_world();

    let rules = RulesBuilder::new_cartesian_3d(models, socket_collection)
        // Use ZForward as the up axis (rotation axis for models) since we are using Bevy in 2D
        .with_rotation_axis(Direction::ZForward)
        .build()
        .unwrap();

    // 2. Grid - Create 3D world space with wrapping behavior (false, false, false)
    let grid = CartesianGrid::new_cartesian_3d(GRID_X, GRID_Y, GRID_Z, false, false, false);

    // 3. Configuring the Algorithm - Set up WFC behavior
    let gen_builder = GeneratorBuilder::new()
        .with_rules(rules)
        .with_grid(grid.clone())
        .with_rng(RngMode::RandomSeed)
        .with_node_heuristic(NodeSelectionHeuristic::MinimumRemainingValue)
        .with_model_heuristic(ModelSelectionHeuristic::WeightedProbability);

    let generator = gen_builder.build().unwrap();

    // 4. Loading Assets - Load sprite atlas and convert to renderable assets
    let tilemap_handles =
        prepare_tilemap_handles(&asset_server, &mut atlas_layouts, ASSETS_PATH, TILEMAP_FILE);
    let models_assets = load_assets(&tilemap_handles, assets_definitions);

    // 5. Spawning the Generator - Create entity with Transform and NodesSpawner
    commands.spawn((
        Transform::from_translation(Vec3 {
            x: -TILE_SIZE * grid.size_x() as f32 / 2.,
            y: -TILE_SIZE * grid.size_y() as f32 / 2.,
            z: 0.,
        }),
        grid,
    ));
}
```



```

        generator,
        NodesSpawner::new(models_assets, NODE_SIZE, ASSETS_SCALE).with_z_offset_from_y(true),
    ));
}

```

Rules Initialization

This creates the **constraint solver** that the WFC algorithm uses. It takes our tile definitions and connection rules and converts them into a format the algorithm can understand.

Why `Direction::ZForward`?

Since we're building a 2D game, we need to tell the system which axis to use for rotations.

`Direction::ZForward` means tiles rotate around the Z-axis (pointing toward/away from the screen), which makes sense for a 2D top-down view.

Grid

This creates our world space where tiles will be placed. The three boolean parameters control wrapping behavior:

1. `(false, false, false)` - Most games like Minecraft, Terraria (hard boundaries)
2. `(true, true, false)` - Classic Asteroids or Pac-Man (wraps left-right and up-down)
3. `(true, true, true)` - Advanced simulations with infinite-feeling 3D worlds

Configuring the Algorithm

This is where we configure how the WFC algorithm behaves:

1. `RngMode::RandomSeed` - Uses random seeds (same seed = same world every time)
2. `NodeSelectionHeuristic::MinimumRemainingValue` - Always picks the most constrained cell (fewest valid tiles)
3. `ModelSelectionHeuristic::WeightedProbability` - Picks tiles based on their weights (higher weight = more likely)

Loading Assets and Spawning the Generator

`prepare_tilemap_handles()` loads our sprite atlas from disk, while `load_assets()` converts our sprite definitions into renderable assets.

The `commands.spawn()` creates the generator entity with a `Transform` that centers the world on screen and a `NodesSpawner` that handles the actual tile creation.

The `with_z_offset_from_y(true)` setting uses Y coordinates for Z-layer positioning - tiles higher up on screen render in front, creating natural depth ordering (e.g., tree at Y=10 appears in front of rock at Y=5).

Final Module Structure

Throughout this chapter, we've been building our procedural generation system across multiple files. Before we integrate everything into your main game, let's make sure your `src/map/mod.rs` file includes all the modules we've created:

```

// src/map/mod.rs
pub mod generate;
pub mod assets;
pub mod models;
pub mod rules;
pub mod sockets;
pub mod tilemap;

```

Make sure your `mod.rs` file matches this structure before proceeding to the integration step.

Integrating the Generator into Your Game

Now that we've built our procedural generation system, we need to integrate it into our main game. We'll update the `main.rs` file to include the procedural generation plugin and set up the window size to match our generated world.

Updating main.rs

We need to add the procedural generation plugin and configure the window size to match our generated world. Update your `main.rs`:

```
// src/main.rs
mod map;
mod player;

use bevy::{
    prelude::*,
    window::{Window, WindowPlugin, WindowResolution},
};

use bevy_procedural_tilemaps::prelude::*;

use crate::map::generate::{map_pixel_dimensions, setup_generator};
use crate::player::PlayerPlugin;

fn main() {
    let map_size = map_pixel_dimensions();

    App::new()
        .insert_resource(ClearColor(Color::WHITE))
        .add_plugins(
            DefaultPlugins
                .set(AssetPlugin {
                    file_path: "src/assets".into(),
                    ..default()
                })
                .set(WindowPlugin {
                    primary_window: Some(Window {
                        resolution: WindowResolution::new(map_size.x as u32, map_size.y as u32),
                        resizable: false,
                        ..default()
                    }),
                    ..default()
                })
                .set(ImagePlugin::default_nearest()),
        )
        .add_plugins(ProcGenSimplePlugin::<Cartesian3D, Sprite>::default())
        .add_systems(Startup, (setup_camera, setup_generator))
        .add_plugins(PlayerPlugin)
        .run();
}

fn setup_camera(mut commands: Commands) {
    commands.spawn(Camera2d);
}
```

What's New:

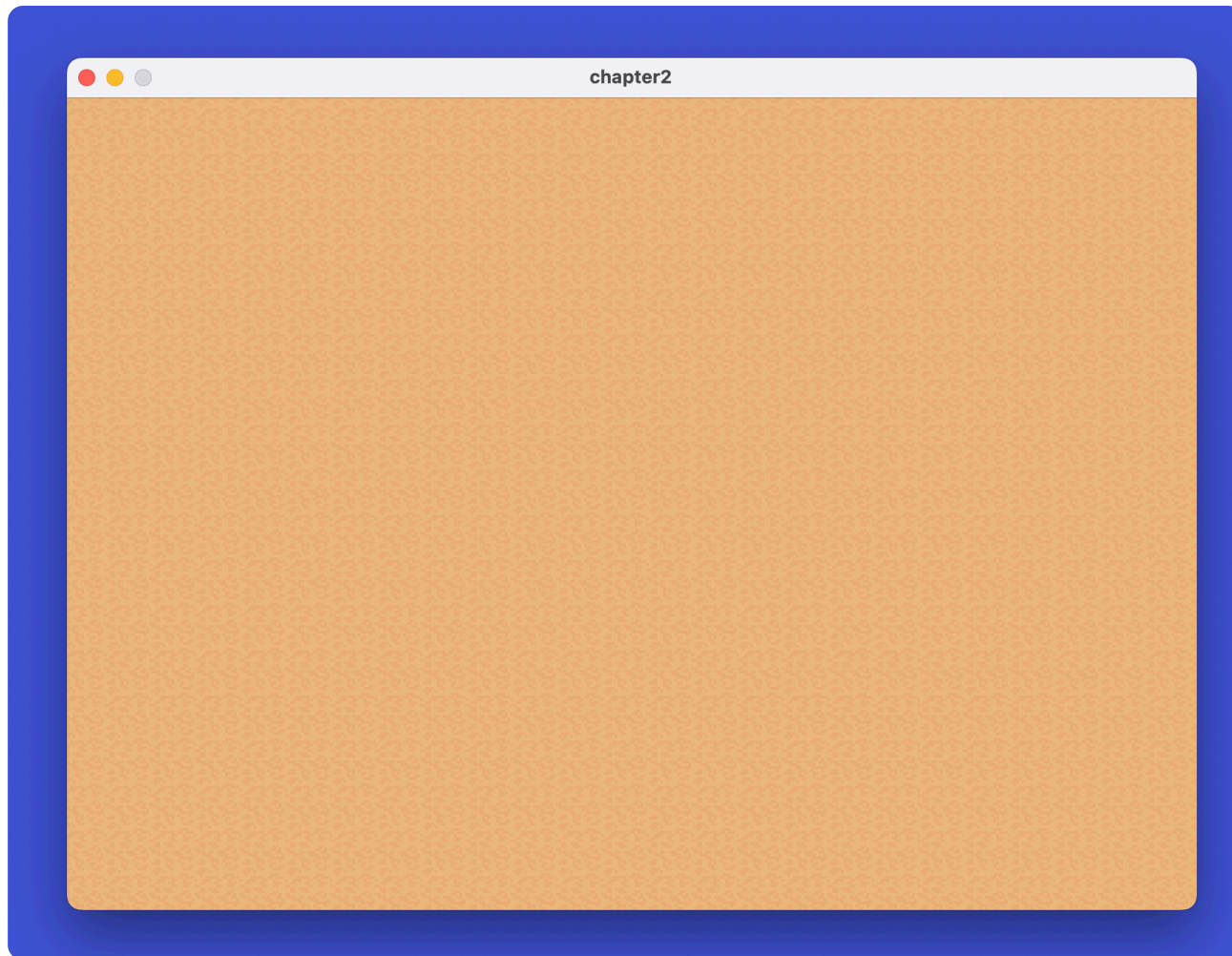
1. **Map module import** - `mod map;` brings in our procedural generation code
2. **Window sizing** - `map_pixel_dimensions()` calculates the window size based on our grid dimensions
3. **Procedural generation plugin** - `ProcGenSimplePlugin` handles the WFC algorithm execution
4. **Generator setup** - `setup_generator` runs at startup to create our world
5. **Image filtering** - `ImagePlugin::default_nearest()` keeps pixel art crisp

Running Your Procedural World

Now run your game:

```
cargo run
```

You should see a procedurally generated world with dirt tiles following the rules we defined! The world will be centered on screen, and the window size will match your grid dimensions (25×18 tiles = 800×576 pixels).



IT WORKS! ALL BROWN! BEAUTIFUL!

Your parents are so proud!



Linux Users with Wayland: If you're seeing white lines appearing between tile rows in your tilemap, this is a known Bevy [rendering issue](#) that affects Linux systems using the Wayland display protocol. To resolve this, run your application using XWayland instead:

```
WINIT_UNIX_BACKEND=x11 WAYLAND_DISPLAY= cargo run
```

Where's the player?

The player is actually there, but it's rendering behind the dirt tiles.

We need to make the player render on top of other layers we have.

Add a Z position constant

```
// src/player.rs, please it below ANIM_DT const
const PLAYER_Z: f32 = 20.0;
```

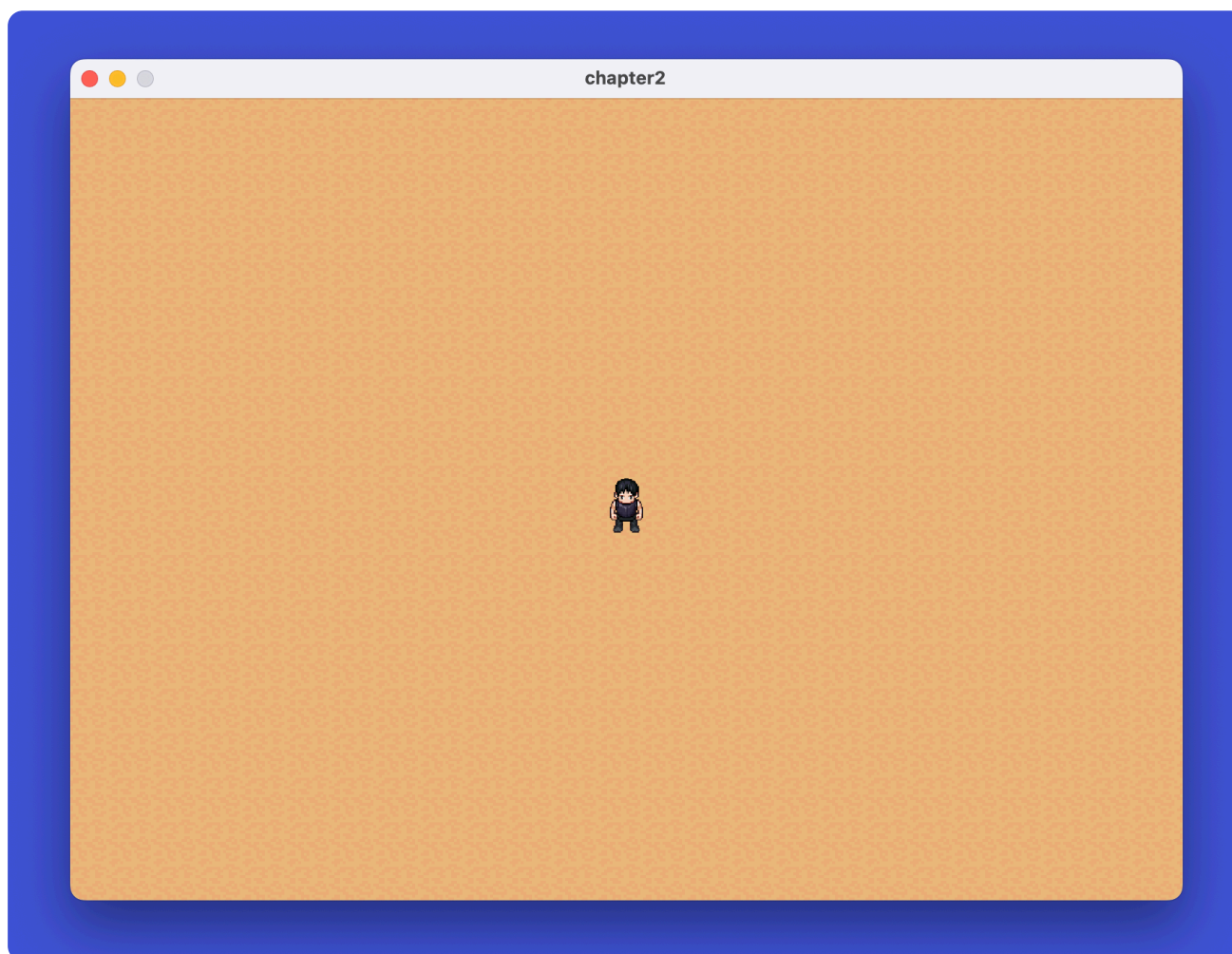
Update the spawn function to use this Z value and scale the player slightly down (for better visual proportion with our generated world).

```
// src/player.rs - Update the Transform line in spawn_player
Transform::from_translation(Vec3::new(0., 0., PLAYER_Z)).with_scale(Vec3::splat(0.8)),
```

Run your again:

```
cargo run
```

Your player renders in front of all tiles and looks proportional to the 32×32 tile world!



Adding the Grass Layer

Now that we have a working dirt foundation, let's add grass on top. The grass layer will create patches of green grass that sit on the dirt, with proper edge tiles for smooth transitions.

Step 1: Adding Grass Sprites to the Tilemap

First, we need to add all the grass sprite coordinates to our tilemap. Append these sprites to the `sprites` array in `tilemap.rs`:

```
// src/map/tilemap.rs - Add these to the sprites array after the dirt sprite inside
TilemapDefinition struct
TilemapSprite {
    name: "green_grass",
    pixel_x: 160,
```

```

    pixel_y: 0,
  },
  TilemapSprite {
    name: "green_grass_corner_in_tl",
    pixel_x: 192,
    pixel_y: 0,
  },
  TilemapSprite {
    name: "green_grass_corner_in_tr",
    pixel_x: 224,
    pixel_y: 0,
  },
  TilemapSprite {
    name: "green_grass_corner_in_bl",
    pixel_x: 192,
    pixel_y: 32,
  },
  TilemapSprite {
    name: "green_grass_corner_in_br",
    pixel_x: 224,
    pixel_y: 32,
  },
  TilemapSprite {
    name: "green_grass_corner_out_tl",
    pixel_x: 0,
    pixel_y: 64,
  },
  TilemapSprite {
    name: "green_grass_corner_out_tr",
    pixel_x: 32,
    pixel_y: 64,
  },
  TilemapSprite {
    name: "green_grass_corner_out_bl",
    pixel_x: 0,
    pixel_y: 96,
  },
  TilemapSprite {
    name: "green_grass_corner_out_br",
    pixel_x: 32,
    pixel_y: 96,
  },
  TilemapSprite {
    name: "green_grass_side_t",
    pixel_x: 64,
    pixel_y: 64,
  },
  TilemapSprite {
    name: "green_grass_side_r",
    pixel_x: 96,
    pixel_y: 64,
  },
  TilemapSprite {
    name: "green_grass_side_l",
    pixel_x: 64,
    pixel_y: 96,
  },
  TilemapSprite {
    name: "green_grass_side_b",
    pixel_x: 96,
    pixel_y: 96,
  },

```

These sprites include the main grass tile, inner corners, outer corners, and side edges for smooth transitions between grass and dirt.

Step 2: Adding Grass Sockets

Now we need to define the sockets for the grass layer. Update your `sockets.rs`:

```
// src/map/sockets.rs - Add this struct after DirtLayerSockets
pub struct GrassLayerSockets {
    pub layer_up: Socket,
    pub layer_down: Socket,
    pub material: Socket,
    pub void_and_grass: Socket,
    pub grass_and_void: Socket,
    pub grass_fill_up: Socket,
}
```

Then update the `TerrainSockets` struct to include grass:

```
// src/map/sockets.rs - Update TerrainSockets
pub struct TerrainSockets {
    pub dirt: DirtLayerSockets,
    pub void: Socket, // line update alert
    pub grass: GrassLayerSockets, // line update alert
}
```

Finally, update the `create_sockets` function to initialize the grass sockets:

```
// src/map/sockets.rs - Update create_sockets function
pub fn create_sockets(socket_collection: &mut SocketCollection) -> TerrainSockets {
    let mut new_socket = || -> Socket { socket_collection.create() };

    let sockets = TerrainSockets {
        dirt: DirtLayerSockets {
            layer_up: new_socket(),
            material: new_socket(),
            layer_down: new_socket(),
        },
        // line update alert
        void: new_socket(),
        // lines update alert
        grass: GrassLayerSockets {
            layer_up: new_socket(),
            material: new_socket(),
            layer_down: new_socket(),
            void_and_grass: new_socket(),
            grass_and_void: new_socket(),
            grass_fill_up: new_socket(),
        },
    };
    sockets
}
```

Why does grass need more sockets than dirt?

Dirt is simple - it fills the entire base layer, so every dirt tile connects to another dirt tile. Grass is different - it creates patches on top of dirt, which means grass tiles need to handle edges where grass meets empty space.

Here's what each socket handles:

- `material` - Connects grass to grass (like dirt's material socket)
- `layer_up` and `layer_down` - Vertical connections (like dirt)
- `void_and_grass` - Transitions from empty space (left) to grass (right)

- `grass_and_void` - Transitions from grass (left) to empty space (right)
- `grass_fill_up` - Allows layers above to fill down into grass areas

These transition sockets (`void_and_grass` and `grass_and_void`) are what create smooth edges. Without them, grass patches would have hard, blocky borders instead of the curved corners and sides we want.

Step 3: Building the Grass Layer Rules

Now let's create the function that builds the grass layer. Append this function to `rules.rs`:

```
// src/map/rules.rs - Add this function after build_dirt_layer
fn build_grass_layer(
    terrain_model_builder: &mut TerrainModelBuilder,
    terrain_sockets: &TerrainSockets,
    socket_collection: &mut SocketCollection,
) {
    // Void model - empty space above dirt where no grass exists
    terrain_model_builder.create_model(
        SocketsCartesian3D::Simple {
            x_pos: terrain_sockets.void,
            x_neg: terrain_sockets.void,
            z_pos: terrain_sockets.grass.layer_up,
            z_neg: terrain_sockets.grass.layer_down,
            y_pos: terrain_sockets.void,
            y_neg: terrain_sockets.void,
        },
        Vec::new(),
    );

    // Main grass tile
    terrain_model_builder
        .create_model(
            SocketsCartesian3D::Multiple {
                x_pos: vec![terrain_sockets.grass.material],
                x_neg: vec![terrain_sockets.grass.material],
                z_pos: vec![
                    terrain_sockets.grass.layer_up,
                    terrain_sockets.grass.grass_fill_up,
                ],
                z_neg: vec![terrain_sockets.grass.layer_down],
                y_pos: vec![terrain_sockets.grass.material],
                y_neg: vec![terrain_sockets.grass.material],
            },
            vec![SpawnableAsset::new("green_grass")],
        )
        .with_weight(5.);

    // Outer corner template
    let green_grass_corner_out = SocketsCartesian3D::Simple {
        x_pos: terrain_sockets.grass.void_and_grass,
        x_neg: terrain_sockets.void,
        z_pos: terrain_sockets.grass.layer_up,
        z_neg: terrain_sockets.grass.layer_down,
        y_pos: terrain_sockets.void,
        y_neg: terrain_sockets.grass.grass_and_void,
    }
    .to_template();

    // Inner corner template
    let green_grass_corner_in = SocketsCartesian3D::Simple {
        x_pos: terrain_sockets.grass.grass_and_void,
        x_neg: terrain_sockets.grass.material,
        z_pos: terrain_sockets.grass.layer_up,
```

```

        z_neg: terrain_sockets.grass.layer_down,
        y_pos: terrain_sockets.grass.material,
        y_neg: terrain_sockets.grass.void_and_grass,
    }
    .to_template();

// Side edge template
let green_grass_side = SocketsCartesian3D::Simple {
    x_pos: terrain_sockets.grass.void_and_grass,
    x_neg: terrain_sockets.grass.grass_and_void,
    z_pos: terrain_sockets.grass.layer_up,
    z_neg: terrain_sockets.grass.layer_down,
    y_pos: terrain_sockets.void,
    y_neg: terrain_sockets.grass.material,
}
.to_template();

// Create rotated versions of outer corners
terrain_model_builder.create_model(
    green_grass_corner_out.clone(),
    vec![SpawnableAsset::new("green_grass_corner_out_tl")],
);
terrain_model_builder.create_model(
    green_grass_corner_out.rotated(ModelRotation::Rot90, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_corner_out_bl")],
);
terrain_model_builder.create_model(
    green_grass_corner_out.rotated(ModelRotation::Rot180, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_corner_out_br")],
);
terrain_model_builder.create_model(
    green_grass_corner_out.rotated(ModelRotation::Rot270, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_corner_out_tr")],
);

// Create rotated versions of inner corners
terrain_model_builder.create_model(
    green_grass_corner_in.clone(),
    vec![SpawnableAsset::new("green_grass_corner_in_tl")],
);
terrain_model_builder.create_model(
    green_grass_corner_in.rotated(ModelRotation::Rot90, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_corner_in_bl")],
);
terrain_model_builder.create_model(
    green_grass_corner_in.rotated(ModelRotation::Rot180, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_corner_in_br")],
);
terrain_model_builder.create_model(
    green_grass_corner_in.rotated(ModelRotation::Rot270, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_corner_in_tr")],
);

// Create rotated versions of side edges
terrain_model_builder.create_model(
    green_grass_side.clone(),
    vec![SpawnableAsset::new("green_grass_side_t")],
);
terrain_model_builder.create_model(
    green_grass_side.rotated(ModelRotation::Rot90, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_side_l")],
);
terrain_model_builder.create_model(
    green_grass_side.rotated(ModelRotation::Rot180, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_side_b")],
);

```



```

);
terrain_model_builder.create_model(
    green_grass_side.rotated(ModelRotation::Rot270, Direction::ZForward),
    vec![SpawnableAsset::new("green_grass_side_r")],
);

// Add connection rules
socket_collection.add_rotated_connection(
    terrain_sockets.dirt.layer_up,
    vec![terrain_sockets.grass.layer_down],
);
socket_collection.add_connections(vec![
    (terrain_sockets.void, vec![terrain_sockets.void]),
    (
        terrain_sockets.grass.material,
        vec![terrain_sockets.grass.material],
    ),
    (
        terrain_sockets.grass.void_and_grass,
        vec![terrain_sockets.grass.grass_and_void],
    ),
]);
}

```

Understanding the Grass Layer Function

This function does several things - let's break it down step by step.

1. The Void Model - Empty Space

This creates an “invisible” tile - a spot where no grass grows. Notice `Vec::new()` means no sprite is rendered. The WFC algorithm needs this to create patches of grass rather than covering everything.

2. The Main Grass Tile

This is the center grass tile. All four horizontal sides use `grass.material`, meaning they connect to other grass tiles. The `z_pos` has two options - allowing either another layer above OR the special `grass_fill_up` socket for yellow grass later.

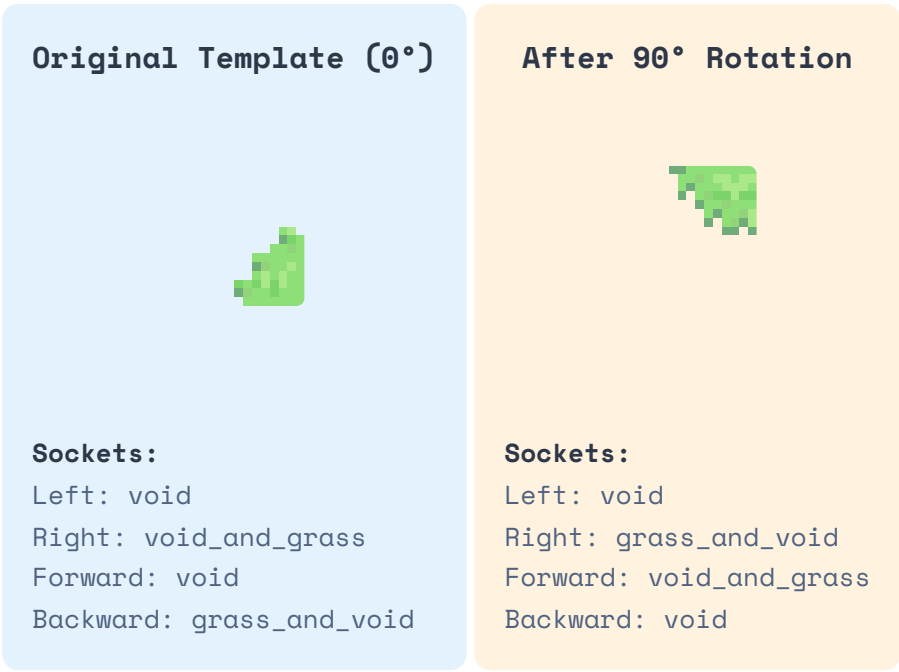
A **template** is a reusable socket pattern. Instead of writing the same socket configuration four times (once for each rotation), we create it once and rotate it. The `.to_template()` converts it into a format that can be rotated.

4. Understanding Rotation - What's Actually Happening?

We're rotating the **socket pattern**.

Each tile has a sprite (the visual) and sockets (the connection rules). When we rotate a template, the sockets shift to different edges.

How Socket Rotation Works



Notice: the sprites are different (top-left vs bottom-left corner), but the socket pattern shifted clockwise by 90°. The `void_and_grass` socket moved from the right edge to the forward edge.

This is powerful because we define one socket pattern and pair it with different sprites at different rotations. The result: four unique corner models from one template definition.

We do the same for corner inside and side edges of grass tiles as well.

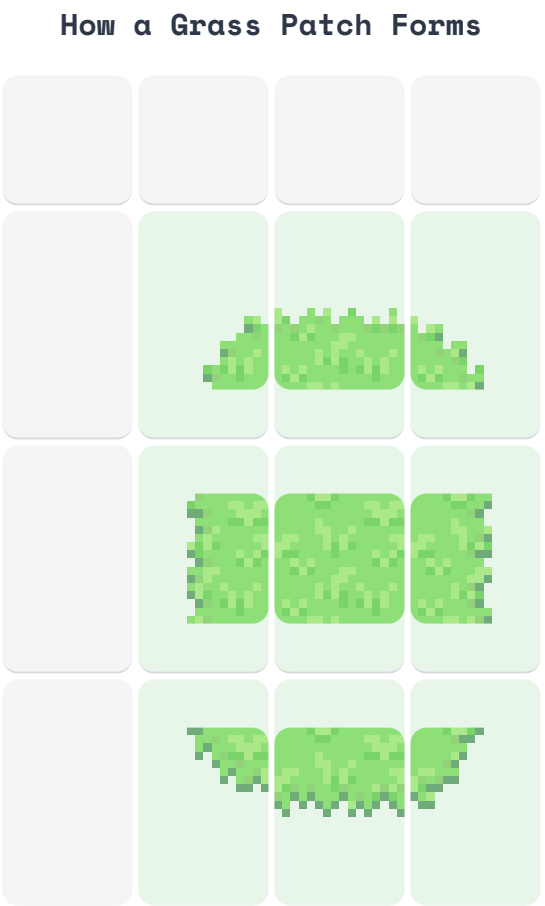
5. How Simple Rules Create Coherent Shapes

Here’s where the magic happens. We define only three connection rules, yet they create complex, natural-looking grass patches. Let’s see how.

The Three Rules:

- 1. `void` connects to `void` - Empty space stays empty
- 2. `grass.material` connects to `grass.material` - Grass centers connect to each other
- 3. `void_and_grass` connects to `grass_and_void` - Transition sockets create smooth edges

That’s it! But how do these simple rules create coherent grass patches? Let’s visualize a 3×3 grass patch forming:



Why Every Edge Matches Perfectly:

Let's trace through the top row to understand how sockets work. Remember: each tile has sockets on its edges that define what can connect to it.

Look at the grass tile on the top left - **green_grass_corner_out_tl** tile (second row, second column in the grid above). This tile has a socket called **void_and_grass** on its **right edge**

Now look at the **green_grass_side_t** tile immediately to its right. This tile has a socket called **grass_and_void** on its **left edge**. When these two tiles sit next to each other, their edges touch. The **void_and_grass** socket (from the corner) connects to the **grass_and_void** socket (from the side) because of Rule 3

The same pattern repeats across the entire grid. **green_grass_side_t** has **grass_and_void** on its **right edge**. **green_grass_corner_out_tr** has **void_and_grass** on its **left edge**. Where they touch, these sockets match perfectly!

The **green_grass** center tile has **material** sockets on all edges, so it connects to any adjacent grass tile that also has **material** on the touching edge.

The WFC algorithm uses these three simple rules to check every tile placement. Before placing a tile, it verifies that all its sockets match the sockets of neighboring tiles. The result: organic-looking grass patches with smooth, curved edges!

6. Layer Connections

This tells the WFC algorithm that grass can sit on top of dirt. The **add_rotated_connection** means this rule applies regardless of how the tiles are rotated - grass can always sit on dirt.

Step 4: Calling the Grass Layer Function

Now update the **build_world** function to call **build_grass_layer**:

```
// src/map/rules.rs - Update build_world function
pub fn build_world() -> (
    Vec<Vec<SpawnableAsset>>,
    ModelCollection<Cartesian3D>,
    SocketCollection,
) {
    let mut socket_collection = SocketCollection::new();
    let terrain_sockets = create_sockets(&mut socket_collection);

    let mut terrain_model_builder = TerrainModelBuilder::new();

    // Build dirt layer
    build_dirt_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    // Line update alert
    // Build grass layer
    build_grass_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    let (assets, models) = terrain_model_builder.into_parts();

    (assets, models, socket_collection)
}
```

Step 5: Updating Grid Layers

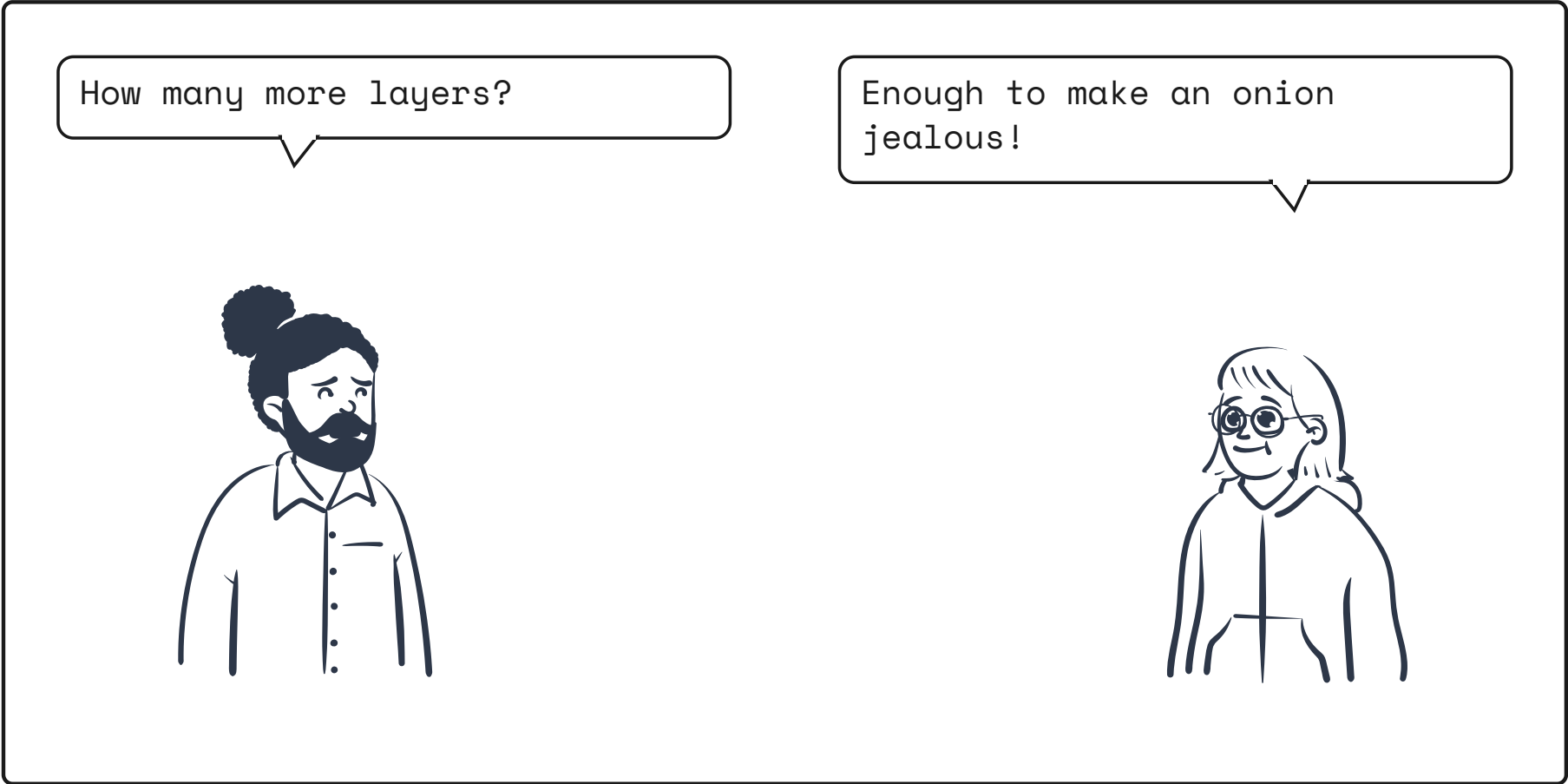
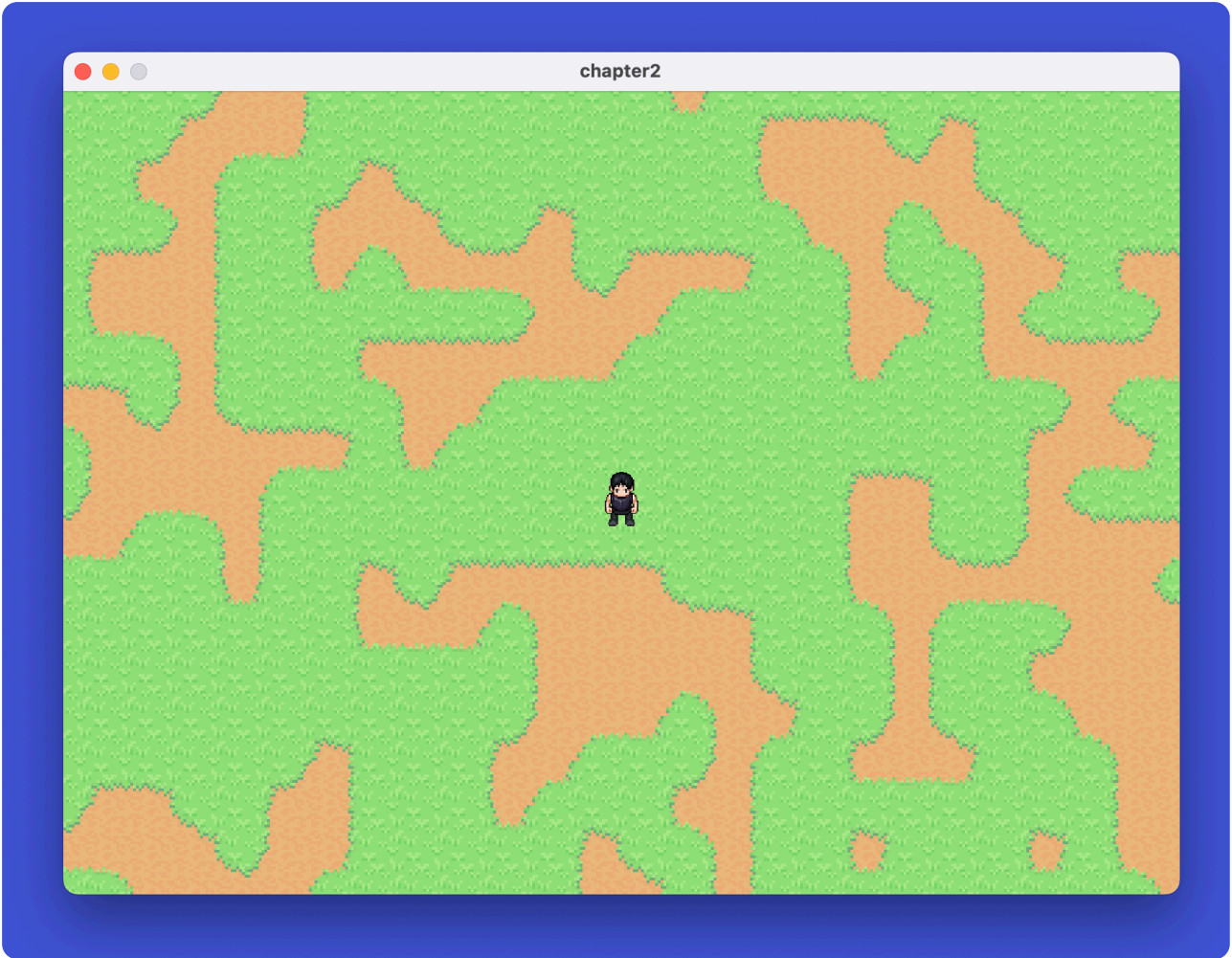
Finally, update `generate.rs` to use 2 layers instead of 1:

```
// src/map/generate.rs - Update GRID_Z constant
const GRID_Z: u32 = 2;
```

Now run your game:

```
cargo run
```

You should see patches of green grass growing on top of the dirt layer, with smooth edges and corners transitioning between grass and dirt!



Adding the Yellow Grass Layer

Now that we have green grass, let's add yellow grass patches that can grow on top of it! Yellow grass creates visual variety and demonstrates how layers can stack.

Step 1: Add Yellow Grass Sprites to Tilemap

First, let's add the yellow grass sprites to our tilemap definition. Open `src/map/tilemap.rs` and add these entries to the `sprites` array:

```
// src/map/tilemap.rs - Add these to the sprites array after the dirt sprite inside
TilemapDefinition struct
TilemapSprite {
    name: "yellow_grass",
    pixel_x: 0,
    pixel_y: 256,
},
TilemapSprite {
    name: "yellow_grass_corner_in_tl",
    pixel_x: 32,
    pixel_y: 256,
},
TilemapSprite {
    name: "yellow_grass_corner_in_tr",
    pixel_x: 64,
    pixel_y: 256,
},
TilemapSprite {
    name: "yellow_grass_corner_in_bl",
    pixel_x: 32,
    pixel_y: 288,
},
TilemapSprite {
    name: "yellow_grass_corner_in_br",
    pixel_x: 64,
    pixel_y: 288,
},
TilemapSprite {
    name: "yellow_grass_corner_out_tl",
    pixel_x: 96,
    pixel_y: 256,
},
TilemapSprite {
    name: "yellow_grass_corner_out_tr",
    pixel_x: 128,
    pixel_y: 256,
},
TilemapSprite {
    name: "yellow_grass_corner_out_bl",
    pixel_x: 96,
    pixel_y: 288,
},
TilemapSprite {
    name: "yellow_grass_corner_out_br",
    pixel_x: 128,
    pixel_y: 288,
},
TilemapSprite {
    name: "yellow_grass_side_t",
    pixel_x: 160,
    pixel_y: 256,
},
TilemapSprite {
    name: "yellow_grass_side_r",
    pixel_x: 192,
    pixel_y: 256,
```

```

},
TilemapSprite {
    name: "yellow_grass_side_l",
    pixel_x: 160,
    pixel_y: 288,
},
TilemapSprite {
    name: "yellow_grass_side_b",
    pixel_x: 192,
    pixel_y: 288,
},

```

Step 2: Define Yellow Grass Sockets

Yellow grass has a special behavior - it sits **on top of green grass**, not on dirt. This means it needs different socket connections.

Add the socket structure to `src/map/sockets.rs`:

```

// src/map/sockets.rs - Add after GrassLayerSockets
pub struct YellowGrassLayerSockets {
    pub layer_up: Socket,
    pub layer_down: Socket,
    pub yellow_grass_fill_down: Socket,
}

```

Then update `TerrainSockets` to include yellow grass:

```

// src/map/sockets.rs - Update TerrainSockets
pub struct TerrainSockets {
    pub dirt: DirtLayerSockets,
    pub void: Socket,
    pub grass: GrassLayerSockets,
    pub yellow_grass: YellowGrassLayerSockets, // Add this line
}

```

Finally, initialize the yellow grass sockets in `create_sockets`:

```

// src/map/sockets.rs - Update create_sockets function
pub fn create_sockets(socket_collection: &mut SocketCollection) -> TerrainSockets {
    let mut new_socket = || -> Socket { socket_collection.create() };

    let sockets = TerrainSockets {
        dirt: DirtLayerSockets {
            layer_up: new_socket(),
            material: new_socket(),
            layer_down: new_socket(),
        },
        void: new_socket(),
        grass: GrassLayerSockets {
            layer_up: new_socket(),
            material: new_socket(),
            layer_down: new_socket(),
            void_and_grass: new_socket(),
            grass_and_void: new_socket(),
            grass_fill_up: new_socket(),
        },
        yellow_grass: YellowGrassLayerSockets {
            layer_up: new_socket(),
            layer_down: new_socket(),
            yellow_grass_fill_down: new_socket(),
        }
    };
}

```



```

    },
};
sockets
}

```

Why does yellow grass only need 3 sockets?

Unlike green grass, yellow grass doesn't need `void_and_grass` transition sockets. Why? Because yellow grass **reuses the green grass edges**. When yellow grass meets empty space, the green grass layer below provides the edge tiles. Yellow grass only appears where green grass already exists, so it uses green grass's `material` socket for horizontal connections.

The `yellow_grass_fill_down` socket is special - it connects to green grass's `grass_fill_up` socket, allowing yellow grass to "fill down" into the green grass layer below.

Step 3: Building the Yellow Grass Layer Rules

Now let's create the function that builds the yellow grass layer. Add this function to `rules.rs`:

```

// src/map/rules.rs - Add this function after build_grass_layer
fn build_yellow_grass_layer(
    terrain_model_builder: &mut TerrainModelBuilder,
    terrain_sockets: &TerrainSockets,
    socket_collection: &mut SocketCollection,
) {
    // Void model - empty space where no yellow grass exists
    terrain_model_builder.create_model(
        SocketsCartesian3D::Simple {
            x_pos: terrain_sockets.void,
            x_neg: terrain_sockets.void,
            z_pos: terrain_sockets.yellow_grass.layer_up,
            z_neg: terrain_sockets.yellow_grass.layer_down,
            y_pos: terrain_sockets.void,
            y_neg: terrain_sockets.void,
        },
        Vec::new(),
    );

    // Main yellow grass tile
    terrain_model_builder
        .create_model(
            SocketsCartesian3D::Simple {
                x_pos: terrain_sockets.grass.material,
                x_neg: terrain_sockets.grass.material,
                z_pos: terrain_sockets.yellow_grass.layer_up,
                z_neg: terrain_sockets.yellow_grass.yellow_grass_fill_down,
                y_pos: terrain_sockets.grass.material,
                y_neg: terrain_sockets.grass.material,
            },
            vec![SpawnableAsset::new("yellow_grass")],
        )
        .with_weight(5.);

    // Outer corner template
    let yellow_grass_corner_out = SocketsCartesian3D::Simple {
        x_pos: terrain_sockets.grass.void_and_grass,
        x_neg: terrain_sockets.void,
        z_pos: terrain_sockets.yellow_grass.layer_up,
        z_neg: terrain_sockets.yellow_grass.yellow_grass_fill_down,
        y_pos: terrain_sockets.void,
        y_neg: terrain_sockets.grass.grass_and_void,
    }
    .to_template();
}

```



```

// Inner corner template
let yellow_grass_corner_in = SocketsCartesian3D::Simple {
  x_pos: terrain_sockets.grass.grass_and_void,
  x_neg: terrain_sockets.grass.material,
  z_pos: terrain_sockets.yellow_grass.layer_up,
  z_neg: terrain_sockets.yellow_grass.yellow_grass_fill_down,
  y_pos: terrain_sockets.grass.material,
  y_neg: terrain_sockets.grass.void_and_grass,
}
.to_template();

// Side edge template
let yellow_grass_side = SocketsCartesian3D::Simple {
  x_pos: terrain_sockets.grass.void_and_grass,
  x_neg: terrain_sockets.grass.grass_and_void,
  z_pos: terrain_sockets.yellow_grass.layer_up,
  z_neg: terrain_sockets.yellow_grass.yellow_grass_fill_down,
  y_pos: terrain_sockets.void,
  y_neg: terrain_sockets.grass.material,
}
.to_template();

// Create rotated versions of outer corners
terrain_model_builder.create_model(
  yellow_grass_corner_out.clone(),
  vec![SpawnableAsset::new("yellow_grass_corner_out_tl")],
);
terrain_model_builder.create_model(
  yellow_grass_corner_out.rotated(ModelRotation::Rot90, Direction::ZForward),
  vec![SpawnableAsset::new("yellow_grass_corner_out_bl")],
);
terrain_model_builder.create_model(
  yellow_grass_corner_out.rotated(ModelRotation::Rot180, Direction::ZForward),
  vec![SpawnableAsset::new("yellow_grass_corner_out_br")],
);
terrain_model_builder.create_model(
  yellow_grass_corner_out.rotated(ModelRotation::Rot270, Direction::ZForward),
  vec![SpawnableAsset::new("yellow_grass_corner_out_tr")],
);

// Create rotated versions of inner corners
terrain_model_builder.create_model(
  yellow_grass_corner_in.clone(),
  vec![SpawnableAsset::new("yellow_grass_corner_in_tl")],
);
terrain_model_builder.create_model(
  yellow_grass_corner_in.rotated(ModelRotation::Rot90, Direction::ZForward),
  vec![SpawnableAsset::new("yellow_grass_corner_in_bl")],
);
terrain_model_builder.create_model(
  yellow_grass_corner_in.rotated(ModelRotation::Rot180, Direction::ZForward),
  vec![SpawnableAsset::new("yellow_grass_corner_in_br")],
);
terrain_model_builder.create_model(
  yellow_grass_corner_in.rotated(ModelRotation::Rot270, Direction::ZForward),
  vec![SpawnableAsset::new("yellow_grass_corner_in_tr")],
);

// Create rotated versions of side edges
terrain_model_builder.create_model(
  yellow_grass_side.clone(),
  vec![SpawnableAsset::new("yellow_grass_side_t")],
);
terrain_model_builder.create_model(
  yellow_grass_side.rotated(ModelRotation::Rot90, Direction::ZForward),

```



```

        vec![SpawnableAsset::new("yellow_grass_side_l")],
    );
    terrain_model_builder.create_model(
        yellow_grass_side.rotated(ModelRotation::Rot180, Direction::ZForward),
        vec![SpawnableAsset::new("yellow_grass_side_b")],
    );
    terrain_model_builder.create_model(
        yellow_grass_side.rotated(ModelRotation::Rot270, Direction::ZForward),
        vec![SpawnableAsset::new("yellow_grass_side_r")],
    );

    // Add connection rules
    socket_collection
        .add_rotated_connection(
            terrain_sockets.grass.layer_up,
            vec![terrain_sockets.yellow_grass.layer_down],
        )
        .add_rotated_connection(
            terrain_sockets.yellow_grass.yellow_grass_fill_down,
            vec![terrain_sockets.grass.grass_fill_up],
        );
}

```

Notice how the yellow grass models reuse green grass's transition sockets (`void_and_grass` and `grass_and_void`) for horizontal connections. This is the clever part - yellow grass doesn't define its own edges, it borrows them from green grass!

The connection rules establish two important relationships:

1. `grass.layer_up` connects to `yellow_grass.layer_down` - Yellow grass sits on top of green grass
2. `yellow_grass_fill_down` connects to `grass_fill_up` - This allows yellow grass to appear where green grass has the special "fill up" socket

Step 4: Calling the Yellow Grass Layer Function

Update `build_world` in `rules.rs` to call `build_yellow_grass_layer`:

```

// src/map/rules.rs - Update build_world function
pub fn build_world() -> (
    Vec<Vec<SpawnableAsset>>,
    ModelCollection<Cartesian3D>,
    SocketCollection,
) {
    let mut socket_collection = SocketCollection::new();
    let terrain_sockets = create_sockets(&mut socket_collection);

    let mut terrain_model_builder = TerrainModelBuilder::new();

    // Build dirt layer
    build_dirt_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    // Build grass layer
    build_grass_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    // Line update alert

```

```
// Build yellow grass layer
build_yellow_grass_layer(
    &mut terrain_model_builder,
    &terrain_sockets,
    &mut socket_collection,
);

let (assets, models) = terrain_model_builder.into_parts();

(assets, models, socket_collection)
}
```

Step 5: Updating Grid Layers

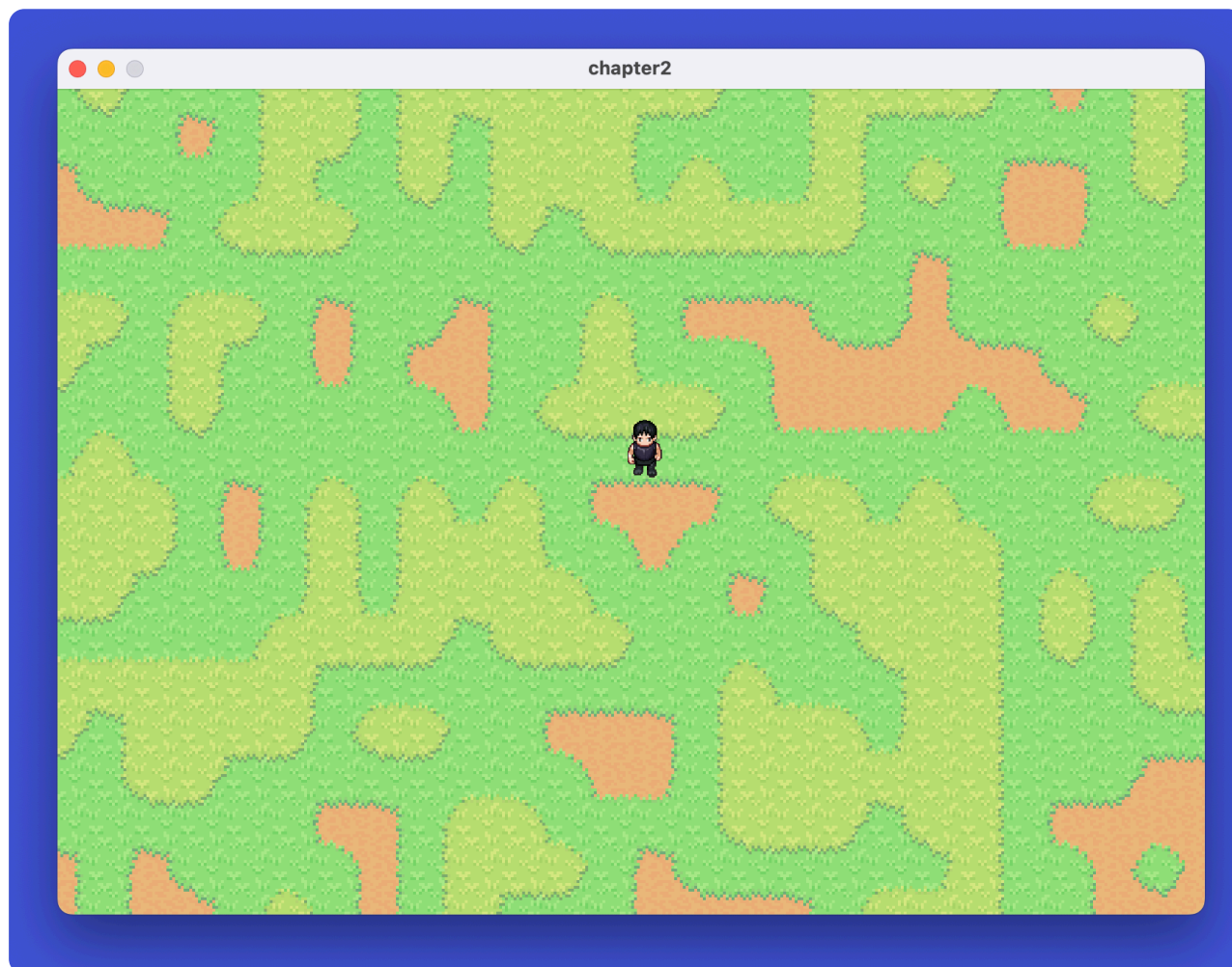
We need one more layer for yellow grass. Update the constant in `generate.rs`:

```
// src/map/generate.rs - Update GRID_Z
const GRID_Z: u32 = 3; // Changed from 2 to 3
```

Now run your game:

```
cargo run
```

You should see yellow grass patches appearing on top of green grass, creating a beautiful layered terrain!



Adding the Water Layer

Water adds life to our procedural world! Unlike grass layers that stack on top of each other, water appears **alongside** yellow grass at the same layer level. This creates interesting terrain where water bodies can form near grassy areas.

Step 1: Add Water Sprites to Tilemap

First, let's add the water sprites to our tilemap definition. Open `src/map/tilemap.rs` and add these entries to the `sprites` array:

```
// src/map/tilemap.rs - Add water sprites to the sprites array
```

```
TilemapSprite {  
    name: "water",  
    pixel_x: 32,  
    pixel_y: 192,  
},  
TilemapSprite {  
    name: "water_corner_in_tl",  
    pixel_x: 64,  
    pixel_y: 192,  
},  
TilemapSprite {  
    name: "water_corner_in_tr",  
    pixel_x: 96,  
    pixel_y: 192,  
},  
TilemapSprite {  
    name: "water_corner_in_bl",  
    pixel_x: 64,  
    pixel_y: 224,  
},  
TilemapSprite {  
    name: "water_corner_in_br",  
    pixel_x: 96,  
    pixel_y: 224,  
},  
TilemapSprite {  
    name: "water_corner_out_tl",  
    pixel_x: 128,  
    pixel_y: 192,  
},  
TilemapSprite {  
    name: "water_corner_out_tr",  
    pixel_x: 160,  
    pixel_y: 192,  
},  
TilemapSprite {  
    name: "water_corner_out_bl",  
    pixel_x: 128,  
    pixel_y: 224,  
},  
TilemapSprite {  
    name: "water_corner_out_br",  
    pixel_x: 160,  
    pixel_y: 224,  
},  
TilemapSprite {  
    name: "water_side_t",  
    pixel_x: 192,  
    pixel_y: 192,  
},  
TilemapSprite {  
    name: "water_side_r",  
    pixel_x: 224,  
    pixel_y: 192,  
},  
TilemapSprite {  
    name: "water_side_l",  
    pixel_x: 192,  
    pixel_y: 224,  
},  
TilemapSprite {  
    name: "water_side_b",  
    pixel_x: 224,  
    pixel_y: 224,  
}
```

```
    pixel_y: 224,  
  },
```

Step 2: Define Water Sockets

Water goes on the next Z-layer above yellow grass. “Layer” here refers to the Z-coordinate in our 3D grid, not geological layers.

We’ve been stacking these: dirt at `Z=0`, green grass at `Z=1`, yellow grass at `Z=2`, and now water at `Z=3`.

At any grid position, you can have dirt at the bottom Z-level and water at a higher Z-level—they occupy the same X,Y position but different Z heights.

Add the socket structure to `src/map/sockets.rs`:

```
// src/map/sockets.rs - Add after YellowGrassLayerSockets  
pub struct WaterLayerSockets {  
    pub layer_up: Socket,  
    pub layer_down: Socket,  
    pub material: Socket,  
    pub void_and_water: Socket,  
    pub water_and_void: Socket,  
    pub ground_up: Socket,  
}
```

Then update `TerrainSockets` to include water:

```
// src/map/sockets.rs - Update TerrainSockets  
pub struct TerrainSockets {  
    pub dirt: DirtLayerSockets,  
    pub void: Socket,  
    pub grass: GrassLayerSockets,  
    pub yellow_grass: YellowGrassLayerSockets,  
    pub water: WaterLayerSockets, // Add this line  
}
```

Finally, initialize the water sockets in `create_sockets`:

```
// src/map/sockets.rs - Update create_sockets function  
pub fn create_sockets(socket_collection: &mut SocketCollection) -> TerrainSockets {  
    let mut new_socket = || -> Socket { socket_collection.create() };  
  
    let sockets = TerrainSockets {  
        dirt: DirtLayerSockets {  
            layer_up: new_socket(),  
            material: new_socket(),  
            layer_down: new_socket(),  
        },  
        void: new_socket(),  
        grass: GrassLayerSockets {  
            layer_up: new_socket(),  
            material: new_socket(),  
            layer_down: new_socket(),  
            void_and_grass: new_socket(),  
            grass_and_void: new_socket(),  
            grass_fill_up: new_socket(),  
        },  
        yellow_grass: YellowGrassLayerSockets {  
            layer_up: new_socket(),  
            layer_down: new_socket(),  
        },  
    };  
}
```

```

        yellow_grass_fill_down: new_socket(),
    },
    // Line update alert
    water: WaterLayerSockets {
        layer_up: new_socket(),
        layer_down: new_socket(),
        material: new_socket(),
        void_and_water: new_socket(),
        water_and_void: new_socket(),
        ground_up: new_socket(),
    },
};
sockets
}

```

Water has 6 sockets because it behaves like green grass - it creates patches with transitions:

1. `material` - Connects water to water (like grass's material socket)
2. `layer_up` and `layer_down` - Vertical connections
3. `void_and_water` and `water_and_void` - Transition sockets for smooth edges
4. `ground_up` - Special socket that allows props above to know they're not on water

Step 3: Building the Water Layer Rules

Now let's create the function that builds the water layer. Add this function to `rules.rs`:

```

// src/map/rules.rs - Add this function after build_yellow_grass_layer
pub fn build_water_layer(
    terrain_model_builder: &mut TerrainModelBuilder,
    terrain_sockets: &TerrainSockets,
    socket_collection: &mut SocketCollection,
) {
    // Void model - represents land areas where no water exists
    terrain_model_builder.create_model(
        SocketsCartesian3D::Multiple {
            x_pos: vec![terrain_sockets.void],
            x_neg: vec![terrain_sockets.void],
            z_pos: vec![
                terrain_sockets.water.layer_up,
                terrain_sockets.water.ground_up,
            ],
            z_neg: vec![terrain_sockets.water.layer_down],
            y_pos: vec![terrain_sockets.void],
            y_neg: vec![terrain_sockets.void],
        },
        Vec::new(),
    );

    // Main water tile
    const WATER_WEIGHT: f32 = 0.02;
    terrain_model_builder
        .create_model(
            SocketsCartesian3D::Simple {
                x_pos: terrain_sockets.water.material,
                x_neg: terrain_sockets.water.material,
                z_pos: terrain_sockets.water.layer_up,
                z_neg: terrain_sockets.water.layer_down,
                y_pos: terrain_sockets.water.material,
                y_neg: terrain_sockets.water.material,
            },
            vec![SpawnableAsset::new("water")],
        )
        .with_weight(10. * WATER_WEIGHT);
}

```

```

// Outer corner template
let water_corner_out = SocketsCartesian3D::Simple {
  x_pos: terrain_sockets.water.void_and_water,
  x_neg: terrain_sockets.void,
  z_pos: terrain_sockets.water.layer_up,
  z_neg: terrain_sockets.water.layer_down,
  y_pos: terrain_sockets.void,
  y_neg: terrain_sockets.water.water_and_void,
}
.to_template()
.with_weight(WATER_WEIGHT);

// Inner corner template
let water_corner_in = SocketsCartesian3D::Simple {
  x_pos: terrain_sockets.water.water_and_void,
  x_neg: terrain_sockets.water.material,
  z_pos: terrain_sockets.water.layer_up,
  z_neg: terrain_sockets.water.layer_down,
  y_pos: terrain_sockets.water.material,
  y_neg: terrain_sockets.water.void_and_water,
}
.to_template()
.with_weight(WATER_WEIGHT);

// Side edge template
let water_side = SocketsCartesian3D::Simple {
  x_pos: terrain_sockets.water.void_and_water,
  x_neg: terrain_sockets.water.water_and_void,
  z_pos: terrain_sockets.water.layer_up,
  z_neg: terrain_sockets.water.layer_down,
  y_pos: terrain_sockets.void,
  y_neg: terrain_sockets.water.material,
}
.to_template()
.with_weight(WATER_WEIGHT);

// Create rotated versions of outer corners
terrain_model_builder.create_model(
  water_corner_out.clone(),
  vec![SpawnableAsset::new("water_corner_out_tl")],
);
terrain_model_builder.create_model(
  water_corner_out.rotated(ModelRotation::Rot90, Direction::ZForward),
  vec![SpawnableAsset::new("water_corner_out_bl")],
);
terrain_model_builder.create_model(
  water_corner_out.rotated(ModelRotation::Rot180, Direction::ZForward),
  vec![SpawnableAsset::new("water_corner_out_br")],
);
terrain_model_builder.create_model(
  water_corner_out.rotated(ModelRotation::Rot270, Direction::ZForward),
  vec![SpawnableAsset::new("water_corner_out_tr")],
);

// Create rotated versions of inner corners
terrain_model_builder.create_model(
  water_corner_in.clone(),
  vec![SpawnableAsset::new("water_corner_in_tl")],
);
terrain_model_builder.create_model(
  water_corner_in.rotated(ModelRotation::Rot90, Direction::ZForward),
  vec![SpawnableAsset::new("water_corner_in_bl")],
);
terrain_model_builder.create_model(

```



```

        water_corner_in.rotated(ModelRotation::Rot180, Direction::ZForward),
        vec![SpawnableAsset::new("water_corner_in_br")],
    );
    terrain_model_builder.create_model(
        water_corner_in.rotated(ModelRotation::Rot270, Direction::ZForward),
        vec![SpawnableAsset::new("water_corner_in_tr")],
    );

    // Create rotated versions of side edges
    terrain_model_builder.create_model(
        water_side.clone(),
        vec![SpawnableAsset::new("water_side_t")],
    );
    terrain_model_builder.create_model(
        water_side.rotated(ModelRotation::Rot90, Direction::ZForward),
        vec![SpawnableAsset::new("water_side_l")],
    );
    terrain_model_builder.create_model(
        water_side.rotated(ModelRotation::Rot180, Direction::ZForward),
        vec![SpawnableAsset::new("water_side_b")],
    );
    terrain_model_builder.create_model(
        water_side.rotated(ModelRotation::Rot270, Direction::ZForward),
        vec![SpawnableAsset::new("water_side_r")],
    );

    // Add connection rules
    socket_collection.add_connections(vec![
        (
            terrain_sockets.water.material,
            vec![terrain_sockets.water.material],
        ),
        (
            terrain_sockets.water.water_and_void,
            vec![terrain_sockets.water.void_and_water],
        ),
    ]);

    // Connect water layer to yellow grass layer
    socket_collection.add_rotated_connection(
        terrain_sockets.yellow_grass.layer_up,
        vec![terrain_sockets.water.layer_down],
    );
}

```

Key Points About Water:

1. **Low Weight Values** - Notice `WATER_WEIGHT: f32 = 0.02`. This makes water appear less frequently than grass, creating occasional water bodies instead of covering everything.
2. **Multiple z_pos Options** - The void model has two options for `z_pos`: `water.layer_up` (another water layer could go here) and `water.ground_up` (props can sit here). This prepares us for the props layer we'll add next.
3. **Same Pattern as Grass** - Water uses the same template and rotation approach as grass, demonstrating how the WFC pattern scales to different terrain types.

Step 4: Calling the Water Layer Function

Update `build_world` in `rules.rs` to call `build_water_layer`:

```

// src/map/rules.rs - Update build_world function
pub fn build_world() -> (
    Vec<Vec<SpawnableAsset>>,

```

```

    ModelCollection<Cartesian3D>,
    SocketCollection,
) {
    let mut socket_collection = SocketCollection::new();
    let terrain_sockets = create_sockets(&mut socket_collection);

    let mut terrain_model_builder = TerrainModelBuilder::new();

    // Build dirt layer
    build_dirt_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    // Build grass layer
    build_grass_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    // Build yellow grass layer
    build_yellow_grass_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    // Line update alert
    // Build water layer
    build_water_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    let (assets, models) = terrain_model_builder.into_parts();

    (assets, models, socket_collection)
}

```

Step 5: Updating Grid Layers

We need one more layer for water. Update the constant in `generate.rs`:

```

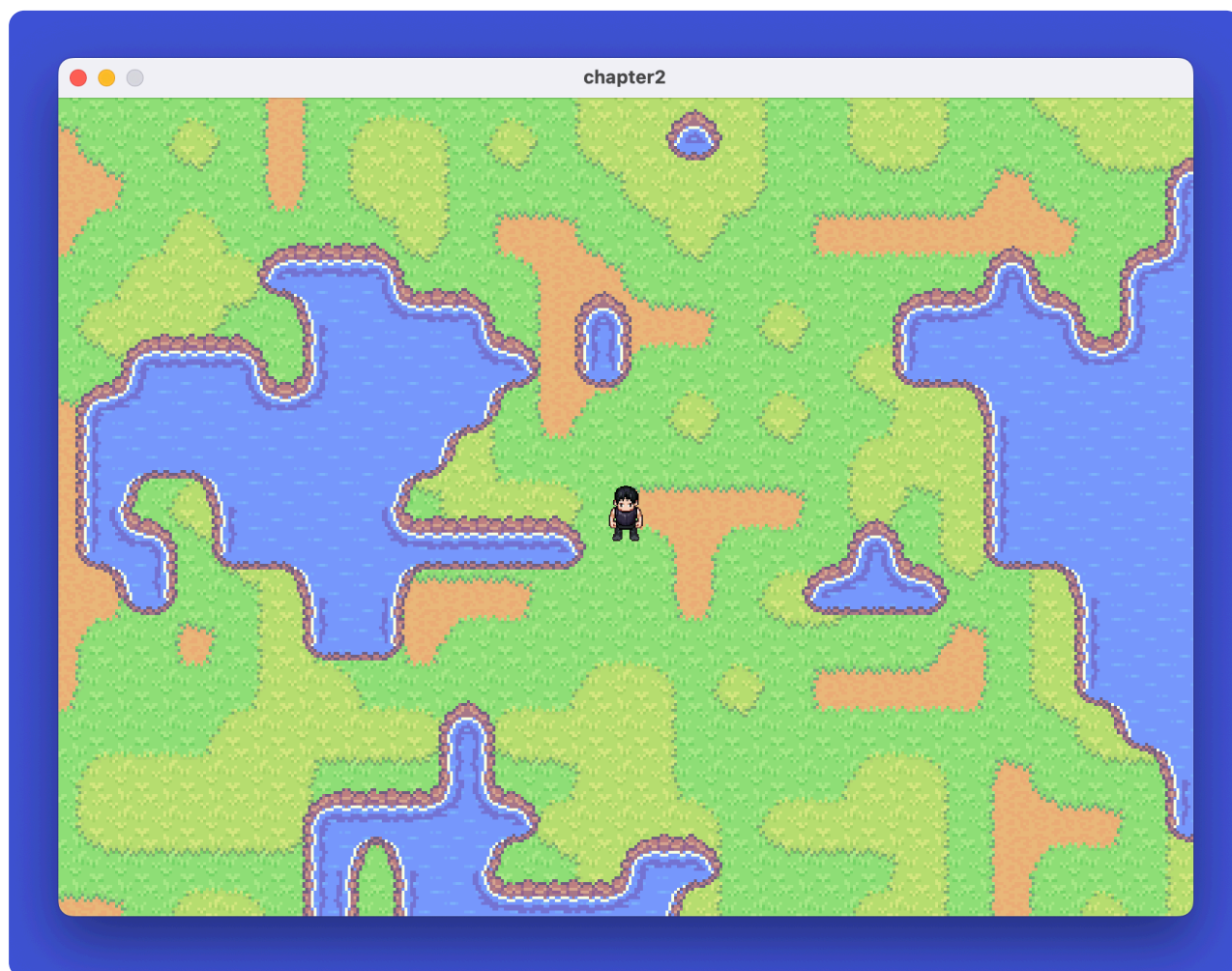
// src/map/generate.rs - Update GRID_Z
const GRID_Z: u32 = 4; // Changed from 3 to 4

```

Now run your game:

```
cargo run
```

You should see water bodies forming on your terrain, creating lakes and ponds alongside the grass patches!



Adding the Props Layer

Props are the final layer that brings our world to life! Trees, rocks, plants, and stumps should appear on land but not in water.

This layer sits at the top of our Z-stack and uses special connection rules to ensure props only spawn on solid ground.

Step 1: Add Props Sprites to Tilemap

First, let's add all the props sprites to our tilemap definition. Open `src/map/tilemap.rs` and add these entries to the `sprites` array:

```
// src/map/tilemap.rs - Add these after the water sprites
TilemapSprite {
    name: "big_tree_1_tl",
    pixel_x: 0,
    pixel_y: 0,
},
TilemapSprite {
    name: "big_tree_1_tr",
    pixel_x: 32,
    pixel_y: 0,
},
TilemapSprite {
    name: "big_tree_1_bl",
    pixel_x: 0,
    pixel_y: 32,
},
TilemapSprite {
    name: "big_tree_1_br",
    pixel_x: 32,
    pixel_y: 32,
},
TilemapSprite {
    name: "big_tree_2_tl",
    pixel_x: 64,
    pixel_y: 0,
},
TilemapSprite {
```

```
    name: "big_tree_2_tr",
    pixel_x: 96,
    pixel_y: 0,
},
TilemapSprite {
    name: "big_tree_2_bl",
    pixel_x: 64,
    pixel_y: 32,
},
TilemapSprite {
    name: "big_tree_2_br",
    pixel_x: 96,
    pixel_y: 32,
},
TilemapSprite {
    name: "plant_1",
    pixel_x: 128,
    pixel_y: 64,
},
TilemapSprite {
    name: "plant_2",
    pixel_x: 160,
    pixel_y: 64,
},
TilemapSprite {
    name: "plant_3",
    pixel_x: 192,
    pixel_y: 64,
},
TilemapSprite {
    name: "plant_4",
    pixel_x: 224,
    pixel_y: 64,
},
TilemapSprite {
    name: "rock_1",
    pixel_x: 0,
    pixel_y: 128,
},
TilemapSprite {
    name: "rock_2",
    pixel_x: 32,
    pixel_y: 128,
},
TilemapSprite {
    name: "rock_3",
    pixel_x: 64,
    pixel_y: 128,
},
TilemapSprite {
    name: "rock_4",
    pixel_x: 96,
    pixel_y: 128,
},
TilemapSprite {
    name: "small_tree_top",
    pixel_x: 128,
    pixel_y: 128,
},
TilemapSprite {
    name: "small_tree_bottom",
    pixel_x: 128,
    pixel_y: 160,
},
TilemapSprite {
```

```

        name: "tree_stump_1",
        pixel_x: 192,
        pixel_y: 128,
    },
    TilemapSprite {
        name: "tree_stump_2",
        pixel_x: 224,
        pixel_y: 128,
    },
    TilemapSprite {
        name: "tree_stump_3",
        pixel_x: 0,
        pixel_y: 192,
    },

```

Step 2: Define Props Sockets

Props need special socket handling because they must only appear on land, never in water. They also include multi-tile objects like big trees that span multiple grid positions.

Add the socket structure to `src/map/sockets.rs`:

```

// src/map/sockets.rs - Add after WaterLayerSockets
pub struct PropsLayerSockets {
    pub layer_up: Socket,
    pub layer_down: Socket,
    pub props_down: Socket,
    pub big_tree_1_base: Socket,
    pub big_tree_2_base: Socket,
}

```

Then update `TerrainSockets` to include props:

```

// src/map/sockets.rs - Update TerrainSockets
pub struct TerrainSockets {
    pub dirt: DirtLayerSockets,
    pub void: Socket,
    pub grass: GrassLayerSockets,
    pub yellow_grass: YellowGrassLayerSockets,
    pub water: WaterLayerSockets,
    pub props: PropsLayerSockets, // Add this line
}

```

Finally, initialize the props sockets in `create_sockets`:

```

// src/map/sockets.rs - Update create_sockets function
pub fn create_sockets(socket_collection: &mut SocketCollection) -> TerrainSockets {
    let mut new_socket = || -> Socket { socket_collection.create() };

    let sockets = TerrainSockets {
        dirt: DirtLayerSockets {
            layer_up: new_socket(),
            material: new_socket(),
            layer_down: new_socket(),
        },
        void: new_socket(),
        grass: GrassLayerSockets {
            layer_up: new_socket(),
            material: new_socket(),
            layer_down: new_socket(),
            void_and_grass: new_socket(),

```

```

        grass_and_void: new_socket(),
        grass_fill_up: new_socket(),
    },
    yellow_grass: YellowGrassLayerSockets {
        layer_up: new_socket(),
        layer_down: new_socket(),
        yellow_grass_fill_down: new_socket(),
    },
    water: WaterLayerSockets {
        layer_up: new_socket(),
        layer_down: new_socket(),
        material: new_socket(),
        void_and_water: new_socket(),
        water_and_void: new_socket(),
        ground_up: new_socket(),
    },
    // Line update alert
    props: PropsLayerSockets {
        layer_up: new_socket(),
        layer_down: new_socket(),
        props_down: new_socket(),
        big_tree_1_base: new_socket(),
        big_tree_2_base: new_socket(),
    },
};
sockets
}

```

Understanding Props Sockets:

Props have 5 sockets with special purposes:

1. `layer_up` and `layer_down` - Standard vertical connections
2. `props_down` - Connects to water's `ground_up` socket (ensures props only on land)
3. `big_tree_1_base` and `big_tree_2_base` - Special sockets for multi-tile trees that need to connect their base parts

Step 3: Building the Props Layer Rules

Now let's create the function that builds the props layer. Add this function to `rules.rs`:

```

// src/map/rules.rs - Add this function after build_water_layer
pub fn build_props_layer(
    terrain_model_builder: &mut TerrainModelBuilder,
    terrain_sockets: &TerrainSockets,
    socket_collection: &mut SocketCollection,
) {
    // Void model - represents areas where no props exist
    terrain_model_builder.create_model(
        SocketsCartesian3D::Multiple {
            x_pos: vec![terrain_sockets.void],
            x_neg: vec![terrain_sockets.void],
            z_pos: vec![terrain_sockets.props.layer_up],
            z_neg: vec![terrain_sockets.props.layer_down],
            y_pos: vec![terrain_sockets.void],
            y_neg: vec![terrain_sockets.void],
        },
        Vec::new(),
    );

    // Weight constants for different prop types
    const PROPS_WEIGHT: f32 = 0.025;
    const ROCKS_WEIGHT: f32 = 0.008;
}

```

```

const PLANTS_WEIGHT: f32 = 0.025;
const STUMPS_WEIGHT: f32 = 0.012;

// Base prop template - single tile props
let prop = SocketsCartesian3D::Simple {
    x_pos: terrain_sockets.void,
    x_neg: terrain_sockets.void,
    z_pos: terrain_sockets.props.layer_up,
    z_neg: terrain_sockets.props.props_down,
    y_pos: terrain_sockets.void,
    y_neg: terrain_sockets.void,
}
.to_template()
.with_weight(PROPS_WEIGHT);

// Create different prop types with different weights
let plant_prop = prop.clone().with_weight(PLANTS_WEIGHT);
let stump_prop = prop.clone().with_weight(STUMPS_WEIGHT);
let rock_prop = prop.clone().with_weight(ROCKS_WEIGHT);

// Small tree (2 tiles high)
terrain_model_builder.create_model(
    plant_prop.clone(),
    vec![
        SpawnableAsset::new("small_tree_bottom"),
        SpawnableAsset::new("small_tree_top").with_grid_offset(GridDelta::new(0, 1, 0)),
    ],
);

// Big tree 1 (2x2 tiles)
terrain_model_builder
    .create_model(
        SocketsCartesian3D::Simple {
            x_pos: terrain_sockets.props.big_tree_1_base,
            x_neg: terrain_sockets.void,
            z_pos: terrain_sockets.props.layer_up,
            z_neg: terrain_sockets.props.props_down,
            y_pos: terrain_sockets.void,
            y_neg: terrain_sockets.void,
        },
        vec![
            SpawnableAsset::new("big_tree_1_bl"),
            SpawnableAsset::new("big_tree_1_tl").with_grid_offset(GridDelta::new(0, 1, 0)),
        ],
    )
    .with_weight(PROPS_WEIGHT);

terrain_model_builder
    .create_model(
        SocketsCartesian3D::Simple {
            x_pos: terrain_sockets.void,
            x_neg: terrain_sockets.props.big_tree_1_base,
            z_pos: terrain_sockets.props.layer_up,
            z_neg: terrain_sockets.props.props_down,
            y_pos: terrain_sockets.void,
            y_neg: terrain_sockets.void,
        },
        vec![
            SpawnableAsset::new("big_tree_1_br"),
            SpawnableAsset::new("big_tree_1_tr").with_grid_offset(GridDelta::new(0, 1, 0)),
        ],
    )
    .with_weight(PROPS_WEIGHT);

// Big tree 2 (2x2 tiles)

```

```

terrain_model_builder
    .create_model(
        SocketsCartesian3D::Simple {
            x_pos: terrain_sockets.props.big_tree_2_base,
            x_neg: terrain_sockets.void,
            z_pos: terrain_sockets.props.layer_up,
            z_neg: terrain_sockets.props.props_down,
            y_pos: terrain_sockets.void,
            y_neg: terrain_sockets.void,
        },
        vec![
            SpawnableAsset::new("big_tree_2_bl"),
            SpawnableAsset::new("big_tree_2_tl").with_grid_offset(GridDelta::new(0, 1, 0)),
        ],
    )
    .with_weight(PROPS_WEIGHT);

terrain_model_builder
    .create_model(
        SocketsCartesian3D::Simple {
            x_pos: terrain_sockets.void,
            x_neg: terrain_sockets.props.big_tree_2_base,
            z_pos: terrain_sockets.props.layer_up,
            z_neg: terrain_sockets.props.props_down,
            y_pos: terrain_sockets.void,
            y_neg: terrain_sockets.void,
        },
        vec![
            SpawnableAsset::new("big_tree_2_br"),
            SpawnableAsset::new("big_tree_2_tr").with_grid_offset(GridDelta::new(0, 1, 0)),
        ],
    )
    .with_weight(PROPS_WEIGHT);

// Tree stumps
terrain_model_builder.create_model(
    stump_prop.clone(),
    vec![SpawnableAsset::new("tree_stump_1")],
);
terrain_model_builder.create_model(
    stump_prop.clone(),
    vec![SpawnableAsset::new("tree_stump_2")],
);
terrain_model_builder.create_model(
    stump_prop.clone(),
    vec![SpawnableAsset::new("tree_stump_3")],
);

// Rocks
terrain_model_builder.create_model(rock_prop.clone(), vec![SpawnableAsset::new("rock_1")]);
terrain_model_builder.create_model(rock_prop.clone(), vec![SpawnableAsset::new("rock_2")]);
terrain_model_builder.create_model(rock_prop.clone(), vec![SpawnableAsset::new("rock_3")]);
terrain_model_builder.create_model(rock_prop.clone(), vec![SpawnableAsset::new("rock_4")]);

// Plants
terrain_model_builder.create_model(plant_prop.clone(), vec![SpawnableAsset::new("plant_1")]);
terrain_model_builder.create_model(plant_prop.clone(), vec![SpawnableAsset::new("plant_2")]);
terrain_model_builder.create_model(plant_prop.clone(), vec![SpawnableAsset::new("plant_3")]);
terrain_model_builder.create_model(plant_prop.clone(), vec![SpawnableAsset::new("plant_4")]);

// Add connection rules
socket_collection.add_connections(vec![
    (
        terrain_sockets.props.big_tree_1_base,
        vec![terrain_sockets.props.big_tree_1_base],
    )
])

```



```

    ),
    (
        terrain_sockets.props.big_tree_2_base,
        vec![terrain_sockets.props.big_tree_2_base],
    ),
]);

// Connect props to water layer
socket_collection
    .add_rotated_connection(
        terrain_sockets.water.layer_up,
        vec![terrain_sockets.props.layer_down],
    )
    .add_rotated_connection(
        terrain_sockets.props.props_down,
        vec![terrain_sockets.water.ground_up],
    );
}

```

Key Points About Props:

1. **Multi-tile Objects** - Big trees use `GridDelta::new(0, 1, 0)` to place the top half one tile up
2. **Weight System** - Different prop types have different spawn probabilities (rocks are rarer than plants)
3. **Land-only Rule** - `props_down` connects to `water.ground_up`, ensuring props never spawn in water
4. **Base Sockets** - Big trees use special base sockets to connect their left and right halves

Step 4: Calling the Props Layer Function

Update `build_world` in `rules.rs` to call `build_props_layer`:

```

// src/map/rules.rs - Update build_world function
pub fn build_world() -> (
    Vec<Vec<SpawnableAsset>>,
    ModelCollection<Cartesian3D>,
    SocketCollection,
) {
    let mut socket_collection = SocketCollection::new();
    let terrain_sockets = create_sockets(&mut socket_collection);

    let mut terrain_model_builder = TerrainModelBuilder::new();

    // Build dirt layer
    build_dirt_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    // Build grass layer
    build_grass_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );

    // Build yellow grass layer
    build_yellow_grass_layer(
        &mut terrain_model_builder,
        &terrain_sockets,
        &mut socket_collection,
    );
}

```



```

// Build water layer
build_water_layer(
    &mut terrain_model_builder,
    &terrain_sockets,
    &mut socket_collection,
);

// Line update alert
// Build props layer
build_props_layer(
    &mut terrain_model_builder,
    &terrain_sockets,
    &mut socket_collection,
);

let (assets, models) = terrain_model_builder.into_parts();

(assets, models, socket_collection)
}

```

Step 5: Updating Grid Layers

We need one final layer for props. Update the constant in `generate.rs`:

```

// src/map/generate.rs - Update GRID_Z
const GRID_Z: u32 = 5; // Changed from 4 to 5

```

Now run your game:

```
cargo run
```

You should see a complete procedural world with dirt, grass, water, and props! Trees and rocks will only appear on land, never in water, creating a realistic and varied landscape.



Congratulations!

You've successfully built a complete procedural terrain generation system using Wave Function Collapse! Your world now has:

- **Dirt base layer** - The foundation
- **Green grass patches** - With smooth edges and corners
- **Yellow grass variety** - Stacking on green grass
- **Water bodies** - Creating lakes and ponds
- **Props** - Trees, rocks, and plants that only appear on land

This demonstrates the power of WFC for creating coherent, natural-looking game worlds with just a few simple rules!

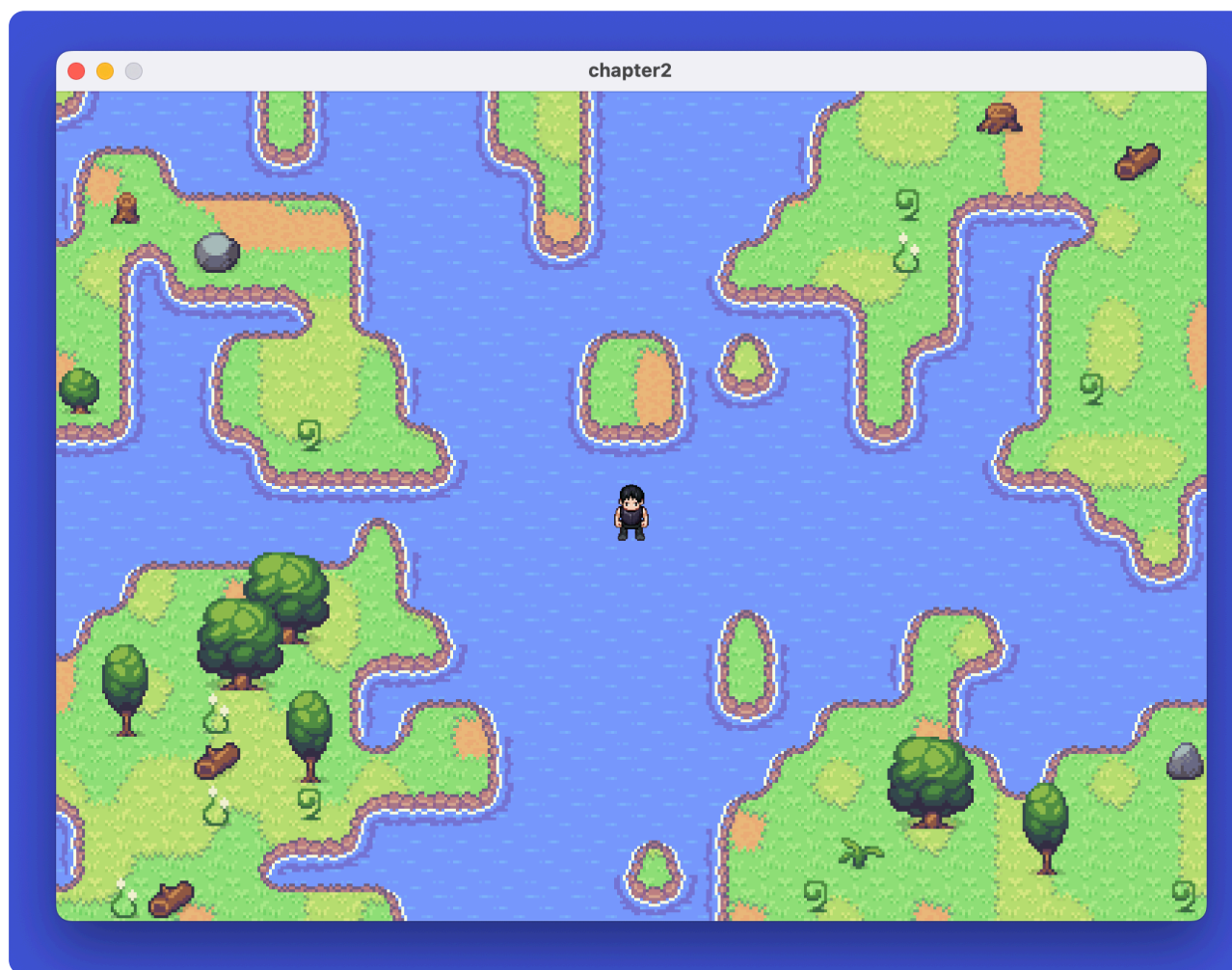
Wait, Just one more thing!

Go to `rules.rs` and change the water weight.

```
const WATER_WEIGHT: f32 = 0.07;
```

Now run your game:

```
cargo run
```



Woah, with a simple modification you are able to change the world to have more water! This demonstrates the power of procedural generation—by tweaking just a few numbers, you can create completely different landscapes.

Try experimenting with the weight values for different layers to see how dramatically you can transform your world.

Also notice our player can walk on water. We will work on collision detection and also on approaches to build larger maps in our upcoming chapters.

My player walks on water
without cheat codes!



Found Jesus mode!



The Impatient Programmer's Guide to Bevy and Rust: Chapter 3 - Let The Data Flow

I was supposed to release this on Halloween, but my code was so scary it kept running away. 🎃
Now that we've debugged the ghosts in the machine, let's begin!

By the end of this tutorial, you'll have a flexible, data-driven character system that supports character switching, multiple animation types (Walk, Run, Jump) configured through a data file.



The Problem with Hardcoded Characters

In Chapter 1, we built a player with movement and animation, however **everything was hardcoded and tightly coupled**.

```
// Pseudo code warning, don't use
// From Chapter 1 player.rs - Everything is hardcoded!
const TILE_SIZE ... = 64;           // ← Hardcoded sprite size
const WALK_FRAMES ... = 9;          // ← Hardcoded frame count
const MOVE_SPEED ... = 140.0;       // ← Hardcoded movement speed
const ANIM_DT ... = 0.1;            // ← Hardcoded animation timing
```

Maintenance Nightmare

Let's see what happens when you add a second character. You'd need to duplicate everything:

```
// Pseudo code warning, don't use
// First character
const WARRIOR_TILE_SIZE ... = 64;
const WARRIOR_WALK_FRAMES ... = 9;
const WARRIOR_MOVE_SPEED ... = 140.0;
```

```
fn spawn_warrior(...) { /* 50 lines of code */ }
fn animate_warrior(...) { /* 80 lines of animation logic */ }
fn warrior_row_zero_based(...) { /* row mapping */ }

// Second character - DUPLICATE ALL THE CODE
const MAGE_TILE_SIZE ... = 64;
const MAGE_WALK_FRAMES ... = 6;
const MAGE_MOVE_SPEED ... = 100.0;

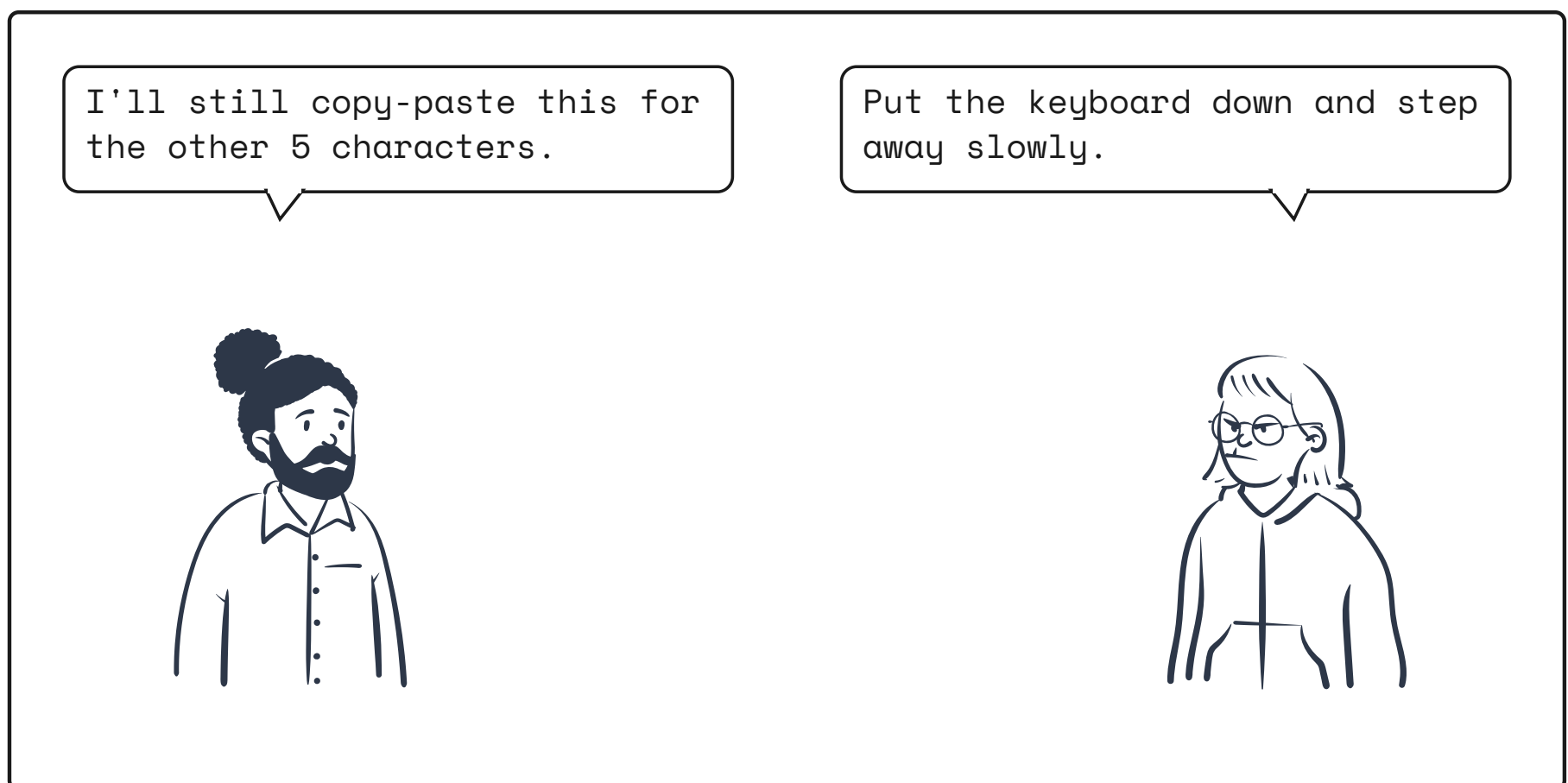
fn spawn_mage(...) { /* 50 lines of IDENTICAL code */ }
fn animate_mage(...) { /* 80 lines of IDENTICAL animation logic */ }
fn mage_row_zero_based(...) { /* different row mapping */ }
```

Now imagine you discover a bug in your animation system. You need to fix it in:

- `animate_warrior()`
- `animate_mage()`
- `animate_rogue()`
- `animate_paladin()`
- ...and 6 more character functions

Miss one? That character breaks. Want to add a “jump” animation? Update 10 functions. Want to change how movement works? Touch every single character’s code.

This is the copy-paste maintenance nightmare you want to avoid.



Data-Driven Design

The solution lies in **data-oriented programming**, a design approach where we **separate what things are (data) from what they do (behavior)**.

Instead of tightly coupling character attributes with character-specific code, we:

1. Separate Data from Code

Move character properties into a single external `.ron` configuration file:

```
// characters.ron, All characters in one file!
(
  characters: [
```



```
(
  (
    name: "Warrior",
    base_move_speed: 140.0,
    max_health: 150.0,
    animations: {...}
  ),
  (
    name: "Mage",
    base_move_speed: 100.0,
    max_health: 80.0,
    animations: {...}
  ),
]
```

What's a `.ron` file?

RON stands for **Rusty Object Notation**, a data format similar to JSON but designed for Rust. It's human-readable, supports Rust types like tuples and structs, and allows comments. Think of it as JSON that feels native to Rust developers.

JSON	RON
Requires quotes on every key	Optional quotes for simple identifiers
No comment support	Inline and multiline comments, document your data directly
Trailing commas cause syntax errors	Trailing commas allowed
Limited to JavaScript types	Native Rust types (tuples, structs, enums), matches your code

RON eliminates JSON's verbosity while adding features that Rust developers need, making it ideal for game configuration.

2. Write Systems that Operate on Data

Build generic systems that work with **any** character data:

```
// Pseudo code warning, don't use
// Before: Code + Data mixed together
fn animate_warrior(...) { /* hardcoded warrior logic */ }
fn animate_mage(...) { /* hardcoded mage logic */ }

// After: Flexible system that operates on data
fn animate_characters(...) {
  // Reads character data and animates accordingly
  // Works for warrior, mage, rogue, or any future character!
}
```

Benefits of Separation

When data lives separately from code, the same animation system adapts to any character automatically.

Fix a bug? **One place**. Change animation speeds or frame counts? **Update the data file, no code changes**. Switch characters? **Load different data from the file**.

But the benefits extend far beyond maintenance. This separation helps with powerful capabilities that would be difficult or impossible with hardcoded values.

You can load characters from a network server at runtime, enabling downloadable content and live game updates without redistributing your entire game binary.

Players can create their own custom characters by simply editing the `.ron` file, opening the door to user-generated content.

What We'll Build in This Chapter:

While the data-driven approach opens up all these possibilities, we'll start with the foundation:

- 1. **Creating & loading external `.ron` file** containing all character data
- 2. **Generic animation system** that works with any character
- 3. **Runtime character switching** with number keys (1-6)
- 4. **Multiple animation types** (Walk, Run, Jump) per character

Ready to build this data-driven character system? Let's dive in!

Setting Up the Characters Module

Find the Chapter 3 project files from the [repo](#) and copy these spritesheets into your project's `src/assets/` directory:

```
male_spritesheet.png
female_spritesheet.png
crimson_count_spritesheet.png
graveyard_reaper_spritesheet.png
lantern_warden_spritesheet.png
starlit_oracle_spritesheet.png
```



Character Schema

Every entry in `characters.ron` follows the same structure:

- `name`: Identifier that shows up in logs/UI.
- `max_health`, `base_move_speed`, `run_speed_multiplier`: Gameplay attributes.
- `texture_path`: Which spritesheet to load.
- `tile_size`: Each frame's width/height in pixels.
- `atlas_columns`: How many columns exist in the spritesheet grid.
- `animations`: Map where the key is an `AnimationType` (`Walk`, `Run`, `Jump`) and the value is:
 - `start_row`: Row number in the spritesheet grid, counting from 0 at the top.

- `frame_count`: Number of frames for that animation.
- `frame_time`: Seconds per frame.
- `directional`: `true` when the spritesheet contains four direction rows (Up, Left, Down, Right) stacked vertically for that animation. If `false`, Bevy uses the same row regardless of facing direction.

Create a folder `src/assets/characters/` and copy the `characters.ron` file from the [repo](#) and add place it inside the folder. It includes data for all 6 characters.

Setting Up the Config

With our data file in place, we need code that reads it and spawns the player. We're going to replace the Chapter 1 `player.rs` with a new `characters` module.

Hence delete `src/player.rs` and remove `mod player;` and `PlayerPlugin` usage from `main.rs`.

Create a new folder `src/characters/` and create the file `config.rs` inside it.

```
characters/
├─ config.rs
```

First we need to define what type of animation are possible. Presently we only support `Walk`, `Run` and `Jump` animations. `AnimationDefinition` captures where each animation lives in the spritesheet, how many frames it has, and how fast it should play.

```
// characters/config.rs
use bevy::prelude::*;
use serde::{Deserialize, Serialize};
use std::collections::HashMap;

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub enum AnimationType {
    Walk,
    Run,
    Jump
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AnimationDefinition {
    pub start_row: usize,
    pub frame_count: usize,
    pub frame_time: f32,
    pub directional: bool, // true = 4 rows (one per direction), false = 1 row
}
```

What's `Hash`, `Serialize`, and `Deserialize`?

`Hash` lets us use `AnimationType` as a key inside `HashMap`, so retrieving the settings for `AnimationType::Run` is just a dictionary lookup. `Serialize` and `Deserialize` allow Rust to turn these structs into `.ron` text (and back) automatically when you load or save character data.

With the animation schema in place, we can define `CharacterEntry`, the asset we load from `characters.ron`. It bundles character attributes, sprite metadata, and the animation map so every system pulls the info it needs from a single struct record.

Append the following code to `characters/config.rs`.

```
// Append this to characters/config.rs
#[derive(Component, Asset, TypePath, Debug, Clone, Serialize, Deserialize)]
pub struct CharacterEntry {
    pub name: String,
    pub max_health: f32,
```

```

    pub base_move_speed: f32,
    pub run_speed_multiplier: f32,
    pub texture_path: String,
    pub tile_size: u32,
    pub atlas_columns: usize,
    pub animations: HashMap<AnimationType, AnimationDefinition>,
}

impl CharacterEntry {
    pub fn calculate_max_animation_row(&self) -> usize {
        self.animations
            .values()
            .map(|def| if def.directional { def.start_row + 3 } else { def.start_row })
            .max()
            .unwrap_or(0)
    }
}

#[derive(Asset, TypePath, Debug, Clone, Serialize, Deserialize)]
pub struct CharactersList {
    pub characters: Vec<CharacterEntry>,
}

```

The `calculate_max_animation_row` helper inspects every animation definition to figure out how many rows the texture atlas needs.

Directional animations like `Walk` often consume four stacked rows (Up, Left, Down, Right), while others, say a climb animation, may only need a single row regardless of facing. This helper keeps those differences data-driven so atlas loading code can stay generic.

Is `self.animations` calling multiple functions sequentially?

Yes! This is called **method chaining**. Each function runs in order: first `.values()` gets all animation definitions from the HashMap, then `.map()` transforms each one, then `.max()` finds the largest value. They execute top-to-bottom, one after another.

What's `map` doing, also is that a closure inside `map`?

`map` transforms each item in the collection. Yes, `|def|` is a **closure** as we studied in the previous chapter. It takes each animation definition (`def`) and calculates its maximum row: if the animation is directional (4 rows), it returns `start_row + 3`; otherwise just `start_row`. Think of it as “for each animation, calculate its end row.”

What's `unwrap_or`?

`max()` returns an `Option<usize>`, it could be `Some(number)` if there are animations, or `None` if the HashMap is empty. `unwrap_or(0)` says “if you got a number, give it to me; if you got `None`, use `0` instead.” This prevents crashes when a character has no animations defined.

`CharactersList` groups all your `CharacterEntry` configs into one loadable asset, so Bevy can read every character's data from a single JSON/RON file instead of loading many separate files.

What's the purpose of using `Asset`, `TypePath` macros?

`Asset` simply tells Bevy “this struct is something you can load from disk and store in the asset server.” `TypePath` gives Bevy a unique name for the type so it knows exactly which asset you're asking for later. Together they turn `CharacterEntry` / `CharactersList` into first-class loadable data, the same way textures or audio files already work.

What's `HashMap<AnimationType, AnimationDefinition>` doing?

Each character needs different timing and sprite rows for `Walk`, `Run`, `Jump`, etc. The `HashMap` is simply a lookup table with key as `AnimationType`, so when the animation system asks for `AnimationType::Run`, it instantly receives the corresponding `AnimationDefinition` (start row, frame count, frame speed, directional flag).

Now that we have a data structure to hold our character information, we need a system to bring it to life.

The Animation Engine

Here we will be building the animation engine to interpret our data structure.

Create a new file `src/characters/animation.rs`. This will house the logic that brings our sprites to life.

```
characters/  
├─ config.rs  
├─ animation.rs  <- Create this
```

Our animation engine needs to help us with the following:

1. **Direction Tracking:** When your character moves right, you want them to face right. When they move up, they should face up. We need a system to convert movement into facing direction.
2. **State Management:** We need to know *when* to change animations. Did the player just start running? Just stop? Just jump? These transitions are when we reset the animation.
3. **Frame Calculation:** Given a character's current state ("running left"), which exact frame from the spritesheet should we display right now?

Let's build each piece.

Direction Tracking



When your player presses the arrow keys, we get a velocity vector like `Vec2 { x: 1.0, y: 0.0 }` for moving right. But our spritesheet doesn't understand vectors—it has specific rows for Up, Down, Left, and Right animations.

We need to translate "moving in this direction" into "show this specific row of sprites." That's what the `Facing` enum does.

Add this to `src/characters/animation.rs`:

```
// src/characters/animation.rs  
use bevy::prelude::*;  
use serde::{Deserialize, Serialize};  
use crate::characters::config::{CharacterEntry, AnimationType};  
  
// Default animation timing (10 FPS = 0.1 seconds per frame)
```

```

pub const DEFAULT_ANIMATION_FRAME_TIME: f32 = 0.1;

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub enum Facing {
    Up,
    Left,
    Down,
    Right,
}

impl Facing {
    // Convert a velocity vector into a discrete direction
    pub fn from_direction(direction: Vec2) -> Self {
        if direction.x.abs() > direction.y.abs() {
            if direction.x > 0.0 { Facing::Right } else { Facing::Left }
        } else {
            if direction.y > 0.0 { Facing::Up } else { Facing::Down }
        }
    }

    // Helper to map direction to row offset (0, 1, 2, 3)
    fn direction_index(self) -> usize {
        match self {
            Facing::Up => 0,
            Facing::Left => 1,
            Facing::Down => 2,
            Facing::Right => 3,
        }
    }
}

```

Converting Movement to Facing

The `from_direction` function takes a velocity vector and figures out which direction is dominant. If the player is moving diagonally (both x and y are non-zero), we pick the stronger component. Moving mostly right with a bit of up? Face right. Moving mostly up with a bit of right? Face up. This ensures your character always faces the most relevant direction during gameplay.

Mapping Direction to Sprite Rows

Our spritesheets follow a convention. For directional animations like “Walk”, the rows are organized as:

- Row 0: Walk Up
- Row 1: Walk Left
- Row 2: Walk Down
- Row 3: Walk Right

The `direction_index` function converts our `Facing` enum into these row offsets (0, 1, 2, 3). So when we know the player is facing `Down` and we want to play the “Walk” animation starting at row 8, we calculate: `8 + Facing::Down.direction_index()` = `8 + 2` = row 10. That’s where the “Walk Down” frames live in the atlas.



Tracking Animation State

We need to know *when* to change animations. If your character transitions from standing still to running, we need to detect that moment and restart the animation from frame 0. Otherwise, the run animation might start mid-cycle, looking janky.

We need these components to track the *current* state of the animation.

- `AnimationController`: It knows what animation we *want* to play (e.g., “Run”) and which way we are facing.
- `AnimationState`: It tracks if we are moving or jumping, and importantly, if we *were* moving last frame. This helps us detect state *changes* (like starting to run) so we can reset the animation timer.
- `AnimationTimer`: Controls how fast frames update.

Add these components to `src/characters/animation.rs`:

```
// Append this to src/characters/animation.rs
// Component that holds animation configuration
#[derive(Component)]
pub struct AnimationController {
    pub current_animation: AnimationType,
    pub facing: Facing,
}

impl Default for AnimationController {
    fn default() -> Self {
        Self {
            current_animation: AnimationType::Walk,
            facing: Facing::Down,
        }
    }
}

#[derive(Component, Default)]
pub struct AnimationState {
    pub is_moving: bool,
    pub was_moving: bool,
    pub is_jumping: bool,
    pub was_jumping: bool,
}
```

```
#[derive(Component, Deref, DerefMut)]
pub struct AnimationTimer(pub Timer);
```

You might notice we're using boolean flags (`is_moving`, `was_moving`, `is_jumping`, `was_jumping`) to track animation state. While this works for our current simple case, it's not the best approach for managing state transitions. As your game grows more complex (adding attacks, dodges, climbing, etc.), managing all these boolean combinations becomes error-prone and hard to maintain.

In Chapter 4, we'll learn about the **state machine design pattern**, which provides a much cleaner and more scalable way to handle state transitions. For now, we'll use this simpler approach to keep our focus on understanding the animation system fundamentals.

Frame Calculation

Now we know *which* animation to play (Walk, Run, Jump) and *which* direction the character is facing. But how do we translate that into “show frame 47 from the texture atlas right now”?

Think of the spritesheet as a numbered grid, reading left-to-right, top-to-bottom. If “Walk Down” is on row 2 and the grid has 12 columns, the first frame of that animation is at position 24 (because we skip the first 2 rows: $2 \times 12 = 24$). If the animation has 6 frames, we'll cycle through positions 24, 25, 26, 27, 28, 29, then loop back to 24.

We'll create an `AnimationClip` struct to handle this math for us.

```
// Append this to src/characters/animation.rs
// Runtime animation clip helper
#[derive(Clone, Copy)]
pub struct AnimationClip {
    first: usize,
    last: usize,
}

impl AnimationClip {
    pub fn new(row: usize, frame_count: usize, atlas_columns: usize) -> Self {
        let first = row * atlas_columns;
        Self {
            first,
            last: first + frame_count - 1,
        }
    }

    pub fn start(self) -> usize {
        self.first
    }

    // Check if a frame index belongs to this clip
    pub fn contains(self, index: usize) -> bool {
        (self.first..=self.last).contains(&index)
    }

    // Calculate the next frame, looping back to start if needed
    pub fn next(self, index: usize) -> usize {
        if index >= self.last {
            self.first
        } else {
            index + 1
        }
    }

    // Check if animation has completed (used for non-looping animations like Jump)
```

```

    pub fn is_complete(self, current_index: usize, timer_finished: bool) -> bool {
        current_index >= self.last && timer_finished
    }
}

```

The `AnimationClip` struct stores just two numbers: the `first` and `last` frame indices for a specific animation sequence. The `new` method calculates these indices from the row, frame count, and atlas width.

The `start` method returns where the animation begins. The `contains` method checks if a given frame index belongs to this clip (useful for detecting if we've wandered into the wrong animation). The `next` method advances to the next frame, automatically looping back to the start when we reach the end.

The `is_complete` method checks if we've reached the last frame and the timer has finished—this is crucial for non-looping animations like Jump, where we need to know when to transition back to Walk.

Connecting Clips to Controllers

Now that we have a way to represent frame ranges, we need to connect it to our `AnimationController`. Remember, the controller knows *what* animation to play (“Run”) and *which* direction we're facing (“Left”). We'll add a helper method that combines this information with the character's configuration data to produce the correct `AnimationClip`.

Now we can add a method to `AnimationController` to easily get the current clip based on the character's config:

```

// Append this to src/characters/animation.rs
impl AnimationController {
    pub fn get_clip(&self, config: &CharacterEntry) -> Option<AnimationClip> {
        // 1. Get the definition (e.g. "Walk" data)
        let def = config.animations.get(&self.current_animation)?;

        // 2. Calculate the actual row based on facing direction
        let row = if def.directional {
            def.start_row + self.facing.direction_index()
        } else {
            def.start_row
        };

        // 3. Create the clip
        Some(AnimationClip::new(row, def.frame_count, config.atlas_columns))
    }
}

```

Animating Characters

Finally, the system that ties it all together. This system runs every frame and:

1. Checks if the animation changed (e.g., started moving).
2. If changed, resets the timer and frame index.
3. If not changed, ticks the timer and advances the frame.

```

// Append this to src/characters/animation.rs
// Generic animation system - works for ALL entities
pub fn animate_characters(
    time: Res<Time>,
    mut query: Query<(
        &AnimationController,
        &AnimationState,
        &mut AnimationTimer,
    )>,
) {
    for (mut controller, state, mut timer) in query.iter_mut() {
        // ... animation logic ...
    }
}

```

```

        &mut Sprite,
        &CharacterEntry,
    )>,
) {
    for (animated, state, mut timer, mut sprite, config) in query.iter_mut() {

        let Some(atlas) = sprite.texture_atlas.as_mut() else { continue; };

        // Get the correct clip for current state/facing
        let Some(clip) = animated.get_clip(config) else { continue; };

        // Get timing info
        let Some(anim_def) = config.animations.get(&animated.current_animation) else { continue; };

        // Safety: If we somehow ended up on a frame outside our clip, reset.
        if !clip.contains(atlas.index) {
            atlas.index = clip.start();
            timer.0.reset();
        }

        // Detect state changes
        let just_started_moving = state.is_moving && !state.was_moving;
        let just_stopped_moving = !state.is_moving && state.was_moving;
        let just_started_jumping = state.is_jumping && !state.was_jumping;
        let just_stopped_jumping = !state.is_jumping && state.was_jumping;

        let should_animate = state.is_jumping || state.is_moving;
        let animation_changed = just_started_moving || just_started_jumping
            || just_stopped_moving || just_stopped_jumping;

        if animation_changed {
            // Reset animation
            atlas.index = clip.start();
            timer.0.set_duration(std::time::Duration::from_secs_f32(anim_def.frame_time));
            timer.0.reset();
        } else if should_animate {
            // Advance animation
            timer.tick(time.delta());
            if timer.just_finished() {
                atlas.index = clip.next(atlas.index);
            }
        } else {
            // When idle (not moving or jumping), stay on frame 0
            if atlas.index != clip.start() {
                atlas.index = clip.start();
            }
        }
    }
}

// Helper to update "was_moving" flags at the end of the frame
pub fn update_animation_flags(mut query: Query<&mut AnimationState>) {
    for mut state in query.iter_mut() {
        state.was_moving = state.is_moving;
        state.was_jumping = state.is_jumping;
    }
}

```

How the Animation System Works

The animation system has three branches that handle different scenarios:

1. **Animation changed** (starting/stopping movement or jumping): Reset to frame 0 and update the timer duration for the new animation.

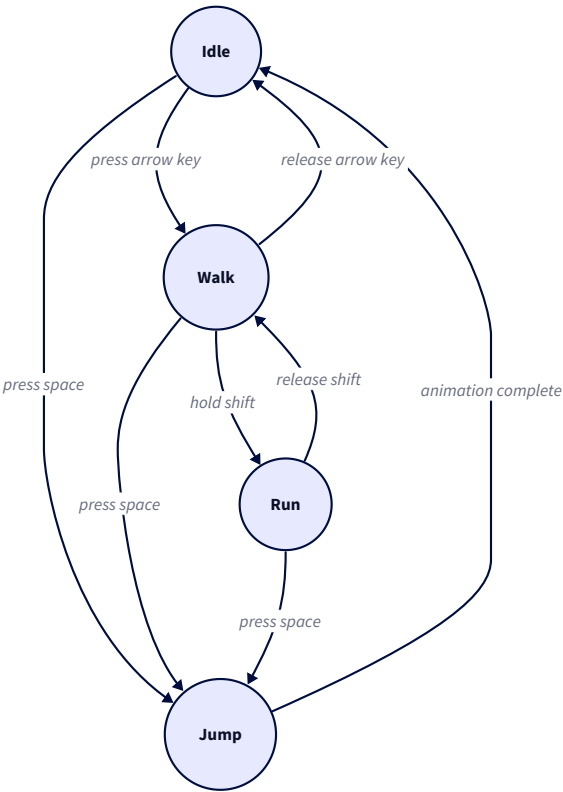
- 2. **Should animate** (actively moving or jumping): Tick the timer and advance through frames.
- 3. **Idle state** (standing still): Ensure we stay on frame 0, the neutral standing pose.

Why State Change Detection Matters

The first branch relies on detecting the *exact moment* a state changes. `is_moving` tells us the current state (“am I moving right now?”), while `was_moving` tells us the previous frame’s state (“was I moving last frame?”). When `is_moving` is true but `was_moving` is false, we know the player *just* pressed a movement key.

This detection is crucial for smooth transitions. Without it, animations would continue from wherever they left off. Imagine your character’s walk cycle is on frame 5, then you press jump—without resetting, the jump animation would start at frame 5 instead of frame 0, looking broken.

The third branch (idle state) handles a different case: when the player *stops* moving, we transition to Walk animation but need to ensure we display the idle pose (frame 0), not whatever frame the walk cycle was on when they stopped.



Why is he running while standing still?



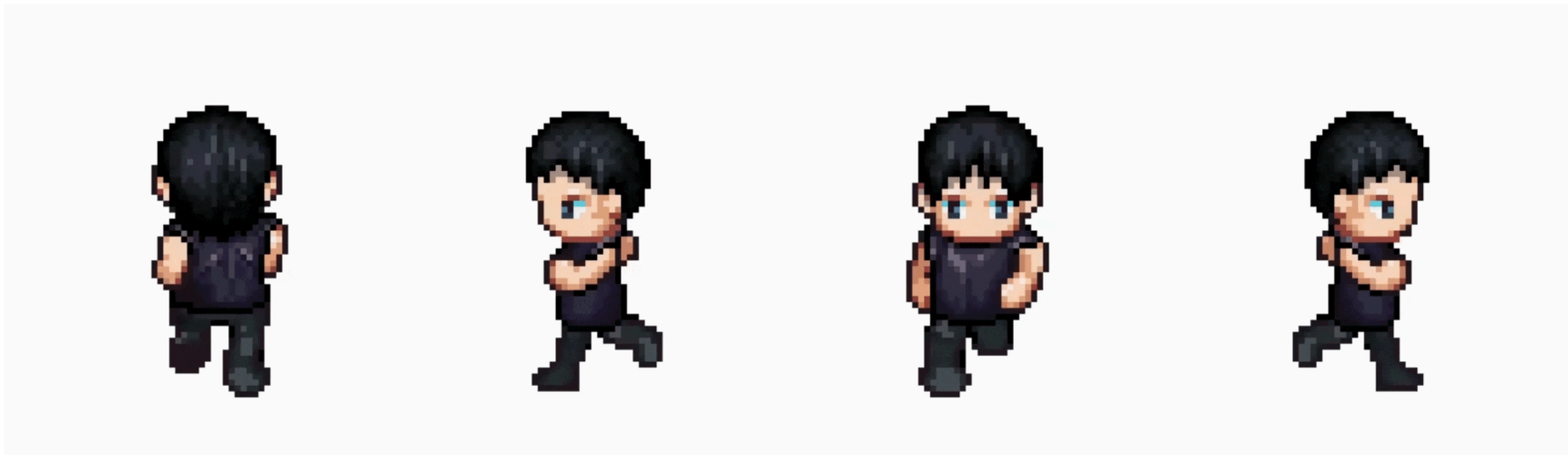
He's just really excited to be idle.



Why do we need `update_animation_flags`?

We need `update_animation_flags` to run *after* all logic is done, so that in the *next* frame, `was_moving` correctly reflects the previous frame's state. This allows us to detect the exact moment a state changes.

The Movement System



Our animation engine can display the right frames, but it needs to know *what* the player is doing. The `AnimationController` we built earlier stores the *current* animation state (“I’m running left”), but something needs to *update* that state based on player input. That’s where the movement system comes in. It reads keyboard input, moves the character, and tells the `AnimationController` which animation to play.

Create a new file `src/characters/movement.rs`:

```
characters/  
├─ config.rs  
├─ animation.rs  
└─ movement.rs <- Create this
```

The movement system has three responsibilities:

1. **Input Reading:** Convert arrow key presses into a direction vector
2. **Movement Calculation:** Apply speed and delta time to move the character smoothly
3. **Animation Coordination:** Tell the animation system when to switch between Walk, Run, and Jump

Here’s how we’ll tackle this:

Reading Player Input

When the player presses arrow keys, we need to convert those discrete button presses into a continuous direction vector. If they press Up and Right simultaneously, we want `Vec2 { x: 1.0, y: 1.0 }` for diagonal movement.

Add this to `src/characters/movement.rs`:

```
// src/characters/movement.rs  
use bevy::prelude::*;
```

```

use crate::characters::animation::*;
use crate::characters::config::{CharacterEntry, AnimationType};

/// Read directional input and return a direction vector
fn read_movement_input(input: &ButtonInput<KeyCode>) -> Vec2 {
    const MOVEMENT_KEYS: [(KeyCode, Vec2); 4] = [
        (KeyCode::ArrowLeft, Vec2::NEG_X),
        (KeyCode::ArrowRight, Vec2::X),
        (KeyCode::ArrowUp, Vec2::Y),
        (KeyCode::ArrowDown, Vec2::NEG_Y),
    ];

    MOVEMENT_KEYS.iter()
        .filter(|(key, _)| input.pressed(*key))
        .map(|(_, dir)| *dir)
        .sum()
}

```

What's happening here?

We define a constant array mapping each arrow key to its direction vector. `Vec2::NEG_X` means “negative X direction” (left), `Vec2::X` means “positive X direction” (right), and so on.

Then we iterate through all four keys, filter to only the ones currently pressed, extract their direction vectors, and sum them. If both Up and Right are pressed, we get `Vec2::Y + Vec2::X` = `Vec2 { x: 1.0, y: 1.0 }`.

What's `_` in `(key, _)` and in `(_, dir)`?

The `_` (underscore) is Rust's “I don't care” placeholder in pattern matching. When destructuring tuples, you use `_` to ignore values you don't need:

- In `(key, _)`: We only need the `key` to check if it's pressed, so we ignore the direction with `_`
- In `(_, dir)`: We only need the `dir` (direction vector), so we ignore the key with `_`

This is more readable than naming unused variables like `(key, _unused_dir)` or `(_unused_key, dir)`. Rust's compiler also knows these values are intentionally ignored, so you won't get warnings about unused variables.

Calculating Movement Speed

Different characters move at different speeds. The Male character might be slower, while the Female character is faster. We also need to support running (holding Shift).

Add this helper function:

```

// Append this to src/characters/movement.rs
/// Calculate movement speed based on character config and running state
fn calculate_movement_speed(character: &CharacterEntry, is_running: bool) -> f32 {
    if is_running {
        character.base_move_speed * character.run_speed_multiplier
    } else {
        character.base_move_speed
    }
}

```

This reads the character's `base_move_speed` from our data file and multiplies it by `run_speed_multiplier` if the player is holding Shift. All the speed values are data-driven—no hardcoded constants!

The Player Marker

We need a way to identify which entity is the player. We'll use a simple marker component:

```
// Append this to src/characters/movement.rs
/// Marker component for the player entity
#[derive(Component)]
pub struct Player;
```

This component has no data, it's just a tag. When we spawn the player entity, we'll attach this component. Then our movement system can query for entities with `Player` to find the player. We have already studied this in the first chapter.

The Movement System

Now we tie it all together. This system runs every frame, reads input, calculates movement, and updates the animation state:

```
// Append this to src/characters/movement.rs
/// Handle player movement input and update transform/animation
pub fn move_player(
    input: Res<ButtonInput<KeyCode>>,
    time: Res<Time>,
    mut query: Query<(
        &mut Transform,
        &mut AnimationController,
        &mut AnimationState,
        &CharacterEntry,
    ), With<Player>>,
) {
    let Ok((mut transform, mut animated, mut state, character)) = query.single_mut() else {
        return;
    };

    let direction = read_movement_input(&input);

    // Check for jump input (space key)
    if input.just_pressed(KeyCode::Space) {
        state.is_jumping = true;
        animated.current_animation = AnimationType::Jump;
    }

    // Check if running
    let is_running = input.pressed(KeyCode::ShiftLeft) || input.pressed(KeyCode::ShiftRight);

    // Handle movement
    if direction != Vec2::ZERO {
        let move_speed = calculate_movement_speed(character, is_running);
        let delta = direction.normalize() * move_speed * time.delta_secs();
        transform.translation += delta.extend(0.0);

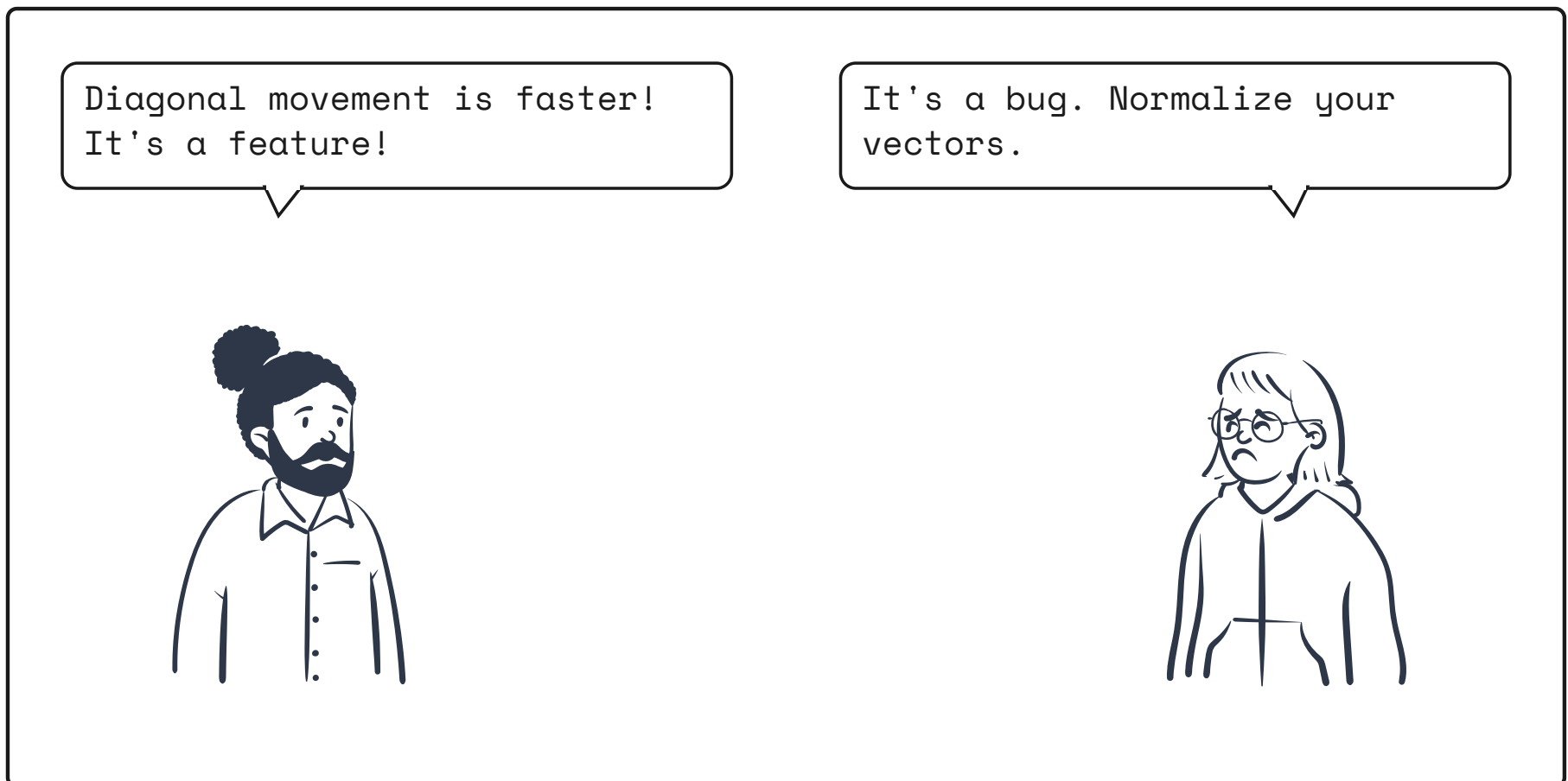
        animated.facing = Facing::from_direction(direction);

        // Only update animation if not jumping
        if !state.is_jumping {
            state.is_moving = true;
            animated.current_animation = if is_running {
                AnimationType::Run
            } else {
                AnimationType::Walk
            };
        }
    } else if !state.is_jumping {
        state.is_moving = false;
        animated.current_animation = AnimationType::Walk;
    }
}
```

```
}  
}
```

Breaking it down:

1. **Query for the player:** `With<Player>` filters to only entities with the `Player` component. `single_mut()` gets the one player entity.
2. **Read input:** Get the direction vector from arrow keys.
3. **Handle jumping:** If Space was just pressed, set `is_jumping` and switch to the Jump animation.
4. **Check for running:** Are either Shift keys pressed?
5. **Move the character:** If there's input, normalize the direction (so diagonal movement isn't faster), multiply by speed and delta time, and update the transform.



1. **Update facing:** Use our `Facing::from_direction` helper to determine which way to face.
2. **Update animation:** If not jumping, set the animation to Run or Walk based on whether `Shift` is held.

Handling Jump Completion



Jump animations are special, they have a beginning and an end. Unlike Walk or Run, which loop forever, Jump plays once and then we need to return to the idle state.

Add this system:

```
// Append this to src/characters/movement.rs
/// Monitor jump animation completion and reset state
pub fn update_jump_state(
    mut query: Query<
        &mut AnimationController,
        &mut AnimationState,
        &AnimationTimer,
        &Sprite,
        &CharacterEntry,
    >, With<Player>>,
) {
    for (mut animated, mut state, timer, sprite, config) in query.iter_mut() {
        if !state.is_jumping {
            continue;
        }

        let Some(atlas) = sprite.texture_atlas.as_ref() else {
            continue;
        };

        let Some(clip) = animated.get_clip(config) else {
            continue;
        };

        // Check if jump animation has completed
        if clip.is_complete(atlas.index, timer.just_finished()) {
            state.is_jumping = false;
            animated.current_animation = AnimationType::Walk;
        }
    }
}
```

This system checks if the jump animation has reached its last frame and the timer has finished (using the `is_complete` method we defined earlier in `AnimationClip`). If so, it resets `is_jumping` to false and switches back to the Walk animation. The player can then move normally again.

What's `.as_ref()`?

In the animation system, we used `.as_mut()` to get a mutable reference to the texture atlas so we could change the frame index. Here, we only need to read the current frame index, not modify it. The `.as_ref()` method converts `Option<TextureAtlas>` into `Option<&TextureAtlas>`, giving us a read-only reference.

Spawning Characters

We have animation, movement, and data structures, but no actual character on screen yet! The spawn system is responsible for:

1. Loading the `characters.ron` file
2. Creating the player entity with all necessary components
3. Setting up the texture atlas from the spritesheet
4. Allowing runtime character switching with number keys (1-6)

Create a new file `src/characters/spawn.rs`:

```
characters/  
├─ config.rs  
├─ animation.rs  
├─ movement.rs  
└─ spawn.rs  <- Create this
```

Resources for Character Management

We'll need two resources: one to track which character is currently active, and another to hold a reference to our loaded character data file.

Add this to `src/characters/spawn.rs`:

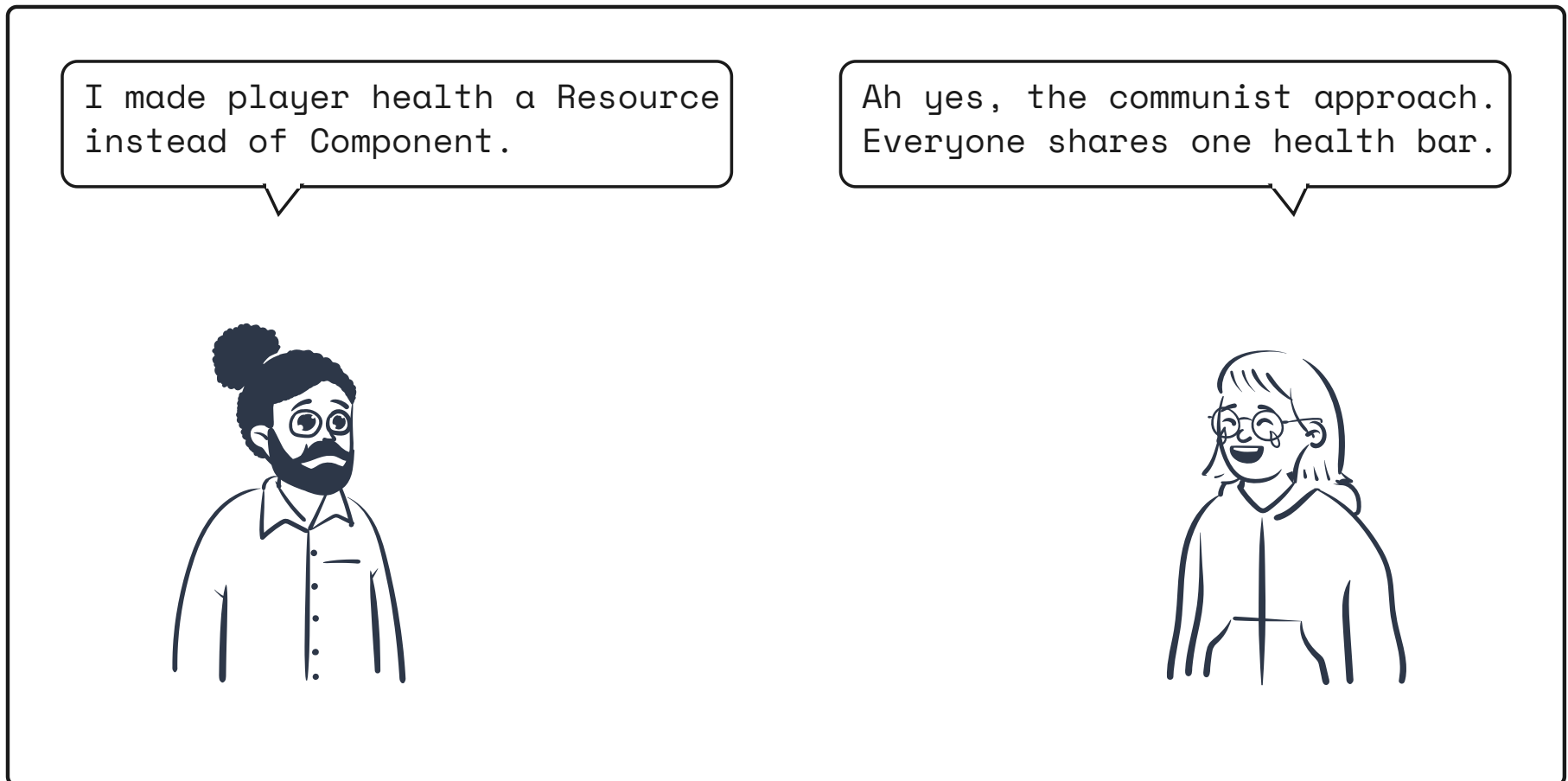
```
// src/characters/spawn.rs  
use bevy::prelude::*;  
use crate::characters::animation::*;  
use crate::characters::config::{CharacterEntry, CharactersList};  
use crate::characters::movement::Player;  
  
const PLAYER_SCALE: f32 = 0.8;  
const PLAYER_Z_POSITION: f32 = 20.0;  
  
#[derive(Resource, Default)]  
pub struct CurrentCharacterIndex {  
    pub index: usize,  
}  
  
#[derive(Resource)]  
pub struct CharactersListResource {  
    pub handle: Handle<CharactersList>,  
}
```

What are these resources for?

- `CurrentCharacterIndex`: Tracks which character is currently active (0 = first character, 1 = second, etc.)
- `CharactersListResource`: Stores the handle to our loaded `characters.ron` file. A handle is like a reference to an asset that Bevy is loading or has loaded.

How to decide between using resources and components?

Use **Components** for data that belongs to specific entities (like a player's health, position, or animation state). Use **Resources** for global data that isn't tied to any particular entity (like the current level number, game settings, or in our case, which character is active). Think of it this way: if you'd ask "which entity does this belong to?" and the answer is "all of them" or "none of them," it's probably a Resource.



What's the `Default` macro?

The `Default` derive macro automatically implements the `Default` trait, which provides a default value for the struct. For `usize`, Rust's default is `0`. When you later in the chapter use `init_resource::<CurrentCharacterIndex>()`, Bevy internally calls `CurrentCharacterIndex::default()`, which creates `CurrentCharacterIndex { index: 0 }`.

This is equivalent to manually writing:

```
// Pseudo code warning, don't use
impl Default for CurrentCharacterIndex {
    fn default() -> Self {
        Self { index: 0 }
    }
}
```

But the derive macro does it for us! Different types have different defaults: `bool` → `false`, `String` → `""`, `Option<T>` → `None`, etc.

Creating the Texture Atlas Layout

Remember how we talked about spritesheets being grids of frames? Bevy doesn't automatically know where one frame ends and another begins. We need to give it instructions: "Each frame is 64×64 pixels, there are 12 columns, and 8 rows." That's what a texture atlas layout does, it's like a map that tells Bevy how to navigate the spritesheet.

This helper function creates that map:

```
// Append this to src/characters/spawn.rs
/// Create a texture atlas layout for a character
fn create_character_atlas_layout(
    atlas_layouts: &mut ResMut<Assets<TextureAtlasLayout>>,
    character_entry: &CharacterEntry,
```



```

) -> Handle<TextureAtlasLayout> {
    let max_row = character_entry.calculate_max_animation_row();

    atlas_layouts.add(TextureAtlasLayout::from_grid(
        UVec2::splat(character_entry.tile_size),
        character_entry.atlas_columns as u32,
        (max_row + 1) as u32,
        None,
        None,
    ))
}

```

Breaking it down:

- `calculate_max_animation_row()`: We defined this earlier in `CharacterEntry`. It figures out how many rows the spritesheet needs based on all the animations.
- `UVec2::splat(tile_size)`: Creates a 2D vector where both x and y are the tile size (e.g., 64×64 pixels per frame).
- `from_grid(...)`: Tells Bevy “this texture is a grid with X columns and Y rows, each cell is this size.”

Spawning the Player Entity

Now we spawn the player. But here’s the catch: loading files from disk takes time. We can’t wait for `characters.ron` to load before creating the player entity, that would freeze the game during startup.

So we use a two stage approach: create a “placeholder” entity immediately, then fill in the details once the data finishes loading.

Stage 1: Create the entity

```

// Append this to src/characters/spawn.rs
pub fn spawn_player(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut character_index: ResMut<CurrentCharacterIndex>,
) {
    // Load the characters list
    let characters_list_handle: Handle<CharactersList> =
asset_server.load("characters/characters.ron");

    // Store the handle in a resource
    commands.insert_resource(CharactersListResource {
        handle: characters_list_handle,
    });

    // Initialize with first character
    character_index.index = 0;

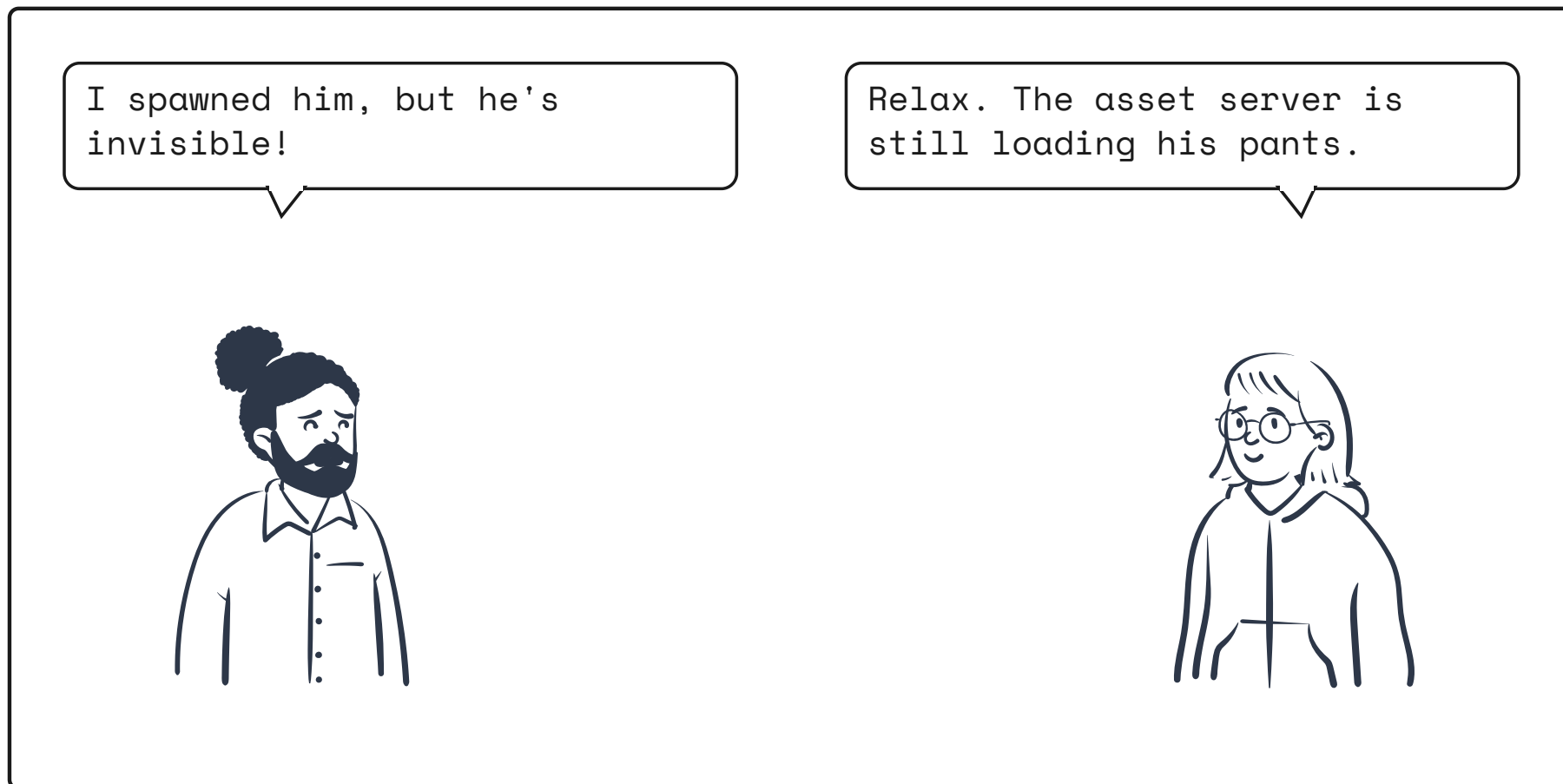
    // Spawn player entity (will be initialized once asset loads)
    commands.spawn((
        Player,
        Transform::from_translation(Vec3::new(0.0, 0.0, PLAYER_Z_POSITION))
            .with_scale(Vec3::splat(PLAYER_SCALE)),
        Sprite::default(),
    ));
}

```

This system runs once at startup. It loads the `characters.ron` file, stores the handle in a resource, and spawns a player entity with just the `Player` marker, a `Transform`, and an empty `Sprite`. We can’t fully initialize it yet because the asset is still loading.

Why not load everything immediately?

Asset loading in Bevy is asynchronous. When you call `asset_server.load()`, Bevy starts loading the file in the background. It might take a few frames (or longer for large files). We need to wait until it's ready.



Stage 2: Initialize once loaded

```
// Append this to src/characters/spawn.rs
pub fn initialize_player_character(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
    characters_lists: Res<Assets<CharactersList>>,
    character_index: Res<CurrentCharacterIndex>,
    characters_list_res: Option<Res<CharactersListResource>>,
    mut query: Query<Entity, (With<Player>, Without<AnimationController>)>,
) {
    let Some(characters_list_res) = characters_list_res else {
        return;
    };

    for entity in query.iter_mut() {
        let Some(characters_list) = characters_lists.get(&characters_list_res.handle) else {
            continue;
        };

        if character_index.index >= characters_list.characters.len() {
            continue;
        };

        let character_entry = &characters_list.characters[character_index.index];

        let texture = asset_server.load(&character_entry.texture_path);
        let layout = create_character_atlas_layout(&mut atlas_layouts, character_entry);

        let sprite = Sprite::from_atlas_image(
            texture,
            TextureAtlas {
                layout,
                index: 0,
            },
        );
        commands.spawn(sprite);
    }
}
```

```

        },
    );

    commands.entity(entity).insert((
        AnimationController::default(),
        AnimationState::default(),
        AnimationTimer(Timer::from_seconds(DEFAULT_ANIMATION_FRAME_TIME, TimerMode::Repeating)),
        character_entry.clone(),
        sprite,
    ));
}
}

```

Earlier in Stage 1, we started loading `characters.ron` in the background. But we don't know *when* it will finish—could be the next frame, could be 10 frames later.

This system runs every frame, checking: “Is the file loaded yet? Is there a player entity that still needs initialization?”

The query `(With<Player>, Without<AnimationController>)` finds player entities that exist but haven't been fully set up yet. Once the file loads, `characters_lists.get()` succeeds, and we can finally add all the animation components.

We grab the character data, load its texture, create the atlas layout, and insert all the necessary components.

Character Switching

One of the coolest features of our data-driven system: switching characters at runtime by pressing number keys!

```

// Append this to src/characters/spawn.rs
pub fn switch_character(
    input: Res<ButtonInput<KeyCode>>,
    mut character_index: ResMut<CurrentCharacterIndex>,
    characters_lists: Res<Assets<CharactersList>>,
    characters_list_res: Option<Res<CharactersListResource>>,
    mut query: Query<(
        &mut CharacterEntry,
        &mut Sprite,
    ), With<Player>>,
    mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
    asset_server: Res<AssetServer>,
) {
    // Map digit keys to indices
    const DIGIT_KEYS: [KeyCode; 9] = [
        KeyCode::Digit1, KeyCode::Digit2, KeyCode::Digit3,
        KeyCode::Digit4, KeyCode::Digit5, KeyCode::Digit6,
        KeyCode::Digit7, KeyCode::Digit8, KeyCode::Digit9,
    ];

    // Find which digit key was pressed
    let new_index = DIGIT_KEYS.iter()
        .position(|&key| input.just_pressed(key));

    let Some(new_index) = new_index else {
        return;
    };

    let Some(characters_list_res) = characters_list_res else {
        return;
    };

    let Some(characters_list) = characters_lists.get(&characters_list_res.handle) else {

```

```

        return;
    };

    if new_index >= characters_list.characters.len() {
        return;
    }

    // Update character index
    character_index.index = new_index;

    // Update player entity
    let Ok((mut current_entry, mut sprite)) = query.single_mut() else {
        return;
    };

    let character_entry = &characters_list.characters[new_index];

    // Update character entry
    *current_entry = character_entry.clone();

    // Update sprite with new texture
    let texture = asset_server.load(&character_entry.texture_path);
    let layout = create_character_atlas_layout(&mut atlas_layouts, character_entry);

    *sprite = Sprite::from_atlas_image(
        texture,
        TextureAtlas {
            layout,
            index: 0,
        },
    );
}

```

How it works:

1. **Detect key press:** Check if any digit key (1-9) was just pressed.
2. **Validate index:** Make sure the index is within bounds (we have 6 characters, so keys 1-6 work).
3. **Update resources:** Set `character_index.index` to the new value.
4. **Swap the character:** Replace the `CharacterEntry` component with the new character's data.
5. **Update the sprite:** Load the new character's texture and create a new atlas layout.

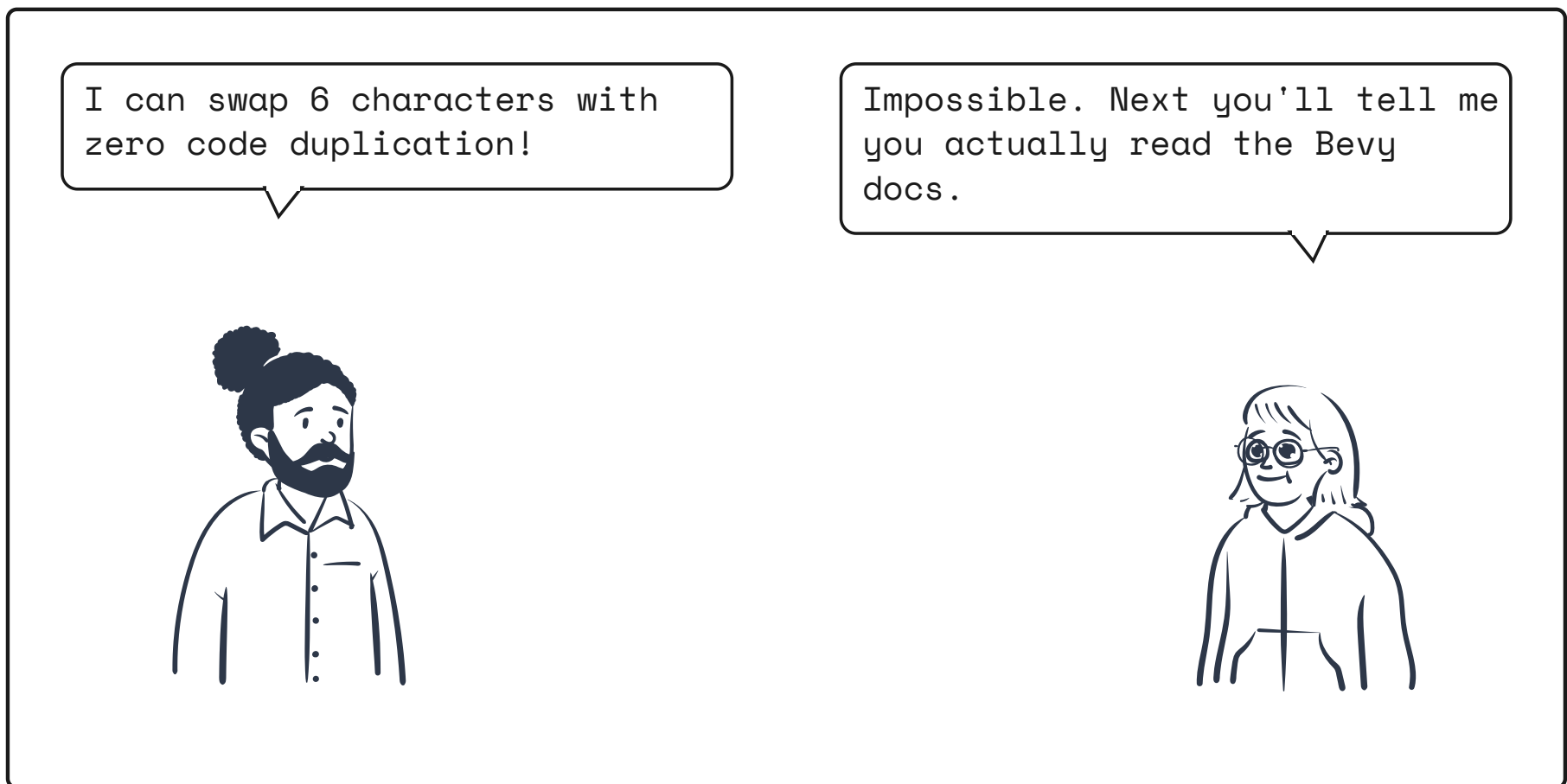
This is the payoff of our data-driven design! Remember how in Chapter 1, adding a second character would have meant duplicating all the animation and movement code? Here, we just swap out the data. The animation system doesn't care if it's animating a Male character or a Crimson Count, it just reads the `CharacterEntry` component and does its job. Same with movement: it reads `base_move_speed` and `run_speed_multiplier` from the new character's data. No code changes needed.

How does `.position()` work?

This iterator method searches through the array and returns the index of the first item where the condition is true. The closure `|&key| input.just_pressed(key)` checks each key: "was this key just pressed?" If the player presses `Digit3`, `.position()` returns `Some(2)` (because `Digit3` is at index 2 in the array). If no digit key is pressed, it returns `None`. It's similar to the filter operation.

What's the `*` in `*current_entry`?

The `*` is the dereference operator. `current_entry` is a mutable reference (`&mut CharacterEntry`), not the actual data. To modify the data it points to, we need to dereference it with `*`. Think of it like this: `current_entry` is a pointer to a box, `*current_entry` is the contents of the box. We're replacing the contents, not the pointer.



Bringing It All Together

We've built all the pieces: animation, movement, spawning, and character switching. Now we need to package them into a plugin and integrate it into our game.

Adding the RON Asset Loader

First, we need to add dependencies that let Bevy load `.ron` files and serialize/deserialize our data structures. Open `Cargo.toml` and add these to your `[dependencies]` section:

```
bevy_common_assets = { version = "0.17", features = ["ron"] }
serde = { version = "1.0", features = ["derive"] }
```

The `bevy_common_assets` crate provides asset loaders for common file formats. We're using the `ron` feature to load our `characters.ron` file. The `serde` crate with the `derive` feature allows us to use `#[derive(Serialize, Deserialize)]` on our structs, which we used in `config.rs` and `animation.rs`.

Creating the Characters Plugin

Create `src/characters/mod.rs`:

```
// src/characters/mod.rs
pub mod animation;
pub mod config;
pub mod movement;
pub mod spawn;

use bevy::prelude::*;
use bevy_common_assets::ron::RonAssetPlugin;
use config::CharactersList;

pub struct CharactersPlugin;

impl Plugin for CharactersPlugin {
    fn build(&self, app: &mut App) {
        app.add_plugins(RonAssetPlugin::<CharactersList>::new(&["characters.ron"]))
            .init_resource::<spawn::CurrentCharacterIndex>()
    }
}
```

```

        .add_systems(Startup, spawn::spawn_player)
        .add_systems(Update, (
            spawn::initialize_player_character,
            spawn::switch_character,
            movement::move_player,
            movement::update_jump_state,
            animation::animate_characters,
            animation::update_animation_flags,
        ));
    }
}

```

Breaking it down:

- **Module declarations:** `pub mod animation;` etc. make our submodules accessible.
- **RonAssetPlugin:** Registers the `.ron` file loader for our `CharactersList` type.
- **init_resource:** Creates the `CurrentCharacterIndex` resource with its default value (0).
- **Startup systems:** `spawn_player` runs once at game start.
- **Update systems:** All other systems run every frame.

Why are we keeping `initialize_player_character` in the Update instead of Startup? Does it initialize player character every frame?

Good catch! The system *does* run every frame, but it doesn't initialize the player every frame. Look at the query: `Query<Entity, (With<Player>, Without<AnimationController>)>`. This only matches player entities that *don't have* an `AnimationController` yet.

Once we add the `AnimationController` component (which happens inside the system), the entity no longer matches the query, so the system does nothing on subsequent frames. It's a self-terminating system—it runs until it finds and initializes uninitialized players, then effectively becomes a no-op.

We can't put it in Startup because the `characters.ron` file might not be loaded yet when Startup runs. By putting it in Update, it keeps checking every frame: "Is the file loaded? Is there an uninitialized player?" Once both conditions are true, it initializes the player and then stops doing anything.

Plugging Into the Game

Now we connect our plugin to the main game. Open `src/main.rs` and add the module declaration at the top:

```

// Update line in src/main.rs
mod map;
mod characters; // Add this line

```

Then add the plugin to your app:

```

// Update line in src/main.rs
fn main() {
    let map_size = map_pixel_dimensions();

    App::new()
        .insert_resource(ClearColor(Color::WHITE))
        .add_plugins(
            DefaultPlugins
                .set(AssetPlugin {
                    file_path: "src/assets".into(),
                    ..default()
                })
                .set(WindowPlugin {
                    primary_window: Some(Window {

```



```

        resolution: WindowResolution::new(map_size.x as u32, map_size.y as u32),
        resizable: false,
        ..default()
    }},
    ..default()
})
.set(ImagePlugin::default_nearest()),
)
.add_plugins(ProcGenSimplePlugin::<Cartesian3D, Sprite>::default())
.add_plugins(characters::CharactersPlugin) // Add this line
.add_systems(Startup, (setup_camera, setup_generator))
.run();
}

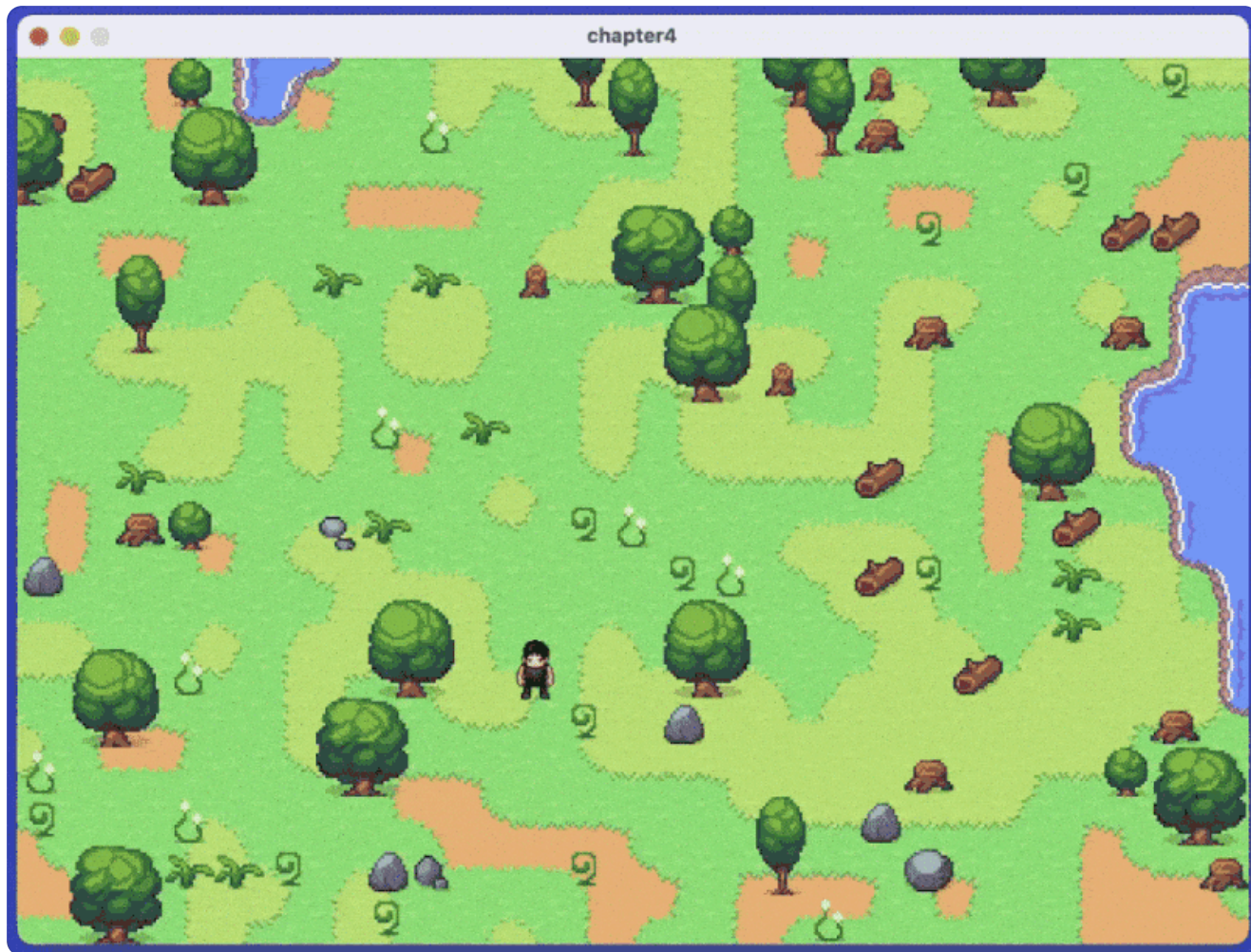
```

That's it! Run your game with `cargo run`, and you should see your character on screen. Press the arrow keys to move, hold Shift to run, press Space to jump, and press number keys 1-6 to switch characters!



The Impatient Programmer's Guide to Bevy and Rust: Chapter 4 - Let There Be Collisions

By the end of this chapter, your player will interact with the world properly, no more walking through trees, water, or rocks. We'll implement z-ordering so they can walk behind objects, giving your 2D game true depth. Also, you'll build a collision visualizer for debugging.



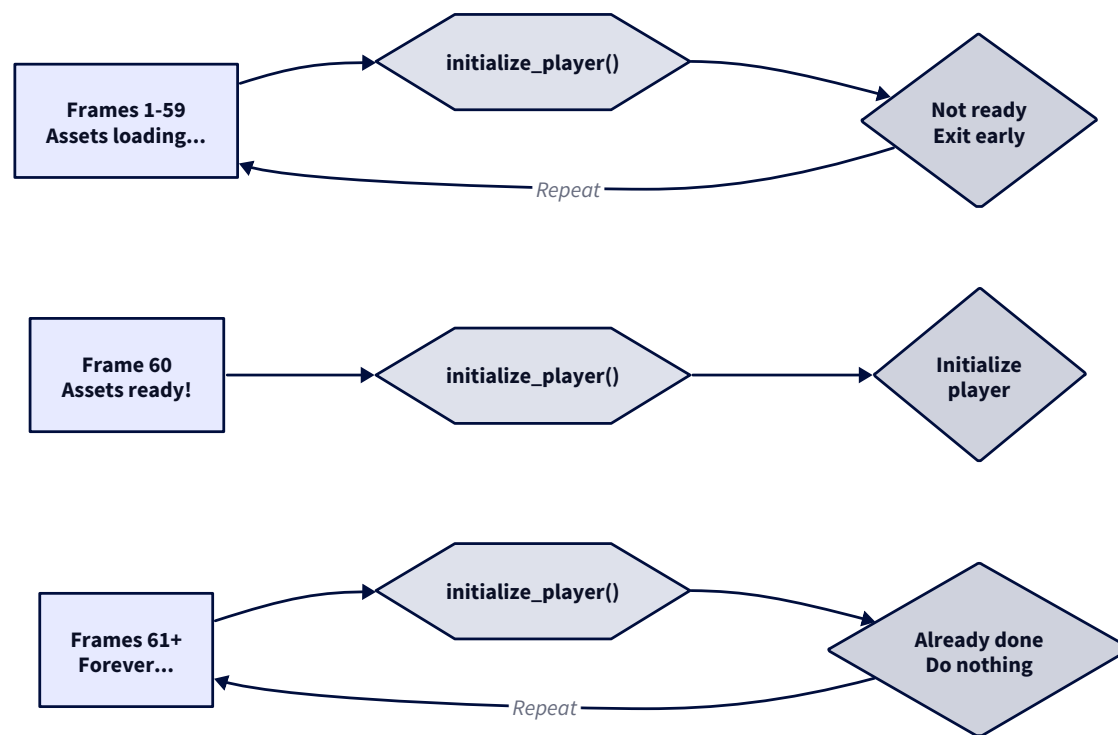
Systems Running When They Shouldn't

In Chapter 3, we built a character system that loads sprite data from a `.ron` file. But there's a problem in how we handle the initialization.

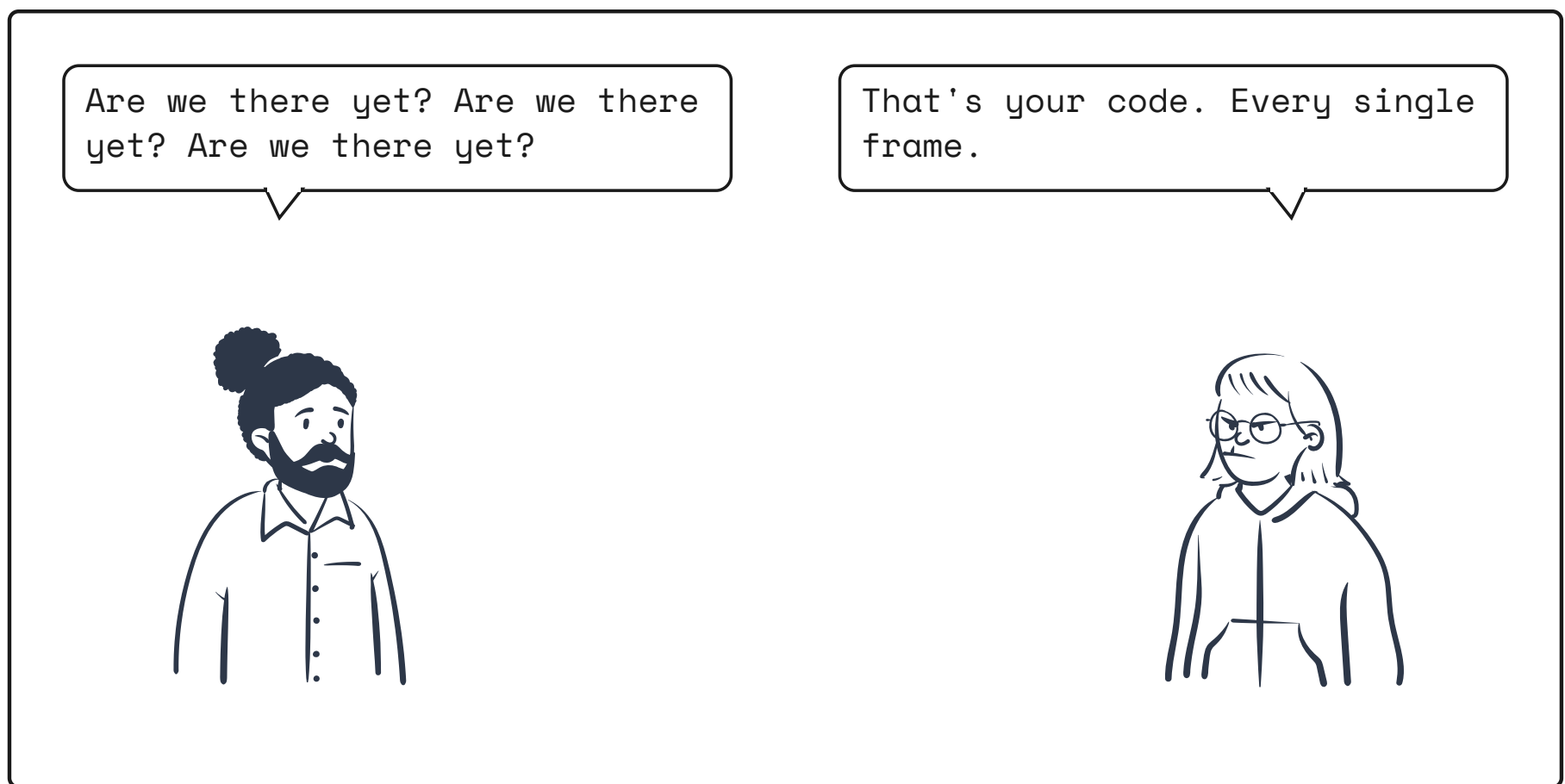
The `spawn_player` system runs once at startup. It spawns the player entity and begins loading the character data file. So far, so good.

But then `initialize_player_character` runs **every single frame**, checking if the assets have finished loading:

We have a **polling pattern** that repeatedly checks if something is ready:



The system runs **every frame forever** with wasted checks while loading, one useful execution, then infinite no-ops causing performance waste, code clutter, and making the code difficult to extend.



Systems That Run Only When Needed

`Startup` and `Update` we use to trigger our systems are **schedules**. Bevy's way of organizing when systems run. `Startup` runs once at launch, `Update` runs every frame.

But what if we need systems that run at *specific moments*? Not every frame, not just at startup, but exactly when something happens like when assets finish loading, or when the player pauses the game.

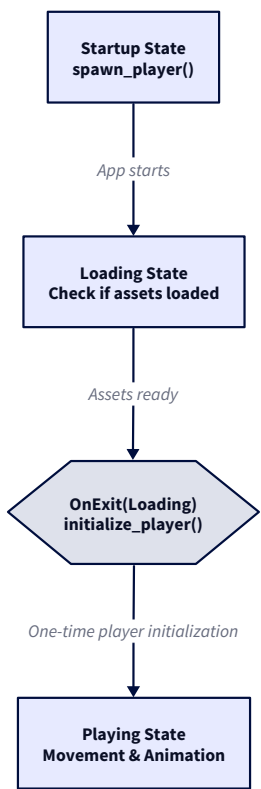
State-Based Schedules

The solution is to organize our game into distinct **phases**. We call these phases **states**: `Loading`, `Playing`, and `Paused`. Each state represents a different mode of the game.

When the game transitions from one state to another (say, from `Loading` to `Playing`), Bevy provides special schedules that run exactly once:

- **OnEnter** - Runs when entering a state
- **OnExit** - Runs when leaving a state

This is how we eliminate polling. Instead of `initialize_player_character` checking every frame “Are assets loaded yet?”, we attach it to `OnExit>Loading`. When assets finish loading and we leave the `Loading` state, Bevy runs it exactly once.



Implementing Game States

Let’s build a state management module with loading screens and game pause functionality. Create the folder `state` inside the `src` folder.

Defining Game States

What states do we need?

Think about your game’s lifecycle. Right now, when the game starts:

1. Assets need time to load from disk
2. Once loaded, gameplay begins
3. Players might want to pause

These are three distinct phases, each needing different systems:

- **Loading**: Show loading screen, check if assets ready, don’t run gameplay
- **Playing**: Run movement/animation, hide loading screen, allow pausing
- **Paused**: Show pause menu, stop gameplay, allow un-pausing

Let’s define these as an enum:

Create `src/state/game_state.rs`:

```
// src/state/game_state.rs
use bevy::prelude::*;

#[derive(States, Default, Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum GameState {
```

```
#[default]
Loading,
Playing,
Paused,
}
```

What's the `States` macro?

The `#[derive(States)]` macro implements the `States` trait, which tells Bevy:

- This enum represents game phases where only one can be active at a time
- Bevy should track which state is active
- Systems can be gated to run only in specific states
- State transitions should trigger `OnEnter/OnExit` schedules

The `#[default]` attribute marks which state the game starts in. Here, Bevy initializes the state to the default value (`Loading` in our case).

Loading Screen

Let's create a loading screen with a full-screen dark background, animated "Loading..." text.

Create `src/state/loading.rs`

```
// src/state/loading.rs
use bevy::prelude::*;

#[derive(Component)]
pub struct LoadingScreen;

#[derive(Component)]
pub struct LoadingText;

pub fn spawn_loading_screen(mut commands: Commands) {
    commands.spawn((
        LoadingScreen,
        Node {
            width: Val::Percent(100.0),
            height: Val::Percent(100.0),
            justify_content: JustifyContent::Center,
            align_items: AlignItems::Center,
            ..default()
        },
        BackgroundColor(Color::srgb(0.1, 0.1, 0.15)),
    )).with_children(|parent| {
        parent.spawn((
            LoadingText,
            Text::new("Loading..."),
            TextFont {
                font_size: FontSize::Px(48.0),
                ..default()
            },
            TextColor(Color::WHITE),
        ));
    });

    info!("Loading screen spawned");
}
```

What's happening:

- `LoadingScreen` and `LoadingText` are marker components to identify our UI entities
- `Node` creates a full-screen container (100% width and height)

- `justify_content` and `align_items` centered means text appears in the middle
- `.with_children()` spawns the text as a child of the background

Now append the animation function to the same file:

```
// Append to src/state/loading.rs
pub fn animate_loading(
    time: Res<Time>,
    mut query: Query<&mut Text, With<LoadingText>>,
) {
    for mut text in query.iter_mut() {
        let dots = (time.elapsed_secs() * 2.0) as usize % 4;
        **text = format!("Loading{}", ".".repeat(dots));
    }
}
```

This animates the text: “Loading” → “Loading.” → “Loading..” → “Loading...” (cycling through 0-3 dots).

What’s `**text`, is this like double dereferencing?

Yes! When we call `.iter_mut()`, Bevy wraps our `&mut Text` in a special type that tracks changes. The first `*` unwraps that to get `&mut Text`, and the second `*` dereferences the reference to reach the actual `Text` value we can modify.

Finally, append the despawn function:

```
// Append to src/state/loading.rs
pub fn despawn_loading_screen(
    mut commands: Commands,
    query: Query<Entity, With<LoadingScreen>>,
) {
    for entity in query.iter() {
        commands.entity(entity).despawn();
    }

    info!("Loading screen despawned");
}
```

Why despawn?

When we transition to Playing state, the loading screen should disappear. Without despawning, the UI entities would remain in the world forever, cluttering memory and rendering on top of the game. The `.despawn()` function removes the loading screen entity. Since `LoadingText` is a child (via `.with_children()`), Bevy automatically removes it too.

Pause Menu

Now let’s create a pause menu that shows a semi-transparent overlay with “PAUSED” text when the player presses Escape, and hides when they press it again.

Create `src/state/pause.rs`:

```
// src/state/pause.rs
use bevy::prelude::*;

#[derive(Component)]
pub struct PauseMenu;

pub fn spawn_pause_menu(mut commands: Commands) {
    commands.spawn((
        PauseMenu,
```



```

Node {
    width: Val::Percent(100.0),
    height: Val::Percent(100.0),
    justify_content: JustifyContent::Center,
    align_items: AlignItems::Center,
    ..default()
},
BackgroundColor(Color::srgba(0.0, 0.0, 0.0, 0.7)),
)).with_children(|parent| {
    parent.spawn((
        Text::new("PAUSED\n\nPress ESC to resume"),
        TextFont {
            font_size: FontSize::Px(36.0),
            ..default()
        },
        TextColor(Color::WHITE),
        TextLayout::justify(Justify::Center),
    ));
});

info!("Pause menu spawned");
}

pub fn despawn_pause_menu(
    mut commands: Commands,
    query: Query<Entity, With<PauseMenu>>,
) {
    for entity in query.iter() {
        commands.entity(entity).despawn();
    }

    info!("Pause menu despawned");
}

```

State Plugin

Now we'll create the `StatePlugin` that wires everything together. We'll build it piece by piece.

Create `src/state/mod.rs`.

```

// src/state/mod.rs
mod game_state;
mod loading;
mod pause;

use bevy::prelude::*;
use crate::characters::spawn::CharactersListResource;
use crate::characters::config::CharactersList;

pub use game_state::GameState;

pub struct StatePlugin;

impl Plugin for StatePlugin {
    fn build(&self, app: &mut App) {
        app
            .init_state::<GameState>()

            // Loading state systems
            .add_systems(OnEnter(GameState::Loading), loading::spawn_loading_screen)
            .add_systems(Update, (
                check_assets_loaded,
                loading::animate_loading,
            ))
    }
}

```

```

        ).run_if(in_state(GameState::Loading)))
        .add_systems(OnExit(GameState::Loading), (
            loading::despawn_loading_screen,
            crate::characters::spawn::initialize_player_character,
        ))
        // Pause state systems
        .add_systems(OnEnter(GameState::Paused), pause::spawn_pause_menu)
        .add_systems(OnExit(GameState::Paused), pause::despawn_pause_menu)

        // Pause toggle (works in Playing or Paused states)
        .add_systems(Update,
            toggle_pause.run_if(in_state(GameState::Playing).or(in_state(GameState::Paused))))
    );
}

fn check_assets_loaded(
    characters_list_res: Option<Res<CharactersListResource>>,
    characters_lists: Res<Assets<CharactersList>>,
    mut next_state: ResMut<NextState<GameState>>,
) {
    let Some(res) = characters_list_res else {
        return;
    };

    if characters_lists.get(&res.handle).is_some() {
        info!("Assets loaded, transitioning to Playing!");
        next_state.set(GameState::Playing);
    }
}

fn toggle_pause(
    input: Res<ButtonInput<KeyCode>>,
    current_state: Res<State<GameState>>,
    mut next_state: ResMut<NextState<GameState>>,
) {
    if input.just_pressed(KeyCode::Escape) {
        match current_state.get() {
            GameState::Playing => {
                info!("Game paused");
                next_state.set(GameState::Paused);
            }
            GameState::Paused => {
                info!("Game resumed");
                next_state.set(GameState::Playing);
            }
            _ => {}
        }
    }
}

```

What's happening here?

We start by telling Bevy to track our custom `GameState` enum.

When the game starts and enters `Loading` state, `OnEnter(GameState::Loading)` runs `spawn_loading_screen` once, showing the loading UI.

While in Loading state, `Update.run_if(in_state(GameState::Loading))` runs two systems one to check if assets are loaded and another to animate the loading text.

Once assets load, `check_assets_loaded` requests a transition to `Playing` state. When this happens, `OnExit(GameState::Loading)` triggers, running two systems that cleans up the loading UI and initialize the player. Now player initialization can happen only once since exiting from loading state is a one-time event.

For pausing, we added systems on `OnEnter(GameState::Paused)` and `OnExit(GameState::Paused)` to show and hide the pause menu. The `toggle_pause` function listens for the Escape key and switches between `Playing` and `Paused` states.

Our `StatePlugin` now orchestrates the entire game flow. The `Loading` state handles asset loading with visual feedback, and the `Playing` state will run gameplay systems. Later in this chapter, we'll gate our gameplay systems to only run in the `Playing` state, which will freeze the game when paused. The beauty of this design is that systems automatically attach to state transitions, no polling, no wasted frames. Everything runs exactly when needed.

Now open `src/characters/mod.rs` and remove `initialize_player_character` from the Update schedule. Since we have already added it through `StatePlugin`.

```
// src/characters/mod.rs
impl Plugin for CharactersPlugin {
    fn build(&self, app: &mut App) {
        app.add_plugins(RonAssetPlugin::<CharactersList>::new(&["characters.ron"]))
            .init_resource::<spawn::CurrentCharacterIndex>()
            .add_systems(Startup, spawn::spawn_player)
            // REMOVE initialize_player_character from here!
            // It now runs in StatePlugin's OnExit>Loading)
            .add_systems(Update, (
                spawn::switch_character,
                movement::move_player,
                movement::update_jump_state,
                animation::animate_characters,
                animation::update_animation_flags,
            ));
    }
}
```

Integrating StatePlugin

Add the state module and plugin to `src/main.rs`:

```
// src/main.rs
mod characters;
mod map;
mod state; // Add this
```

Important: Add `StatePlugin` BEFORE `CharactersPlugin` so the state system is initialized before character systems try to use it.

```
// Add state plugin inside main function of src/main.rs
// Previous code as it is
.add_plugins(ProcGenSimplePlugin::<Cartesian3D, Sprite>::default())
.add_plugins(state::StatePlugin) // Add BEFORE CharactersPlugin!
.add_plugins(characters::CharactersPlugin)
.add_systems(Startup, setup_camera)
.run();
```

Run your game:

```
cargo run
```

You might not see the loading screen (assets load quickly, but you can manually add a delay if needed). The game starts and your character appears, ready to move. Press Escape to toggle the pause overlay. Note that at this point, the game will continue running in the background when

paused, we'll fix that later in the chapter by gating our gameplay systems to only run in the `Playing` state.

The State Pattern for Characters

We just used states to control our *game flow* (`Loading` → `Playing` → `Paused`). Now let's apply the same pattern to something else: *character behavior*.

Have a look at our `AnimationState` component:

```
// Pseudo code, don't use
#[derive(Component, Default)]
pub struct AnimationState {
    pub is_moving: bool,
    pub was_moving: bool,
    pub is_jumping: bool,
    pub was_jumping: bool,
}
```

Four booleans tracking two pieces of information: what the character is doing *now* and what they were doing *last frame*. We needed `was_moving` and `was_jumping` to detect transitions like “just started jumping” or “just stopped moving”.

This works to help us with animation, but it has problems.

Too Many Boolean Flags

What if we add running? We'd need:

```
// Pseudo code, don't use
pub is_running: bool,
pub was_running: bool,
```

Attacking?

```
// Pseudo code, don't use
pub is_attacking: bool,
pub was_attacking: bool,
```

Soon our component is drowning in booleans, and our animation system is drowning in transition logic:

```
// Pseudo code, don't use
let just_started_moving = state.is_moving && !state.was_moving;
let just_stopped_moving = !state.is_moving && state.was_moving;
let just_started_jumping = state.is_jumping && !state.was_jumping;
let just_stopped_jumping = !state.is_jumping && state.was_jumping;
let just_started_running = state.is_running && !state.was_running;
// ... it keeps growing
```

Worse, what happens if `is_moving` and `is_jumping` are both true? Or `is_running` and `is_attacking`? Boolean flags don't prevent impossible states.

A developer might accidentally set both flags, or forget to clear one when setting another. Your animation system then has to decide: which flag wins? You end up writing priority logic, and bugs creep in when the priorities aren't consistent across systems.

The State Pattern Solution

Remember how `GameState` worked? We defined an enum with `Loading`, `Playing`, and `Paused`, and Bevy tracked which state we were in. We can apply the same idea to characters: define an enum of possible states, and let the current state determine behavior.

```
// Pseudo code, don't use
#[derive(Component, Debug, Clone, Copy, PartialEq, Eq, Default)]
pub enum CharacterState {
    #[default]
    Idle,
    Walking,
    Running,
    Jumping,
}
```

A character can only be in *one* state at a time. No more impossible combinations. No more boolean math.

Why This Is Better

1. Impossible states become impossible:

With an enum, the compiler enforces that the character is in exactly one state:

```
// Pseudo code, don't use
// With booleans: you can do this (but shouldn't!)
is_walking = true;
is_jumping = true; // Now both are true - invalid!

// With enum: you can't have both true
let state = CharacterState::Walking;
// state is Walking. To jump, you must replace it:
let state = CharacterState::Jumping; // Now it's only Jumping
```

The variable holds one value. You can't be Walking and Jumping simultaneously. This approach is called *making illegal states unrepresentable*, a key principle in type-driven development. Instead of writing code to check for invalid combinations, you design your types so invalid combinations can't exist.

2. Bevy detects changes for us:

Remember manually tracking `was_moving` and `was_jumping`? That was change detection done by hand. Bevy has this built in. When you use `Changed<CharacterState>`, Bevy only runs your code when the state actually changes.

Your animation update system only runs when the character transitions between states. Your sound effect system only runs when entering a new state. Less code, fewer bugs, and we'll use this later in the chapter.

3. Animation selection becomes a simple match:

With an enum, picking the right animation is straightforward. You match on the current state, and each state maps to exactly one animation. There's no ambiguity, no priority logic, no "what if both flags are true?" dilemma.

```
// Pseudo code, don't use
let new_animation = match state {
    CharacterState::Idle | CharacterState::Walking => AnimationType::Walk,
    CharacterState::Running => AnimationType::Run,
    CharacterState::Jumping => AnimationType::Jump,
};
```

The compiler warns you if you forget to handle a state. If you later add a new state to the enum, every match statement becomes a compile error until you handle the new case. The compiler forces you to think through all possibilities.

Now let's put this into practice and implement `CharacterState`.

Implementing Character States

Create a new file `src/characters/state.rs`:

```
characters/  
├─ config.rs  
├─ animation.rs  
├─ movement.rs  
├─ mod.rs  
├─ spawn.rs  
└─ state.rs ← Create this
```

The CharacterState Enum

We need an enum that represents all possible states our character can be in:

```
// src/characters/state.rs  
use bevy::prelude::*;  
  
/// Character states. Only one can be active at a time.  
#[derive(Component, Debug, Clone, Copy, PartialEq, Eq, Default)]  
pub enum CharacterState {  
    #[default]  
    Idle,  
    Walking,  
    Running,  
    Jumping,  
}
```

Querying State with Methods

Earlier, we explained how boolean flags can create invalid combinations. Instead of tracking `is_jumping` and `was_jumping` flags separately, we now have a single `CharacterState`. But we still need to answer questions like “can this character jump right now?”

That's where these query methods come in. They let us ask questions about the current state without maintaining separate flag variables:

```
// Append to src/characters/state.rs  
impl CharacterState {  
    /// Check if this is a grounded state (can jump from here)  
    pub fn is_grounded(&self) -> bool {  
        matches!(self, CharacterState::Idle | CharacterState::Walking | CharacterState::Running)  
    }  
}
```

This method replaces boolean flag logic for jump control. Instead of tracking an `is_jumping` flag and checking it manually, we query the state. The logic is simple: you can only jump when grounded (Idle, Walking, or Running). You can't jump while already in the Jumping state.

What's `matches!`?

The `matches!` macro checks if a value matches a pattern. `matches!(self, CharacterState::Idle)` returns `true` if `self` is `Idle`, `false` otherwise. The `|` means “or” so `matches!(self, CharacterState::Walking | CharacterState::Running)` checks if it’s either Walking or Running.

Animation Refactoring

Now we’ll refactor `animation.rs` to use our new state-based approach. While we’re at it, let’s also clean up a code organization issue: currently, `AnimationController` stores both the current animation type and the facing direction. But these are owned by different systems, facing decides direction, animation decides the clip. Separating them makes each system’s responsibility clearer.

First, create a new file `src/characters/facing.rs`. By making `Facing` its own component, the movement system owns direction updates, and the animation system focuses on sprite animation.

```
// src/characters/facing.rs
use bevy::prelude::*;

/// The direction a character is facing.
/// Separate from movement - character can face one way while moving another.
#[derive(Component, Debug, Clone, Copy, PartialEq, Eq, Default)]
pub enum Facing {
    Up,
    Left,
    #[default]
    Down,
    Right,
}

impl Facing {
    pub fn from_velocity(velocity: Vec2) -> Self {
        if velocity.x.abs() > velocity.y.abs() {
            if velocity.x > 0.0 { Facing::Right } else { Facing::Left }
        } else {
            if velocity.y > 0.0 { Facing::Up } else { Facing::Down }
        }
    }

    /// Helper to map direction to row offset (0, 1, 2, 3)
    pub(crate) fn direction_index(self) -> usize {
        match self {
            Facing::Up => 0,
            Facing::Left => 1,
            Facing::Down => 2,
            Facing::Right => 3,
        }
    }
}
```

Expose both `state` and `facing` in `src/characters/mod.rs`:

```
// src/characters/mod.rs
pub mod config;
pub mod animation;
pub mod movement;
pub mod state; // Add this line
pub mod facing; // Add this line
```

Open `src/characters/animation.rs`. We’ll update it section by section.

First, delete the old `Facing` enum and `AnimationState` struct. We’ll be using `CharacterState` instead.

```
// src/characters/animation.rs - Delete the following sections

// DELETE this (Facing moved to facing.rs)
pub enum Facing { ... }
impl Facing { ... }

// DELETE this (AnimationState replaced by CharacterState)
pub struct AnimationState {
    pub is_moving: bool,
    ...
}
```

Remove `facing` from `AnimationController` since it's now a separate component. Also derive `Default` macro for `AnimationController`.

```
// src/characters/animation.rs - Update AnimationController
#[derive(Component, Default)] // Line update alert
pub struct AnimationController {
    pub current_animation: AnimationType,
    // Facing is removed now, line update alert
}
```

You'll also need to delete the old manual `Default` implementation for `AnimationController`. Previously, it looked like this:

```
// DELETE the following old implementation from src/characters/animation.rs
impl Default for AnimationController {
    fn default() -> Self {
        Self {
            current_animation: AnimationType::Walk,
            facing: Facing::Down,
        }
    }
}
```

Delete it entirely. Since `AnimationController` now only has one field (`current_animation`), we need `AnimationType` to have a default value. Add `#[derive(Default)]` to `AnimationType` and mark `Walk` as the default:

```
// src/characters/config.rs - Set default animation type to Walk
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize, Default)] // Line update alert
pub enum AnimationType {
    #[default] // Line update alert
    Walk,
    Run,
    Jump
}
```

Now update the imports at the top of the file. Let's add imports for `CharacterState` and `Facing`.

```
// src/characters/animation.rs - Update imports
use bevy::prelude::*;
use crate::characters::config::{CharacterEntry, AnimationType};
use crate::characters::state::CharacterState; // Line update alert
use crate::characters::facing::Facing; // Line update alert
```

Since we moved `Facing` out of `AnimationController`, `get_clip` can no longer access `self.facing`. We need to pass facing as a parameter. This is actually cleaner: the method now explicitly declares

what data it needs:

```
// src/characters/animation.rs - Update get_clip signature
impl AnimationController {
    /// Get the animation clip for the current animation and facing direction.
    /// `facing` is passed in since it's now a separate component.
    pub fn get_clip(&self, config: &CharacterEntry, facing: Facing) -> Option<AnimationClip> {
        let def = config.animations.get(&self.current_animation)?;

        let row = if def.directional {
            def.start_row + facing.direction_index()
        } else {
            def.start_row
        };

        Some(AnimationClip::new(row, def.frame_count, config.atlas_columns))
    }
}
```

Delete update_animation_flags

We no longer need this function since we will be using `Changed<CharacterState>` instead of manual tracking.

```
// src/characters/animation.rs - DELETE this entire function
pub fn update_animation_flags(...) { ... }
```

Replace Animation System

Here's where the state pattern really pays off. The old `animate_characters` function manually tracked state changes with boolean flags. With `CharacterState`, Bevy's `Changed` filter does this for us automatically.

Delete `animate_characters` entirely.

```
// src/characters/animation.rs - DELETE this entire function
pub fn animate_characters(...) { ... }
```

We'll write one system that responds to state changes (using `Changed<CharacterState>`), and another that does animation playback. This separation means state-change logic only runs when needed, not every frame.

System 1: Handle Character State Changes

This system runs only when `CharacterState` changes, using Bevy's `Changed` filter. When triggered, it updates the animation type so the playback system knows which animation to play.

```
// src/characters/animation.rs - Add this new function
pub fn on_state_change_update_animation(
    mut query: Query<
        (&CharacterState, &mut AnimationController, &mut AnimationTimer),
        Changed<CharacterState>
    >,
) {
    for (state, mut controller, mut timer) in query.iter_mut() {
        // Select animation based on new state
        let new_animation = match state {
            CharacterState::Idle | CharacterState::Walking => AnimationType::Walk,
            CharacterState::Running => AnimationType::Run,
            CharacterState::Jumping => AnimationType::Jump,
        };
    }
};
```



```

        // Only update and reset timer if animation actually changed
        if controller.current_animation != new_animation {
            controller.current_animation = new_animation;
            timer.0.reset();
        }
    }
}

```

What's `Changed<CharacterState>`?

This is the change detection we discussed earlier! The query only returns entities whose `CharacterState` changed since last frame. No manual tracking needed.

Why check `controller.current_animation != new_animation` if `Changed` already filters?

Because multiple states can map to the same animation. Look at the match: both `Idle` and `Walking` use `AnimationType::Walk`. If the player transitions from `Idle` to `Walking`, `Changed<CharacterState>` fires (state changed), but the animation type stays `Walk`. Without this guard, we'd reset the timer and cause a visual stutter even though we're playing the same animation.

System 2: Animation Playback

While the first system picks *which* animation to play when state changes, this one handles the frame-by-frame playback. Together they form a complete animation pipeline: state changes set up the animation, and this system keeps it running.

```

// src/characters/animation.rs - Add this new function
pub fn animations_playback(
    time: Res<Time>,
    mut query: Query<(
        &CharacterState,
        &Facing,
        &AnimationController,
        &mut AnimationTimer,
        &mut Sprite,
        &CharacterEntry,
    )>,
) {
    for (state, facing, controller, mut timer, mut sprite, config) in query.iter_mut() {
        // Don't animate when idle
        if *state == CharacterState::Idle {
            // Ensure idle sprite is at frame 0
            if let Some(atlas) = sprite.texture_atlas.as_mut() {
                if let Some(clip) = controller.get_clip(config, *facing) {
                    if atlas.index != clip.start() {
                        atlas.index = clip.start();
                    }
                }
            }
            continue;
        }

        let Some(atlas) = sprite.texture_atlas.as_mut() else { continue; };
        let Some(clip) = controller.get_clip(config, *facing) else { continue; };
        let Some(anim_def) = config.animations.get(&controller.current_animation) else { continue; };

        // Safety: If we somehow ended up on a frame outside our clip, reset.
        if !clip.contains(atlas.index) {
            atlas.index = clip.start();
            timer.0.reset();
        }

        // Update timer duration if needed
    }
}

```

```

    let expected_duration = std::time::Duration::from_secs_f32(anim_def.frame_time);
    if timer.0.duration() != expected_duration {
        timer.0.set_duration(expected_duration);
    }

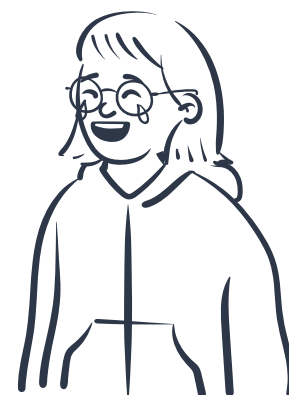
    // Advance animation
    timer.tick(time.delta());
    if timer.just_finished() {
        atlas.index = clip.next(atlas.index);
    }
}
}

```

With enums, my character can't be walking AND idle!



Schrödinger's cat would like a word!



Completing the State-Based Refactoring

We've updated the animation system to use `CharacterState` instead of boolean flags. But where does `CharacterState` get set? Right now, our `movement.rs` still uses the old approach, it directly modifies `Transform` and sets boolean flags in `AnimationState`. We need to refactor it to work with our new state-based design.

Look at the current `movement.rs`. It does three things at once:

1. Reads input (arrow keys, shift, space)
2. Moves the character on screen
3. Decides which animation to play using boolean flags

This mixing of concerns made sense before, but now that we have `CharacterState`, we can separate these responsibilities. We'll split `movement.rs` into:

- **input.rs** - Reads keyboard input and decides what the character should do
- **physics.rs** - Handles moving the character based on velocity

This separation means the animation system we just built will work automatically. When input changes `CharacterState`, our `on_state_change_update_animation` system reacts. When input sets `Velocity`, our physics system moves the entity. Each piece focuses on one job.

Create `src/characters/physics.rs`:

```
// src/characters/physics.rs
use bevy::prelude::*;
use super::{state::CharacterState, config::CharacterEntry};

/// Linear velocity in world units per second.
/// Systems that want to move an entity modify this.
/// A physics system reads this to update Transform.
#[derive(Component, Debug, Clone, Copy, Default, Deref, DerefMut)]
pub struct Velocity(pub Vec2);

impl Velocity {
    pub const ZERO: Self = Self(Vec2::ZERO);

    pub fn is_moving(&self) -> bool {
        self.0 != Vec2::ZERO
    }
}
```

Now add the velocity calculation based on state:

```
// Append to src/characters/physics.rs

/// Calculate velocity based on character state, direction, and configuration.
pub fn calculate_velocity(
    state: CharacterState,
    direction: Vec2,
    character: &CharacterEntry,
) -> Velocity {
    match state {
        CharacterState::Idle => Velocity::ZERO,
        CharacterState::Jumping => Velocity::ZERO, // No movement during jump
        CharacterState::Walking => {
            Velocity(direction.normalize_or_zero() * character.base_move_speed)
        }
        CharacterState::Running => {
            Velocity(direction.normalize_or_zero() * character.base_move_speed *
character.run_speed_multiplier)
        }
    }
}
```

Notice how `CharacterState` directly determines velocity. No boolean flags, no conditionals about `is_jumping && !is_moving`. The state tells us everything we need to know.

Finally, add the system that actually moves the character. It reads the velocity and updates the character's position on screen:

```
// Append to src/characters/physics.rs

/// Applies velocity to transform. Pure physics, no game logic.
pub fn apply_velocity(
    time: Res<Time>,
    mut query: Query<(&Velocity, &mut Transform)>,
) {
    for (velocity, mut transform) in query.iter_mut() {
        if velocity.is_moving() {
            transform.translation += velocity.0.extend(0.0) * time.delta_secs();
        }
    }
}
```

This system knows nothing about input, characters, or states. It just moves things based on their velocity. Add any entity with `Velocity` and `Transform`, and it moves automatically.

Refactoring Player Input

Now for the second half of splitting `movement.rs`. We have physics handling the “how to move” part. Now we need input handling for the “what the player wants to do” part.

This is where we connect everything together. The input system will:

1. Read keyboard input
2. Update `CharacterState` (which triggers our animation system via `Changed<CharacterState>`)
3. Set `Velocity` (which our physics system uses to move the entity)

Create `src/characters/input.rs`. This replaces the input-handling parts of `movement.rs`.

```
// src/characters/input.rs
use bevy::prelude::*;
use super::{
    state::CharacterState,
    physics::Velocity,
    facing::Facing,
    config::CharacterEntry,
    animation::{AnimationController, AnimationTimer},
};

#[derive(Component)]
pub struct Player;
```

We moved the `Player` marker component here since input handling is player-specific.

```
// Append to src/characters/input.rs

/// Read directional input and return a direction vector
fn read_movement_input(input: &ButtonInput<KeyCode>) -> Vec2 {
    const MOVEMENT_KEYS: [(KeyCode, Vec2); 4] = [
        (KeyCode::ArrowLeft, Vec2::NEG_X),
        (KeyCode::ArrowRight, Vec2::X),
        (KeyCode::ArrowUp, Vec2::Y),
        (KeyCode::ArrowDown, Vec2::NEG_Y),
    ];

    MOVEMENT_KEYS.iter()
        .filter(|(key, _)| input.pressed(*key))
        .map(|(_, dir)| *dir)
        .sum()
}
```

This is the same input reading we had before, just isolated into its own function.

State Machine Logic

Now we need to decide what state the character should be in based on that input. Remember earlier we said “the state tells us everything we need to know”? This is where we translate raw input into meaningful state transitions.

Instead of scattered `if` statements that set boolean flags (`is_moving = true`, `is_jumping = true`), we have one function that returns the new state:

```
// Append to src/characters/input.rs
```

```
fn determine_new_state(
    current: CharacterState,
    direction: Vec2,
    is_running: bool,
    wants_jump: bool,
) -> CharacterState {
    match current {
        // Can't transition out of jumping until it completes
        CharacterState::Jumping => CharacterState::Jumping,

        // Jump takes priority when grounded
        _ if wants_jump && current.is_grounded() => CharacterState::Jumping,

        // Movement states
        _ if direction != Vec2::ZERO => {
            if is_running { CharacterState::Running } else { CharacterState::Walking }
        }

        // Default to idle
        _ => CharacterState::Idle,
    }
}
```

What's this strange pattern of having `if` conditions inside `match`?

This is called a *match guard*. The syntax `_ if condition =>` means “match anything, but only if this condition is also true.” It combines pattern matching with boolean logic.

Why use it here?

We need to check two things: what state we're currently in, and what the player is doing (moving, jumping, etc.). Match guards let us handle both in one clean expression. The `_` means “any state not already matched above,” and the `if` adds the extra condition.

You can use this pattern when you need to match on one thing but also check something else that isn't part of the enum itself.

Alright, the new approach puts all state transition logic in one function. You can read it top to bottom and understand the priority.

The Main Input Handler

We've built the helper functions: `read_movement_input` reads keys, `determine_new_state` decides the state. Now we need the main system that ties them together and actually updates the entity's components.

This is the new version of the old `move_player` function from `movement.rs`. Instead of directly modifying `Transform` and `AnimationController`, it now updates `CharacterState`, `Velocity`, and `Facing`. Other systems react to these changes: the animation system responds to `Changed<CharacterState>`, and the physics system reads `Velocity` to move the entity.

The function works in four steps:

1. **Read input** - Check which keys are pressed (arrows for direction, shift for running, space for jump)
2. **Update facing** - If moving, update the facing direction so the character looks where they're going
3. **Determine new state** - Use the state machine to figure out the next state based on current state and input
4. **Calculate velocity** - Based on the new state, calculate how fast and which direction to move

```
// Append to src/characters/input.rs
```

```

/// Reads player input and updates movement-related components.
pub fn handle_player_input(
    input: Res<ButtonInput<KeyCode>>,
    mut query: Query<(
        &mut CharacterState,
        &mut Velocity,
        &mut Facing,
        &CharacterEntry,
    ), With<Player>>,
) {
    let Ok((mut state, mut velocity, mut facing, character)) = query.single_mut() else {
        return;
    };

    // Step 1: Read what keys are pressed
    let direction = read_movement_input(&input);
    let is_running = input.pressed(KeyCode::ShiftLeft) || input.pressed(KeyCode::ShiftRight);
    let wants_jump = input.just_pressed(KeyCode::Space);

    // Step 2: Update facing direction (which way the character looks)
    if direction != Vec2::ZERO {
        let new_facing = Facing::from_velocity(direction);
        if *facing != new_facing {
            *facing = new_facing;
        }
    }

    // Step 3: Use our state machine to determine the new state
    // This calls the determine_new_state function we wrote earlier
    let new_state = determine_new_state(*state, direction, is_running, wants_jump);
    if *state != new_state {
        *state = new_state; // This triggers Changed<CharacterState>!
    }

    // Step 4: Calculate velocity based on state
    // Idle and Jumping = no movement, Walking/Running = movement
    *velocity = super::physics::calculate_velocity(*state, direction, character);
}

```

Handling Jump Completion

There's one edge case our main input handler doesn't cover. Look at `determine_new_state`, when the character is `Jumping`, it stays `Jumping`. But how does jumping ever end?

Unlike walking or running (which end when you release the key), jumping needs to complete its animation before transitioning back to idle. We need a separate system that watches for this:

```

// Append to src/characters/input.rs

/// Checks if jump animation completed and transitions back to idle
pub fn update_jump_state(
    mut query: Query<(
        &mut CharacterState,
        &Facing,
        &AnimationController,
        &AnimationTimer,
        &Sprite,
        &CharacterEntry,
    ), With<Player>>,
) {
    let Ok((mut state, facing, controller, timer, sprite, config)) = query.single_mut() else {
        return;
    };
}

```



```

// Only check if currently jumping
if *state != CharacterState::Jumping {
    return;
}

let Some(atlas) = sprite.texture_atlas.as_ref() else {
    return;
};

let Some(clip) = controller.get_clip(config, *facing) else {
    return;
};

// Check if jump animation has completed
if clip.is_complete(atlas.index, timer.just_finished()) {
    *state = CharacterState::Idle;
}
}

```

This system only runs meaningful logic when the player is jumping. It uses `clip.is_complete()` to check if the animation finished, then transitions to Idle. The state change triggers our `on_state_change_update_animation` system, which updates the animation to Walk.

Before we update `mod.rs`, we need to update `spawn.rs`. We moved `Player` to `input.rs`, and our new systems expect entities to have `CharacterState`, `Velocity`, and `Facing` components.

Update the imports at the top of `src/characters/spawn.rs`:

```

// src/characters/spawn.rs - Update imports
use bevy::prelude::*;
use crate::characters::animation::*;
use crate::characters::config::{CharacterEntry, CharactersList};
use crate::characters::input::Player; // Changed from movement::Player
use crate::characters::state::CharacterState; // Line update alert
use crate::characters::physics::Velocity; // Line update alert
use crate::characters::facing::Facing; // Line update alert

```

Then in `initialize_player_character`, update the components attached to the player entity.

```

// src/characters/spawn.rs - Inside initialize_player_character
// Update commands.entity(entity).insert(...) function call
// Remove the line AnimationState::default(), and add the following lines:
commands.entity(entity).insert((
    AnimationController::default(),
    CharacterState::default(), // Line update alert
    Velocity::default(), // Line update alert
    Facing::default(), // Line update alert
    AnimationTimer(Timer::from_seconds(DEFAULT_ANIMATION_FRAME_TIME, TimerMode::Repeating)),
    character_entry.clone(),
    sprite,
));

```

Now the player entity has all the components our refactored systems need: `CharacterState` for the animation system to react to, `Velocity` for the physics system to read, and `Facing` for sprite direction.

Wiring Up the New Systems

Update `src/characters/mod.rs` to include the new modules:


```
// src/characters/mod.rs - Update module declarations
pub mod animation;
pub mod config;
pub mod facing;      // Line update alert
pub mod input;        // Line update alert
pub mod physics;      // Line update alert
pub mod spawn;
pub mod state;        // Line update alert

// DELETE this line:
// pub mod movement;

use crate::state::GameState;
```

Now update the system registration. Remove the old systems and add the new ones:

```
// src/characters/mod.rs - DELETE these old systems from add_systems
movement::move_player,      // DELETE
movement::update_jump_state, // DELETE
animation::animate_characters, // DELETE
animation::update_animation_flags, // DELETE
```

Replace them with our new systems. Notice we're using `.chain()` to ensure they run in order, and `.run_if(in_state(GameState::Playing))` so they only run during gameplay. **This is where we actually implement the freeze-on-pause behavior**, when the game is in the `Paused` state, these systems won't run, freezing all character movement and animation:

```
// src/characters/mod.rs - Add new systems
.add_systems(Update, (
    input::handle_player_input,
    spawn::switch_character,
    input::update_jump_state,
    animation::on_state_change_update_animation,
    physics::apply_velocity,
    animation::animations_playback,
).chain().run_if(in_state(GameState::Playing))));
```

The `.chain()` ensures systems run in order. Input sets state and velocity, animation responds to state changes, physics moves the entity, and animation playback animates the character.

Building a Collision System

Right now our character can walk anywhere, even through trees and into water. We need a collision system that prevents movement into obstacles.

Our approach is simple, each tile in our world has a *type* (grass, water, tree, etc.), and each type is either walkable or not. When the player tries to move, we check if the destination is walkable. If not, we block the move.

Defining Tile Types

First, we need to categorize what kinds of tiles exist in our world. Create `src/collision/tile_type.rs`:

```
// src/collision/tile_type.rs
use bevy::prelude::*;

/// Tile types for collision detection.
/// Each type has different walkability and collision behavior.
```

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Default)]
pub enum TileType {
    // Walkable terrain
    #[default]
    Empty,
    Dirt,
    Grass,
    YellowGrass,
    Shore, // Water edges (walkable)
    // Non-walkable obstacles
    Water,
    Tree,
    Rock,
}
```

Now add a method to check walkability:

```
// Append to src/collision/tile_type.rs
impl TileType {
    /// Check if this tile type allows movement through it.
    pub fn is_walkable(&self) -> bool {
        !matches!(self, TileType::Water | TileType::Tree | TileType::Rock)
    }

    /// Get the collision adjustment for this tile type.
    /// Positive = push player away, negative = allow corner cutting.
    pub fn collision_adjustment(&self) -> f32 {
        match self {
            TileType::Tree | TileType::Rock => -0.2, // Allow cutting corners
            _ => 0.0,
        }
    }
}
```

Notice how we define walkability: instead of listing everything that **is** walkable, we list what **isn't**. This means new tile types are walkable by default, a safer choice since forgetting to add a tile would make it passable rather than creating invisible walls.

What's `collision_adjustment`?

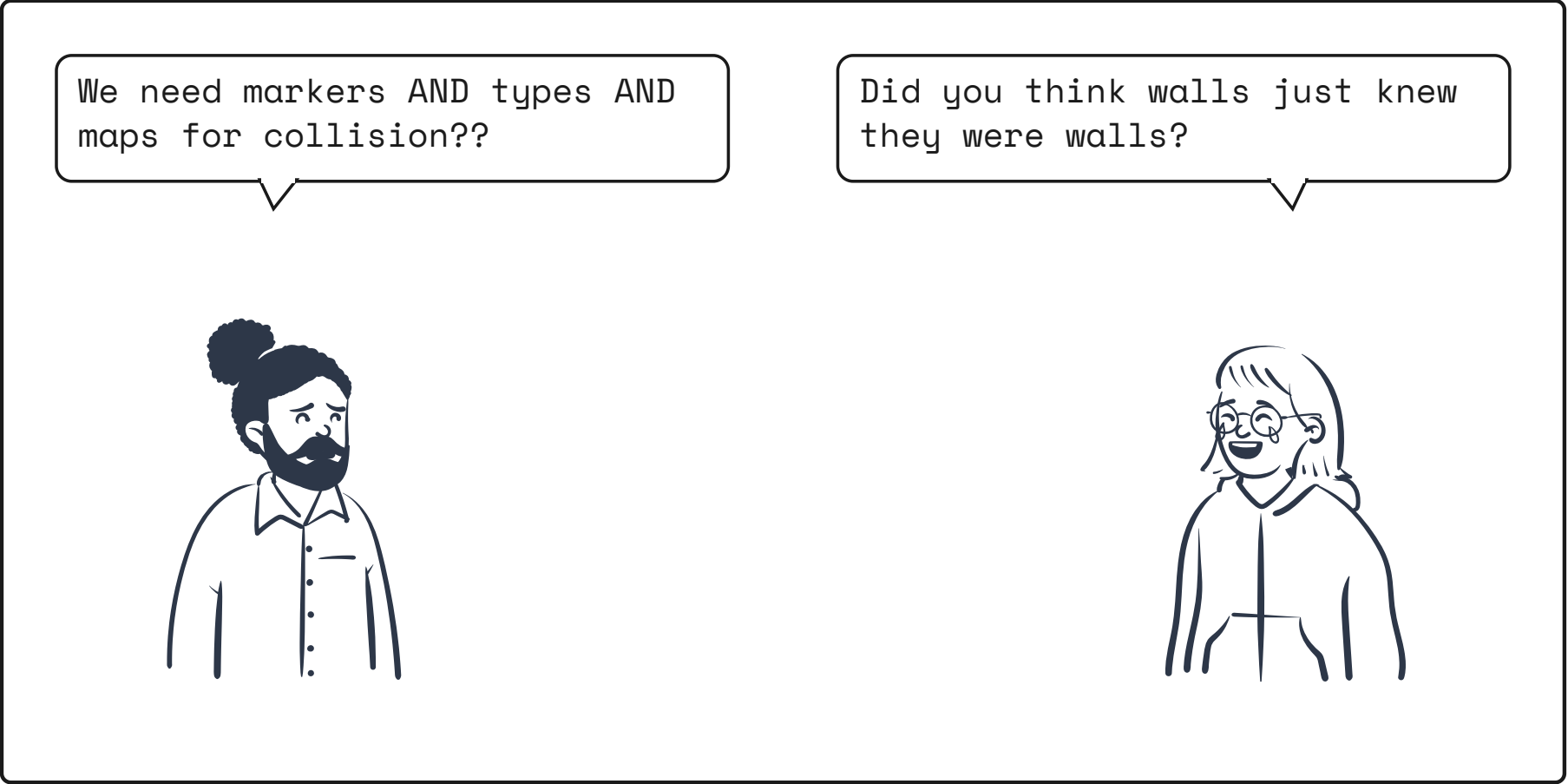
Some tiles feel better with adjusted collision. A negative value (like -0.2 for trees and rocks) lets players cut corners more naturally instead of getting stuck on edges. Positive values would push players away from tiles.

We also need a marker component to attach tile type information to entities:

```
// Append to src/collision/tile_type.rs

/// Component to mark entities with their collision type.
/// Attached to tiles during map generation.
#[derive(Component, Debug, Clone)]
pub struct TileMarker {
    pub tile_type: TileType,
}

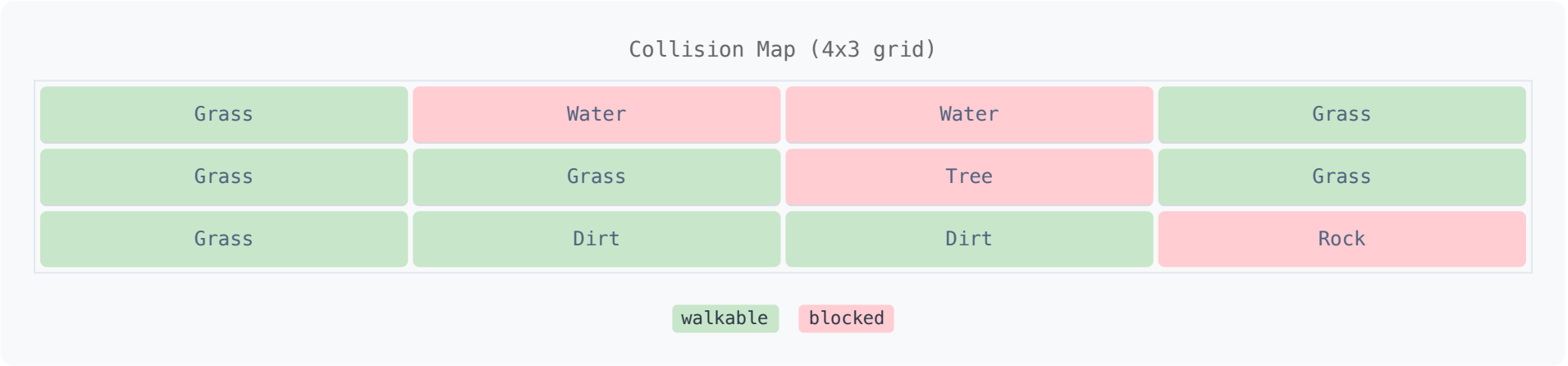
impl TileMarker {
    pub fn new(tile_type: TileType) -> Self {
        Self { tile_type }
    }
}
```



The Collision Map

Every frame, when the player tries to move, we need to answer “can they move there?” quickly. Checking every tree and water tile in the world each frame would be slow. Instead, we build a lookup table once: a grid where each cell knows if it’s walkable or not.

Here’s what a collision map looks like for a small 4x3 area:



This is `CollisionMap`. When the level loads, we scan all tiles and record their types. During gameplay, checking “is position (x, y) walkable?” is just an array lookup.

Create `src/collision/map.rs`:

```
// src/collision/map.rs
use bevy::prelude::*;
use super::TileType;

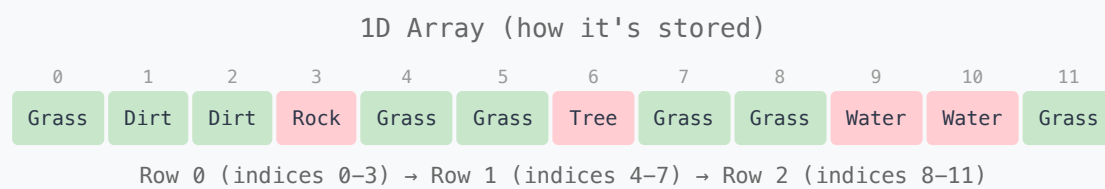
/// Collision map resource that stores walkability information.
/// Provides efficient spatial queries for movement validation.
#[derive(Resource)]
pub struct CollisionMap {
    /// Flat array of tile types (row-major order)
    tiles: Vec<TileType>,
    /// Grid dimensions
    width: i32,
    height: i32,
    /// Size of each tile in world units
    tile_size: f32,
    /// World position of grid origin (bottom-left corner)
    origin_x: f32,
```

```
origin_y: f32,
}
```

We store tiles in a flat `Vec` rather than a 2D array for better performance.

Why a flat array?

In memory, we flatten the grid above row by row into a single array:

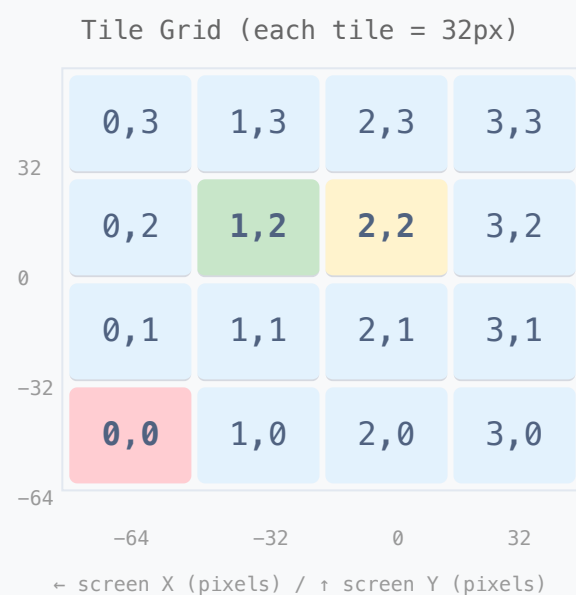


All 12 tiles sit next to each other in memory. When you access index 6 (Tree), indices 7 and 8 are likely already loaded into fast memory. A 2D array (`Vec<Vec<TileType>>`) stores each row separately, which can be slower.

Why store origin?

In Bevy, the default camera places (0, 0) at the center of the screen. If we center a 4x4 tile grid, where does each tile sit?

Our grid uses **tile coordinates**: the bottom-left tile is (0, 0), the one to its right is (1, 0), and so on.



origin (tile 0,0) screen center player

The grid is 4 tiles wide × 32 pixels each = 128 pixels total. To center it, we shift left by half: $128 \div 2 = 64$ pixels. So the grid's bottom-left corner (tile 0,0) sits at screen position **(-64, -64)**. That's the **origin**.

Screen center (0, 0) lands inside tile (2, 2), not tile (0, 0)!

Now the player stands at screen position (-32, 0). Which tile?

- **With origin:** $(-32 - (-64)) / 32 = 1$, $(0 - (-64)) / 32 = 2$ → Tile (1, 2) ✓
- **Without origin:** $(-32) / 32 = -1$, $(0) / 32 = 0$ → Tile (-1, 0) **(wrong tile!)**

That's why our `CollisionMap` stores the origin. Let's implement the struct with methods to handle this conversion automatically.

The constructor creates an empty map filled with `TileType::Empty`. We also need two internal helpers:

- `xy_to_idx` converts 2D coordinates like (3, 7) to a single number for array access, since tiles are stored in a 1D array.
- `in_bounds` checks if coordinates are inside the grid, this serves double duty: it prevents accessing invalid memory and treats anything outside the map as "blocked" for collision

purposes.

```
// Append to src/collision/map.rs
impl CollisionMap {
    /// Create a new collision map with specified dimensions and origin.
    pub fn new(width: i32, height: i32, tile_size: f32, origin_x: f32, origin_y: f32) -> Self {
        let size = (width * height) as usize;
        Self {
            tiles: vec![TileType::Empty; size],
            width,
            height,
            tile_size,
            origin_x,
            origin_y,
        }
    }

    /// Convert 2D grid coordinates to 1D array index.
    #[inline]
    fn xy_to_idx(&self, x: i32, y: i32) -> usize {
        (y * self.width + x) as usize
    }

    /// Check if grid coordinates are within bounds.
    #[inline]
    pub fn in_bounds(&self, x: i32, y: i32) -> bool {
        x >= 0 && x < self.width && y >= 0 && y < self.height
    }
}
```

Players move in screen positions like (150.5, -32.0). We need to convert these to tile coordinates like (4, -1) to check collision. That's what these two functions do.

What's `#[inline]`?

This hints to the compiler that these small, frequently-called functions should be inlined (copied directly into the caller function) rather than called as separate functions.

```
// Append to src/collision/map.rs
// Add this inside impl CollisionMap {

    // ... earlier functions inside impl CollisionMap

    /// Convert world position to grid coordinates.
    pub fn world_to_grid(&self, world_pos: Vec2) -> IVec2 {
        let grid_x = ((world_pos.x - self.origin_x) / self.tile_size).floor() as i32;
        let grid_y = ((world_pos.y - self.origin_y) / self.tile_size).floor() as i32;
        IVec2::new(grid_x, grid_y)
    }

    /// Convert grid coordinates to world position (tile center).
    pub fn grid_to_world(&self, grid_x: i32, grid_y: i32) -> Vec2 {
        Vec2::new(
            self.origin_x + (grid_x as f32 + 0.5) * self.tile_size,
            self.origin_y + (grid_y as f32 + 0.5) * self.tile_size,
        )
    }
}
```

Understanding `grid_to_world`

This function does the opposite of `world_to_grid`: it takes grid coordinates like (3, 7) and converts them back to a world position.

Notice the `+ 0.5` in the code—this returns the **center** of the tile rather than its corner. If tile (3, 7) spans pixels 96-127, adding 0.5 gives us the center at pixel 112:

```
origin_x + (grid_x * tile_size)      // ← This gives the left edge (96)
origin_x + (grid_x + 0.5) * tile_size // ← This gives the center (112)
```

This is useful because when you spawn a sprite in Bevy, its `Transform` position represents its center point by default. So `grid_to_world` gives you the exact position to place entities that should be visually centered in their tiles.

Now add tile access methods. Here's how they fit together:

- `set_tile` is used when building the map (during level load, we scan tiles and call `set_tile` for each one)
- `get_tile` retrieves a tile type at grid coordinates
- `is_walkable` asks **can I walk on tile (3, 7)?** It calls `get_tile`, then checks if that tile type is walkable
- `is_world_pos_walkable` is the same question, but starts from screen position (150, -32). It converts to grid coordinates, then calls `is_walkable`

In practice, the circle collision code uses these internally. You rarely call them directly.

```
// Append to src/collision/map.rs
// Add this inside impl CollisionMap {

    // ... earlier functions inside impl CollisionMap

    /// Get the tile type at grid coordinates.
    pub fn get_tile(&self, x: i32, y: i32) -> Option<TileType> {
        if self.in_bounds(x, y) {
            Some(self.tiles[self.xy_to_idx(x, y)])
        } else {
            None
        }
    }

    /// Set a tile at grid coordinates.
    pub fn set_tile(&mut self, x: i32, y: i32, tile_type: TileType) {
        if self.in_bounds(x, y) {
            let idx = self.xy_to_idx(x, y);
            self.tiles[idx] = tile_type;
        }
    }

    /// Check if a grid position is walkable.
    pub fn is_walkable(&self, x: i32, y: i32) -> bool {
        self.get_tile(x, y).map_or(false, |t| t.is_walkable())
    }

    /// Check if a world position is walkable.
    pub fn is_world_pos_walkable(&self, world_pos: Vec2) -> bool {
        let grid_pos = self.world_to_grid(world_pos);
        self.is_walkable(grid_pos.x, grid_pos.y)
    }
}
```

Circle Collision

The methods above check if a single **point** is walkable. But our character isn't a point, they have a body! If we only check the player's center position, they could overlap walls.

We model the player's collision area as a circle at their center. To check if a circle collides with a tile (rectangle), we need to solve a problem: **how do you test if a circle overlaps a rectangle?**

The trick is to find the point on the rectangle closest to the circle's center. If that point is inside the circle (distance less than radius), they overlap.

```
// Append to src/collision/map.rs
// Add this inside impl CollisionMap {

    // ... earlier functions inside impl CollisionMap

    /// Check if a circle intersects with a tile's bounding box.
    fn circle_intersects_tile(&self, center: Vec2, radius: f32, gx: i32, gy: i32) -> bool {
        // Tile bounding box
        let tile_min = Vec2::new(
            self.origin_x + gx as f32 * self.tile_size,
            self.origin_y + gy as f32 * self.tile_size,
        );
        let tile_max = tile_min + Vec2::splat(self.tile_size);

        // Find closest point on tile to circle center
        let closest = Vec2::new(
            center.x.clamp(tile_min.x, tile_max.x),
            center.y.clamp(tile_min.y, tile_max.y),
        );

        // Check if closest point is within radius
        center.distance_squared(closest) <= radius * radius
    }
}
```

With `circle_intersects_tile` ready, we can build `is_circle_clear`. But first, we need to prevent the player from walking off the edge of the map.

```
// Append to src/collision/map.rs
// Add this inside impl CollisionMap {

    // ... earlier functions inside impl CollisionMap
    /// Check if a position with radius is within map bounds.
    fn is_within_bounds(&self, center: Vec2, radius: f32) -> bool {
        let left = self.origin_x;
        let right = self.origin_x + self.width as f32 * self.tile_size;
        let bottom = self.origin_y;
        let top = self.origin_y + self.height as f32 * self.tile_size;

        center.x - radius >= left
            && center.x + radius <= right
            && center.y - radius >= bottom
            && center.y + radius <= top
    }
}
```

Now the main collision check.

```
// Append to src/collision/map.rs
// Add this inside impl CollisionMap {

    // ... earlier functions inside impl CollisionMap

    /// Check if a circle at the given world position is clear of obstacles.
```



```

pub fn is_circle_clear(&self, center: Vec2, radius: f32) -> bool {
    // Early bounds check
    if !self.is_within_bounds(center, radius) {
        return false;
    }

    // Point collision if no radius
    if radius <= 0.0 {
        return self.is_world_pos_walkable(center);
    }

    // Find grid cells that could overlap the circle
    let min_gx = ((center.x - radius - self.origin_x) / self.tile_size).floor() as i32;
    let max_gx = ((center.x + radius - self.origin_x) / self.tile_size).floor() as i32;
    let min_gy = ((center.y - radius - self.origin_y) / self.tile_size).floor() as i32;
    let max_gy = ((center.y + radius - self.origin_y) / self.tile_size).floor() as i32;

    for gy in min_gy..=max_gy {
        for gx in min_gx..=max_gx {
            if !self.in_bounds(gx, gy) {
                return false; // Out of bounds = blocked
            }

            if let Some(tile) = self.get_tile(gx, gy) {
                if !tile.is_walkable() {
                    // Apply tile-specific collision adjustment
                    let effective_radius = radius + tile.collision_adjustment() *
self.tile_size;

                    if self.circle_intersects_tile(center, effective_radius, gx, gy) {
                        return false;
                    }
                }
            }
        }
    }
}
true
}

```

Here's how `is_circle_clear` works:

1. Early exit if circle is outside map bounds (`is_within_bounds`)
2. Find all grid cells the circle might touch (based on circle bounds)
3. For each unwalkable tile, use `circle_intersects_tile` to check overlap
4. Apply `collision_adjustment()` to allow corner cutting on certain tiles

Notice that `is_circle_clear` builds on everything we've created: `in_bounds`, `get_tile`, `is_walkable`, `is_within_bounds`, and `circle_intersects_tile`. Each layer builds on the previous one.

Swept Collision

We can now check if a position is valid using `is_circle_clear`. But there's a problem: if a player moves fast enough in one frame, they could jump over a thin wall. The collision check at the destination would pass, but they'd skip right through the obstacle.

`sweep_circle` solves this by checking the **entire path** from start to end. It uses `is_circle_clear` repeatedly along small steps, ensuring we catch any collision along the way:

```

// Append to src/collision/map.rs
// Add this inside impl CollisionMap {

    // ... earlier functions inside impl CollisionMap

```

```

/// Perform swept circle movement with axis-sliding.
/// Returns the furthest valid position the circle can reach.
pub fn sweep_circle(&self, start: Vec2, end: Vec2, radius: f32) -> Vec2 {
    let delta = end - start;

    // No movement needed
    if delta.length() < 0.001 {
        return start;
    }

    // Step size (quarter tile for smooth collision)
    let max_step = self.tile_size * 0.25;
    let steps = (delta.length() / max_step).ceil().max(1.0) as i32;
    let step_vec = delta / steps as f32;

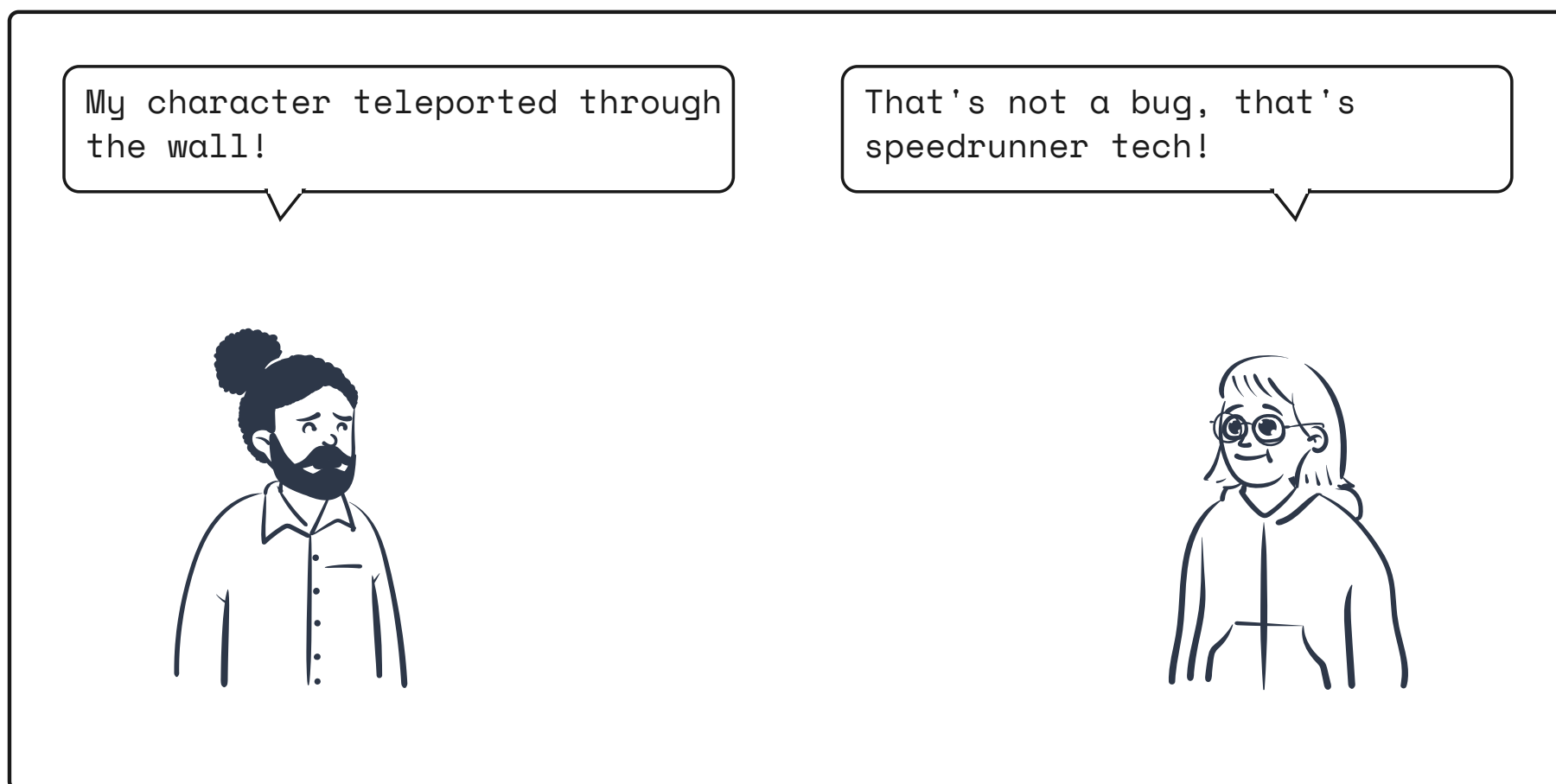
    let mut pos = start;
    for _ in 0..steps {
        let candidate = pos + step_vec;

        if self.is_circle_clear(candidate, radius) {
            pos = candidate;
        } else {
            // Try sliding along X axis only
            let try_x = Vec2::new(candidate.x, pos.y);
            if self.is_circle_clear(try_x, radius) {
                pos = try_x;
                continue;
            }

            // Try sliding along Y axis only
            let try_y = Vec2::new(pos.x, candidate.y);
            if self.is_circle_clear(try_y, radius) {
                pos = try_y;
                continue;
            }

            // Completely blocked
            break;
        }
    }
    pos
}

```



We'll be soon working on a feature to visually debug collisions. We need the following helper functions to calculate the collision map. The `#[cfg(debug_assertions)]` attribute means these only exist in debug builds, they're completely removed from release builds:

```
// Append to src/collision/map.rs
// Append to src/collision/map.rs
// Add this inside impl CollisionMap {

    // ... earlier functions inside impl CollisionMap
    #[cfg(debug_assertions)]
    pub fn width(&self) -> i32 { self.width }

    #[cfg(debug_assertions)]
    pub fn height(&self) -> i32 { self.height }

    #[cfg(debug_assertions)]
    pub fn tile_size(&self) -> f32 { self.tile_size }

    #[cfg(debug_assertions)]
    pub fn origin(&self) -> Vec2 { Vec2::new(self.origin_x, self.origin_y) }
```

Building the Collision Map

We have the `CollisionMap` data structure, but when does it get populated? We need a system that scans all the tiles after WFC generation and builds the collision map.

Create `src/collision/systems.rs`:

The collision map should only be built once after tiles are spawned. We need a way to track this. A simple boolean resource does the job: it starts as `false`, and once we build the map, we flip it to `true`.

We register this resource with Bevy's scheduler using `run_if(resource_equals(CollisionMapBuilt(false)))`, which means the build system won't even run after the map is built.

```
// src/collision/systems.rs
use bevy::prelude::*;
```

```

use std::collections::{HashMap, hash_map::Entry};

use super::{CollisionMap, TileMarker, TileType};
use crate::config::map::{TILE_SIZE, GRID_X, GRID_Y};

/// Resource to track if collision map has been built.
#[derive(Resource, Default, PartialEq, Eq)]
pub struct CollisionMapBuilt(pub bool);

```

Now the main system that builds the map. This is the longest function we've written, so let's understand what it does before looking at the code.

The system detects when WFC has finished by querying for tiles, if no tiles exist yet, it exits early. Once tiles exist, it:

1. **Calculates the grid origin** - Our map is centered on the screen, so the bottom-left corner of the grid isn't at (0, 0). We need to figure out where it actually is.
2. **Scans all tiles** - Each tile entity has a `Transform` (its position in the world) and a `TileMarker` (what type of tile it is). We loop through every tile and read both.
3. **Handles overlapping tiles** - Our WFC generator can place tiles on top of each other. For example, a tree sprite sits on top of grass at the same (x, y) position but at a higher Z (depth). For collision, we only care about the topmost tile: if there's a tree on grass, the player collides with the tree.
4. **Tracks bounds** - We track the leftmost, rightmost, topmost, and bottommost tile coordinates. From these, we calculate the dimensions of the `CollisionMap`.
5. **Creates and populates the CollisionMap** - Using the `world_to_grid` logic we built earlier, we convert each tile's world position to grid coordinates and call `set_tile` to record its type.
6. **Post-processes** - Finally, we run shore conversion to turn water tiles at the edge of lakes into walkable shore tiles. This gives players a better experience at water boundaries.

```

// Append to src/collision/systems.rs

/// System that builds the collision map from spawned tiles.
/// Handles multi-layer maps by keeping only the TOPMOST tile at each (x,y).
pub fn build_collision_map(
    mut commands: Commands,
    mut built: ResMut<CollisionMapBuilt>,
    tile_query: Query<(&TileMarker, &Transform)>,
) {
    // Need at least one tile to proceed
    let mut tile_iter = tile_query.iter();
    let Some((first_marker, first_transform)) = tile_iter.next() else {
        return; // WFC hasn't generated tiles yet
    };

    // Calculate grid origin (centered map)
    let grid_origin_x = -TILE_SIZE * GRID_X as f32 / 2.0;
    let grid_origin_y = -TILE_SIZE * GRID_Y as f32 / 2.0;

    // Track bounds and layer info
    let (mut min_x, mut max_x) = (i32::MAX, i32::MIN);
    let (mut min_y, mut max_y) = (i32::MAX, i32::MIN);
    let mut layer_tracker: HashMap<i32, i32>, (TileType, f32)> = HashMap::new();
    let mut tile_count: usize = 0;

    // Process all tiles, keeping only the topmost at each position
    let mut process_tile = |marker: &TileMarker, transform: &Transform| {
        tile_count += 1;

        let world_x = transform.translation.x;

```

```

    let world_y = transform.translation.y;
    let world_z = transform.translation.z;

    let grid_x = ((world_x - grid_origin_x) / TILE_SIZE).floor() as i32;
    let grid_y = ((world_y - grid_origin_y) / TILE_SIZE).floor() as i32;

    min_x = min_x.min(grid_x);
    max_x = max_x.max(grid_x);
    min_y = min_y.min(grid_y);
    max_y = max_y.max(grid_y);

    // Keep only the topmost layer (highest Z)
    match layer_tracker.entry((grid_x, grid_y)) {
        Entry::Occupied(mut entry) => {
            if world_z > entry.get().1 {
                *entry.get_mut() = (marker.tile_type, world_z);
            }
        }
        Entry::Vacant(entry) => {
            entry.insert((marker.tile_type, world_z));
        }
    }
};

// Process first tile and remaining
process_tile(first_marker, first_transform);
for (marker, transform) in tile_iter {
    process_tile(marker, transform);
}

// Calculate actual dimensions
let actual_width = (max_x - min_x + 1) as i32;
let actual_height = (max_y - min_y + 1) as i32;

// Create the collision map
let mut map = CollisionMap::new(
    actual_width,
    actual_height,
    TILE_SIZE,
    grid_origin_x,
    grid_origin_y,
);

// Populate the map from layer tracker
for ((grid_x, grid_y), (tile_type, _z)) in layer_tracker.iter() {
    // Convert world grid to local array coordinates
    let local_x = grid_x - min_x;
    let local_y = grid_y - min_y;
    map.set_tile(local_x, local_y, *tile_type);
}

// Post-processing: Convert water edges to shore
convert_water_edges_to_shore(&mut map);
// Insert as resource and mark built
commands.insert_resource(map);
built.0 = true;
}

```

Shore Conversion

Imagine a lake in your game. Water tiles block movement completely, but what about the edge of the lake? If the player walks up to water at the edge of grass, they should be able to get close to the waterline rather than stopping a full tile away.

Shore conversion solves this by finding water tiles that touch walkable terrain and marking them as `Shore`. Shore tiles are walkable (remember our `TileType::Shore` from earlier), so the player can walk right up to the water's edge while still being blocked by the deep water in the middle of the lake.

The algorithm is straightforward: scan every tile, find water tiles, check their 8 neighbors, and if any neighbor is walkable, mark this water tile as shore.

```
// Append to src/collision/systems.rs

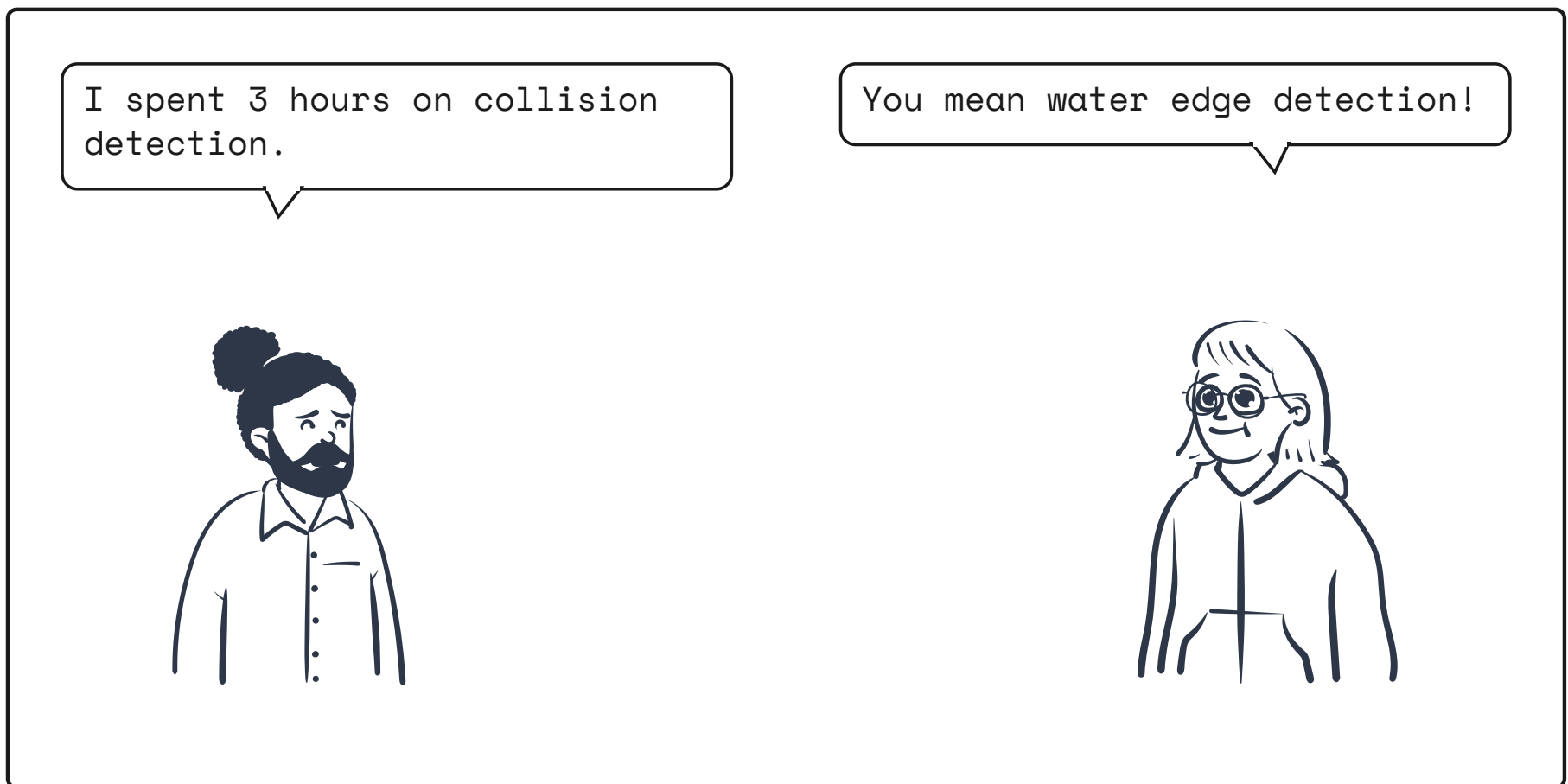
/// Convert water tiles adjacent to walkable tiles into shore tiles.
fn convert_water_edges_to_shore(map: &mut CollisionMap) {
    let mut shores = Vec::new();

    // Find water tiles that touch walkable tiles
    for y in 0..map.height() {
        for x in 0..map.width() {
            if map.get_tile(x, y) != Some(TileType::Water) {
                continue;
            }

            // Check 8 neighbors
            let neighbors = [
                (x - 1, y),      (x + 1, y),      // left, right
                (x, y - 1),      (x, y + 1),      // down, up
                (x - 1, y - 1),  (x + 1, y - 1),  // bottom corners
                (x - 1, y + 1),  (x + 1, y + 1),  // top corners
            ];

            for (nx, ny) in neighbors {
                if map.is_walkable(nx, ny) {
                    shores.push((x, y));
                    break;
                }
            }
        }
    }

    for (x, y) in shores {
        map.set_tile(x, y, TileType::Shore);
    }
}
```

Debug Visualization

When collision isn't working, you need to see what's happening. Is the player's collider in the right place? Are tiles marked correctly? Visual debugging makes these problems obvious.

We'll create a debug overlay that:

- Shows green for walkable tiles, red for blocked
- Draws the player's collision circle
- Highlights which grid cell the player is in
- Only exists in debug builds (removed from release)

Create `src/collision/debug.rs`:

```
// src/collision/debug.rs
use bevy::prelude::*;
use super::CollisionMap;
use crate::characters::input::Player;
use crate::characters::collider::Collider;

/// Resource to toggle debug visualization.
#[derive(Resource, Default)]
pub struct DebugCollisionEnabled(pub bool);
```

This `DebugCollisionEnabled` resource controls whether debug drawing is active. We start with it off and let the player toggle it with a key.

```
// Append to src/collision/debug.rs

/// Toggle collision debug visualization with F3 key.
pub fn toggle_debug_collision(
    keyboard: Res<ButtonInput<KeyCode>>,
    mut debug_enabled: ResMut<DebugCollisionEnabled>,
) {
    if keyboard.just_pressed(KeyCode::F3) {
        debug_enabled.0 = !debug_enabled.0;
        if debug_enabled.0 {
            info!("🔍 Collision debug ENABLED (F3 to toggle)");
        } else {
            info!("🔍 Collision debug DISABLED (F3 to toggle)");
        }
    }
}
```



```

        info!("Collision debug disabled");
    }
}
}

```

Why `debug_enabled.0` instead of just `debug_enabled`?

Our `DebugCollisionEnabled` is a “tuple struct” - a struct that wraps a single value. The `.0` accesses the first (and only) field inside it. This pattern is common in Rust for creating type-safe wrappers: instead of passing around a raw `bool`, we wrap it in a named type that makes the code’s intent clearer.

We have used F3 as keyboard shortcut to toggle the debug overlay on and off in the gameplay.

What’s `Gizmos`?

Gizmos are Bevy’s built-in tool for drawing debug shapes like lines, circles, and rectangles. Unlike regular sprites, gizmos only last for one frame and automatically disappear. This makes them perfect for debug visualization: you draw what you need each frame, and when you stop drawing, they’re gone.

```

// Append to src/collision/debug.rs

/// Draw colored rectangles over tiles showing walkability.
pub fn debug_draw_collision(
    map: Option<Res<CollisionMap>>,
    debug_enabled: Res<DebugCollisionEnabled>,
    mut gizmos: Gizmos,
) {
    if !debug_enabled.0 {
        return;
    }

    let Some(map) = map else { return };

    let tile_size = map.tile_size();
    let origin = map.origin();

    // Draw each tile
    for y in 0..map.height() {
        for x in 0..map.width() {
            let world_pos = Vec2::new(
                origin.x + (x as f32 + 0.5) * tile_size,
                origin.y + (y as f32 + 0.5) * tile_size,
            );

            let color = if map.is_walkable(x, y) {
                Color::srgba(0.0, 1.0, 0.0, 0.25) // Green, 25% opacity
            } else {
                Color::srgba(1.0, 0.0, 0.0, 0.4) // Red, 40% opacity
            };

            gizmos.rect_2d(
                world_pos,
                Vec2::splat(tile_size * 0.9),
                color,
            );
        }
    }
}

```

This loops through every tile and draws a colored rectangle: green for walkable, red for blocked. The 0.9 multiplier makes each rectangle slightly smaller than the tile so you can see the grid lines.

Next, we visualize the player's collider:

```
// Append to src/collision/debug.rs

/// Draw player position and collider visualization.
pub fn debug_player_position(
    player_query: Query<(&Transform, &Collider), With<Player>>,
    map: Option<Res<CollisionMap>>,
    debug_enabled: Res<DebugCollisionEnabled>,
    mut gizmos: Gizmos,
) {
    if !debug_enabled.0 {
        return;
    }

    let Some(map) = map else { return };
    let Ok((transform, collider)) = player_query.single() else { return };

    let center = transform.translation.truncate();

    // Get actual collider position
    let collider_pos = collider.world_position(transform);
    let grid = map.world_to_grid(collider_pos);

    // Draw line from center to collider (shows offset)
    gizmos.line_2d(center, collider_pos, Color::srgba(1.0, 1.0, 0.0, 0.5));

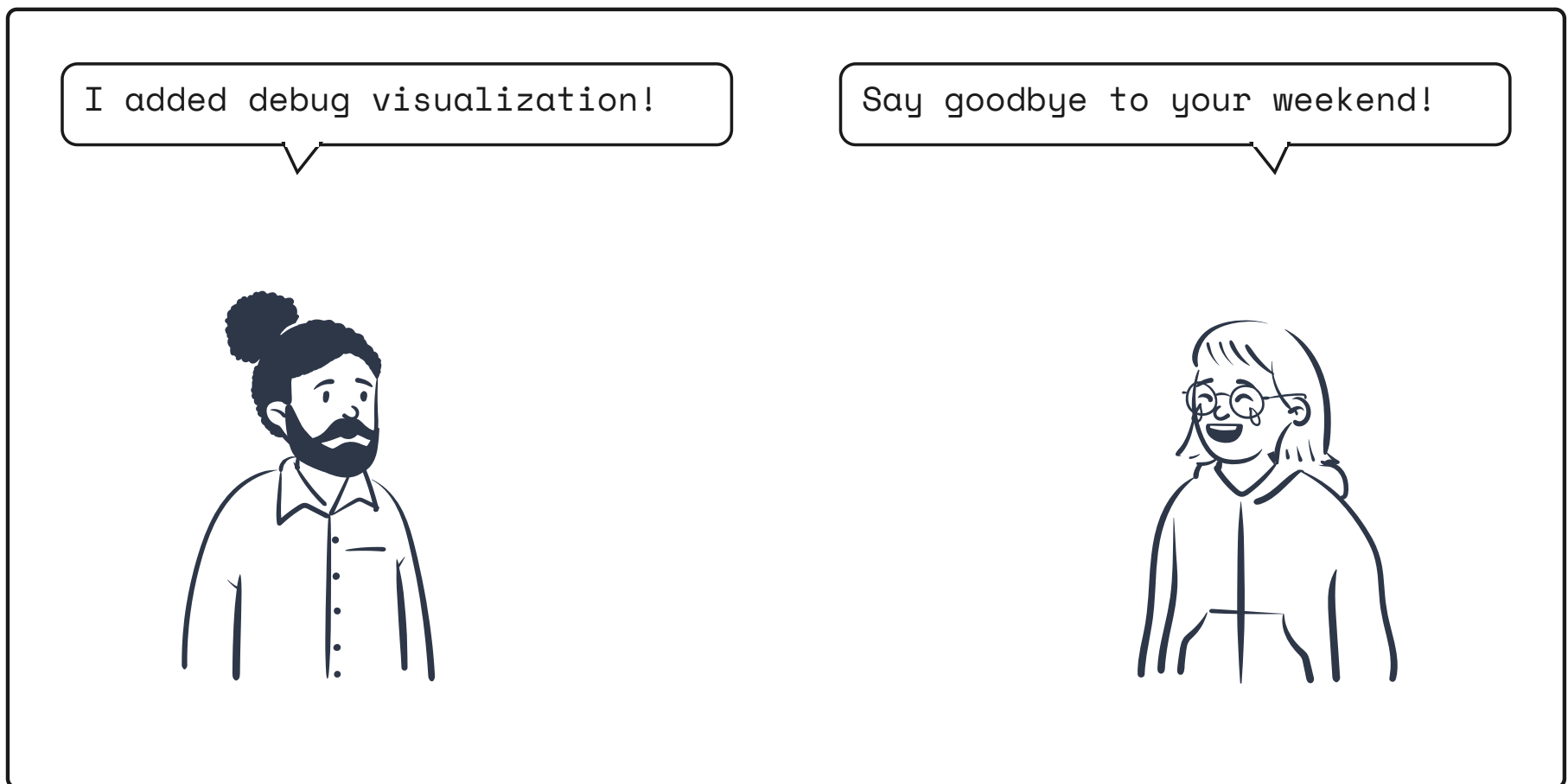
    // Draw actual collider circle
    gizmos.circle_2d(collider_pos, collider.radius, Color::srgb(0.0, 1.0, 1.0));

    // Draw current grid cell outline
    if map.in_bounds(grid.x, grid.y) {
        let cell_center = map.grid_to_world(grid.x, grid.y);
        gizmos.rect_2d(
            cell_center,
            Vec2::splat(map.tile_size()),
            Color::srgb(1.0, 1.0, 0.0),
        );
    }

    // Draw red X if on unwalkable tile
    if !map.is_walkable(grid.x, grid.y) {
        let offset = 15.0;
        gizmos.line_2d(
            collider_pos + Vec2::new(-offset, -offset),
            collider_pos + Vec2::new(offset, offset),
            Color::srgb(1.0, 0.0, 0.0),
        );
        gizmos.line_2d(
            collider_pos + Vec2::new(-offset, offset),
            collider_pos + Vec2::new(offset, -offset),
            Color::srgb(1.0, 0.0, 0.0),
        );
    }
}
```

This draws:

- **Cyan circle** - the player's collision radius
- **Yellow rectangle** - which grid cell the player is in
- **Red X** - appears if the player is somehow on an unwalkable tile



Wiring It Together: The Collision Module

Now we tie everything together in `src/collision/mod.rs`. This file declares our submodules, re-exports the public types, and defines the `CollisionPlugin`.

```
// src/collision/mod.rs
mod tile_type;
mod map;
mod systems;

#[cfg(debug_assertions)]
mod debug;

use bevy::prelude::*;
use crate::state::GameState;

// Re-export commonly used types
pub use tile_type::{TileType, TileMarker};
pub use map::CollisionMap;
pub use systems::CollisionMapBuilt;

#[cfg(debug_assertions)]
pub use debug::DebugCollisionEnabled;
```

The `pub use` lines make these types available to other modules. Instead of writing `collision::tile_type::TileType`, other code can simply use `collision::TileType`.

Now the plugin that registers everything with Bevy:

```
// Append to src/collision/mod.rs

/// Plugin for collision detection functionality
pub struct CollisionPlugin;

impl Plugin for CollisionPlugin {
    fn build(&self, app: &mut App) {
        app.init_resource::()
            .add_systems(
                Update,
                systems::build_collision_map
            );
    }
}
```

```

        .run_if(resource_equals(CollisionMapBuilt(false)))
        .run_if(in_state(GameState::Playing)),
    );

    // Debug systems - only in debug builds
    #[cfg(debug_assertions)]
    {
        app.init_resource::<DebugCollisionEnabled>()
            .add_systems(
                Update,
                (
                    debug::toggle_debug_collision,
                    debug::debug_draw_collision,
                    debug::debug_player_position,
                )
                .run_if(in_state(GameState::Playing)),
            );
    }
}

```

Notice the two `run_if` conditions on `build_collision_map`:

- `resource_equals(CollisionMapBuilt(false))` - Only runs when the map hasn't been built yet
- `in_state(GameState::Playing)` - Only runs during gameplay, not menus or loading

The debug systems are wrapped in `#[cfg(debug_assertions)]`, so they don't exist in release builds at all.

Also add the collision module to your `main.rs`:

```

// In main.rs
mod collision; // Line update alert

// In your app setup:
.add_plugins(ProcGenSimplePlugin::<Cartesian3D, Sprite>::default())
.add_plugins(state::StatePlugin)
.add_plugins(collision::CollisionPlugin) // Line update alert
.add_plugins(characters::CharactersPlugin)
.add_systems(Startup, (setup_camera, setup_generator))
.run();

```

Integrating Collision with Map Assets

Now we need to modify our map generation to attach `TileMarker` components to spawned tiles. This connects the visual tiles to the collision system.

First, update `src/map/assets.rs` to support tile types:

```

// src/map/assets.rs - Updated imports
use bevy::prelude::*;
use bevy_procedural_tilemaps::prelude::*;

use crate::collision::{TileMarker, TileType}; // Line update alert
use crate::map::tilemap::TILEMAP;

```

Update the `SpawnableAsset` struct to include tile type. Note: We are replacing `components_spawner` we had earlier with `tile_type`, so this include multiple changes in struct, function arguments, etc.

```

// src/map/assets.rs - Updated SpawnableAsset struct
#[derive(Clone)]
pub struct SpawnableAsset {
    /// Name of the sprite inside our tilemap atlas
    sprite_name: &'static str,
    /// Offset in grid coordinates (for multi-tile objects)
    grid_offset: GridDelta,
    /// Offset in world coordinates (fine positioning)
    offset: Vec3,
    /// The tile type for collision detection
    tile_type: Option<TileType>, // Line update alert
}

impl SpawnableAsset {
    pub fn new(sprite_name: &'static str) -> Self {
        Self {
            sprite_name,
            grid_offset: GridDelta::new(0, 0, 0),
            offset: Vec3::ZERO,
            tile_type: None, // Line update alert
        }
    }

    /// Set grid offset for multi-tile objects.
    pub fn with_grid_offset(mut self, offset: GridDelta) -> Self {
        self.grid_offset = offset;
        self
    }

    /// Set tile type for collision detection.
    // Function added alert
    pub fn with_tile_type(mut self, tile_type: TileType) -> Self {
        self.tile_type = Some(tile_type);
        self
    }
}

```

The builder pattern lets us chain methods:

```
SpawnableAsset::new("grass").with_tile_type(TileType::Grass).
```

Update the `load_assets` function to use a spawner:

```

// src/map/assets.rs - Updated load_assets function
pub fn load_assets(
    tilemap_handles: &TilemapHandles,
    assets_definitions: Vec<Vec<SpawnableAsset>>,
) -> ModelsAssets<Sprite> {
    let mut models_assets = ModelsAssets::<Sprite>::new();

    for (model_index, assets) in assets_definitions.into_iter().enumerate() {
        for asset_def in assets {
            let SpawnableAsset {
                sprite_name,
                grid_offset,
                offset,
                tile_type, // Line update alert
            } = asset_def;

            let Some(atlas_index) = TILEMAP.sprite_index(sprite_name) else {
                panic!("Unknown atlas sprite '{}'", sprite_name);
            };

            // Create the spawner function that adds components

```

```

        let spawner = create_spawner(tile_type); // Line update alert

        models_assets.add(
            model_index,
            ModelAsset {
                assets_bundle: tilemap_handles.sprite(atlas_index),
                grid_offset,
                world_offset: offset,
                spawn_commands: spawner, // Line update alert
            },
        );
    }
}
models_assets
}

```

Now add the `create_spawner` function that generates the right component insertion based on tile type:

```

// src/map/assets.rs - Add create_spawner function

fn create_spawner(
    tile_type: Option<TileType>,
) -> fn(&mut EntityCommands) {
    match tile_type {
        // Tile types without pickable
        Some(TileType::Dirt) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Dirt));
        },
        Some(TileType::Grass) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Grass));
        },
        Some(TileType::YellowGrass) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::YellowGrass));
        },
        Some(TileType::Water) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Water));
        },
        Some(TileType::Shore) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Shore));
        },
        Some(TileType::Tree) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Tree));
        },
        Some(TileType::Rock) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Rock));
        },
        Some(TileType::Empty) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Empty));
        },
        // Default: no components
        _ => |_: &mut EntityCommands| {},
    }
}

```

Updating Map Rules with Tile Types

Now update `src/map/rules.rs` to specify tile types for each spawnable asset. Add the import at the top:

```

// src/map/rules.rs - Updated imports
use crate::collision::TileType; //Line update alert

```



```

use crate::map::assets::SpawnableAsset;
use crate::map::models::TerrainModelBuilder;
use crate::map::sockets::*;
use bevy_procedural_tilemaps::prelude::*;

```

Now update each layer function. Here's the dirt layer:

```

// src/map/rules.rs - Updated build_dirt_layer
fn build_dirt_layer(
    terrain_model_builder: &mut TerrainModelBuilder,
    terrain_sockets: &TerrainSockets,
    socket_collection: &mut SocketCollection,
) {
    terrain_model_builder
        .create_model(
            SocketsCartesian3D::Simple {
                x_pos: terrain_sockets.dirt.material,
                x_neg: terrain_sockets.dirt.material,
                z_pos: terrain_sockets.dirt.layer_up,
                z_neg: terrain_sockets.dirt.layer_down,
                y_pos: terrain_sockets.dirt.material,
                y_neg: terrain_sockets.dirt.material,
            },
            //Line update alert
            vec![SpawnableAsset::new("dirt").with_tile_type(TileType::Dirt)],
        )
        .with_weight(20.);

    socket_collection.add_connections(vec![(
        terrain_sockets.dirt.material,
        vec![terrain_sockets.dirt.material],
    )]);
}

```

The grass layer (showing the main tile and one corner as example), you will need to repeat this to all other grass corner and side tiles.

```

// src/map/rules.rs - Updated build_grass_layer (partial)
// Main grass tile
terrain_model_builder
    .create_model(
        SocketsCartesian3D::Multiple {
            x_pos: vec![terrain_sockets.grass.material],
            x_neg: vec![terrain_sockets.grass.material],
            z_pos: vec![
                terrain_sockets.grass.layer_up,
                terrain_sockets.grass.grass_fill_up,
            ],
            z_neg: vec![terrain_sockets.grass.layer_down],
            y_pos: vec![terrain_sockets.grass.material],
            y_neg: vec![terrain_sockets.grass.material],
        },
        //Line update alert
        vec![SpawnableAsset::new("green_grass").with_tile_type(TileType::Grass)],
    )
    .with_weight(5.);

// Outer corners
terrain_model_builder.create_model(
    green_grass_corner_out.clone(),
    vec![SpawnableAsset::new("green_grass_corner_out_tl").with_tile_type(TileType::Grass)],
)

```



```
);  
// ... repeat for all corner and side variants with .with_tile_type(TileType::Grass)
```

Ensure to repeat for all corner and side variants with `.with_tile_type(TileType::Grass)`, if you are confused check the github repo for the full code.

The yellow grass layer follows the same pattern:

```
// Main yellow grass tile  
vec![SpawnableAsset::new("yellow_grass").with_tile_type(TileType::YellowGrass)]  
  
// All yellow grass corners and sides  
vec![SpawnableAsset::new("yellow_grass_corner_out_tl").with_tile_type(TileType::YellowGrass)]  
// ... etc
```

Ensure to repeat for all corner and side variants with `.with_tile_type(TileType::YellowGrass)`.

The water layer:

```
// Main water tile  
vec![SpawnableAsset::new("water").with_tile_type(TileType::Water)]  
  
// All water corners and sides  
vec![SpawnableAsset::new("water_corner_out_tl").with_tile_type(TileType::Water)]  
// ... etc
```

Ensure to repeat for all corner and side variants with `.with_tile_type(TileType::Water)`.

The props layer has mixed tile types. Trees and rocks block movement, plants are walkable:

```
// src/map/rules.rs - Updated build_props_layer (key examples)  
  
// Small tree - bottom blocks, top is walkable (canopy)  
terrain_model_builder.create_model(  
    plant_prop.clone(),  
    vec![  
        // Line update alert  
        SpawnableAsset::new("small_tree_bottom").with_tile_type(TileType::Tree),  
        SpawnableAsset::new("small_tree_top").with_grid_offset(GridDelta::new(0, 1, 0)),  
    ],  
);  
  
// Big tree 1 - left side (base blocks, canopy walkable)  
vec![  
    SpawnableAsset::new("big_tree_1_bl").with_tile_type(TileType::Tree),  
    SpawnableAsset::new("big_tree_1_tl").with_grid_offset(GridDelta::new(0, 1, 0)),  
]  
  
// Big tree 1 - right side  
vec![  
    SpawnableAsset::new("big_tree_1_br").with_tile_type(TileType::Tree),  
    SpawnableAsset::new("big_tree_1_tr").with_grid_offset(GridDelta::new(0, 1, 0)),  
]  
  
// Big tree 2 - left side  
vec![  
    SpawnableAsset::new("big_tree_2_bl").with_tile_type(TileType::Tree),  
    SpawnableAsset::new("big_tree_2_tl").with_grid_offset(GridDelta::new(0, 1, 0)),  
]  
  
// Big tree 2 - right side  
vec![
```

```

    SpawnableAsset::new("big_tree_2_br").with_tile_type(TileType::Tree),
    SpawnableAsset::new("big_tree_2_tr").with_grid_offset(GridDelta::new(0, 1, 0)),
]

// Tree stumps block movement
vec![SpawnableAsset::new("tree_stump_1").with_tile_type(TileType::Tree)]
vec![SpawnableAsset::new("tree_stump_2").with_tile_type(TileType::Tree)]
vec![SpawnableAsset::new("tree_stump_3").with_tile_type(TileType::Tree)]

// Rocks block movement
// Lines updated
vec![SpawnableAsset::new("rock_1").with_tile_type(TileType::Rock)]
vec![SpawnableAsset::new("rock_2").with_tile_type(TileType::Rock)]
vec![SpawnableAsset::new("rock_3").with_tile_type(TileType::Rock)]
vec![SpawnableAsset::new("rock_4").with_tile_type(TileType::Rock)]

// Plants are walkable (grass tile type)
vec![SpawnableAsset::new("plant_1").with_tile_type(TileType::Grass)]
vec![SpawnableAsset::new("plant_2").with_tile_type(TileType::Grass)]
vec![SpawnableAsset::new("plant_3").with_tile_type(TileType::Grass)]
vec![SpawnableAsset::new("plant_4").with_tile_type(TileType::Grass)]

```

Notice tree canopies (the “top” parts) don’t have a tile type. This is intentional: the player can walk under tree canopies but collides with the trunk.

Centralizing Configuration

Before we create the player’s collider, let’s centralize our magic numbers. Scattered constants make tuning gameplay tedious. Create a single configuration file:

Create `src/config.rs`:

```

// src/config.rs
//! Centralized configuration constants for the game.

/// Player-related configuration
pub mod player {
    /// Collision radius for the player's collider (in world units)
    pub const COLLIDER_RADIUS: f32 = 16.0;

    /// Z-position for player rendering (above terrain, below UI)
    pub const PLAYER_Z_POSITION: f32 = 20.0;

    /// Visual scale of the player sprite
    pub const PLAYER_SCALE: f32 = 0.8;
}

/// Map/terrain configuration
pub mod map {
    /// Size of a single tile in world units
    pub const TILE_SIZE: f32 = 32.0;

    /// Grid dimensions
    pub const GRID_X: u32 = 25;
    pub const GRID_Y: u32 = 18;
}

```

These constants define our collision radius (16 pixels) and map dimensions. Having them in one place means you can tweak collision behavior without hunting through multiple files.

All my magic numbers in one file!



Now you can break everything from one place!



Add this module to `src/main.rs`:

```
// src/main.rs
mod config; // Add this line
```

Creating the Player Collider

Now we need a component that represents the player's collision shape. We'll use a circle positioned at the player's center.

Create `src/characters/collider.rs`:

```
// src/characters/collider.rs
use bevy::prelude::*;

use crate::collision::CollisionMap;
use crate::characters::physics::Velocity;
use crate::config::player::{COLLIDER_RADIUS};

/// A circular collider for collision detection.
///
/// The collider position is offset from the entity's transform,
#[derive(Component, Debug, Clone)]
pub struct Collider {
    /// Radius of the circular collider in world units
    pub radius: f32,
    /// Offset from entity center
    pub offset: Vec2,
}

impl Default for Collider {
    fn default() -> Self {
        Self {
            radius: COLLIDER_RADIUS,
            offset: Vec2::ZERO,
        }
    }
}
```

```
impl Collider {
    /// Get the world position of this collider given an entity's transform.
    pub fn world_position(&self, transform: &Transform) -> Vec2 {
        transform.translation.truncate() + self.offset
    }
}
```

Validating Movement Against Collision

Now the crucial system: checking if the player can actually move where they want to go.

```
// Append to src/characters/collider.rs

/// System that validates movement against the collision map.
///
/// Runs after input (which sets velocity) but before physics (which applies velocity).
/// Modifies velocity to prevent movement into unwalkable tiles.
pub fn validate_movement(
    map: Option<Res<CollisionMap>>,
    time: Res<Time>,
    mut query: Query<(&Transform, &mut Velocity, &Collider)>,
) {
    let Some(map) = map else { return };

    for (transform, mut velocity, collider) in query.iter_mut() {
        // Skip if not moving
        if !velocity.is_moving() {
            continue;
        }

        // Current collider position
        let current_pos = collider.world_position(transform);

        // Desired new position based on velocity
        let delta = velocity.0 * time.delta_secs();
        let desired_pos = current_pos + delta;

        // Use swept collision to find valid position
        let valid_pos = map.sweep_circle(current_pos, desired_pos, collider.radius);

        // Calculate what velocity would get us to valid_pos
        let actual_delta = valid_pos - current_pos;

        // Only update velocity if collision modified our path
        if (actual_delta - delta).length_squared() > 0.001 {
            // Convert position delta back to velocity
            let dt = time.delta_secs();
            if dt > 0.0 {
                velocity.0 = actual_delta / dt;
            }
        }
    }
}
```

How this works:

1. Get the current collider position
2. Calculate where the player *wants* to go based on their velocity
3. Use `sweep_circle` to find the furthest valid position along that path
4. If collision blocked part of the movement, adjust velocity accordingly

The key insight: we don't block movement entirely. If the player tries to walk diagonally into a wall, `sweep_circle` might allow sliding along the wall's surface. This feels much better than getting stuck.

Updating the Characters Module

Add the collider module to `src/characters/mod.rs`:

```
// src/characters/mod.rs - Add at the top
pub mod collider; // Add this line
```

Now update the system registration. We need `validate_movement` to run **after** input sets velocity but **before** physics applies it:

```
// src/characters/mod.rs - Update the system chain
.add_systems(Update, (
    // 1. Input determines state + velocity + facing
    input::handle_player_input,
    spawn::switch_character,
    input::update_jump_state,

    // 2. State changes trigger animation updates
    animation::on_state_change_update_animation,

    // 3. Collision validation adjusts velocity <-- Line update alert!
    collider::validate_movement,

    // 4. Physics applies velocity to transform
    physics::apply_velocity,

    // 5. Animation ticks frames
    animation::tick_animations,
).chain().run_if(in_state(GameState::Playing)));
```

Attaching Collider to the Player

Finally, update `src/characters/spawn.rs` to attach the `Collider` component when initializing the player.

Update the imports:

```
// src/characters/spawn.rs - Add import
use crate::characters::collider::Collider; // Line update alert!
use crate::config::player::{PLAYER_SCALE, PLAYER_Z_POSITION}; // Line update alert!
```

Also remove these existing constants since we are importing the constants from the config.

```
// Delete these lines
// const PLAYER_SCALE: f32 = 0.8;
// const PLAYER_Z_POSITION: f32 = 20.0;
```

Then in `initialize_player_character`, add `Collider::default()` to the component bundle:

```
// src/characters/spawn.rs - In initialize_player_character
commands.entity(entity).insert((
    AnimationController::default(),
    CharacterState::default(),
    Velocity::default(),
```

```
Facing::default(),
Collider::default(), // Line update alert!
AnimationTimer(Timer::from_seconds(DEFAULT_ANIMATION_FRAME_TIME, TimerMode::Repeating)),
character_entry.clone(),
sprite,
));
```

Now run the game:

```
cargo run
```

Walk your character around. You should now collide with trees, rocks, and water! The character slides smoothly along obstacles rather than getting stuck. Press F3 to toggle the debug overlay and see the collision map visualized.

The Illusion of Depth

There's one more problem. Walk your character behind a tree. Notice anything odd?

The player always renders on top. They don't disappear behind the tree trunk. In a top-down game, objects higher on the screen (further away from you) should appear "behind" objects lower on the screen (closer to you). When your character walks upward and passes behind a tree, they should disappear behind it.

This is **Y-based depth sorting**. The concept is simple: objects with a lower Y position (closer to the bottom of the screen) are "in front" of objects with a higher Y position.

In 2D rendering, Z determines draw order. Higher Z draws on top. So we need:

- **Higher Y** (top of screen) → **Lower Z** (drawn first, appears behind)
- **Lower Y** (bottom of screen) → **Higher Z** (drawn last, appears in front)

Our tilemap generator already does this for tiles using `with_z_offset_from_y(true)`. But the player's Z position is fixed! We need to update the player's Z dynamically based on their Y position.

Creating the Rendering Module

Create `src/characters/rendering.rs`:

```
// src/characters/rendering.rs
//! Rendering utilities for depth sorting.

use bevy::prelude::*;

use crate::characters::input::Player;
use crate::config::map::{GRID_Y, TILE_SIZE};
use crate::config::player::PLAYER_SCALE;

/// Z-depth constants for proper layering.
/// The tilemap uses `with_z_offset_from_y(true)` which assigns Z based on Y position.
/// We need to match this formula for the player.
const NODE_SIZE_Z: f32 = 1.0; // Same as tilemap generator
const PLAYER_BASE_Z: f32 = 4.0; // Match props layer Z range
const PLAYER_Z_OFFSET: f32 = 0.5; // Small offset to stay above ground props
```

These constants need to match how the tilemap generator calculates Z. `PLAYER_BASE_Z` positions the player in the same Z range as props (trees, rocks). `PLAYER_Z_OFFSET` adds a tiny buffer so the player doesn't z-fight with ground-level decorations.

Now the depth sorting system. The key is to dynamically adjust the player's Z position based on their Y position. When the player moves up the screen (higher Y), we lower their Z so they render behind objects. When they move down (lower Y), we raise their Z so they render in front.

But there's a subtlety: we use the player's **feet position**, not their sprite center, for depth calculation. Why? Imagine your character standing just in front of a tree. The sprite's center is at chest level, but visually, what matters is where their feet are. If we used the center, the player's head might incorrectly poke out above a tree they're standing in front of. Using feet position makes occlusion look natural.

```
// Append to src/rendering.rs

/// System to update player depth based on Y position.
///
/// Objects with lower Y (further down screen) get higher Z (rendered in front).
/// This creates proper occlusion when walking behind trees.
pub fn update_player_depth(
    mut player_query: Query<&mut Transform, (With<Player>, Changed<Transform>)>,
) {
    // Map dimensions for normalization
    let map_height = TILE_SIZE * GRID_Y as f32;
    let map_y0 = -TILE_SIZE * GRID_Y as f32 / 2.0; // Map origin Y (centered)

    // Player sprite height for feet position calculation
    let player_sprite_height = 64.0 * PLAYER_SCALE;

    for mut transform in player_query.iter_mut() {
        let player_center_y = transform.translation.y;

        // Use player's FEET position for depth sorting (not center)
        let player_feet_y = player_center_y - (player_sprite_height / 2.0);

        // Normalize feet Y to [0, 1] across the grid height
        let t = ((player_feet_y - map_y0) / map_height).clamp(0.0, 1.0);

        // Y-to-Z formula:
        // Lower Y (bottom of screen) = higher t = lower Z offset = rendered in front
        // Higher Y (top of screen) = lower t = higher Z offset = rendered behind
        let player_z = PLAYER_BASE_Z + NODE_SIZE_Z * (1.0 - t) + PLAYER_Z_OFFSET;

        transform.translation.z = player_z;
    }
}
```

What's `Changed<Transform>`?

This is a Bevy query filter that only matches entities whose `Transform` was modified this frame. It's a performance optimization: we only recalculate Z when the player actually moves. If they're standing still, the system skips them entirely.

Integrating Depth Sorting

Now, you can add it to the character system chain in `src/characters/mod.rs` after `physics::apply_velocity`:

```
// src/characters/mod.rs

mod rendering; // Line update alert!

.add_systems(Update, (
    // ... existing systems ...
    physics::apply_velocity,
```



```
rendering::update_player_depth, // Add after physics , line update alert!  
animation::tick_animations,  
).chain().run_if(in_state(GameState::Playing)));
```

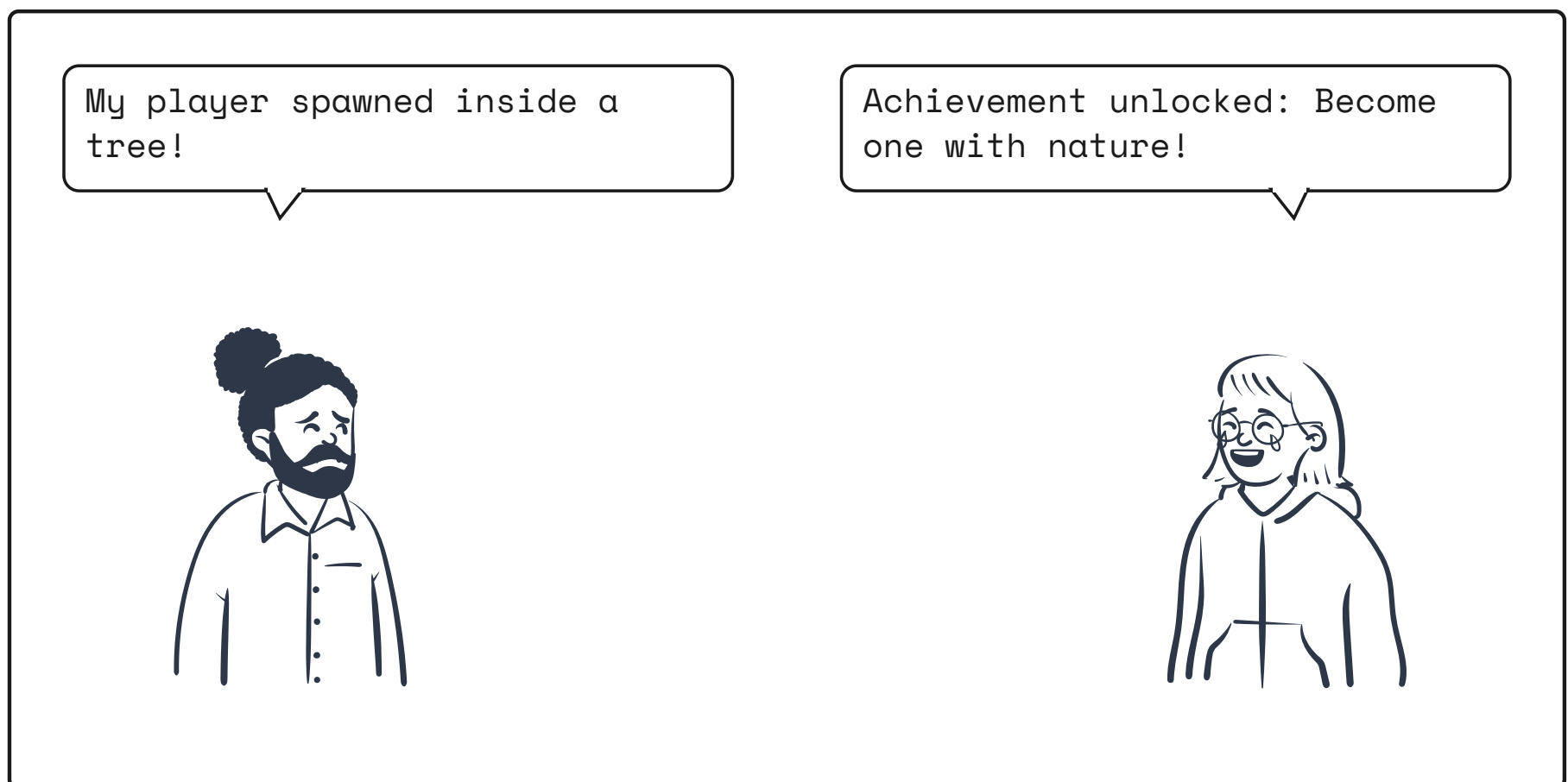
Adjusting Water Generation

Before running the game, make one small tweak to prevent the player from spawning in water. Open `src/map/rules.rs` and find the water layer weight:

```
// src/map/rules.rs - In build_water_layer function  
const WATER_WEIGHT: f32 = 0.001; // Reduce from 0.02 to 0.001
```

This reduces water generation, making it less likely the player spawns in an inaccessible area.

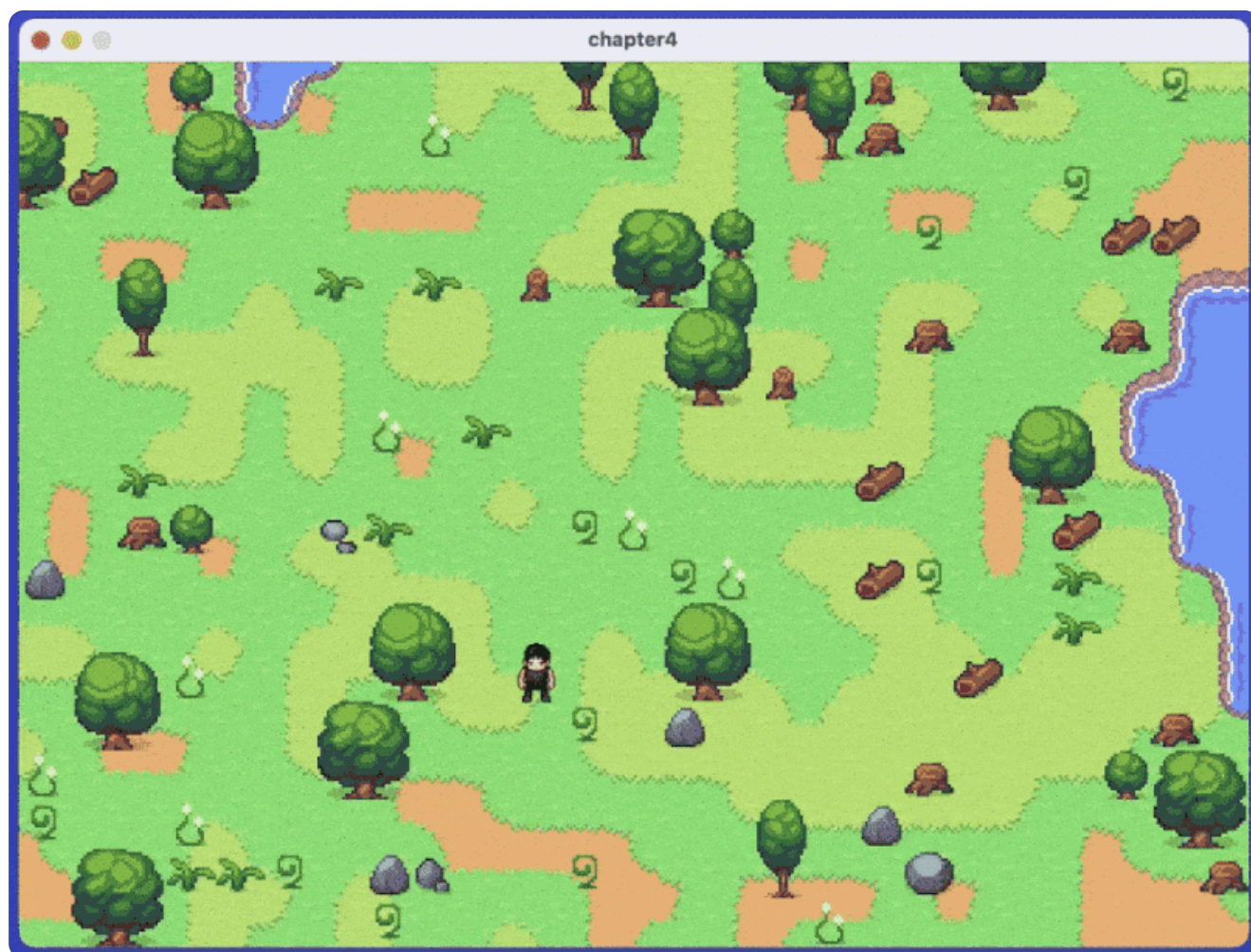
Spawn Troubleshooting: There's a small chance the procedural generation places the player on top of a blocking object (tree, rock) at spawn. If you can't move when the game starts, simply restart to generate a new map. This is a quirk of random generation we'll address in future chapters.



Run the game again:

```
cargo run
```

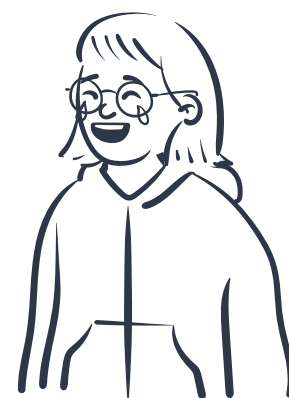
Now walk behind a tree. Your character should disappear behind the trunk! Walk back down, and they reappear. The illusion of depth makes the world feel three-dimensional even though it's all 2D sprites.



My 2D game has depth now!



That's what every indie dev says on Twitter!



What We Built

Let's take a step back and appreciate what we've accomplished in this chapter:

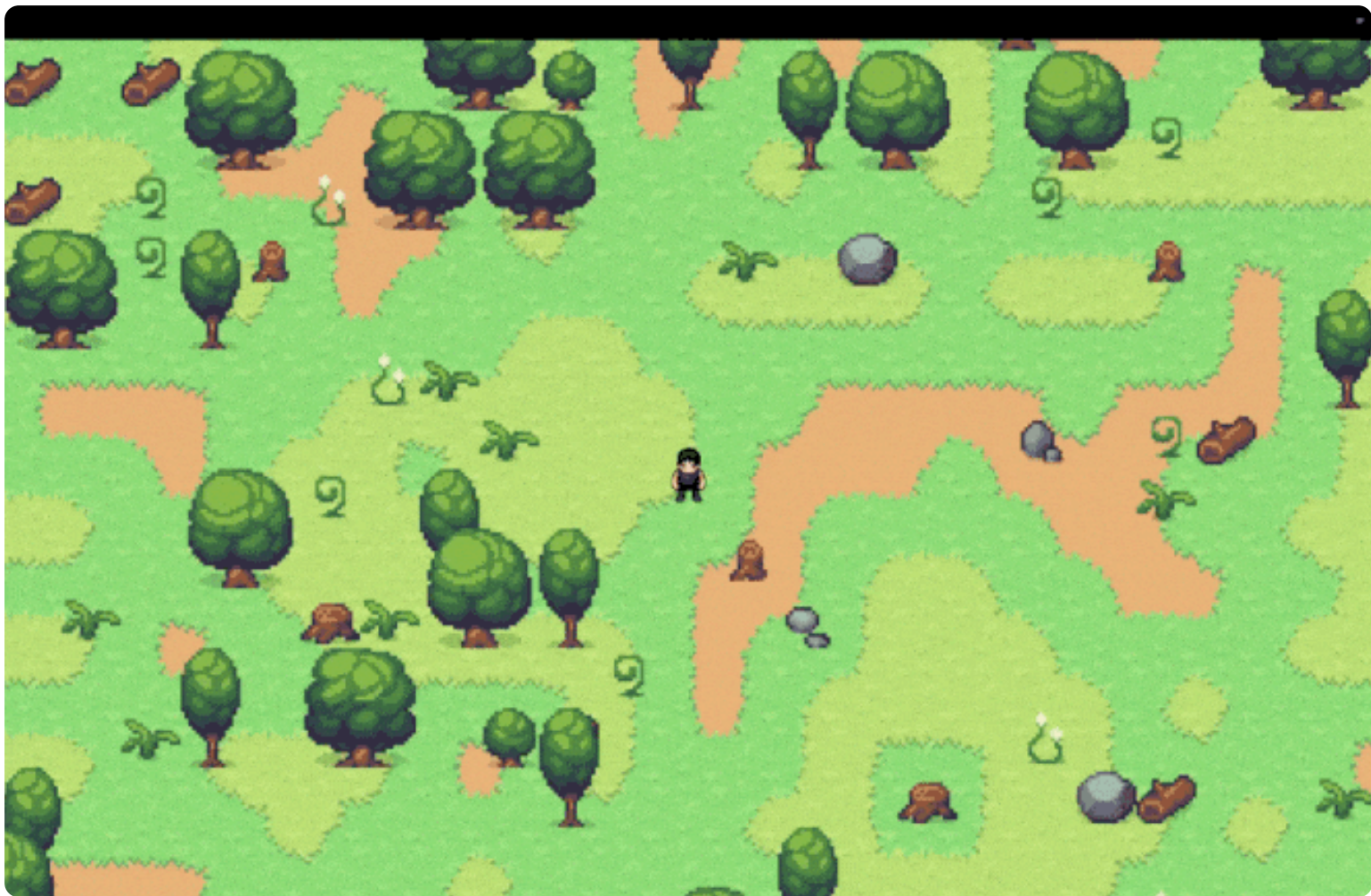
1. **Game States** - Proper lifecycle management with Loading, Playing, and Paused states. No more polling patterns.
2. **Character States** - An enum-based state machine that prevents impossible states and simplifies animation logic.
3. **Collision System** - A tile-based collision map with circle colliders, swept collision for smooth wall-sliding, and shore detection for water edges.
4. **Debug Visualization** - Press F3 to see the collision map, player collider, and current grid cell.
5. **Depth Sorting** - Y-based rendering that creates the illusion of walking behind objects.

Your character can now walk through a procedurally generated world, colliding realistically with obstacles, sliding around corners, and disappearing behind trees. That's a lot of systems working together!

In the next chapter, we'll add interactivity: picking up items and building an inventory system. Until then, experiment with the collision parameters, try different collider sizes, and see how the debug visualization helps you understand what's happening under the hood.

The Impatient Programmer's Guide to Bevy and Rust: Chapter 5 - Let There Be Pickups

By the end of this chapter, you'll have an inventory system that lets your player collect items from the world, and a smooth camera that follows them around. Walk near a plant or mushroom, and it disappears into your inventory. Move across the map, and the camera keeps you centered with buttery-smooth motion.



Building the Pickup System

In Chapter 4, we made solid objects block the player. Now let's do the opposite: make items the player can walk through and collect.

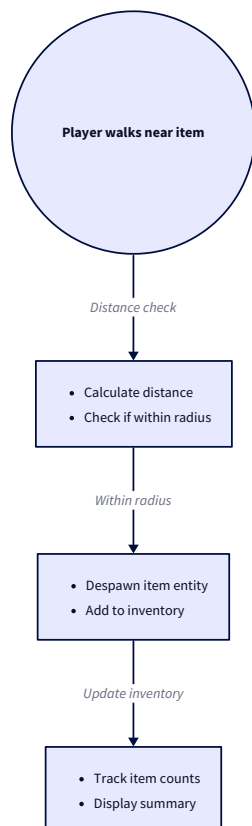
The Pickup System

Think about how pickups work in games you've played:

1. **Detection:** "Is the player close enough to this item?"
2. **Collection:** "Remove the item from the world"
3. **Storage:** "Remember what was collected"

We'll build this in three parts:

- A `Pickable` component to mark items as collectible
- A `handle_pickups` system to detect when the player is near items
- An `Inventory` resource to track what's been collected



Configuring Pickup Radius

Before we build the inventory, let's add configuration for pickup detection. We already have a `config.rs` file from Chapter 4 where we centralized collision settings. Let's add pickup configuration there.

Open `src/config.rs` and you'll see it's organized into modules: `player`, `map`, `debug`. Let's add a `pickup` module:

```
// src/config.rs - Add this section after the player module

/// Pickup/inventory configuration
pub mod pickup {
    /// Default radius for item pickup detection (in world units)
    pub const DEFAULT_RADIUS: f32 = 40.0;
}
```

What's a world unit?

In Bevy, positions are measured in world units. Our tiles are 32 units wide (from `map::TILE_SIZE`), so a radius of 40 units means the player can pick up items from slightly more than one tile away. Not too far, not too close.

Building the Inventory System

Create a new folder `src/inventory/` for our inventory module.

Defining Item Types

What kinds of items can the player collect? In our game, we have plants, mushrooms(or something that looks like a mushroom) scattered around the map. Each needs a unique identifier.

Create `src/inventory/inventory.rs`:

```
// src/inventory/inventory.rs
use bevy::prelude::*;
use std::collections::HashMap;
use std::fmt;
```

```

use crate::config::pickup::DEFAULT_RADIUS;

/// Types of items that can be collected.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum ItemKind {
    Plant1,
    Plant2,
    Plant3,
    Plant4,
}

```

Display Names

Players don't want to see "Plant1" in their inventory. They want "Herb" or "Flower". Let's add human-readable names:

```

// Append to src/inventory/inventory.rs

impl ItemKind {
    pub fn display_name(&self) -> &'static str {
        match self {
            ItemKind::Plant1 => "Herb",
            ItemKind::Plant2 => "Flower",
            ItemKind::Plant3 => "Mushroom",
            ItemKind::Plant4 => "Fern",
        }
    }
}

impl fmt::Display for ItemKind {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str(self.display_name())
    }
}

```

What's `fmt::Display` for?

`fmt::Display` is like a predefined contract Rust makes with your type. It's like saying, "If you implement this trait, I'll let you use `{}` in print statements."

Without it, `println!("Collected: {}", item_kind)` would fail to compile. Rust wouldn't know how to convert your `ItemKind::Plant1` into text for display. By implementing `Display`, we fulfill the contract, we tell Rust "when you need to print this, call this `fmt` method I wrote."

The Pickable Component

Now we need a component to mark entities as collectible. When we spawn a plant in the world, we'll attach this component to say "the player can pick this up."

```

// Append to src/inventory/inventory.rs

/// Component marking an entity as collectible.
#[derive(Component, Debug)]
pub struct Pickable {
    pub kind: ItemKind,
    pub radius: f32,
}

impl Pickable {
    pub fn new(kind: ItemKind) -> Self {

```

```

        Self {
            kind,
            radius: DEFAULT_RADIUS,
        }
    }
}

```

Why store `radius` per item?

Most items use the default radius (40 units), but maybe you want a treasure chest to have a smaller radius (must be right next to it) or a glowing orb to have a larger one (magnetic pull). Storing it in the component gives you flexibility.

The Inventory Resource

The inventory persists across the entire game session, it's not tied to any single entity. That's what Resources are for: global game state that any system can access.

```

// Append to src/inventory/inventory.rs

/// Resource storing collected items.
#[derive(Resource, Default, Debug)]
pub struct Inventory {
    items: HashMap<ItemKind, u32>,
}

```

Why `HashMap<ItemKind, u32>` instead of `Vec<ItemKind>`?

A `Vec` would store every individual item: `[Plant1, Plant1, Plant2, Plant1...]`. To count how many `Plant1`s you have, you'd scan the whole list. A `HashMap` stores counts directly: `{Plant1: 3, Plant2: 1}`. It also makes lookups efficient.

Adding Items to Inventory

Now let's implement the logic to add items to the inventory:

```

// Append to src/inventory/inventory.rs

impl Inventory {
    /// Add an item to the inventory, returns new count.
    pub fn add(&mut self, kind: ItemKind) -> u32 {
        let entry = self.items.entry(kind).or_insert(0);
        *entry += 1;
        *entry
    }

    /// Get a summary string of inventory contents.
    pub fn summary(&self) -> String {
        if self.items.is_empty() {
            return "empty".to_string();
        }

        let mut parts: Vec<String> = self
            .items
            .iter()
            .map(|(kind, count)| format!("{: }: {}", kind, count))
            .collect();
        parts.sort();
        parts.join(", ")
    }
}

```


What's `.entry(kind).or_insert(0)` doing?

This is HashMap's "get or create" pattern. If `kind` exists in the map, `.entry(kind)` gets a mutable reference to its value. If not, `.or_insert(0)` creates it with a count of 0. Then we increment and return the new count.

Why return the count from `add`?

So the pickup system can log "Picked up Herb (total: 3)". It's a small quality-of-life feature that makes debugging easier.

What's the `summary` method for?

It formats the inventory for display: "Herb: 3, Flower: 1, Wood: 2". We sort the items alphabetically so the order is consistent. Later, you could show this in a UI panel or debug overlay.

Pickup Detection System

Now for the system that actually detects when the player is near an item. Create `src/inventory/systems.rs`:

```
// src/inventory/systems.rs
use bevy::prelude::*;

use crate::characters::input::Player;
use super::inventory::{Pickable, Inventory};

/// System that checks for and processes item pickups.
pub fn handle_pickups(
    mut commands: Commands,
    mut inventory: ResMut<Inventory>,
    player_query: Query<&Transform, With<Player>>,
    pickables: Query<(Entity, &GlobalTransform, &Pickable)>,
) {
    let Ok(player_transform) = player_query.single() else {
        return;
    };

    let player_pos = player_transform.translation.truncate();
    let mut collected = Vec::new();

    // Check distance to each pickable
    for (entity, global_transform, pickable) in pickables.iter() {
        let item_pos = global_transform.translation().truncate();
        let distance_sq = player_pos.distance_squared(item_pos);

        if distance_sq <= pickable.radius * pickable.radius {
            collected.push((entity, pickable.kind));
        }
    }

    // Process collected items
    for (entity, kind) in collected {
        commands.entity(entity).despawn();
        let count = inventory.add(kind);
        info!(
            " Picked up {} (total: {}) - inventory: {}",
            kind, count, inventory.summary()
        );
    }
}
```

Let's break this down step by step.

Getting the Player Position

We query for the player's position as we did in the earlier chapters.

What's `.truncate()`?

`Transform.translation` is a `Vec3` (x, y, z). We're in 2D, so we only care about x and y. `.truncate()` converts `Vec3` to `Vec2`, dropping the z component.

Collecting Items to Process

Before we can despawn collected items, we need to identify which ones are within pickup range.

We create an empty `Vec` and iterate through all pickable items. For each item within range, we store its entity ID and item kind as a tuple `(entity, pickable.kind)`. This gives us a list of everything the player just picked up.

Why `distance_squared` instead of `distance`?

The actual distance formula is `sqrt((x2-x1)2 + (y2-y1)2)`. Square root is expensive. But we're just comparing distances, so we can compare the *squared* distances instead:

- `distance < radius` is the same as `distance2 < radius2`
- No square root needed, just multiplication

This is a common game dev optimization. When checking hundreds of items every frame, it adds up.

Processing Collected Items

Later we go through the collected items and do the following:

1. **Despawn the entity:** Removes it from the world (no more rendering, no more queries)
2. **Add to inventory:** Updates the count in the HashMap
3. **Log the pickup:** Prints to console for debugging

Why collect items into a `Vec` first before despawning them?

You might notice we use a two-step pattern: first collect items into a list, then process them. In our Bevy code, we could actually skip this and despawn directly in the loop, but we follow this pattern because it's a Rust best practice that makes code safer and clearer.

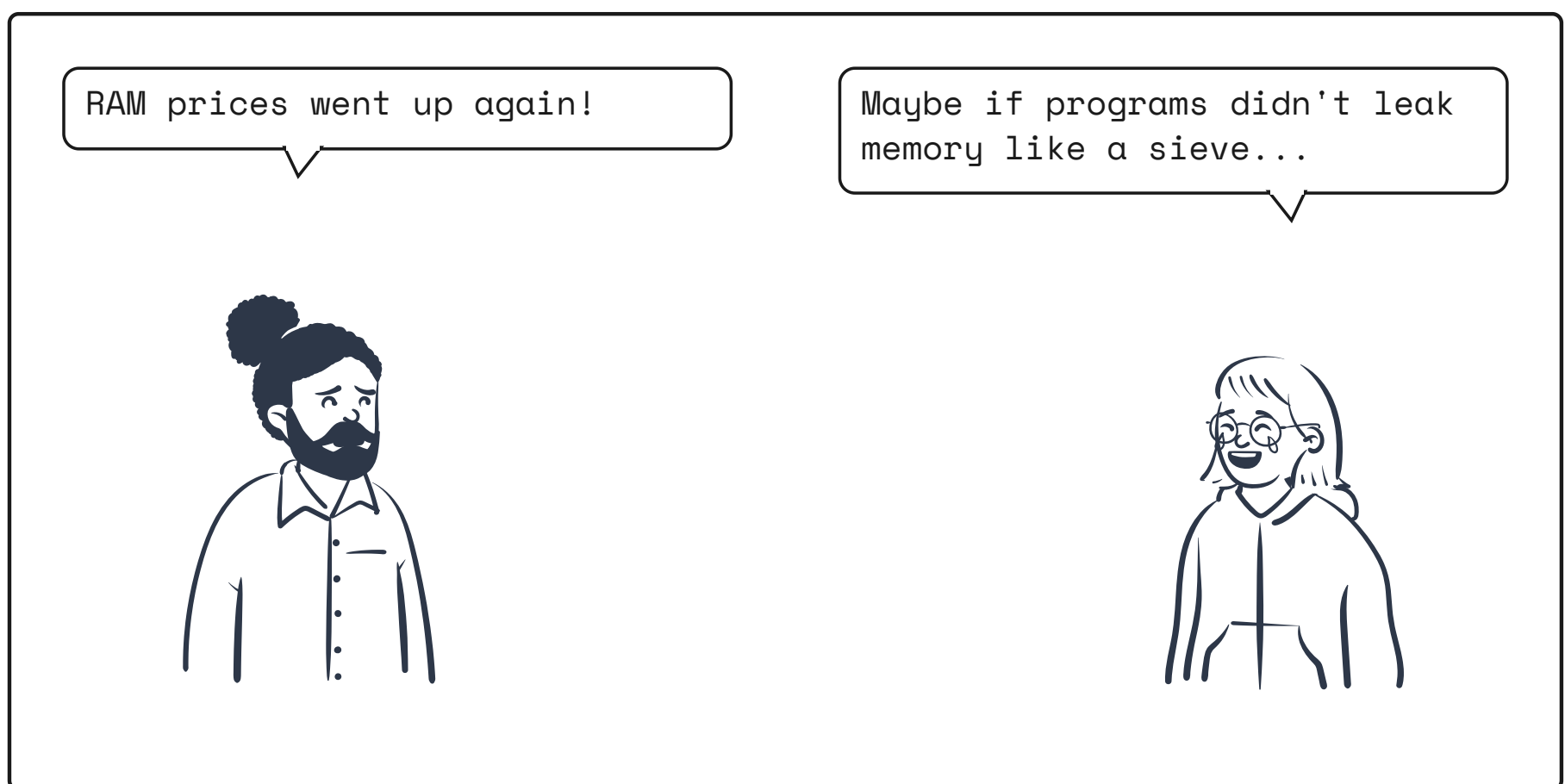
Let's learn why this pattern exists:

The Borrow Checker

Remember in Chapter 1, where we learned about how Rust gets mad when we don't put `mut` when declaring a variable we want to change? That was just the beginning. Rust has more rules about how you can access and modify data, enforced by something called the **borrow checker**.

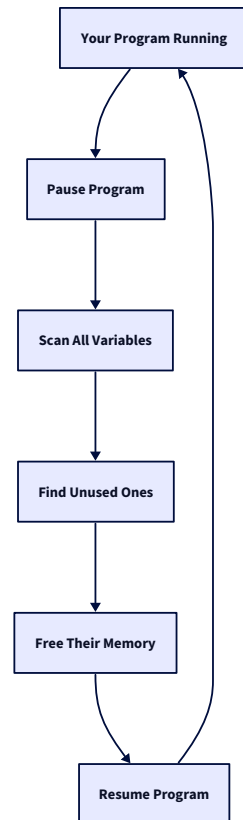
What's a borrow checker?

Let's start with a fundamental problem: when should we free memory? When you create a variable in most languages, it allocates memory. When you're done with it, that memory should be freed:



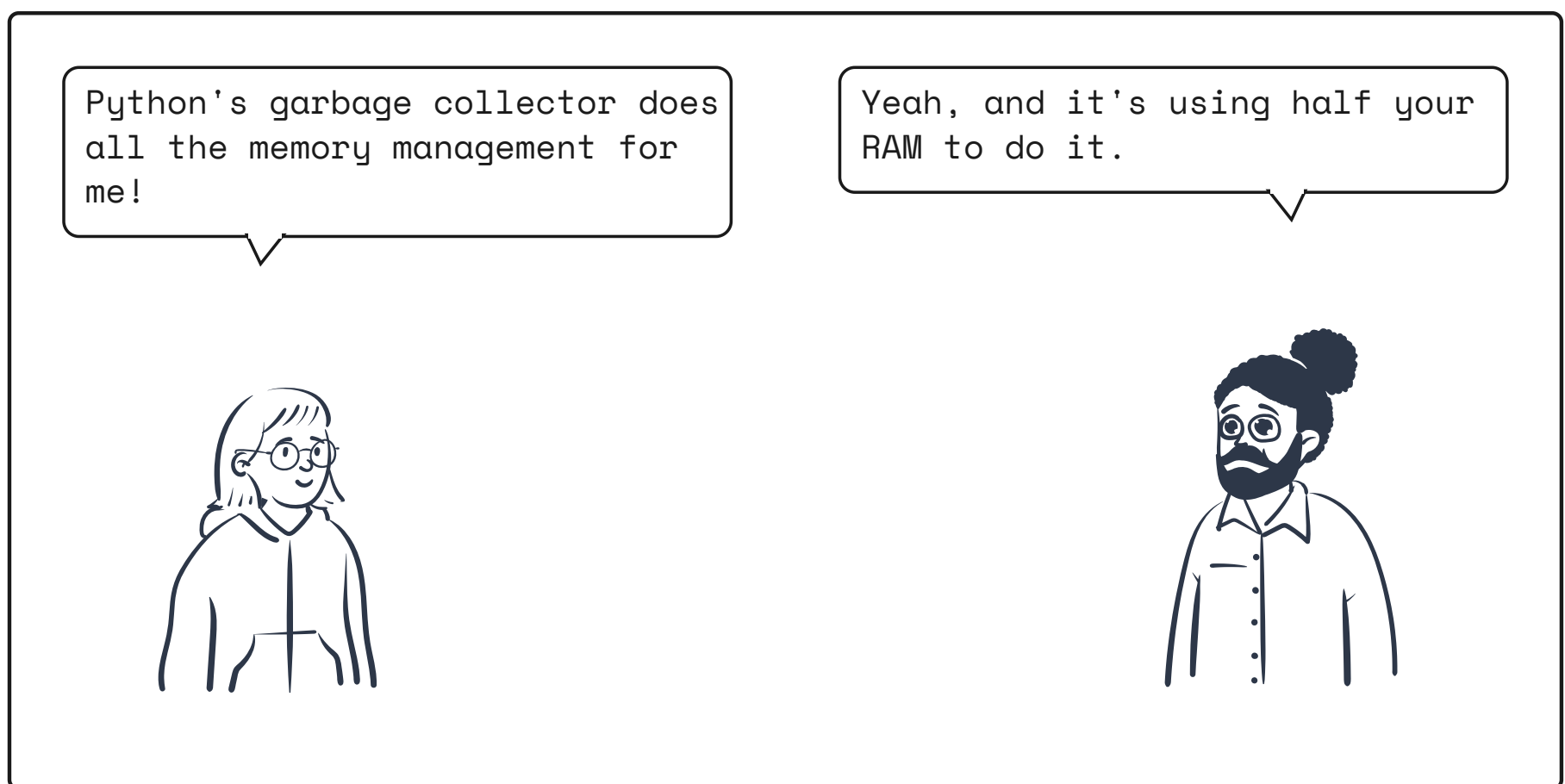
Languages like Python and JavaScript: Garbage Collection

In Python or JavaScript, you never explicitly free variables. A *garbage collector* runs periodically, checking which variables are still being used:



This is convenient, you don't think about memory. But it has costs:

- **Pause times:** The garbage collector must pause your program to scan memory
- **Overhead:** It tracks every variable at runtime, using extra memory
- **Unpredictability:** You don't know when pauses happen



Rust's Approach: Compile-Time Checks

Rust takes a different approach. It checks at compile time that your code follows strict rules. When the compiler sees these rules followed, it knows *exactly* when each variable's memory can be freed, no runtime tracking needed:

```
// Pseudo code, don't use
fn example() {
    let items = vec![1, 2, 3]; // items owns the vector

    process(items);           // ownership transferred to process()
}
```

```
    // items can't be used anymore
} // Compiler inserts cleanup here - no garbage collector!
```

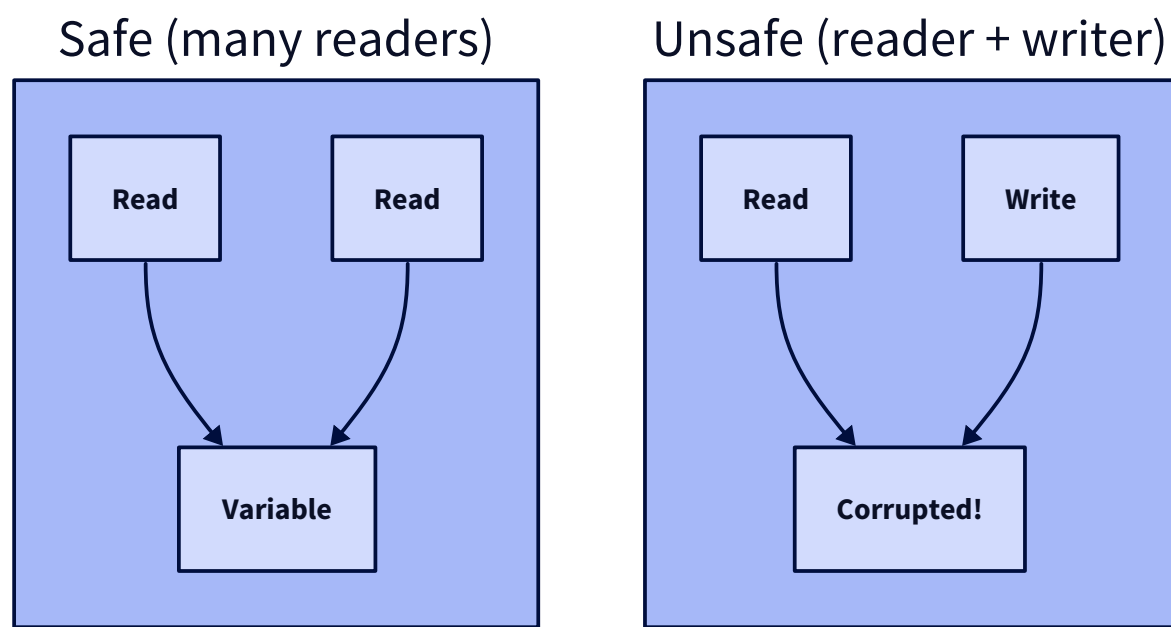
The borrow checker enforces rules that let the compiler figure out memory lifetimes. Let's learn these rules.

The Borrowing Rules

Many readers OR one writer, never both

You can have many immutable references OR one mutable reference, but not both at the same time.

Why? Imagine you're reading a map while someone is erasing and redrawing it. You might read half-old, half-new data—corruption! This rule prevents that:



Example code:

```
// Pseudo code, don't use
let mut inventory = vec!["Herb", "Flower"];

let reader1 = &inventory; // ✓ Immutable borrow
let reader2 = &inventory; // ✓ Another immutable borrow is fine
println!("{:?}", reader1); // Both can read
println!("{:?}", reader2);

// After the immutable borrows are done being used, we can create a mutable borrow:
let writer = &mut inventory; // ✓ Now this works
writer.push("Mushroom");
```

The key insight: borrows end when they're last used, not when they go out of scope. Once `reader1` and `reader2` are done (after the `println!` calls), the mutable borrow is allowed.

This won't compile:

```
// Pseudo code, don't use
let mut inventory = vec!["Herb", "Flower"];

let reader = &inventory;
let writer = &mut inventory; // ✗ ERROR: Can't borrow as mutable while immutably borrowed
```

```
println!("{:?}", reader); // reader still used here
```

References must always be valid (no dangling pointers)

A reference can't outlive the data it points to.

Why? In C++, you can create a pointer to memory that gets freed, leaving a “dangling pointer.” Reading it accesses random memory—crash or worse, subtle corruption.

Here's a C++ example that compiles but crashes at runtime:

```
int* create_dangling_pointer() {
    int x = 5;           // x is allocated on the stack
    int* ptr = &x;       // ptr points to x
    return ptr;          // Return pointer to x
} // x goes out of scope here - memory freed!

int main() {
    int* ptr = create_dangling_pointer();

    // ptr now points to freed memory (dangling pointer)
    std::cout << *ptr;   // CRASH! Reading freed memory

    return 0;
}
```

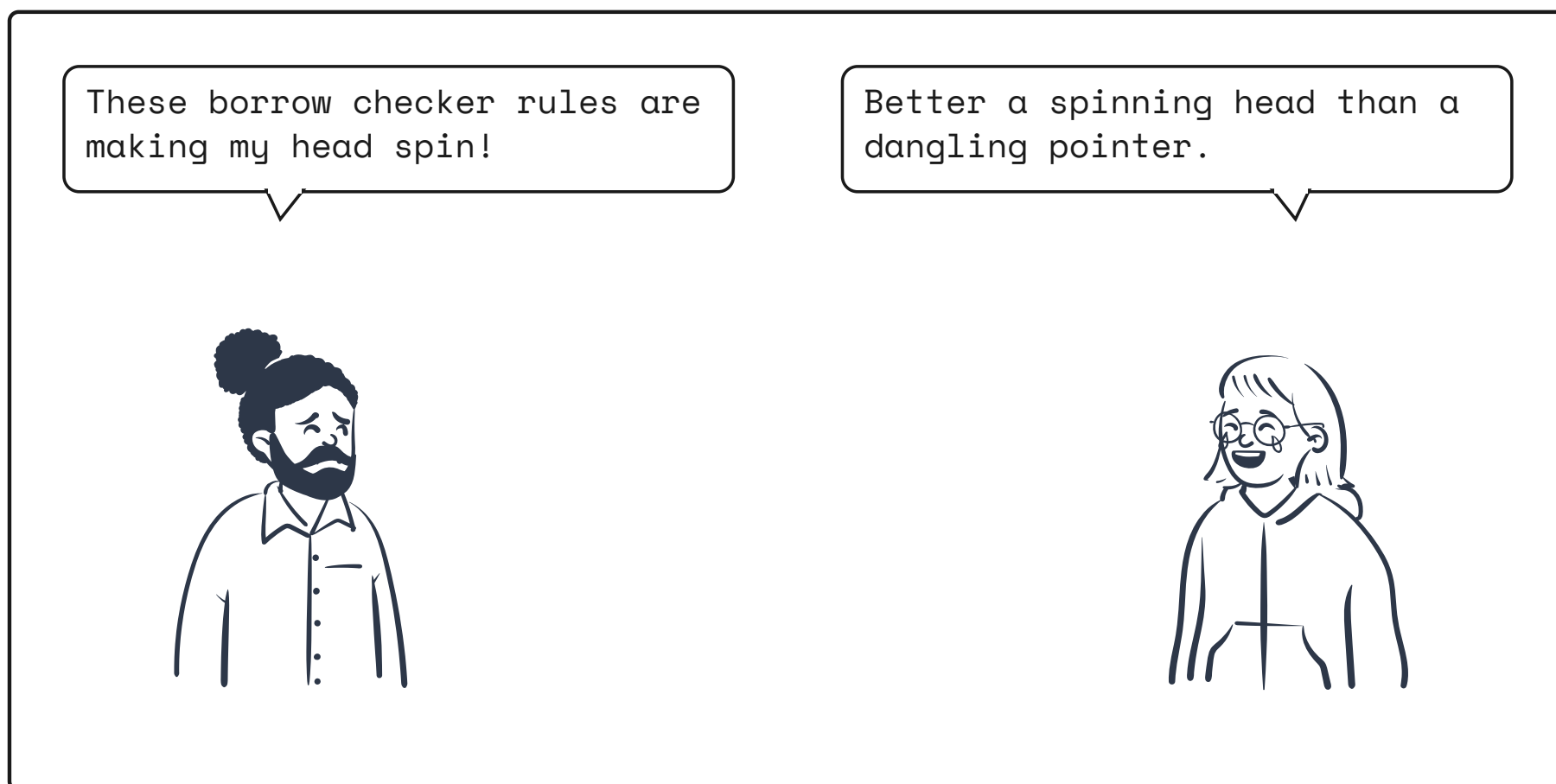
What happens:

1. Function creates local variable `x` on the stack
2. Function returns a pointer to `x`
3. When function exits, `x` is destroyed, but `ptr` still holds its address
4. Back in `main`, using `ptr` reads memory that's been freed
5. Result: undefined behavior (crash, garbage data, or worse)

Rust prevents this at compile time:

```
// Pseudo code, don't use
fn dangling() -> &String {
    let s = String::from("hello");
    &s // X ERROR: `s` doesn't live long enough
} // s is dropped here, but we're trying to return a reference to it!
```

The compiler rejects this. You must either return the owned value or ensure the reference outlives the data.



How this helps with memory management:

When Rust sees code following these rules, it can prove at compile time that:

- No variable is used after being freed
- No variable is modified while being read
- Memory can be safely freed when the owner goes out of scope

The compiler inserts cleanup code automatically, knowing it's safe. No garbage collector needed!

Why This Pattern Exists in General Rust

In most Rust code (unlike Bevy's special Commands), you can't modify a collection while iterating over it. Here's a real example that won't compile:

```
// Pseudo code, don't use
let mut items = vec![1, 2, 3, 4, 5];

for (index, item) in items.iter().enumerate() { // ← Immutable borrow starts
    if *item % 2 == 0 {
        items.remove(index); // ✗ ERROR: Can't mutably borrow while immutably borrowed!
    }
}
```

Rust rejects this with: “cannot borrow as mutable because it is also borrowed as immutable.”

Why? While `.iter()` is reading through the list, `.remove()` tries to modify it. That's like trying to read a book while someone's tearing pages out—you might try to read a page that's gone!

The Safe Pattern: Collect-Then-Process

```
// Pseudo code, don't use
let mut items = vec![1, 2, 3, 4, 5];
let mut to_remove = Vec::new();

// Step 1: Just reading (immutable borrow)
for (index, item) in items.iter().enumerate() {
    if *item % 2 == 0 {
        to_remove.push(index);
    }
} // ← Immutable borrow ends
```



```
// Step 2: Now we can modify (mutable borrow)
for index in to_remove.iter().rev() { // Remove from back to front
    items.remove(*index); // ✓ Safe! No conflicting borrows
}
```

This pattern separates reading from writing. The borrow checker is happy, and we avoid crashes.

In Our Bevy Code

Even though Bevy's Commands doesn't strictly require this pattern (it's deferred), we use it anyway because:

1. It makes our intent clear: "find eligible items" then "process them"
2. It follows Rust best practices that work everywhere
3. If we later need to work with non-deferred collections, the pattern still applies

Wiring It Together

Create `src/inventory/mod.rs` to expose the inventory system as a plugin:

```
// src/inventory/mod.rs
use bevy::prelude::*;

use crate::state::GameState;

mod inventory;
mod systems;

pub use inventory::{ItemKind, Pickable, Inventory};
use systems::handle_pickups;

/// Plugin for inventory and pickup functionality.
pub struct InventoryPlugin;

impl Plugin for InventoryPlugin {
    fn build(&self, app: &mut App) {
        app.init_resource::()
            .add_systems(
                Update,
                handle_pickups.run_if(in_state(GameState::Playing)),
            );
    }
}
```

What does `.init_resource::()` do?

It creates the `Inventory` resource by calling `Default::default()` (which creates an empty HashMap) and registers it with Bevy. Now any system can access it with `Res<Inventory>` or `ResMut<Inventory>`.

Integrating the Plugin

Open `src/main.rs` and add the inventory module and plugin:

```
// src/main.rs - Add to module declarations
mod inventory;
```

Then add the plugin to your app:

```
// src/main.rs - Add to the plugin chain
.add_plugins(state::StatePlugin)
.add_plugins(characters::CharactersPlugin)
.add_plugins(inventory::InventoryPlugin) // Add this line
.add_plugins(collision::CollisionPlugin)
```

Making Items Pickable

Now we have the infrastructure, but no items to pick up! Our procedural map generation system needs to know which decorative objects should be pickable.

Updating the Asset Spawner

Open `src/map/assets.rs`. First, add the inventory imports at the top of the file:

```
// src/map/assets.rs - Add to the imports section
use crate::inventory::{ItemKind, Pickable};
```

Now add a method to the `SpawnableAsset` struct that marks an asset as pickable:

```
// src/map/assets.rs - Add this method to the impl block for SpawnableAsset

// Inside impl SpawnableAsset {

/// Make this asset a pickable item.
pub fn with_pickable(mut self, kind: ItemKind) -> Self {
    self.pickable = Some(kind);
    self
}
}
```

This method lets us chain `.with_pickable(ItemKind::Plant1)` when defining spawnable assets.

Next, update the `SpawnableAsset` struct to store the pickable item kind:

```
// src/map/assets.rs - Add this field to SpawnableAsset struct

#[derive(Clone)]
pub struct SpawnableAsset {
    sprite_name: &'static str,
    grid_offset: GridDelta,
    offset: Vec3,
    tile_type: Option<TileType>,
    pickable: Option<ItemKind>, // Add this line
}
```

Update the constructor to initialize this field:

```
// src/map/assets.rs - Update the new() method

impl SpawnableAsset {
    pub fn new(sprite_name: &'static str) -> Self {
        Self {
            sprite_name,
            grid_offset: GridDelta::new(0, 0, 0),
            offset: Vec3::ZERO,
            tile_type: None,
            pickable: None, // Add this line
        }
    }
}
```

```
}  
// ... rest of the methods
```

Update the asset loading code:

Since we added a new field to `SpawnableAsset`, we need to update the destructuring pattern in the `load_assets` function:

```
// src/map/assets.rs - Update the destructuring in load_assets function  
  
pub fn load_assets(  
    tilemap_handles: &TilemapHandles,  
    assets_definitions: Vec<Vec<SpawnableAsset>>,  
) -> ModelsAssets<Sprite> {  
    let mut models_assets = ModelsAssets::<Sprite>::new();  
  
    for (model_index, assets) in assets_definitions.into_iter().enumerate() {  
        for asset_def in assets {  
            let SpawnableAsset {  
                sprite_name,  
                grid_offset,  
                offset,  
                tile_type,  
                pickable, // Add this line  
            } = asset_def;  
  
            let Some(atlas_index) = TILEMAP.sprite_index(sprite_name) else {  
                panic!("Unknown atlas sprite '{}'", sprite_name);  
            };  
  
            // Create the spawner function that adds components  
            let spawner = create_spawner(tile_type, pickable); // Line update alert  
  
            models_assets.add(  
                model_index,  
                ModelAsset {  
                    assets_bundle: tilemap_handles.sprite(atlas_index),  
                    grid_offset,  
                    world_offset: offset,  
                    spawn_commands: spawner,  
                },  
            );  
        }  
    }  
    models_assets  
}
```

Making the Spawner Attach Components

Now update the spawner function to actually attach `Pickable` components when entities are created. Find the `create_spawner` function in `assets.rs` and add cases for pickable items:

```
// src/map/assets.rs - Update create_spawner function to handle pickables  
  
fn create_spawner(  
    tile_type: Option<TileType>,  
    pickable: Option<ItemKind>,  
) -> fn(&mut EntityCommands) {  
    match (tile_type, pickable) {  
        // Tile types without pickable
```

```

        (Some(TileType::Dirt), None) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Dirt));
        },
        (Some(TileType::Grass), None) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Grass));
        },
        (Some(TileType::YellowGrass), None) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::YellowGrass));
        },
        (Some(TileType::Water), None) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Water));
        },
        (Some(TileType::Shore), None) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Shore));
        },
        (Some(TileType::Tree), None) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Tree));
        },
        (Some(TileType::Rock), None) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Rock));
        },
        (Some(TileType::Empty), None) => |e: &mut EntityCommands| {
            e.insert(TileMarker::new(TileType::Empty));
        },
    },

    // Pickable plants (with grass tile type)
    (Some(TileType::Grass), Some(ItemKind::Plant1)) => |e: &mut EntityCommands| {
        e.insert((TileMarker::new(TileType::Grass), Pickable::new(ItemKind::Plant1)));
    },
    (Some(TileType::Grass), Some(ItemKind::Plant2)) => |e: &mut EntityCommands| {
        e.insert((TileMarker::new(TileType::Grass), Pickable::new(ItemKind::Plant2)));
    },
    (Some(TileType::Grass), Some(ItemKind::Plant3)) => |e: &mut EntityCommands| {
        e.insert((TileMarker::new(TileType::Grass), Pickable::new(ItemKind::Plant3)));
    },
    (Some(TileType::Grass), Some(ItemKind::Plant4)) => |e: &mut EntityCommands| {
        e.insert((TileMarker::new(TileType::Grass), Pickable::new(ItemKind::Plant4)));
    },

    // Default: no components
    _ => |_: &mut EntityCommands| {},
}
}

```

Marking Props as Pickable

Finally, open `src/map/rules.rs`. First, add the import at the top:

```

// src/map/rules.rs - Add to the imports section
use crate::inventory::ItemKind;

```

Now find the `build_props_layer` function. This is where decorative objects like plants are defined. Add `.with_pickable()` to make them collectible:

```

// src/map/rules.rs - Update the plants section in build_props_layer

// Plants - make them all pickable
terrain_model_builder.create_model(
    plant_prop.clone(),
    vec![SpawnableAsset::new("plant_1")
        .with_tile_type(TileType::Grass)
        .with_pickable(ItemKind::Plant1) // Add this line
    ]
)

```

```

    ]
);
terrain_model_builder.create_model(
    plant_prop.clone(),
    vec![SpawnableAsset::new("plant_2")
        .with_tile_type(TileType::Grass)
        .with_pickable(ItemKind::Plant2) // Add this line
    ]
);
terrain_model_builder.create_model(
    plant_prop.clone(),
    vec![SpawnableAsset::new("plant_3")
        .with_tile_type(TileType::Grass)
        .with_pickable(ItemKind::Plant3) // Add this line
    ]
);
terrain_model_builder.create_model(
    plant_prop.clone(),
    vec![SpawnableAsset::new("plant_4")
        .with_tile_type(TileType::Grass)
        .with_pickable(ItemKind::Plant4) // Add this line
    ]
);

```

What's happening here?

We're using the builder pattern to configure each spawnable asset.

`.with_tile_type(TileType::Grass)` says "this is grass for collision purposes," and
`.with_pickable(ItemKind::Plant1)` says "the player can collect this as a Plant1 item."

Run your game:

```
cargo run
```

Walk near a plant or mushroom. It should disappear, and you'll see a log message:

```

Picked up Herb (total: 1) - inventory: Herb: 1
Picked up Flower (total: 1) - inventory: Flower: 1, Herb: 1
Picked up Herb (total: 2) - inventory: Flower: 1, Herb: 2

```

Spawn Troubleshooting: There's a small chance the procedural generation places the player on top of a blocking object (tree, rock) at spawn. If you can't move when the game starts, simply restart to generate a new map. This is a quirk of random generation we'll address in future chapters.

Zooming In and Following the Player

Right now, our game window shows the **entire map** at once. While functional, this has no sense of exploration. You can see everything at a glance, removing the mystery

Let's **zoom in** to show only part of the map, making everything larger and more detailed. But this creates a new problem: if the camera stays fixed and only shows part of the map, the player can walk off camera view and disappear!

So we need **two changes**:

1. First, scale up the world (larger tiles and sprites) for a zoomed-in view
2. Then, make the camera smoothly follow the player to keep them on screen

We need three things:

1. **Updated configuration** - Bigger tiles, player scale, and camera settings
2. **Map generation updates** - Scale sprites 2× larger
3. **Camera system** - Logic to smoothly track the player

Updating the Config

Our `config.rs` file already has `player`, `pickup`, and `map` modules. Let's add camera configuration.

Open `src/config.rs` and add this new module after the `map` module: I

```
// src/config.rs - Add this after the map module

/// Camera configuration
pub mod camera {
    /// How fast the camera interpolates toward the player (higher = snappier)
    pub const CAMERA_LERP_SPEED: f32 = 6.0;

    /// Z position for the camera (must be high to see all layers)
    pub const CAMERA_Z: f32 = 1000.0;
}
```

Why these values?

`CAMERA_LERP_SPEED` controls how quickly the camera catches up to the player. A value of 6.0 means the camera closes 60% of the distance each second. Higher values make the camera snappier (instantly follows), lower values make it lag behind (cinematic feel).

`CAMERA_Z` must be high enough to see all game layers. Our player is around Z=20, props are around Z=4, and we need the camera above everything to render the scene.

What's "lerp"?

"Lerp" is short for **linear interpolation**. It's a smooth transition between two values. Instead of jumping from point A to point B instantly, lerp gradually moves from A to B over time.

Think of it like this: if the camera is at position (0, 0) and the player is at (100, 0), with a lerp factor of 0.6, the camera moves to (60, 0) in the first frame. Next frame, the player is still at (100, 0), but the camera is at (60, 0), so it moves 60% of the remaining distance (40 units) to (84, 0). It keeps closing the gap smoothly until it catches up.

We also need to update some existing values for better scaling. While we're in `config.rs`, let's update the player and map configurations:

```
// src/config.rs - Update the player module

/// Player-related configuration
pub mod player {
    /// Collision radius for the player's collider (in world units)
    pub const COLLIDER_RADIUS: f32 = 24.0; // Line update alert

    /// Z-position for player rendering (above terrain, below UI)
    pub const PLAYER_Z_POSITION: f32 = 20.0;

    /// Visual scale of the player sprite
    pub const PLAYER_SCALE: f32 = 1.2; // Line update alert (was 0.8)
}
```

Now update the map module:


```
// src/config.rs - Update the map module

/// Map/terrain configuration
pub mod map {
    /// Size of a single tile in world units (64px base * 1.0 scale = 64)
    /// NOTE: This must match TILE_SIZE in generate.rs!
    pub const TILE_SIZE: f32 = 64.0; // Line update alert (was 32.0)

    /// Grid dimensions
    pub const GRID_X: u32 = 25;
    pub const GRID_Y: u32 = 18;

    /// Z-height of each layer (used for Y-based depth sorting)
    pub const NODE_SIZE_Z: f32 = 1.0; // Add this line
}
```

Updating Map Generation

Now we need to update `src/map/generate.rs` to use our centralized config values and scale up the sprites.

Open `src/map/generate.rs` and make these changes:

1. Update the imports to use config values:

We now import `GRID_X`, `GRID_Y`, `NODE_SIZE_Z`, and `TILE_SIZE` from config instead of defining them locally

```
// src/map/generate.rs - Update imports
use bevy_procedural_tilemaps::prelude::*;
use bevy::prelude::*;

use crate::config::map::{GRID_X, GRID_Y, NODE_SIZE_Z, TILE_SIZE}; // Line update alert
use crate::map::{
    assets::{load_assets, prepare_tilemap_handles},
    rules::build_world,
};
```

2. Delete these local constants (we're using config values now):

```
// src/map/generate.rs
// DELETE THESE LINES:
// DELETE pub const GRID_X: u32 = 25;
// DELETE pub const GRID_Y: u32 = 18;
// DELETE pub const TILE_SIZE: f32 = 32.;
```

3. Delete the `map_pixel_dimensions` function (no longer needed):

```
// src/map/generate.rs
// DELETE THIS ENTIRE FUNCTION:
// DELETE pub fn map_pixel_dimensions() -> Vec2 {
// DELETE     Vec2::new(TILE_SIZE * GRID_X as f32, TILE_SIZE * GRID_Y as f32)
// DELETE }
```

4. Update `NODE_SIZE` to use the config constant:

Uses `NODE_SIZE_Z` from config for consistency

```
// src/map/generate.rs

// BEFORE:
```



```
const NODE_SIZE: Vec3 = Vec3::new(TILE_SIZE, TILE_SIZE, 1.);

// AFTER:
const NODE_SIZE: Vec3 = Vec3::new(TILE_SIZE, TILE_SIZE, NODE_SIZE_Z); // Line update alert
```

5. Update ASSETS_SCALE to 2× for larger sprites:

Change from `Vec3::ONE` (1.0, 1.0, 1.0) to `Vec3::new(2.0, 2.0, 1.0)` to scale sprites 2× larger

Our tilemap uses 32px sprites but we're rendering them at 64px. The 2× scale factor in `ASSETS_SCALE` makes this happen, creating the zoomed-in view.

```
// src/map/generate.rs

// BEFORE:
const ASSETS_SCALE: Vec3 = Vec3::ONE;

// AFTER:
const ASSETS_SCALE: Vec3 = Vec3::new(2.0, 2.0, 1.0); // Line update alert
```

Building the Camera System

Create a new folder `src/camera/` and add `src/camera/camera.rs`:

```
// src/camera/camera.rs
use bevy::prelude::*;

use crate::characters::input::Player;
use crate::config::camera::{CAMERA_LERP_SPEED, CAMERA_Z};

/// Marker component for the main game camera.
#[derive(Component)]
pub struct MainCamera;

/// Spawn the main 2D camera.
pub fn setup_camera(mut commands: Commands) {
    commands.spawn((Camera2d::default(), MainCamera));
}

/// Smoothly follow the player with the camera.
///
/// Uses linear interpolation for smooth movement and snaps to pixel boundaries
/// to prevent subpixel rendering artifacts (grid shimmer).
pub fn follow_camera(
    time: Res<Time>,
    player_query: Query<&Transform, (With<Player>, Changed<Transform>)>,
    mut camera_query: Query<&mut Transform, (With<MainCamera>, Without<Player>)>,
) {
    // The Changed<Transform> filter in player_query means this system only processes
    // when the player has moved - Bevy filters it at the query level
    let Some(player_transform) = player_query.iter().next() else {
        return;
    };

    let Ok(mut camera_transform) = camera_query.single_mut() else {
        return;
    };

    let player_pos = player_transform.translation.truncate();
    let camera_pos = camera_transform.translation.truncate();

    // Early exit if camera is already very close (within 0.5 pixels)
```

```

let distance = player_pos.distance(camera_pos);
if distance < 0.5 {
    return;
}

// Smooth interpolation toward player
let lerp_factor = (CAMERA_LERP_SPEED * time.delta_secs()).clamp(0.0, 1.0);
let new_pos = camera_pos.lerp(player_pos, lerp_factor);

// Snap to pixel boundaries to prevent grid shimmer
camera_transform.translation.x = new_pos.x.round();
camera_transform.translation.y = new_pos.y.round();
camera_transform.translation.z = CAMERA_Z;
}

```

Breaking it down:

MainCamera marker: Just like we use `Player` to identify the player entity, we use `MainCamera` to find the camera entity.

setup_camera: Spawns a 2D camera with the `MainCamera` marker. This runs once at startup.

follow_camera: This is where the magic happens. Let's dissect it:

1. **Changed filter:** `Changed<Transform>` in the query definition means Bevy only calls this system when the player's transform has changed. This is a query-level optimization - if the player hasn't moved, Bevy won't even run this function. No movement? No wasted CPU cycles.
2. **Early exits:** If there's no player or no camera, bail out early. Also, if the camera is already within 0.5 pixels of the player, don't bother moving it.
3. **Lerp calculation:** We calculate how much to move based on `CAMERA_LERP_SPEED` and the time since last frame (`time.delta_secs()`). This gives us smooth movement regardless of frame rate.
4. **Pixel snapping:** `.round()` snaps the position to whole pixels. Without this, the camera can land on fractional pixel coordinates (like 10.3), causing the grid to look jittery. Snapping prevents this "shimmer" effect.

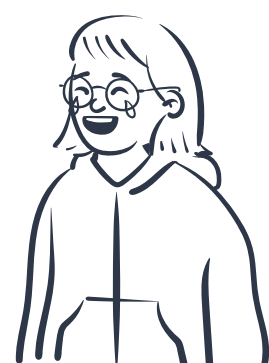
Why clamp lerp_factor?

If the frame rate is very slow (say, 2 FPS), `delta_secs()` could be 0.5 seconds. Multiplying by `CAMERA_LERP_SPEED` (6.0) gives 3.0, which would overshoot. `.clamp(0.0, 1.0)` ensures we never move more than 100% of the distance in a single frame.

I removed `.clamp()` to make it "faster." Now my camera goes WHOOSH past the player.



Congratulations, you invented the hyperdrive. Wrong game though.



Camera Module

Now create `src/camera/mod.rs` to expose the camera systems:

```
// src/camera/mod.rs
mod camera;

use bevy::prelude::*;
use crate::state::GameState;

// Re-export public items
pub use camera::MainCamera;

/// Plugin for camera systems.
pub struct CameraPlugin;

impl Plugin for CameraPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(
            Startup,
            camera::setup_camera,
        )
        .add_systems(
            Update,
            camera::follow_camera.run_if(in_state(GameState::Playing)),
        );
    }
}
```

What's happening:

The `CameraPlugin` bundles our camera systems:

- `setup_camera` runs once at `Startup` to spawn the camera
- `follow_camera` runs every frame during `Update`, but only when `GameState::Playing`

Why gate on `GameState::Playing`?

During the `Loading` state, the player entity might not exist yet. Running `follow_camera` would cause errors. By using `.run_if(in_state(GameState::Playing))`, we ensure the follow system only runs when the game is actually playing.

Integrating the Camera

Now we wire everything together in `main.rs`. But first, we need to remove the old camera setup.

Open `src/main.rs`. Find and **delete** the `setup_camera` function at the bottom:

```
// src/main.rs - DELETE this entire function
//DELETE fn setup_camera(mut commands: Commands) {
//DELETE     commands.spawn(Camera2d);
//DELETE }
```

Now update the module declarations and imports:

```
// src/main.rs - Update module declarations at the top
mod map;
mod characters;
mod state;
mod collision;
mod config;
```

```
mod inventory;
mod camera; // Add this line
```

Update the imports to include the `CameraPlugin`:

```
// src/main.rs - Update imports
use bevy::{
    prelude::*,
    window::{MonitorSelection, Window, WindowMode, WindowPlugin}, // Line update alert
};

use bevy_procedural_tilemaps::prelude::*;

use crate::camera::CameraPlugin; // Add this line
use crate::map::generate::setup_generator; // Line update alert - remove map_pixel_dimensions
```

Now update the `main` function to use the `CameraPlugin` instead of the old `setup_camera`:

```
// src/main.rs - Update the main function
fn main() {
    App::new()
        .insert_resource(ClearColor(Color::BLACK)) // Line update alert
        .add_plugins(
            DefaultPlugins
                .set(AssetPlugin {
                    file_path: "src/assets".into(),
                    ..default()
                })
                .set(WindowPlugin {
                    primary_window: Some(Window {
                        title: "Bevy Game".into(),
                        mode: WindowMode::BorderlessFullscreen(MonitorSelection::Current), // Add
this line
                        ..default()
                    }),
                    ..default()
                })
                .set(ImagePlugin::default_nearest()),
        )
        .add_plugins(ProcGenSimplePlugin::<Cartesian3D, Sprite>::default())
        .add_plugins(state::StatePlugin)
        .add_plugins(CameraPlugin) // Add this line
        .add_plugins(inventory::InventoryPlugin)
        .add_plugins(collision::CollisionPlugin)
        .add_plugins(characters::CharactersPlugin)
        .add_systems(Startup, setup_generator) // Line update alert - remove setup_camera here
        .run();
}
```

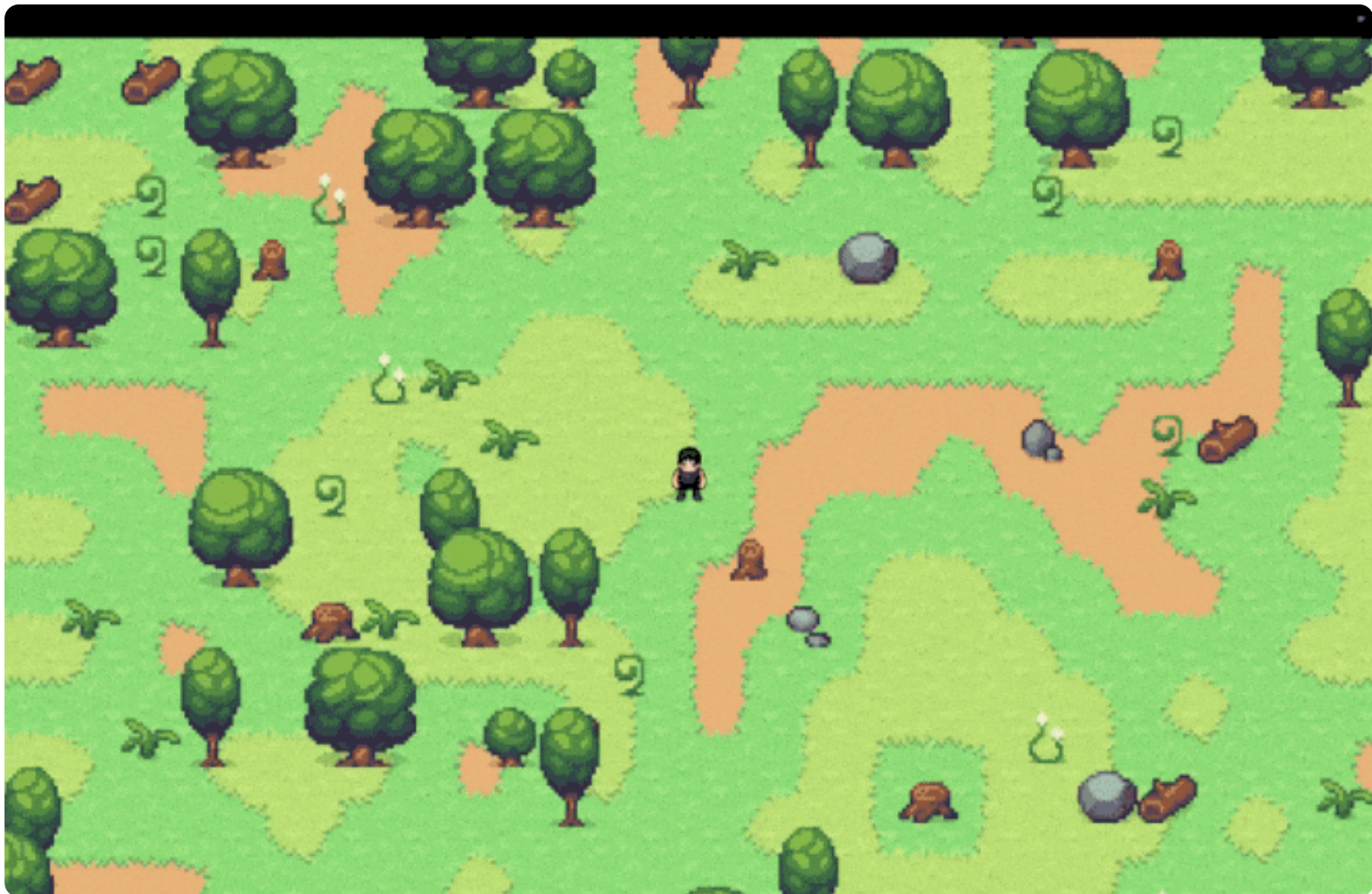
What changed:

1. Added `mod camera;` to declare the camera module
2. Imported `CameraPlugin` and window mode types
3. Added `.add_plugins(CameraPlugin)` to the app
4. Removed `setup_camera` from the `Startup` systems (it's now inside `CameraPlugin`)
5. Changed to **borderless fullscreen mode** - perfect for our zoomed-in camera view! It uses your full screen, making the zoomed world feel immersive without window borders getting in the way
6. Changed background to black for a cleaner look

Run your game:

cargo run

Walk around using the arrow keys. Notice how the camera smoothly follows your character instead of staying fixed. The camera should feel responsive but not jarring - that's the lerp in action!



The camera follows me everywhere now! I'm basically a celebrity!

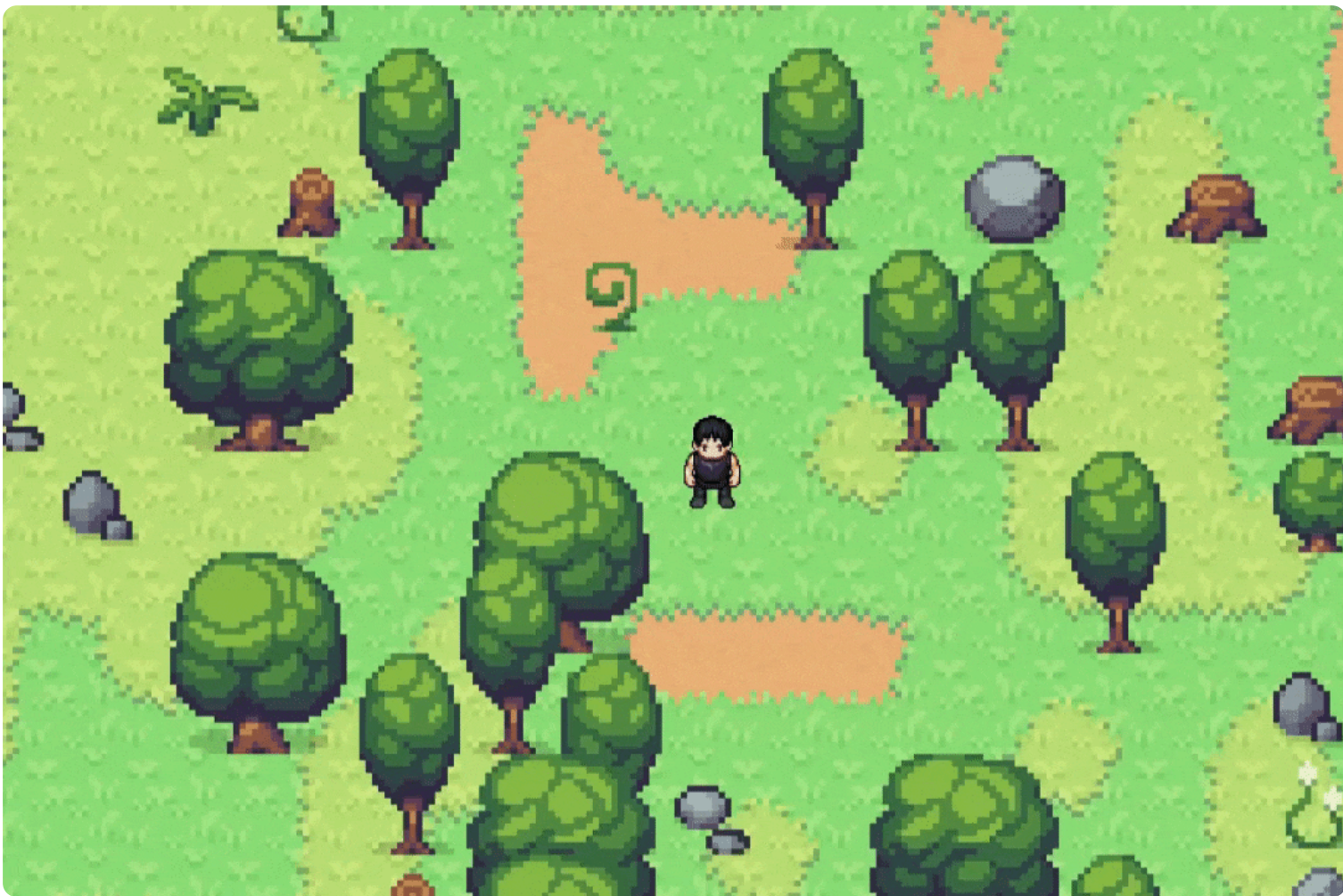


You have exactly zero viewers, but hey, at least your camera's loyal. That's one fan!



The Impatient Programmer's Guide to Bevy and Rust: Chapter 6 - Let There Be Particles

By the end of this chapter, you'll have built a particle system that brings magical powers to life. You'll create four unique effects (Fire, Arcane, Shadow, and Poison), each with glowing particles that move, rotate, and fade. You'll learn how to create particle emitters, write a custom shader for glowing effects, and use additive blending to make particles that feel magical.

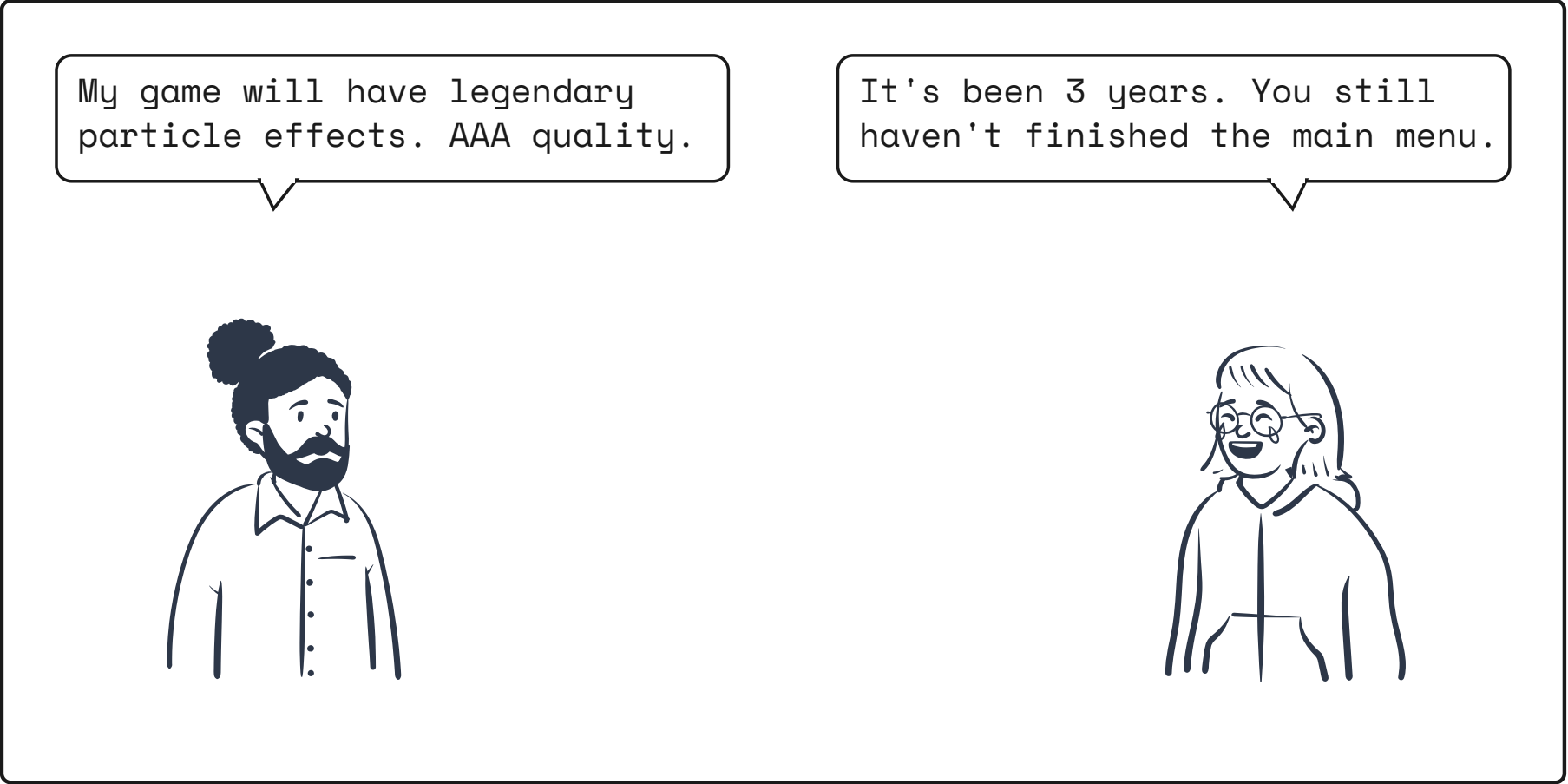


Let's Give Your Players Magic Powers

By the end of this chapter, you'll learn:

- How to spawn and update thousands of particles efficiently
- Add variance for organic, natural looking effects
- Custom shaders with additive blending for that magical glow
- Building a flexible system that's easy to extend
- Give your player magical powers

Understanding Particle Systems



A particle system spawns many small sprites that each:

1. **Spawn** from an emitter with initial properties (position, velocity, color, size)
2. **Live** for a short time, moving and changing
3. **Die** when their lifetime expires

The magic is in the numbers: spawn enough particles with slight variations, and they combine to create complex, beautiful effects.

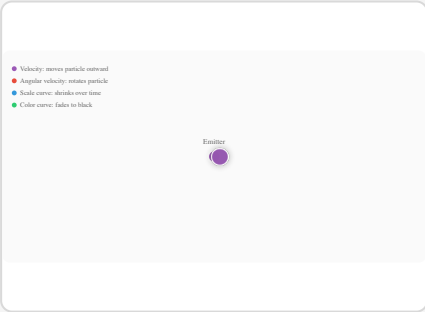
Building the Particle System

Each particle needs to be independent—moving, rotating, fading, and shrinking on its own. To achieve this, we need two types of properties: **physics** (how it moves) and **visuals** (how it looks). The physics properties like velocity, acceleration, and angular velocity give particles realistic motion.

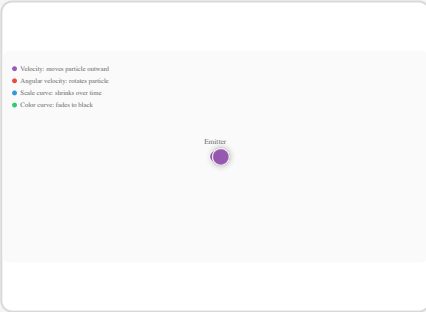
Physics + Visual Properties in Action

Watch particles move (velocity), rotate (angular velocity), shrink (scale curve), and fade (color curve)

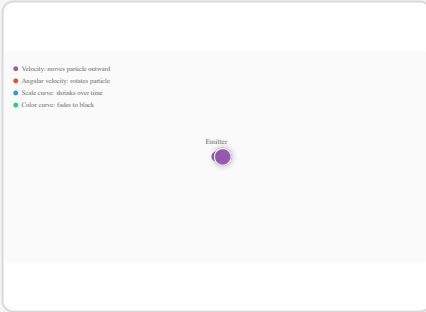
- Velocity: moves particle outward
- Angular velocity: rotates particle
- Scale curve: shrinks over time
- Color curve: fades to black



- Velocity: moves particle outward
- Angular velocity: rotates particle
- Scale curve: shrinks over time
- Color curve: fades to black



- Velocity: moves particle outward
- Angular velocity: rotates particle
- Scale curve: shrinks over time
- Color curve: fades to black



Particles shouldn't live forever. A fire particle needs to burn out, a magic spell needs to fade away. But particles also need smooth animations as they age, they should gradually fade, shrink, and change color over time, not just blink out of existence.

To make this happen, particles need to track two things: when to die and how far along they are in their life. That's why we use a countdown timer paired with progress tracking.

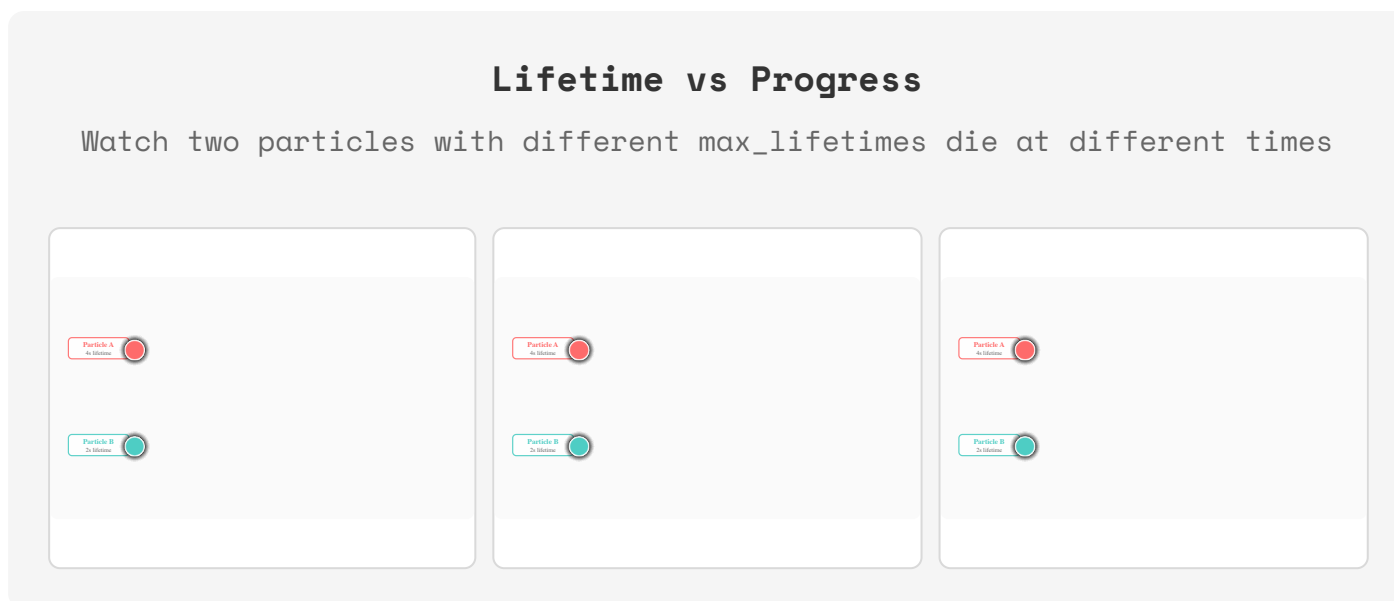
A countdown timer (`lifetime`) tells us when to delete the particle, but not its progress value. A particle with 0.5s left could be 25% done (started at 2.0s) or 99% done (started at 0.51s).

We need both values:

- `lifetime` - time remaining
- `max_lifetime` - original duration

Then: `progress = 1.0 - (lifetime / max_lifetime)`

Now we know exactly where the particle is: 0% at birth, 50% at midpoint, 100% at death. This progress value drives all animations like color, size, opacity. Without both values, particles just blink on/off. With both, they transition smoothly.



Particle Components

Now that we understand what properties particles need, let's create these variables for our particle system. We'll bundle them into a `Particle` component that tracks everything from physics to visuals.

Create `particles` folder inside `src` and add `src/particles/components.rs`:

```
// src/particles/components.rs
use bevy::prelude::*;

/// A single particle in the particle system
#[derive(Component, Clone)]
pub struct Particle {
    pub velocity: Vec3,           // Movement speed and direction (units/sec)
    pub lifetime: f32,           // Remaining time before death (seconds)
    pub max_lifetime: f32,       // Original lifetime for progress calculation
    pub scale: f32,              // Current size multiplier
    pub angular_velocity: f32,   // Rotation speed (radians/sec)
    pub acceleration: Vec3,      // Forces like gravity (units/sec²)
    // Color curve support (start → mid → end)
    pub start_color: Color,      // Color at birth (0% lifetime)
    pub mid_color: Color,        // Color at midpoint (50% lifetime)
    pub end_color: Color,        // Color at death (100% lifetime)
    // Scale curve support
    pub start_scale: f32,        // Size at birth
    pub end_scale: f32,          // Size at death (usually smaller)
}
```

Why three colors?

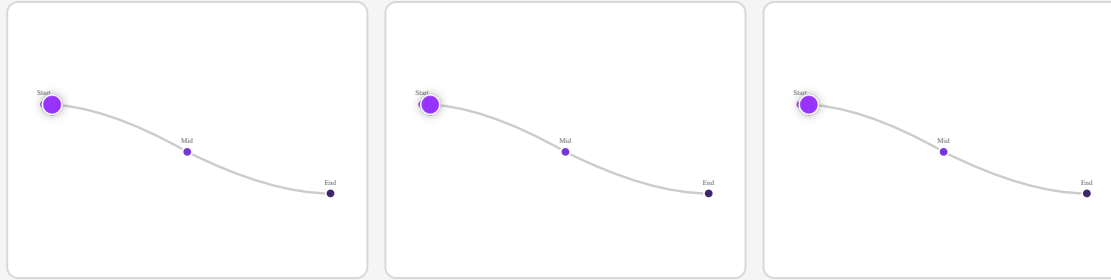
We animate color over the particle's lifetime using a **curve**:

- Start (bright) → Mid (dimmer) → End (fade to black)

This creates smooth transitions. A simple blend between two colors looks linear and boring. Three control points give us more expressive fading.

Particle Color Curve

Watch how a Shadow particle smoothly transitions through three color keyframes



The `Particle` struct holds all the data, but we need methods to:

1. **Create particles easily** - A constructor that sets sensible defaults
2. **Customize particles** - Builder methods to override specific properties
3. **Calculate animations** - Methods that compute current color and scale based on lifetime progress

Without these methods, every system that renders particles would need to duplicate this logic. By centralizing it here, we ensure consistency and make the code easier to maintain.

Let's implement the particle methods:

```
// Append to src/particles/components.rs
```

```
impl Particle {
    pub fn new(velocity: Vec3, lifetime: f32, scale: f32, start_color: Color) -> Self {
        Self {
            velocity,
            lifetime,
            max_lifetime: lifetime,
            scale,
            angular_velocity: 0.0,
            acceleration: Vec3::ZERO,
            start_color,
            mid_color: start_color, // Default to same color
            end_color: start_color,
            start_scale: scale,
            end_scale: scale * 0.5, // Default: shrink to half
        }
    }

    pub fn with_angular_velocity(mut self, angular_velocity: f32) -> Self {
        self.angular_velocity = angular_velocity;
        self
    }

    pub fn with_acceleration(mut self, acceleration: Vec3) -> Self {
        self.acceleration = acceleration;
        self
    }

    /// Set color curve for smooth color transitions
    pub fn with_color_curve(mut self, mid: Color, end: Color) -> Self {
        self.mid_color = mid;
        self.end_color = end;
        self
    }

    /// Set scale curve
    pub fn with_scale_curve(mut self, end_scale: f32) -> Self {
        self.end_scale = end_scale;
        self
    }
}
```

```

    }

    /// Returns the normalized lifetime progress (0.0 to 1.0)
    pub fn progress(&self) -> f32 {
        1.0 - (self.lifetime / self.max_lifetime)
    }

    /// Get interpolated color based on lifetime progress
    pub fn current_color(&self) -> Color {
        let progress = self.progress();

        if progress < 0.5 {
            // First half: start → mid
            let t = progress * 2.0; // Remap 0.0-0.5 to 0.0-1.0
            self.start_color.mix(&self.mid_color, t)
        } else {
            // Second half: mid → end
            let t = (progress - 0.5) * 2.0; // Remap 0.5-1.0 to 0.0-1.0
            self.mid_color.mix(&self.end_color, t)
        }
    }

    /// Get interpolated scale based on lifetime progress
    pub fn current_scale(&self) -> f32 {
        let progress = self.progress();
        self.start_scale.lerp(self.end_scale, progress)
    }
}

```

The Builder Pattern

Methods like `with_angular_velocity()` and `with_color_curve()` use the **builder pattern**, a Rust idiom for constructing complex objects step-by-step. Each method:

- Takes `self` (ownership of the particle)
- Modifies one field
- Returns `self` back

This lets us chain calls together:

```

// Pseudo code, don't use
Particle::new(velocity, 1.0, 2.0, Color::RED)
    .with_angular_velocity(3.14)
    .with_color_curve(Color::ORANGE, Color::BLACK)
    .with_scale_curve(0.1)

```

Clean, readable, and flexible. You only specify what you need to customize, everything else uses defaults from `new()`.

The Calculation Methods

Methods like `progress()`, `current_color()`, and `current_scale()` are where the magic happens. They **compute** values based on the particle's current state:

- `progress()` - Converts remaining lifetime into a 0.0-1.0 percentage, so we know exactly where the particle is in its life cycle (just born at 0%, halfway through at 50%, about to die at 100%)
- `current_color()` - Blends smoothly between the three colors based on progress, creating that magical fade effect where fire particles glow bright orange then dim to black, or poison clouds shift from sickly green to dark purple
- `current_scale()` - Gradually shrinks the particle from full size to tiny as it ages, making effects feel more dynamic and preventing particles from just blinking out of existence

The `current_color()` method:

1. First, it calls `progress()` to get a value between 0.0 (particle just spawned) and 1.0 (particle about to die)
2. Then it splits the lifetime into two halves:
 - **First half (0% to 50%):** Blends from `start_color` to `mid_color`
 - **Second half (50% to 100%):** Blends from `mid_color` to `end_color`

What's `.mix()`?

Bevy's color interpolation method. `color1.mix(&color2, 0.5)` gives you 50% between the two colors.

Why the `* 2.0`?

The `mix()` function needs input from 0.0 to 1.0 to do a complete blend. During the first half of life, progress only reaches 0.5. That's not enough, it would only blend halfway. Multiplying by 2 makes progress reach 1.0 by the halfway point, giving `mix()` the full range it needs.

The `current_scale()` method:

You know how good particle effects don't just blink out, they shrink and fade away naturally? That's what `current_scale()` creates. To achieve this, we gradually reduce the particle's size from its starting size to a tiny ending size based on how much of its life has passed.

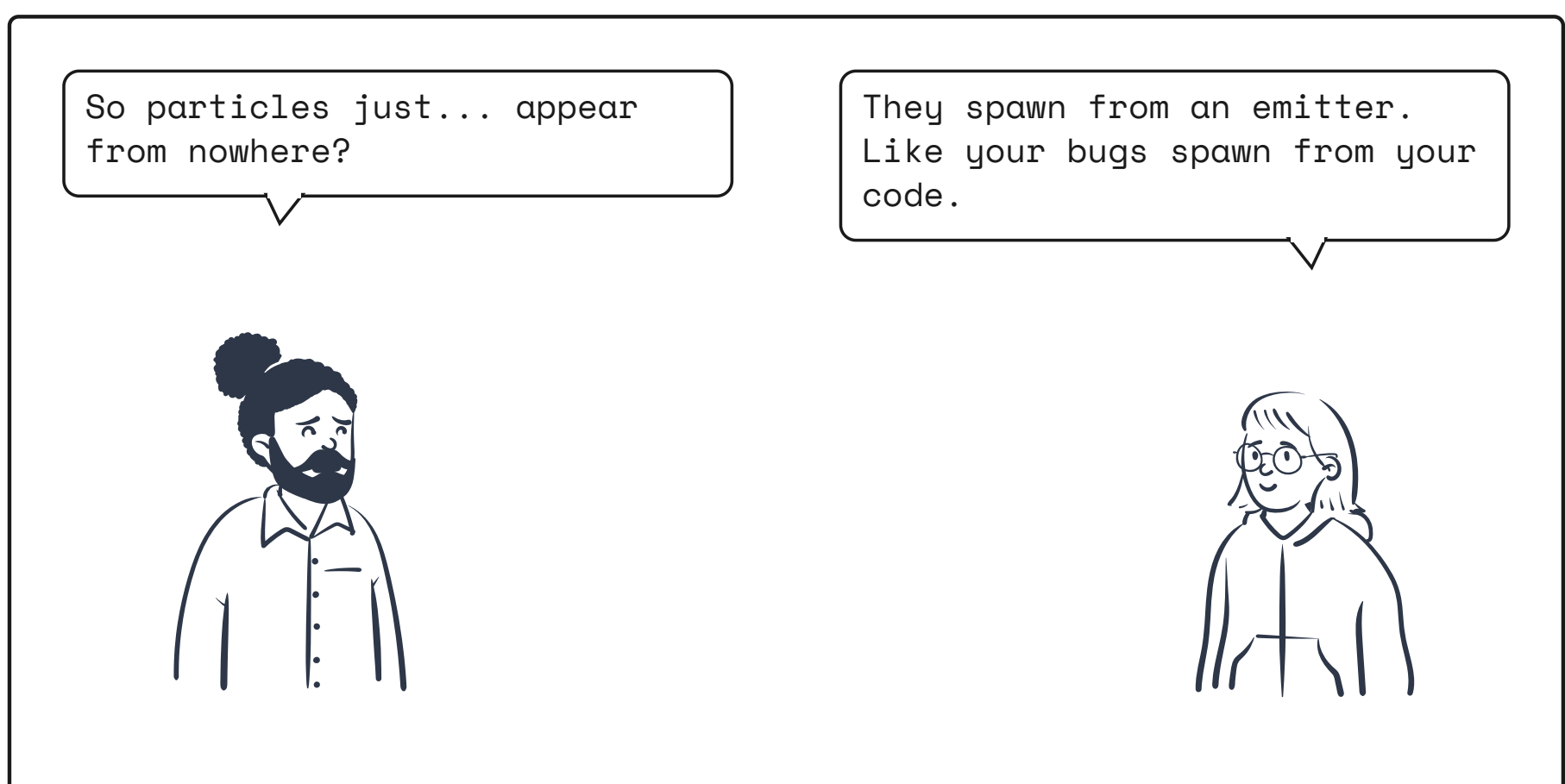
For example, imagine a particle that starts at size 2.0 and should end at 0.5:

- At 0% progress: size is 2.0 (full size, just spawned)
- At 50% progress: size is 1.25 (halfway between)
- At 100% progress: size is 0.5 (tiny, about to disappear)

The particle smoothly shrinks over time, creating that satisfying dissipation effect.

Particle Emitter

Now we need something to actually **create** particles. That's the job of the `ParticleEmitter` component. Think of it as a particle factory attached to an entity (like your player character).



The emitter needs to track:

- **When to spawn** - A timer that ticks down and triggers particle creation
- **How many to spawn** - Burst size (e.g., 5 particles at once)
- **What kind to spawn** - The template for creating particles

- **Whether it's active** - Can be turned on/off
- **One-shot vs continuous** - Fire once or keep firing

Here's the emitter component:

```
// Append to src/particles/components.rs

/// Configuration for a particle emitter
#[derive(Component, Clone)]
pub struct ParticleEmitter {
    pub spawn_timer: Timer,
    pub particles_per_spawn: u32,
    pub particle_config: ParticleConfig,
    pub active: bool,
    pub one_shot: bool,
    pub has_spawned: bool,
}

impl ParticleEmitter {
    pub fn new(spawn_rate: f32, particles_per_spawn: u32, particle_config: ParticleConfig) -> Self {
        Self {
            spawn_timer: Timer::from_seconds(spawn_rate, TimerMode::Repeating),
            particles_per_spawn,
            particle_config,
            active: true,
            one_shot: false,
            has_spawned: false,
        }
    }

    pub fn one_shot(mut self) -> Self {
        self.one_shot = true;
        self
    }
}
```

How does `one_shot` work?

Without `one_shot`, the emitter keeps spawning particles forever (or until you manually set `active = false`). This is perfect for continuous effects like a torch flame or a magic aura.

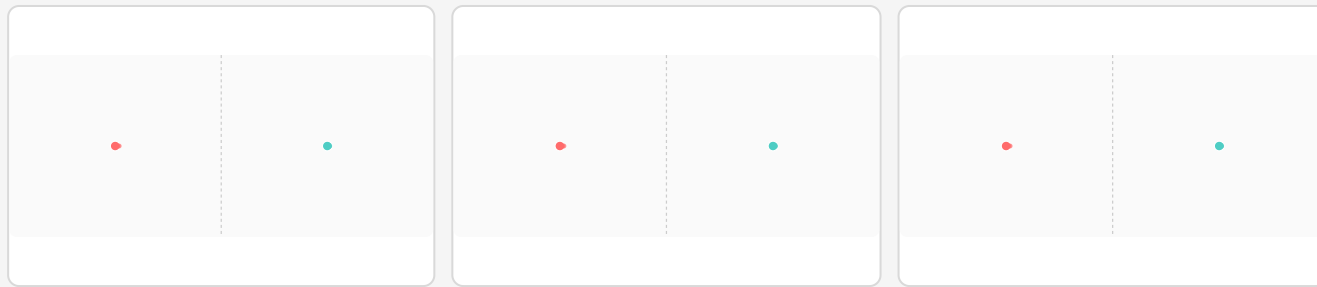
With `one_shot = true`, the emitter spawns particles **once** and then automatically deactivates. This is ideal for one-time effects like a spell cast or an explosion—you don't want those repeating every frame!

Particle Configuration

The `ParticleConfig` struct is the DNA for creating particles. It defines all the properties each particle should have, but with a twist: **variance**.

Variance

Without variance (left) vs with variance (right) - see the difference!



Without variance, every particle would be identical (boring!). With variance, each particle gets slightly randomized values, creating organic, natural looking effects. For each property, we store:

- **Base value** - The target value (e.g., `lifetime: 1.0` seconds)
- **Variance** - How much to randomize it (e.g., `lifetime_variance: 0.2` means ± 0.2 seconds)

So a particle might live for 0.8 seconds, another for 1.1 seconds, another for 0.95 seconds, all slightly different, making the effect feel alive.

Now let's create the struct that holds all particle properties. This `ParticleConfig` serves as a template that particle emitters use to spawn new particles. Instead of hardcoding values, we define them once in a config and reuse it.

Here's the configuration struct:

```
// Append to src/particles/components.rs

/// Configuration for spawning particles
#[derive(Clone)]
pub struct ParticleConfig {
    pub lifetime: f32,
    pub lifetime_variance: f32,
    pub speed: f32,
    pub speed_variance: f32,
    pub direction: Vec3,
    pub direction_variance: f32, // In radians
    pub scale: f32,
    pub scale_variance: f32,
    pub color: Color,
    pub angular_velocity: f32,
    pub angular_velocity_variance: f32,
    pub acceleration: Vec3,
    pub emission_shape: EmissionShape,
}

impl Default for ParticleConfig {
    fn default() -> Self {
        Self {
            lifetime: 1.0,
            lifetime_variance: 0.1,
            speed: 100.0,
            speed_variance: 10.0,
            direction: Vec3::X,
            direction_variance: 0.1,
            scale: 1.0,
            scale_variance: 0.1,
            color: Color::WHITE,
            angular_velocity: 0.0,
            angular_velocity_variance: 0.0,
        }
    }
}
```



```

        acceleration: Vec3::ZERO,
        emission_shape: EmissionShape::Point,
    }
}

#[derive(Clone)]
pub enum EmissionShape {
    Point,
    Circle { radius: f32 },
    Cone { angle: f32 },
}

```

Understanding the config attributes:

- `lifetime` / `lifetime_variance` - How long particles exist (seconds)
- `speed` / `speed_variance` - Initial velocity magnitude
- `direction` / `direction_variance` - Which way particles fly (direction_variance in radians creates spread)
- `scale` / `scale_variance` - Size of particles
- `color` - Base particle tint (can be HDR values above 1.0)
- `angular_velocity` / `angular_velocity_variance` - How fast particles spin
- `acceleration` - Constant force applied (like gravity or wind)
- `emission_shape` - Where particles spawn (Point, Circle, or Cone)

Particle System Updates

So far we've defined the **data structures**, what particles and emitters *are*. Now we need the **systems**, the code that actually *does* things every frame.

We need two key behaviors:

1. **Spawn particles** - Check each emitter's timer, and when it fires, create new particle entities
2. **Update particles** - Move them, rotate them, fade their colors, shrink their size, and delete them when they die

Without these systems, our components would just sit there doing nothing. Let's start with the spawning system. Create `src/particles/systems.rs`:

To spawn particles, we need two functions working together:

1. `update_emitters` - Runs every frame, checks each emitter's timer, and when the timer fires, triggers particle creation
2. `spawn_particle` - A helper function that creates a single particle entity with randomized properties

```

// src/particles/systems.rs
use super::components::*;
use super::material::ParticleMaterial;
use bevy::prelude::*;
use rand::Rng;

/// System to update particle emitters and spawn new particles
pub fn update_emitters(
    mut commands: Commands,
    time: Res<Time>,
    mut emitters: Query<(Entity, &mut ParticleEmitter, &GlobalTransform)>,
    mut meshes: ResMut<Assets<Mesh>>,
    mut materials: ResMut<Assets<ParticleMaterial>>,
) {
    let mut rng = rand::thread_rng();

```



```

    for (entity, mut emitter, global_transform) in emitters.iter_mut() {
        if !emitter.active {
            continue;
        }

        // Handle one-shot emitters
        if emitter.one_shot && emitter.has_spawned {
            emitter.active = false;
            continue;
        }

        emitter.spawn_timer.tick(time.delta());

        if emitter.spawn_timer.just_finished() {
            emitter.has_spawned = true;

            // Spawn particles
            for i in 0..emitter.particles_per_spawn {
                spawn_particle(
                    &mut commands,
                    &emitter.particle_config,
                    global_transform,
                    &mut rng,
                    &mut meshes,
                    &mut materials,
                    Some(entity),
                    i,
                );
            }

            if emitter.one_shot {
                emitter.active = false;
            }
        }
    }
}

```

How `update_emitters` Works

This system runs every frame and manages all particle emitters in the game. Here's the flow:

1. **Setup** - Create a random number generator (we'll need it for variance)
2. **Loop through all emitters** - The `Query` gives us every entity with a `ParticleEmitter` component
3. **Skip inactive emitters** - If `active` is false, don't spawn anything
4. **Handle one-shot logic** - If it's a one-shot emitter that already spawned, deactivate it
5. **Tick the timer** - Advance the spawn timer by the frame's delta time
6. **Check if timer finished** - When the timer completes, it's time to spawn!
7. **Spawn a burst** - Create `particles_per_spawn` particles using the `spawn_particle` helper
8. **Deactivate one-shots** - If it's a one-shot emitter, turn it off after spawning

Emitters don't spawn particles every frame, they use a timer to control the spawn rate. A timer of 0.1 seconds means 10 bursts per second.

What's `rand::thread_rng()`?

Creates a random number generator for this thread. We use it to add variance to particle properties, each particle gets slightly different lifetime, speed, direction, etc.

Now the particle spawning function:

`update_emitters` handles the emitter's timer and decides when to spawn particles. But it delegates the actual particle creation to a helper function called `spawn_particle`. This function takes the emitter's configuration and creates a single particle entity with randomized properties (lifetime, speed, direction, etc.), a visual mesh, and all the necessary Bevy components.

```

// Append to src/particles/systems.rs

/// Helper function to spawn a single particle
pub fn spawn_particle(
    commands: &mut Commands,
    config: &ParticleConfig,
    global_transform: &GlobalTransform,
    rng: &mut rand::rngs::ThreadRng,
    meshes: &mut ResMut<Assets<Mesh>>,
    materials: &mut ResMut<Assets<ParticleMaterial>>,
    owner: Option<Entity>,
    _particle_index: u32,
) {
    // Calculate randomized values
    let lifetime =
        config.lifetime + rng.gen_range(-config.lifetime_variance..config.lifetime_variance);
    let speed = config.speed + rng.gen_range(-config.speed_variance..config.speed_variance);
    let scale = config.scale + rng.gen_range(-config.scale_variance..config.scale_variance);
    let angular_velocity = config.angular_velocity
        + rng.gen_range(-config.angular_velocity_variance..config.angular_velocity_variance);

    // Calculate direction with variance
    let base_direction = config.direction.normalize_or_zero();
    let direction = if config.direction_variance > 0.0 {
        apply_direction_variance(base_direction, config.direction_variance, rng)
    } else {
        base_direction
    };

    // Calculate emission offset based on shape
    let emission_offset = match config.emission_shape {
        EmissionShape::Point => Vec3::ZERO,
        EmissionShape::Circle { radius } => {
            let angle = rng.gen_range(0.0..std::f32::consts::TAU);
            let distance = rng.gen_range(0.0..radius);
            Vec3::new(angle.cos() * distance, angle.sin() * distance, 0.0)
        }
        EmissionShape::Cone { angle } => {
            let cone_angle = rng.gen_range(-angle..angle);
            let rotated = rotate_vector_2d(base_direction, cone_angle);
            rotated * rng.gen_range(0.0..1.0)
        }
    };

    let velocity = direction * speed;

    // Get position directly from GlobalTransform
    let emitter_position = global_transform.translation();
    let mut position = emitter_position + emission_offset;

    // Ensure particles are at a visible Z layer (above player)
    position.z = 25.0;

    // Create particle with color curves
    let start_color = config.color;
    // Create color curve: bright → slightly dimmer → fade to black
    let mid_color = {
        let linear = config.color.to_linear();
        Color::LinearRgba(LinearRgba::new(
            linear.red * 0.7,
            linear.green * 0.7,
            linear.blue * 0.7,
            linear.alpha,
        ))
    }
}

```

```

};
let end_color = Color::srgba(
    config.color.to_linear().red * 0.3,
    config.color.to_linear().green * 0.3,
    config.color.to_linear().blue * 0.3,
    0.0,
); // Fading to black

let particle = Particle::new(velocity, lifetime, scale, start_color)
    .with_angular_velocity(angular_velocity)
    .with_acceleration(config.acceleration)
    .with_color_curve(mid_color, end_color)
    .with_scale_curve(scale * 0.2); // Shrink to 20%

// Create a mesh for the particle
let size = 24.0 * scale;
let mesh = meshes.add(Rectangle::new(size, size));
let material = materials.add(ParticleMaterial::new(start_color));

commands.spawn((
    particle,
    Mesh2d(mesh),
    MeshMaterial2d(material),
    Transform::from_translation(position),
));
}

```

How `spawn_particle` Works

This function creates a single particle entity with randomized properties.

Step 1: Add randomness to basic properties

- We don't want every particle to be identical, it won't look natural
- Take the base values (how long it lives, how fast it moves, how big it is, how fast it spins)
- Add a random amount within the variance range
- Now each particle is unique!

Step 2: Randomize the direction

- Particles shouldn't all fly in exactly the same direction
- Start with the base direction (e.g., "fly to the right")
- If there's direction variance, randomly rotate it a bit, otherwise keep it straight

Step 3: Pick a spawn position within the emitter's shape

- `Point`: all particles spawn at the exact same spot (like a laser beam origin)
- `Circle`: particles spawn randomly within a circular area (like a campfire)
- `Cone`: particles spawn in a cone shape (like a flamethrower)

Step 4: Figure out where the particle starts

- Combine direction and speed to get velocity (how it moves each frame)
- Start at the emitter's position in the world
- Add the shape offset from Step 3
- Put it at Z = 25.0 so it appears above the player

Step 5: Set up the color fade animation

- Particles should fade out as they die, not just disappear
- Start: full brightness (the color you configured)
- Middle: 70% brightness (getting dimmer)
- End: 30% brightness and transparent (fading to nothing)

Step 6: Create the particle with all its settings

- Use the builder pattern to chain all the properties together
- Set velocity, lifetime, scale, rotation speed, color curve, scale curve
- Everything is configured and ready to go

Step 7: Make a visual square for the particle

- Create a 24-pixel square mesh (scaled by the particle's size)
- Create a material that uses the particle's starting color
- This is what you'll actually see on screen

Step 8: Tell Bevy to create the particle entity

- Bundle everything together: the particle data, the mesh, the material, the position
- Bevy spawns a new entity with all these components
- The particle is now alive in the game world!

Why `std::f32::consts::TAU`?

TAU is 2π (approximately 6.28), a full circle in radians. For the `Circle` emission shape, we pick a random angle from 0 to TAU to get a point anywhere around the circle.

What's `.normalize_or_zero()`?

Converts a vector to length 1.0, making it a pure direction. If the vector is zero (no direction), it returns (0,0,0) instead of nan (Not a Number). This is safer than `.normalize()` which can panic on zero vectors.

Now add the helper functions:

These are small utility functions that help with the math in `spawn_particle`. They handle the geometry of spreading particles in different directions, essential for creating cone and spray effects instead of straight lines.

```
// Append to src/particles/systems.rs

/// Apply directional variance to a vector
fn apply_direction_variance(
    direction: Vec3,
    variance: f32,
    rng: &mut rand::rngs::ThreadRng,
) -> Vec3 {
    let angle = rng.gen_range(-variance..variance);
    rotate_vector_2d(direction, angle)
}

/// Rotate a 2D vector by an angle (in radians)
fn rotate_vector_2d(vec: Vec3, angle: f32) -> Vec3 {
    let cos = angle.cos();
    let sin = angle.sin();
    Vec3::new(vec.x * cos - vec.y * sin, vec.x * sin + vec.y * cos, vec.z)
}
```

What's this rotation math?

This is a 2D rotation matrix. To rotate a vector by an angle:

- New X = old X × cos(angle) - old Y × sin(angle)
- New Y = old X × sin(angle) + old Y × cos(angle)

Now the particle update system:

We've created the spawning system, but now we need to bring particles to life. The `update_particles` system runs every frame and handles everything that happens during a particle's

lifetime: moving it, spinning it, fading its color, shrinking its size, and removing it when it dies.

```
// Append to src/particles/systems.rs

/// System to update particle lifetime and properties
pub fn update_particles(
    mut commands: Commands,
    time: Res<Time>,
    mut particles: Query<(
        Entity,
        &mut Particle,
        &mut Transform,
        &MeshMaterial2d<ParticleMaterial>,
    )>,
    mut materials: ResMut<Assets<ParticleMaterial>>,
) {
    for (entity, mut particle, mut transform, material_handle) in particles.iter_mut() {
        particle.lifetime -= time.delta_secs();

        if particle.lifetime <= 0.0 {
            commands.entity(entity).despawn();
            continue;
        }

        // Update position
        let acceleration = particle.acceleration;
        particle.velocity += acceleration * time.delta_secs();
        transform.translation += particle.velocity * time.delta_secs();

        // Update rotation
        transform.rotate_z(particle.angular_velocity * time.delta_secs());

        // Apply color curve interpolation
        let current_color = particle.current_color();

        // Apply scale curve interpolation
        let current_scale = particle.current_scale();
        transform.scale = Vec3::splat(current_scale);

        // Update material color
        if let Some(mut material) = materials.get_mut(&material_handle.0) {
            material.color = current_color.to_linear();
        }
    }
}
```

Breaking it down:

1. **Countdown lifetime:** Subtract frame time from particle's remaining life
2. **Despawn dead particles:** When lifetime hits zero, remove the entity
3. **Apply acceleration:** Forces modify velocity over time
4. **Update position:** Move by velocity each frame
5. **Rotate:** Spin the particle based on angular velocity
6. **Update color:** Use the curve from `current_color()`
7. **Update scale:** Shrink/grow using `current_scale()`
8. **Update material:** Push the new color to the shader

Finally, add emitter cleanup:

```
// Append to src/particles/systems.rs
```

```

/// System to clean up inactive emitters that are one-shot
pub fn cleanup_finished_emitters(
    mut commands: Commands,
    emitters: Query<(Entity, &ParticleEmitter)>,
) {
    for (entity, emitter) in emitters.iter() {
        if emitter.one_shot && !emitter.active {
            commands.entity(entity).despawn();
        }
    }
}

```

This removes one-shot emitters after they've spawned their particles. Continuous emitters stick around until manually despawned.

The Shader

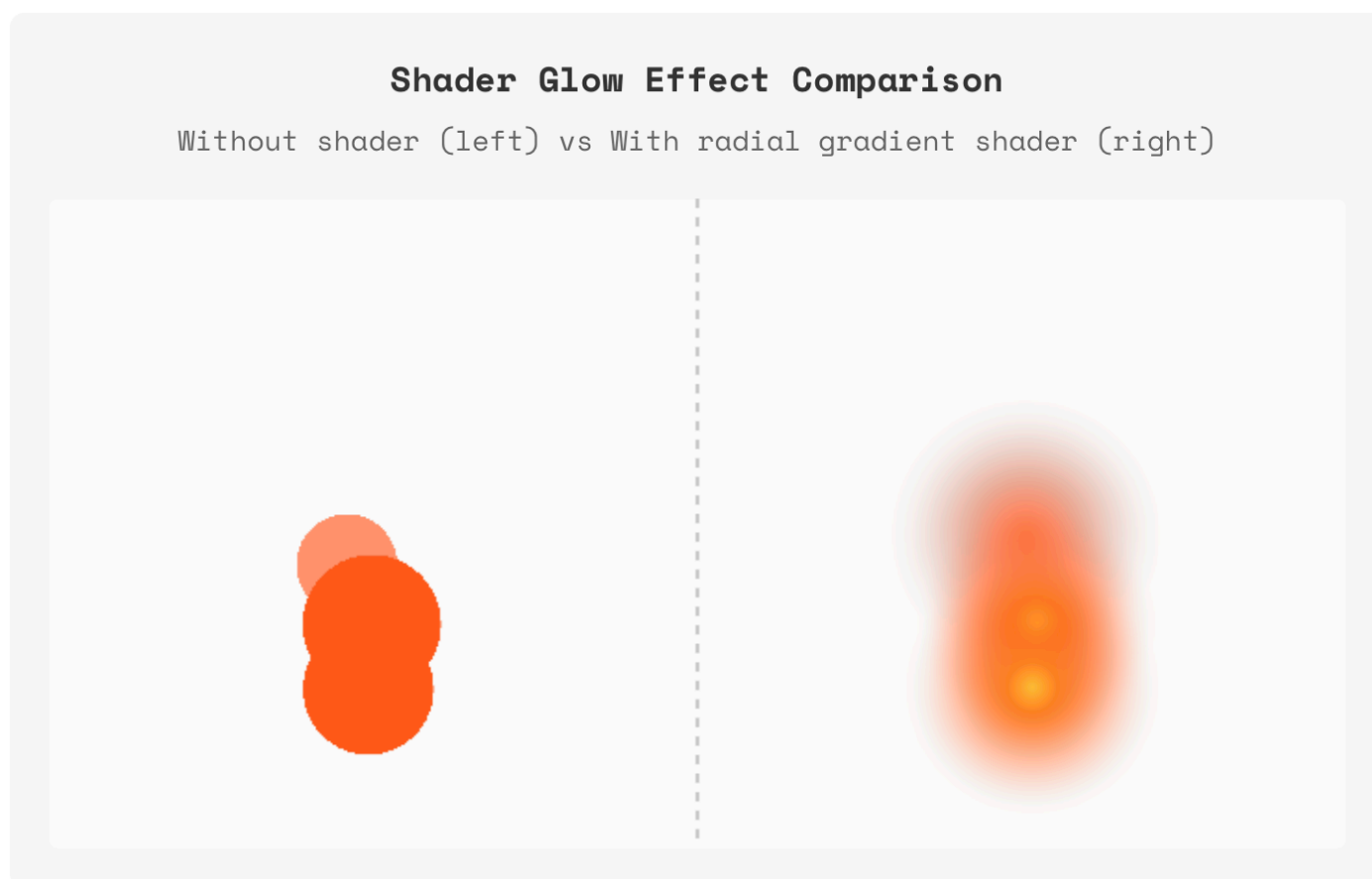
We now have particles spawning, moving, and dying. But they still look like flat colored squares. To make them glow like magical energy, we need a custom shader that runs on the GPU.

What is a shader?

A shader is a small program that runs on your GPU (graphics card) for every pixel on screen. While our Rust code runs on the CPU and manages game logic, shaders run massively in parallel on the GPU to create visual effects.

What we're achieving:

We're creating a radial glow effect where each particle is bright and intense at the center, smoothly fading to transparent at the edges. This makes particles look like glowing orbs of energy instead of flat squares.



The language:

This shader is written in **WGSL** (WebGPU Shading Language), Bevy's shader language. It's similar to Rust in some ways but designed specifically for GPU programming.

Create the folder `shaders` in `src/assets` and add the shader file `particle_glow.wgsl`, the final path should be `src/assets/shaders/particle_glow.wgsl`.

```

// src/assets/shaders/particle_glow.wgsl
// Custom shader for particles

```



```
// Creates a radial gradient glow effect with additive blending
#import bevy_sprite::mesh2d_vertex_output::VertexOutput

@group(#{MATERIAL_BIND_GROUP}) @binding(0) var<uniform> color: vec4<f32>;

@fragment
fn fragment(mesh: VertexOutput) -> @location(0) vec4<f32> {
    // Calculate distance from center (UV space is 0-1)
    let center = vec2<f32>(0.5, 0.5);
    let dist = distance(mesh.uv, center) * 2.0; // *2 to normalize to 0-1

    // Create radial gradient
    // Bright center → fades to edges
    let radial = 1.0 - smoothstep(0.0, 1.0, dist);

    // Add extra glow in the center
    let glow = pow(1.0 - dist, 3.0);

    // Combine radial gradient with center glow
    let intensity = radial * 0.7 + glow * 0.5;

    // Boost brightness near center for hot glow effect
    let brightness = 1.0 + glow * 0.5;

    // Apply to color (supports HDR - values > 1.0)
    let final_rgb = color.rgb * brightness;
    let final_alpha = color.a * intensity;

    return vec4<f32>(final_rgb, final_alpha);
}
```

How the shader creates the glow effect:

Inputs the shader receives:

- `color` - The particle's color sent from Rust code via `ParticleMaterial` (remember the `#[uniform(0)]` binding)
- `mesh.uv` - The pixel's position on the particle square. When Bevy renders a sprite with `Material2d`, it automatically creates a quad (rectangle) mesh and assigns UV coordinates to each corner: (0,0) at bottom-left, (1,1) at top-right. The GPU interpolates these for each pixel in between.

What's a mesh and what are UV coordinates?

A **mesh** is a 3D shape made of triangles. For 2D sprites, Bevy creates a simple quad (2 triangles forming a rectangle) to display the image.

UV coordinates are like a map that tells the shader where each pixel is on that rectangle. Think of it like a grid:

- U goes left to right (0.0 = left edge, 1.0 = right edge)
- V goes bottom to top (0.0 = bottom edge, 1.0 = top edge)
- So (0.5, 0.5) is the exact center of the particle

When the shader runs, every pixel knows its UV position. With these inputs (color and UV coordinates), the shader creates a radial gradient by calculating each pixel's distance from the particle's center. Pixels near the center get bright colors, while pixels at the edges fade to transparent.

Here's the technique:

1. **Calculate distance** - Measure how far this pixel is from the particle center
2. **Create smooth falloff** - Use `smoothstep` to gradually fade from bright (center) to dark (edges)
3. **Boost center brightness** - Multiply center pixels by values above 1.0 for that "hot core" effect

4. **Combine** - Mix the smooth fade with the bright center for a natural-looking glow

Without the shader, particles are just flat colored squares. With it, they become glowing orbs of energy.

What's `smoothstep`?

A function that creates smooth transitions. `smoothstep(edge0, edge1, x)` returns 0 when x is at edge0, 1 when x is at edge1, and smoothly transitions between them. Unlike a straight line transition, it starts slow, speeds up in the middle, then slows down at the end, creating natural looking fades.

Custom Shader Material

Now that we understand shaders, let's create the Rust code that uses our shader. The `ParticleMaterial` struct is our bridge between Rust code and the GPU shader, it holds the particle's color and tells Bevy which shader file to use for rendering.

Create `src/particles/material.rs`:

```
// src/particles/material.rs
use bevy::{
    prelude::*,
    reflect::TypePath,
    render::render_resource::{
        AsBindGroup, BlendComponent, BlendFactor, BlendOperation, BlendState, ColorWrites,
        RenderPipelineDescriptor, SpecializedMeshPipelineError,
    },
    shader::ShaderRef,
    sprite_render::{AlphaMode2d, Material2d, Material2dKey},
    mesh::MeshVertexBufferLayoutRef,
};

/// Custom material for particles with radial gradient shader and additive blending
#[derive(Asset, TypePath, AsBindGroup, Debug, Clone)]
pub struct ParticleMaterial {
    #[uniform(0)]
    pub color: LinearRgba,
}

impl ParticleMaterial {
    pub fn new(color: Color) -> Self {
        Self {
            color: color.to_linear(),
        }
    }
}
```

What's `AsBindGroup`?

This macro tells Bevy how to send data from your Rust code to the GPU shader. Think of it like packing a box to ship: the `#[uniform(0)]` label says "put the color value in slot 0 so the shader can find it."

Understanding the terms:

- **Material:** A material defines how a surface looks when rendered. It combines a shader (the rendering program) with properties (like color). Our `ParticleMaterial` is a custom material specifically for particles.
- **Fragment shader:** A shader program that runs for each pixel being drawn. It calculates the final color of that pixel. Our fragment shader creates the radial glow effect.
- **Alpha blending:** How transparent objects are combined with what's behind them. Normal alpha blending makes things see-through. Additive blending (what we use) adds brightness values

together for glowing effects.

- **Specialize:** Customizing the rendering pipeline for this specific material. We use it to configure additive blending instead of normal transparency.

Now implement the Material2d trait:

The `Material2d` trait tells Bevy how to render our custom material. We implement three methods:

- `fragment_shader()` - Returns the path to our shader file
- `alpha_mode()` - Enables transparency blending
- `specialize()` - Configures the rendering pipeline for additive blending

```
// Append to src/particles/material.rs

impl Material2d for ParticleMaterial {
    fn fragment_shader() -> ShaderRef {
        "shaders/particle_glow.wgsl".into()
    }

    fn alpha_mode(&self) -> AlphaMode2d {
        AlphaMode2d::Blend
    }

    fn specialize(
        descriptor: &mut RenderPipelineDescriptor,
        _layout: &MeshVertexBufferLayoutRef,
        _key: Material2dKey<Self>,
    ) -> Result<(), SpecializedMeshPipelineError> {
        // Set up additive blending for glowing effect
        if let Some(fragment) = &mut descriptor.fragment {
            if let Some(target) = fragment.targets.first_mut() {
                if let Some(target_state) = target.as_mut() {
                    target_state.blend = Some(BlendState {
                        color: BlendComponent {
                            src_factor: BlendFactor::SrcAlpha,
                            dst_factor: BlendFactor::One, // Additive!
                            operation: BlendOperation::Add,
                        },
                        alpha: BlendComponent {
                            src_factor: BlendFactor::One,
                            dst_factor: BlendFactor::One,
                            operation: BlendOperation::Add,
                        },
                    },
                ));
                    target_state.write_mask = ColorWrites::ALL;
                }
            }
        }
        Ok(())
    }
}
```

The `specialize` function configures the GPU's rendering pipeline for our particle material. It tells the GPU how to blend each rendered particle with what's already on screen.

What this means for rendered particles: When a particle is drawn, the GPU needs to know how to combine its color with the background. Normal transparency makes overlapping particles darker. Additive blending makes them brighter and glowing. This is what creates that magical fire/magic look.

Here's what the function does:

1. **Access the pipeline descriptor** - Gets the configuration for how this material will be rendered
2. **Find the color target** - Locates where color output is defined

3. **Set up additive blending** - Configures the blend mode:

- `src_factor: SrcAlpha` - Multiply particle color by its transparency
- `dst_factor: One` - Keep the background color at full strength (don't darken it)
- `operation: Add` - Add them together

Result: Overlapping particles add their brightness together, creating intense glows where they overlap. Ten overlapping fire particles create a bright white-hot center!

What's additive blending?

Normal alpha blending: `new_color = particle_color * alpha + background * (1 - alpha)`

Additive blending: `new_color = particle_color + background`

Overlapping particles add their brightness together, creating intense glows. This is how fire, magic, and explosions get that magical glowing look.



Particles Module Plugin

Create `src/particles/mod.rs`:

```
// src/particles/mod.rs
pub mod components;
pub mod material;
pub mod systems;

use crate::state::GameState;
use bevy::{prelude::*, sprite_render::Material2dPlugin};

pub use material::*;
pub use systems::*;

pub struct ParticlesPlugin;

impl Plugin for ParticlesPlugin {
    fn build(&self, app: &mut App) {
        info!("Initializing ParticlesPlugin");
        app.add_plugins(Material2dPlugin::<ParticleMaterial>::default())
            .add_systems(
                Update,
```

```

        (update_emitters, update_particles, cleanup_finished_emitters)
        .chain()
        .run_if(in_state(GameState::Playing)),
    );
    info!("ParticlesPlugin initialized");
}
}

```

Why `.chain()` ?

Systems in a chain run sequentially in the order specified. We want:

1. `update_emitters` - Spawn new particles
2. `update_particles` - Update existing particles
3. `cleanup_finished_emitters` - Remove dead emitters

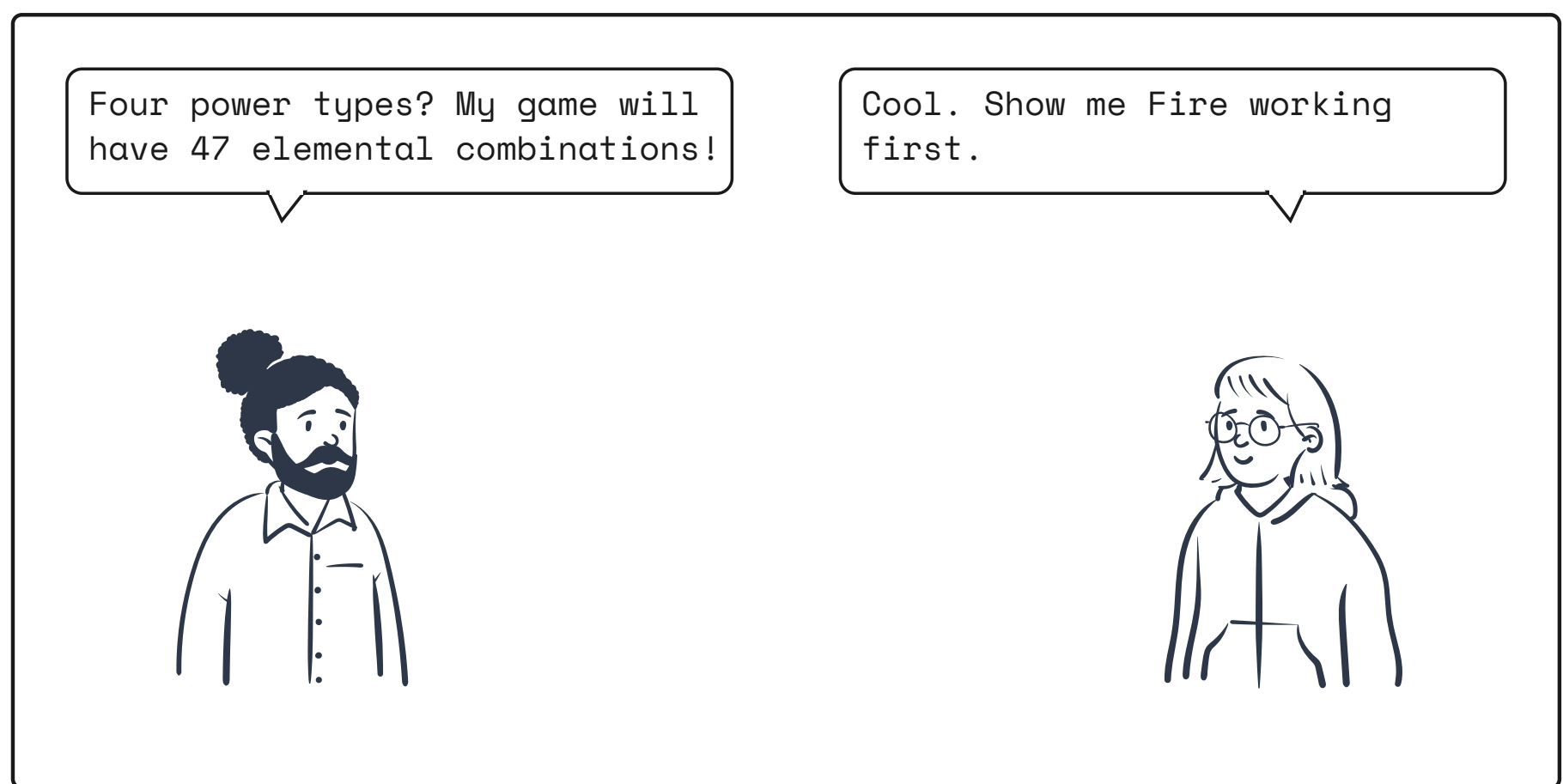
This prevents edge cases where an emitter spawns particles then immediately despawns.

Creating the Combat Module

Now that we have a particle system, let's build the combat system that uses it! Create a new folder `src/combat/` for our combat system.

Power Types

Different powers need different behaviors and visuals. Let's start by defining what makes each power unique.



Create `src/combat/power_type.rs` :

```

// src/combat/power_type.rs
use bevy::prelude::*;
use crate::particles::components::{EmissionShape, ParticleConfig};

/// The different magical powers available
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub enum PowerType {
    #[default]

```

```

    Fire,
    Arcane,
    Shadow,
    Poison,
}

```

Now let's define the visual configuration for each power. We'll separate visuals from behavior (future chapters will add damage, collision, etc.):

```

// Append to src/combat/power_type.rs

/// Visual configuration for a power - decoupled from behavior
#[derive(Clone)]
pub struct PowerVisuals {
    pub primary: ParticleConfig,
    pub core: Option<ParticleConfig>,
    pub particles_per_spawn: u32,
    pub core_particles_per_spawn: u32,
}

```

What's the difference between `primary` and `core`?

Many effects have two layers:

- **Primary:** The outer glow/trail (lots of particles, less bright)
- **Core:** The bright center (fewer particles, very bright)

Imagine a fireball, the core is the white-hot center, the primary is the orange flames around it. Not all powers need a core, poison is just a single layer of green particles.

Now we can use the `ParticleConfig` and `EmissionShape` types we defined earlier! Let's implement the visual configurations for each power type:

Now we'll configure how each power looks and behaves using the `ParticleConfig` struct. Each power gets unique visual properties (colors, speeds, sizes, variances) that define its character. Fire gets HDR orange colors and wide spread, while Shadow uses tight precision with many fast particles.

```

// Append to src/combat/power_type.rs

impl PowerType {
    /// Get visual configuration for this power
    pub fn visuals(&self, direction: Vec3) -> PowerVisuals {
        match self {
            PowerType::Fire => Self::fire_visuals(direction),
            PowerType::Arcane => Self::arcane_visuals(direction),
            PowerType::Shadow => Self::shadow_visuals(direction),
            PowerType::Poison => Self::poison_visuals(direction),
        }
    }

    fn fire_visuals(direction: Vec3) -> PowerVisuals {
        PowerVisuals {
            primary: ParticleConfig {
                lifetime: 1.0,
                lifetime_variance: 0.2,
                speed: 350.0,
                speed_variance: 40.0,
                direction,
                direction_variance: 0.12,
                scale: 1.5,
                scale_variance: 0.5,
                color: Color::srgb(3.0, 0.5, 0.1), // Bright orange-red
            },
            core: None,
            particles_per_spawn: 100,
            core_particles_per_spawn: 0,
        }
    }
}

```

```

        angular_velocity: 3.0,
        angular_velocity_variance: 2.0,
        acceleration: Vec3::ZERO,
        emission_shape: EmissionShape::Circle { radius: 10.0 },
    },
    core: Some(ParticleConfig {
        lifetime: 0.8,
        lifetime_variance: 0.2,
        speed: 350.0,
        speed_variance: 30.0,
        direction,
        direction_variance: 0.08,
        scale: 1.0,
        scale_variance: 0.3,
        color: Color::srgb(4.0, 1.0, 0.2), // Very bright yellow-white
        angular_velocity: 5.0,
        angular_velocity_variance: 2.0,
        acceleration: Vec3::ZERO,
        emission_shape: EmissionShape::Circle { radius: 5.0 },
    }),
    particles_per_spawn: 5,
    core_particles_per_spawn: 3,
}
}

```

```

fn arcane_visuals(direction: Vec3) -> PowerVisuals {
    PowerVisuals {
        primary: ParticleConfig {
            lifetime: 1.2,
            lifetime_variance: 0.2,
            speed: 300.0,
            speed_variance: 30.0,
            direction,
            direction_variance: 0.05, // Very precise
            scale: 1.2,
            scale_variance: 0.3,
            color: Color::srgb(0.5, 0.8, 2.5), // Blue arcane energy
            angular_velocity: 2.0,
            angular_velocity_variance: 1.0,
            acceleration: Vec3::ZERO,
            emission_shape: EmissionShape::Circle { radius: 6.0 },
        },
        core: Some(ParticleConfig {
            lifetime: 1.0,
            lifetime_variance: 0.1,
            speed: 300.0,
            speed_variance: 20.0,
            direction,
            direction_variance: 0.03, // Even more precise
            scale: 0.8,
            scale_variance: 0.2,
            color: Color::srgb(0.9, 0.95, 3.0), // Bright white-blue
            angular_velocity: 0.5,
            angular_velocity_variance: 0.5,
            acceleration: Vec3::ZERO,
            emission_shape: EmissionShape::Point,
        }),
        particles_per_spawn: 4,
        core_particles_per_spawn: 2,
    }
}

```

```

fn shadow_visuals(direction: Vec3) -> PowerVisuals {
    PowerVisuals {
        primary: ParticleConfig {

```



```

        lifetime: 0.6, // Short-lived
        lifetime_variance: 0.15,
        speed: 600.0, // Very fast
        speed_variance: 100.0,
        direction,
        direction_variance: 0.04,
        scale: 1.0,
        scale_variance: 0.4,
        color: Color::srgb(0.6, 0.2, 1.2), // Dark purple
        angular_velocity: 8.0, // Spins fast
        angular_velocity_variance: 4.0,
        acceleration: Vec3::ZERO,
        emission_shape: EmissionShape::Point,
    },
    core: Some(ParticleConfig {
        lifetime: 0.5,
        lifetime_variance: 0.1,
        speed: 650.0,
        speed_variance: 80.0,
        direction,
        direction_variance: 0.02,
        scale: 1.3,
        scale_variance: 0.3,
        color: Color::srgb(1.0, 0.5, 1.8), // Brighter purple core
        angular_velocity: 12.0,
        angular_velocity_variance: 5.0,
        acceleration: Vec3::ZERO,
        emission_shape: EmissionShape::Point,
    }),
    particles_per_spawn: 7, // Many particles
    core_particles_per_spawn: 3,
}

}

fn poison_visuals(direction: Vec3) -> PowerVisuals {
    PowerVisuals {
        primary: ParticleConfig {
            lifetime: 1.5, // Long-lived
            lifetime_variance: 0.4,
            speed: 200.0, // Slow
            speed_variance: 50.0,
            direction,
            direction_variance: 0.25, // Spreads a lot
            scale: 1.8, // Large particles
            scale_variance: 0.6,
            color: Color::srgb(0.3, 2.0, 0.3), // Toxic green
            angular_velocity: 1.0,
            angular_velocity_variance: 2.0,
            acceleration: Vec3::new(0.0, 20.0, 0.0), // Rises slightly
            emission_shape: EmissionShape::Circle { radius: 15.0 },
        },
        core: None, // No core - just a cloud
        particles_per_spawn: 6,
        core_particles_per_spawn: 0,
    }
}

}

```

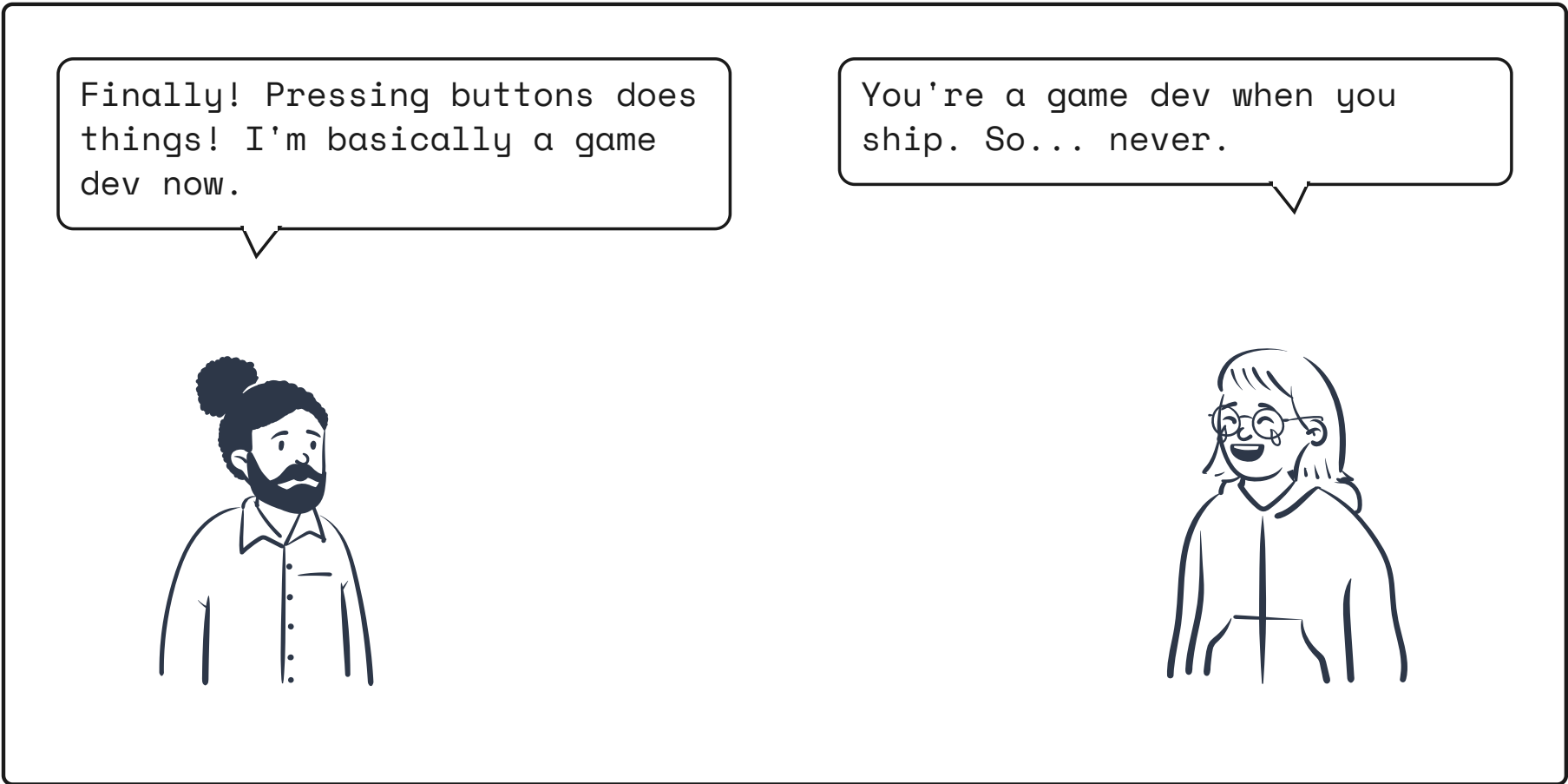
The configuration numbers define each power's unique character. Fire has high speed (350), wide spread (0.12 variance), and HDR orange colors. Shadow has very high speed (500), tight beam (0.05 variance), and dark purple.

Why `Color::srgb(3.0, 0.5, 0.1)` with values above 1.0?

Values above 1.0 create **HDR (High Dynamic Range) colors** that glow brighter than normal. When combined with additive blending (from our particle shader), these create the magical glow effect. It's like cranking the brightness past 100%, perfect for fire and magic.

What's `direction_variance` ?

Controls how much particles spread. Low variance (0.03 for Arcane) means particles stay in a tight beam. High variance (0.25 for Poison) creates a wide, spreading cloud. It's measured in radians.



Now notice the design pattern here: each power has a distinct **character** expressed through numbers:

Power	Character	Speed	Lifetime	Spread	Particles
Fire	Hot, chaotic	Medium	Short	Medium	Medium
Arcane	Precise, magical	Medium	Long	Tight	Few
Shadow	Fast, deadly	Fast	Very short	Tight	Many
Poison	Spreading, lingering	Slow	Long	Wide	Medium

This data-driven approach means adding a new power is just adding a new function with different numbers, no code logic changes needed.

Player Combat Component

Now we need a component to attach to the player that tracks their current power and prevents rapid-fire spam.

My player should fire 60 fireballs per second. Maximum DPS!



Your laptop would catch fire before the enemies do.



We use a countdown `Timer` that must finish before the next attack. When the player attacks, we check if the timer is finished. If yes, spawn particles and reset the timer back to 0.5 seconds. If no, ignore the input. This creates a smooth attack rate without complex cooldown tracking.

Create `src/combat/player_combat.rs`:

```
// src/combat/player_combat.rs
use super::power_type::PowerType;
use bevy::prelude::*;

/// Attach to any entity that can use powers (player, NPCs)
#[derive(Component)]
pub struct PlayerCombat {
    pub power_type: PowerType,
    pub cooldown: Timer,
}

impl Default for PlayerCombat {
    fn default() -> Self {
        Self {
            power_type: PowerType::Fire,
            cooldown: Timer::from_seconds(0.5, TimerMode::Once),
        }
    }
}

impl PlayerCombat {
    pub fn new(power_type: PowerType) -> Self {
        Self {
            power_type,
            cooldown: Timer::from_seconds(0.5, TimerMode::Once),
        }
    }

    pub fn with_cooldown(mut self, seconds: f32) -> Self {
        self.cooldown = Timer::from_seconds(seconds, TimerMode::Once);
        self
    }
}
```

What's `TimerMode::Once`?

Timers in Bevy can be `Once` (stop when finished) or `Repeating` (restart automatically). For cooldowns, we want `Once`, the timer counts down from 0.5 seconds to 0, then stops. We manually reset it when the player attacks.

Why 0.5 seconds?

This creates a ~2 attacks per second rate. Too fast feels spammy, too slow feels unresponsive. You can tweak this with `.with_cooldown()` for different weapons or upgrades.

Combat Systems

Now for the system that handles player input and spawns projectiles.

Create `src/combat/systems.rs`:

```
// src/combat/systems.rs
use super::player_combat::PlayerCombat;
use super::power_type::{PowerType, PowerVisuals};
use crate::characters::facing::Facing;
use crate::characters::input::Player;
use crate::particles::components::ParticleEmitter;
use bevy::prelude::*;

/// Marker for projectile effects
#[derive(Component)]
pub struct ProjectileEffect {
    pub power_type: PowerType,
}

pub fn handle_power_input(
    mut commands: Commands,
    input: Res<ButtonInput<KeyCode>>,
    time: Res<Time>,
    mut player_query: Query<(&GlobalTransform, &Facing, &mut PlayerCombat), With<Player>>,
) {
    let Ok((global_transform, facing, mut combat)) = player_query.single_mut() else {
        return;
    };

    combat.cooldown.tick(time.delta());

    let ctrl_pressed =
        input.just_pressed(KeyCode::ControlLeft) || input.just_pressed(KeyCode::ControlRight);

    if !ctrl_pressed {
        return;
    }

    // Only fire if cooldown has elapsed
    if combat.cooldown.elapsed_secs() < combat.cooldown.duration().as_secs_f32() {
        return;
    }

    combat.cooldown.reset();

    let position: Vec3 = global_transform.translation();
    let direction = facing_to_vec3(facing);
    let spawn_position = position + direction * 5.0;

    // Get visuals from power type
    let visuals = combat.power_type.visuals(direction);

    spawn_projectile(&mut commands, spawn_position, combat.power_type, &visuals);
}
```

```
    info!("{:?} projectile fired!", combat.power_type);
}
```

Breaking it down:

1. **Tick the cooldown:** `combat.cooldown.tick(time.delta())` counts down by the frame time
2. **Check input:** Only proceed if Ctrl is pressed
3. **Check cooldown:** If `elapsed_secs() < duration()`, we're still on cooldown
4. **Reset cooldown:** `combat.cooldown.reset()` starts the timer over
5. **Calculate spawn position:** Offset slightly from the player in the facing direction
6. **Get visuals:** Power type knows its own visual configuration
7. **Spawn projectile:** Create the particle emitters using our particle system!

Now let's implement the projectile spawning system. This is where we convert player input into visible magical effects.

```
// Append to src/combat/systems.rs

fn spawn_projectile(
    commands: &mut Commands,
    position: Vec3,
    power_type: PowerType,
    visuals: &PowerVisuals,
) {
    // Primary particles
    let primary_emitter =
        ParticleEmitter::new(0.016, visuals.particles_per_spawn, visuals.primary.clone())
            .one_shot();

    commands.spawn((
        primary_emitter,
        Transform::from_translation(position),
        GlobalTransform::from(Transform::from_translation(position)),
        ProjectileEffect { power_type },
    ));

    // Core particles (if the power has a core)
    if let Some(ref core_config) = visuals.core {
        let core_emitter =
            ParticleEmitter::new(0.016, visuals.core_particles_per_spawn, core_config.clone())
                .one_shot();

        commands.spawn((
            core_emitter,
            Transform::from_translation(position),
            GlobalTransform::from(Transform::from_translation(position)),
            ProjectileEffect { power_type },
        ));
    }
}
```

How the spawning function works:

1. **Create primary emitter** - Spawn the main particle layer with configured count and settings
2. **Mark as one-shot** - Emitter spawns once then deactivates (perfect for projectiles)
3. **Position it** - Place at the specified position
4. **Add marker component** - `ProjectileEffect` tags this as a projectile for other systems
5. **Spawn core (if exists)** - Some powers have a bright center layer

The function takes the visual configuration and converts it into actual particle emitters. Each emitter is its own entity with the `ParticleEmitter` component.

Now add the helper function to convert facing to direction:

```
// Append to src/combat/systems.rs

fn facing_to_vec3(facing: &Facing) -> Vec3 {
    match facing {
        Facing::Right => Vec3::X,
        Facing::Left => Vec3::NEG_X,
        Facing::Up => Vec3::Y,
        Facing::Down => Vec3::NEG_Y,
    }
}
```

Finally, let's add a debug system to quickly switch between powers for testing:

```
// Append to src/combat/systems.rs

/// Switch powers with number keys (for testing)
pub fn debug_switch_power(
    input: Res<ButtonInput<KeyCode>>,
    mut player_query: Query<&mut PlayerCombat, With<Player>>,
) {
    let Ok(mut combat) = player_query.single_mut() else {
        return;
    };

    let new_power = if input.just_pressed(KeyCode::Digit1) {
        Some(PowerType::Fire)
    } else if input.just_pressed(KeyCode::Digit2) {
        Some(PowerType::Arcane)
    } else if input.just_pressed(KeyCode::Digit3) {
        Some(PowerType::Shadow)
    } else if input.just_pressed(KeyCode::Digit4) {
        Some(PowerType::Poison)
    } else {
        None
    };

    if let Some(power) = new_power {
        combat.power_type = power;
        info!("Switched to {:?}", power);
    }
}
```

This lets you press 1-4 to instantly switch powers while playing. Essential for testing visual effects without restarting the game.

Combat Module Plugin

Create `src/combat/mod.rs`:

```
// src/combat/mod.rs
mod player_combat;
mod power_type;
mod systems;

pub use player_combat::PlayerCombat;
pub use power_type::{PowerType, PowerVisuals};
pub use systems::{ProjectileEffect, debug_switch_power, handle_power_input};

use bevy::prelude::*;
```

```
pub struct CombatPlugin;

impl Plugin for CombatPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(Update, (handle_power_input, debug_switch_power));
    }
}
```

Perfect! Now our combat module is ready to use the particle system.

Putting it all together

Now let's wire everything together!

First, add `rand` to your `Cargo.toml`:

```
[dependencies]
bevy = "0.19"
bevy_procedural_tilemaps = "0.4.0"
bevy_common_assets = { version = "0.17", features = ["ron"] }
serde = { version = "1.0", features = ["derive"] }
rand = "0.8" # Add this line
```

Open `src/main.rs` and add the modules:

```
// src/main.rs - Add to module declarations
mod combat;
mod particles;
```

Then add the plugins:

```
// src/main.rs - Add to the plugin chain
.add_plugins(combat::CombatPlugin)
.add_plugins(particles::ParticlesPlugin)
```

The player needs the `PlayerCombat` component. Open `src/characters/spawn.rs`:

```
// src/characters/spawn.rs - Add import
use crate::combat::PlayerCombat;
```

Find the `initialize_player_character` system and add the combat component when inserting the player:

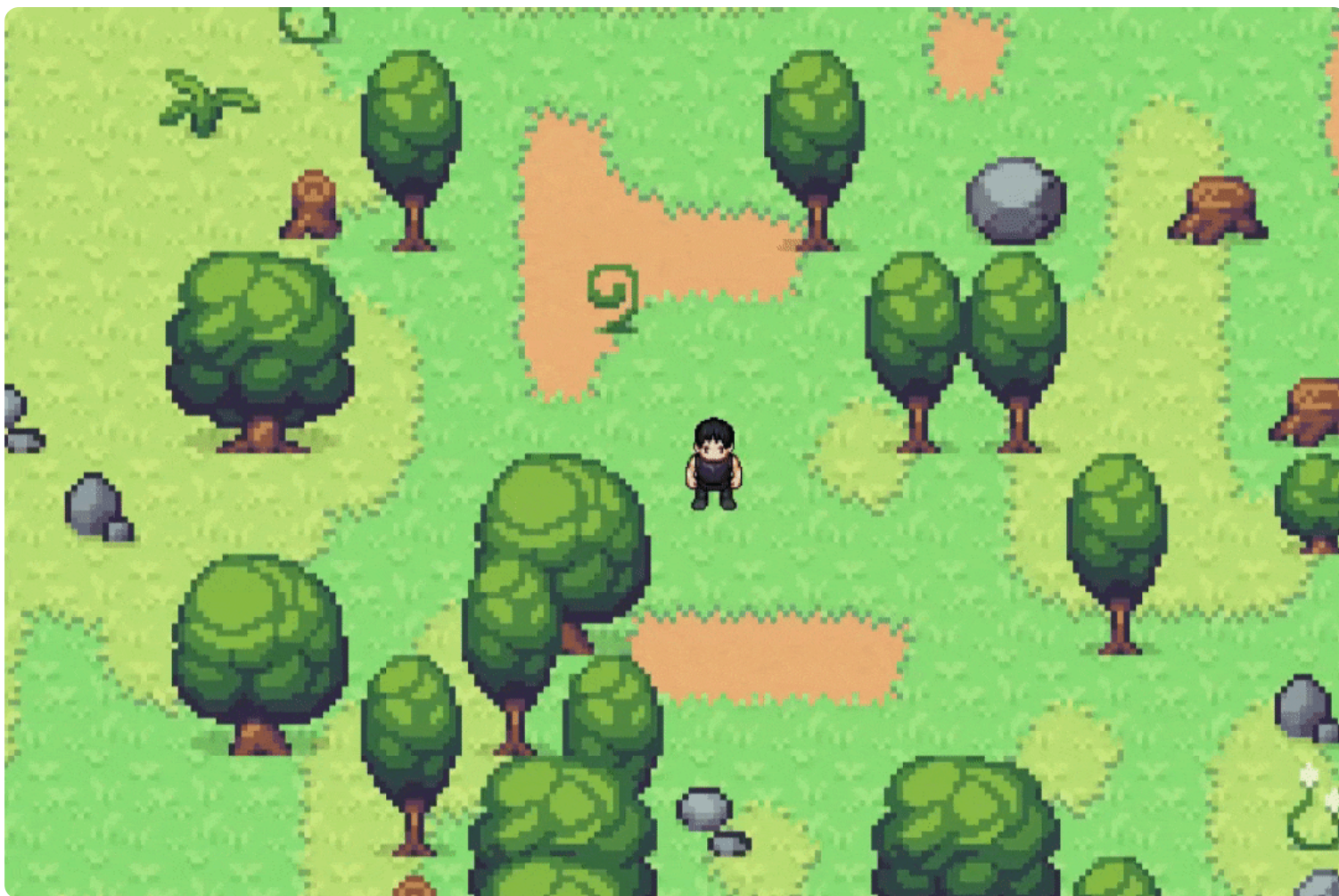
```
// In the initialize_player_character function, add PlayerCombat to the character entity bundle
commands.entity(entity).insert((
    AnimationController::default(),
    CharacterState::default(),
    Velocity::default(),
    Facing::default(),
    Collider::default(),
    PlayerCombat::default(), // Add this line
    AnimationTimer(Timer::from_seconds(
        DEFAULT_ANIMATION_FRAME_TIME,
        TimerMode::Repeating,
    )),
    character_entry.clone(),
```



```
    sprite,  
  ));
```

Run your game:

```
cargo run
```



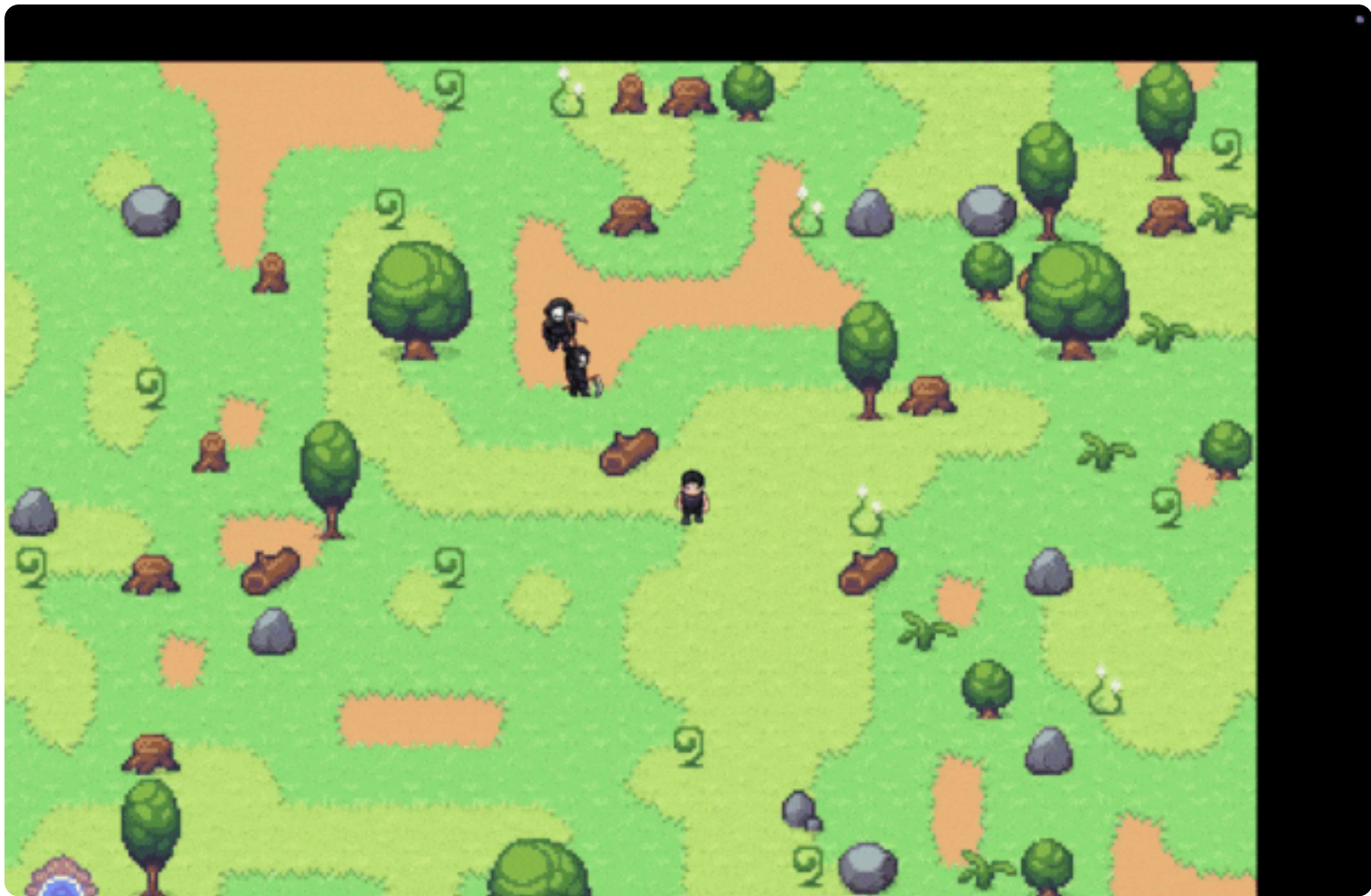
Spawn Troubleshooting: There's a small chance the procedural generation places the player on top of a blocking object (tree, rock) at spawn. If you can't move when the game starts, simply restart to generate a new map. This is a quirk of random generation we'll address in future chapters.

Controls:

- **Arrow keys:** Move
- **Ctrl:** Fire current power
- **1-4:** Switch powers (Fire, Arcane, Shadow, Poison)

The Impatient Programmer's Guide to Bevy and Rust: Chapter 7 - Let There Be Enemies

By the end of this chapter, you'll have enemies that follow and attack your player.



What Makes an Enemy Different from a Player?

Your player character moves, animates, collides with objects, and attacks. Now you want enemies do almost the same things except for decision making.

Players make decisions through keyboard input. You press `↑`, the character walks up. You press `Ctrl`, they attack. The input system reads your keypresses and sets components like `Velocity` and `CharacterState`.

Enemies need to make the same decisions, but automatically. Instead of reading a keyboard, they read the game world. “Where is the player? Am I close enough to attack? Which direction should I face?”

Both players and enemies need to:

- Move around the world
- Animate based on state
- Collide with walls and objects
- Attack with magical powers

The only difference is **who decides what to do**. For players, you decide. For enemies, AI decides. But once the decision is made, the rest is identical. Hence can we re-use the code we have written for player for enemy? Let's find out.

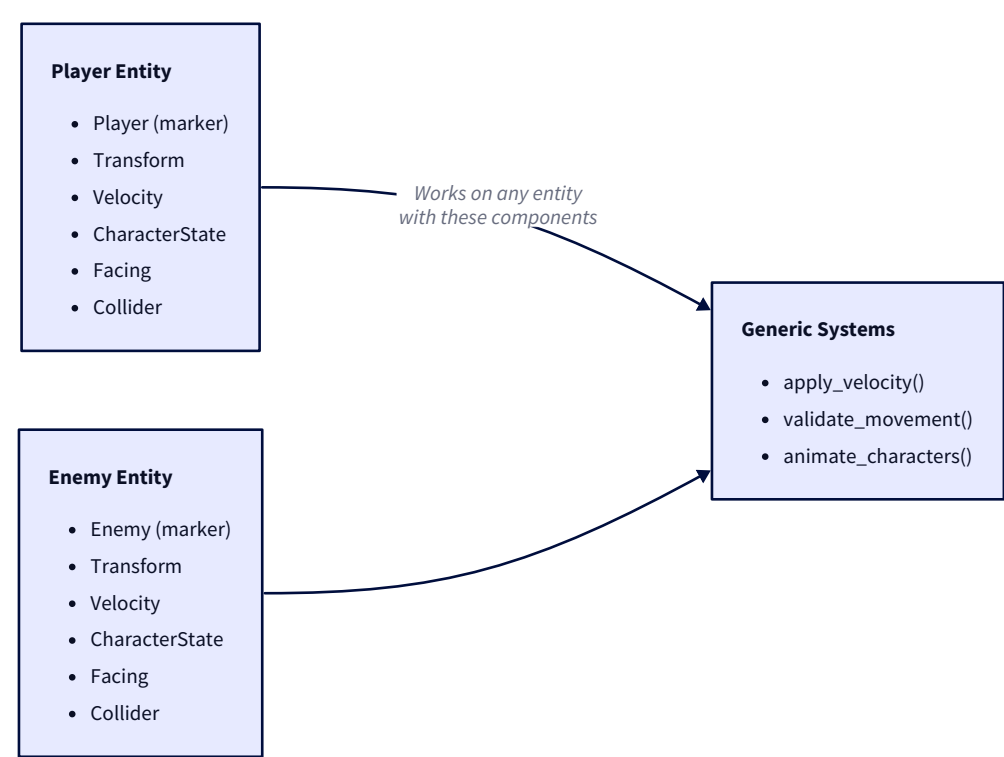
Thinking in Components

In Bevy’s ECS architecture, entities are just IDs. Components are the data. Systems are the logic. This separation lets us build systems that work on **any** entity with the right components.

Your player entity has these components:

- `Player` (marker)
- `Transform` (position)
- `Velocity` (movement)
- `CharacterState` (Idle, Walking, etc.)
- `Facing` (direction)
- `AnimationController` (sprite animation)
- `Collider` (physics)

An enemy entity needs most of the same components. The key insight: systems like `apply_velocity`, `validate_movement`, and `animate_characters` don’t care whether an entity is a player or enemy. They just process components.



Creating the Enemy Module

Create a new folder `src/enemy` with the following structure:

```
src/  
├─ enemy/  
|   ├─ mod.rs  
|   ├─ components.rs  
|   ├─ ai.rs  
|   ├─ combat.rs  
|   └─ spawn.rs
```

Enemy Components

We need three components to define enemy behavior. The `Enemy` marker identifies which entities are enemies. The `EnemyCombat` component gives them attack capabilities. The `AIBehavior` component controls their decision-making.

Create `src/enemy/components.rs`:

```

// src/enemy/components.rs
use crate::combat::PowerType;
use bevy::prelude::*;

/// Marker component for enemy entities
#[derive(Component)]
pub struct Enemy;

/// Combat capabilities for enemies
#[derive(Component)]
pub struct EnemyCombat {
    pub power_type: PowerType,
    pub cooldown: Timer,
}

impl Default for EnemyCombat {
    fn default() -> Self {
        Self {
            power_type: PowerType::Shadow, // Graveyard reaper uses shadow magic
            cooldown: Timer::from_seconds(2.0, TimerMode::Once), // Slower than player
        }
    }
}

impl EnemyCombat {
    pub fn new(power_type: PowerType, cooldown_seconds: f32) -> Self {
        Self {
            power_type,
            cooldown: Timer::from_seconds(cooldown_seconds, TimerMode::Once),
        }
    }
}

/// AI behavior state for enemies
#[derive(Component)]
pub struct AIBehavior {
    pub attack_range: f32,
    pub detection_range: f32,
}

impl Default for AIBehavior {
    fn default() -> Self {
        Self {
            attack_range: 150.0, // Stop and attack within this range
            detection_range: 500.0, // Start following player within this range
        }
    }
}

impl AIBehavior {
    pub fn new(attack_range: f32, detection_range: f32) -> Self {
        Self {
            attack_range,
            detection_range,
        }
    }
}

```

What's happening here?

The `Enemy` component is just a marker. It has no data, it just tags entities as enemies so systems can query for them.

`EnemyCombat` stores the enemy's attack capabilities. The `power_type` determines what kind of projectile they fire (Shadow magic for our graveyard reaper). The `cooldown` timer prevents them from attacking every frame.

`AIBehavior` defines two ranges. The `detection_range` is how far away the enemy can “see” the player. The `attack_range` is how close they need to be before they stop moving and start attacking.

Why use Default?

The `Default` trait lets us spawn enemies quickly without specifying every field. We can call `EnemyCombat::default()` to get sensible starting values, then customize specific enemies with `EnemyCombat::new()` if needed.

Making Enemies Follow You

Now for the interesting part. We need a system that makes enemies track and follow the player.

Simple, move enemies directly toward the player? But what if there's a tree in the way? The enemy walks into the obstacle and gets stuck.

We need **pathfinding**, the ability to find a route around obstacles. The industry-standard solution is the A* (“A-star”) algorithm.

Understanding A* Pathfinding

A* is a pathfinding algorithm that finds the shortest path between two points on a grid. It's used everywhere in games, robotics, GPS navigation because it's both efficient and guaranteed to find the shortest path.

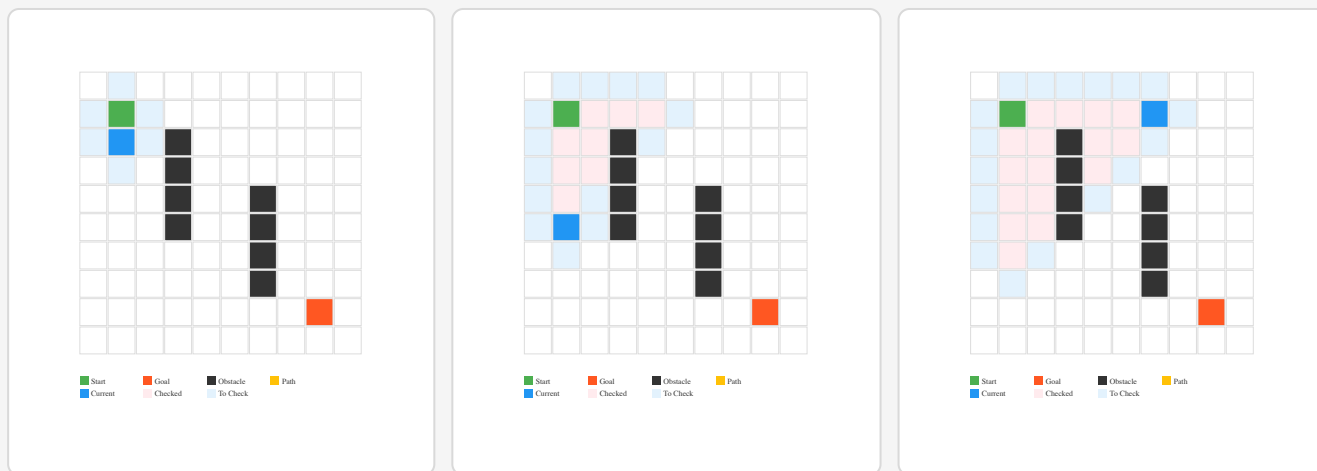
A* explores the grid by calculating a score for each cell. A cell is a single square in the grid (like a tile in your game world). Cost here refers to how expensive it is to travel somewhere.

- **g-cost**: Actual distance traveled from start to this cell
- **h-cost**: Heuristic - Estimated distance from this cell to goal
- **f-cost**: Total cost - g + h (total estimated cost if we take this path)

The algorithm always explores the cell with the lowest f-cost first. This makes it “greedy” (always picking what looks best) but informed (using the heuristic to guide the search).

A* Pathfinding

Path found! Length: 14 steps



The **green** cell is where the enemy starts, and the **red** cell is the player's position. **Black obstacles** block the way, forcing the algorithm to navigate around them.

As A* explores, it marks cells in **light blue** (candidates to check next) and **pink** (already checked and ruled out). The **dark blue** cell shows what A* is currently examining, and once the path is found, it lights up in **yellow**.

Notice how the algorithm doesn't waste time exploring cells that are obviously far from the goal, it intelligently prioritizes the most promising directions!

Adding the Pathfinding Crate

We'll use the `pathfinding` crate for the A* algorithm.

Update your `Cargo.toml`:

```
[dependencies]
bevy = "0.19"
bevy_procedural_tilemaps = "0.4.0"
bevy_common_assets = { version = "0.17", features = ["ron"] }
serde = { version = "1.0", features = ["derive"] }
rand = "0.8"
pathfinding = "4.9" # Add this line
```

Now that we have the pathfinding algorithm, we need to build the **infrastructure** to use it. This involves three pieces:

1. **EnemyPath Component**: Stores the calculated path and tracks which waypoint the enemy is currently moving toward
2. **CollisionMap Methods**: Extends our collision system with pathfinding capabilities
3. **AI System**: Uses both to make enemies intelligently navigate to the player

Let's build each piece step by step.

What are waypoints?

Waypoints are like breadcrumbs along a path. Instead of storing "move 3 units north, then 2 units east," we store actual positions like `[(100, 50), (150, 50), (150, 100)]`. The enemy moves toward the first waypoint, and when it gets close enough (16 units), it advances to the next one. This makes pathfinding flexible—waypoints can adapt to any terrain shape.

How Waypoints Work

Following waypoint 2 (40px away)



The enemy (green) follows each waypoint in sequence. When it gets within 16 pixels of the current waypoint, it moves to the next one. Eventually it navigates the path to reach the player (red)!

The EnemyPath Component

So let's create an `EnemyPath` component that caches the waypoints, tracks the current position along the path, and manages a recalculation timer.

Update `src/enemy/components.rs` to add this at the end:

```
// src/enemy/components.rs
// Add this after the AIBehavior implementation

/// Cached path for enemy navigation
#[derive(Component, Default)]
pub struct EnemyPath {
    /// Waypoints in world coordinates
    pub waypoints: Vec<Vec2>,
    /// Current waypoint index
    pub current_index: usize,
    /// Timer for path recalculation
    pub recalc_timer: f32,
}

impl EnemyPath {
    /// Distance threshold to consider a waypoint reached
    pub const WAYPOINT_THRESHOLD: f32 = 16.0;
    /// How often to recalculate path (seconds)
    pub const RECALC_INTERVAL: f32 = 0.5;

    /// Get current waypoint position
    pub fn current_waypoint(&self) -> Option<Vec2> {
        self.waypoints.get(self.current_index).copied()
    }

    /// Advance to next waypoint, returns true if path completed
    pub fn advance(&mut self) -> bool {
        self.current_index += 1;
        self.current_index >= self.waypoints.len()
    }

    /// Set a new path (skips first waypoint since it's the starting position)
    pub fn set_path(&mut self, waypoints: Vec<Vec2>) {
        // Skip waypoint 0 - it's the enemy's current position
        // This prevents jitter from briefly facing backwards
        let new_waypoints = if waypoints.len() > 1 {
            waypoints[1..].to_vec()
        } else {
            waypoints
        }
    }
}
```



```

};

// If we have an existing path, check if new path is similar
// This prevents flickering when paths are recalculated
if let Some(current_target) = self.current_waypoint() {
    if let Some(new_first) = new_waypoints.first() {
        // If new first waypoint is close to our current target,
        // keep the current path - we're already heading the right way
        if current_target.distance(*new_first) < Self::WAYPOINT_THRESHOLD * 1.5 {
            return;
        }
    }
}

self.waypoints = new_waypoints;
self.current_index = 0;
}

/// Check if we have a valid path
pub fn has_path(&self) -> bool {
    !self.waypoints.is_empty() && self.current_index < self.waypoints.len()
}
}

```

Let's break down each piece:

- `waypoints`: Stores the path as a vector of world positions. Once we calculate a path, we cache it here.
- `current_index`: Tracks which waypoint we're heading toward. When we reach waypoint 0, we increment to waypoint 1, etc.
- `recalc_timer`: Counts down from 0.5 seconds. When it hits zero, we recalculate the path (in case the player moved).
- `current_waypoint()`: Returns the waypoint we're currently heading toward (or None if we've completed the path).
- `advance()`: Moves to the next waypoint, returns true if we've completed the entire path.
- `set_path()`: Stores a new path with smart optimizations:
 - Skips waypoint 0 (the enemy's current position) to avoid jitter
 - Checks if the new path is similar to the current one - if so, keeps the existing path to reduce flickering
- `has_path()`: Returns true if we have a valid, incomplete path to follow.

Why cache paths?

Running A* every frame would slow down the game. By recalculating every 0.5 seconds, we get smooth movement with minimal performance cost.

Why have a waypoint threshold?

Due to movement speed and frame timing, an enemy might never land exactly on a waypoint. The 16-unit threshold lets us consider it "reached" when close enough.

Now that we can store paths, we need to **calculate** them. But who does the pathfinding? The `CollisionMap`!

Why does `CollisionMap` do pathfinding? Shouldn't the enemy figure out its own path?

Good question! The `CollisionMap` already knows which tiles are walkable and which aren't, it has all the terrain data. Rather than duplicating this knowledge in the enemy AI, we extend `CollisionMap` with pathfinding methods. This way:

- **Enemies** just ask: "Give me a path to position X"
- **CollisionMap** responds: "Here's the list of waypoints to get there"

This keeps our code clean and follows the principle: *the component with the data provides the operations on that data.*

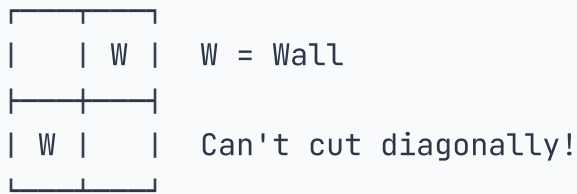
Extending CollisionMap for Pathfinding

Our collision map knows about walkable tiles, but it doesn't know how to *find a path* through them. We need to:

1. Get valid neighboring cells (ensuring diagonal moves don't cut corners through walls)
2. Find the nearest walkable tile if the target is blocked
3. Run A* pathfinding to calculate the optimal path

What does “don't cut corners” mean?

Imagine two walls meeting at a corner diagonally. Without proper checks, an enemy could move diagonally through that gap, which looks like walking through walls! We prevent this by only allowing diagonal movement when both adjacent cardinal cells are also walkable.



So let's add these three methods to `CollisionMap` that work together to provide intelligent pathfinding.

Open `src/collision/map.rs` and add these methods at the end of the `impl CollisionMap` block:

First, add the import at the top of `src/collision/map.rs`:

```
// src/collision/map.rs
use pathfinding::prelude::astar;
```

Now add these methods at the end of the `impl CollisionMap` block:

```
// src/collision/map.rs
// Add at the end of impl CollisionMap, before the closing brace

/// Get walkable neighboring grid cells (8 directions)
/// Diagonal movement only allowed if both adjacent cardinals are clear
pub fn get_neighbors(&self, pos: IVec2) -> Vec<IVec2> {
    let mut neighbors = Vec::new();

    // Cardinal directions (always allowed if walkable)
    let cardinals = [
        IVec2::new(0, 1), IVec2::new(0, -1), IVec2::new(-1, 0), IVec2::new(1, 0),
    ];

    for dir in cardinals {
        let neighbor = pos + dir;
        if self.is_walkable(neighbor.x, neighbor.y) {
            neighbors.push(neighbor);
        }
    }

    // Diagonal directions - only if both adjacent cardinals are clear
    // This prevents corner-cutting through diagonal walls
    let diagonals = [
        IVec2::new(-1, 1), IVec2::new(-1, 0), IVec2::new(0, 1)), // Up-Left
        IVec2::new(1, 1), IVec2::new(1, 0), IVec2::new(0, 1)), // Up-Right
        IVec2::new(-1, -1), IVec2::new(-1, 0), IVec2::new(0, -1)), // Down-Left
    ];
```

```

        (IVec2::new(1, -1), IVec2::new(1, 0), IVec2::new(0, -1)), // Down-Right
    ];

    for (diagonal, adj1, adj2) in diagonals {
        let diag_pos = pos + diagonal;
        let adj1_pos = pos + adj1;
        let adj2_pos = pos + adj2;

        // Only allow diagonal if destination AND both adjacent cells are walkable
        if self.is_walkable(diag_pos.x, diag_pos.y)
            && self.is_walkable(adj1_pos.x, adj1_pos.y)
            && self.is_walkable(adj2_pos.x, adj2_pos.y)
        {
            neighbors.push(diag_pos);
        }
    }

    neighbors
}

/// Find path using A* algorithm
pub fn find_path(&self, start: Vec2, goal: Vec2) -> Option<Vec<Vec2>> {
    use pathfinding::prelude::astar;

    let start_grid = self.world_to_grid(start);
    let goal_grid = self.world_to_grid(goal);

    if !self.is_walkable(start_grid.x, start_grid.y) {
        return None;
    }

    let actual_goal = if self.is_walkable(goal_grid.x, goal_grid.y) {
        goal_grid
    } else {
        self.find_nearest_walkable(goal_grid)?
    };

    let result = astar(
        &start_grid,
        |pos| {
            let pos = *pos;
            self.get_neighbors(pos).into_iter().map(move |n| {
                let cost = if (n.x - pos.x).abs() + (n.y - pos.y).abs() == 2 {
                    14u32 // Diagonal
                } else {
                    10u32 // Cardinal
                };
                (n, cost)
            })
        },
        |pos| {
            let dx = (pos.x - actual_goal.x).abs();
            let dy = (pos.y - actual_goal.y).abs();
            ((dx + dy) * 10) as u32
        },
        |pos| *pos == actual_goal,
    );

    result.map(|(path, _cost)| {
        path.into_iter().map(|p| self.grid_to_world(p.x, p.y)).collect()
    })
}

/// Find nearest walkable cell
pub fn find_nearest_walkable(&self, pos: IVec2) -> Option<IVec2> {

```

```

        for radius in 1i32..10 {
            for dx in -radius..radius {
                for dy in -radius..radius {
                    if dx.abs() == radius || dy.abs() == radius {
                        let check = IVec2::new(pos.x + dx, pos.y + dy);
                        if self.is_walkable(check.x, check.y) {
                            return Some(check);
                        }
                    }
                }
            }
        }
        None
    }

    /// Find nearest position where a circle of the given radius is fully clear.
    /// Searches expanding rings up to 20 tiles from the given world position.
    /// Returns a world-space position (tile center) or None if nothing found.
    pub fn find_nearest_clear_position(&self, world_pos: Vec2, radius: f32) -> Option<Vec2> {
        let grid_pos = self.world_to_grid(world_pos);

        for ring in 0i32..20 {
            for dx in -ring..ring {
                for dy in -ring..ring {
                    if ring > 0 && dx.abs() != ring && dy.abs() != ring {
                        continue; // Only check the ring perimeter
                    }
                    let candidate_grid = IVec2::new(grid_pos.x + dx, grid_pos.y + dy);
                    let candidate_world = self.grid_to_world(candidate_grid.x, candidate_grid.y);
                    if self.is_circle_clear(candidate_world, radius) {
                        return Some(candidate_world);
                    }
                }
            }
        }
        None
    }
}

```

Let's break down the pathfinding code:

1. Neighbor Detection (`get_neighbors`):

- First checks all 4 cardinal directions (up/down/left/right)
- Then checks all 4 diagonal directions **only if** both adjacent cardinals are walkable
- This prevents enemies from “cutting corners” through diagonal walls

2. Nearest Walkable Search (`find_nearest_walkable`):

- Uses a spiral search pattern starting from the target
- Checks increasingly larger squares until it finds a walkable tile
- Returns None if nothing found within 10 tiles (player might be unreachable)

2b. Nearest Clear Position (`find_nearest_clear_position`):

- Like `find_nearest_walkable`, but validates the **full collider circle** at each candidate, not just the tile type
- Used for spawning, where we need to guarantee the entity won't overlap any obstacle after placement
- Searches up to 20 tiles and returns a world-space position ready to use

3. A* Pathfinding (`find_path`):

- Converts world positions to grid coordinates
- Validates start position; if goal is blocked, finds nearest walkable alternative

- Runs A* algorithm using:
 - **Successors**: Get neighbors with movement cost (10 for cardinal, 14 for diagonal)
 - **Heuristic**: Manhattan distance estimate to goal
 - **Success**: Checks if the current position is the goal
- Converts resulting grid path back to world coordinates

Understanding the `astar` Function

The A* function doesn't care if you're navigating a dungeon, flying a spaceship, or routing a delivery truck, it just needs to know "what positions exist" (nodes) and "how expensive is it to move between them" (costs).

Hence it's just a generic function that works with any node type `N` and cost type `C`. Node in our situation, is a position on the board where the enemy can stand (in our case, grid coordinates like `(5, 3)`)

```
// Pseudo code, don't use
pub fn astar<...>{
  start: &N,
  successors: FN,
  heuristic: FH,
  success: FS,
} -> Option<(Vec<N>, C)>
```

What matters are the **four parameters**:

1. `start: &N` - Enemy's starting position
2. `successors: FN` - A function that answers "where can I move from here?" It helps us know from the enemy's current tile, which adjacent tiles can it walk to and how much does each move cost?
3. `heuristic: FH` - A function that guesses distance to target or "how many tiles away is the player from here?". It helps enemy prioritize which direction to explore first.
4. `success: FS` - A function that checks "are we there yet?". It returns true when the enemy has reached the player's position.

The function returns `Option<(Vec<N>, C)>`:

- `Some((path, cost))` A list of grid positions the enemy should walk through to reach the player, plus the total cost.
- `None` if no path exists or player is completely blocked off by obstacles, enemy can't reach them.

Now let's see how we provide these functions using **closures**:

Understanding Closures in the A* Implementation:

You might notice the `astar` function looks a bit odd, it takes **closures** (anonymous functions) as arguments. This is a powerful Rust pattern called **higher-order functions**.

```
// Pseudo code, don't use
let result = astar(
  &start_grid,           // Starting position
  |pos| { /* closure 1 */ }, // How to get neighbors and costs
  |pos| { /* closure 2 */ }, // How to estimate distance to goal
  |pos| { /* closure 3 */ }, // How to check if we reached the goal
);
```

Why closures? The `pathfinding` crate's `astar` function is generic - it works for ANY type of pathfinding problem (chess moves, graph traversal, tile grids, etc.). By accepting closures, it lets YOU define:

1. What counts as a “neighbor” (straight lines only? diagonals too? teleportation?)
2. What the movement costs are (flat terrain? hills?)
3. How to estimate distance (Manhattan? Euclidean?)
4. What the goal condition is

How do closures capture data?

Notice that the closures reference `self` and `actual_goal`, variables from the outer scope. Rust closures can “capture” these variables:

```
// Pseudo code don't use
|pos| {
    let pos = *pos;
    self.get_neighbors(pos) // Uses 'self' from outer scope!
        .into_iter()
        .map(move |n| {
            // Calculate cost based on movement type
            let cost = if (n.x - pos.x).abs() + (n.y - pos.y).abs() == 2 {
                14u32 // Diagonal movement
            } else {
                10u32 // Cardinal movement
            };
            (n, cost)
        })
}
```

What's the `move` keyword?

In the inner closure (the one with `.map(move |n| ...)`), the `move` keyword forces the closure to **take ownership** of the captured variable `pos`. Without `move`, the closure would borrow `pos`, but since we're returning this closure from `.map()`, it needs to own its data. Think of it like: “This closure is leaving home, so it needs to pack its own copy of `pos`.”

How does borrowing work with `self.get_neighbors`?

Great question! When the closure captures `self`, it's actually borrowing it:

- The outer closure borrows `self` immutably (just reading from it)
- `self.get_neighbors(pos)` only needs to read the collision map data
- The `astar` function promises to only call the closure while we're in the `find_path` method
- No ownership is transferred, so no borrowing rules are broken!

The borrowing is temporary and safe because:

1. We're not trying to modify `self` (immutable borrow is fine)
2. The closure only exists during the `astar` call, not after
3. Rust's borrow checker verifies this at compile time

This closure is essentially a custom function that says: “For any position, here's how to find its neighbors and the cost to reach them.”

The AI System with Pathfinding

Now we can implement AI that uses pathfinding.

We need enemies to intelligently pursue the player by:

1. Detecting when the player is in range
2. Using pathfinding to navigate around obstacles
3. Stopping to attack when close enough
4. Handling edge cases (player too far, pathfinding fails, etc.)
5. Preventing jitter and oscillation with smart state transitions

Build an AI system that manages three behavior states (idle, following, attacking) with smooth transitions between them.

Create `src/enemy/ai.rs`:

```
// src/enemy/ai.rs
use super::components::{AIBehavior, Enemy, EnemyPath};
use crate::characters::{
    config::CharacterEntry,
    facing::Facing,
    input::Player,
    physics::{Velocity, calculate_velocity},
    state::CharacterState,
};
use crate::collision::CollisionMap;
use bevy::prelude::*;

/// AI system that makes enemies follow the player using A* pathfinding
pub fn enemy_follow_player(
    time: Res<Time>,
    collision_map: Option<Res<CollisionMap>>,
    mut enemy_query: Query<
        (
            &Transform,
            &mut CharacterState,
            &mut Velocity,
            &mut Facing,
            &CharacterEntry,
            &AIBehavior,
            &mut EnemyPath,
        ),
        With<Enemy>,
    >,
    player_query: Query<&Transform, With<Player>>,
) {
    let Ok(player_transform) = player_query.single() else {
        return;
    };

    let Some(collision_map) = collision_map else {
        return;
    };

    let player_pos = player_transform.translation.truncate();
    let delta = time.delta_secs();

    for (enemy_transform, mut state, mut velocity, mut facing, character, ai, mut path) in
        enemy_query.iter_mut()
    {
        let enemy_pos = enemy_transform.translation.truncate();
        let to_player = player_pos - enemy_pos;
        let distance = to_player.length();

        // Outside detection range - go idle
        if distance > ai.detection_range {
            if *state != CharacterState::Idle {
                *state = CharacterState::Idle;
            }
            *velocity = Velocity::ZERO;
            continue;
        }

        // Within attack range - stop and attack
        // Use hysteresis: different threshold for staying vs entering attack mode
        // This prevents oscillation at the boundary
```

```

let attack_threshold = if *state == CharacterState::Idle {
    ai.attack_range + 20.0 // Stay in attack mode even if player moves slightly away
} else {
    ai.attack_range // Enter attack mode at normal range
};

if distance <= attack_threshold {
    if *state != CharacterState::Idle {
        *state = CharacterState::Idle;
    }
    *velocity = Velocity::ZERO;

    // Face the player while attacking
    let direction = to_player.normalize_or_zero();
    if direction != Vec2::ZERO {
        let new_facing = Facing::from_velocity(direction);
        if *facing != new_facing {
            *facing = new_facing;
        }
    }
    continue;
}

// Need to move toward player - use pathfinding
path.recalc_timer -= delta;

// Recalculate path if we don't have one
if !path.has_path() {
    if let Some(waypoints) = collision_map.find_path(enemy_pos, player_pos) {
        path.set_path(waypoints);
        path.recalc_timer = EnemyPath::RECALC_INTERVAL;
    }
} else if path.recalc_timer <= 0.0 {
    // Periodically update existing path
    path.recalc_timer = EnemyPath::RECALC_INTERVAL;

    if let Some(waypoints) = collision_map.find_path(enemy_pos, player_pos) {
        path.set_path(waypoints);
    }
}

// Follow current waypoint
if let Some(waypoint) = path.current_waypoint() {
    let to_waypoint = waypoint - enemy_pos;
    let waypoint_distance = to_waypoint.length();

    // Check if we reached the waypoint
    if waypoint_distance < EnemyPath::WAYPOINT_THRESHOLD {
        path.advance();
    }

    // Recalculate direction for current waypoint (might have advanced)
    if let Some(current_wp) = path.current_waypoint() {
        let to_waypoint = current_wp - enemy_pos;
        let direction = to_waypoint.normalize_or_zero();

        // Update state
        if *state != CharacterState::Walking {
            *state = CharacterState::Walking;
        }

        // Update facing
        if direction != Vec2::ZERO {
            let new_facing = Facing::from_velocity(direction);
            if *facing != new_facing {

```

```

        *facing = new_facing;
    }
}

// Calculate velocity toward waypoint
*velocity = calculate_velocity(*state, direction, character);
}
} else {
    // No path available - fallback to direct movement
    let direction = to_player.normalize_or_zero();

    if *state != CharacterState::Walking {
        *state = CharacterState::Walking;
    }

    if direction != Vec2::ZERO {
        let new_facing = Facing::from_velocity(direction);
        if *facing != new_facing {
            *facing = new_facing;
        }
    }

    *velocity = calculate_velocity(*state, direction, character);
}
}
}

```

How does this work?

The query fetches all enemies with their path component. We also get the `CollisionMap` resource and player's position.

For each enemy, we check three ranges:

- **Too far** (beyond detection range): Go idle
- **In attack range**: Stop moving, face player
- **Between ranges**: Use pathfinding to approach

Attack range hysteresis: We use different thresholds for entering vs staying in attack mode:

- Enter attack: `distance <= attack_range` (150)
- Stay attacking: `distance <= attack_range + 20` (170)

This prevents **oscillation**, a buggy behavior where the enemy rapidly flickers between states. Imagine the player is exactly 150 units away (the attack range). The enemy enters attack mode, but as soon as it stops moving, the distance might increase to 151 units, triggering follow mode again. This causes the enemy to jitter back and forth between “attack” and “follow” dozens of times per second! By adding a buffer zone, the enemy must move further away (170 units) before switching back to following.

Path management: We separate path creation from path updates:

1. **No path** → Create immediately and reset timer
2. **Have path, timer expired** → Update periodically (every 0.5s)

This prevents the path from resetting mid-frame when enemies complete waypoints.

Path similarity check: Before replacing an existing path, we check if the new path's first waypoint is close to our current target. If so, we keep the existing path since we're already heading the right direction. This reduces flickering during periodic updates.

Skipping the first waypoint: A* includes the starting position as waypoint 0. If we don't skip it, enemies briefly face backward (toward their own position) before turning to the actual destination. By skipping waypoint 0 in `set_path()`, enemies immediately move toward the first real destination.

Waypoint advancement: When we reach a waypoint, we call `advance()` and immediately recalculate the direction for the NEW current waypoint in the same frame. This prevents jitter from skipping frames between waypoints.

Fallback behavior: If pathfinding fails, we fall back to direct movement. This ensures enemies don't get completely stuck.

Why check if state changed?

Bevy's change detection tracks when components are modified. If we set `*state = new_state` every frame even when the value doesn't change, Bevy thinks the state changed. This triggers systems that run on `Changed<CharacterState>`, like animation updates. By only updating when the value actually changes, we avoid unnecessary work.

Making Enemies Attack

Enemies can follow the player, but they need to attack. We'll create a combat system that fires when enemies are in range and their cooldown is ready.

The combat system needs to:

1. Tick the cooldown timer
2. Check if the player is in attack range
3. Fire a projectile when ready
4. Reset the cooldown

Create `src/enemy/combat.rs`:

```
// src/enemy/combat.rs
use super::components::{AIBehavior, Enemy, EnemyCombat};
use crate::characters::input::Player;
use crate::combat::systems::spawn_projectile;
use bevy::prelude::*;

/// System that handles enemy attacks
pub fn enemy_attack(
    mut commands: Commands,
    time: Res<Time>,
    mut enemy_query: Query<&GlobalTransform, &mut EnemyCombat, &AIBehavior>, With<Enemy>>,
    player_query: Query<&Transform, With<Player>>,
) {
    let Ok(player_transform) = player_query.single() else {
        return;
    };

    for (enemy_transform, mut combat, ai) in enemy_query.iter_mut() {
        // Tick the cooldown timer
        combat.cooldown.tick(time.delta());

        let enemy_pos = enemy_transform.translation();
        let player_pos = player_transform.translation();

        // Calculate distance to player
        let distance = enemy_pos.distance(player_pos);

        // Attack if in range and cooldown is ready
        if distance <= ai.attack_range && combat.cooldown.elapsed() >= combat.cooldown.duration() {
            // Calculate direction to player
            let to_player = (player_pos - enemy_pos).normalize();
            let spawn_position = enemy_pos + to_player * 5.0;

            // Get visuals from power type (using actual direction to player)
            let visuals = combat.power_type.visuals(to_player);
```

```

        // Spawn projectile (reuse existing function!)
        spawn_projectile(&mut commands, spawn_position, combat.power_type, &visuals);

        // Reset cooldown for next attack
        combat.cooldown.reset();

        info!("Enemy fired {:?} projectile at player!", combat.power_type);
    }
}
}

```

How does the attack system work?

Every frame, we tick each enemy's cooldown timer. We check if `elapsed() >= duration()` to see if the cooldown is ready. This allows enemies to attack immediately when spawned (the timer starts in a finished state) and then every 2 seconds after.

We calculate the actual direction vector from enemy to player using `(player_pos - enemy_pos).normalize()`. This ensures projectiles always aim at the player's current position, not just in a cardinal direction.

We check if the player is within attack range. If both conditions are met (cooldown ready AND player in range), we fire a projectile toward the player and reset the cooldown.

The `spawn_projectile` function is the same one the player uses. We made it public in `combat/systems.rs` so both players and enemies can call it. This is code reuse at its best.

Fixing Player Spawning

Before we spawn enemies, we need to fix a critical bug in how the player spawn. Right now, the player spawns at `(0, 0)` during `Startup`, before the collision map exists. This means the player might spawn on an obstacle (rock, tree, water) and get stuck!

To prevent this, let's use a **validate-then-spawn** pattern:

1. Load character assets → `Startup`
2. Wait for collision map → `Update` (run condition)
3. Validate position → Check `is_circle_clear`
4. Spawn at valid position

This ensures characters **never** spawn on obstacles.

Let's update `src/characters/spawn.rs`. We'll split the old system into two cleaner functions:

First, add these imports at the top of the file:

```

// src/characters/spawn.rs
use crate::collision::CollisionMapBuilt; // Add this line
use crate::config::player::COLLIDER_RADIUS; // Add this line
use crate::collision::CollisionMap; // Add this line

```

Remove these old functions:

```

//DELETE spawn_player
//DELETE initialize_player_character

```

Add these new functions:

```

// src/characters/spawn.rs
// Update the resource name at the top of the file
/// Resource to track if player has been spawned (prevents spawning multiple times)

```

```

#[derive(Resource, Default, PartialEq, Eq)]
pub struct PlayerSpawned(pub bool);

/// Get a valid spawn position, checking collision map and adjusting if needed
fn get_valid_spawn_position(collision_map: &CollisionMap, desired_pos: Vec2) -> Vec2 {
    // Use the same radius as the runtime collision system
    if collision_map.is_circle_clear(desired_pos, COLLIDER_RADIUS) {
        return desired_pos;
    }

    // Find nearest position where the full collider circle is clear
    if let Some(clear_pos) = collision_map.find_nearest_clear_position(desired_pos, COLLIDER_RADIUS)
    {
        info!(
            "Adjusted player spawn from {:?} to {:?} (was on obstacle)",
            desired_pos, clear_pos
        );
        return clear_pos;
    }

    // Fallback to original
    warn!("Could not find walkable spawn position near {:?}", desired_pos);
    desired_pos
}

// Replace spawn_player with this
/// Load character assets at startup (before collision map is built)
pub fn load_character_assets(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut character_index: ResMut<CurrentCharacterIndex>,
) {
    // Load the characters list
    let characters_list_handle: Handle<CharactersList> =
        asset_server.load("characters/characters.ron");

    // Store the handle in a resource
    commands.insert_resource(CharactersListResource {
        handle: characters_list_handle,
    });

    // Initialize with first character
    character_index.index = 0;

    info!("Character assets loading started");
}

// Add this new function
/// Spawn player at a valid position AFTER collision map is built
pub fn spawn_player_at_valid_position(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
    characters_lists: Res<Assets<CharactersList>>,
    character_index: Res<CurrentCharacterIndex>,
    characters_list_res: Option<Res<CharactersListResource>>,
    collision_map: Option<Res<CollisionMap>>,
    mut player_spawned: ResMut<PlayerSpawned>,
) {
    // Wait for collision map
    let Some(collision_map) = collision_map else {
        return;
    };

    // Wait for character list resource

```



```

let Some(characters_list_res) = characters_list_res else {
    return;
};

// Get the character list asset
let Some(characters_list) = characters_lists.get(&characters_list_res.handle) else {
    return;
};

if character_index.index >= characters_list.characters.len() {
    warn!("Invalid character index: {}", character_index.index);
    return;
}

let character_entry = &characters_list.characters[character_index.index];

// Calculate valid spawn position
let desired_pos = Vec2::new(0.0, 0.0);
let valid_pos = get_valid_spawn_position(&collision_map, desired_pos);

// Create sprite
let texture = asset_server.load(&character_entry.texture_path);
let layout = create_character_atlas_layout(&mut atlas_layouts, character_entry);
let sprite = Sprite::from_atlas_image(texture, TextureAtlas { layout, index: 0 });

// Spawn player with all components at valid position
commands.spawn((
    Player,
    Transform::from_translation(Vec3::new(valid_pos.x, valid_pos.y, PLAYER_Z_POSITION))
        .with_scale(Vec3::splat(PLAYER_SCALE)),
    sprite,
    AnimationController::default(),
    CharacterState::default(),
    Velocity::default(),
    Facing::default(),
    Collider::default(),
    PlayerCombat::default(),
    AnimationTimer(Timer::from_seconds(
        DEFAULT_ANIMATION_FRAME_TIME,
        TimerMode::Repeating,
    )),
    character_entry.clone(),
));

// Mark player as spawned
player_spawned.0 = true;
info!("Player spawned at validated position {:?}", valid_pos);
}

```

What changed?

1. **load_character_assets** (Startup) - Only loads the asset, doesn't spawn anything
2. **spawn_player_at_valid_position** (Update) - Waits for collision map, validates position, then spawns with ALL components in one step
3. **get_valid_spawn_position** - Shared helper that checks `is_circle_clear` (not just single tile!)

Notice we use `COLLIDER_RADIUS` – the same constant the runtime collision system uses. This is important: if spawn validation checks a smaller radius than what the movement system enforces, an entity could pass the spawn check but still overlap an obstacle at runtime and get stuck. Using a single source of truth eliminates that mismatch. The fallback `find_nearest_clear_position` also validates the full circle, not just the tile type.

Updating the Plugin

Now update `src/characters/mod.rs`:

```
// src/characters/mod.rs
// Update imports
use spawn::PlayerSpawned; // Add this line
use crate::collision::CollisionMapBuilt; // Add this line

impl Plugin for CharactersPlugin {
    fn build(&self, app: &mut App) {
        app.add_plugins(RonAssetPlugin::<CharactersList>::new(&["characters.ron"]))
            .init_resource::<spawn::CurrentCharacterIndex>()
            .init_resource::<PlayerSpawned>() // Add this line
            // Load character assets at startup (before collision map)
            .add_systems(Startup, spawn::load_character_assets) // Change function name
            // Spawn player at valid position AFTER collision map is built
            // Add this
            .add_systems(
                Update,
                spawn::spawn_player_at_valid_position
                    .run_if(resource_equals(CollisionMapBuilt(true)))
                    .run_if(resource_equals(PlayerSpawned(false)))
                    .run_if(in_state(GameState::Playing)),
            )
            .add_systems(
                Update,
                (
                    input::handle_player_input,
                    spawn::switch_character,
                    input::update_jump_state,
                    animation::on_state_change_update_animation,
                    collider::validate_movement,
                    physics::apply_velocity,
                    rendering::update_player_depth,
                    animation::animations_playback,
                )
                    .chain()
                    .run_if(in_state(GameState::Playing)),
            );
    }
}
```

And one more fix in `src/state/mod.rs` - remove the old initialization from `OnExit>Loading)`:

```
// src/state/mod.rs
.add_systems(OnExit(GameState::Loading),
    loading::despawn_loading_screen,
    // REMOVE: crate::characters::spawn::initialize_player_character,
)
```

Adding Enemy Configuration

Now, let's add configuration constants for enemies in `src/config.rs`:

```
// src/config.rs
pub mod player {
    // ... existing player config ...
}

pub mod enemy {
    /// Z-position for enemy rendering (same as player for consistent layering)
    pub const ENEMY_Z_POSITION: f32 = 20.0;
```

```

    /// Visual scale of enemy sprites (same as player for consistency)
    pub const ENEMY_SCALE: f32 = 1.2;
} // Add this

pub mod pickup {
    // ... existing pickup config ...
}
// ... rest of config ...

```

We create a separate `enemy` module to keep enemy constants organized and avoid confusion with player constants.

Spawning Enemies

Now we need to actually create enemy entities in the world. We'll write a spawn function that creates an enemy with all the necessary components, then a system that spawns test enemies - using the **exact same pattern** we just used for the player.

The spawn function needs to:

1. Load the character configuration from `characters.ron`
2. Create the sprite atlas
3. Spawn an entity with all required components

Create `src/enemy/spawn.rs`:

```

// src/enemy/spawn.rs
use super::components::{AIBehavior, Enemy, EnemyCombat, EnemyPath};
use crate::characters::{
    animation::{AnimationController, AnimationTimer, DEFAULT_ANIMATION_FRAME_TIME},
    collider::Collider,
    config::{CharacterEntry, CharactersList},
    facing::Facing,
    physics::Velocity,
    spawn::CharactersListResource, // Add this line
    state::CharacterState,
};
use crate::collision::CollisionMap;
use crate::config::enemy::{ENEMY_SCALE, ENEMY_Z_POSITION};
use crate::config::player::COLLIDER_RADIUS;
use bevy::prelude::*;

/// Spawn an enemy at the given position
pub fn spawn_enemy(
    commands: &mut Commands,
    asset_server: &AssetServer,
    atlas_layouts: &mut ResMut<Assets<TextureAtlasLayout>>,
    characters_list: &CharactersList,
    position: Vec3,
    character_name: &str,
) -> Option<Entity> {
    // Find the character entry by name
    let character_entry = characters_list
        .characters
        .iter()
        .find(|c| c.name == character_name)?;

    // Create atlas layout
    let max_row = character_entry.calculate_max_animation_row();
    let layout = atlas_layouts.add(TextureAtlasLayout::from_grid(
        UVec2::splat(character_entry.tile_size),
        character_entry.atlas_columns as u32,
    ));

```

```

        (max_row + 1) as u32,
        None,
        None,
    ));

    // Load texture
    let texture = asset_server.load(&character_entry.texture_path);

    // Create sprite
    let sprite = Sprite::from_atlas_image(texture, TextureAtlas { layout, index: 0 });

    // Spawn enemy entity with all necessary components
    let entity = commands
        .spawn((
            Enemy,
            sprite,
            Transform::from_translation(position).with_scale(Vec3::splat(ENEMY_SCALE)),
            GlobalTransform::default(),
            AnimationController::default(),
            CharacterState::default(),
            Velocity::default(),
            Facing::default(),
            Collider::default(),
            EnemyCombat::default(),
            AIBehavior::default(),
            EnemyPath::default(), // Add this line
            AnimationTimer(Timer::from_seconds(
                DEFAULT_ANIMATION_FRAME_TIME,
                TimerMode::Repeating,
            )),
            character_entry.clone(),
        ))
        .id();

    info!("Spawned enemy '{}' at {:?}", character_name, position);

    Some(entity)
}

/// Resource to track if enemies have been spawned
#[derive(Resource, Default, PartialEq, Eq)]
pub struct EnemiesSpawned(pub bool);

/// Validate and adjust spawn position to ensure it's on a walkable tile
fn get_valid_spawn_position(collision_map: &CollisionMap, desired_pos: Vec2) -> Vec2 {
    // Use the same radius as the runtime collision system
    if collision_map.is_circle_clear(desired_pos, COLLIDER_RADIUS) {
        return desired_pos;
    }

    // Find nearest position where the full collider circle is clear
    if let Some(clear_pos) = collision_map.find_nearest_clear_position(desired_pos, COLLIDER_RADIUS)
    {
        info!(
            "Adjusted spawn from {:?} to {:?} (was on obstacle)",
            desired_pos, clear_pos
        );
        return clear_pos;
    }

    // Fallback to original (shouldn't happen in a valid map)
    warn!("Could not find walkable spawn position near {:?}", desired_pos);
    desired_pos
}

```

```

/// System to spawn test enemies when collision map is ready
pub fn spawn_test_enemies(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
    characters_lists: Res<Assets<CharactersList>>,
    characters_list_res: Option<Res<CharactersListResource>>, // Add this line
    collision_map: Option<Res<CollisionMap>>,
    mut enemies_spawned: ResMut<EnemiesSpawned>,
) {
    // Wait for collision map
    let Some(collision_map) = collision_map else {
        return;
    };

    // Wait for character list resource
    let Some(characters_list_res) = characters_list_res else {
        return;
    };

    // Get the character list asset
    let Some(characters_list) = characters_lists.get(&characters_list_res.handle) else {
        return;
    };

    // Define desired spawn positions
    let spawn_positions = [Vec2::new(200.0, 0.0), Vec2::new(-200.0, 100.0)];

    for desired_pos in spawn_positions {
        // Validate position against collision map
        let valid_pos = get_valid_spawn_position(&collision_map, desired_pos);

        spawn_enemy(
            &mut commands,
            &asset_server,
            &mut atlas_layouts,
            characters_list,
            Vec3::new(valid_pos.x, valid_pos.y, ENEMY_Z_POSITION),
            "graveyard_reaper",
        );
    }

    // Mark enemies as spawned so this system doesn't run again
    enemies_spawned.0 = true;
    info!("Enemies spawned with validated positions");
}

```

What's happening in spawn_enemy?

This function follows the same pattern as the player spawn system. Both players and enemies need the same components for movement, animation, and collision.

Both player and enemy spawning now follow the same “validate-then-spawn” approach:

1. **Wait for collision map** (run condition: `CollisionMapBuilt(true)`)
2. **Check spawn position** (using `is_circle_clear` with `COLLIDER_RADIUS`)
3. **Find nearest clear position if blocked** (using `find_nearest_clear_position`, which validates the full collider circle)
4. **Spawn with all components** (once valid position is determined)

The key differences between player and enemy entities are:

- **Marker component:** `Player` vs `Enemy`
- **Behavior components:** `PlayerCombat` vs `EnemyCombat` + `AIBehavior`

- **Context-specific:** Player gets inventory, enemies get pathfinding

Notice how we reuse `CharacterEntry` from `characters.ron`. The “graveyard_reaper” character we defined in Chapter 3 works perfectly for enemies. Same sprite system, same animation definitions, zero duplication.

Why return `Option<Entity>`?

The `find()` call might fail if the character name doesn’t exist in `characters.ron`. Returning `Option<Entity>` lets the caller handle this gracefully. If the character is found, we return `Some(entity_id)`. If not, we return `None`.

The Enemy Plugin

Now we need to wire everything together into a plugin. The plugin will register all our enemy systems and schedule them to run at the right time.

Create `src/enemy/mod.rs`:

```
// src/enemy/mod.rs
pub mod ai;
pub mod combat;
pub mod components;
pub mod spawn;

use crate::collision::CollisionMapBuilt;
use crate::state::GameState;
use bevy::prelude::*;
use spawn::EnemiesSpawned;

pub use components::{AIBehavior, Enemy, EnemyCombat};
pub use spawn::spawn_enemy;

pub struct EnemyPlugin;

impl Plugin for EnemyPlugin {
    fn build(&self, app: &mut App) {
        app
            .init_resource::()
            // Spawn enemies AFTER collision map is ready (prevents spawning on obstacles)
            .add_systems(
                Update,
                spawn::spawn_test_enemies
                    .run_if(resource_equals(CollisionMapBuilt(true)))
                    .run_if(resource_equals(EnemiesSpawned(false)))
                    .run_if(in_state(GameState::Playing)),
            )
            // Enemy AI and combat systems
            .add_systems(
                Update,
                (ai::enemy_follow_player, combat::enemy_attack)
                    .chain()
                    .run_if(in_state(GameState::Playing)),
            );
    }
}
```

How does the plugin work?

We use **run conditions** to control when `spawn_test_enemies` executes:

- `CollisionMapBuilt(true)` - collision map must exist before validating spawn positions
- `EnemiesSpawned(false)` - spawns only once (resource tracks spawn state)

- `in_state(GameState::Playing)` - only spawns during actual gameplay

This ensures enemies spawn at validated positions AFTER the collision map is ready, preventing them from spawning on obstacles.

The AI and combat systems run every frame during `Playing` state. We chain them together so AI runs first (updating positions), then combat runs (firing projectiles).

Also, our enemy combat system needs to call `spawn_projectile`, but it's currently private to the combat module. We need to make two changes: make the systems module public, and export the `spawn_projectile` function.

Open `src/combat/mod.rs` and update it:

```
// src/combat/mod.rs
mod player_combat;
mod power_type;
pub mod systems; // Line update alert: Make module public

pub use player_combat::PlayerCombat;
pub use power_type::{PowerType, PowerVisuals};
pub use systems::{debug_switch_power, handle_power_input, spawn_projectile}; // Line update alert:
Export spawn_projectile
```

Then open `src/combat/systems.rs` and make the `spawn_projectile` function public:

```
// src/combat/systems.rs
// Find this function and add 'pub' keyword
pub fn spawn_projectile( // Line update alert: Add 'pub' keyword
    commands: &mut Commands,
    position: Vec3,
    power_type: PowerType,
    visuals: &PowerVisuals,
) {
    // ... rest of the function
}
```

Now both players and enemies can call `spawn_projectile` from outside the combat module.

Fixing Enemy Depth Sorting

There's one more issue to address. Remember in Chapter 5 when we implemented depth sorting for the player? The player's Z position updates based on Y position, so they render correctly behind or in front of objects.

Enemies need the same treatment. Right now, they spawn with a static Z position and never update. This means an enemy standing at the bottom of the screen (low Y) might render behind a tree at the top of the screen (high Y), which looks wrong.

The good news? We already have the depth sorting logic. We just need to make it work for all characters, not just the player.

Open `src/characters/rendering.rs` and update it:

```
// src/characters/rendering.rs
use bevy::prelude::*;

use crate::characters::state::CharacterState; // Line update alert: Change from Player to
CharacterState
use crate::config::map::{GRID_Y, TILE_SIZE};
use crate::config::player::PLAYER_SCALE;
```

```

/// Z-depth constants for proper layering.
/// The tilemap uses `with_z_offset_from_y(true)` which assigns Z based on Y position.
/// We need to match this formula for all characters (player and enemies). // Line update alert
const NODE_SIZE_Z: f32 = 1.0; // Same as tilemap generator
const CHARACTER_BASE_Z: f32 = 4.0; // Match props layer Z range // Line update alert
const CHARACTER_Z_OFFSET: f32 = 0.5; // Small offset to stay above ground props // Line update alert

pub fn update_character_depth( // Line update alert: Renamed from update_player_depth
    //Updated from PlayerState to CharacterState
    mut character_query: Query<&mut Transform, (With<CharacterState>, Changed<Transform>)>,
) {
    // Map dimensions for normalization
    let map_height = TILE_SIZE * GRID_Y as f32;
    let map_y0 = -TILE_SIZE * GRID_Y as f32 / 2.0; // Map origin Y (centered)

    // Character sprite height for feet position calculation // Line update alert
    let character_sprite_height = 64.0 * PLAYER_SCALE; // Line update alert

    for mut transform in character_query.iter_mut() { // Line update alert
        let character_center_y = transform.translation.y; // Line update alert

        // Use character's FEET position for depth sorting (not center) // Line update alert
        let character_feet_y = character_center_y - (character_sprite_height / 2.0); // Line update alert

        // Normalize feet Y to [0, 1] across the grid height
        let t = ((character_feet_y - map_y0) / map_height).clamp(0.0, 1.0); // Line update alert

        // Y-to-Z formula:
        // Lower Y (bottom of screen) = higher t = lower Z offset = rendered in front
        // Higher Y (top of screen) = lower t = higher Z offset = rendered behind
        let character_z = CHARACTER_BASE_Z + NODE_SIZE_Z * (1.0 - t) + CHARACTER_Z_OFFSET; // Line update alert

        transform.translation.z = character_z; // Line update alert
    }
}

```

What changed?

We changed the query from `With<Player>` to `With<CharacterState>`. Since both players and enemies have the `CharacterState` component, this system now runs for all characters.

We also renamed the function and variables to reflect that it's no longer player-specific.

Now update `src/characters/mod.rs` to use the new function name:

```

// src/characters/mod.rs
// Find the Update systems and update this line:
.add_systems(Update, (
    input::handle_player_input,
    spawn::switch_character,
    input::update_jump_state,
    animation::on_state_change_update_animation,
    collider::validate_movement,
    physics::apply_velocity,
    rendering::update_character_depth, // Line update alert: Renamed
    animation::animations_playback,
).chain().run_if(in_state(GameState::Playing)));

```

Why does this work?

Both players and enemies have the `CharacterState` component. By querying for `With<CharacterState>` instead of `With<Player>`, the depth sorting system automatically applies to all character entities.

The system uses `Changed<Transform>` to only recalculate Z-depth when entities move, keeping it efficient.

Preventing Entity Stacking

We have one more collision problem to solve. Right now, the `validate_movement` system only checks against the **collision map** (tiles and obstacles). It doesn't check against other **entities**. This means:

1. Enemies can stack on top of each other
2. Enemies pass right through the player

We need **entity-to-entity collision detection**.

The Approach:

After validating against the collision map, we'll check each entity against all other entities. If two entities overlap (their collision circles intersect), we push them apart.

Open `src/characters/collider.rs` and add this new system at the end:

```
// src/characters/collider.rs
// Add at the end of the file

/// Resolve collisions between entities (player and enemies)
/// Prevents entities from moving into each other
pub fn resolve_entity_collisions(
    mut query: Query<Entity, &Transform, &mut Velocity, &Collider>,
) {
    // Collect all entity positions first to avoid multiple mutable borrows
    let entities: Vec<_> = query
        .iter()
        .map(|(e, t, _, c)| (e, c.world_position(t), c.radius))
        .collect();

    // Check each entity against all others
    for (entity, transform, mut velocity, collider) in query.iter_mut() {
        // Skip if not moving
        if !velocity.is_moving() {
            continue;
        }

        let pos = collider.world_position(transform);
        let radius = collider.radius;

        for &(other_entity, other_pos, other_radius) in &entities {
            // Skip self
            if entity == other_entity {
                continue;
            }

            let delta = other_pos - pos;
            let distance = delta.length();
            let min_distance = radius + other_radius;

            // Check if entities are overlapping or very close
            if distance < min_distance * 1.1 {
                // Calculate the direction toward the other entity
                if distance > 0.01 {
                    let direction = delta / distance;
                }
            }
        }
    }
}
```

```

        // Project velocity onto the direction toward the other entity
        let velocity_toward = velocity.0.dot(direction);

        // If moving toward the other entity, block that movement
        if velocity_toward > 0.0 {
            // Remove the component of velocity moving toward the other entity
            velocity.0 -= direction * velocity_toward;
        }
    }
}
}
}
}
}
}
}
}
}
}

```

How does this work?

This uses **vector projection** to block movement toward other entities:

1. **Collect positions first** - Store all entity positions to avoid borrowing issues
2. **Skip stationary entities** - Only check moving entities
3. **Check proximity** - If entities are within `1.1 * (radius1 + radius2)`, they're close enough to block
4. **Project velocity** - Use dot product to find how much velocity points toward the other entity
5. **Remove that component** - Subtract the "toward" velocity, leaving only tangential movement

Why vector projection?

Instead of pushing entities apart (which feels jarring), we let them slide past each other. If an enemy walks toward the player, the component of velocity moving directly toward the player is removed, but sideways velocity remains. This creates smooth "sliding" collisions.

Example:

- Enemy moving diagonally toward player: ↗
- After blocking: Enemy slides sideways past player: →

This feels natural and prevents the jittery pushback effect.

Why modify velocity instead of directly moving entities?

Modifying velocity keeps physics consistent. The pushback force is applied as acceleration, then `apply_velocity` moves the entity. This ensures collision response feels smooth and natural.

Now add this system to the chain in `src/characters/mod.rs`:

```

// src/characters/mod.rs
.add_systems(
    Update,
    (
        input::handle_player_input,
        spawn::switch_character,
        input::update_jump_state,
        animation::on_state_change_update_animation,
        collider::validate_movement,
        collider::resolve_entity_collisions, // NEW: Prevent entity stacking
        physics::apply_velocity,
        rendering::update_character_depth,
        animation::animations_playback,
    )
    .chain()
    .run_if(in_state(GameState::Playing)),
)

```

System order matters:

```
validate_movement    → Check vs tiles/obstacles  
resolve_entity_collisions → Check vs other entities  
apply_velocity       → Actually move the character
```

Both collision checks happen BEFORE movement is applied. This prevents entities from getting stuck inside obstacles or each other.

Integrating the Enemy Module

Finally, let's add the enemy module to our game. Open `src/main.rs` and add the module declaration and plugin:

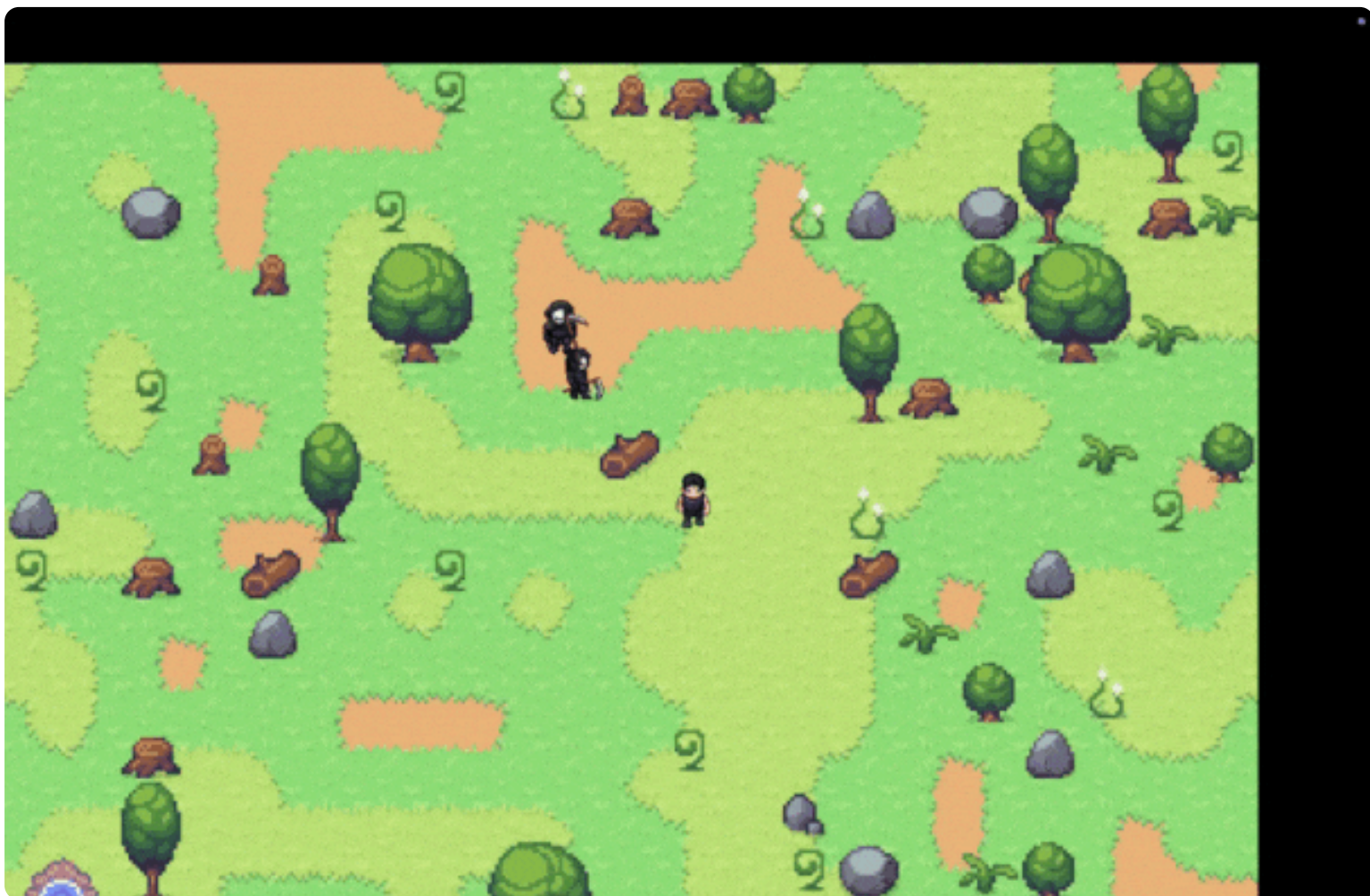
```
// src/main.rs  
mod camera;  
mod characters;  
mod collision;  
mod combat;  
mod config;  
mod enemy; // Add this line  
mod inventory;  
mod map;  
mod particles;  
mod state;
```

Then add the plugin to your app:

```
// Inside main function of src/main.rs  
fn main() {  
    App::new()  
        // ... existing plugins ...  
        .add_plugins(combat::CombatPlugin)  
        .add_plugins(enemy::EnemyPlugin) // Add this line  
        .add_plugins(inventory::InventoryPlugin)  
        // ... rest of the code ...  
        .run();  
}
```

Run your game:

```
cargo run
```

You might notice enemies don't actually deal damage yet. In **Chapter 8**, we'll add health, damage, and death. Enemies will become threatening, and your survival will depend on dodging their attacks.

Optimizing Debug Builds

***Community Tip:** This optimization was pointed out by one of our [community members](#), thank you for helping make this tutorial better!*

Bevy's default debug configuration can lead to performance issues, scenes that should run smoothly might drop to unplayable framerates, or large assets might take minutes to load.

The Bevy team [documents this issue](#) and provides a solution.

Add these optimizations to your `Cargo.toml`:

```
# At the bottom of your Cargo.toml

# Enable a small amount of optimization in the dev profile
[profile.dev]
opt-level = 1

# Enable a large amount of optimization in the dev profile for dependencies
[profile.dev.package."*"]
opt-level = 3
```

What does this do?

- `opt-level = 1` **for your code**: Applies minimal optimization to your own code, keeping compile times fast while improving runtime performance
- `opt-level = 3` **for dependencies**: Heavily optimizes Bevy and other dependencies (which rarely change), dramatically improving framerate

The trade-off:

Your first build after adding this will take longer (dependencies need to recompile with optimizations). But subsequent builds remain fast since dependencies are cached. You get much

better debug performance without sacrificing development speed!

The Impatient Programmer's Guide to Bevy and Rust: Chapter 8 - Let There Be Damage

Right now, your magic powers are purely visual. They fly through the air with particle effects, but when they hit an enemy, nothing happens. The enemy keeps following the player, completely unbothered.

This chapter will make combat meaningful. When a projectile collides with an enemy, the enemy takes damage. A health bar appears above them, visually showing how much HP they have left. When their health reaches zero, they're removed from the game. The same system will work for both player and enemies.



The Health Component

We need a way to track how much health any entity has. This applies to both the player and enemies, so we'll create a generic component that works for both.

Create `src/combat/health.rs`:

```
use bevy::prelude::*;

/// Health component for any damageable entity (player, enemies).
#[derive(Component)]
pub struct Health {
    pub current: f32,
    pub max: f32,
}

impl Health {
    pub fn new(max: f32) -> Self {
        Self { current: max, max }
    }
}
```

```

pub fn take_damage(&mut self, amount: f32) {
    self.current = (self.current - amount).max(0.0);
}

pub fn is_alive(&self) -> bool {
    self.current > 0.0
}

/// Returns health as a ratio in [0, 1].
pub fn ratio(&self) -> f32 {
    self.current / self.max
}
}

```

The `Health` component stores two floats: `current` represents the entity's current HP, and `max` represents the maximum HP capacity.

The `new()` constructor creates an entity at full health by setting both values to the same number.

The `take_damage()` method subtracts damage from current health. The `.max(0.0)` ensures health never goes negative. If you take 30 damage with only 10 HP remaining, your health becomes 0, not -20.

The `is_alive()` method returns true if the entity still has health remaining. We'll use this to determine when to delete defeated enemies.

The `ratio()` method calculates health as a percentage by dividing current by max. This returns a value between 0.0 (dead) and 1.0 (full health). We'll use this to scale the health bar width and determine its color.

Making Powers Deal Damage

Each power type should have different damage values. Fire might deal high damage with a large hitbox, while shadow magic deals less damage but travels faster.

Open `src/combat/power_type.rs` and add these two methods to the `impl PowerType` block:

```

// src/combat/power_type.rs
// Add these methods to the impl PowerType block

impl PowerType {
    /// Damage dealt by this power type.
    pub fn damage(&self) -> f32 {
        match self {
            PowerType::Fire => 25.0,
            PowerType::Arcane => 35.0,
            PowerType::Shadow => 20.0,
            PowerType::Poison => 15.0,
        }
    }

    /// Hitbox collision radius. Larger for powers that spread visually.
    pub fn hitbox_radius(&self) -> f32 {
        match self {
            PowerType::Fire => 30.0,
            PowerType::Arcane => 18.0,
            PowerType::Shadow => 15.0,
            PowerType::Poison => 25.0,
        }
    }
}

```

```
// ... existing visuals() method and others below
}
```

The `damage()` method returns how much HP is removed when this power hits an entity. Arcane magic deals the most damage (35) but will have a smaller hitbox. Poison deals the least damage (15) but spreads widely.

The `hitbox_radius()` method defines the collision detection distance. Fire has a 30-unit radius because visually it spreads out. Shadow has only 15 units because it's a focused beam. This radius determines how close a projectile needs to be to an entity before we consider it a "hit".

Creating Invisible Hitboxes

Currently, projectiles are just visual particle emitters. We need invisible hitbox entities that move forward and check for collisions with enemies.

Why do we need invisible hitboxes when there's particles?

Particle emitters are visual effects made of dozens or hundreds of individual particles, each with its own position, velocity, and lifetime. Using them for collision detection would require checking 50+ particles per projectile against every enemy extremely expensive.

Worse, particles spread out unpredictably in a spray pattern, making it unclear which particle determines a hit. A hitbox is a single point with a radius, requiring just one distance check per enemy with precise, predictable collision detection.

The invisible hitbox travels alongside the particle emitter but handles all collision logic independently.

NOTE: From this point forward, when we say "projectile" we're referring to the **hitbox entity** with the `Projectile` component, not the visual particle effect. The particle emitter is a separate entity spawned alongside the hitbox.

We'll create two new components: `ProjectileOwner` to track who fired the projectile, and `Projectile` to store the hitbox data.

Open `src/combat/systems.rs` and add these components at the top, after the existing `ProjectileEffect`:

```
// src/combat/systems.rs
// Add these after the existing ProjectileEffect component

/// Who fired a projectile determines which entities it can hit.
#[derive(Component, Clone, Copy)]
pub enum ProjectileOwner {
    Player,
    Enemy,
}

/// Invisible hitbox that travels and checks for collisions.
#[derive(Component)]
pub struct Projectile {
    pub velocity: Vec3,
    pub lifetime: f32,
    pub power_type: PowerType,
    pub owner: ProjectileOwner,
    pub radius: f32,
}
```

The `ProjectileOwner` enum identifies who created the projectile. Player projectiles can only damage enemies, and enemy projectiles can only damage the player. This prevents friendly fire.

The `Projectile` component stores everything needed for a moving hitbox:

- `velocity` determines how fast and in what direction it moves
- `lifetime` tracks how long the projectile exists (in seconds) before auto-despawning
- `power_type` links back to the visual effect and damage value
- `owner` determines which entities this projectile can damage
- `radius` is the collision detection distance

Spawning Projectile Hitboxes

Now we need to modify the `spawn_projectile` function to create a hitbox entity alongside the visual particles.

Update the `spawn_projectile` function signature and add the hitbox spawning logic at the end:

```
// src/combat/systems.rs
// Update the spawn_projectile function

pub fn spawn_projectile(
    commands: &mut Commands,
    position: Vec3,
    power_type: PowerType,
    visuals: &PowerVisuals,
    owner: ProjectileOwner, // Add this line
) {
    // Primary particles
    let primary_emitter =
        ParticleEmitter::new(0.016, visuals.particles_per_spawn, visuals.primary.clone())
            .one_shot();

    commands.spawn((
        primary_emitter,
        Transform::from_translation(position),
        GlobalTransform::from(Transform::from_translation(position)),
        ProjectileEffect { power_type },
    ));

    // Core particles (if the power has a core)
    if let Some(ref core_config) = visuals.core {
        let core_emitter =
            ParticleEmitter::new(0.016, visuals.core_particles_per_spawn, core_config.clone())
                .one_shot();

        commands.spawn((
            core_emitter,
            Transform::from_translation(position),
            GlobalTransform::from(Transform::from_translation(position)),
            ProjectileEffect { power_type },
        ));
    }

    // Add this line
    // Hitbox: invisible entity that moves and checks for hits
    let direction = visuals.primary.direction.normalize_or_zero();
    let speed = visuals.primary.speed;
    commands.spawn((
        Projectile {
            velocity: direction * speed,
            lifetime: 2.0,
            power_type,
            owner,
            radius: power_type.hitbox_radius(),
        },
    ));
}
```



```

        Transform::from_translation(position),
    ));
}

```

The function now accepts an `owner` parameter to identify who fired the projectile.

After spawning the visual particle emitters (primary and core), we extract the direction and speed from the particle configuration. The direction vector is normalized to ensure it has length 1.0, then multiplied by speed to create the velocity.

The hitbox is spawned as a separate entity with a `Projectile` component. It starts at the same position as the particles but has its own `Transform`. The hitbox will move independently using its velocity, while the particles remain stationary and fade out.

We set `lifetime: 2.0`, giving the projectile 2 seconds before it auto-despawns. The radius comes from `power_type.hitbox_radius()`, which we added to `PowerType` earlier.

Now update the call site in `handle_power_input` to pass the owner:

```

// src/combat/systems.rs
// Update this line in handle_power_input function

pub fn handle_power_input(
    mut commands: Commands,
    input: Res<ButtonInput<KeyCode>>,
    time: Res<Time>,
    mut player_query: Query<(&GlobalTransform, &Facing, &mut PlayerCombat), With<Player>>,
) {
    // ... existing code ...

    // Get visuals from power type
    let visuals = combat.power_type.visuals(direction);

    // Updated line:
    spawn_projectile(&mut commands, spawn_position, combat.power_type, &visuals,
        ProjectileOwner::Player);

    info!("{:?} projectile fired!", combat.power_type);
}

```

We pass `ProjectileOwner::Player` to identify that this projectile was fired by the player.

You also need to update the enemy's `spawn_projectile` call in `src/enemy/combat.rs`:

```

// src/enemy/combat.rs
// Add this to the imports at the top
use crate::combat::systems::{spawn_projectile, ProjectileOwner}; // Line update alert

// Update this line in the enemy_attack function
spawn_projectile(&mut commands, spawn_position, combat.power_type, &visuals,
    ProjectileOwner::Enemy);

```

Now both player and enemy projectiles will work with the damage system. Player projectiles hit enemies, and enemy projectiles hit the player.

Moving Projectiles Forward

The hitboxes exist, but they're not moving yet. We need a system that moves each projectile forward based on its velocity, and despawns projectiles that have exceeded their lifetime.

Add this new system to `src/combat/systems.rs`:


```
// src/combat/systems.rs
// Add this new system

/// Moves projectile hitboxes forward and despawns them on timeout.
pub fn move_projectiles(
    mut commands: Commands,
    time: Res<Time>,
    mut projectiles: Query<(Entity, &mut Projectile, &mut Transform)>,
) {
    let dt = time.delta_secs();
    for (entity, mut proj, mut transform) in projectiles.iter_mut() {
        proj.lifetime -= dt;
        if proj.lifetime <= 0.0 {
            commands.entity(entity).despawn();
            continue;
        }
        transform.translation += proj.velocity * dt;
    }
}
```

We query for all entities with both a `Projectile` and `Transform` component. The `Entity` ID lets us despawn the projectile when needed.

For each frame, we decrease the projectile's lifetime by the delta time (the time elapsed since the last frame). If lifetime drops to zero or below, we despawn the entity and skip the movement step.

If the projectile is still alive, we move its position forward by adding `velocity * dt`. If velocity is `(300, 0, 0)` and delta time is 0.016 seconds, the projectile moves 4.8 units to the right.

This approach creates smooth movement regardless of frame rate. At 60 FPS, the projectile takes many small steps. At 30 FPS, it takes fewer but larger steps, covering the same distance.

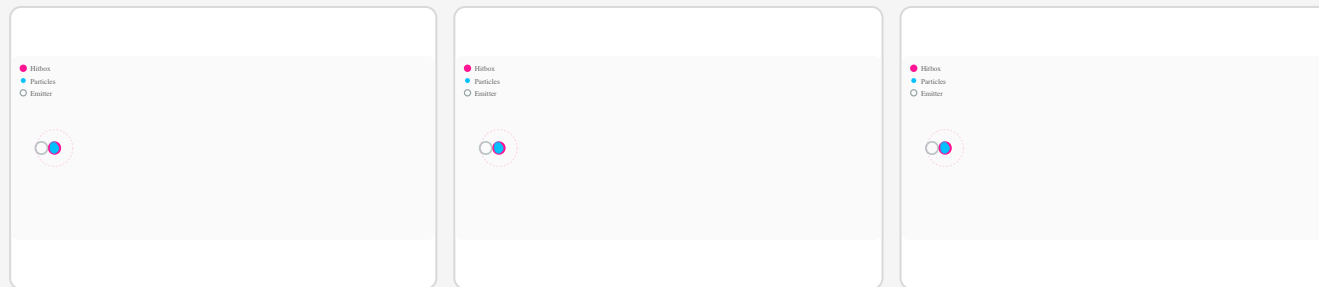
So these invisible hitboxes, do they move along with particles, do they share same position as they move?

No, they're completely separate entities. When you fire a projectile, `spawn_projectile` creates multiple entities:

Both start at the same position, but they move independently:

- **Hitbox:** Moves in a straight line via `velocity * dt`. Clean, predictable physics.
- **Particles:** Each has unique velocity with variance, creating the visual spray effect.

Hitbox vs Particles: Separate Entities



The hitbox and emitter stay roughly aligned (same base trajectory), but the individual particles drift around the emitter. When the hitbox detects a collision, we despawn both the hitbox entity and the emitter entity, which stops particle generation.

Understanding the Observer Pattern

When a projectile hits an enemy, we need to:

1. Apply damage to the enemy's health
2. Despawn the projectile (both hitbox and particle emitter)

Later, we might want to add more effects when hits occur:

- Play a hit sound
- Spawn impact particles at the collision point
- Trigger screen shake
- Show floating damage numbers
- Award points for hits

The naive approach puts all this logic directly in the hit detection code. This creates tight coupling.

The tight coupling problem:

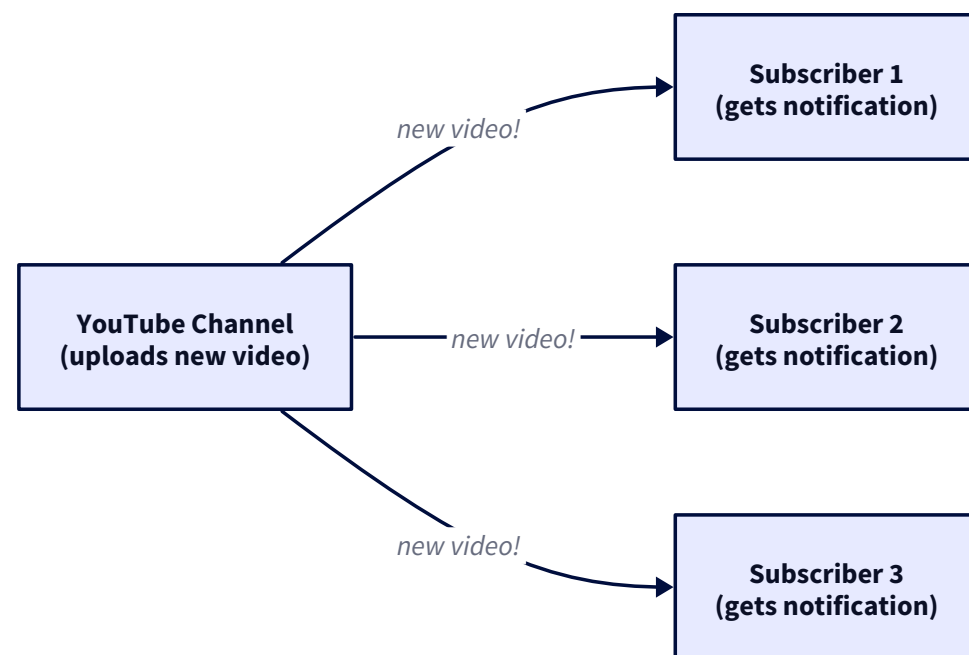
The `check_projectile_hits()` function in our combat system would become a tangled mess of responsibilities. It detects hits, applies damage, cleans up entities, plays sounds, triggers visual effects... Adding any new hit effect means modifying this function every time.

The Observer Pattern Solution

The Observer Pattern solves this by flipping the control flow. Instead of one system commanding everyone else what to do, it simply announces that something happened. This is called “Event-Driven Architecture.”

Think of a YouTube channel (the Publisher) and its subscribers (the Observers).

- **Without Observers:** You have to constantly refresh the channel page to check for new videos.
- **With Observers:** You subscribe to the channel. When a new video is uploaded, YouTube notifies you automatically. The channel doesn't need to know who you are or what you'll do with the notification; it just sends it out.



Applying this to Projectiles

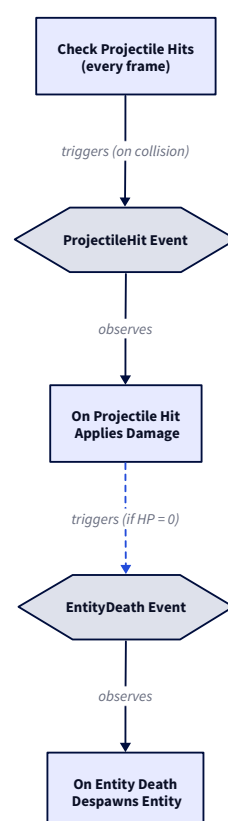
In our game, we can make the combat system act like the YouTube channel.

Instead of `check_projectile_hits()` trying to do everything itself, it just triggers a `ProjectileHit` event. Other systems observe this event and react independently:

- One observer applies damage.
- Another observer despawns the projectile.
- Future observers could play sounds, trigger screen shake, or spawn particles.

This keeps the hit detection code focused purely on detection.

Visualizing the Flow



So when a projectile hits an enemy:

1. Hit detection triggers the event: `commands.trigger(ProjectileHit { target, damage })`
2. The damage observer sees the event and applies damage to health

3. If health reaches zero, `Health` triggers a `EntityDeath` event
4. The death observer sees that event and despawns the entity

This is called the observer pattern because systems observe events and react to them, rather than constantly polling for changes.

Why this is better:

Event handlers only run when events occur, not every frame. If 10 hits happen in 60 seconds, the damage system runs 10 times, not 3,600 times.

New reactions are easy to add. Want to play a sound on hit? Add a sound observer. Want to spawn blood particles on death? Add a particle observer. The hit detection system doesn't change.

Systems stay decoupled. Hit detection doesn't know about damage. Damage doesn't know about despawning. Each system has one job.

Bevy provides this through the observer API. You define event types, trigger them with `commands.trigger()`, and register observers with `.add_observer()`. The observer function receives the event data and can query components or run commands like any system.

What observers don't replace:

Observers are for reactions to discrete events, not for continuous processes. Systems that check conditions every frame still need to be regular systems:

- Collision detection (`check_projectile_hits`) must run every frame to calculate distances
- Position tracking of healthbar (`update_healthbars`) must run every frame to follow moving characters
- Movement (`move_projectiles`) must run every frame to update positions

The observer pattern handles **what happens** when a collision occurs, not the detection itself. Hit detection triggers events. Observers react to those events. Both are necessary.

Let's implement this.

Creating Combat Events

First, create event types that represent things that happen in combat.

Create `src/combat/events.rs`:

```
// src/combat/events.rs
use super::power_type::PowerType;
use bevy::prelude::*;

/// Event triggered when a projectile hits a target entity.
#[derive(Event)]
pub struct ProjectileHit {
    pub target: Entity,
    pub damage: f32,
    pub power_type: PowerType,
}

/// Event triggered when an entity's health reaches zero.
#[derive(Event)]
pub struct EntityDeath {
    pub entity: Entity,
}
```

The `#[derive(Event)]` macro marks these structs as Bevy events. Events are just data structures that get passed from the code that triggers them to the observers that handle them.

`ProjectileHit` carries the target entity that was hit, how much damage to apply, and which power type caused the hit.

`EntityDeath` only needs the entity ID. Observers can use this to despawn the entity, spawn death effects, or update scores.

Events are simple data containers. They don't contain logic. The logic lives in the observers.

Detecting Projectile Hits

Now that projectiles are moving, we need to check if they collide with valid targets. Instead of directly applying damage, we'll trigger events that observers will handle.

Add this new system to `src/combat/systems.rs`:

```
// src/combat/systems.rs
// Add at the top with other imports
use crate::enemy::Enemy;

// Add this new system

/// Checks each projectile hitbox against its valid targets; triggers hit events on collision.
pub fn check_projectile_hits(
    mut commands: Commands,
    projectiles: Query<(Entity, &Projectile, &Transform)>,
    players: Query<(Entity, &GlobalTransform), With<Player>>,
    enemies: Query<(Entity, &GlobalTransform), With<Enemy>>,
) {
    for (proj_entity, proj, proj_transform) in &projectiles {
        let proj_pos = proj_transform.translation;

        let hit_target = match proj.owner {
            ProjectileOwner::Player => enemies
                .iter()
                .find(|(_, t)| proj_pos.distance(t.translation()) <= proj.radius)
                .map(|(e, _)| e),
            ProjectileOwner::Enemy => players
                .iter()
                .find(|(_, t)| proj_pos.distance(t.translation()) <= proj.radius)
                .map(|(e, _)| e),
        };

        if let Some(target) = hit_target {
            // Trigger hit event instead of directly applying damage
            commands.trigger(super::events::ProjectileHit {
                target,
                damage: proj.power_type.damage(),
                power_type: proj.power_type,
            });
            commands.entity(proj_entity).despawn();
        }
    }
}
```

We query for all projectiles, all players, and all enemies. The projectiles query includes the `Entity` ID so we can despawn the projectile when it hits something.

For each projectile, we check its owner. If the owner is `Player`, we search the enemies list for any enemy within collision radius. If the owner is `Enemy`, we search the players list instead.

The `.find()` method returns the first enemy (or player) where the distance to the projectile is less than or equal to the hitbox radius. We use `.map(|(e, _)| e)` to extract just the entity ID.

When we find a target, we trigger a `ProjectileHit` event using `commands.trigger()`. The event carries the target entity, damage amount, and power type. We don't apply damage here. We just report that a hit occurred.

The hit detection system has one job: detect collisions and report them. Observers will handle the rest.

Creating Observer Handlers

Now we create the systems that respond to combat events.

Create `src/combat/observers.rs`:

```
// src/combat/observers.rs
use super::events::{EntityDeath, ProjectileHit};
use super::health::Health;
use bevy::prelude::*;

/// Observer that handles projectile hits by applying damage to the target.
pub fn on_projectile_hit(
    hit: On<ProjectileHit>,
    mut healths: Query<&mut Health>,
    mut commands: Commands,
) {
    let Ok(mut health) = healths.get_mut(hit.target) else {
        return;
    };

    health.take_damage(&mut commands, hit.target, hit.damage);

    info!(
        "{:?} hit for {} damage! HP: {:.0}/{:.0}",
        hit.power_type, hit.damage, health.current, health.max
    );
}

/// Observer that handles entity death by despawning the entity.
pub fn on_entity_death(death: On<EntityDeath>, mut commands: Commands) {
    info!("Entity {:?} defeated!", death.entity);
    commands.entity(death.entity).despawn();
}
```

Observer functions look like regular systems, but they receive event data through the `On<EventType>` parameter. When a `ProjectileHit` event triggers, Bevy calls `on_projectile_hit` and passes the event data.

The `hit` parameter acts like a reference to the event. You can access its fields directly: `hit.target`, `hit.damage`, `hit.power_type`.

The `on_projectile_hit` observer queries for health components and applies damage. If the target doesn't have health (maybe it was already despawned or dead), we return early.

We call `health.take_damage()`, passing the commands, entity, and damage amount. This will reduce health and potentially trigger a death event.

The `on_entity_death` observer is even simpler. It just logs the death and despawns the entity. In a full game, this is where you'd spawn death animations, play sounds, or update scores. Each of those could be separate observers.

Observers are just functions. They're not registered as regular systems in the `Update` schedule. Instead, they're registered with `.add_observer()` and only run when their event triggers.

Updating Health to Trigger Death Events

The `Health` component needs to trigger `EntityDeath` events when health reaches zero.

Update `src/combat/health.rs`:


```
// src/combat/health.rs
use bevy::prelude::*;

// ... existing Health struct ...

impl Health {
    // ... existing new(), is_alive(), ratio() methods ...

    pub fn take_damage(
        &mut self,
        commands: &mut Commands, // Add this line
        entity: Entity, // Add this line
        amount: f32
    ) {
        self.current = (self.current - amount).max(0.0);

        // Add this
        // Trigger death event when health reaches zero
        if !self.is_alive() {
            commands.trigger(super::events::EntityDeath { entity });
        }
    }
}
```

The `take_damage` method now accepts `commands` and `entity` parameters so it can trigger events.

After reducing health, we check if the entity is still alive. If not, we trigger an `EntityDeath` event. Any observer watching for death events will see this and react accordingly.

This creates a chain: projectile hit triggers damage → damage triggers death → death triggers despawn. Each step is handled by a separate observer, keeping the code modular.

The signature change means any code calling `take_damage()` needs to pass the additional parameters. Observers already have access to `commands`, so this works seamlessly.

The `.find()` method returns the first enemy (or player) where the distance to the projectile is less than or equal to the hitbox radius. We use `.map(|(e, _)| e)` to extract just the entity ID.

Health Bars

Seeing numbers in the console is helpful for debugging, but players need visual feedback. We'll create health bars that appear above any entity with a Health component.

Create `src/combat/healthbar.rs`:

First, we define the look of our health bars and the components we'll need.

```
// src/combat/healthbar.rs
use bevy::prelude::*;
use super::health::Health;

const HEALTHBAR_WIDTH: f32 = 50.0;
const HEALTHBAR_HEIGHT: f32 = 6.0;
const HEALTHBAR_Y_OFFSET: f32 = 43.0;
const HEALTHBAR_Z_OFFSET: f32 = 1.0;
/// Small z bump so the foreground always renders on top of the background.
const HEALTHBAR_FG_Z_BUMP: f32 = 0.01;

/// Marker: this entity is the colored fill of a healthbar.
#[derive(Component)]
pub struct HealthBarForeground;

/// Links a healthbar entity back to its owner character.
```

```
#[derive(Component)]
pub struct HealthBarOwner(pub Entity);
```

We use simple constants for sizing and positioning. The `HealthBarOwner` component lets the health bar know which entity's health it should display.

`HealthBarForeground` marks the foreground part of the bar that will change size and color as health changes.

Spawning Health Bars

We need a system that automatically attaches a health bar to any entity that has health. Bevy's query filters make this easy.

When we query for `Added<Health>`, it matches in two cases:

1. When an entity is first spawned with the component.
2. When the component is added to an existing entity later.

This means we don't need to manually call a "setup health bar" function every time we create an enemy. The system just reacts to the presence of `Health`.

```
// Append to src/combat/healthbar.rs
/// Spawns a background + foreground healthbar pair for each entity that gains Health.
pub fn spawn_healthbars(
    mut commands: Commands,
    new_health: Query<(Entity, &GlobalTransform, &Health), Added<Health>>,
    mut meshes: ResMut<Assets<Mesh>>,
    mut materials: ResMut<Assets<ColorMaterial>>,
) {
    for (owner, transform, health) in &new_health {
        let pos = transform.translation();
        let bg_pos = Vec3::new(pos.x, pos.y + HEALTHBAR_Y_OFFSET, pos.z + HEALTHBAR_Z_OFFSET);
        let fg_pos = Vec3::new(pos.x, pos.y + HEALTHBAR_Y_OFFSET, pos.z + HEALTHBAR_Z_OFFSET +
HEALTHBAR_FG_Z_BUMP);

        // Background: dark gray
        let bg_mesh = meshes.add(Rectangle::new(HEALTHBAR_WIDTH, HEALTHBAR_HEIGHT));
        let bg_mat = materials.add(ColorMaterial::from(Color::srgb(0.2, 0.2, 0.2)));
        commands.spawn((
            Mesh2d(bg_mesh),
            MeshMaterial2d(bg_mat),
            Transform::from_translation(bg_pos),
            HealthBarOwner(owner),
        ));

        // Foreground: color derived from actual health ratio
        let ratio = health.ratio();
        let fg_mesh = meshes.add(Rectangle::new(HEALTHBAR_WIDTH, HEALTHBAR_HEIGHT));
        let fg_mat = materials.add(ColorMaterial::from(health_color(ratio)));
        commands.spawn((
            Mesh2d(fg_mesh),
            MeshMaterial2d(fg_mat),
            Transform::from_translation(fg_pos).with_scale(Vec3::new(ratio.max(0.001), 1.0, 1.0)),
            HealthBarOwner(owner),
            HealthBarForeground,
        ));
    }
}
```

We use a `Query` with `Added<Health>`. This ensures the system runs only once per entity, exactly when it enters the game world (or first gets health component).

For each entity, we spawn two rectangular meshes: a dark gray background and a colored foreground. We position them at slightly different z-coordinates using `HEALTHBAR_FG_Z_BUMP` (0.01). This tiny offset ensures the foreground always renders firmly on top of the background.

Finally, we attach the `HealthBarOwner` component to these new bars. This link is what allows our healthbar update system to find the character and follow it as it moves.

Updating Health Bars

Finally, we need a system that runs every frame to update the health bars. It needs to do two things:

1. Move the health bar to follow its owner.
2. Update the width and color of the foreground bar based on current health.

```
// Append to src/combat/healthbar.rs
/// Each frame: repositions bars to follow owner, updates foreground width and color.
/// Despawns bars whose owner no longer exists.
pub fn update_healthbars(
    mut commands: Commands,
    mut bars: Query<(
        Entity,
        &HealthBarOwner,
        &mut Transform,
        Has<HealthBarForeground>,
        &MeshMaterial2d<ColorMaterial>,
    )>,
    owners: Query<(&GlobalTransform, &Health)>,
    mut materials: ResMut<Assets<ColorMaterial>>,
) {
    for (bar_entity, owner_ref, mut transform, is_foreground, mat_handle) in bars.iter_mut() {
        let Ok((owner_transform, health)) = owners.get(owner_ref.0) else {
            // If the owner is gone (despawnd), clean up the health bar
            commands.entity(bar_entity).despawn();
            continue;
        };

        let owner_pos = owner_transform.translation();
        let ratio = health.ratio();

        if is_foreground {
            // Scale foreground width to match health ratio
            transform.scale.x = ratio.max(0.001);

            // Reposition foreground to stay left-aligned as it shrinks
            // (Scaling happens from center, so we need to offset position)
            transform.translation = Vec3::new(
                owner_pos.x - (HEALTHBAR_WIDTH * (1.0 - ratio) / 2.0),
                owner_pos.y + HEALTHBAR_Y_OFFSET,
                owner_pos.z + HEALTHBAR_Z_OFFSET + HEALTHBAR_FG_Z_BUMP,
            );

            // Update color (Green -> Yellow -> Red)
            if let Some(mut mat) = materials.get_mut(&mat_handle.0) {
                mat.color = health_color(ratio);
            }
        } else {
            // Background simply stays centered above the owner
            transform.translation = Vec3::new(
                owner_pos.x,
                owner_pos.y + HEALTHBAR_Y_OFFSET,
                owner_pos.z + HEALTHBAR_Z_OFFSET,
            );
        }
    }
}
```

```

    }
}

/// Green → Yellow → Red, continuous at ratio = 0.5.
fn health_color(ratio: f32) -> Color {
    if ratio >= 0.5 {
        let t = (ratio - 0.5) * 2.0; // 1.0 at full, 0.0 at half
        Color::srgb(1.0 - t * 0.8, 0.8, 0.2)
    } else {
        let t = ratio * 2.0; // 1.0 at half, 0.0 at empty
        Color::srgb(1.0, t * 0.8, 0.2)
    }
}

```

The `update_healthbars` system runs every frame. For each healthbar, it queries the owner's transform and health. If the owner no longer exists (defeated enemy), we despawn the healthbar.

For the foreground bar, we scale its width by the health ratio. At 100% health, `scale.x = 1.0` (full width). At 50% health, `scale.x = 0.5` (half width). We also shift the translation to keep the bar left-aligned as it shrinks.

The `health_color()` function returns green when above 50% health, transitioning to yellow, then red as health decreases. The formula uses linear interpolation to create smooth color transitions.

Adding Health to Characters

We need to give the player and enemies health when they spawn. Open the files where characters are created and add the Health component.

For the player, update `src/characters/spawn.rs`:

```

// src/characters/spawn.rs
// Add this import at the top
use crate::combat::Health;

// In the spawn_player_at_valid_position function,
// add Health to the entity bundle:

pub fn spawn_player_at_valid_position(
    mut commands: Commands,
    // ... existing parameters
) {
    // ... existing code ...

    commands.spawn((
        Player,
        // ... existing components ...
        Health::new(character_entry.max_health), // Add this line
    ));
}

```

For enemies, update `src/enemy/spawn.rs`:

```

// src/enemy/spawn.rs
// Add this import at the top
use crate::combat::Health;

// In the spawn function where enemies are created,
// add Health to the entity bundle:

commands.spawn((

```

```

    Enemy,
    // ... existing components ...
    Health::new(character_entry.max_health), // Add this line
  ));

```

When spawning a character, we read their `max_health` from the character definition loaded from the RON file. This value is passed to `Health::new()`, which creates a Health component with current HP equal to max HP (full health).

Now every player and enemy entity will have health tracking and automatically get a health bar.

Registering the Combat Systems

We've created several new systems, but they won't run until we register them with Bevy's app.

Open `src/combat/mod.rs` and update it:

```

// src/combat/mod.rs
mod events; // Add this line
mod observers; // Add this line
pub mod health; // Add this line
pub mod healthbar; // Add this line

mod player_combat;
mod power_type;
pub mod systems;

pub use health::Health; // Add this line
pub use player_combat::PlayerCombat;

pub use power_type::{PowerType, PowerVisuals};

// Update this import
pub use systems::{debug_switch_power, handle_power_input, spawn_projectile, ProjectileOwner};

use bevy::prelude::*;
use crate::state::GameState; // Add this line

pub struct CombatPlugin;

impl Plugin for CombatPlugin {
    fn build(&self, app: &mut App) {
        app
            // Register observers for combat events
            .add_observer(observers::on_projectile_hit) // Add this line
            .add_observer(observers::on_entity_death) // Add this line
            // Update the following systems
            .add_systems(
                Update,
                (
                    handle_power_input,
                    debug_switch_power,
                    systems::move_projectiles,
                    systems::check_projectile_hits,
                    healthbar::spawn_healthbars,
                    healthbar::update_healthbars,
                )
                .chain()
                .run_if(in_state(GameState::Playing)),
            );
    }
}

```

We declare the new `events` and `observers` modules at the top. These are private modules since other systems don't need direct access to the event definitions or observer implementations.

In the plugin's `build` method, we register observers using `.add_observer()`. Observers are not added to the `Update` schedule. Instead, they're registered as callbacks that Bevy will invoke when their specific events trigger.

The plugin registers two observers:

- `on_projectile_hit` applies damage when projectiles hit targets
- `on_entity_death` despawns entities when their health reaches zero

And five systems in a chain:

- `move_projectiles` moves hitboxes forward and despawns them on timeout
- `check_projectile_hits` detects collisions and triggers `ProjectileHit` events
- `spawn_healthbars` creates health bars for new Health components
- `update_healthbars` keeps health bars synced with character position and HP

Adding Game Over and Restart

Right now, when the player dies, the game freezes with no feedback. The player entity despawns, but there's no indication of what happened and no way to restart. We need a game over screen and restart capability.

Adding the GameOver State

First, add a new state variant for game over.

Update `src/state/game_state.rs`:

```
// src/state/game_state.rs
use bevy::prelude::*;

#[derive(States, Default, Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum GameState {
    #[default]
    Loading,
    Playing,
    Paused,
    GameOver, // Add this line
}
```

Adding `GameOver` to the state enum creates a new game state. When the player dies, we'll transition from `Playing` to `GameOver`. When the player presses R to restart, we'll transition back to `Playing`.

This creates a loop: Loading → Playing → GameOver → Playing → GameOver → ... allowing infinite restarts.

Creating the Game Over Screen

Now create the systems that handle the game over state.

Create `src/state/game_over.rs`:

```
// src/state/game_over.rs
use bevy::prelude::*;

use crate::characters::spawn::PlayerSpawned;
use crate::combat::healthbar::HealthBarOwner;
```



```

use crate::combat::systems::{Projectile, ProjectileEffect};
use crate::enemy::{spawn::EnemiesSpawned, Enemy};
use crate::particles::components::{Particle, ParticleEmitter};

use super::GameState;

#[derive(Component)]
pub struct GameOverScreen;

pub fn spawn_game_over_screen(mut commands: Commands) {
    commands
        .spawn((
            GameOverScreen,
            Node {
                width: Val::Percent(100.0),
                height: Val::Percent(100.0),
                justify_content: JustifyContent::Center,
                align_items: AlignItems::Center,
                ..default()
            },
            BackgroundColor(Color::srgba(0.0, 0.0, 0.0, 0.85)),
        ))
        .with_children(|parent| {
            parent.spawn((
                Text::new("GAME OVER\n\nPress R to restart"),
                TextFont {
                    font_size: FontSize::Px(48.0),
                    ..default()
                },
                TextColor(Color::WHITE),
                TextLayout::justify(Justify::Center),
            ));
        });

    info!("Game over screen spawned");
}

pub fn despawn_game_over_screen(mut commands: Commands, query: Query<Entity, With<GameOverScreen>>>)
{
    for entity in query.iter() {
        commands.entity(entity).despawn();
    }
    info!("Game over screen despawned");
}

pub fn handle_restart_input(
    input: Res<ButtonInput<KeyCode>>,
    mut next_state: ResMut<NextState<GameState>>,
) {
    if input.just_pressed(KeyCode::KeyR) {
        info!("Restarting game...");
        next_state.set(GameState::Playing);
    }
}

/// Despawns all gameplay entities and resets spawn flags so they re-trigger.
pub fn cleanup_game_world(
    mut commands: Commands,
    enemies: Query<Entity, With<Enemy>>,
    projectiles: Query<Entity, With<Projectile>>,
    projectile_effects: Query<Entity, With<ProjectileEffect>>,
    emitters: Query<Entity, With<ParticleEmitter>>,
    particles: Query<Entity, With<Particle>>,
    healthbars: Query<Entity, With<HealthBarOwner>>,
    mut player_spawned: ResMut<PlayerSpawned>,

```

```

    mut enemies_spawned: ResMut<EnemiesSpawned>,
) {

    for entity in enemies.iter() {
        commands.entity(entity).despawn();
    }
    for entity in projectiles.iter() {
        commands.entity(entity).despawn();
    }
    for entity in projectile_effects.iter() {
        commands.entity(entity).despawn();
    }
    for entity in emitters.iter() {
        commands.entity(entity).despawn();
    }
    for entity in particles.iter() {
        commands.entity(entity).despawn();
    }
    for entity in healthbars.iter() {
        commands.entity(entity).despawn();
    }

    player_spawned.0 = false;
    enemies_spawned.0 = false;

}

```

`spawn_game_over_screen` creates a UI overlay with a semi-transparent black background and centered white text. The text shows “GAME OVER” and restart instructions.

`despawn_game_over_screen` removes the game over UI when transitioning back to playing.

`handle_restart_input` checks if the R key was pressed. If so, it transitions to the `Playing` state, which will trigger cleanup and respawn systems.

`cleanup_game_world` queries for all gameplay entities (enemies, projectiles, particle effects, emitters, particles, health bars) and despawns them. It then resets the `PlayerSpawned` and `EnemiesSpawned` flags to `false`, which tells the spawn systems to create fresh characters when entering the Playing state.

Without resetting these flags, characters wouldn’t respawn because the system would think they already exist.

Detecting Player Death

Update the death observer to detect when the player dies and transition to game over.

Update `src/combat/observers.rs`:

```

// src/combat/observers.rs
use super::events::{EntityDeath, ProjectileHit};
use super::health::Health;
use crate::characters::input::Player; // Add this import
use crate::state::GameState; // Add this import
use bevy::prelude::*;

// ... on_projectile_hit stays the same ...

/// Observer that handles entity death by despawning the entity.
/// If the dying entity is the player, transitions to GameOver state.
pub fn on_entity_death(
    death: On<EntityDeath>,
    mut commands: Commands,
    players: Query<(), With<Player>>, // Add this parameter

```

```

    mut next_state: ResMut<NextState<GameState>>, // Add this parameter
) {
    let entity = death.entity; // Add this line
    let is_player = players.get(entity).is_ok(); // Add this line

    info!("Entity {:?} defeated!", entity);
    commands.entity(entity).despawn();

    // Add this line
    if is_player {
        info!("Player defeated! Game Over.");
        next_state.set(GameState::GameOver);
    }
}

```

We add a `players` query that matches entities with the `Player` component. When an entity dies, we check if it's in the players query using `.get(entity).is_ok()`.

If the dying entity is the player, we log a special message and transition to the `GameOver` state. This will trigger `spawn_game_over_screen` through the state plugin.

Enemies still just despawn without changing state. Here's a homework for you, try to make "You are a victorious!" message appear when the player defeats all enemies.

The `cleanup_game_world` system needs to query for health bars.

So export it from `src/combat/mod.rs`:

```

// src/combat/mod.rs
// Add to existing exports
pub use healthbar::HealthBarOwner;

```

Making `HealthBarOwner` public allows other modules (like `state/game_over.rs`) to query for health bar entities. The cleanup system needs this to despawn health bars during restart.

Registering Game Over Systems

Update the state plugin to register game over systems.

Update `src/state/mod.rs`:

```

// src/state/mod.rs
mod game_over; // Add this line
mod game_state;
mod loading;
mod pause;

// ... existing imports and code ...

impl Plugin for StatePlugin {
    fn build(&self, app: &mut App) {
        app.init_state::<GameState>()
            // ... existing Loading and Pause systems ...

        // Game Over state systems
        .add_systems(OnEnter(GameState::GameOver), game_over::spawn_game_over_screen)
        .add_systems(
            OnExit(GameState::GameOver),
            (game_over::despawn_game_over_screen, game_over::cleanup_game_world),
        )
        .add_systems(
            Update,
            game_over::handle_restart_input.run_if(in_state(GameState::GameOver)),

```

```
    );  
}  
}
```

We register three sets of systems for the `GameOver` state:

`OnEnter(GameState::GameOver)` runs `spawn_game_over_screen` when transitioning to game over. This happens immediately after the player dies.

`OnExit(GameState::GameOver)` runs when leaving game over (when R is pressed). It runs both `despawn_game_over_screen` to remove the UI and `cleanup_game_world` to reset the game. These run before entering the `Playing` state, so the world is clean when characters respawn.

The `Update` system runs `handle_restart_input` every frame, but only when in the `GameOver` state. This detects the R key press.

Run your game.

```
cargo run
```



Mac Users: If you're on macOS, pressing `Ctrl` + `Left/Right Arrow` keys quickly can trigger the system shortcut to switch workspaces, interrupting your game.

Homework: Try adding support for the `Meta` (Command) key as an alternative fire button to fix this!

The Impatient Programmer's Guide to Bevy and Rust: Chapter 9 - Let The World Scale

Our game world is tiny, let's make the map bigger. But when you try to increase `GRID_X` and `GRID_Y`, the Wave Function Collapse algorithm literally collapses. By the end of this chapter, we'll break those limits and generate a massive world.



Why WFC Struggles with Large Grids

As we learned in chapter 2, Wave Function Collapse is a constraint satisfaction algorithm. Every tile placement constrains which tiles can be placed nearby. As the grid grows, the number of constraints explodes exponentially.

A 25x18 grid has 450 positions to fill. A 75x54 grid has 4,050 positions. That's 9x more tiles, but the computational cost is far worse than 9x. Each failed placement forces the algorithm to undo and retry, and with exponentially more states to search, that cost compounds fast. WFC can fail completely when it runs out of valid options for a tile.

The solution isn't to make WFC generate bigger maps. It's to make the problem smaller. Generate multiple small chunks that WFC can handle easily, then stitch them together to form a massive world.

This chapter builds heavily on Wave Function Collapse (WFC) from Chapter 2, a quick recap will strengthen your foundation before diving in. And if parts of this chapter feel tough on the first read, that's expected. The concepts here are just as challenging, so don't give up, try again, it gets clearer.

The Multi-Chunk Strategy

We'll understand the technique using a 3x3 grid of chunks, then scale it to 100. Think of a **chunk** as a separate, mini-map that WFC generates in isolation. Each one is 25x18 tiles, small enough for the algorithm to solve instantly. But we'll generate them in a specific order and pre-seed their borders to ensure they connect.

What do you mean by pre-seed?

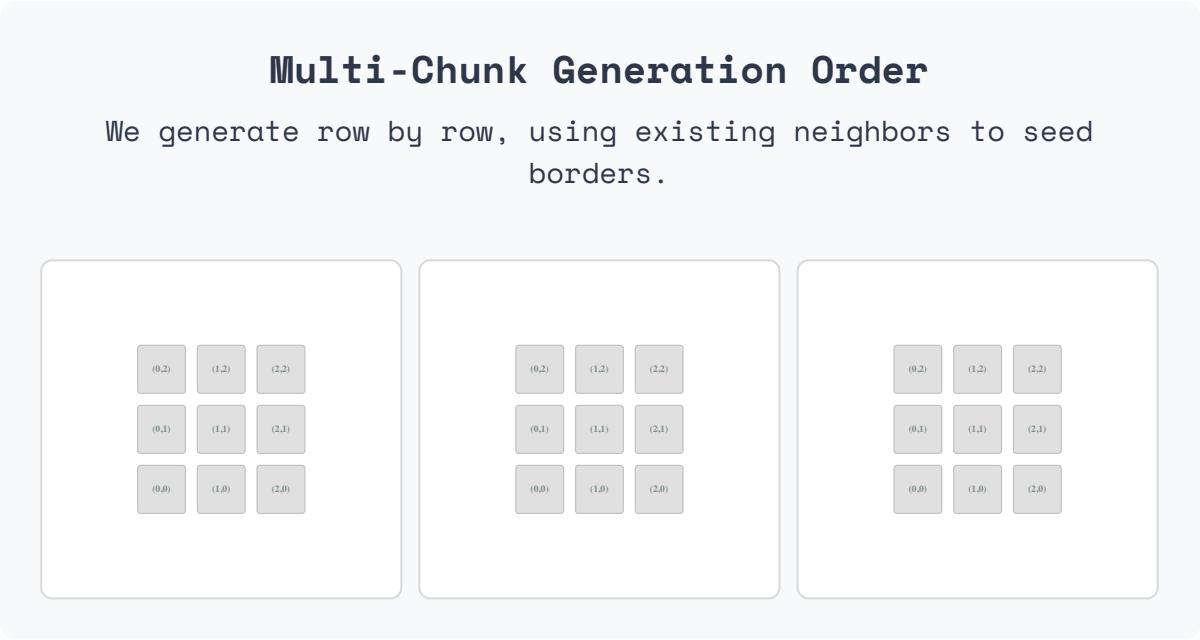
Normally, WFC starts with a completely empty grid, any tile can go anywhere. This creates a problem when generating chunks side-by-side.

Imagine chunk A generates with grass along its right edge. Now you generate chunk B next to it. Since chunk B doesn't know anything about chunk A, WFC might fill chunk B's left edge with water or stone. When you place them side-by-side: grass meets water. It would make a jarring transition and the world breaks.

Pre-seeding solves this. Each chunk is a small, self-contained map (like a 25x18 grid). When we generate chunk B, we lock in its left edge first by copying the exact grass tiles from chunk A's right edge. Now when WFC runs on chunk B, it sees: "The left edge is already grass, figure out the rest." WFC naturally generates terrain that flows from that grass border. The chunks connect perfectly because chunk B was literally built to fit chunk A.

So the generation order matters?

Yes, we generate chunks in row major order: bottom left first, then across the row, then up to the next row. This ensures that when we generate a chunk, its left and bottom neighbors already exist.





The fix: make chunks spatially overlap by exactly 1 tile width. If chunk (0,1) spans from `x=0` to `x=1600`, we start chunk (1,1) a bit earlier at `x=1536` (which is 1600 minus our 64px tile size) instead of `x=1600`. This way, the duplicated border tiles render at the **exact same coordinate**, merging seamlessly.

Instead of `TOTAL_GRID_X = CHUNKS_X * GRID_X` (which would place chunks adjacent), we use:

```
// Pseudo code don't use
TOTAL_GRID_X = CHUNKS_X * GRID_X - (CHUNKS_X - 1)
```

With 3 chunks across, 25 tiles wide each, we have 2 overlaps. So the total width is 73 tiles $((3 \times 25) - 2)$. For height, chunks are 18 tiles tall, so the total height is 52 tiles $((3 \times 18) - 2)$.

To make them overlap by 1 tile, we place each chunk exactly (chunk width - 1) tiles apart.

To support the latest features and async generation capabilities, we need to update our dependencies.

Open `Cargo.toml` and ensure you are using version `0.3` of `bevy_procedural_tilemaps`:

```
[dependencies]
bevy_procedural_tilemaps = "0.4"
# ... other dependencies
```

The Need for Async Generation

Generating 9 chunks is fast. Generating 100 chunks (10x10 world) is not. If we run WFC for 100 chunks on the main thread, the game will completely freeze for several seconds on startup. The window might even be flagged as “not responding” by the OS.

To fix this, we’ll run the generation in a **background task**. The main thread will stick to rendering a loading screen, while a separate thread churns through the WFC logic.

This requires a few architectural changes:

1. **State Management:** We need to pause the game in a `Loading` state until the map is ready.
2. **Task Polling:** We need a system to check “is the map done yet?” every frame.
3. **Thread Safety:** We need to share data (like WFC rules) between threads safely.

With the architecture planned, let's start by defining the scale of our new world.

Adding Chunk Configuration

Open `src/config.rs` and add chunk dimensions to the `map` module:

```
// src/config.rs
// Add these constants to the map module

/// Map/terrain configuration
pub mod map {
    /// Size of a single tile in world units (64px base * 1.0 scale = 64)
    /// NOTE: This must match TILE_SIZE in generate.rs!
    pub const TILE_SIZE: f32 = 64.0;

    /// Grid dimensions (per chunk)
    pub const GRID_X: u32 = 25;
    pub const GRID_Y: u32 = 18;

    /// Add this
    /// Number of chunks in each direction
    /// We're going big: 10x10 = 100 chunks!
    pub const CHUNKS_X: u32 = 10;
    pub const CHUNKS_Y: u32 = 10;

    /// Add this
    /// Total unique grid dimensions (chunks overlap by 1 tile at borders)
    pub const TOTAL_GRID_X: u32 = CHUNKS_X * GRID_X - (CHUNKS_X - 1);
    pub const TOTAL_GRID_Y: u32 = CHUNKS_Y * GRID_Y - (CHUNKS_Y - 1);

    /// Z-height of each layer (used for Y-based depth sorting)
    pub const NODE_SIZE_Z: f32 = 1.0;
}
```

`CHUNKS_X` and `CHUNKS_Y` define the chunk grid (10x10 = 100 chunks). `TOTAL_GRID_X` and `TOTAL_GRID_Y` calculate the actual unique tile count accounting for the 1-tile overlap at each border.

Rewriting the Generator for Async

Now for the heavy lifting. We need to completely rewrite `src/map/generate.rs` to move the WFC logic off the main thread.

Replace the entire contents of `src/map/generate.rs` with the following sections.

Imports and Resources

To make map generation asynchronous, we can't just throw a function onto a thread and forget about it. The main thread (which runs the UI) and the background thread (which runs the generator) need a way to coordinate.

We need to **launch** the background task, **track its progress** so the loading screen can update (e.g., "5/100 chunks done"), and **store the generated grids** until the main thread is ready to spawn them. Finally, we need a way to **signal completion** so we can clean up the resources and transition the game state.

Let's define the resources that will facilitate this coordination.

We need Bevy's `tasks` module for background execution, and standard library `sync` to share data safely.

```
// Rewrite src/map/generate.rs
use std::collections::HashMap;
```

```

use std::sync::atomic::{AtomicU32, Ordering};
use std::sync::Arc;

use bevy::prelude::*;
use bevy::tasks::{block_on, futures_lite::future::poll_once, AsyncComputeTaskPool, Task};
use bevy_procedural_tilemaps::prelude::*;
use bevy_procedural_tilemaps::proc_gen::generator::model::ModelInstance;
use bevy_procedural_tilemaps::proc_gen::generator::rules::Rules;
use bevy_procedural_tilemaps::proc_gen::grid::GridData;

use crate::config::map::{
    CHUNKS_X, CHUNKS_Y, GRID_X, GRID_Y, NODE_SIZE_Z, TILE_SIZE, TOTAL_GRID_X, TOTAL_GRID_Y,
};
use crate::map::{
    assets::{load_assets, prepare_tilemap_handles},
    rules::build_world,
};

const ASSETS_PATH: &str = "tile_layers";
const TILEMAP_FILE: &str = "tilemap.png";
const NODE_SIZE: Vec3 = Vec3::new(TILE_SIZE, TILE_SIZE, NODE_SIZE_Z);
const ASSETS_SCALE: Vec3 = Vec3::new(2.0, 2.0, 1.0);
const GRID_Z: u32 = 5;

/// Safety cap to avoid infinite loops if strict seam constraints cannot be solved.
const MAX_BACKTRACKS: u32 = 2048;

/// Shared progress counter for the loading screen.
#[derive(Resource)]
pub struct MapGenProgress {
    pub current: Arc<AtomicU32>,
    pub total: u32,
}

/// Marker resource: inserted when the map is fully spawned.
#[derive(Resource)]
pub struct MapReady;

/// Stores the spawner and grid template needed after background generation completes.
#[derive(Resource)]
pub struct MapSpawnResources {
    spawner: NodesSpawner<Sprite>,
    grid_template: CartesianGrid<Cartesian3D>,
}

/// Background task producing generated chunk data.
#[derive(Resource)]
pub struct MapGenTask(Task<Vec<ChunkResult>>);

struct ChunkResult {
    grid_data: GridData<Cartesian3D, ModelInstance, CartesianGrid<Cartesian3D>>,
    chunk_offset: Vec3,
    chunk_x: u32,
    chunk_y: u32,
}

```

Here's what each piece does:

MapGenProgress: This tracks how many chunks have been generated. It uses two special Rust types to handle thread safety: **Arc** (Atomic Reference Counted): It's like a shared "ticket" that lets multiple threads (UI and Background) point to the *same* piece of data in memory without fighting over ownership.

AtomicU32: A special integer that supports safe, simultaneous updates. If two threads try to write to a normal **u32** at the same time, the program could crash. An **Atomic** integer handles the

traffic safely, ensuring our progress bar updates smoothly without glitches.

Why is it called Atomic Reference Counted?

“Reference Counted” means it keeps track of how many owners exist. When the count drops to zero, the data is automatically cleaned up. “Atomic” means this counting happens safely across threads, without it, two threads dropping the data at the same time could cause a memory error.

MapReady: A simple “marker” component. When this exists, we know map generation is complete.

MapSpawnResources: The background thread can generate *data* (grids), but it cannot spawn *Entities*. Only the main thread has access to **Commands**. So we store the **NodesSpawner** and **CartesianGrid** in this resource to keep them readily available for the main thread when it’s time to spawn the finished map.

MapGenTask: This is our direct link to the running background task. We hold onto it so we can check it every frame and ask: “Are you done yet?”

ChunkResult: This struct acts as a data container sent from the background thread. Since the background thread can’t access **Commands** or the Bevy **World** to spawn entities directly, it packs the raw generation data (grid, position, ID) into this struct and sends it back to the main thread.

The Setup System

The **setup_generator** system has one job: **prepare everything the background thread needs and launch it**.

It parses the assets, builds the WFC rules, and creates the grid template. Crucially, it wraps the rules in an **Arc** so they can be safely shared. It doesn’t actually *run* the generation; it just spawns the task in the background and leaves a **MapGenTask** resource for us to check later.

```
// Append this to src/map/generate.rs

pub fn setup_generator(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
) {
    // 1. Build rules, models, and assets (shared across all chunks)
    let (assets_definitions, models, socket_collection) = build_world();

    let rules = RulesBuilder::new_cartesian_3d(models, socket_collection)
        .with_rotation_axis(Direction::ZForward)
        .build()
        .unwrap();
    let rules_arc = Arc::new(rules);

    let grid_template =
        CartesianGrid::new_cartesian_3d(GRID_X, GRID_Y, GRID_Z, false, false, false);

    let tilemap_handles =
        prepare_tilemap_handles(&asset_server, &mut atlas_layouts, ASSETS_PATH, TILEMAP_FILE);
    let models_assets = load_assets(&tilemap_handles, assets_definitions);
    let spawner = NodesSpawner::new(models_assets, NODE_SIZE, ASSETS_SCALE);

    // Store resources needed for spawning later
    commands.insert_resource(MapSpawnResources {
        spawner,
        grid_template: grid_template.clone(),
    });

    // Initialize progress tracking
    let progress = Arc::new(AtomicU32::new(0));
    commands.insert_resource(MapGenProgress {
```

```

        current: progress.clone(),
        total: CHUNKS_X * CHUNKS_Y,
    });

    // Spawn the background task
    let pool = AsyncComputeTaskPool::get();
    let task = pool.spawn(async move {
        generate_all_chunks(rules_arc, grid_template, progress)
    });
    commands.insert_resource(MapGenTask(task));
}

```

Here is the breakdown of the setup process:

Build Shared Data: We load assets and build rules just like before. But instead of using them immediately, we wrap the `rules` in `Arc::new(rules)`. This makes them thread-safe and shareable.

Initialize Progress: We create an `AtomicU32` initialized to 0 and wrap it in an `Arc`. We give a clone to the `MapGenProgress` resource (for the UI) and will soon give another clone to the background task.

Spawn the Task: `AsyncComputeTaskPool::get().spawn(...)` is the Bevy way to say “run this on a background thread.” We pass in our `Arc` clones. Inside the `async move` block, the task takes ownership of these clones and runs our generation function.

Save the Handle: The `spawn` function returns a `Task`. We wrap this in `MapGenTask` and save it as a resource. This is our “ticket” to check back later.

This system runs instantly, so the game frame rate remains smooth. The `generate_all_chunks` function (which we’ll write next) will run on a separate thread.

The Polling System

The background task is running, but the main game loop doesn’t know about it. We need a system that checks every single frame: “Is the map done?”

If it’s not done, we do nothing (and let the loading screen show progress). If it *is* done, we need to take the results, spawn the actual entities into the world, and clean up our tools.

```

// Append this to src/map/generate.rs

pub fn poll_map_generation(
    mut commands: Commands,
    task: Option<ResMut<MapGenTask>>,
    resources: Option<Res<MapSpawnResources>>,
) {
    let (Some(mut task), Some(resources)) = (task, resources) else {
        return;
    };

    // Check if the task is done
    let Some(chunks) = block_on(poll_once(&mut task.0)) else {
        return; // Still running...
    };

    // Task finished! Spawn everything.
    for chunk in &chunks {
        spawn_chunk_tiles(
            &mut commands,
            &resources.grid_template,
            &resources.spawner,
            &chunk.grid_data,
            chunk.chunk_offset,
            chunk.chunk_x,

```

```

        chunk.chunk_y,
    );
}

// Cleanup and mark as ready
commands.remove_resource::<MapGenTask>();
commands.remove_resource::<MapSpawnResources>();
commands.remove_resource::<MapGenProgress>();
commands.insert_resource(MapReady);

info!(
    "Map generation complete: {}x{} chunks, {}x{} total tiles",
    CHUNKS_X, CHUNKS_Y, TOTAL_GRID_X, TOTAL_GRID_Y
);
}

```

Here's what happens inside this system:

Poll the Task: `block_on(poll_once(&mut task.0))` is the key line. It checks the background thread *without blocking the main thread*. If it returns `None`, the background thread is still working. We return and wait for the next frame. If it returns `Some(chunks)`, the map is ready!

Spawn Entities: Now that we're back on the main thread, we can iterate through the `chunks` vector and spawn the actual sprites using `spawn_chunk_tiles`.

Cleanup: We remove the `MapGenTask`, `MapSpawnResources`, and `MapGenProgress` because they've done their job.

Signal Completion: Finally, we insert the `MapReady` resource. This tells our State Management system (which we'll write soon) to switch the game state from `Loading` to `Playing`.

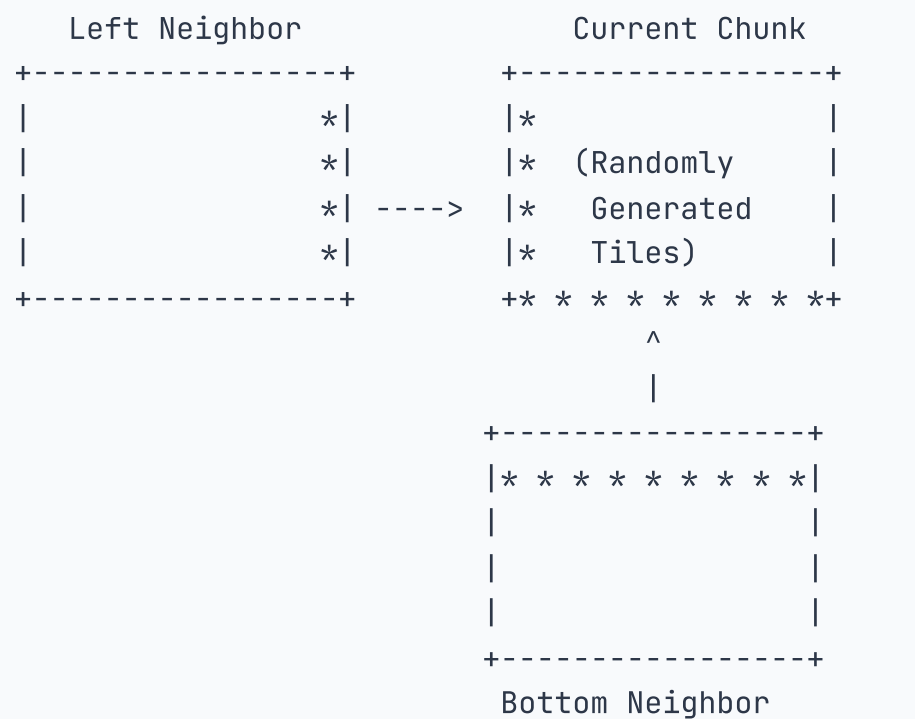
Background Generation Logic

Now for the core generation logic. As we established earlier, chunks overlap territorially by exactly one tile to prevent visible gaps. To ensure these overlapped edges merge perfectly, our background generator pre-seeds each new chunk with the exact tiles its neighbors just generated.

This brings us to the fundamental rule of our map builder:

Once overlap tiles are copied from already generated neighbors into a new chunk as starting seeds, they become immutable.

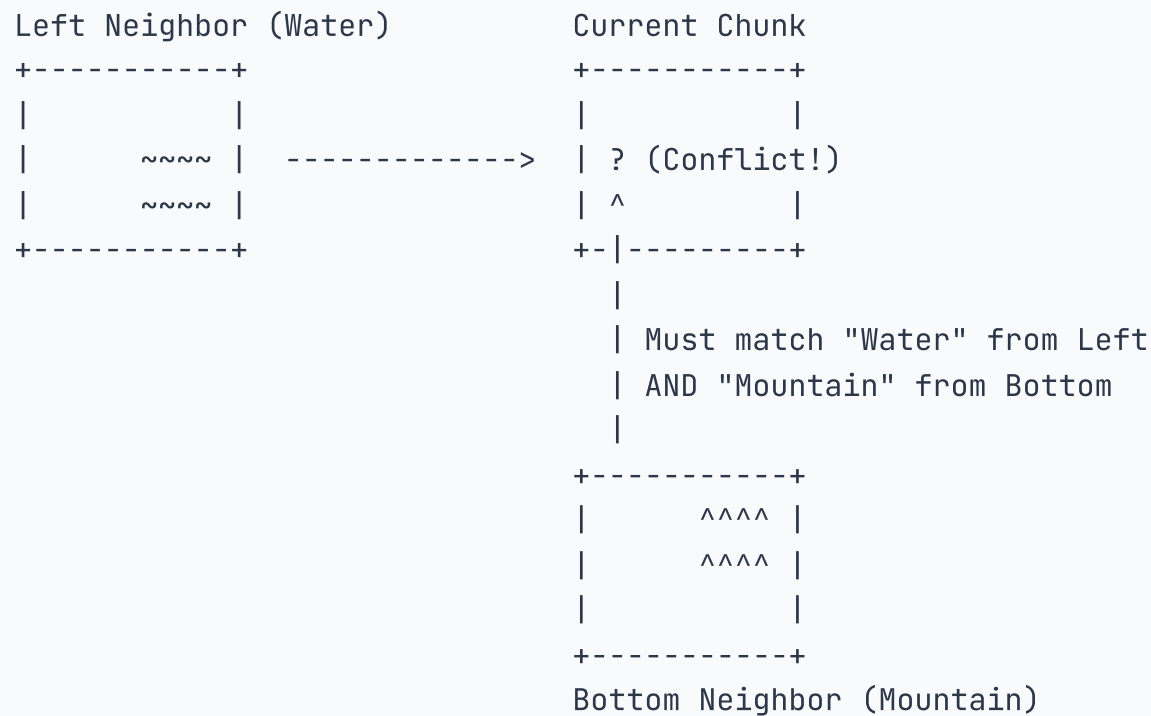
The Wave Function Collapse algorithm has no permission to alter these pinned border tiles; it must adapt the rest of the chunk to respect them.



However, this strictness can paint the algorithm into a corner literally. Because we generate in a grid (bottom-left to top-right), a new chunk often inherits constraints from *both* its left and bottom neighbors simultaneously.

If the left neighbor insists on generating Water along the edge, and the bottom neighbor generates Mountain, these two boundaries converge at the bottom-left corner of the new chunk. If our game’s rules dictate that Water and Mountain cannot legally touch without a transition tile between them, WFC instantly faces an unsolvable contradiction. It cannot satisfy the adjacency rule, nor can it modify the immutable pinned cells to fix it. The generation fails.

Here is the corner collision problem visualized:



When the algorithm gets stuck and cannot place valid tiles without altering these pinned seeds, we don't relax the rules. Instead, we **backtrack**: we throw away the current chunk, and take a few steps backward to un-generate the previously generated left and bottom neighboring chunks that caused the problem, regenerating them to find a valid connected pattern.

```
// Append this to src/map/generate.rs

fn generate_all_chunks(
    rules_arc: Arc<Rules<Cartesian3D>>,
    grid_template: CartesianGrid<Cartesian3D>,
    progress: Arc<AtomicU32>,
) -> Vec<ChunkResult> {
    let chunk_order = build_chunk_order();
    let mut generated_chunks: HashMap<
        (u32, u32),
        GridData<Cartesian3D, ModelInstance, CartesianGrid<Cartesian3D>>,
    > = HashMap::new();
    let mut index: usize = 0;
    let mut backtracks: u32 = 0;

    while index < chunk_order.len() {
        let (cx, cy) = chunk_order[index];
        let initial_nodes = build_initial_nodes(cx, cy, &generated_chunks, &grid_template);

        if let Some(grid_data) = try_generate_chunk(&rules_arc, &grid_template, &initial_nodes) {
            generated_chunks.insert((cx, cy), grid_data);
            progress.store((index as u32) + 1, Ordering::Relaxed);
            info!("Generated chunk ({} , {})", cx, cy);
            index += 1;
            continue;
        }

        let Some(backtrack_to) = backtrack_start_index(index, cx, cy) else {
            panic!(
                "Chunk ({} , {}) failed with strict seam pins and has no valid backtrack target",
                cx, cy
            );
        };

        backtracks += 1;
        if backtracks > MAX_BACKTRACKS {
            panic!(
                "Exceeded max backtracks ({} ) while generating strict stitched map",
                MAX_BACKTRACKS
            );
        }
    }
}
```

```

    let (bt_x, bt_y) = chunk_order[backtrack_to];
    warn!(
        "Chunk ({} , {}) failed with strict seam pins; backtracking to chunk ({} , {}) [{} / {}]",
        cx, cy, bt_x, bt_y, backtracks, MAX_BACKTRACKS
    );

    for rollback_index in backtrack_to..index {
        let (rx, ry) = chunk_order[rollback_index];
        generated_chunks.remove(&(rx, ry));
    }
    progress.store(backtrack_to as u32, Ordering::Relaxed);
    index = backtrack_to;
}

// Convert HashMap into results for spawning
let mut results = Vec::with_capacity((CHUNKS_X * CHUNKS_Y) as usize);
for cy in 0..CHUNKS_Y {
    for cx in 0..CHUNKS_X {
        let grid_data = generated_chunks.remove(&(cx, cy)).unwrap();
        let chunk_offset = Vec3::new(
            (cx as f32 * (GRID_X - 1) as f32 - TOTAL_GRID_X as f32 / 2.0) * TILE_SIZE,
            (cy as f32 * (GRID_Y - 1) as f32 - TOTAL_GRID_Y as f32 / 2.0) * TILE_SIZE,
            0.0,
        );
        results.push(ChunkResult {
            grid_data,
            chunk_offset,
            chunk_x: cx,
            chunk_y: cy,
        });
    }
}
results
}

```

Here is the step-by-step logic:

Iterate Chunks: We build row-major chunk coordinates once, then walk through them with an index.

Strict Seeding: For every chunk, we pre-seed overlap cells from already-generated neighbors and keep those seeds immutable.

Generate: We run the WFC algorithm on the chunk, with the pre-seeded border tiles locked in place as its starting state.

Backtrack on Failure: If generation fails, we throw away the current chunk and roll back our progress far enough to un-generate both the left and bottom neighbors causing the conflict. We then try to generate them again, creating a new, valid pattern for our pinned borders.

Update Progress: Progress reflects how far we are in the ordered chunk list and is adjusted when backtracking occurs.

Package: We convert the generated chunks into `ChunkResult` objects, calculating their exact 2D world offsets so they are ready for spawning.

Now that we understand the high-level flow of the generation loop, let's look closer at the helper functions that make it work. The most critical pieces are exactly how we determine the generation order, how we pre-seed the borders to guarantee a match, and how we execute the backtracking strategy when the algorithm gets cornered.

Generation Order and Backtracking

First, we need to establish the order in which chunks are built.

```
// Append this to src/map/generate.rs

fn build_chunk_order() -> Vec<(u32, u32)> {
    let mut order = Vec::with_capacity((CHUNKS_X * CHUNKS_Y) as usize);
    for cy in 0..CHUNKS_Y {
        for cx in 0..CHUNKS_X {
            order.push((cx, cy));
        }
    }
    order
}
```

Because we generate chunks in a strict row-major order (bottom-left to top-right), a chunk only ever has left and bottom neighbors at the time it's being generated.

If their constraints collide at a corner (like our Water/Mountain example), simply retrying the same chunk won't help. We must roll back far enough to un-generate and regenerate the previously generated neighboring chunks that caused the collision. We calculate exactly how far back to go with `backtrack_start_index`:

```
// Append this to src/map/generate.rs

fn backtrack_start_index(index: usize, cx: u32, cy: u32) -> Option<usize> {
    if cx > 0 && cy > 0 {
        // Corner conflict: reopen the 2x2 dependency root.
        Some(index - CHUNKS_X as usize - 1)
    } else if cx > 0 {
        Some(index - 1)
    } else if cy > 0 {
        Some(index - CHUNKS_X as usize)
    } else {
        None
    }
}
```

With our generation order defined and our backtracking in place, we can finally look at the function that enforces our overlapping boundaries.

How do we actually copy and pin those tiles before the new chunk generation algorithm begins so that borders match up perfectly?

Pre-Seeding Borders

Now let's see how we actually tell the generator that the new chunk's left and bottom edges are already decided. We copy the exact border tiles from the already-finished neighboring chunks and pin them as the starting state for the new chunk's generation.

```
// Append this to src/map/generate.rs

fn build_initial_nodes(
    cx: u32,
    cy: u32,
    generated_chunks: &HashMap<
        (u32, u32),
        GridData<Cartesian3D, ModelInstance, CartesianGrid<Cartesian3D>>,
    >,
    grid_template: &CartesianGrid<Cartesian3D>,
) -> Vec<((u32, u32, u32), ModelInstance)> {
    let mut initial_nodes = Vec::new();

    // Seed left column (x=0) from left neighbor's right column (x=GRID_X-1)
```

```

    if cx > 0 {
        let left_data = &generated_chunks[&(cx - 1, cy)];
        for y in 0..GRID_Y {
            for z in 0..GRID_Z {
                let src_index = grid_template.index_from_coords(GRID_X - 1, y, z);
                let model = *left_data.get(src_index);
                initial_nodes.push(((0, y, z), model));
            }
        }
    }

    // Seed bottom row (y=0) from bottom neighbor's top row (y=GRID_Y-1)
    if cy > 0 {
        let bottom_data = &generated_chunks[&(cx, cy - 1)];
        let start_x = if cx > 0 { 1 } else { 0 };
        for x in start_x..GRID_X {
            for z in 0..GRID_Z {
                let src_index = grid_template.index_from_coords(x, GRID_Y - 1, z);
                let model = *bottom_data.get(src_index);
                initial_nodes.push(((x, 0, z), model));
            }
        }
    }

    initial_nodes
}

```

Left Neighbor: If we have a neighbor to the left (`cx > 0`), we look at its *rightmost* column (`x = GRID_X - 1`). We copy those tiles and force them to be the *leftmost* column (`x = 0`) of the new chunk. **Bottom Neighbor:** If we have a neighbor below (`cy > 0`), we look at its *topmost* row (`y = GRID_Y - 1`). We copy those tiles and force them to be the *bottommost* row (`y = 0`) of the new chunk.

This ensures that before WFC even generates the first random tile for a chunk, it already has the literal exact tiles from its neighbor’s borders securely locked into the grid.

Wait, if both the left chunk and the new chunk store that border column, won’t those tiles be rendered twice?

Yes! That’s a real problem we have to address. When we later spawn tile entities for rendering, we must skip the rightmost column (and topmost row) of every chunk except the edge of the world, since that tile is already owned by the next chunk’s pre-seeded border. This is covered in the “Skipping Duplicate Overlap Tiles” section.

The Core Generation Function

With the borders pinned and our fallback plan ready, we finally call `try_generate_chunk`. This is the function that actually runs the WFC algorithm. It takes our shared rules, the grid template, and any initial pinned nodes (seeds) we collected from neighbors.

Its job is to:

1. **Protect Edges:** Ensure our seeded nodes don’t get deleted by the solver before it even begins.
2. **Configure WFC:** Set up the heuristics (e.g., Minimum Remaining Value) to optimize the search space.
3. **Run:** Execute `generate_grid()` and return the generated chunk block.

Why do we need to protect edges? (Border Zones)

We copy the finished bottom chunk’s top row into the new chunk, pinned at `y=0`. When WFC starts, it checks every tile in all directions – including the pinned ones. The tile at `y=0` looks **down** to `y=-1`, but that position doesn’t exist in this chunk’s grid. WFC sees void there and deletes the tile, wiping out the seeded border before generation starts.

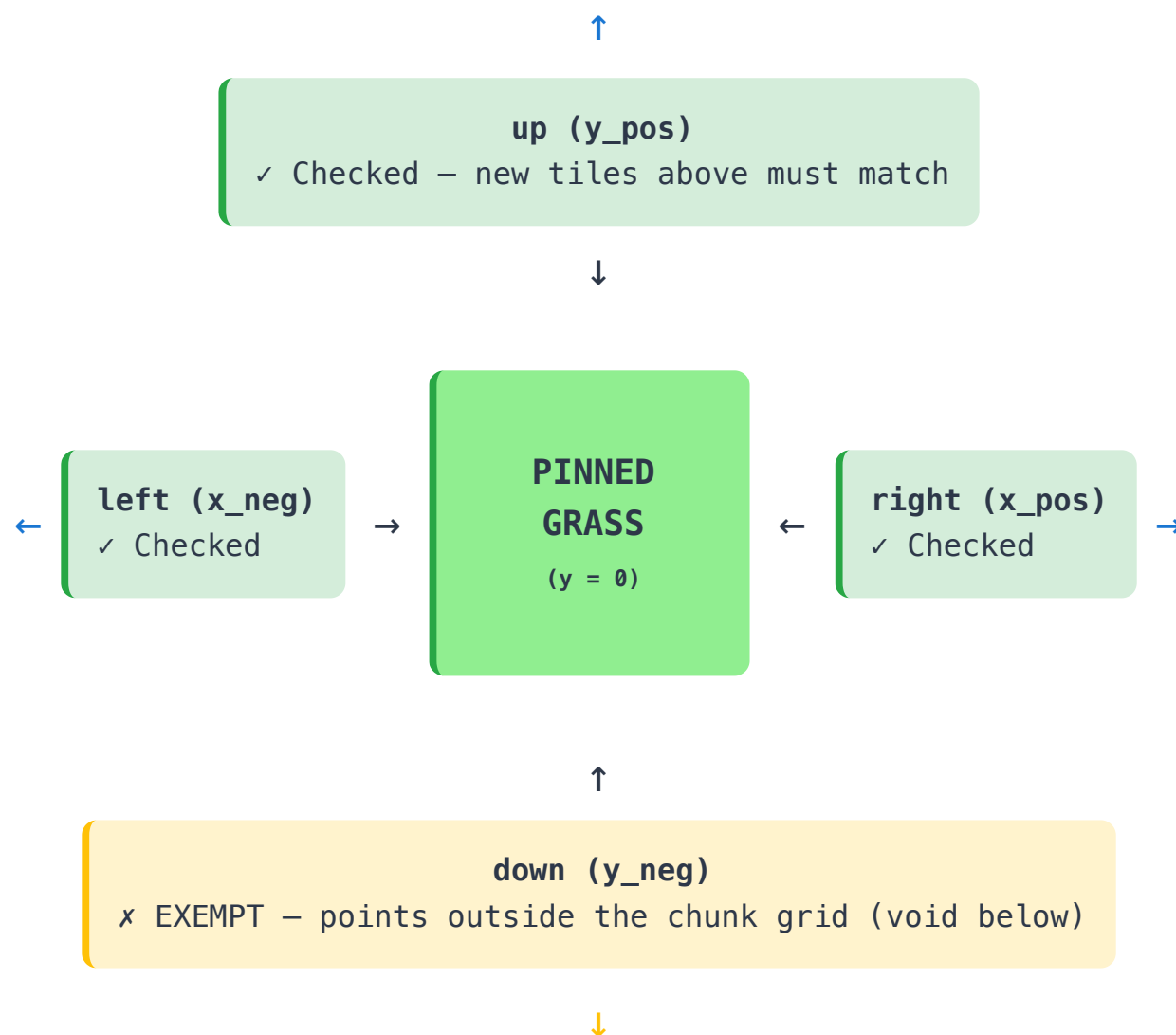
```

    Inside Chunk (y=0)
+-----+
|  G  G  G  G  G  G  | <-- Pinned border (copied from bottom neighbor)
+-----+
      |
      v  Solver looks DOWN at (y=-1)
+-----+
|      V O I D      | <-- Outside Grid (chunk grid ends here)
|  (Empty Space)   |
+-----+
      |
      +--> "I expected Grass, but I found Void!"
           "DELETE EVERYTHING!"

```

The fix is a **Border Zone**: for each pinned tile, we tell the solver to skip the void check on the face that points outside the chunk. In our bottom-border case, that is the **downward face** (`y_neg`).

If we are pinning the left-border instead (from the left neighbor), it would be the **leftward face** (`x_neg`) that gets exempted. All other connections, especially the ones facing into the new chunk, are still checked normally, so new tiles must still legally match the pinned border.



The code below does exactly this. For each pinned tile it checks which faces point outside the chunk boundary and registers a Border Zone.

```

// Append this to src/map/generate.rs

fn try_generate_chunk(
    rules: &Arc<Rules<Cartesian3D>>,
    grid: &CartesianGrid<Cartesian3D>,
    initial_nodes: &[(u32, u32, u32), ModelInstance]],
) -> Option<GridData<Cartesian3D, ModelInstance, CartesianGrid<Cartesian3D>>> {
    // Border exemptions are directional: only outward chunk-boundary directions.
    let mut border_zones = Vec::with_capacity(initial_nodes.len() * 2);
    let dir_x_forward = usize::from(Direction::XForward);
    let dir_y_forward = usize::from(Direction::YForward);

```



```

let dir_x_backward = usize::from(Direction::XBackward);
let dir_y_backward = usize::from(Direction::YBackward);
let dir_z_forward = usize::from(Direction::ZForward);
let dir_z_backward = usize::from(Direction::ZBackward);

for &((x, y, z), _) in initial_nodes {
    let idx = grid.index_from_coords(x, y, z);
    if x == 0 {
        border_zones.push((idx, dir_x_backward));
    }
    if x == GRID_X - 1 {
        border_zones.push((idx, dir_x_forward));
    }
    if y == 0 {
        border_zones.push((idx, dir_y_backward));
    }
    if y == GRID_Y - 1 {
        border_zones.push((idx, dir_y_forward));
    }
    if z == 0 {
        border_zones.push((idx, dir_z_backward));
    }
    if z == GRID_Z - 1 {
        border_zones.push((idx, dir_z_forward));
    }
}

let gen_builder = GeneratorBuilder::new()
    .with_shared_rules(rules.clone())
    .with_grid(grid.clone())
    .with_rng(RngMode::RandomSeed)
    .with_node_heuristic(NodeSelectionHeuristic::MinimumRemainingValue)
    .with_model_heuristic(ModelSelectionHeuristic::WeightedProbability)
    .with_border_zones(border_zones);

let gen_builder = if !initial_nodes.is_empty() {
    match gen_builder.with_initial_nodes(initial_nodes.to_vec()) {
        Ok(b) => b,
        Err(_) => return None,
    }
} else {
    gen_builder
};

let mut generator = match gen_builder.build() {
    Ok(g) => g,
    Err(_) => return None,
};

match generator.generate_grid() {
    Ok((_, data)) => Some(data),
    Err(_) => None,
}
}

```

The function does two things: it runs the actual WFC generation for a single chunk, and it makes sure the pre-seeded border tiles we copied from our neighbors aren't accidentally rejected before generation even begins.

First, we build a list telling the solver which edges of each pinned tile face outside the chunk. For those edges, we say: "Don't check – there's another already-solved chunk there." For all the edges that face inside our new chunk, the solver still enforces its rules normally, meaning newly placed tiles must still respect our pinned borders.

Then we configure and launch the WFC generator with the rules, grid, and our pinned starting tiles. If WFC succeeds, we get back a completed chunk. If it fails – because no valid pattern can satisfy the constraints, we return nothing, and the caller triggers backtracking.

Skipping Duplicate Overlap Tiles

Finally, when spawning tiles, we add a check to **skip redundant tiles**. Since the right column of Chunk A is identical to the left column of Chunk B (due to overlap), we only need to spawn one of them. We skip the right-most and top-most tiles of each chunk, unless it's the very last chunk on the edge of the world.

```
// Append this to src/map/generate.rs

fn spawn_chunk_tiles(
    commands: &mut Commands,
    grid: &CartesianGrid<Cartesian3D>,
    spawner: &NodesSpawner<Sprite>,
    grid_data: &GridData<Cartesian3D, ModelInstance, CartesianGrid<Cartesian3D>>,
    chunk_offset: Vec3,
    chunk_x: u32,
    chunk_y: u32,
) {
    for (node_index, instance) in grid_data.iter().enumerate() {
        let Some(node_assets) = spawner.assets.get(&instance.model_index) else {
            continue;
        };
        let position = grid.pos_from_index(node_index);

        // Optimization: Skip overlap tiles
        // The right column and top row are spawned by the next chunk.
        if position.x == GRID_X - 1 && chunk_x < CHUNKS_X - 1 {
            continue;
        }
        if position.y == GRID_Y - 1 && chunk_y < CHUNKS_Y - 1 {
            continue;
        }

        for asset in node_assets.iter() {
            let mut local_pos = Vec3::new(
                asset.world_offset.x
                    + NODE_SIZE.x
                    * (position.x as f32 + asset.grid_offset.dx as f32 + 0.5),
                asset.world_offset.y
                    + NODE_SIZE.y
                    * (position.y as f32 + asset.grid_offset.dy as f32 + 0.5),
                asset.world_offset.z
                    + NODE_SIZE.z
                    * (position.z as f32 + asset.grid_offset.dz as f32 + 0.5),
            );

            // Global z_offset for correct depth sorting across all chunks
            let global_y = chunk_y * (GRID_Y - 1) + position.y;
            local_pos.z += NODE_SIZE_Z * (1.0 - global_y as f32 / TOTAL_GRID_Y as f32);

            let world_pos = Vec3::new(
                chunk_offset.x + local_pos.x,
                chunk_offset.y + local_pos.y,
                local_pos.z,
            );

            let entity = commands.spawn_empty().id();
            let entity_commands = &mut commands.entity(entity);
            asset.assets_bundle.insert_bundle(
                entity_commands,
```

```

        world_pos,
        ASSETS_SCALE,
        instance.rotation,
    );
    (asset.spawn_commands)(entity_commands);
}
}
}

```

Iterate: We loop through every node (tile) in the generated `grid_data`.

Lookup: We use `spawner.assets.get()` to find the specific game object (like a **Tree**, **Rock**, or **Grass** sprite) associated with the model ID assigned by WFC.

Local Position: We calculate where the tile sits *inside* the chunk. This includes the base grid position plus any specific offsets defined in the asset itself.

Global Z-Sorting: We calculate `global_y` based on the chunk's Y row. We use this to adjust the Z-coordinate so that tiles lower in the world (front) render in front of tiles higher in the world (back), regardless of which chunk they belong to.

World Position: We add the `chunk_offset` to the local position to get the final `world_pos`. This places the tile in the correct spot in the massive 10x10 world.

Spawn: We spawn the entity with `commands.spawn_empty()`, add the sprite bundle at the calculated world position, and trigger any custom spawn commands associated with the asset.

The Global Z-Sorting Problem In a 2D game with a top-down view, “depth” is fake. We simulate it by drawing lower objects (smaller Y) in front of higher objects (larger Y). This usually works fine within a single chunk. But what happens at the **seam** between two chunks?

- A tree at the very bottom of the Top Chunk (high Y).
- A tree at the very top of the Bottom Chunk (low Y).

If we only sorted based on local chunk positions, the Top Chunk might be drawn *after* the Bottom Chunk, causing the top tree to accidentally cover the bottom tree. So we calculate a `global_y` that spans the *entire* 10x10 world. This ensures that a tree at `y=50` in the bottom chunk correctly sorts in front of a tree at `y=100` in the top chunk, maintaining a perfect visual illusion across the whole map.

Removing the Plugin-Based Approach

Currently, `src/main.rs` uses `ProcGenSimplePlugin` which handles generation and spawning automatically. We need to replace that with our custom async setup.

Open `src/main.rs` and remove the plugin:

```

// src/main.rs
mod map;
mod characters;
mod state;
mod collision;
mod config;
mod inventory;
mod camera;
mod combat;
mod particles;
mod enemy;

use bevy::{
    prelude::*,
    window::{MonitorSelection, Window, WindowMode, WindowPlugin},
};

```

```

use crate::camera::CameraPlugin;

//Update this import
use crate::map::generate::{setup_generator, poll_map_generation};

// Add this import
use crate::state::GameState;

fn main() {
    App::new()
        .insert_resource(ClearColor(Color::BLACK))
        .add_plugins(
            DefaultPlugins
                .set(AssetPlugin {
                    file_path: "src/assets".into(),
                    ..default()
                })
                .set(WindowPlugin {
                    primary_window: Some(Window {
                        title: "Bevy Game".into(),
                        mode: WindowMode::BorderlessFullscreen(MonitorSelection::Current),
                        ..default()
                    }),
                    ..default()
                })
                .set(ImagePlugin::default_nearest()),
        )
        // Remove this line:
        // .add_plugins(ProcGenSimplePlugin::<Cartesian3D, Sprite>::default())
        .add_plugins(state::StatePlugin)
        .add_plugins(CameraPlugin)
        .add_plugins(inventory::InventoryPlugin)
        .add_plugins(collision::CollisionPlugin)
        .add_plugins(characters::CharactersPlugin)
        .add_plugins(combat::CombatPlugin)
        .add_plugins(enemy::EnemyPlugin)
        .add_plugins(particles::ParticlesPlugin)
        .add_systems(Startup, setup_generator)

        // Add our custom generation systems
        .add_systems(Update, poll_map_generation.run_if(in_state(GameState::Loading)))
        .run();
}

```

We added `poll_map_generation` which will run every frame during the `Loading` state to check if our background thread has finished.

State Management

Since generation takes time, we need to ensure the game doesn't start `Playing` until the map is ready. We'll utilize our `GameState::Loading` state.

Open `src/state/mod.rs`. We need to register the loading screen systems and update the transition logic.

```

// src/state/mod.rs
// .. Other imports
use crate::map::generate::MapReady; // Import our new marker
// .. Existing imports

fn check_assets_loaded(
    characters_list_res: Option<Res<CharactersListResource>>,
    characters_lists: Res<Assets<CharactersList>>,

```

```

    map_ready: Option<Res<MapReady>>, // Add this resource
    mut next_state: ResMut<NextState<GameState>>,
) {
    let Some(res) = characters_list_res else {
        return;
    };

    // Wait for both character assets AND map generation to complete
    // Add an additional check to ensure the map is ready
    if characters_lists.get(&res.handle).is_some() && map_ready.is_some() {
        info!("Assets loaded, transitioning to Playing!");
        next_state.set(GameState::Playing);
    }
}

```

We updated `check_assets_loaded` to check for `Res<MapReady>`. This resource will only exist when our async map generator finishes.

The Loading Screen

Rewrite `src/state/loading.rs` to give user feedback while the background thread works.

```

// src/state/loading.rs
use std::sync::atomic::Ordering;

use bevy::prelude::*;

use crate::map::generate::{MapGenProgress, MapReady};

#[derive(Component)]
pub struct LoadingScreen;

#[derive(Component)]
pub struct LoadingText;

pub fn spawn_loading_screen(mut commands: Commands) {
    commands.spawn((
        LoadingScreen,
        Node {
            width: Val::Percent(100.0),
            height: Val::Percent(100.0),
            justify_content: JustifyContent::Center,
            align_items: AlignItems::Center,
            ..default()
        },
        BackgroundColor(Color::srgb(0.1, 0.1, 0.15)),
    )).with_children(|parent| {
        parent.spawn((
            LoadingText,
            Text::new("Loading..."),
            TextFont {
                font_size: FontSize::Px(48.0),
                ..default()
            },
            TextColor(Color::WHITE),
        ));
    });

    info!("Loading screen spawned");
}

pub fn animate_loading(

```

```

    time: Res<Time>,
    mut query: Query<&mut Text, With<LoadingText>>,
    progress: Option<Res<MapGenProgress>>,
    map_ready: Option<Res<MapReady>>,
) {
    for mut text in query.iter_mut() {
        if map_ready.is_some() {
            **text = "Starting...".to_string();
        } else if let Some(ref progress) = progress {
            // Read atomic progress safely
            let current = progress.current.load(Ordering::Relaxed);
            **text = format!("Generating world: {}/{}", current, progress.total);
        } else {
            let dots = (time.elapsed_secs() * 2.0) as usize % 4;
            **text = format!("Loading{}", ".".repeat(dots));
        }
    }
}

pub fn despawn_loading_screen(
    mut commands: Commands,
    query: Query<Entity, With<LoadingScreen>>,
) {
    for entity in query.iter() {
        commands.entity(entity).despawn();
    }

    info!("Loading screen despawned");
}

```

Map Ready: If `Res<MapReady>` exists, the job is done! We change the text to “Starting...”.

In Progress: If `Res<MapGenProgress>` exists, we read the atomic counter. `progress.current.load(Ordering::Relaxed)` is the thread-safe way to peek at the value being updated by the background thread. We update the text to show the exact count: “Generating world: 5/100”.

Just Started: If neither resource exists yet, we run a simple animation (“Loading...”, “Loading..”, etc.) based on `time.elapsed_secs()`. This gives the user immediate, accurate feedback on what the game is doing in the background.

What’s `.current.load(...)`, what’s `Ordering::Relaxed`?

`.load(...)`: Unlike normal integers, we can’t just read an atomic variable directly. We must use `.load()` to safely read its value while other threads might be writing to it.

`Ordering::Relaxed`: This tells the CPU, “I just want the latest number. I don’t care about strict synchronization with other memory operations.” This is the fastest option and is perfect for a simple UI counter where being off by a microsecond doesn’t matter.

Updating the Collision Map

The collision map needs to use the total map dimensions instead of the single-chunk dimensions.

Because our collision system uses a fixed-size grid (a 2D vector). If we initialized it with just `GRID_X` (25), it would only be big enough for *one* chunk.

As soon as we walked into the second chunk ($x > 25$), the game would crash or wrap around incorrectly. We need `TOTAL_GRID_X` to create a massive grid that can hold `Collision` data for all 100 chunks at once.

Open `src/collision/systems.rs` and update the imports and grid origin calculation:

```

// src/collision/systems.rs
use bevy::prelude::*;

```



```

use std::collections::{HashMap, hash_map::Entry};

use super::{CollisionMap, TileMarker, TileType};
// Update the following import
use crate::config::map::{TILE_SIZE, TOTAL_GRID_X, TOTAL_GRID_Y};

/// Resource to track if collision map has been built.
#[derive(Resource, Default, PartialEq, Eq)]
pub struct CollisionMapBuilt(pub bool);

pub fn build_collision_map(
    mut commands: Commands,
    mut built: ResMut<CollisionMapBuilt>,
    tile_query: Query<(&TileMarker, &Transform)>,
) {
    // Need at least one tile to proceed
    let mut tile_iter = tile_query.iter();
    let Some((first_marker, first_transform)) = tile_iter.next() else {
        return; // WFC hasn't generated tiles yet
    };

    // Update the following grid calculations
    // Calculate grid origin (centered map) using total dimensions
    let grid_origin_x = -TILE_SIZE * TOTAL_GRID_X as f32 / 2.0;
    let grid_origin_y = -TILE_SIZE * TOTAL_GRID_Y as f32 / 2.0;

    // ... rest of the function unchanged
}

```

Change the import from `GRID_X, GRID_Y` to `TOTAL_GRID_X, TOTAL_GRID_Y`, and update the grid origin calculation to use these total dimensions. The rest of the function remains the same because it dynamically calculates the actual map bounds from tile positions.

Updating Character Depth Sorting

Also, the depth sorting system needs to normalize character positions across the total map height.

As mentioned in earlier chapters, we simulate depth by drawing objects lower on the screen in front of objects higher up. This illusion needs to stay consistent across the entire map.

If we calculated this “layering” using only the height of a single chunk, the math would break as soon as you walked north into the next chunk. The game would think you were “off the chart” deep, and you’d suddenly start rendering *behind* trees that you are actually standing *in front* of.

By using `TOTAL_GRID_Y` (the height of the entire world) instead of `GRID_Y`, we ensure the depth calculation scales smoothly from the very bottom of the map to the very top. No matter where you are, you’ll always correctly appear in front of or behind other objects.

Open `src/characters/rendering.rs` and update the imports and map dimensions:

```

// src/characters/rendering.rs
use bevy::prelude::*;

use crate::characters::state::CharacterState;

// Update the following import
use crate::config::map::{TOTAL_GRID_Y, TILE_SIZE};
use crate::config::player::PLAYER_SCALE;

const NODE_SIZE_Z: f32 = 1.0;
const CHARACTER_BASE_Z: f32 = 4.0;
const CHARACTER_Z_OFFSET: f32 = 0.5;

```

```
pub fn update_character_depth(
    mut character_query: Query<&mut Transform, (With<CharacterState>, Changed<Transform>)>,
) {
    // Update the following calculations
    let map_height = TILE_SIZE * TOTAL_GRID_Y as f32;
    let map_y0 = -TILE_SIZE * TOTAL_GRID_Y as f32 / 2.0;

    let character_sprite_height = 64.0 * PLAYER_SCALE;

    for mut transform in character_query.iter_mut() {
        let character_center_y = transform.translation.y;

        let character_feet_y = character_center_y - (character_sprite_height / 2.0);

        let t = ((character_feet_y - map_y0) / map_height).clamp(0.0, 1.0);

        let character_z = CHARACTER_BASE_Z + NODE_SIZE_Z * (1.0 - t) + CHARACTER_Z_OFFSET;

        transform.translation.z = character_z;
    }
}
```

Change `GRID_Y` to `TOTAL_GRID_Y` in the import and in the calculations for `map_height` and `map_y0`. This ensures characters are depth-sorted correctly across the full 10x10 map, not just within a single chunk.

Run the game:

```
cargo run
```

Once the generation finishes, you'll be dropped into a massive world. Walk around. It goes on for screens and screens. The collision detection works everywhere. Enemies pathfind across chunk boundaries.

The Impatient Programmer's Guide to Bevy and Rust: Chapter 10 - Let There Be Memory

By the end of this chapter, you will have a complete save and load system. Your players will be able to save their progress across multiple slots, come back days later, and continue exactly where they left off.

Save systems also reveal how your game works. When you save a game, you need to capture everything that makes the current moment unique: where the player stands, what items they carry, which enemies are alive, how the world looks. This forces you to think clearly about your game's state.

What We Need to Save

Think about everything that makes your game session unique:

Category	What We Save
Player State	World position, current health, selected power type, direction
World State	Tile's attributes, pickable items, procedurally generated layout
Enemy State	Enemy's position and health, direction, enemy type
Inventory State	Item and its quantity

When you load a save, the game should look exactly as it did when you saved. The player should be in the same spot, with the same health, surrounded by the same enemies, in the same world.

Saving a game means converting Bevy's runtime data into a format that can be written to disk. This process is called serialization. When you load, you reverse it (deserialization).

In our game, the player's state lives in memory as Rust structs:

```
// In memory (runtime)
position: Vec3 { x: 120.5, y: -84.0, z: 3.0 }
health: Health { current: 75.0, max: 100.0 }
facing: Facing::Left
```

Serialization converts this into a format that can be written to a file. In JSON, that same data looks like:

```
{
  "position": [120.5, -84.0, 3.0],
  "health": { "current": 75.0, "max": 100.0 },
  "facing": "Left"
}
```

JSON is human-readable but large. Bincode encodes the same data as compact bytes: 00 00 F1 42 00 00 A8 C2 The file is smaller and faster to read/write, but not human-readable. We use bincode because save files don't need to be read by humans.

When you load, deserialization reads those bytes and reconstructs the same structs with the same values.

Rust provides excellent tools for this through the `serde` crate. Serde provides a framework that allows serialization formats (like bincode or JSON) to convert your structs and enums into bytes that can be saved to files.

But there are challenges:

1. Not Everything Can Be Saved

Some Bevy components hold references to GPU textures or internal engine state. You can't serialize these. You need to separate the data you own (position, health) from the data Bevy manages (sprite handles, texture atlases).

2. Files Can Be Tampered With

Players might edit save files to cheat. Or files might get corrupted. We need to verify that loaded data is valid.

3. Save Versions

As you update your game, your data structures change. Old saves might not work with new code. You need a way to detect this.

Let's solve these problems one by one.

We need two new crates. `bincode` serializes Rust data structures into compact binary format. `chrono` gives us timestamps for save files.

Update your `Cargo.toml`:

```
bincode = "1.3"
chrono = { version = "0.4", features = ["serde"] }
```

What does bincode do?

Bincode takes your structs and converts them into a sequence of bytes. Unlike JSON or YAML, bincode produces compact binary files that are faster to read and write. A JSON save might be 10KB; the same data in bincode could be 2KB.

Why do we need chrono?

When you save a game, you want to know when. Chrono gives you properly formatted timestamps like "16 Feb 2026, 3:45 PM" that we'll display in the save slot UI.

Before we can save data, we need to tell Rust which types can be serialized. Serde requires that every field in a struct implements `Serialize` and `Deserialize`.

Let's start with the enums that describe game state.

Facing Direction

Your characters face different directions. We need to save which way they were facing, so when you load, they're looking the same direction.

Open `src/characters/facing.rs` and add the serde derives:

```
// src/characters/facing.rs
use bevy::prelude::*;
use serde::{Serialize, Deserialize}; // Add this import

/// Add Serialize and Deserialize to the derive list
#[derive(Component, Debug, Clone, Copy, PartialEq, Eq, Default, Serialize, Deserialize)]
pub enum Facing {
    Up,
    Left,
    #[default]
```

```
    Down,  
    Right,  
}
```

We imported the `Serialize` and `Deserialize` traits from `serde`, then added them to the `derive` list. Now Rust knows how to convert a `Facing` enum into bytes and back.

The implementation is automatic. Serde generates all the conversion code at compile time. When you serialize `Facing::Down`, serde encodes which variant it is. When you deserialize, it reconstructs the exact same enum value.

Is it similar to the serialization in other languages?

Yes. Other languages have similar features. Python's `pickle` can serialize objects automatically. Java has `Serializable`. But they all work at runtime, which means errors show up when your code is already running in production.

Rust's `derive` macros generate serialization code at compile time. If you add a new field to a struct, the serialization updates automatically. If a field's type doesn't implement `Serialize`, the compiler rejects it before you can even run the program. You catch mistakes while coding, not after your players hit a corrupted save file.

Power Types

Characters have magical powers. When you save, you need to remember which power the player had equipped.

Open `src/combat/power_type.rs` and add `serde` support:

```
// src/combat/power_type.rs  
use bevy::prelude::*;  
use crate::particles::components::{EmissionShape, ParticleConfig};  
use serde::{Serialize, Deserialize}; // Add this import  
  
// Add Serialize and Deserialize to the derive list  
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default, Serialize, Deserialize)]  
pub enum PowerType {  
    #[default]  
    Fire,  
    Arcane,  
    Shadow,  
    Poison,  
}
```

Character State

Characters can be idle or walking. This affects their animation state.

Open `src/characters/state.rs` and add `derive`:

```
// src/characters/state.rs  
use bevy::prelude::*;  
use serde::{Serialize, Deserialize}; // Add this import  
  
// Add Serialize and Deserialize to the derive list  
#[derive(Component, Default, Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]  
pub enum CharacterState {  
    #[default]  
    Idle,  
    Walking,  
    Running,  
}
```

```
    Jumping,  
}
```

Tile Types

The world is made of different tile types. Each tile in your procedurally generated world has a type that determines if you can walk on it.

Open `src/collision/tile_type.rs` and add derives:

```
// src/collision/tile_type.rs  
use bevy::prelude::*;  
use serde::{Serialize, Deserialize}; // Add this import  
  
// Add Serialize and Deserialize to the derive list  
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Default, Serialize, Deserialize)]  
pub enum TileType {  
    #[default]  
    Empty,  
    Dirt,  
    Grass,  
    YellowGrass,  
    Shore,  
    Water,  
    Tree,  
    Rock,  
}
```

Why save tile types?

Your procedurally generated world has thousands of tiles. When you save, each tile's type gets recorded. When you load, we reconstruct the exact same world layout. Without saving tile types, you'd generate a different random world each time you load.

Item Types

Players collect items scattered across the world. The inventory tracks how many of each item type you have.

Open `src/inventory/inventory.rs` and update the `ItemKind` enum:

```
// src/inventory/inventory.rs  
use bevy::prelude::*;  
use serde::{Serialize, Deserialize}; // Add this import  
use std::collections::HashMap;  
use std::fmt;  
  
// Add Serialize and Deserialize to the derive list  
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]  
pub enum ItemKind {  
    Plant1,  
    Plant2,  
    Plant3,  
    Plant4,  
}
```

Then add two methods to the `Inventory` impl block to expose the internal HashMap:

```
// src/inventory/inventory.rs  
// Append these inside impl Inventory
```



```
impl Inventory {
    // ... existing methods ...

    /// Get a reference to the items map for serialization.
    pub fn items(&self) -> &HashMap<ItemKind, u32> {
        &self.items
    }

    /// Replace the entire items map (used when loading saves).
    pub fn set_items(&mut self, items: HashMap<ItemKind, u32>) {
        self.items = items;
    }
}
```

The `Inventory` resource holds items in a private `HashMap`. To save, we need to read this map. To load, we need to replace it. These methods provide controlled access without exposing the internal implementation to the rest of the codebase.

Why didn't we add these methods to other enums?

Enums like `Facing` and `PowerType` are already public and Copy. You can read and write them directly. `Inventory` is different because it wraps a private `HashMap`. Without `items()` and `set_items()`, the save system would have no way to access or replace the contents.

Building the Save Data Structure

Now we design the actual save file format. We need structures that capture the entire game state without depending on Bevy-specific types.

The save system will have two layers. The outer layer is a wrapper that verifies the save file hasn't been damaged. The inner layer is the actual game state.

Create a new directory `src/save/` and add `src/save/data.rs`:

```
// src/save/data.rs
use std::collections::HashMap;
use serde::{Serialize, Deserialize};

use crate::characters::facing::Facing;
use crate::collision::TileType;
use crate::combat::PowerType;
use crate::inventory::ItemKind;

pub const SAVE_VERSION: u32 = 1;
pub const MAX_SLOTS: usize = 5;

#[derive(Serialize, Deserialize)]
pub struct SaveFile {
    pub checksum: u64,
    pub data: Vec<u8>,
}

#[derive(Serialize, Deserialize)]
pub struct SaveData {
    pub version: u32,
    pub timestamp: String,
    pub slot_name: String,
    pub player: PlayerSave,
    pub enemies: Vec<EnemySave>,
    pub inventory: HashMap<ItemKind, u32>,
    pub tiles: Vec<TileSave>,
}
```

```

#[derive(Serialize, Deserialize)]
pub struct PlayerSave {
    pub position: [f32; 3],
    pub health_current: f32,
    pub health_max: f32,
    pub power_type: PowerType,
    pub character_name: String,
    pub character_index: usize,
    pub facing: Facing,
}

#[derive(Serialize, Deserialize)]
pub struct EnemySave {
    pub position: [f32; 3],
    pub health_current: f32,
    pub health_max: f32,
    pub character_name: String,
    pub power_type: PowerType,
    pub facing: Facing,
}

#[derive(Serialize, Deserialize)]
pub struct TileSave {
    pub position: [f32; 3],
    pub rotation: [f32; 4], // Quat (x, y, z, w) - WFC applies Z-axis rotation to tiles
    pub scale: [f32; 3],
    pub atlas_index: usize,
    pub tile_type: TileType,
    pub pickable: Option<ItemKind>,
}

#[derive(Serialize, Deserialize, Clone)]
pub struct SaveMetadata {
    pub timestamp: String,
    pub character_name: String,
    pub player_health: f32,
    pub player_max_health: f32,
}

```

`SaveFile` is the outer wrapper. It contains the actual game data as bytes plus a checksum. A checksum is a number calculated from the data, like a fingerprint. When you save, you compute the checksum and store it alongside the data. When you load, you compute it again. If the two numbers don't match, something changed the file (corruption, a crash mid-write, or manual editing) and the save is rejected.

`SaveData` is the game state. It holds the version number, a timestamp, and all the entities that matter: player, enemies, inventory, and tiles.

How does a version number help?

As you develop your game, you'll add new fields, rename structs, or change data layouts. A save file created with version 1 of your code won't deserialize correctly with version 2. The version number lets you detect this upfront: if the loaded version doesn't match the current `SAVE_VERSION`, you can choose to reject or provide compatibility.

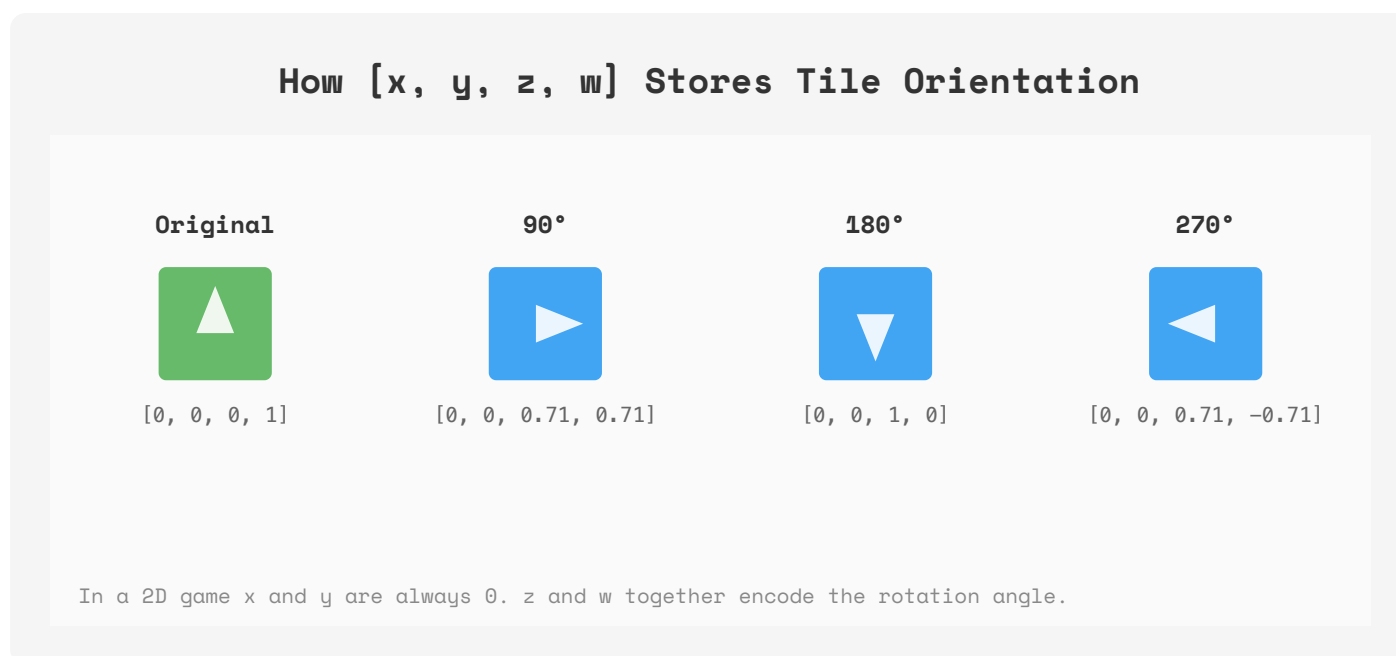
Each entity type has its own struct. Notice how we store positions as `[f32; 3]` arrays instead of `Vec3`. This is because `glam::Vec3` (the math library Bevy uses) doesn't implement Serde's `Serialize` and `Deserialize` traits by default. While we could enable the `serde` feature flag for `glam`, using a plain array `[f32; 3]` is an easy alternative that keeps our save format simple and decoupled from engine types.

For tiles, we store the rotation as a quaternion `[f32; 4]` (x, y, z, w components). Remember how in [Chapter 2](#) we created corner and edge tiles? A single grass-to-dirt corner sprite gets rotated

to cover all four directions. The save file needs to record which rotation each tile ended up with, so the loaded world looks identical.

What's a quaternion?

A way to store rotation using four numbers instead of an angle. You don't need to understand the math behind it. Just know that `[0, 0, 0, 1]` means "no rotation" and changing the z and w values rotates the tile around the screen axis.



`SaveMetadata` is a smaller structure that we save separately. It contains just enough information to display in the save slot UI (timestamp, character name, health) without loading the entire save file. This makes the UI responsive because we don't need to deserialize thousands of tiles just to show "Slot 1: Saved on 17 Feb 2026".

File Path Functions

Add these helper functions to `data.rs`:

```
// Append this to src/save/data.rs

pub fn saves_directory() -> std::path::PathBuf {
    let mut path = std::env::current_exe()
        .unwrap_or_default()
        .parent()
        .unwrap_or(std::path::Path::new("."))
        .to_path_buf();
    path.push("saves");
    path
}

pub fn save_file_path(slot: usize) -> std::path::PathBuf {
    saves_directory().join(format!("slot_{}.sav", slot))
}

pub fn meta_file_path(slot: usize) -> std::path::PathBuf {
    saves_directory().join(format!("slot_{}.meta", slot))
}
```

`saves_directory()` finds where the game executable lives, then creates a `saves` subfolder path. On Windows, this might be `C:\Users\YourName\Desktop\game\saves`. On Mac, it could be `/Users/YourName/Desktop/game/saves`.

The `current_exe()` call gets the path to your compiled binary. The `parent()` method gets the directory containing it. The `unwrap_or_default()` and `unwrap_or` calls handle edge cases where the executable path can't be determined.

`save_file_path()` and `meta_file_path()` generate filenames like `slot_0.sav` and `slot_0.meta` for a specific slot number. By keeping these centralized, if you ever want to change the naming scheme, you only change it in one place.

Checksum Function

Add this function to verify data integrity:

```
// Append this to src/save/data.rs

pub fn compute_checksum(data: &[u8]) -> u64 {
    // Simple FNV-1a hash
    let mut hash: u64 = 0xcbf29ce484222325;
    for &byte in data {
        hash ^= byte as u64;
        hash = hash.wrapping_mul(0x100000001b3);
    }
    hash
}
```

What is FNV-1a?

It's a function that takes your save data and produces a fingerprint.

Say your save file contains the player at position (120, -84) with 75 health. FNV-1a reads every byte of that data and produces a number like `14695981039346656037`. We store that number alongside the save data. When loading, we recalculate the fingerprint and compare. If the numbers don't match, we reject the file.

This acts as a basic guard against tampering. Someone could decode the binary data and edit their health, but the fingerprint would no longer match and the game would refuse to load it.

Can't they edit the data, recalculate and store it?

Yes, someone determined enough can still get around it. They can figure out the hashing algorithm, modify the data, recalculate the fingerprint, and the game would accept it. For a single-player game, that's an acceptable tradeoff. Multiplayer games solve this differently by keeping the real game state on the server, where players can't touch it.

What does `wrapping_mul` do?

A `u64` can hold numbers up to 18,446,744,073,709,551,615. The FNV-1a algorithm multiplies the running total by a large number on each step, and the result often exceeds that limit. In Rust, that's an error by default and your program crashes.

`wrapping_mul` tells Rust to discard the extra and keep only the remainder. Imagine a counter that can only display 0 to 99. If you add 10 to 95, instead of crashing, it shows 05. That's wrapping. The hash algorithm relies on this behavior to scramble the data into a unique fingerprint.

Building the Save/Load UI

The UI needs to do several things: display 5 save slots, show which slots have saves with timestamps, let players click a slot to save or load, and handle the actual operations.

We'll use a deferred execution pattern. When you click a slot, we don't save or load immediately. Instead, we record a pending action. Next frame, a separate system processes it. Because loading a save requires despawning every entity and respawning new ones. That needs `&mut World`, which gives us **exclusive access** to the entire game world.

In Bevy, if you have exclusive access to the whole world, you cannot also ask for specific pieces of it (like `Query` or `Commands`) in the same system. They are mutually exclusive. We apply the same deferred pattern to saving for consistency.

```
// Pseudo code, don't use
// You can't ask for specific queries at the same time.
fn handle_click_and_load(
    world: &mut World,           // ← "I need everything"
    buttons: Query<&Interaction>, // ← "I need just buttons" (conflict!)
    mut commands: Commands,
) { }

// Split into two systems
fn handle_click(buttons: Query<&Interaction>, mut pending: ResMut<PendingAction>) {
    // Regular system: record which slot was clicked
}
fn execute_load(world: &mut World) {
    // Exclusive system: full world access to despawn and respawn
}
```

We need a menu for saving/loading that can overlay:

1. The **Pause Menu** (to save or load during gameplay)
2. The **Main Menu** (to load a game from the title screen)

Visually, it should have:

- A title (“Save Game” or “Load Game”)
- Five slots, each showing a timestamp if a save exists
- A “Back” button to close the menu

To make this work in Bevy, we need a resource to track whether the menu is open `SaveLoadUIState`, a resource to handle the deferred save/load logic `PendingSaveLoadAction`, and marker components to identify which button is which `SlotButton`, `BackButton`.

Create `src/save/ui.rs`

```
// src/save/ui.rs
use bevy::prelude::*;

use crate::characters::config::{CharacterEntry, CharactersList};
use crate::characters::facing::Facing;
use crate::characters::input::Player;
use crate::characters::spawn::{CharactersListResource, CurrentCharacterIndex, PlayerSpawned};
use crate::collision::{CollisionMapBuilt, TileMarker};
use crate::combat::healthbar::HealthBarOwner;
use crate::combat::systems::{Projectile, ProjectileEffect};
use crate::combat::{Health, PlayerCombat};
use crate::enemy::spawn::EnemiesSpawned;
use crate::enemy::Enemy;
use crate::inventory::{Inventory, Pickable};
use crate::map::assets::TilemapHandles;
use crate::particles::components::{Particle, ParticleEmitter};
use crate::state::GameState;
use crate::state::pause::PauseMenu;
use crate::characters::animation::*;
use crate::characters::collider::Collider;
use crate::characters::physics::Velocity;
use crate::characters::state::CharacterState;
use crate::config::player::PLAYER_SCALE;

use super::data::*;
use super::systems;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum SaveLoadMode {
```

```

        Save,
        Load,
    }

#[derive(Resource, Default)]
pub struct PendingSaveLoadAction(pub Option<(SaveLoadMode, usize)>);

#[derive(Resource)]
pub struct SaveLoadUIState {
    pub active: bool,
    pub mode: SaveLoadMode,
}

impl Default for SaveLoadUIState {
    fn default() -> Self {
        Self {
            active: false,
            mode: SaveLoadMode::Save,
        }
    }
}

#[derive(Component)]
pub struct SaveLoadUI;

#[derive(Component)]
pub struct SlotButton(pub usize);

#[derive(Component)]
pub struct BackButton;

```

`SaveLoadUIState` tracks whether the UI is visible and what mode it's in (saving or loading). When the player clicks "Save Game" in the pause menu, we set `active: true` and `mode: SaveLoadMode::Save`.

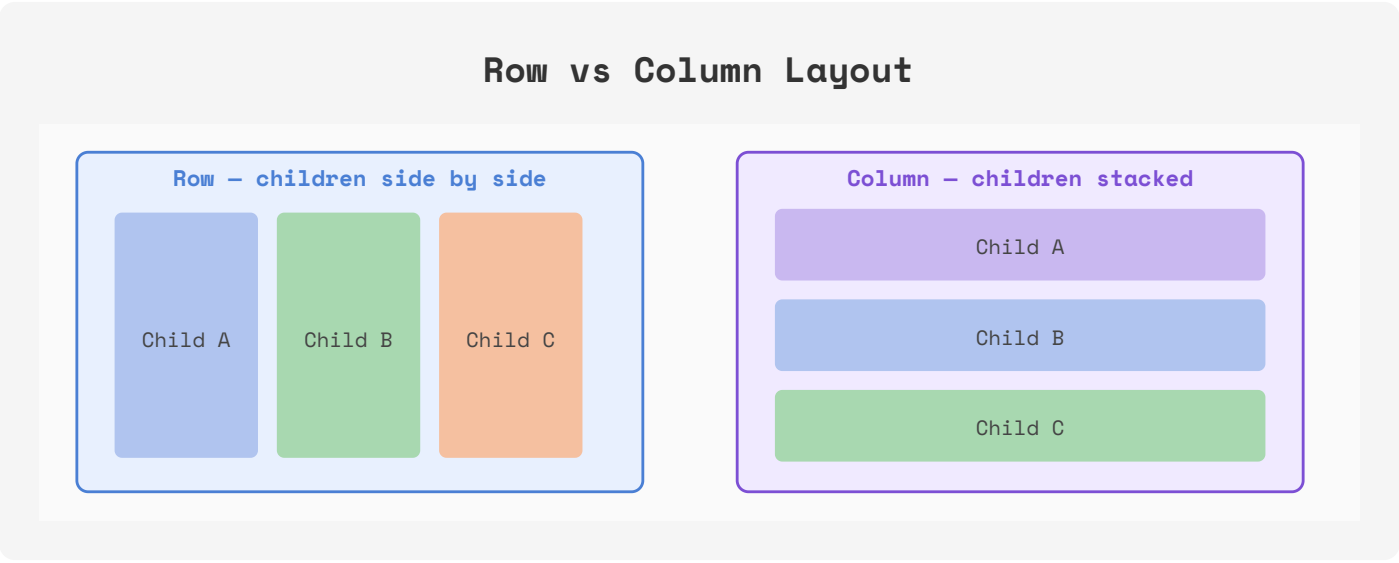
`PendingSaveLoadAction` is the deferred action resource we discussed. It queues the operation so it can be processed safely in the next frame with exclusive world access.

The component types (`SaveLoadUI`, `SlotButton`, `BackButton`) let us query specific UI elements for interaction handling.

Bevy UI

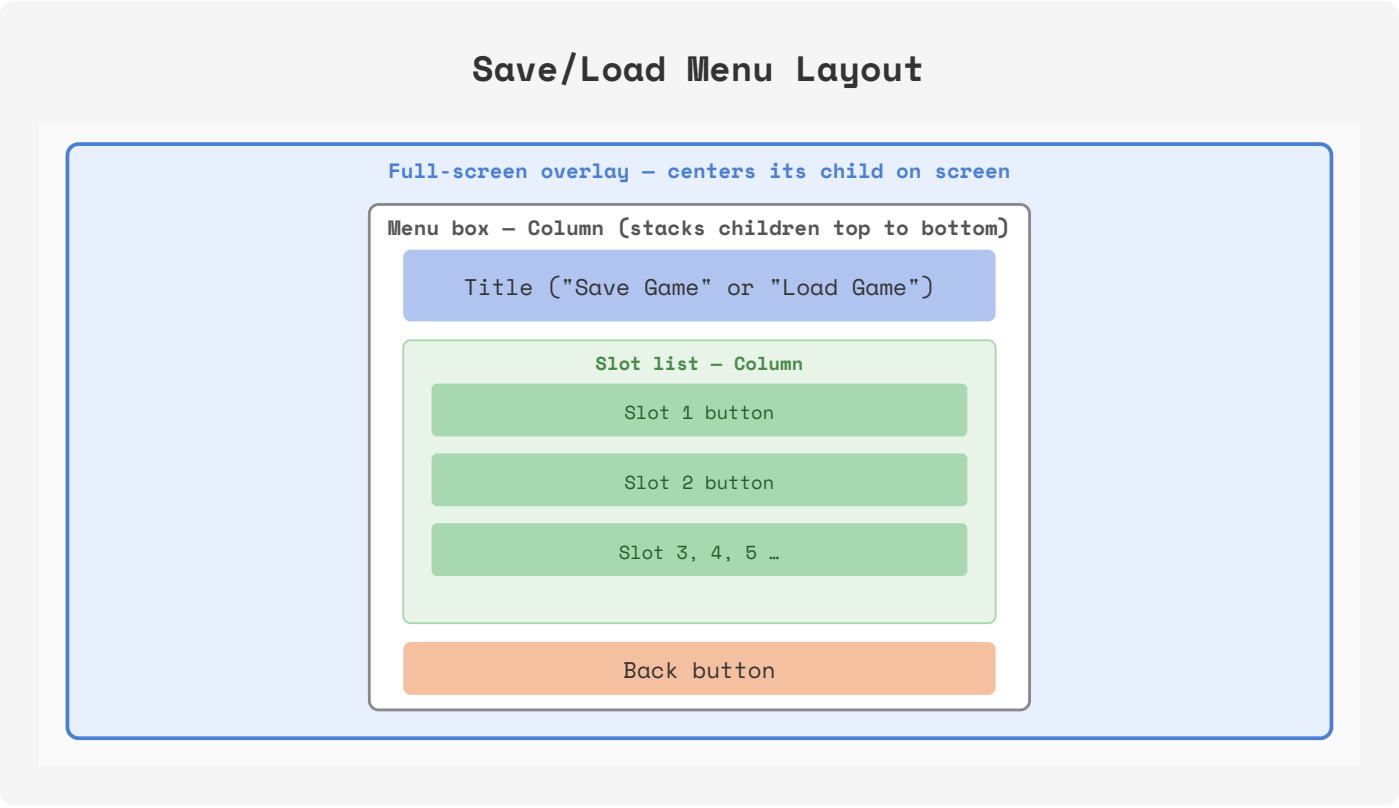
We need to build a layout that centers our menu on the screen. Bevy's UI system uses **Flexbox**, the same layout model the web uses. Every UI element is a **Node**. A Node can contain child Nodes, and you control how they are arranged using a direction.

There are two directions: **Row** and **Column**. Think of a spreadsheet, a row is horizontal (side by side) and a column is vertical (stacked). Flexbox works the same way.



Our save menu needs to stack vertically (title on top, slots in the middle, Back button at the bottom), so we use **Column**. The outer container needs to **fill the whole screen** and place the menu in the **center**, both horizontally and vertically.

Here is how those two Nodes nest together:



Spawning the UI

With that structure in mind, here is the system that builds it:

```
// Append this to src/save/ui.rs

pub fn handle_save_load_ui(
    mut commands: Commands,
    ui_state: Res<SaveLoadUIState>,
    existing_ui: Query<Entity, With<SaveLoadUI>>,
) {
    if !ui_state.is_changed() {
        return;
    }

    for entity in existing_ui.iter() {
        commands.entity(entity).despawn();
    }

    if !ui_state.active {
        return;
    }

    let title = match ui_state.mode {
```

```

        SaveLoadMode::Save => "SAVE GAME",
        SaveLoadMode::Load => "LOAD GAME",
};

let mut slot_infos: Vec<Option<SaveMetadata>> = Vec::new();
for slot in 0..MAX_SLOTS {
    slot_infos.push(systems::load_slot_metadata(slot));
}

commands
    .spawn((
        SaveLoadUI,
        Node {
            width: Val::Percent(100.0),
            height: Val::Percent(100.0),
            justify_content: JustifyContent::Center,
            align_items: AlignItems::Center,
            flex_direction: FlexDirection::Column,
            ..default()
        },
        BackgroundColor(Color::srgba(0.05, 0.05, 0.1, 1.0)),
        GlobalZIndex(100),
    ))
    .with_children(|parent| {
        parent.spawn((
            Text::new(title),
            TextFont {
                font_size: FontSize::Px(42.0),
                ..default()
            },
            TextColor(Color::WHITE),
            Node {
                margin: UiRect::bottom(Val::Px(30.0)),
                ..default()
            },
        ));

        for slot in 0..MAX_SLOTS {
            let info = &slot_infos[slot];
            let label = match info {
                Some(meta) => format!(
                    "Slot {} - {}",
                    slot + 1,
                    meta.timestamp,
                ),
                None => format!("Slot {} - Empty", slot + 1),
            };

            let is_empty = info.is_none();
            let is_load_mode = ui_state.mode == SaveLoadMode::Load;
            let disabled = is_load_mode && is_empty;

            let bg_color = if disabled {
                Color::srgba(0.2, 0.2, 0.2, 0.5)
            } else {
                Color::srgba(0.15, 0.15, 0.3, 0.9)
            };

            let text_color = if disabled {
                Color::srgba(0.5, 0.5, 0.5, 1.0)
            } else {
                Color::WHITE
            };

            let mut btn = parent.spawn((

```

```

        SlotButton(slot),
        Button,
        Node {
            width: Val::Px(500.0),
            height: Val::Px(50.0),
            justify_content: JustifyContent::Center,
            align_items: AlignItems::Center,
            margin: UiRect::vertical(Val::Px(5.0)),
            ..default()
        },
        BackgroundColor(bg_color),
    ));

    if disabled {
        btn.remove::<Button>();
    }

    btn.with_children(|btn_parent| {
        btn_parent.spawn((
            Text::new(label),
            TextFont {
                font_size: FontSize::Px(20.0),
                ..default()
            },
            TextColor(text_color),
        ));
    });
}

parent
    .spawn((
        BackButton,
        Button,
        Node {
            width: Val::Px(200.0),
            height: Val::Px(45.0),
            justify_content: JustifyContent::Center,
            align_items: AlignItems::Center,
            margin: UiRect::top(Val::Px(20.0)),
            ..default()
        },
        BackgroundColor(Color::srgba(0.4, 0.1, 0.1, 0.9)),
    ))
    .with_children(|btn_parent| {
        btn_parent.spawn((
            Text::new("Back"),
            TextFont {
                font_size: FontSize::Px(24.0),
                ..default()
            },
            TextColor(Color::WHITE),
        ));
    });
});
}

```

How Bevy UI nesting works

Here is the mental model to keep:

Spawn a parent entity, then call `.with_children(|parent| { ... })` to attach children inside it.

That closure gives you a `parent` builder. Any entity you `.spawn(...)` inside that closure becomes a child. Bevy positions it *inside* the parent according to the parent's flex settings. You can go as many levels deep as you need.

```
// Pseudo-code, don't use
commands.spawn(ParentNode).with_children(|parent| {

    parent.spawn(ChildA);

    parent.spawn(ChildB).with_children(|child_b| {
        child_b.spawn(GrandchildOfB);    // nested one more level
    });

    parent.spawn(ChildC);
});
```

We apply this pattern to build our menu: the overlay is the parent, and the title, slot list, and back button are its children.

Within that structure, here are the key decisions we made:

- `GlobalZIndex(100)` – Without this, the UI would render at the same depth as the pause menu behind it. Setting a high z-index pushes it visually on top of everything else.
- **Early return when inactive** – The system runs whenever `SaveLoadUIState` changes. If it changed to `active: false` (e.g. player closed the menu), we despawn the UI and return immediately. We don't rebuild anything we don't need.
- **Slot metadata loaded first** – Before spawning any buttons, we read the save files from disk so we know which slots are empty. This lets us set the colors and disabled state once, at spawn time, instead of polling the disk every frame. We will implement the `load_slot_metadata` later in the chapter.
- **Disabling empty slots in Load mode** – In save mode, all slots are clickable (you can overwrite any slot). In load mode, empty slots have no data, so clicking them would do nothing. We handle this by removing the `Button` component from empty slots. Bevy's interaction system only fires on entities that *have* `Button`, so removing it effectively makes the button inactive.

Handling Button Clicks

Now that the UI is visible, we need to know when the player clicks something. Bevy handles this with the `Interaction` component, which is automatically added to buttons. It tracks whether a node is `None` (idle), `Hovered`, or `Pressed`.

We need two systems: one to handle saving/loading when a slot is clicked, and one to close the menu when “Back” is clicked (or Escape is pressed).

Add these systems to `src/save/ui.rs`:

```
// Append this to src/save/ui.rs

pub fn handle_slot_buttons(
    mut ui_state: ResMut<SaveLoadUIState>,
    mut pending: ResMut<PendingSaveLoadAction>,
    interaction_query: Query<(&Interaction, &SlotButton), Changed<Interaction>>,
) {
    for (interaction, slot_btn) in interaction_query.iter() {
        if *interaction != Interaction::Pressed {
            continue;
        }
        // Button was just clicked!
        // 1. Record what needs to happen (Save or Load this slot)
        pending.0 = Some((ui_state.mode, slot_btn.0));
    }
}
```

```

        // 2. Close the UI
        ui_state.active = false;
    }
}

pub fn handle_back_button(
    mut ui_state: ResMut<SaveLoadUIState>,
    interaction_query: Query<&Interaction, (Changed<Interaction>, With<BackButton>)>,
    input: Res<ButtonInput<KeyCode>>,
) {
    // Close on ESC
    if input.just_pressed(KeyCode::Escape) {
        ui_state.active = false;
        return;
    }
    // Close on Back button click
    for interaction in interaction_query.iter() {
        if *interaction == Interaction::Pressed {
            ui_state.active = false;
        }
    }
}

```

- `Query<..., Changed<Interaction>>` – This is a performance optimization. Bevy only runs the loop if a button’s state *actually changed* (e.g. from Hovered to Pressed) this frame. We don’t waste time checking idle buttons.
- `PendingSaveLoadAction` – When a slot is clicked, we **do not** save immediately. We just record “Player wants to [Save] to [Slot 1]” in the pending resource. The actual heavy lifting happens in the deferred system we’ll write next.
- `ui_state.active = false` – As soon as an action is picked, we hide the menu so the game can resume (or the save operation can start).

Implementing Save

Now let’s add the system that writes saves to disk.

The button click didn’t save the game, it just left a note in `PendingSaveLoadAction`. Now we need a system to read that note and do the actual work.

This system will:

1. Check if there is a pending **Save** action.
2. If yes, query all the game data (player position, enemy stats, map layout).
3. Package it into our `SaveData` struct.
4. Serialize it to binary and write it to disk.

Add these functions to `src/save/ui.rs`:

```

// Append this to src/save/ui.rs

pub fn execute_save(
    mut pending: ResMut<PendingSaveLoadAction>,
    tile_query: Query<(&Transform, &Sprite, &TileMarker, Option<&Pickable>)>,
    player_query: Query<
        (&Transform, &Health, &PlayerCombat, &CharacterEntry, &Facing),
        With<Player>,
    >,
    enemy_query: Query<(&Transform, &Health, &CharacterEntry, &Facing), With<Enemy>>,
    inventory: Res<Inventory>,
    character_index: Res<CurrentCharacterIndex>,
) {
    let Some((SaveLoadMode::Save, slot)) = pending.0 else {
        return;
    }
}

```

```

};
pending.0 = None;

let Ok((player_tf, player_health, player_combat, player_entry, player_facing)) =
    player_query.single()
else {
    error!("No player found for save");
    return;
};

let player_save = PlayerSave {
    position: [
        player_tf.translation.x,
        player_tf.translation.y,
        player_tf.translation.z,
    ],
    health_current: player_health.current,
    health_max: player_health.max,
    power_type: player_combat.power_type,
    character_name: player_entry.name.clone(),
    character_index: character_index.index,
    facing: *player_facing,
};

let mut enemies = Vec::new();
for (tf, health, entry, facing) in enemy_query.iter() {
    enemies.push(EnemySave {
        position: [tf.translation.x, tf.translation.y, tf.translation.z],
        health_current: health.current,
        health_max: health.max,
        character_name: entry.name.clone(),
        power_type: crate::combat::PowerType::Fire,
        facing: *facing,
    });
}

let mut tiles = Vec::new();
for (tf, sprite, tile_marker, pickable) in tile_query.iter() {
    let atlas_index = sprite
        .texture_atlas
        .as_ref()
        .map(|a| a.index)
        .unwrap_or(0);

    let rot = tf.rotation;
    tiles.push(TileSave {
        position: [tf.translation.x, tf.translation.y, tf.translation.z],
        rotation: [rot.x, rot.y, rot.z, rot.w],
        scale: [tf.scale.x, tf.scale.y, tf.scale.z],
        atlas_index,
        tile_type: tile_marker.tile_type,
        pickable: pickable.map(|p| p.kind),
    });
}

let timestamp = chrono::Local::now().format("%d %b %Y, %I:%M %p").to_string();

let save_data = SaveData {
    version: SAVE_VERSION,
    timestamp: timestamp.clone(),
    slot_name: format!("Slot {}", slot + 1),
    player: player_save,
    enemies,
    inventory: inventory.items().clone(),
    tiles,
}

```



```

};

match do_write_save(slot, &save_data, &timestamp) {
  Ok(()) => info!("Saved to slot {}", slot + 1),
  Err(e) => error!("Failed to save: {}", e),
}

}

fn do_write_save(slot: usize, save_data: &SaveData, timestamp: &str) -> Result<(), String> {
  let data_bytes =
    bincode::serialize(save_data).map_err(|e| format!("Serialize error: {}", e))?;
  let checksum = compute_checksum(&data_bytes);
  let save_file = SaveFile {
    checksum,
    data: data_bytes,
  };
  let file_bytes =
    bincode::serialize(&save_file).map_err(|e| format!("Serialize error: {}", e))?;

  let dir = saves_directory();
  std::fs::create_dir_all(&dir).map_err(|e| format!("Create dir error: {}", e))?;
  std::fs::write(save_file_path(slot), &file_bytes)
    .map_err(|e| format!("Write error: {}", e))?;

  let metadata = SaveMetadata {
    timestamp: timestamp.to_string(),
    character_name: save_data.player.character_name.clone(),
    player_health: save_data.player.health_current,
    player_max_health: save_data.player.health_max,
  };
  let meta_bytes =
    bincode::serialize(&metadata).map_err(|e| format!("Meta serialize error: {}", e))?;
  std::fs::write(meta_file_path(slot), &meta_bytes)
    .map_err(|e| format!("Meta write error: {}", e))?;

  Ok(())
}

```

The system checks if there's a pending save action. If not, it returns immediately.

Then it queries all the entities we need: player, enemies, and tiles. For each one, it extracts the data that matters and converts it into the save format.

Notice how we query `Option<&Pickable>` for tiles. Some tiles have pickable items, some don't. The `Option` handles both cases.

For the timestamp, we use chrono to format the current time. The format string `%d %b %Y, %I:%M %p` produces "16 Feb 2026, 3:45 PM".

The `do_write_save` helper serializes everything. It creates the save directory if it doesn't exist (important for first-time saves), writes the main save file, and writes the metadata file separately.

The serialization happens in two steps: first serialize the `SaveData`, compute its checksum, wrap it in `SaveFile`, then serialize that wrapper. This creates the two-layer structure we designed earlier.

Implementing Loading

Loading happens in two steps: clear the old world, then rebuild from the save file. The first step queries for all game entities (player, enemies, tiles, etc.) and despawns them. If the player isn't in a game world yet, such as when loading from the main menu, the query returns empty and nothing is touched.

Here is what the system does when it finds a pending load:

1. Read the slot index from `PendingSaveLoadAction` and clear it.
2. Query and despawn all game entities (tiles, player, enemies, projectiles, particles).
3. Read the binary save file from disk and deserialize it.
4. Spawn new entities from the deserialized data.

```
// Append this to src/save/ui.rs

pub fn execute_load(world: &mut World) {
    let slot = {
        let pending = world.resource::();
        match pending.0 {
            Some((SaveLoadMode::Load, slot)) => slot,
            _ => return,
        }
    };
    world.resource_mut::().0 = None;

    let save_data = match systems::load_save_data(slot) {
        Ok(data) => data,
        Err(e) => {
            error!("Failed to load: {}", e);
            return;
        }
    };

    // Despawn all gameplay entities and UI overlays
    let mut to_despawn = Vec::new();
    for entity in world.query_filtered::();
```

```

ui_state.active = false;

let tilemap_handles = match world.get_resource::<TilemapHandles>() {
    Some(h) => h.clone(),
    None => {
        error!("TilemapHandles not available for loading");
        return;
    }
};

// Spawn tiles
for tile in &save_data.tiles {
    let sprite = tilemap_handles.sprite(tile.atlas_index);
    let transform = Transform {
        translation: Vec3::new(tile.position[0], tile.position[1], tile.position[2]),
        rotation: Quat::from_xyzw(
            tile.rotation[0],
            tile.rotation[1],
            tile.rotation[2],
            tile.rotation[3],
        ),
        scale: Vec3::new(tile.scale[0], tile.scale[1], tile.scale[2]),
    };

    let mut entity = world.spawn((sprite, transform, TileMarker::new(tile.tile_type)));
    if let Some(item_kind) = tile.pickable {
        entity.insert(Pickable::new(item_kind));
    }
}

let characters_list_handle = {
    let Some(res) = world.get_resource::<CharactersListResource>() else {
        error!("CharactersListResource not available");
        return;
    };
    res.handle.clone()
};

let characters_list = {
    let lists = world.resource::<Assets<CharactersList>>();
    let Some(list) = lists.get(&characters_list_handle) else {
        error!("Characters list not loaded");
        return;
    };
    list.clone()
};

// Spawn player
let player_data = &save_data.player;
let char_idx = player_data.character_index.min(characters_list.characters.len() - 1);
let character_entry = characters_list.characters[char_idx].clone();

let max_row = character_entry.calculate_max_animation_row();
let layout = {
    let mut layouts = world.resource_mut::<Assets<TextureAtlasLayout>>();
    layouts.add(TextureAtlasLayout::from_grid(
        UVec2::splat(character_entry.tile_size),
        character_entry.atlas_columns as u32,
        (max_row + 1) as u32,
        None,
        None,
    ))
};

let texture = {
    let asset_server = world.resource::<AssetServer>();
    asset_server.load(&character_entry.texture_path)
}

```

```

};
let sprite = Sprite::from_atlas_image(texture, TextureAtlas { layout, index: 0 });

world.spawn((
    Player,
    Transform::from_translation(Vec3::new(
        player_data.position[0],
        player_data.position[1],
        player_data.position[2],
    ))
    .with_scale(Vec3::splat(PLAYER_SCALE)),
    sprite,
    AnimationController::default(),
    CharacterState::default(),
    Velocity::default(),
    player_data.facing,
    Collider::default(),
    PlayerCombat::new(player_data.power_type),
    Health {
        current: player_data.health_current,
        max: player_data.health_max,
    },
    AnimationTimer(Timer::from_seconds(
        DEFAULT_ANIMATION_FRAME_TIME,
        TimerMode::Repeating,
    )),
    character_entry,
));

// Spawn enemies
for enemy_data in &save_data.enemies {
    let enemy_entry = characters_list
        .characters
        .iter()
        .find(|c| c.name == enemy_data.character_name);

    let Some(enemy_entry) = enemy_entry else {
        warn!("Unknown enemy character: {}", enemy_data.character_name);
        continue;
    };
    let enemy_entry = enemy_entry.clone();

    let max_row = enemy_entry.calculate_max_animation_row();
    let layout = {
        let mut layouts = world.resource_mut::<Assets<TextureAtlasLayout>>();
        layouts.add(TextureAtlasLayout::from_grid(
            UVec2::splat(enemy_entry.tile_size),
            enemy_entry.atlas_columns as u32,
            (max_row + 1) as u32,
            None,
            None,
        ))
    };
    let texture = {
        let asset_server = world.resource::<AssetServer>();
        asset_server.load(&enemy_entry.texture_path)
    };
    let sprite = Sprite::from_atlas_image(texture, TextureAtlas { layout, index: 0 });

    use crate::config::enemy::ENEMY_SCALE;
    use crate::enemy::components::*;

    world.spawn((
        Enemy,
        sprite,

```

```

        Transform::from_translation(Vec3::new(
            enemy_data.position[0],
            enemy_data.position[1],
            enemy_data.position[2],
        ))
        .with_scale(Vec3::splat(ENEMY_SCALE)),
        GlobalTransform::default(),
        AnimationController::default(),
        CharacterState::default(),
        Velocity::default(),
        enemy_data.facing,
        Collider::default(),
        EnemyCombat::default(),
        Health {
            current: enemy_data.health_current,
            max: enemy_data.health_max,
        },
        AIBehavior::default(),
        EnemyPath::default(),
        AnimationTimer(Timer::from_seconds(
            DEFAULT_ANIMATION_FRAME_TIME,
            TimerMode::Repeating,
        )),
        enemy_entry,
    ));
}

world
    .resource_mut::<Inventory>()
    .set_items(save_data.inventory);

world.resource_mut::<PlayerSpawned>().0 = true;
world.resource_mut::<EnemiesSpawned>().0 = true;
world.resource_mut::<CurrentCharacterIndex>().index = save_data.player.character_index;
world.resource_mut::<CollisionMapBuilt>().0 = false;
world.insert_resource(crate::map::generate::MapReady);

world
    .resource_mut::<NextState<GameState>>()
    .set(GameState::Playing);

info!("Game loaded from slot {}", slot + 1);
}

```

This function takes `&mut World` instead of individual queries. This is called an “exclusive system” and gives us complete control. We can query, spawn, and despawn without worrying about Rust’s borrowing rules. However, because it demands exclusive access, **no other systems can run in parallel** while it executes. We use it here because loading is a “stop the world” event.

The function first loads the save data from disk. Then it despawns everything: tiles, player, enemies, projectiles, particles, health bars, and any open menu UI to clear the game world. We also despawn the `PauseMenu` and `SaveLoadUI` entities directly and reset `SaveLoadUIState`. This is important because `execute_load` transitions the state to `Playing`, and the `handle_save_load_ui` system only runs in the `Paused` or `MainMenu` states. Without this cleanup, the save/load overlay could get stranded on screen after loading since no system would be running to despawn it.

Next, it spawns tiles using the saved data. For each tile, it reconstructs the full `Transform` including position, rotation, and scale. The rotation is converted from the `[f32; 4]` array back into a `Quat` using `Quat::from_xyzw`.

For the player and enemies, it needs to rebuild the sprite components. It loads the character data, creates texture atlas layouts, and spawns entities with all the required components.

Finally, it updates resources like the inventory and marks flags that let other systems know the world is ready. Setting `CollisionMapBuilt` to false triggers the collision system to rebuild the

map from the new tiles. Inserting `MapReady` tells other systems that the map exists.

The state transition to `GameState::Playing` starts the game with the loaded state.

Inside `execute_load`, a call to `load_save_data(slot)` does the heavy lifting of reading the file from disk and handing back usable Rust data. Let's implement that now.

Implementing Load Logic

The load system needs to read files, verify them, and convert bytes back into Rust structures.

Create `src/save/systems.rs` and add these functions to it:

```
// src/save/systems.rs

use std::fs;

use super::data::*;

pub fn load_save_data(slot: usize) -> Result<SaveData, String> {
    let path = save_file_path(slot);
    let file_bytes = fs::read(&path).map_err(|e| format!("Read error: {}", e))?;

    let save_file: SaveFile =
        bincode::deserialize(&file_bytes).map_err(|e| format!("Deserialize error: {}", e))?;

    // Verify checksum
    let computed = compute_checksum(&save_file.data);
    if computed != save_file.checksum {
        return Err("Save file corrupted or tampered with".into());
    }

    let save_data: SaveData = bincode::deserialize(&save_file.data)
        .map_err(|e| format!("Data deserialize error: {}", e))?;

    if save_data.version != SAVE_VERSION {
        return Err(format!(
            "Incompatible save version: {} (expected {})",
            save_data.version, SAVE_VERSION
        ));
    }

    Ok(save_data)
}

pub fn load_slot_metadata(slot: usize) -> Option<SaveMetadata> {
    let path = meta_file_path(slot);
    let bytes = fs::read(&path).ok()?;
    bincode::deserialize(&bytes).ok()
}
```

First, it reads the raw file bytes using `fs::read`. The `?` operator propagates any IO errors up the call stack.

Then it deserializes those bytes into a `SaveFile` struct using bincode. This gives us access to the checksum and embedded data.

Next, it computes the checksum of the embedded data and compares it to the stored checksum. If they don't match, the file is corrupt or was edited. We return an error immediately.

If the checksum is valid, it deserializes the actual `SaveData` from the embedded bytes. This is a second deserialization step, which is why we have the two-layer structure.

Finally, it checks the version number. If the save was created by a different version of the game, we reject it to avoid crashes from incompatible data structures. As your game evolves, you might add new fields to `PlayerSave` or change the meaning of existing fields. Version checking prevents loading saves that don't match the current code.

What is `load_slot_metadata` doing?

It loads the metadata file. We use this to display save slot info in the UI without loading the entire save. The metadata file is tiny (a few hundred bytes). The save file is large (potentially megabytes with thousands of tiles). By separating them, we can quickly scan all 5 slots to show what's saved without reading massive files.

Integrating the Save System

Create `src/save/mod.rs` to tie everything together:

```
// src/save/mod.rs

pub mod data;
pub mod systems;
pub mod ui;

use bevy::prelude::*;
use crate::state::GameState;

pub use ui::{SaveLoadUIState, SaveLoadMode};

pub struct SavePlugin;

impl Plugin for SavePlugin {
    fn build(&self, app: &mut App) {
        app.init_resource::()
            .init_resource::()
            .add_systems(
                Update,
                ui::handle_save_load_ui
                    .run_if(in_state(GameState::Paused).or(in_state(GameState::MainMenu))),
            )
            .add_systems(
                Update,
                ui::handle_slot_buttons
                    .run_if(|ui_state: Res<SaveLoadUIState>| ui_state.active)
                    .run_if(in_state(GameState::Paused).or(in_state(GameState::MainMenu))),
            )
            .add_systems(
                Update,
                ui::handle_back_button
                    .run_if(|ui_state: Res<SaveLoadUIState>| ui_state.active)
                    .run_if(in_state(GameState::Paused).or(in_state(GameState::MainMenu))),
            )
            .add_systems(
                Update,
                ui::execute_save
                    .run_if(|p: Res<ui::PendingSaveLoadAction>| matches!(p.0,
Some((SaveLoadMode::Save, _)))),
            )
            .add_systems(
                Update,
                ui::execute_load
                    .run_if(|p: Res<ui::PendingSaveLoadAction>| matches!(p.0,
Some((SaveLoadMode::Load, _)))),
            );
    }
}
```

```
}  
}
```

The UI spawn system runs whenever `SaveLoadUIState` changes in the Paused or MainMenu states. The button handlers only run when the UI is active. The execute systems run conditionally when there's a pending action.

The `run_if` conditions ensure systems only execute when needed. The `handle_save_load_ui` system could technically run in any state, but we restrict it to Paused and MainMenu because those are the only places where the save/load UI makes sense.

The `matches!(p.0, Some((SaveLoadMode::Save, _)))` condition checks if there's a pending save action.

This separation keeps systems focused. Each system does one job and doesn't interfere with others.

Adding Game States

Before we can add save/load buttons to menus, we need a proper state system. Games have different states: main menu, loading, playing, paused, game over. Each state shows different UI and runs different systems.

Open `src/state/game_state.rs` and update it:

```
// Update src/state/game_state.rs  
  
use bevy::prelude::*;  
  
#[derive(States, Default, Debug, Clone, Copy, PartialEq, Eq, Hash)]  
pub enum GameState {  
    #[default]  
    MainMenu, // Add this line, ensure default is placed on to of it.  
    Loading,  
    Playing,  
    Paused,  
    GameOver,  
}
```

What changed?

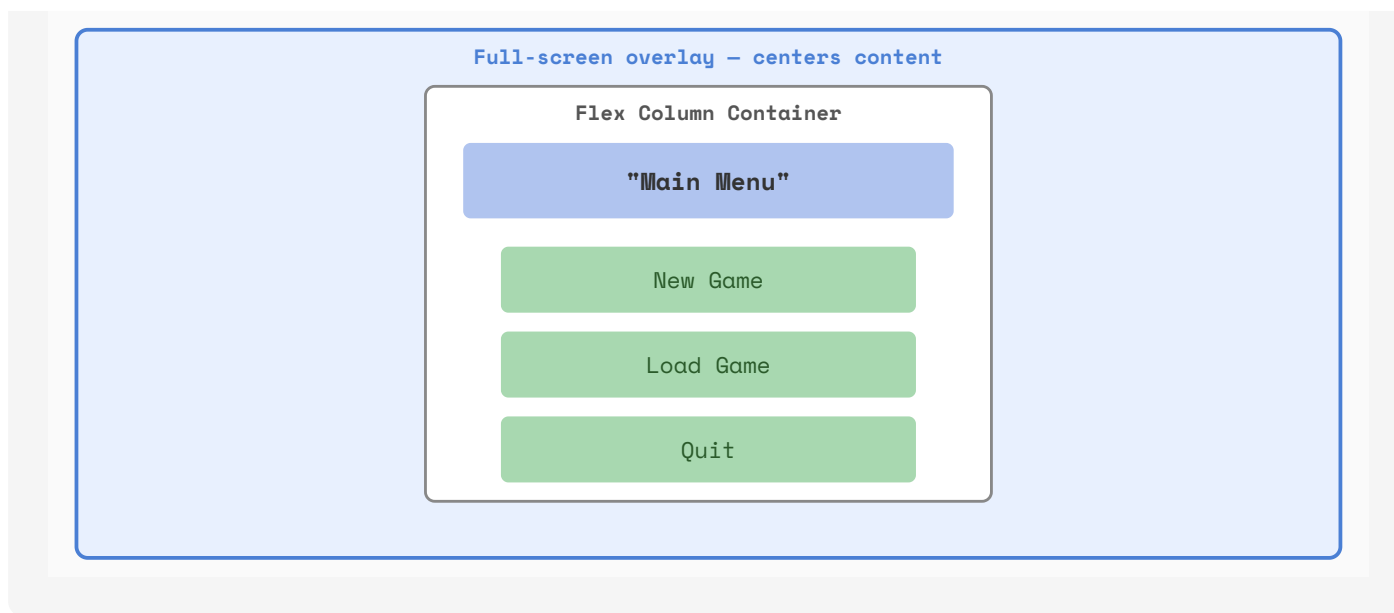
The game now starts at `MainMenu` instead of loading directly into gameplay. This gives players a chance to load an existing save before starting a new game.

As we learned in the earlier chapters, the `#[default]` attribute marks which state the game starts in. The `States` derive makes this work with Bevy's state management system.

Creating the Main Menu

We'll define a UI structure with a root node that fills the screen, holding a title text and three buttons. We also need to handle the button interactions: "New Game" triggers the loading state, "Load Game" opens the save/load menu we just built, and "Quit" exits the app.

Main Menu Structure



Players need a place to start new games or load saved games. Create `src/state/main_menu.rs`:

```
// src/state/main_menu.rs

use bevy::prelude::*;

use crate::save::SaveLoadUIState;
use crate::save::ui::SaveLoadMode;

use super::GameState;

#[derive(Component)]
pub struct MainMenuScreen;

#[derive(Component)]
pub enum MainMenuButton {
    NewGame,
    LoadGame,
    Quit,
}

pub fn spawn_main_menu(mut commands: Commands) {
    commands
        .spawn((
            MainMenuScreen,
            Node {
                width: Val::Percent(100.0),
                height: Val::Percent(100.0),
                justify_content: JustifyContent::Center,
                align_items: AlignItems::Center,
                flex_direction: FlexDirection::Column,
                ..default()
            },
            BackgroundColor(Color::srgb(0.05, 0.05, 0.1)),
        ))
        .with_children(|parent| {
            parent.spawn((
                Text::new("Main Menu"),
                TextFont {
                    font_size: FontSize::Px(64.0),
                    ..default()
                },
                TextColor(Color::srgb(0.8, 0.7, 1.0)),
                Node {
                    margin: UiRect::bottom(Val::Px(60.0)),
                    ..default()
                },
            ));

            let buttons = [
```

```

        (MainMenuButton::NewGame, "New Game"),
        (MainMenuButton::LoadGame, "Load Game"),
        (MainMenuButton::Quit, "Quit"),
    ];

    for (btn_type, label) in buttons {
        parent
            .spawn((
                btn_type,
                Button,
                Node {
                    width: Val::Px(300.0),
                    height: Val::Px(55.0),
                    justify_content: JustifyContent::Center,
                    align_items: AlignItems::Center,
                    margin: UiRect::vertical(Val::Px(8.0)),
                    ..default()
                },
                BackgroundColor(Color::srgba(0.15, 0.15, 0.3, 0.9)),
            ))
            .with_children(|btn_parent| {
                btn_parent.spawn((
                    Text::new(label),
                    TextFont {
                        font_size: FontSize::Px(28.0),
                        ..default()
                    },
                    TextColor(Color::WHITE),
                ));
            });
    }
});

pub fn despawn_main_menu(mut commands: Commands, query: Query<Entity, With<MainMenuScreen>>()) {
    for entity in query.iter() {
        commands.entity(entity).despawn();
    }
}

pub fn handle_main_menu_buttons(
    mut next_state: ResMut<NextState<GameState>>,
    mut ui_state: ResMut<SaveLoadUIState>,
    interaction_query: Query<(&Interaction, &MainMenuButton), Changed<Interaction>>,
    mut exit: MessageWriter<AppExit>,
) {
    for (interaction, button) in interaction_query.iter() {
        if *interaction != Interaction::Pressed {
            continue;
        }

        match button {
            MainMenuButton::NewGame => {
                next_state.set(GameState::Loading);
            }
            MainMenuButton::LoadGame => {
                ui_state.active = true;
                ui_state.mode = SaveLoadMode::Load;
            }
            MainMenuButton::Quit => {
                exit.write(AppExit::Success);
            }
        }
    }
}

```

```
pub fn handle_main_menu_hover(
    mut interaction_query: Query<
        (&Interaction, &mut BackgroundColor),
        (Changed<Interaction>, With<MainMenuButton>),
    >,
) {
    for (interaction, mut bg) in interaction_query.iter_mut() {
        *bg = match interaction {
            Interaction::Hovered => BackgroundColor(Color::srgba(0.25, 0.25, 0.5, 0.9)),
            Interaction::Pressed => BackgroundColor(Color::srgba(0.35, 0.35, 0.6, 0.9)),
            Interaction::None => BackgroundColor(Color::srgba(0.15, 0.15, 0.3, 0.9)),
        };
    }
}
```

We have two systems managing the menu: one for logic and one for visuals.

`handle_main_menu_buttons` listens for clicks. It queries for buttons that have `Changed<Interaction>`, which means it only runs on the specific frame a user clicks or releases a button. When it detects a `Pressed` interaction, it checks which `MainMenuButton` type it is and acts accordingly:

- `NewGame`: Transitions the game state to `Loading`, kicking off the world generation.
- `LoadGame`: Doesn't change the game state yet. Instead, it activates our `SaveLoadUIState`, which overlays the save slot menu on top of the main menu.
- `Quit`: Sends an `AppExit` message, which Bevy listens for to cleanly close the application.

`handle_main_menu_hover` handles the visual polish. It also looks for `Changed<Interaction>` but updates the `BackgroundColor` component. This gives immediate feedback—buttons light up when hovered and darken when pressed—making the UI feel responsive and alive. Bevy's `Interaction` component (None, Hovered, Pressed) makes this kind of state-based styling very easy to implement.

Updating the Pause Menu

Let's upgrade the pause menu with Save and Load buttons. Clicking these will trigger our `SaveLoadUIState`, overlaying the save slots while keeping the game paused in the background.

Let's rewrite `src/state/pause.rs`:

```
// Rewrite src/state/pause.rs

use bevy::prelude::*;

use crate::save::SaveLoadUIState;
use crate::save::ui::SaveLoadMode;

use super::GameState;

#[derive(Component)]
pub struct PauseMenu;

#[derive(Component)]
pub enum PauseButton {
    Resume,
    SaveGame,
    LoadGame,
    MainMenu,
    Quit,
}

pub fn spawn_pause_menu(mut commands: Commands) {
    commands
```

```

        .spawn((
            PauseMenu,
            Node {
                width: Val::Percent(100.0),
                height: Val::Percent(100.0),
                justify_content: JustifyContent::Center,
                align_items: AlignItems::Center,
                flex_direction: FlexDirection::Column,
                ..default()
            },
            BackgroundColor(Color::srgba(0.0, 0.0, 0.0, 0.7)),
        ))
        .with_children(|parent| {
            parent.spawn((
                Text::new("PAUSED"),
                TextFont {
                    font_size: FontSize::Px(42.0),
                    ..default()
                },
                TextColor(Color::WHITE),
                Node {
                    margin: UiRect::bottom(Val::Px(30.0)),
                    ..default()
                },
            ));

            let buttons = [
                (PauseButton::Resume, "Resume"),
                (PauseButton::SaveGame, "Save Game"),
                (PauseButton::LoadGame, "Load Game"),
                (PauseButton::MainMenu, "Main Menu"),
                (PauseButton::Quit, "Quit"),
            ];

            for (btn_type, label) in buttons {
                parent
                    .spawn((
                        btn_type,
                        Button,
                        Node {
                            width: Val::Px(250.0),
                            height: Val::Px(50.0),
                            justify_content: JustifyContent::Center,
                            align_items: AlignItems::Center,
                            margin: UiRect::vertical(Val::Px(5.0)),
                            ..default()
                        },
                        BackgroundColor(Color::srgba(0.15, 0.15, 0.3, 0.9)),
                    ))
                    .with_children(|btn_parent| {
                        btn_parent.spawn((
                            Text::new(label),
                            TextFont {
                                font_size: FontSize::Px(24.0),
                                ..default()
                            },
                            TextColor(Color::WHITE),
                        ));
                    });
            }
        });

        info!("Pause menu spawned");
    }
}

```



```

pub fn despawn_pause_menu(mut commands: Commands, query: Query<Entity, With<PauseMenu>>) {
    for entity in query.iter() {
        commands.entity(entity).despawn();
    }

    info!("Pause menu despawned");
}

pub fn handle_pause_buttons(
    mut next_state: ResMut<NextState<GameState>>,
    mut ui_state: ResMut<SaveLoadUIState>,
    interaction_query: Query<(&Interaction, &PauseButton), Changed<Interaction>>,
    mut exit: MessageWriter<AppExit>,
) {
    if ui_state.active {
        return;
    }

    for (interaction, button) in interaction_query.iter() {
        if *interaction != Interaction::Pressed {
            continue;
        }

        match button {
            PauseButton::Resume => {
                next_state.set(GameState::Playing);
            }
            PauseButton::SaveGame => {
                ui_state.active = true;
                ui_state.mode = SaveLoadMode::Save;
            }
            PauseButton::LoadGame => {
                ui_state.active = true;
                ui_state.mode = SaveLoadMode::Load;
            }
            PauseButton::MainMenu => {
                next_state.set(GameState::MainMenu);
            }
            PauseButton::Quit => {
                exit.write(AppExit::Success);
            }
        }
    }
}

pub fn handle_pause_hover(
    mut interaction_query: Query<
        (&Interaction, &mut BackgroundColor),
        (Changed<Interaction>, With<PauseButton>),
    >,
) {
    for (interaction, mut bg) in interaction_query.iter_mut() {
        *bg = match interaction {
            Interaction::Hovered => BackgroundColor(Color::srgba(0.25, 0.25, 0.5, 0.9)),
            Interaction::Pressed => BackgroundColor(Color::srgba(0.35, 0.35, 0.6, 0.9)),
            Interaction::None => BackgroundColor(Color::srgba(0.15, 0.15, 0.3, 0.9)),
        };
    }
}

```

This setup is similar to the main menu but with a few key differences:

Overlay Effect: The background is a semi-transparent black (`srgba(0.0, 0.0, 0.0, 0.7)`). This darkens the game world without hiding it, reinforcing that the game is still there, just paused.

Save and Load Buttons: We've added `Save Game` and `Load Game` buttons. Usually, clicking a major menu button changes the entire game state (like switching from `Playing` to `Paused`). But here, we don't want to leave the pause screen, we just want to pop up the save slots on top of it. That's why clicking these buttons doesn't change `GameState`. Instead, it updates a separate resource called `SaveLoadUIState`, turning its `active` field to `true`. Our UI system watches this resource, and when it sees `active` become `true`, it immediately draws the save slots overlay over whatever menu you're currently in.

Input Guarding: The pause menu buttons are still on screen even when the save/load UI sits on top of them. Notice the guard at the start of `handle_pause_buttons`. This prevents a user from accidentally interacting with the underlying pause menu when clicking on the save/load UI.

State Transitions: `Resume` switches back to `GameState::Playing`, `MainMenu` cleanly transitions to `GameState::MainMenu`, and `Quit` instructs the app to exit. Visual hover states are handled independently in `handle_pause_hover` for immediate UI feedback.

Updating Game Over

Before save/load, this function only needed to handle the Game Over → Restart flow. It cleaned up enemies, projectiles, particles, health bars.

With save/load, this function now runs in two scenarios:

1. **Game Over → Loading** – the original path.
2. **Pause → Main Menu** – a new flow where the player returns to the main menu mid-game.

The second case forces us to do a **complete world teardown**, not just a partial cleanup. Here's why each step matters:

- **Tiles** must be despawned – otherwise loading a save would stack new tiles on top of the existing ones.
- `MapReady` must be removed – this resource tells Loading to skip WFC. If it lingers, pressing “New Game” would jump straight to Playing without generating a fresh map.
- `CollisionMapBuilt` must be reset – so the collision system rebuilds from the new tiles.
- `CollisionMap` must be removed – because `build_collision_map` inserts it via deferred commands while setting the built flag immediately; without removing the old resource, spawn systems would validate positions against the stale map from the previous game.
- **Inventory** must be cleared – otherwise items from a previous session carry into a new game.

Update `src/state/game_over.rs`:

```
// Update src/state/game_over.rs
use bevy::prelude::*;

use crate::characters::input::Player; // Add this import
use crate::characters::spawn::PlayerSpawned;
use crate::collision::{CollisionMap, TileMarker, CollisionMapBuilt}; // Add this import
use crate::combat::healthbar::HealthBarOwner;
use crate::combat::systems::{Projectile, ProjectileEffect};
use crate::enemy::{spawn::EnemiesSpawned, Enemy};
use crate::inventory::Inventory; // Add this import
use crate::map::generate::MapReady; // Add this import
use crate::particles::components::{Particle, ParticleEmitter};

use super::GameState;

#[derive(Component)]
pub struct GameOverScreen;

pub fn spawn_game_over_screen(mut commands: Commands) {
    commands
        .spawn((
            GameOverScreen,
```

```

        Node {
            width: Val::Percent(100.0),
            height: Val::Percent(100.0),
            justify_content: JustifyContent::Center,
            align_items: AlignItems::Center,
            ..default()
        },
        BackgroundColor(Color::srgba(0.0, 0.0, 0.0, 0.85)),
    ))
    .with_children(|parent| {
        parent.spawn((
            Text::new("GAME OVER\n\nPress R to restart"),
            TextFont {
                font_size: FontSize::Px(48.0),
                ..default()
            },
            TextColor(Color::WHITE),
            TextLayout::justify(Justify::Center),
        ));
    });

    info!("Game over screen spawned");
}

pub fn despawn_game_over_screen(mut commands: Commands, query: Query<Entity, With<GameOverScreen>>)
{
    for entity in query.iter() {
        commands.entity(entity).despawn();
    }
    info!("Game over screen despawned");
}

pub fn handle_restart_input(
    input: Res<ButtonInput<KeyCode>>,
    mut next_state: ResMut<NextState<GameState>>,
) {
    if input.just_pressed(KeyCode::KeyR) {
        info!("Restarting game...");
        next_state.set(GameState::Loading); // Updated from Playing to Loading
    }
}

pub fn cleanup_game_world(
    mut commands: Commands,
    tiles: Query<Entity, With<TileMarker>>, // Add this query
    players: Query<Entity, With<Player>>, // Add this query
    enemies: Query<Entity, With<Enemy>>,
    projectiles: Query<Entity, With<Projectile>>,
    projectile_effects: Query<Entity, With<ProjectileEffect>>,
    emitters: Query<Entity, With<ParticleEmitter>>,
    particles: Query<Entity, With<Particle>>,
    healthbars: Query<Entity, With<HealthBarOwner>>,
    mut player_spawned: ResMut<PlayerSpawned>,
    mut enemies_spawned: ResMut<EnemiesSpawned>,
    mut collision_map_built: ResMut<CollisionMapBuilt>, // Add this mutable resource
    mut inventory: ResMut<Inventory>, // Add this mutable resource
) {

    // Add the following loop to despawn all tiles
    for entity in tiles.iter() {
        commands.entity(entity).despawn();
    }
    // Add the following loop to despawn all players
    for entity in players.iter() {

```

```

        commands.entity(entity).despawn();
    }
    for entity in enemies.iter() {
        commands.entity(entity).despawn();
    }
    for entity in projectiles.iter() {
        commands.entity(entity).despawn();
    }
    for entity in projectile_effects.iter() {
        commands.entity(entity).despawn();
    }
    for entity in emitters.iter() {
        commands.entity(entity).despawn();
    }
    for entity in particles.iter() {
        commands.entity(entity).despawn();
    }
    for entity in healthbars.iter() {
        commands.entity(entity).despawn();
    }
}

player_spawned.0 = false;
enemies_spawned.0 = false;

// Add the following lines to reset the collision map and inventory
collision_map_built.0 = false;
commands.remove_resource::<CollisionMap>();
inventory.set_items(Default::default());
commands.remove_resource::<MapReady>();

info!("Game world cleaned up: {} entities despawned, spawn flags reset", count);
}

```

Each query targets a different entity category and despawns it. The flags and resources at the end restore the game to a clean slate, as if the player just launched the app. Whether the next move is “New Game” or “Load Game”, the world is ready for it.

Wiring Up the State System

Update `src/state/mod.rs` to connect all the pieces:

```

// Update src/state/mod.rs

mod game_over;
mod game_state;
mod loading;
pub mod main_menu; // Add this import
pub(crate) mod pause; // pub(crate) so the save system can import PauseMenu

use crate::characters::config::CharactersList;
use crate::characters::spawn::CharactersListResource;
use crate::map::generate::MapReady;
use crate::save::SaveLoadUIState; // Add this import
use bevy::prelude::*;

pub use game_state::GameState;

pub struct StatePlugin;

impl Plugin for StatePlugin {
    fn build(&self, app: &mut App) {
        app.init_state::<GameState>()
            // Add these three systems
    }
}

```

```

        .add_systems(
            OnEnter(GameState::MainMenu),
            (game_over::cleanup_game_world, main_menu::spawn_main_menu).chain(),
        )
        .add_systems(OnExit(GameState::MainMenu), main_menu::despawn_main_menu)
        .add_systems(
            Update,
            main_menu::handle_main_menu_buttons
                .run_if(in_state(GameState::MainMenu)),
        )
        .add_systems(
            Update,
            main_menu::handle_main_menu_hover
                .run_if(in_state(GameState::MainMenu)),
        )
        .add_systems(OnEnter(GameState::Loading), loading::spawn_loading_screen)
        .add_systems(
            Update,
            (check_assets_loaded, loading::animate_loading)
                .run_if(in_state(GameState::Loading)),
        )
        .add_systems(
            OnExit(GameState::Loading),
            loading::despawn_loading_screen,
        )
        .add_systems(OnEnter(GameState::Paused), pause::spawn_pause_menu)
        // Add close_save_load_ui to the following system
        .add_systems(OnExit(GameState::Paused), (pause::despawn_pause_menu, close_save_load_ui))
        // Add these two systems
        .add_systems(
            Update,
            pause::handle_pause_buttons
                .run_if(in_state(GameState::Paused)),
        )
        .add_systems(
            Update,
            pause::handle_pause_hover
                .run_if(in_state(GameState::Paused)),
        )
        .add_systems(
            Update,
            toggle_pause.run_if(in_state(GameState::Playing).or(in_state(GameState::Paused))),
        )
        .add_systems(OnEnter(GameState::GameOver), game_over::spawn_game_over_screen)
        .add_systems(
            OnExit(GameState::GameOver),
            (game_over::despawn_game_over_screen, game_over::cleanup_game_world),
        )
        .add_systems(
            Update,
            game_over::handle_restart_input.run_if(in_state(GameState::GameOver)),
        );
    }
}

fn check_assets_loaded(
    characters_list_res: Option<Res<CharactersListResource>>,
    characters_lists: Res<Assets<CharactersList>>,
    map_ready: Option<Res<MapReady>>,
    mut next_state: ResMut<NextState<GameState>>,
) {
    let Some(res) = characters_list_res else {
        return;
    };
};

```

```

        if characters_lists.get(&res.handle).is_some() && map_ready.is_some() {
            info!("Assets loaded, transitioning to Playing!");
            next_state.set(GameState::Playing);
        }
    }

fn toggle_pause(
    input: Res<ButtonInput<KeyCode>>,
    current_state: Res<State<GameState>>,
    mut next_state: ResMut<NextState<GameState>>,
    ui_state: Res<SaveLoadUIState>, // Add this
) {
    if input.just_pressed(KeyCode::Escape) {

        // Add this if block
        if ui_state.active {
            return;
        }

        match current_state.get() {
            GameState::Playing => {
                info!("Game paused");
                next_state.set(GameState::Paused);
            }
            GameState::Paused => {
                info!("Game resumed");
                next_state.set(GameState::Playing);
            }
            _ => {}
        }
    }
}

// Add this function
fn close_save_load_ui(mut ui_state: ResMut<SaveLoadUIState>) {
    ui_state.active = false;
}

```

When entering the MainMenu state, we first cleanup the game world (despawning entities), then spawn the main menu UI. The `.chain()` ensures these run in order.

When exiting the Paused state, we close the save/load UI. This ensures the UI doesn't persist if you resume the game.

The `toggle_pause` function checks `ui_state.active` before handling the Escape key. If the save/load UI is open, we ignore Escape in this system because the back button handler will close the UI instead.

The `check_assets_loaded` system waits for both character assets and map generation to complete before transitioning to Playing. This prevents the game from starting with an incomplete world.

There's one problem with our current implementation. The load system needs `TilemapHandles` to spawn tiles, but those handles aren't created until the `Loading` state. If you try to load a save from the main menu, the handles don't exist yet.

The solution is to prepare the tilemap handles during `Startup`, before the main menu appears. This makes them available throughout the entire game lifecycle.

Open `src/map/assets.rs` and make `TilemapHandles` a resource:

```

// Update src/map/assets.rs

#[derive(Clone, Resource)] // Add Resource to the derive macro
pub struct TilemapHandles {
    pub image: Handle<Image>,
}

```



```
pub layout: Handle<TextureAtlasLayout>,
}
```

The `#[derive(Clone, Resource)]` on `TilemapHandles` allows us to store it globally. This is critical because multiple systems—map generation, loading, and saving—need access to these assets independently.

We also updated `create_spawner` to handle the default case. Previously, decorative tiles (like tree tops) fell into the empty default branch and spawned without a `TileMarker`. Since our save system queries for `With<TileMarker>`, these tiles were invisible to it and vanished on load. By explicitly adding `TileMarker::new(TileType::Empty)`, we ensure every visual element in the world is tracked and saved.

```
fn create_spawner(
    tile_type: Option<TileType>,
    pickable: Option<ItemKind>,
) -> fn(&mut EntityCommands) {
    match (tile_type, pickable) {
        // ... all the specific tile type cases ...

        // Default: treat as empty (walkable) tile
        _ => |e: &mut EntityCommands| { // Replace _ with e
            e.insert(TileMarker::new(TileType::Empty)); // Update this line
        },
    }
}
```

Now, back to `TilemapHandles`. Just making it a resource isn't enough; we also need to ensure it exists *before* the main menu appears so we can load saves immediately. Currently, it's created during the `Loading` state, which is too late.

To fix this, we'll move the handle creation to a Startup system.

Update `src/map/generate.rs`

```
// Update src/map/generate.rs
use crate::map::{
    // Add TilemapHandles to the import
    assets::{load_assets, prepare_tilemap_handles, TilemapHandles},
    rules::build_world,
};

// Add this function
pub fn prepare_tilemap_handles_resource(
    mut commands: Commands,
    asset_server: Res<AssetServer>,
    mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,
) {
    let tilemap_handles =
        prepare_tilemap_handles(&asset_server, &mut atlas_layouts, ASSETS_PATH, TILEMAP_FILE);
    commands.insert_resource(tilemap_handles);
}
```

This runs once when the game starts, before the main menu appears. It creates the tilemap handles and inserts them as a resource. Now the handles exist throughout the entire game lifecycle, available for both map generation and save loading.

The system uses the existing `prepare_tilemap_handles` function from `assets.rs`, but wraps it to work as a Bevy system.

Now update the `setup_generator` system to use the existing resource instead of creating it:

```
// Update setup_generator in src/map/generate.rs
pub fn setup_generator(
    mut commands: Commands,
    tilemap_handles: Res<TilemapHandles>, // Add this
    // Delete asset_server: Res<AssetServer>,
    // Delete mut atlas_layouts: ResMut<Assets<TextureAtlasLayout>>,

) {
    let (assets_definitions, models, socket_collection) = build_world();

    let rules = RulesBuilder::new_cartesian_3d(models, socket_collection)
        .with_rotation_axis(Direction::ZForward)
        .build()
        .unwrap();
    let rules_arc = Arc::new(rules);

    let grid_template =
        CartesianGrid::new_cartesian_3d(GRID_X, GRID_Y, GRID_Z, false, false, false);

    // Delete this line
    // let tilemap_handles = prepare_tilemap_handles(&asset_server, &mut atlas_layouts, ASSETS_PATH,
    TILEMAP_FILE);

    let models_assets = load_assets(&tilemap_handles, assets_definitions);
    let spawner = NodesSpawner::new(models_assets, NODE_SIZE, ASSETS_SCALE);

    commands.insert_resource(MapSpawnResources {
        spawner,
        grid_template: grid_template.clone(),
    });

    let progress = Arc::new(AtomicU32::new(0));
    commands.insert_resource(MapGenProgress {
        current: progress.clone(),
        total: CHUNKS_X * CHUNKS_Y,
    });

    let pool = AsyncComputeTaskPool::get();
    let task = pool.spawn(async move {
        generate_all_chunks(rules_arc, grid_template, progress)
    });
    commands.insert_resource(MapGenTask(task));
}
```

The system now takes `tilemap_handles: Res<TilemapHandles>` as a parameter instead of creating them. It uses the existing handles to load assets and create the spawner.

This system still runs on `OnEnter(GameState::Loading)`, but now it assumes the handles already exist. Since we prepare them during Startup, this is guaranteed.

Final Integration

Open `src/main.rs` and wire everything together:

```
mod map;
mod characters;
mod state;
mod collision;
mod config;
mod inventory;
mod camera;
mod combat;
```

```

mod particles;
mod enemy;
mod save; // Add this

use bevy::{
    prelude::*,
    window::{MonitorSelection, Window, WindowMode, WindowPlugin},
};

use crate::camera::CameraPlugin;
// Add prepare_tilemap_handles_resource to the import
use crate::map::generate::{prepare_tilemap_handles_resource, setup_generator, poll_map_generation};
use crate::state::GameState;

fn main() {
    App::new()
        .insert_resource(ClearColor(Color::BLACK))
        .add_plugins(
            DefaultPlugins
                .set(AssetPlugin {
                    file_path: "src/assets".into(),
                    ..default()
                })
                .set(WindowPlugin {
                    primary_window: Some(Window {
                        title: "Bevy Game".into(),
                        mode: WindowMode::BorderlessFullscreen(MonitorSelection::Current),
                        ..default()
                    }),
                    ..default()
                })
                .set(ImagePlugin::default_nearest()),
        )
        .add_plugins(state::StatePlugin)
        .add_plugins(CameraPlugin)
        .add_plugins(inventory::InventoryPlugin)
        .add_plugins(collision::CollisionPlugin)
        .add_plugins(characters::CharactersPlugin)
        .add_plugins(combat::CombatPlugin)
        .add_plugins(enemy::EnemyPlugin)
        .add_plugins(particles::ParticlesPlugin)
        // Delete .add_systems(Startup, setup_generator)
        .add_plugins(save::SavePlugin) // Add this system
        .add_systems(Startup, prepare_tilemap_handles_resource) // Add this system
        .add_systems(OnEnter(GameState::Loading), setup_generator) // Add this system
        .add_systems(Update, poll_map_generation.run_if(in_state(GameState::Loading)))
        .run();
}

```

The `prepare_tilemap_handles_resource` system runs during `Startup`, before any states activate. This ensures the handles exist when the main menu loads.

The `setup_generator` system runs on `OnEnter(GameState::Loading)`, whether you start a new game or restart after game over.

The `poll_map_generation` system runs every frame during the `Loading` state, checking if the background generation task has completed.

The `SavePlugin` is added alongside the other plugins, giving you access to save/load functionality throughout the game.

Run the game:

```
cargo run
```

You should be able to start a new game from the main menu, pause the game with **Escape**, save your progress, and load it back later from either the main menu or the pause menu.

The Impatient Programmer's Guide to Bevy and Rust: Chapter 11 - Let There Be Sound

We have a playable world, enemies, magic powers, etc. But our fun adventure is missing something: it is completely silent.

In this chapter, we're going to bring our game to life with audio. We'll add looping background music that changes based on the game state (menu vs battle), and responsive sound effects for jumping, shooting, and taking damage.

Building the Audio Module

Before diving into the code, you'll need the sound files. You can find the required audio assets in the `chapter11/src/assets/audio` directory of our [GitHub repository](#). Download the entire `audio` folder and place it directly into your project's `src/assets/` directory.

Bevy supports audio playback out of the box through `bevy_audio` (included in `DefaultPlugins`). However, by default, it might only support certain compressed formats like `.ogg` or `.flac`.

Our asset pack contains `.mp3` background tracks and `.wav` sound effects. We need to explicitly tell Bevy to compile support for these formats.

Open your `Cargo.toml` and update the `bevy` dependency:

```
# Update this line
bevy = { version = "0.19", features = ["mp3", "wav"] }
```

Loading Audio Assets

We need to load our audio files into memory when the game starts. Loading from disk is slow. If we try to load the “jump” sound at the exact millisecond the player presses Spacebar, the game might stutter while it waits for the hard drive.

To prevent this, we load all audio files into a single `Resource` on startup.

Create a new directory `src/audio` and add `assets.rs`:

```
// src/audio/assets.rs
use bevy::prelude::*;

#[derive(Resource)]
pub struct AudioAssets {
    pub menu_music: Handle,
    pub battle_music: Handle,
    pub spell_generic: Handle,
    pub spell_fire: Handle,
    pub enemy_shoot: Handle,
    pub hit: Handle,
    pub player_death: Handle,
    pub enemy_death: Handle,
    pub pickup: Handle,
    pub button_click: Handle,
    pub jump: Handle,
}
```

```
pub fn load_audio_assets(mut commands: Commands, asset_server: Res<AssetServer>) {
    commands.insert_resource(AudioAssets {
        menu_music: asset_server.load("audio/music/menu_ambient.mp3"),
        battle_music: asset_server.load("audio/music/battle_theme.ogg"),
        spell_generic: asset_server.load("audio/sfx/spell_01.ogg"),
        spell_fire: asset_server.load("audio/sfx/spell_fire.wav"),
        enemy_shoot: asset_server.load("audio/sfx/spell_enemy.ogg"),
        hit: asset_server.load("audio/sfx/hit.wav"),
        player_death: asset_server.load("audio/sfx/death_player.mp3"),
        enemy_death: asset_server.load("audio/sfx/death_enemy.mp3"),
        pickup: asset_server.load("audio/sfx/pickup.wav"),
        button_click: asset_server.load("audio/sfx/button_click.wav"),
        jump: asset_server.load("audio/sfx/jump.wav"),
    });
}
```

Now, any system that needs to play a sound can just grab `Res<AudioAssets>` and use the pre-loaded `Handle<AudioSource>`.

Managing Background Music

Background music (BGM) has two special requirements:

1. It needs to loop indefinitely.
2. It needs to change when the game state changes (e.g., from Main Menu to Playing).

When you transition from the quiet Main Menu to battle, the music should change smoothly.

To achieve this, we need a way to track the currently playing music so our systems can find it and stop it before starting a new track. We will create a `MusicTrack` marker component to tag our audio entities.

We will also write three functions: one to start the menu music, one to start the battle music, and one to stop all music entirely when the game is over. Create `src/audio/music.rs`:

```
// src/audio/music.rs
use bevy::prelude::*;
use super::assets::AudioAssets;

/// Marker for music entities so we can find and stop them.
#[derive(Component)]
pub struct MusicTrack;

pub fn start_menu_music(
    mut commands: Commands,
    audio_assets: Option<Res<AudioAssets>>,
    existing: Query<Entity, With<MusicTrack>>,
) {
    // 1. Stop any currently playing music
    for entity in existing.iter() {
        commands.entity(entity).despawn();
    }

    // 2. Safely get our loaded audio assets
    let Some(audio) = audio_assets else { return; };

    // 3. Spawn the new audio player
    commands.spawn((
        MusicTrack,
        AudioPlayer::new(audio.menu_music.clone()),
        PlaybackSettings::LOOP,
    ));
}
```



```
pub fn start_battle_music(
    mut commands: Commands,
    audio_assets: Option<Res<AudioAssets>>,
    existing: Query<Entity, With<MusicTrack>>,
) {
    for entity in existing.iter() {
        commands.entity(entity).despawn();
    }
    let Some(audio) = audio_assets else { return; };
    commands.spawn((
        MusicTrack,
        AudioPlayer::new(audio.battle_music.clone()),
        PlaybackSettings::LOOP,
    ));
}

pub fn stop_all_music(
    mut commands: Commands,
    existing: Query<Entity, With<MusicTrack>>,
) {
    for entity in existing.iter() {
        commands.entity(entity).despawn();
    }
}
```

Let's break down how this works.

First, `start_menu_music` and `start_battle_music` query for any existing entity that has the `MusicTrack` marker and despawns them all. Despawning the entity immediately stops the audio from playing, ensuring we have a clean slate.

Next, they grab our pre-loaded music files from the `AudioAssets` resource.

Finally, they spawn a single new entity with three crucial components:

1. Our `MusicTrack` marker, so we can find and stop this track later.
2. The `AudioPlayer` itself, holding the loaded `.ogg` or `.mp3` track.
3. `PlaybackSettings::LOOP`, an instruction to Bevy's audio engine to keep restarting the song forever.

Sound Effects with Observers

Now we need sound effects (SFX) for hitting enemies, shooting fireballs, and picking up plants.

In our previous chapters, we used **Observers** to decouple our combat logic, spawning hit effects and handling entity deaths without scattering that logic everywhere. Observers are the perfect tool for audio, too. By using them, we can fire a sound trigger from anywhere in our codebase, while keeping our core game systems completely decoupled from our audio logic.

To build this decoupled audio system, we need two things:

1. An `enum` (`SfxKind`) that acts as a menu of every sound in our game like jumping, dying, shooting, UI clicks, etc. We will derive `Event` on it so it can act as a trigger.
2. An observer function (`on_sfx`) that constantly listens for these `SfxKind` events. When it hears one, it will grab the corresponding audio file from our `AudioAssets` resource and play it.

Create `src/audio/sfx.rs`:

```
// src/audio/sfx.rs
use bevy::prelude::*;
use crate::combat::PowerType;
use super::assets::AudioAssets;
```

```
#[derive(Event, Clone, Copy)]
pub enum SfxKind {
    PlayerShoot(PowerType),
    EnemyShoot,
    Hit,
    PlayerDeath,
    EnemyDeath,
    Pickup,
    ButtonClick,
    Jump,
}

pub fn on_sfx(
    trigger: On<SfxKind>,
    audio_assets: Option<Res<AudioAssets>>,
    mut commands: Commands,
) {
    let Some(audio) = audio_assets else { return; };

    let handle = match *trigger.event() {
        SfxKind::PlayerShoot(PowerType::Fire) => audio.spell_fire.clone(),
        SfxKind::PlayerShoot(_) => audio.spell_generic.clone(),
        SfxKind::EnemyShoot => audio.enemy_shoot.clone(),
        SfxKind::Hit => audio.hit.clone(),
        SfxKind::PlayerDeath => audio.player_death.clone(),
        SfxKind::EnemyDeath => audio.enemy_death.clone(),
        SfxKind::Pickup => audio.pickup.clone(),
        SfxKind::ButtonClick => audio.button_click.clone(),
        SfxKind::Jump => audio.jump.clone(),
    };

    commands.spawn((AudioPlayer::new(handle), PlaybackSettings::DESPAWN));
}
```

By adding `#[derive(Event)]` to `SfxKind`, we tell Bevy that this enum can be broadcasted globally and caught by an observer. Notice that `PlayerShoot` even holds data (`PowerType`) so we can play different sounds for fireballs versus arcane blasts!

The `on_sfx` system acts as the catcher. It intercepts the trigger, reads the event data, and matches it against our loaded `AudioAssets`. Once it finds the correct audio handle, say `audio.hit`, it passes it into a new `AudioPlayer` and spawns it into the world.

What is `PlaybackSettings::DESPAWN`?

Unlike our music which loops forever, sound effects play once and finish. If we don't clean up the `AudioPlayer` entities, we'll permanently leak memory.

`PlaybackSettings::DESPAWN` instructs Bevy's audio engine to automatically despawn the entity the moment the audio file finishes playing. It is the perfect fire-and-forget solution.

What's a memory leak?

A memory leak happens when a program allocates memory (like spawning an entity for a sound effect) but forgets to free that memory when it is no longer needed. Over time, as the player shoots thousands of fireballs, those "dead" audio entities would pile up invisibly in the game world, consuming more and more RAM until your game eventually slows down or crashes!

Audio Plugin

Finally, let's register all these systems and observers in our main audio module.

Do you remember in our previous combat chapters when we registered the `on_projectile_hit` and `on_entity_death` observers? We do the exact same thing here for our audio system by passing our `on_sfx` system into `.add_observer()`.

Create `src/audio/mod.rs`:

```
// src/audio/mod.rs
mod assets;
mod music;
mod sfx;

pub use sfx::SfxKind;

use crate::state::GameState;
use bevy::prelude::*;

pub struct AudioManagerPlugin;

impl Plugin for AudioManagerPlugin {
    fn build(&self, app: &mut App) {
        app
            // SFX handled via global observer
            .add_observer(sfx::on_sfx)

            // Start loading assets and music on app startup
            .add_systems(
                Startup,
                (assets::load_audio_assets, music::start_menu_music).chain(),
            )

            // Music follows game state.
            .add_systems(OnEnter(GameState::MainMenu), music::start_menu_music)
            .add_systems(OnEnter(GameState::Playing), music::start_battle_music)
            .add_systems(OnEnter(GameState::GameOver), music::stop_all_music);
    }
}
```

Now remember to register `AudioManagerPlugin` in your `src/main.rs` file inside `App::new()` alongside the other plugins.

We have our audio engine built and listening! In the next section, we will wire up the triggers in our game's combat, input, and inventory systems to actually fire these sounds.

Triggering Sound Effects

Because we used an Observer pattern for our audio, triggering a sound effect is incredibly simple. We don't need to add `AudioAssets` to every system or worry about managing entities manually. We just need access to `Commands`.

Let's start integrating these triggers into our existing systems.

The Player Jump

When the player presses the Spacebar and transitions into the jumping state, we want to play the jump sound. Let's update `src/characters/input.rs`. First, import our new enum:

```
// In src/characters/input.rs
use crate::audio::SfxKind;
```

Now find the `handle_player_input` system. Before we can trigger sounds, we need access to `Commands`. Add it as a parameter to the function signature:

```
// In src/characters/input.rs
pub fn handle_player_input(
```

```

    mut commands: Commands, // <-- Add this!
    input: Res<ButtonInput<KeyCode>>,
    // ...
) {

```

Then, further down in the system where we determine the new state:

```

// In src/characters/input.rs
// let new_state = determine_new_state(*state, direction, is_running, wants_jump);
// if *state != new_state {

    // Add the following if block
    // If we transitioned into a jump, fire the SFX!
    if new_state == CharacterState::Jumping {
        commands.trigger(SfxKind::Jump);
    }

//     *state = new_state;
// }

```

Casting Spells

When the player shoots, we can use the spell's data to play specific sounds. For instance, we can play a unique sound for our `PowerType::Fire` spell, and fall back to a generic spell sound for the others. Our `SfxKind::PlayerShoot(PowerType)` takes the power type as data!

Open `src/combat/systems.rs` and import `SfxKind`:

```

// In src/combat/systems.rs
use crate::audio::SfxKind;

```

```

// In src/combat/systems.rs
// Inside `handle_power_input`:

let visuals = combat.power_type.visuals(direction);

spawn_projectile(
    &mut commands,
    spawn_position,
    combat.power_type,
    &visuals,
    ProjectileOwner::Player,
);

// Add this
commands.trigger(SfxKind::PlayerShoot(combat.power_type));

```

Likewise, when an enemy fires back, we update `src/enemy/combat.rs`:

```

// In src/enemy/combat.rs
use crate::audio::SfxKind;

```

```

// In src/combat/systems.rs
// Inside enemy_attack function
spawn_projectile(&mut commands, spawn_position, combat.power_type, &visuals,
    ProjectileOwner::Enemy);

```

```
// Add this
commands.trigger(SfxKind::EnemyShoot);
```

Combat

Now when the player or the enemy takes a hit.

Remember how we set up global observers for `ProjectileHit` and `EntityDeath` in Chapter 8? This is where keeping things decoupled really shines. What happens when an observer triggers *another* observer? It works perfectly.

Open `src/combat/observers.rs`:

```
// In src/combat/observers.rs
use crate::audio::SfxKind; // Add this import

pub fn on_projectile_hit(
    hit: On<ProjectileHit>,
    mut healths: Query<&mut Health>,
    mut commands: Commands,
) {
    let Ok(mut health) = healths.get_mut(hit.target) else { return; };

    health.take_damage(&mut commands, hit.target, hit.damage);

    // Add this
    commands.trigger(SfxKind::Hit);

    // ...
}
```

And for entity death:

```
pub fn on_entity_death(
    death: On<EntityDeath>,
    mut commands: Commands,
    players: Query<(), With<Player>>,
    mut next_state: ResMut<NextState<GameState>>,
) {
    let entity = death.entity;
    let is_player = players.get(entity).is_ok();

    commands.entity(entity).despawn();

    // Update this
    if is_player {
        // Did the player die?
        commands.trigger(SfxKind::PlayerDeath); // Add this
        next_state.set(GameState::GameOver);
    } else { // Add this else block
        // Was it an enemy?
        commands.trigger(SfxKind::EnemyDeath);
    }
}
```

Inventory

Finally, our game has collectable items like trees and drops. When the player walks over an item, we update the `Inventory` resource and despawn the entity. Let's add that satisfying "pickup" ping.

Open `src/inventory/systems.rs`:

```
// In src/inventory/systems.rs
use crate::audio::SfxKind; // Add this import

// Inside handle_pickups:
    for (entity, kind) in collected {
        commands.entity(entity).despawn();
        let count = inventory.add(kind);

        // Trigger pickup sound
        commands.trigger(SfxKind::Pickup); // Add this

        info!(
            " Picked up {} (total: {}) – inventory: {}",
            kind, count, inventory.summary()
        );
    }
}
```

UI Buttons

We also want to add a click sound when the user navigates the menus. This applies to both the `main_menu.rs` and the `pause.rs` modules. Since they both follow the same pattern of checking button interactions, the update is nearly identical.

First, import our enum in both files:

```
// In src/state/main_menu.rs AND src/state/pause.rs
use crate::audio::SfxKind;
```

In `src/state/main_menu.rs`, update `handle_main_menu_buttons`. Make sure to add `mut commands: Commands` to the function parameters:

```
// In src/state/main_menu.rs
pub fn handle_main_menu_buttons(
    mut commands: Commands, // <-- Add this!
    // ...
) {
    for (interaction, button) in interaction_query.iter() {
        if *interaction != Interaction::Pressed {
            continue;
        }

        // Trigger the click sound!
        commands.trigger(SfxKind::ButtonClick);

        // rest of the code ...
    }
}
```

Do the exact same thing in `src/state/pause.rs` for `handle_pause_buttons`:

```
// In src/state/pause.rs
pub fn handle_pause_buttons(
    mut commands: Commands, // <-- Add this!
    // ...
) {
    for (interaction, button) in interaction_query.iter() {
        if *interaction != Interaction::Pressed {
            continue;
        }
    }
}
```



```
}

// Trigger the click sound!
commands.trigger(SfxKind::ButtonClick);

// rest of the code ...
```

Wiring It Up in Main

Open `src/main.rs` and add the plugin to our `App`:

```
// In src/main.rs
// ...
mod audio; // Add this!

use crate::camera::CameraPlugin;
use crate::map::generate::{prepare_tilemap_handles_resource, setup_generator, poll_map_generation};
use crate::state::GameState;

fn main() {
    App::new()
        // ... (existing configuration and plugins)
        .add_plugins(save::SavePlugin)

        // Register our new Audio Plugin!
        .add_plugins(audio::AudioManagerPlugin)

        .add_systems(Startup, prepare_tilemap_handles_resource)
        // ...
        .run();
}
```

Now run your game and enjoy the new sound effects.

```
cargo run
```

Packaging

There's one critical issue we need to address if we want to share our game with others. Up until now, we've hardcoded our asset path in `src/main.rs`:

```
// Pseudo code, don't use
.set(AssetPlugin {
    file_path: "src/assets".into(),
    ..default()
})
```

Relative paths like `"src/assets"` are resolved against the *current working directory*. This works perfectly when we use `cargo run` during development because Cargo sets the working directory to our project root.

But if you build the game for release (`cargo build --release`) and someone double-clicks the executable, the game will crash trying to find `"src/assets"`!

We need to fix this by copying our assets next to the compiled executable automatically, and configuring our game to look there first.

Copy Assets at Build Time

Create a new file named `build.rs` at the absolute root of your project (next to `Cargo.toml`). Cargo automatically runs this script before compiling your game.

To fix this, our first step is to ensure that whenever Cargo builds our game, it also copies the entire `src/assets` folder right next to the finished executable. We can automate this by writing a build script. Cargo specifically looks for a file named `build.rs` at the root of a project and runs it *before* compiling the rest of the game.

```
// build.rs <- This needs to be created at the root of the project along with Cargo.toml
use std::env;
use std::fs;
use std::path::Path;

fn main() {
    let profile = env::var("PROFILE").unwrap();
    let manifest_dir = env::var("CARGO_MANIFEST_DIR").unwrap();

    let src_assets = Path::new(&manifest_dir).join("src").join("assets");
    let dst_assets = Path::new(&manifest_dir)
        .join("target")
        .join(&profile)
        .join("assets");

    if src_assets.exists() {
        if dst_assets.exists() {
            let _ = fs::remove_dir_all(&dst_assets);
        }
        let _ = copy_dir_all(&src_assets, &dst_assets);
    }

    println!("cargo:rerun-if-changed=src/assets");
}

fn copy_dir_all(src: &Path, dst: &Path) -> std::io::Result<()> {
    fs::create_dir_all(dst)?;
    for entry in fs::read_dir(src)? {
        let entry = entry?;
        let ty = entry.file_type()?;
        let dst_path = dst.join(entry.file_name());
        if ty.is_dir() {
            copy_dir_all(&entry.path(), &dst_path)?;
        } else {
            fs::copy(entry.path(), &dst_path)?;
        }
    }
    Ok(())
}
```

Let's break down this script. First, we determine if we are building a `debug` or `release` profile, and we find the absolute path to our project directory.

We define our source (`src/assets`) and destination (`target/release/assets` or `target/debug/assets`). If the source exists, we recursively copy all files and folders over using our custom `copy_dir_all` function, wiping the destination first if it already exists to ensure a clean slate.

This script explicitly checks the active profile (`debug` or `release`) and copies the `src/assets` folder directly into `target/release/assets`. The `cargo:rerun-if-changed` bit ensures the script only runs when we actually modify an asset, saving us compile time.

Dynamic Asset Paths

Now we update `src/main.rs` to prioritize the newly copied folder next to the executable over the hardcoded `"src/assets"`.

Now that our assets are automatically bundled next to our executable, we need to instruct Bevy to look there. But we still want `cargo run` to work smoothly during development!

To handle both scenarios, we will create a helper function that dynamically determines the correct asset path. It will first check if an `assets` folder exists right next to the running executable (our release build scenario). If it doesn't find one, it falls back to using the hardcoded `"src/assets"` path (our development scenario).

```
// In src/main.rs
// Add this helper function above main()

fn get_assets_path() -> String {
    // Check for assets/ next to the executable first (release builds)
    if let Ok(exe_path) = std::env::current_exe() {
        if let Some(exe_dir) = exe_path.parent() {
            let exe_assets = exe_dir.join("assets");
            if exe_assets.exists() {
                return exe_assets.to_string_lossy().to_string();
            }
        }
    }
    // Fallback for `cargo run` from project root
    "src/assets".to_string()
}
```

This function uses `std::env::current_exe()` to find the exact location of the running program on the user's hard drive. It then looks at the parent directory and appends `/assets`. If that path actually exists, we found our bundled assets! If any step fails or the folder isn't there, it simply returns our classic `"src/assets"`.

Then, update your `App::new()` builder in `main()` to use it:

```
// In src/main.rs

fn main() {
    let assets_path = get_assets_path(); // Add this line!

    App::new()
        .insert_resource(ClearColor(Color::BLACK))
        .add_plugins(
            DefaultPlugins
                .set(AssetPlugin {
                    file_path: assets_path.into(), // Update this line!
                    ..default()
                })
        )
}
```

Removing Debug Assertions

Finally, in `src/collision/map.rs`, we had several helper methods explicitly hidden behind `#[cfg(debug_assertions)]`. As our logic has evolved during development, we actually depend on these methods returning proper values in our release builds too!

Open `src/collision/map.rs` and remove the `#[cfg(debug_assertions)]` macro from `width()`, `height()`, `tile_size()`, and `origin()`.

```
// In src/collision/map.rs
// Remove the #[cfg(debug_assertions)] tags from these methods:

// Delete #[cfg(debug_assertions)] but keep the method
pub fn width(&self) -> i32 {
    self.width
}
```

```
}

// Delete #[cfg(debug_assertions)] but keep the method
pub fn height(&self) -> i32 {
    self.height
}
```

With these fixes in place, your compiled executable will always be able to locate its assets! If you run `cargo build --release`, you can zip the generated executable along with its generated `assets` sibling folder to keep them bundled.

Depending on the operating system, you may still need to handle code signing and notarization before others can run your game without security warnings. Check the [Bevy Cheatbook](#) for more details on specific platform requirements.

More chapters are coming soon, including Multiplayer, AI-driven NPCs, ...

The Impatient Programmer's Guide to Bevy and Rust: Chapter 12 - Let There Be Networking

Thinking in Systems for Multiplayer

I was 12, maybe 13. My brother and I wanted to play Age of Empires together. I spent an afternoon wrestling with modem settings, LAN cables, configs that made no sense. Then, his computer appeared on the multiplayer screen. I still remember that feeling. Now it's time to code it to life.

In Chapter 1, we asked: *“What do we need to build a game where a player can move?”* We broke it down into two systems, a Setup System and an Update System, and suddenly Bevy made sense.

Let's do the same thing for multiplayer. Ignore servers, databases, WebSockets. Start with the player's experience and ask: what systems does that require?

Here's what we want to build, assume every player connects to the same shared game world:

Action	What Happens
John launches the game	John appears in the shared world with his name displayed
Sara launches the game	Sara appears in the same world; John can see Sara
Sara walks left	John sees Sara move left in real time
Sara closes the game	Sara disappears from John's screen
Sara relaunches the game	Sara reappears at her last position with the same name

Simple enough. Now let's think about what systems are actually required to make it work.

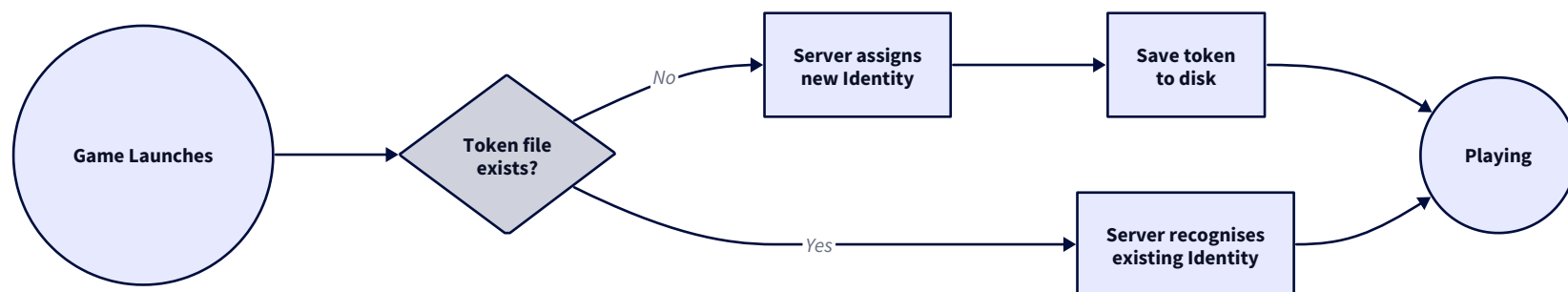
System 1: Player Identification (Identity & Auth)

When John launches the game, the server needs to know: *is this a new player or a returning one?*

In a web app, you'd build a login page, hash passwords, issue JWT tokens, manage sessions. That's weeks of work before writing any game logic.

For our game, we want something simpler: the first time you launch, you get a unique ID. Every time after that, you come back with the same ID automatically. No login, no signup.

The standard approach: a token. A unique string the server generates the first time you connect, like handing you a key. Your game saves it to disk. Every launch after that, you show up, present the token, and the server says “ah, it's John”, no password, no email, no account.

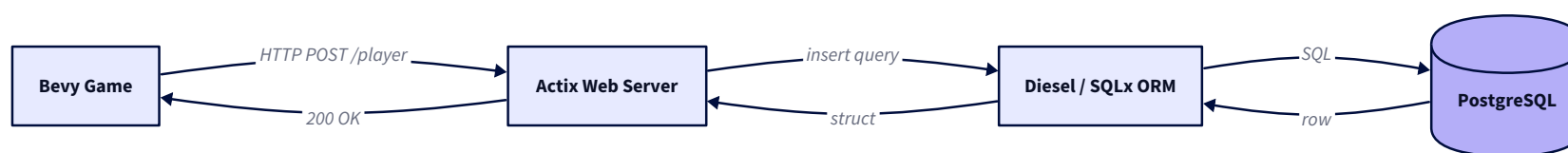


Building this yourself means writing three separate things: a server that hands out tokens, a client that saves them to disk, and a gatekeeper that checks them on every connection. Also handling edge cases like expired tokens, reinstalls that wipe the save file.

System 2: Storing Player State (Persistence)

When John connects, we save his name and position so tomorrow, he picks up where he left off.

In a traditional setup, saving a single player record involves four separate pieces working together:



The **Bevy Game** sends an HTTP request to the **Actix Web Server**. The server passes the data to the **ORM**, which then writes to **PostgreSQL**. The response travels back the same way.

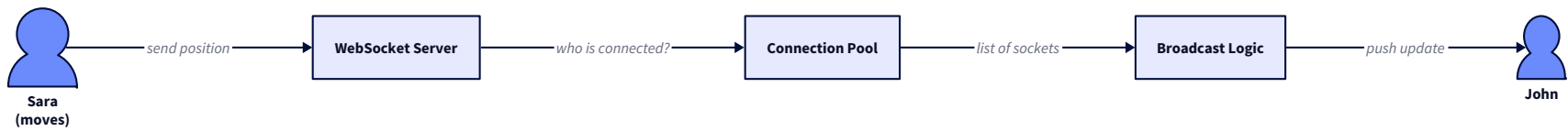
Four things to set up, keep in sync, and debug. Database, ORM, server, all wired together and every schema change touches all of them.

System 3: Real-Time Updates (State Synchronization)

Sara moves, how does John’s screen update?

You could poll: John asks the server “where is everyone?” 30 times a second. Slow, expensive, and doesn’t scale.

The right way is push-based: the server tells John the moment Sara moves. But in a traditional setup, you build that architecture yourself:

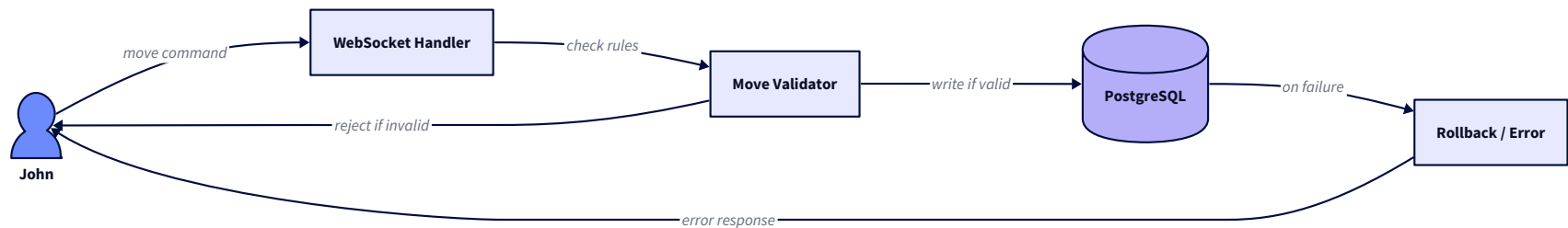


What makes this hard is that your game code and your networking code end up being two separate things that have to stay in sync. Every time you add something new, a door that opens, an item you can pick up, you have to update both. One missed update and you’re debugging for hours.

System 4: Game Logic Validation (Server Authority)

Who decides if a move is valid? If the player’s computer does, cheaters just edit their memory and teleport. Or send a fake message straight to your server. The client can’t be trusted. The server decides.

In a traditional setup that means writing a check for every action. Move? Check the destination is reachable. Pick up item? Check it’s actually there. Every new mechanic, a new check. And if anything fails midway, you need code to undo the partial change.



SpacetimeDB

In summary, here’s everything you’d need to build from scratch:

System	Functionalities
Identity	Auth server, token generation, client-side storage, middleware
Persistence	PostgreSQL, migrations, ORM configuration, connection pool
Real-Time	WebSocket server, connection pool, broadcast logic
Validation	Endpoint validators, rollback logic, consistency rules

And none of these systems live in isolation. Your WebSocket server needs to talk to your database. Your move checks need to run before anything gets saved. Your token system needs to verify who’s talking before any of the above can happen. Every place one system touches another is a place where things can silently break.

All of this before two players have seen each other move on screen.

It’s a lot.

But what if you didn’t have to build any of it?

What if one thing handled all of it, the database, the websocket server, the auth, the game logic?

That thing is **SpacetimeDB**.

Instead of four separate systems stitched together, you write one Rust module that runs *inside* the database engine. The module defines your data as Rust structs (tables) and your logic as Rust functions (reducers). SpacetimeDB handles everything else:

- **Identity:** SpacetimeDB has authentication built in. Every player gets a permanent ID the moment they connect, no login system, no token management, no auth server to build.
- **Persistence:** mark a Rust struct with `#[spacetimedb::table]` and it becomes a database table. No PostgreSQL to install, no ORM to configure, no need of manual data wiring.
- **Real-Time:** mark a table as `public` and SpacetimeDB pushes row-level changes to every subscribed client automatically. No WebSocket code to write.
- **Validation:** reducers run as atomic transactions. If your logic returns an error, the entire transaction rolls back, guaranteed.

What took four separate systems to build, deploy, and maintain now fits in a single Rust module, one thing to write, one thing to publish, one place to look when something breaks. Let's set it up now.

Installing SpacetimeDB

Let's set up the tools. SpacetimeDB's CLI handles everything: creating modules, building, running locally, deploying.

Installing the CLI

On macOS and Linux:

```
curl -sSf https://install.spacetimedb.com | sh
```

On other platforms, follow the [official install guide](#).

Verify it installed correctly:

```
spacetime version list
```

Upgrading to v2

SpacetimeDB is actively developed. For this chapter, we need **v2.1.0 or newer**. Check what version you have running by default, and if you're on v1.x, install and switch to v2:

```
# Install v2.1.0
spacetime version install 2.1.0

# Set it as the default
spacetime version use 2.1.0

# Confirm
spacetime version list
# Should show: 2.1.0 (current)
```

Upgrading Rust

SpacetimeDB v2.0 requires Rust **1.93.0 or newer**. Check your version and update if needed:

```
rustc --version
rustup update stable
```

Starting a Local Server

Open a new terminal window and start a local SpacetimeDB instance. Keep this running as you work:

```
spacetime start
```

This gives you a local server at <http://127.0.0.1:3000>. The same way you might run a local Postgres database during development, except SpacetimeDB is already a WebSocket server as well.

Creating the Server Module

The server-side code for our game lives in a `server/` directory alongside our Bevy game. A separate Rust crate that compiles to WebAssembly and uploads to SpacetimeDB.

From your `chapter12/` project root, create the directory:

```
mkdir -p server/src
```

Create `server/Cargo.toml`:

```
[package]
name = "bevy-game-server"
version = "0.1.0"
edition = "2024"

[lib]
crate-type = ["cdylib"]

[dependencies]
spacimedb = "2.0"
log = "0.4"
petname = { version = "2.0", default-features = false, features = ["default-words"] }
getrandom = { version = "0.2", features = ["custom"] }
```

Let's walk through each decision here.

`crate-type = ["cdylib"]` tells Cargo to produce a library SpacetimeDB can load as a WASM binary.

`spacimedb = "2.0"` is the SDK. It gives you tables and reducers, the two things you'll use throughout this chapter.

`petname` generates readable random names like `"brave-purple-fox"` for auto-assigning usernames. We disable default features and only pull in `default-words` because petname's default RNG doesn't work in WASM.

`getrandom = { version = "0.2", features = ["custom"] }` This fixes a build issue. Petname uses getrandom internally, which normally asks the OS for a random seed but inside WASM there is no OS. The custom feature switches it to use SpacetimeDB's own RNG instead. Declaring it in your own Cargo.toml forces the entire build to use that version. Without this line, compilation fails.

Building Blocks of SpacetimeDB

The two concepts you will use throughout this chapter are **tables** and **reducers**.

A **table** is a Rust struct with `#[spacimedb::table]` on it. SpacetimeDB creates and manages the actual storage for you, no `CREATE TABLE`, no SQL. The struct's fields become the columns. Here's the `Player` table we'll define shortly:

```
//Pseudo code, don't use
#[spacimedb::table(accessor = player, public)]
pub struct Player {
    #[primary_key]
    identity: Identity,
    #[unique]
    username: String,
    position_x: f32,
    position_y: f32,
    is_online: bool,
}
```

Reading a row looks like `ctx.db.player().identity().find(sender)`. Writing a row looks like `ctx.db.player().insert(...)`. The `public` flag means subscribed clients receive every change to

this table in real time.

A reducer is a Rust function marked with `#[spacetime::reducer]`. When a client calls it, it runs on the server as an all-or-nothing transaction – either everything succeeds and saves, or something fails and nothing is written. Here’s the simplest one:

```
//Pseudo code, don't use
#[spacetime::reducer(init)]
pub fn init(_ctx: &ReducerContext) {
    log::info!("Server module initialized");
}
```

To see why all-or-nothing matters, imagine a reducer that makes two database changes and the second one fails. Consider this `pick_up_item` reducer:

```
//Pseudo code, don't use
#[spacetime::reducer]
pub fn pick_up_item(ctx: &ReducerContext, item_id: u64) -> Result<(), String> {
    // Step 1: remove the item from the world
    ctx.db.world_item().id().delete(item_id);

    // Step 2: check the player has room in their inventory
    let player = ctx.db.player().identity().find(ctx.sender())
        .ok_or("Player not found"?);

    if player.inventory_count >= 10 {
        // Inventory full – return an error.
        // SpacetimeDB rolls back Step 1 automatically:
        // the item reappears in the world as if nothing happened.
        return Err("Inventory is full".to_string());
    }

    // Step 3: add the item to the player's inventory
    ctx.db.inventory().insert(InventoryItem { owner: ctx.sender(), item_id });

    Ok(())
}
```

First we delete the item from the world. However, if later we find the inventory is full, SpacetimeDB undoes the initial delete, the item reappears as if it was never touched. You don’t have to write cleanup code for failure cases; the atomic transaction guarantees it.

Defining the Player Table

Now let’s actually build the module. Here’s what it needs to handle:

Event	Action
John connects for the first time	We create a record for him, generated name, position assigned and marked online
John connects again	We find his existing record and mark him online, name and position exactly as he left them
John disconnects	We mark him offline
John wants a custom name	We call a reducer; the server validates and updates his name

Everything flows through one table, `Player` and a handful of reducers. Let’s define the table first.

Create `server/src/lib.rs` and start with the table definition:

```
// server/src/lib.rs
use petname::Generator;
use spacetimedb::{Identity, ReducerContext, Table};

// Map dimensions: 241 tiles × 169 tiles × 64px per tile
const SPAWN_X: f32 = 7712.0; // center of world
const SPAWN_Y: f32 = 5408.0;

#[spacetimedb::table(accessor = player, public)]
pub struct Player {
    #[primary_key]
    identity: Identity,
    #[unique]
    username: String,
    position_x: f32,
    position_y: f32,
    is_online: bool,
}
```

```
#[spacetimedb::table(accessor = player, public)]
```

This macro transforms the `Player` struct into a **database table**. SpacetimeDB will create a persistent table with these exact columns. Every field in the struct becomes a column.

The `accessor = player` argument gives the table a name. Inside reducers, you'll access it as `ctx.db.player()`, it helps with querying and modifying this table.

The `public` flag means clients are allowed to **subscribe** to this table. When a client subscribes (which we'll implement in the next chapter), SpacetimeDB will push every row to them immediately, and then push any future changes in real-time. Without `public`, only server-side reducer code can see the data.

```
#[primary_key] identity: Identity
```

`Identity` is a SpacetimeDB built-in type, a 256-bit unique identifier that the server automatically assigns to each client connection. It persists across sessions: the same player connecting from the same device will always have the same `Identity`.

So is Identity based on the user's device?

Not exactly the device, but the token file on it. When a client connects for the first time, SpacetimeDB's SDK generates a cryptographic key and saves it as a token file on disk. The Identity is derived from that key. As long as that token file exists on the same machine, every future connection produces the same Identity.

If John deletes the token file, reinstalls the game, or connects from a different computer, SpacetimeDB sees a new token and creates a new Identity.

`#[unique] username: String` – the `#[unique]` attribute tells SpacetimeDB to create an index on this field and enforce that no two rows share the same value. It also generates a `.username()` accessor, so you can look up a player by name directly with `ctx.db.player().username().find(&name)` – a fast indexed lookup rather than a full table scan.

`position_x` and `position_y` will store where the player is standing in the world. We initialize them to the center of the map (`SPAWN_X`, `SPAWN_Y`), which we calculated from our world dimensions: 241 tiles × 64 pixels wide, 169 tiles × 64 pixels tall.

`is_online` tracks whether the player is currently connected. This is how we distinguish between “this player exists but left” and “this player is actively playing right now.”

Generating Human-Readable Usernames

Before writing the reducers, let's build the username generator. Ideally a player would type their own name, but that requires a name-entry screen, input handling, and form submission before

the game even starts. By generating a name automatically on first connect, we can keep it simple.

```
// server/src/lib.rs (continued)

fn generate_username(ctx: &ReducerContext) -> String {
    let mut rng = ctx.rng();
    let petnames = petname::Petnames::default();
    petnames
        .generate(&mut rng, 3, "-")
        .unwrap_or_else(|| format!("player-{}", ctx.timestamp.to_micros_since_unix_epoch()))
}
```

A normal random number generator produces a different result every time you run it. But SpacetimeDB requires reducers to be *deterministic*: given the same inputs, they must always produce the same result. This is what allows SpacetimeDB to safely replay or audit transactions.

`ctx.rng()` satisfies that requirement. It's seeded from the transaction's timestamp, so it produces the same sequence for a given transaction but a different sequence for different transactions. We pass `&mut rng` to `petnames.generate()` along with `3` (three words) and `"-"` as the separator. The result is a name like `"brave-purple-fox"` or `"calm-dancing-hawk"`.

So if the same inputs produce the same result, won't all players get the same name?

No, each connection is a separate transaction with a different timestamp. "Same inputs" means the same transaction, not all transactions. Two players connecting at different moments have different timestamps, get different RNG seeds, and therefore get different names. The determinism guarantee is per-transaction, not across transactions.

SpacetimeDB also processes transactions sequentially, each one completes before the next begins. So even two players connecting at the exact same instant are queued and executed one after the other, each receiving a distinct timestamp in order. No two transactions ever share the same timestamp.

You'll also notice the `use petname::Generator;` added in the imports. The `generate()` method lives on the `Generator` trait, not directly on the `Petnames` struct. In Rust, traits must be in scope for their methods to be callable. Without this import, the compiler would tell you the method doesn't exist, even though it's implemented right there.

Writing the Reducers

Now for the heart of our server module, the reducers.

Wait, why are they called reducers?

The name comes from functional programming. A "reduce" operation takes some existing state and an input, and produces new state. Redux, the popular JavaScript state management library, made this pattern mainstream: a reducer is a function of the form `(currentState, action) => newState`.

SpacetimeDB uses the same idea. A reducer takes the current state of the database and an incoming action, the call from a client, and produces a new database state. The key property is that given the same starting state and the same input, a reducer always produces the same result. That determinism is what makes it safe to run as a transaction: SpacetimeDB can apply it, roll it back, or replay it, and the outcome is always predictable.

Now back to writing the reducers. We need four of them.

Module Startup

The first reducer SpacetimeDB calls is `init`. It runs exactly once, when you publish the module for the first time. Think of it as a setup hook: a place to run any one-time initialization before players start connecting. In later chapters, this is where we'll seed the world with spawn points or map configuration. For now, we just log that it ran so we can confirm the publish worked.

```
// server/src/lib.rs (continued)

#[spacetime::reducer(init)]
pub fn init(_ctx: &ReducerContext) {
    log::info!("Server module initialized");
}
```

A Client Connects

Every time a client opens a connection, whether it's John connecting for the first time or returning after a week away, SpacetimeDB automatically calls the `client_connected` reducer. We don't invoke it ourselves; SpacetimeDB fires it the moment the connection is established, before the client receives any data.

This reducer has two jobs depending on who's connecting:

- **First-time player:** create a fresh record with a generated name and place them at the world spawn point.
- **Returning player:** find their existing record and mark them online, their name and last position from the previous session are already saved.

```
// server/src/lib.rs (continued)

#[spacetime::reducer(client_connected)]
pub fn identity_connected(ctx: &ReducerContext) {
    let sender = ctx.sender();
    if let Some(player) = ctx.db.player().identity().find(sender) {
        // Returning player - mark them online
        log::info!("Player '{}' reconnected", player.username);
        ctx.db.player().identity().update(Player {
            is_online: true,
            ..player
        });
    } else {
        // New player - create their record
        let username = generate_username(ctx);
        log::info!("New player '{}' joined", username);
        ctx.db.player().insert(Player {
            identity: sender,
            username,
            position_x: SPAWN_X,
            position_y: SPAWN_Y,
            is_online: true,
        });
    }
}
```

`ctx.sender()` gives us the Identity of whoever just connected. We then call `.identity().find(sender)` to look for an existing row in the `Player` table.

If a row is found, John has connected before, we update just `is_online` to `true`. The `..player` syntax is Rust's struct update shorthand: it means "copy all fields from `player` unchanged, except the ones I've listed here." So John's username, position, and everything else stays exactly as it was.

If no row is found, John is new, we call `generate_username(ctx)` to produce a random name and insert a brand new row at the world spawn coordinates (`SPAWN_X`, `SPAWN_Y`).

A Client Disconnects

When a connection closes, whether John quit the game, lost his internet, or the game crashed, SpacetimeDB automatically calls the `client_disconnected` reducer.

The goal is simple: find John's record and mark him offline. We don't delete the row. That's intentional, his position stays saved in the database, so the next time he connects, he'll reappear exactly where he left off.

```
// server/src/lib.rs (continued)

#[spacetimedb::reducer(client_disconnected)]
pub fn identity_disconnected(ctx: &ReducerContext) {
    let sender = ctx.sender();
    if let Some(player) = ctx.db.player().identity().find(sender) {
        log::info!("Player '{}' disconnected", player.username);
        ctx.db.player().identity().update(Player {
            is_online: false,
            ..player
        });
    } else {
        log::warn!("Disconnect for unknown identity: {:?}", sender);
    }
}
```

We look up the disconnecting player's row with `ctx.sender()`. If found, we flip `is_online` to `false`, the same `..player` struct update pattern as before, copying everything else unchanged.

The `else` branch handles a disconnect for an Identity with no record. This shouldn't happen under normal operation, every disconnect should follow a connect, but network edge cases exist. Rather than crashing the server, we log a warning and move on. It's safe to ignore, but visible in the logs if something unexpected is happening.

Choosing a Custom Name

The three reducers above are all lifecycle reducers, SpacetimeDB calls them automatically.

`register_player` is different: it's a regular reducer the client calls deliberately. It's also useful right now as a way to manually test whether our code is working, we can call it from the CLI and then check the database to confirm the change went through.

```
// server/src/lib.rs (continued)

#[spacetimedb::reducer]
pub fn register_player(ctx: &ReducerContext, username: String) -> Result<(), String> {
    if username.is_empty() {
        return Err("Username must not be empty".to_string());
    }
    if username.len() > 32 {
        return Err("Username must be 32 characters or less".to_string());
    }

    let sender = ctx.sender();

    // Check if the username is already taken by a different player
    if ctx.db.player().username().find(&username)
        .is_some_and(|p| p.identity != sender)
    {
        return Err(format!("'{}' is already taken", username));
    }

    if let Some(player) = ctx.db.player().identity().find(sender) {
        log::info!("Player '{}' renamed to '{}'", player.username, username);
        ctx.db.player().identity().update(Player {
            username,
            ..player
        });
    }
}
```

```

    });
    Ok(())
} else {
    Err("Cannot rename: player not found. Connect first.".to_string())
}
}

```

The function runs three checks before writing anything. First, basic validation: the name can't be empty or over 32 characters. Second, uniqueness: `ctx.db.player().username().find(&username)` looks up any existing row with that name using the index created by `#[unique]`, a direct lookup rather than scanning every row. The `.is_some_and(|p| p.identity != sender)` check returns `true` only if the name is taken by a *different* player, so a player can re-submit their own current name without hitting an error. If the name is taken, we return `Err` and nothing is written.

If all checks pass, we find the caller's record with `ctx.sender()`, update the `username` field, and return `Ok(())`. The `log::info!` records both the old and new name, which is useful when reading server logs.

Building the Module

We don't use `cargo build` for SpacetimeDB modules. We use `spacetime build`, which compiles to `wasm32-unknown-unknown` (the WebAssembly target) and packages the result correctly.

From inside the `server/` directory:

```

cd server
spacetime build

```

You should see Cargo compile all the dependencies, then finish with:

```
Build finished successfully.
```

You may also see a warning about `wasm-opt` not being found. This is an optional optimizer that shrinks the compiled WASM file for faster uploads. You can safely ignore it during development, or install it with `brew install binaryen` to make it go away.

Publishing to Your Local Server

Make sure `spacetime start` is still running in its terminal, then publish the module:

```
spacetime publish --server http://127.0.0.1:3000 bevy-game
```

You should see:

```

Build finished successfully.
Uploading to http://127.0.0.1:3000 ...
Publishing module...
Updated database bevy-game

```

The module is now running inside your local SpacetimeDB instance.

Viewing the Logs

Open another terminal and watch the server logs:

```
spacetime logs --server http://127.0.0.1:3000 bevy-game -f
```

You should see `Server module initialized` – that’s your `init` reducer confirming it ran.

Testing with SQL

SpacetimeDB supports SQL queries directly against your tables. Let’s verify the schema was created correctly:

```
spacetime sql --server http://127.0.0.1:3000 bevy-game "SELECT * FROM player"
```

You’ll see the column headers with an empty table – which is exactly right!

```
identity | username | position_x | position_y | is_online
-----+-----+-----+-----+-----
```

The table is empty because the `Player` rows are only created when a client **connects**. No client has connected yet, our Bevy game doesn’t have the connection code yet. But the schema is there and correct.

Testing Reducers from the CLI

We don’t need to build the Bevy client to test our reducers. The `spacetime call` command lets us invoke any reducer directly from the terminal, which is incredibly useful for verifying your server logic in isolation.

```
# This works
spacetime call --server http://127.0.0.1:3000 bevy-game register_player '"TestPlayer"'
```

When you run `spacetime call`, something interesting happens behind the scenes. The CLI doesn’t just fire the reducer in isolation, it opens a real WebSocket connection to SpacetimeDB using its own identity. That means our `identity_connected` lifecycle reducer fires first, creating a fresh player row with a generated name. Then `register_player` runs, finds that row, and renames it to `"TestPlayer"`. Finally, the CLI connection closes and `identity_disconnected` fires, setting `is_online` to `false`.

You can see all of this by querying the table immediately after:

```
spacetime sql --server http://127.0.0.1:3000 bevy-game "SELECT * FROM player"
```

```
identity | username      | position_x | position_y | is_online
-----+-----+-----+-----+-----
0x.....  | "TestPlayer" | 7712      | 5408      | false
```

Three things worth noticing here:

- `identity` is a long hex value, that’s the 256-bit Identity the CLI is using. Unlike a session cookie, this identity is stored persistently in the CLI’s config on your machine. Every `spacetime call` you make from this machine reuses the same token and therefore the same identity.
- `position_x` and `position_y` are `7712` and `5408`, exactly our `SPAWN_X` and `SPAWN_Y` constants. The player was placed at the center of the world automatically by `identity_connected`.
- `is_online` is `false`, the CLI connection closed immediately after the reducer returned, so `identity_disconnected` fired and marked the player offline. This is correct behaviour. When our

Bevy client connects and stays open, this field will be `true` the entire session.

Why does calling `register_player` again update the same row instead of creating a new player?

Because the CLI behaves exactly like your game client will. It stores a token file on your machine. Every `spacetime call` reuses that token → same Identity → `identity_connected` finds your existing row and marks it online instead of creating a new one. You are always the same player on this machine.

This is the intended behaviour. When John installs the game on his computer and Sara installs it on hers, they each have different token files, different identities, and different rows. The CLI mirrors that exactly.

To simulate a second player from the terminal, get a new server-issued identity:

```
# Save your current token first so you can restore it later
spacetime login show --token

# Log out, then get a fresh identity from the local server
spacetime logout
spacetime login --server-issued-login http://127.0.0.1:3000
```

Now the next `spacetime call` connects with a fresh identity – `identity_connected` finds no existing row and inserts a new player. Run the SQL query and you'll see two rows.

To restore your original identity:

```
spacetime login --token <your-original-token>
```

Here's what we'll implement, clicking **Multiplayer** on the main menu will connect the game to the local SpacetimeDB server, show a live connection status screen, and recognize the same player on every subsequent reconnect using a saved token. We'll keep the single player as it is.

Connecting the Bevy Client

The server works. Now let's connect the game to it.

In this architecture, our Bevy game is the **client**, the program running on the player's machine. Its job is to show the game world, respond to player input, and talk to the server. It doesn't make decisions about what's valid or what gets saved; it just sends requests and reacts to what the server sends back.

Everything we've built so far, the map, the characters, the combat, runs entirely on the player's machine. In single-player, that's all it ever needs to do. In multiplayer, the client also needs to stay in sync with the server: other players' positions, who's online, what changed while you were away.

Now, we add that connection layer. It only activates when the player clicks **Multiplayer**, single-player is completely untouched.

`spacetimedb-sdk` is the Rust client library. It handles WebSocket connections, authentication, message parsing, and the client-side cache of subscribed table rows. We don't build any of that ourselves.

Open `Cargo.toml` in your `chapter_12/` root and add one line:

```
[dependencies]
bevy = { version = "0.19", features = ["mp3", "wav"] }
bevy_procedural_tilemaps = "0.4"
# ... other deps ...
spacetimedb-sdk = "2.1.0" # ← add this
```


Generate Client Bindings

If you've used GraphQL, this will feel familiar. A GraphQL schema describes what data your server exposes, and tools like Apollo or codegen generate typed client code from it, so your frontend gets auto-complete and compiler errors instead of hand-written fetch calls.

Bindings work the same way here: the SpacetimeDB CLI reads your compiled server module and generates typed Rust code for your game client, methods like `conn.db.player()` and `ctx.db.player().identity().find(...)` that map directly to the tables and reducers you defined. Change your server schema and regenerate, the client code updates to match.

Run this from inside the `server/` directory:

Ensure to replace `{ADD_FULL_PATH_TO_CHAPTER12}` with path to your `chapter12` folder

```
cd server
spacetime generate --lang rust \
  --out-dir {ADD_FULL_PATH_TO_CHAPTER12}/src/module_bindings \
  --module-path .
```

You'll see:

```
Writing file src/module_bindings/player_table.rs
Writing file src/module_bindings/player_type.rs
Writing file src/module_bindings/register_player_reducer.rs
Writing file src/module_bindings/mod.rs
Generate finished successfully.
```

These files are generated code, don't edit them by hand. Any time you change the server schema (add a field, rename a reducer, add a new table), re-run `spacetime generate` and they'll be regenerated automatically.

Track Game Mode

`GameMode` helps us track the user's choice of if they want a single player or a multiplayer.

Add this to `src/state/game_state.rs`, right below the `GameState` enum:

```
// src/state/game_state.rs

use bevy::prelude::*;

#[derive(States, Default, Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum GameState {
    #[default]
    MainMenu,
    Loading,
    Playing,
    Paused,
    GameOver,
}

// ↓ Add these
#[derive(Resource, Debug, Clone, PartialEq, Eq)]
pub enum GameMode {
    SinglePlayer,
    Multiplayer,
}

pub fn in_multiplayer(mode: Option<Res<GameMode>>) -> bool {
    mode.is_some_and(|m| *m == GameMode::Multiplayer)
}
```

`GameMode` is a plain Rust enum decorated with `#[derive(Resource)]`. That's all Bevy needs to treat it as a global resource, no component, no entity, just a value attached to the world.

`in_multiplayer` is a **run condition**, a function Bevy evaluates before running a system. If it returns `false`, the system is skipped entirely. We define it here in the state module (not in the network module) so that `main.rs` and the loading screen can also use it to gate non-network systems like map generation.

Update the Main Menu

Now we wire `GameMode` to the buttons. Three changes to `src/state/main_menu.rs`:

Import `GameMode`

```
use super::{GameState, GameMode}; // was: use super::GameState;
```

Add the `Multiplayer` variant to the button enum:

```
#[derive(Component)]
pub enum MainMenuButton {
    NewGame,
    LoadGame,
    Multiplayer, // ← add this
    Quit,
}
```

Add the button to the list and handle both new clicks:

```
let buttons = [
    (MainMenuButton::NewGame, "New Game"),
    (MainMenuButton::LoadGame, "Load Game"),
    (MainMenuButton::Multiplayer, "Multiplayer"), // ← add this
    (MainMenuButton::Quit, "Quit"),
];
```

And in the button handler, update the `match` to set the mode on each relevant button:

```
match button {
    MainMenuButton::NewGame => {
        commands.insert_resource(GameMode::SinglePlayer); // ← add this
        next_state.set(GameState::Loading);
    }
    MainMenuButton::LoadGame => {
        ui_state.active = true;
        ui_state.mode = SaveLoadMode::Load;
    }
    MainMenuButton::Multiplayer => { // ← add this arm
        commands.insert_resource(GameMode::Multiplayer);
        next_state.set(GameState::Loading);
    }
    MainMenuButton::Quit => {
        exit.write(AppExit::Success);
    }
}
```

Initialize GameMode in the StatePlugin

We set a default so `GameMode` always exists in the world, even before any button is pressed.

In `src/state/mod.rs`, add three lines:

```

pub use game_state::GameState;
pub use game_state::GameMode;      // ← re-export so other modules can use it
pub use game_state::in_multiplayer; // ← re-export the run condition

impl Plugin for StatePlugin {
    fn build(&self, app: &mut App) {
        app
            .insert_resource(GameMode::SinglePlayer) // ← add default
            .init_state::<GameState>()
            // Gate the loading screen: multiplayer shows its own connection screen
            .add_systems(OnEnter(GameState::Loading),
                // update below line
                loading::spawn_loading_screen.run_if(not(in_multiplayer)))
            // ... rest unchanged
    }
}

```

We gate the loading screen behind `not(in_multiplayer)`. When the player clicks Multiplayer, we show our own connection status screen instead of the single player map loading process.

Create the NetworkPlugin

All the pieces are in place, the SDK is installed, the bindings are generated, the mode is tracked, and the menu sets it. Now we need to implement basic networking.

We'll create `NetworkPlugin`, it owns everything network-related: opening the connection, processing incoming messages each frame, showing a connection status screen, and cleaning up when the player leaves.

We split it across two files: `mod.rs` for the plugin definition, and `connection.rs` for everything else.

Create `network` folder inside the `src` directory and inside `src/network/mod.rs`

```

// src/network/mod.rs

mod connection;

use bevy::prelude::*;

use crate::module_bindings::DbConnection;
use crate::state::{in_multiplayer, GameState};

use connection::{
    cleanup_network, connect_to_spacetimedb, despawn_multiplayer_screen,
    handle_multiplayer_back, process_spacetimedb_messages, spawn_multiplayer_screen,
    update_multiplayer_screen,
};

#[derive(Resource)]
pub struct SpacetimeConnection {
    pub conn: DbConnection,
}

pub struct NetworkPlugin;

impl Plugin for NetworkPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(
            OnEnter(GameState::Loading),
            (connect_to_spacetimedb, spawn_multiplayer_screen).run_if(in_multiplayer),
        )
        .add_systems(

```

```

        Update,
        (
            process_spacetimeadb_messages
                .run_if(resource_exists::<SpacetimeConnection>),
            update_multiplayer_screen.run_if(in_state(GameState::Loading)),
            handle_multiplayer_back.run_if(in_state(GameState::Loading)),
        )
        .run_if(in_multiplayer),
    )
    .add_systems(
        OnExit(GameState::Loading),
        despawn_multiplayer_screen,
    )
    .add_systems(
        OnEnter(GameState::MainMenu),
        cleanup_network.run_if(resource_exists::<SpacetimeConnection>),
    );
}
}

```

The `.run_if(...)` conditions make sure systems only run when they're needed. `in_multiplayer` is the top-level gate, in single-player, none of these systems run at all.

Here when the state enters loading, we initialize the connect to spacetimeadb server and we spawn the multiplayer screen.

Later, we process messages from server if the connection goes through. We also update multiplayer screen with the status and allow the user to go back to main menu.

Now let's implement this, create `src/network/connection.rs`

This has five mini systems: opening the connection, ticking incoming messages every frame, showing a status screen, updating the status text, and cleaning up when the player leaves. Let's go through each one.

Mini System 1: Connecting

`connect_to_spacetimeadb` runs once when multiplayer loading begins. Its job: load the saved identity from disk, open a WebSocket connection to the server, register callbacks for connection events, subscribe to the `Player` table, and store the live connection as a Bevy resource so other systems can use it.

The callbacks `on_connect`, `on_connect_error`, `on_disconnect`, are closures you register at build time. The SDK fires them when the corresponding event arrives over the wire.

```

// src/network/connection.rs

use std::path::PathBuf;

use bevy::prelude::*;
use spacetimeadb_sdk::{DbContext, Table};

use crate::module_bindings::player_table::{playerQueryTableAccess, PlayerTableAccess};
use crate::module_bindings::DbConnection;
use crate::state::{GameMode, GameState};
use super::SpacetimeConnection;

const SPACETIMEADB_URI: &str = "http://127.0.0.1:3000";
const DATABASE_NAME: &str = "bevy-game";
const TOKEN_FILENAME: &str = "spacetimeadb_token";

fn token_path() -> Option<PathBuf> {
    let exe = std::env::current_exe().ok()?;
    Some(exe.parent()?.join(TOKEN_FILENAME))
}

```

```

}

fn load_token() -> Option<String> {
    let path = token_path()?;
    let contents = std::fs::read_to_string(&path).ok()?;
    let trimmed = contents.trim();
    if trimmed.is_empty() { None } else { Some(trimmed.to_string()) }
}

fn save_token(token: &str) -> std::io::Result<()> {
    let path = token_path().ok_or_else(|| {
        std::io::Error::new(std::io::ErrorKind::Other, "could not determine executable path")
    })?;
    std::fs::write(path, token)
}

pub fn connect_to_spacetimedb(mut commands: Commands) {
    let token = load_token();

    let conn = DbConnection::builder()
        .with_uri(SPACETIMEDB_URI)
        .with_database_name(DATABASE_NAME)
        .with_token(token)
        .on_connect(|ctx, _identity, token| {
            if let Err(e) = save_token(token) {
                error!("Failed to save SpacetimeDB token: {e}");
            }
            info!("Connected to SpacetimeDB");
            ctx.subscription_builder()
                .on_applied(|ctx| {
                    if let Some(identity) = ctx.try_identity() {
                        if let Some(player) = ctx.db.player().identity().find(&identity) {
                            info!("Playing as: {}", player.username);
                            return;
                        }
                    }
                    info!("Player subscription applied");
                })
                .on_error(|_ctx, err| {
                    error!("Subscription error: {err}");
                })
                .add_query(|q| q.from.player())
                .subscribe();
        })
        .on_connect_error(|_ctx, err| {
            error!("SpacetimeDB connection error: {err}");
        })
        .on_disconnect(|_ctx, err| {
            if let Some(e) = err {
                warn!("Disconnected from SpacetimeDB with error: {e}");
            } else {
                info!("Disconnected from SpacetimeDB");
            }
        })
        .build();

    match conn {
        Ok(conn) => {
            info!("SpacetimeDB connection initiated");
            commands.insert_resource(SpacetimeConnection { conn });
        }
        Err(e) => {
            error!("Failed to initiate SpacetimeDB connection: {e}");
        }
    }
}

```

```

    }
}

```

Note the two imports from the generated bindings: `PlayerTableAccess` brings in the `.player()` method for querying the client cache; `playerQueryTableAccess` brings in the `.player()` method for building subscription queries. Both are needed, both come from `spacetime generate`. We also import `Table` from the SDK, that trait provides the `.iter()` method we'll use later to list online players.

This system runs once when we enter `GameState::Loading`, but only in multiplayer mode.

`token_path()` finds the running executable with `std::env::current_exe()` and places the token file next to it, `target/debug/spacetimedb_token` in development, and next to the binary in a release build. `load_token()` reads and trims that file, returning `None` on first launch. `save_token()` writes the raw token string. When the token is `None`, SpacetimeDB creates a new identity; when it's `Some`, you reconnect as the same player.

The builder. `DbConnection::builder()` is a fluent API. We chain:

- `.with_uri(...)` – the server address
- `.with_database_name(...)` – which database to connect to (the name you used in `spacetime publish`)
- `.with_token(token)` – the saved identity, if any
- `.on_connect(...)` – callback that fires when the WebSocket is established
- `.on_connect_error(...)` – callback if the server is unreachable
- `.on_disconnect(...)` – callback when the connection closes

Inside `on_connect`. The first thing we do is save the token the server just issued, that's what makes the same identity persist across sessions. Then we subscribe to the `Player` table. The `on_applied` callback fires once SpacetimeDB has pushed all the initial rows, at which point we look up the player row by identity and log their username.

`.build()` **is non-blocking**. It returns immediately with `Ok(conn)` or `Err(...)`. The connection happens in the background; callbacks fire when messages arrive. On success, we insert `SpacetimeConnection` as a Bevy resource, that's the signal to the rest of the game that a live connection exists.

Mini System 2: Processing Messages Every Frame

```

// src/network/connection.rs (continued)

pub fn process_spacetimedb_messages(connection: Res<SpacetimeConnection>) {
    if let Err(e) = connection.conn.frame_tick() {
        error!("SpacetimeDB frame_tick error: {e}");
    }
}

```

`frame_tick()` processes all WebSocket messages that have arrived since the last call and fires the corresponding callbacks. Call it once per frame and your client stays sync with everything happening on the server.

When you connect for the first time, the server runs `identity_connected` and inserts a new `Player` row. That insert is packaged as a message and sent back over the WebSocket. The next time `frame_tick()` runs, it picks up that message and writes the new row into the **client-side cache**, an in-memory copy of the subscribed table rows that the SDK maintains automatically. Once it's in the cache, you can read it instantly with `ctx.db.player().identity().find(...)`, no network round-trip needed.

Later milestones will add more messages: other players' positions updating, chat arriving, world state syncing. All of them come through this same call.

`resource_exists::<SpacetimeConnection>` guards it: if we haven't connected yet or have already cleaned up, the system is skipped.

But `process_spacetimedb_messages` doesn't do anything except check for errors, so it's not storing anything, right?

It looks that way, but the storage is happening inside `frame_tick()` itself. When a message arrives, say, a new `Player` row was inserted, the SDK processes it and writes the row into the client-side cache automatically. You don't write any code for that part; it's built into the SDK. `frame_tick()` is the trigger that lets the SDK do its work. The `if let Err(e)` is just there to surface any problems. In future milestones, you'll also register row callbacks (`on_insert`, `on_update`) that fire during this same call when you need to react to specific changes, like spawning a Bevy entity for a player who just joined.

Why `frame_tick()` instead of running the SDK on its own thread?

The SDK offers `run_threaded()` as an alternative, which spawns a background thread that processes messages automatically. But callbacks would then fire on that thread, not the Bevy main thread, making it unsafe to access Bevy resources or entities from inside them.

`frame_tick()` keeps everything on the main thread. Callbacks fire during this call, in the same frame as everything else. In future milestones, when a player moves and we need to update their Bevy transform component, the callback will be able to do that directly. No synchronization overhead, no `Arc<Mutex<...>>` wrappers.

Mini System 3: Multiplayer Connection Screen

While the connection is being established, we show a full-screen overlay with three pieces of information: who you're connecting as, who else is currently online.

To update those pieces independently each frame, we need a way to find the specific entities. That's what the three marker components below are for, each one tags a single node in the UI tree so `update_multiplayer_screen` can query and update them without touching the rest:

- `MultiplayerScreen` tags the root node so the entire screen can be despawned in one query when loading ends
- `ConnectionStatusText` tags the line that shows your own connection status
- `OnlinePlayersText` tags the line that lists every other player currently online

```
// src/network/connection.rs (continued)

#[derive(Component)]
pub struct MultiplayerScreen;

#[derive(Component)]
pub struct ConnectionStatusText;

#[derive(Component)]
pub struct OnlinePlayersText;

pub fn spawn_multiplayer_screen(mut commands: Commands) {
    commands
        .spawn((
            MultiplayerScreen,
            Node {
                width: Val::Percent(100.0),
                height: Val::Percent(100.0),
                justify_content: JustifyContent::Center,
                align_items: AlignItems::Center,
                flex_direction: FlexDirection::Column,
                ..default()
            },
            BackgroundColor(Color::srgb(0.05, 0.05, 0.1)),
        ))
}
```

```

        .with_children(|parent| {
            parent.spawn((
                Text::new("Multiplayer"),
                TextFont { font_size: FontSize::Px(48.0), ..default() },
                TextColor(Color::srgb(0.8, 0.7, 1.0)),
                Node { margin: UiRect::bottom(Val::Px(40.0)), ..default() },
            ));
            parent.spawn((
                ConnectionStatusText,
                Text::new("Connecting..."),
                TextFont { font_size: FontSize::Px(24.0), ..default() },
                TextColor(Color::WHITE),
                Node { margin: UiRect::bottom(Val::Px(20.0)), ..default() },
            ));
            parent.spawn((
                OnlinePlayersText,
                Text::new(""),
                TextFont { font_size: FontSize::Px(18.0), ..default() },
                TextColor(Color::srgb(0.7, 0.9, 0.7)),
                Node { margin: UiRect::bottom(Val::Px(40.0)), ..default() },
            ));
            parent.spawn((
                Text::new("Press Backspace to return to Main Menu"),
                TextFont { font_size: FontSize::Px(18.0), ..default() },
                TextColor(Color::srgba(0.6, 0.6, 0.6, 0.8)),
            ));
        });
    }
}

```

`spawn_multiplayer_screen` builds a full-screen flex column: a title, your connection status, a green-tinted list of who else is online, and a hint at the bottom. The `OnlinePlayersText` node starts empty, it won't show anything until a connection is established and other players are found in the local cache.

Now we need a system that updates both text nodes every frame:

```

// src/network/connection.rs (continued)

pub fn update_multiplayer_screen(
    connection: Option<Res<SpacetimeConnection>>,
    mut status_query: Query<
        &mut Text,
        (With<ConnectionStatusText>, Without<OnlinePlayersText>),
    >,
    mut online_query: Query<
        &mut Text,
        (With<OnlinePlayersText>, Without<ConnectionStatusText>),
    >,
) {
    let local_identity = connection.as_ref().and_then(|c| c.conn.try_identity());

    let status = if let Some(conn) = &connection {
        if let Some(identity) = local_identity {
            if let Some(player) = conn.conn.db.player().identity().find(&identity) {
                format!("Connected as: {}", player.username)
            } else {
                "Connected, waiting for player data...".to_string()
            }
        } else {
            "Authenticating...".to_string()
        }
    } else {
        "Connecting...".to_string()
    };
}

```

```

let online_list = if let Some(conn) = &connection {
    let mut others: Vec<String> = conn
        .conn
        .db
        .player()
        .iter()
        .filter(|p| p.is_online && Some(p.identity) != local_identity)
        .map(|p| p.username)
        .collect();
    others.sort();

    if others.is_empty() {
        "No other players online".to_string()
    } else {
        let mut s = String::from("Online players:");
        for name in others {
            s.push_str("\n- ");
            s.push_str(&name);
        }
        s
    }
} else {
    String::new()
};

for mut text in status_query.iter_mut() {
    text.0 = status.clone();
}
for mut text in online_query.iter_mut() {
    text.0 = online_list.clone();
}
}

```

The function updates two separate text nodes, so it declares two queries. Both ask for `&mut Text`, which makes Bevy nervous, it can't tell at a glance whether the same entity might match both and end up mutably borrowed twice. The `Without<>` on each query is how we reassure it: `status_query` only matches entities that have `ConnectionStatusText` but not `OnlinePlayersText`, and vice versa. They're guaranteed to be different entities, so both queries are safe to run at the same time.

But why aren't the `With<>` filters using the distinct tags `ConnectionStatusText` and `OnlinePlayersText` enough?

Because Bevy checks for conflicts at schedule-build time, before your game has spawned a single entity. At that point it only looks at the query type signatures, not at what you actually created. It sees two queries both asking for `&mut Text` and asks: *could* an entity exist that satisfies both filters at the same time? `With<ConnectionStatusText>` only says what an entity must have, it says nothing about what it must not have. So an entity carrying `Text + ConnectionStatusText + OnlinePlayersText` would match both queries simultaneously, giving you two mutable references to the same `Text` component. Bevy can't rule that out from the types alone, so it panics.

The `Without<>` closes that gap. Query A requires `Without<OnlinePlayersText>`, Query B requires `With<OnlinePlayersText>`. Those two conditions can never both be true on the same entity, it's a logical impossibility Bevy can verify from the type signatures. That's the proof it needs to allow both queries in the same system.

The status string builds up through the connection stages shown in the table below. The online list iterates all rows in the client cache, keeps only the ones where `is_online` is true and the identity isn't yours, sorts by name, and joins them into a list. If nobody else is online it shows "No other players online" instead of leaving the space blank.

The status transitions through three phases every session:

Status shown	What it means
Connecting...	SpacetimeConnection resource not yet inserted, connection still being built
Authenticating...	WebSocket open, waiting for server to confirm identity
Connected as: brave-purple-fox	Identity confirmed, player row found in client cache

Mini System 4: Back to Main Menu

```
// src/network/connection.rs (continued)

pub fn handle_multiplayer_back(
    input: Res<ButtonInput<KeyCode>>,
    mut next_state: ResMut<NextState<GameState>>,
) {
    if input.just_pressed(KeyCode::Backspace) {
        next_state.set(GameState::MainMenu);
    }
}
```

Pressing Backspace transitions back to `GameState::MainMenu`, which triggers `cleanup_network`. When the `Loading` state exits, the screen is removed:

```
// src/network/connection.rs (continued)

pub fn despawn_multiplayer_screen(
    mut commands: Commands,
    query: Query<Entity, With<MultiplayerScreen>>,
) {
    for entity in query.iter() {
        commands.entity(entity).despawn();
    }
}
```

Mini System 5: Cleanup on Return to Menu

```
// src/network/connection.rs (continued)

pub fn cleanup_network(mut commands: Commands, connection: Res<SpacetimeConnection>) {
    let _ = connection.conn.disconnect();
    commands.remove_resource::<SpacetimeConnection>();
    commands.insert_resource(GameMode::SinglePlayer);
    info!("Network cleaned up");
}
```

This runs when we enter `GameState::MainMenu` from a multiplayer session. It:

- 1. Disconnects cleanly (the server’s `identity_disconnected` reducer fires, marking the player offline)
- 2. Removes `SpacetimeConnection` from the world so `process_spacetimedb_messages` no longer runs
- 3. Resets `GameMode` back to `SinglePlayer` so “New Game” works normally after

Wiring Up

Four changes to `src/main.rs`:

```
// src/main.rs

mod map;
mod characters;
mod state;
mod collision;
mod config;
mod inventory;
mod camera;
mod combat;
mod particles;
mod enemy;
mod save;
mod audio;
mod module_bindings; // ← add this
mod network;          // ← add this

// ...

fn main() {
    App::new()
        // ... existing plugins ...
        .add_plugins(audio::AudioManagerPlugin)
        .add_plugins(network::NetworkPlugin) // ← add this
        .add_systems(Startup, prepare_tilemap_handles_resource)
        // Gate map generation: only run in single-player
        .add_systems(OnEnter(GameState::Loading),
            setup_generator.run_if(not(state::in_multiplayer))) // ← add run_if
        .add_systems(Update,
            poll_map_generation
                .run_if(in_state(GameState::Loading))
                .run_if(not(state::in_multiplayer))) // ← add run_if
        .run();
}
```

In multiplayer mode, the map doesn't exist yet. We'll address that in a later chapter when we build the shared world.

Let's run it, both in debug and release mode, so you can see two players connected to the server. Ensure to click on multiplayer on both.

```
cargo run
```

```
cargo run --release
```

Multiplayer

Connected as: tacitly-purifying-rockfish

Online players:
- stupidly-dauntless-swan

Press Backspace to return to Main Menu

In the upcoming chapter, we'll see how we can create a shared multiplayer world and sync player movements.

Stay tuned for more chapters, including Multiplayer, AI-driven NPCs, ...

The Impatient Programmer's Guide to Bevy and Rust: Chapter 13 - Let There Be a Shared World

Multiplayer Map Generation

In the last chapter, players could connect to the same server and see each other's names. But the world itself was still local. Every client was generating its own random map.

That means two players could be in the same multiplayer session, synced through the same server, but standing in completely different worlds. Player A might see a lake. Player B might see a desert. Their positions are shared, but the ground under their feet is not.

Back in Chapter 2, we used Wave Function Collapse to generate the map. WFC starts with an empty grid and fills it tile by tile. Each tile has to fit with the tiles around it.

The important detail is randomness. When WFC has more than one valid tile to choose from, it picks one randomly. That randomness comes from a seed: a number given to the random generator at the start. The seed decides the sequence of random numbers the generator will produce.

Same seed, same choices, same map.

In single-player, we used `RngMode::RandomSeed`, which picks a fresh seed every time the game runs. That is exactly what we want there: a new world each run.

In multiplayer, we need every client to start from the same seed, so everyone generates the same map.

The Problem

When two players start a multiplayer session, each of their machines picks its own random number and runs WFC independently. Player A's machine might pick `3841927465` and generate a forest with a lake. Player B's machine picks `9173654820` and generates a desert with a river. Even if we manage to sync player positions they will be standing in completely different worlds.

Player A walks into a rock. From Player B's view, A is standing in an open field.

Co-op works! Meet me by the lake!



What lake? I'm standing in a desert.



The big one. You're right next to it.



I'm next to a cactus. You're standing inside the cactus.

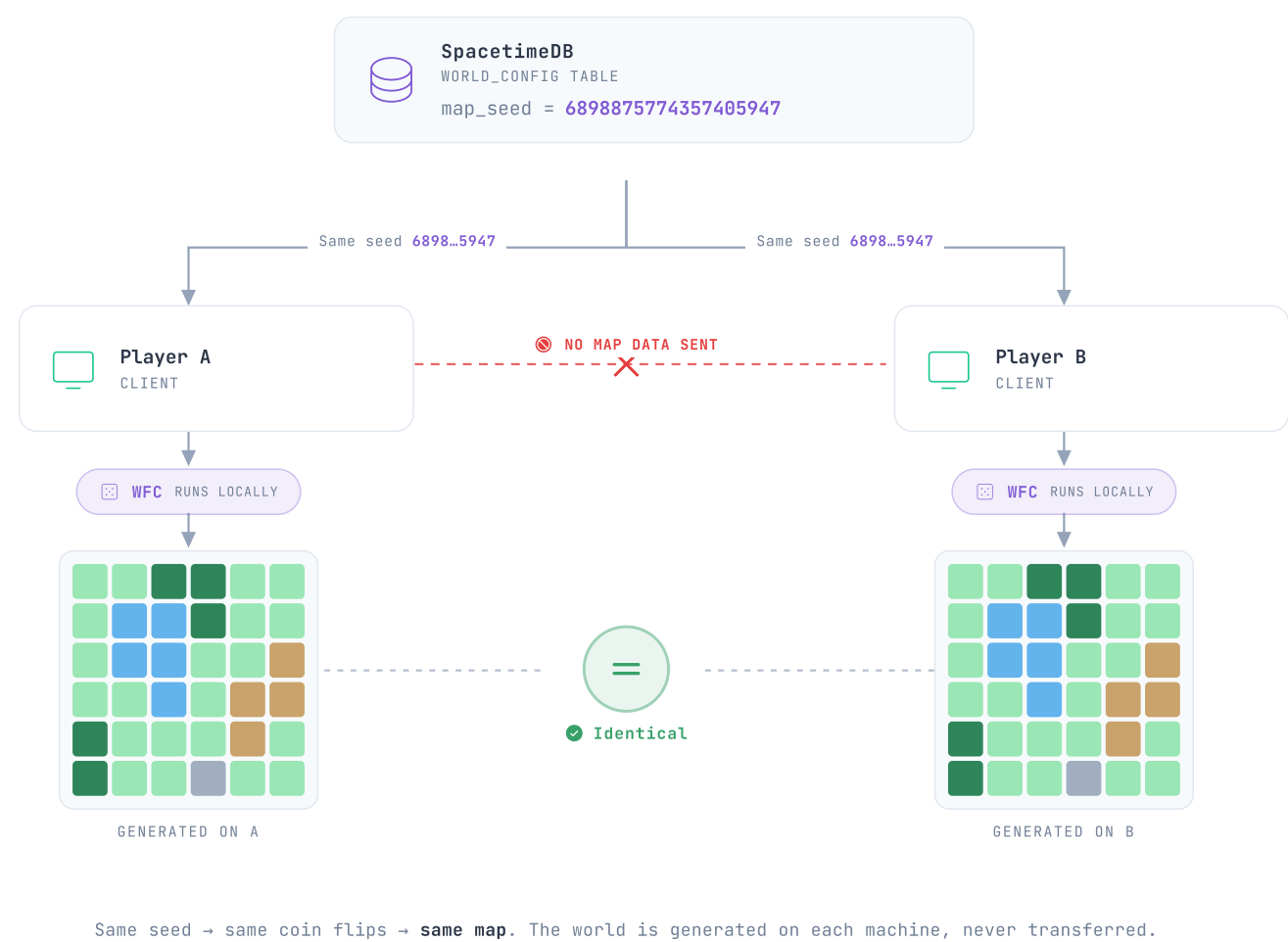


...You definitely forgot to install the lake.



The Solution

The fix is simple, just make both players use the same starting number. If both WFC generators start with `6898875774357405947`, they'll both produce the exact same map. Every tree in the same spot, every water tile in the same place. Neither machine sends map data to the other. They just both run the same algorithm from the same starting point.



So where does that shared number come from?

When we publish a SpacetimeDB module for the first time, it calls an `init` reducer exactly once. That's where we generate a random number and save it to a table. Every client that connects subscribes to that table and gets the number. The world seed exists in the database before any player ever connects.

Single-player stays exactly as-is, still generates a random seed locally. Multiplayer reads from the server.

Setup World Configuration

In SpacetimeDB, database tables are declared right inside the Rust module, the same way we defined the `Player` table back in Chapter 12. We're adding a new table, `WorldConfig`, that stores the world's settings in the server's database, plus an `init` reducer that fills it with a seed the first time the module is published.

```
server/src/lib.rs

Update server/src/lib.rs

+use spacetimedb::rand::RngCore;
    | Gives us next_u64() so the server can create a seed.

// ... Player table and reducers from Chapter 12 ...

THE WORLD CONFIG TABLE
| A SpacetimeDB table, just like Player in Chapter 12, but this one lives in the server's database to hold the
| world's settings.

+#[spacetimedb::table(accessor = world_config, public)]
```

`public` means every connected client receives this row automatically when it subscribes.

```
+ pub struct WorldConfig {  
+     #[primary_key]
```

We only ever store one row; `id = 0` is its key. A common pattern for global server config: one table, one row.

```
+     id: u32,  
+     map_seed: u64,
```

The shared seed. Every client reads this same number, so everyone runs WFC identically.

```
+     world_width: u32,  
+     world_height: u32,  
+ }
```

UPDATE THE INIT REDUCER

Chapter 12 already had an `init` reducer. Now we change it from "just log startup" into "create and store the world seed".

```
- #[spacetime::reducer(init)]  
- pub fn init(_ctx: &ReducerContext) {  
-     log::info!("Server module initialized");  
- }
```

```
+ #[spacetime::reducer(init)]  
+ pub fn init(ctx: &ReducerContext) {  
+     let seed = ctx.rng().next_u64();
```

SpacetimeDB's own server-side RNG picks the seed.

```
+     ctx.db.world_config().insert(WorldConfig {
```

Write the one and only row into the table.

```
+         id: 0,  
+         map_seed: seed,  
+         world_width: 10,  
+         world_height: 10,  
+     });  
+     log::info!("Server module initialized with world seed {}", seed);  
+ }
```

Rebuild and Republish

We changed `init`, so we need to wipe the database and let it run fresh:

```
cd server  
spacetime build  
spacetime publish --server http://127.0.0.1:3000 --delete-data -y bevy-game
```

`--clear-database` is the key flag here. Without it, SpacetimeDB sees an existing database and skips `init` entirely, which means no seed gets generated. You'll need this flag any time you change what `init` does.

Check the logs to make sure it ran:

```
spacetime logs --server http://127.0.0.1:3000 bevy-game -f
```

You should see something like: `Server module initialized with world seed 6898875774357405947` (yours will be different).

And you can peek at the table directly:

```
spacetime sql --server http://127.0.0.1:3000 bevy-game "SELECT * FROM world_config"
```

You should see something like the following:

```
id | map_seed          | world_width | world_height
----+-----+-----+-----
0  | 6898875774357405947 | 10          | 10
```

Good. The seed is in the database. Now the client needs to know about it.

Regenerate Client Bindings

Run `spacetime generate` from inside the `server/` directory (replace `{ADD_FULL_PATH_TO_CHAPTER13}` with the actual path to your chapter 13 folder):

```
cd server
spacetime generate --lang rust \
  --out-dir {ADD_FULL_PATH_TO_CHAPTER13}/src/module_bindings \
  --module-path .
```

You'll see two new files alongside the ones from Chapter 12:

```
Writing file src/module_bindings/world_config_table.rs
Writing file src/module_bindings/world_config_type.rs
```

These files add the client-side support for `world_config`. One helper lets the client ask SpacetimeDB to send the table. The other lets the client read that table after the data arrives. We'll use both shortly.

Create the WorldSeed Resource

The seed now lives in the database, but the Bevy client still needs an in-memory place to keep the seed it will use for map generation.

That's what `WorldSeed` is: a local Bevy resource. Single-player fills it with a local random seed. Multiplayer fills it with the seed received from SpacetimeDB. After that, the map generator reads `WorldSeed` the same way in both modes.

Create a new file `src/map/seed.rs`:

```
src/map/seed.rs

Create src/map/seed.rs

use bevy::prelude::*;
use rand::rngs::StdRng;
use rand::{RngCore, SeedableRng};

#[derive(Resource)]
pub struct WorldSeed(pub u64);

pub fn init_single_player_seed(mut commands: Commands) {
    let seed = StdRng::from_entropy().next_u64();
```

```
info!("Single-player world seed: {}", seed);
commands.insert_resource(WorldSeed(seed));
}
```

`init_single_player_seed` handles the single-player case: ask the OS for some entropy or randomness and add it in `WorldSeed`.

Add it to `src/map/mod.rs`:

```
src/map/mod.rs

pub mod assets;
pub mod generate;
pub mod rules;
pub mod models;
pub mod sockets;
pub mod tilemap;
+ pub mod seed;
```

Make WFC Deterministic

The map generator currently picks its own random seed internally. We need to change that so it uses whatever's in `WorldSeed`.

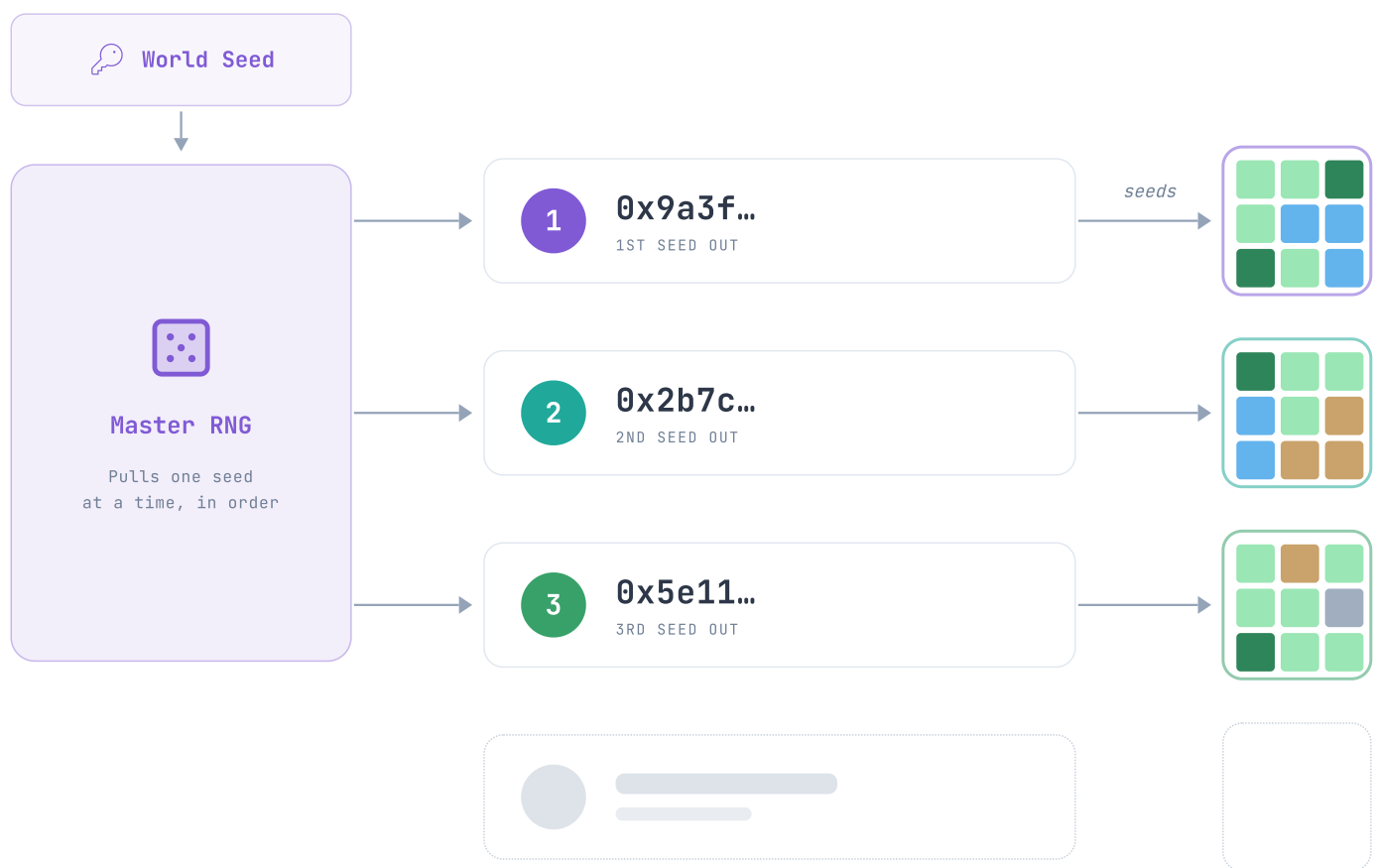
Three functions need changing. Here's how those functions fit together.

`setup_generator` is the entry point. Once `WorldSeed` exists, we want it to read the seed, prepare the shared WFC rules, then start the background generation task.

That task runs `generate_all_chunks`, and the first thing we want it to do with the world seed is build one master random number generator from it.

Master random number generator gives us a fixed sequence of numbers. Seed it once, then every time we ask it for the next number, it hands back the same one, in the same order. Same seed, same sequence.

The map is built one chunk at a time, so we need one seed per chunk, and we'll pull them straight off that sequence. For each chunk, we want `generate_all_chunks` to ask the master generator for its next number and use that as the chunk's seed. The first chunk takes the first number off the stream, the second chunk the next, and so on. That way one world seed fans out into an ordered stream of per-chunk seeds.



Each of those chunk seeds goes to `try_generate_chunk`, where the WFC library actually runs for that chunk.

If two clients start with the same `WorldSeed`, they build the same master seed stream, give each chunk attempt the same seed, and generate the same map.

Let's change `setup_generator` to read `WorldSeed` and pass the value into the background task. We also add the `rand` imports at the top of the file.

```

src/map/generate.rs

Update src/map/generate.rs

+ use crate::map::seed::WorldSeed;
    | Bring in the Bevy resource that stores the world seed.
+ use rand::rngs::StdRng;
    | The seedable RNG we build the master seed stream from.
+ use rand::{RngCore, SeedableRng};
    | SeedableRng provides seed_from_u64; RngCore provides next_u64.

pub fn setup_generator(
    mut commands: Commands,
    tilemap_handles: Res<TilemapHandles>,
+   world_seed: Res<WorldSeed>,
    | Read the seed from Bevy instead of creating one inside the generator.
) {
    // ... build rules, grid_template, spawner as before ...

SEED THE BACKGROUND TASK
    | Pull the number out of WorldSeed, then move it into the async task so chunk generation can stay deterministic.

+   let seed = world_seed.0;
+   info!("Starting map generation with seed {}", seed);
    let pool = AsyncComputeTaskPool::get();
    let task = pool.spawn(async move {
-   generate_all_chunks(rules_arc, grid_template, progress)
+   generate_all_chunks(rules_arc, grid_template, progress, seed)

```

```
| | Pass the world seed into the chunk generator.
});
commands.insert_resource(MapGenTask(task));
}
```

Now let's modify the `generate_all_chunks` function to take the seed and build a master RNG from it. Each chunk should get its own seed pulled from that master RNG:

Update src/map/generate.rs

```
fn generate_all_chunks(
    rules_arc: Arc<Rules<Cartesian3D>>,
    grid_template: CartesianGrid<Cartesian3D>,
    progress: Arc<AtomicU32>,
+   world_seed: u64,
    | | The world seed now comes from setup_generator .
) -> Vec<ChunkResult> {

    MASTER SEED STREAM

    | | Turn the one world seed into a repeatable stream of chunk seeds.
+   let mut master_rng = StdRng::seed_from_u64(world_seed);

    // ... chunk loop ...

    while index < chunk_order.len() {
        let (cx, cy) = chunk_order[index];
        let initial_nodes = build_initial_nodes(cx, cy, &generated_chunks, &grid_template);
+       let chunk_seed = master_rng.next_u64();

        | | Pull the next deterministic seed for this chunk attempt.

-       if let Some(grid_data) = try_generate_chunk(&rules_arc, &grid_template, &initial_nodes) {
+       if let Some(grid_data) =
+       try_generate_chunk(&rules_arc, &grid_template, &initial_nodes, chunk_seed)

        | | Give the chunk seed to WFC.
        {
            // ...
        }
        // ... backtracking logic unchanged ...
    }
}
```

Why not just do `world_seed ^ chunk_index` to get a seed for each chunk?

The reason is backtracking. If a chunk fails to generate, usually because it can't satisfy strict seam constraints with its neighbour, we roll back to an earlier chunk and try again. With `^chunk_index`, we'd retry with the exact same seed and fail in the exact same way, forever.

With the master RNG approach, each attempt (including retries) consumes a fresh number from the stream. Two clients pulling from the same master RNG in the same order will both backtrack the same number of times and land on the same seeds. Same map.

Finally, `try_generate_chunk` is where we hand control to the WFC generator for one chunk. Earlier, we let the library pick a random seed for itself. Now, we pass in the chunk seed from `generate_all_chunks` and tell the library to use that exact seed:

Update src/map/generate.rs

```
fn try_generate_chunk(
```

```

rules: &Arc<Rules<Cartesian3D>>,
grid: &CartesianGrid<Cartesian3D>,
initial_nodes: &[(u32, u32, u32), ModelInstance]],
+ seed: u64,
| | The per-chunk seed from generate_all_chunks.
) -> Option<GridData<Cartesian3D, ModelInstance, CartesianGrid<Cartesian3D>>> {
    // ... border_zones setup unchanged ...

    let gen_builder = GeneratorBuilder::new()
        .with_shared_rules(rules.clone())
        .with_grid(grid.clone())
-     .with_rng(RngMode::RandomSeed)
+     .with_rng(RngMode::Seeded(seed))
| | Force WFC to use the seed we pass in.
    // ...
}

```

Now we need to connect `WorldSeed` to the app schedule.

In single-player, the seed can be created as soon as the game enters `Loading`, because it comes from the local machine. In multiplayer, we must wait until the network code receives the shared seed from SpacetimeDB.

So `setup_generator` should no longer run just because the game is loading. It should run only when the game is loading **and** `WorldSeed` exists. That makes both paths use the same rule: no seed, no map generation.

Update `src/main.rs` to import the seed module, create the single-player seed on loading, and gate `setup_generator` behind `WorldSeed`.

```

src/main.rs

Update src/main.rs

+ use crate::map::seed::{init_single_player_seed, WorldSeed};
+ use crate::map::generate::{MapReady, MapSpawnResources};

fn main() {
    App::new()
        // ... existing plugins ...
-     .add_systems(
-         OnEnter(GameState::Loading),
-         setup_generator.run_if(not(state::in_multiplayer)),
-     )
+     .add_systems(
+         OnEnter(GameState::Loading),
+         init_single_player_seed.run_if(not(state::in_multiplayer)),
| | Single-player creates a local seed as soon as loading starts.
+     )
+     .add_systems(
+         Update,
+         setup_generator
+             .run_if(in_state(GameState::Loading))
+             .run_if(resource_exists::<WorldSeed>)
| | Do not start map generation until a seed exists.
+             .run_if(not(resource_exists::<MapSpawnResources>))
+             .run_if(not(resource_exists::<MapReady>)),
+     )
-     .add_systems(
-         Update,
-         poll_map_generation
-             .run_if(in_state(GameState::Loading))

```

```

-         .run_if(not(state::in_multiplayer)),
-     )
+     .add_systems(
+         Update,
+         poll_map_generation.run_if(in_state(GameState::Loading)),
+         | | Poll in both modes; it only does work after generation starts.
+     )
    .run();
}

```

In Chapter 12, we blocked map generation during multiplayer with `not(in_multiplayer)` because multiplayer did not have a shared map yet. That is no longer the right condition.

Now the rule is simpler: map generation starts only when `WorldSeed` exists. In single-player, `WorldSeed` is created immediately. In multiplayer, it is created only after the player clicks Join. So `setup_generator` can run in both modes, as long as it waits for `WorldSeed`.

`poll_map_generation` can also run in both modes. Its job is to check whether the background map generation task has finished, then spawn the generated tiles when they are ready. Generation decides what the chunks should contain; spawning turns that generated data into actual Bevy entities in the world.

Fetching WorldConfig

The server has the shared seed now. Next, the client needs to receive that seed and keep it ready. We will not start map generation immediately; we will wait until the player clicks Join.

In Chapter 12, the client only subscribed to the `player` table. Now it also needs to subscribe to `world_config`.

We'll also add a Join button so the player can choose when to enter the shared world.

Add these to `src/network/connection.rs`:

```

src/network/connection.rs

Update src/network/connection.rs

use std::path::PathBuf;
+use std::sync::atomic::Ordering;
    | The map generator updates progress on a background thread; this lets the UI read that number safely.

use bevy::prelude::*;
use spacetimedb_sdk::{DbContext, Table};

+use crate::map::generate::{MapGenProgress, MapReady};
    | Used by the multiplayer screen to show generation progress and entering-world status.
+use crate::map::seed::WorldSeed;
    | The active seed resource. When this exists, map generation can start.

use crate::module_bindings::player_table::{PlayerTableAccess, playerQueryTableAccess};
+use crate::module_bindings::world_config_table::{
+    world_configQueryTableAccess, WorldConfigTableAccess,
    | Generated helpers for subscribing to and reading the world_config table.
+};

use crate::module_bindings::DbConnection;
use crate::state::{GameMode, GameState};

use super::SpacetimeConnection;

const SPACETIMEDB_URI: &str = "http://127.0.0.1:3000";
const DATABASE_NAME: &str = "bevy-game";

```

```

const TOKEN_FILENAME: &str = "spacetimedb_token";

// ... existing connection helpers and components ...

+ /// Seed received from the server but not yet applied.
+ /// Promoted to `WorldSeed` when the player clicks Join.
+ #[derive(Resource)]
+ pub struct PendingWorldSeed(pub u64);
    | Holds the server seed while the Join button is still waiting for the player.

+
+ #[derive(Component)]
+ pub struct JoinButton;
    | Marks the button the player clicks when they are ready to enter the shared world.

```

We're using two separate resources because the seed has two different states in the client.

When the seed arrives, we store it in `PendingWorldSeed`: the client has the seed, but the player has not joined yet. When the player clicks Join, we move that value into `WorldSeed`, which allows map generation to start.

Why not store the server seed directly in `WorldSeed`?

Because we use `WorldSeed` as a signal to start map generation. If we inserted `WorldSeed` as soon as the seed arrived, the map would start generating immediately. `PendingWorldSeed` gives us a waiting state, so the Join button controls when generation begins.

But before we can store anything in `PendingWorldSeed`, the client has to receive the seed from SpacetimeDB.

When the client connects, it tells SpacetimeDB which tables it wants to receive. We already ask for `player`; now we also ask for `world_config` so the seed is sent to the client.

Add `world_config` to the subscription:

src/network/connection.rs

Update src/network/connection.rs

```

pub fn connect_to_spacetimedb(mut commands: Commands) {
    let token = load_token();

    let conn = DbConnection::builder()
        .with_uri(SPACETIMEDB_URI)
        .with_database_name(DATABASE_NAME)
        .with_token(token)
        .on_connect(|ctx, _identity, token| {
            if let Err(e) = save_token(token) {
                error!("Failed to save SpacetimeDB token: {e}");
            }
            info!("Connected to SpacetimeDB");
            ctx.subscription_builder()
                .on_applied(|ctx| {
                    if let Some(identity) = ctx.try_identity() {
                        if let Some(player) = ctx.db.player().identity().find(&identity) {
                            info!("Playing as: {}", player.username);
                            return;
                        }
                    }
                    info!("Player subscription applied");
                })
                .on_error(|_ctx, err| {
                    error!("Subscription error: {err}");
                })
                .add_query(|q| q.from(player()))

```

```

+ .add_query(|q| q.from.world_config())
|
| Also subscribe to the server's shared world seed.
|
| .subscribe();
|
| })
| .on_connect_error(|_ctx, err| {
|     error!("SpacetimeDB connection error: {err}");
| })
| .on_disconnect(|_ctx, err| {
|     if let Some(e) = err {
|         warn!("Disconnected from SpacetimeDB with error: {e}");
|     } else {
|         info!("Disconnected from SpacetimeDB");
|     }
| })
| .build();
|
| // ... match conn unchanged ...
}

```

Subscribing to `world_config` asks SpacetimeDB to send the row, but it does not appear on the client immediately. It arrives over the network shortly after the connection is running.

So we add a small system that checks whether the row has arrived yet. Once it finds the `world_config` row, it copies the seed into `PendingWorldSeed`:

src/network/connection.rs

Append this to src/network/connection.rs

```

pub fn fetch_world_seed(
    mut commands: Commands,
    connection: Res<SpacetimeConnection>,
) {
    if let Some(config) = connection.conn.db.world_config().id().find(&0) {
        |
        | Look for the single world_config row in the client's local cache.
        |
        | info!("Received multiplayer world seed: {}", config.map_seed);
        | commands.insert_resource(PendingWorldSeed(config.map_seed));
        |
        | Store the seed as pending until the player clicks Join.
        |
    }
}

```

This reads from the client-side cache, not directly from the server. Once the subscribed row has arrived, the system stores the seed in `PendingWorldSeed`. Later, the Join button will move that value into `WorldSeed` and start map generation.

Shared World Generation

At this point the client can receive the seed and store it as `PendingWorldSeed`. Now we need to give the player a way to say, “I’m ready, start building the world.”

We’ll add a Join button to the multiplayer screen, but keep it hidden at first. It should only appear after the seed has arrived.

Update `spawn_multiplayer_screen` to include the hidden Join button:

src/network/connection.rs

Update spawn_multiplayer_screen in src/network/connection.rs

```

pub fn spawn_multiplayer_screen(mut commands: Commands) {
    commands

```



```

        .spawn((
            MultiplayerScreen,
            Node {
                width: Val::Percent(100.0),
                height: Val::Percent(100.0),
                justify_content: JustifyContent::Center,
                align_items: AlignItems::Center,
                flex_direction: FlexDirection::Column,
                ..default()
            },
            BackgroundColor(Color::srgb(0.05, 0.05, 0.1)),
        ))
        .with_children(|parent| {
            parent.spawn((
                Text::new("Multiplayer"),
                TextFont { font_size: FontSize::Px(48.0), ..default() },
                TextColor(Color::srgb(0.8, 0.7, 1.0)),
                Node { margin: UiRect::bottom(Val::Px(40.0)), ..default() },
            ));

            parent.spawn((
                ConnectionStatusText,
                Text::new("Connecting..."),
                TextFont { font_size: FontSize::Px(24.0), ..default() },
                TextColor(Color::WHITE),
                Node { margin: UiRect::bottom(Val::Px(20.0)), ..default() },
            ));

            parent.spawn((
                OnlinePlayersText,
                Text::new(""),
                TextFont { font_size: FontSize::Px(18.0), ..default() },
                TextColor(Color::srgb(0.7, 0.9, 0.7)),
                Node { margin: UiRect::bottom(Val::Px(40.0)), ..default() },
                Node { margin: UiRect::bottom(Val::Px(30.0)), ..default() },
            ));
        });

```

HIDDEN JOIN BUTTON

The button exists in the UI tree now, but starts invisible. Another system will show it after the seed arrives.

```

+         parent
+         .spawn((
+             JoinButton,
+
+             | Marks this as the button that lets the player enter the shared world.
+
+             Button,
+             Node {
+                 display: Display::None,
+
+                 | Hidden until PendingWorldSeed exists.
+
+                 width: Val::Px(220.0),
+                 height: Val::Px(55.0),
+                 justify_content: JustifyContent::Center,
+                 align_items: AlignItems::Center,
+                 margin: UiRect::bottom(Val::Px(30.0)),
+                 ..default()
+             },
+             BackgroundColor(Color::srgba(0.15, 0.3, 0.15, 0.9)),
+         ))
+         .with_children(|btn_parent| {
+             btn_parent.spawn((
+                 Text::new("Join"),
+                 TextFont { font_size: FontSize::Px(28.0), ..default() },
+                 TextColor(Color::WHITE),
+             ));
+         });

```

```
+
    });

    parent.spawn((
        Text::new("Press Backspace to return to Main Menu"),
        TextFont { font_size: FontSize::Px(18.0), ..default() },
        TextColor(Color::srgba(0.6, 0.6, 0.6, 0.8)),
    ));
});
}
```

The button is spawned with `Display::None`, so it starts invisible. Now we need a system that decides when it should appear.

It's simple, show Join after the seed has arrived, but before map generation has started. In code terms, that means `PendingWorldSeed` exists and `WorldSeed` does not.

```

Append this to src/network/connection.rs

pub fn update_join_button(
    pending: Option<Res<PendingWorldSeed>>,
    world_seed: Option<Res<WorldSeed>>,
    mut query: Query<&mut Node, With<JoinButton>>,
) {
    let should_show = pending.is_some() && world_seed.is_none();
    for mut node in query.iter_mut() {
        node.display = if should_show {
            Display::Flex
        } else {
            Display::None
        };
    }
}
```

Showing the button only gives the player something to click. Now we need to make that click mean something.

When the player presses Join, we take the pending server seed and turn it into the active `WorldSeed`. That is the moment map generation is allowed to start.

```

Append this to src/network/connection.rs

pub fn handle_join_button(
    mut commands: Commands,
    interaction_query: Query<&Interaction, (Changed<Interaction>, With<JoinButton>>>,
    | Only react when the Join button interaction changes.

    pending: Option<Res<PendingWorldSeed>>,
    | The server seed may or may not have arrived yet.
) {
    for interaction in interaction_query.iter() {
        if *interaction != Interaction::Pressed {
            | Ignore hover and release events; only the actual press joins the world.

            continue;
        }
        let Some(pending) = pending.as_ref() else {
            | If the seed has not arrived, there is nothing to join yet.

            continue;
        };
        info!("Joining multiplayer world with seed {}", pending.0);
    }
}
```

```

        commands.insert_resource(WorldSeed(pending.0));
    | Promote the pending seed into the active seed that starts generation.
        commands.remove_resource::();
    | The seed is no longer waiting; the player has joined.
    }
}

```

Clicking Join turns the pending seed into the active `WorldSeed`. Since `setup_generator` waits for `WorldSeed`, map generation starts from there.

The Join button works now, but it should also feel like a real button. We'll reuse the same hover and pressed colour feedback pattern from the earlier menu buttons.

```

Append this to src/network/connection.rs

pub fn handle_join_button_hover(
    mut interaction_query: Query<
        (&Interaction, &mut BackgroundColor),
        (Changed<Interaction>, With<JoinButton>),
    >,
) {
    for (interaction, mut bg) in interaction_query.iter_mut() {
        *bg = match interaction {
            Interaction::Hovered => BackgroundColor(Color::srgba(0.25, 0.45, 0.25, 0.9)),
            Interaction::Pressed => BackgroundColor(Color::srgba(0.35, 0.6, 0.35, 0.9)),
            Interaction::None     => BackgroundColor(Color::srgba(0.15, 0.3, 0.15, 0.9)),
        };
    }
}

```

The status text now needs to show the new multiplayer states and the map generation progress. Let's update `update_multiplayer_screen`.

```

src/network/connection.rs

Update update_multiplayer_screen in src/network/connection.rs

pub fn update_multiplayer_screen(
    connection: Option<Res<SpacetimeConnection>>,
+   pending_seed: Option<Res<PendingWorldSeed>>,
    | Seed has arrived, but the player has not clicked Join yet.
+   world_seed: Option<Res<WorldSeed>>,
    | Player clicked Join; map generation is allowed to start.
+   progress: Option<Res<MapGenProgress>>,
    | Current chunk generation progress, if WFC is running.
+   map_ready: Option<Res<MapReady>>,
    | Map generation has finished.
    mut status_query: Query<
        &mut Text,
        (With<ConnectionStatusText>, Without<OnlinePlayersText>),
    >,
    mut online_query: Query<
        &mut Text,
        (With<OnlinePlayersText>, Without<ConnectionStatusText>),
    >,
) {
    let local_identity = connection.as_ref().and_then(|c| c.conn.try_identity());

```

```

- let status = if let Some(conn) = &connection {
+ let status = if world_seed.is_some() {
    | After Join, the status is about world generation, not connection setup.

+     if map_ready.is_some() {
+         "Entering world...".to_string()
+     } else if let Some(progress) = progress.as_ref() {
+         let current = progress.current.load(Ordering::Relaxed);

    | Read progress written by the background generation task.

+         format!("Generating world: {}/{} chunks", current, progress.total)
+     } else {
+         "Generating world...".to_string()
+     }
+ } else if let Some(conn) = &connection {
    if let Some(identity) = local_identity {
        if let Some(player) = conn.conn.db.player().identity().find(&identity) {
- format!("Connected as: {}", player.username)
+         if pending_seed.is_some() {
+             format!("Ready to join as: {}", player.username)
+         } else {
+             format!("Connected as: {}", player.username)
+         }
        } else {
            "Connected, waiting for player data...".to_string()
        }
    } else {
        "Authenticating...".to_string()
    }
} else {
    "Connecting...".to_string()
};

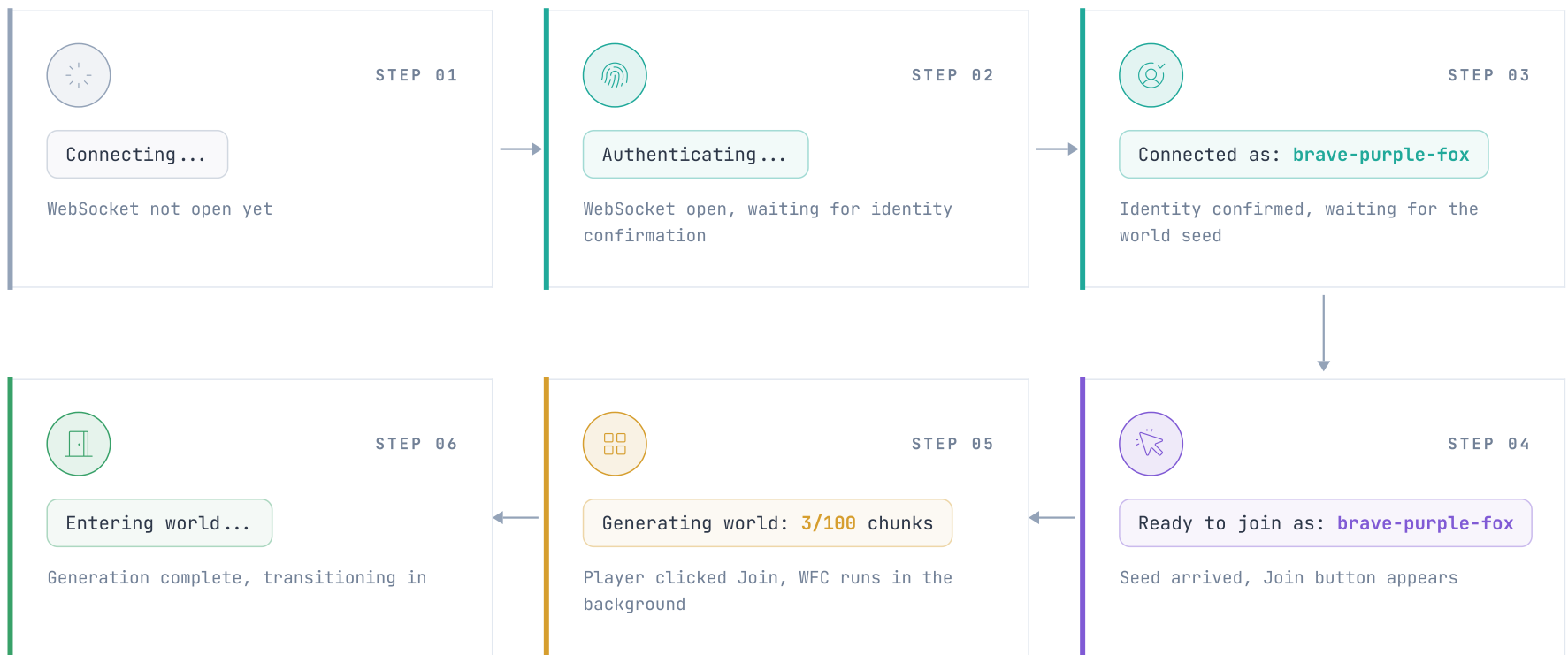
let online_list = if let Some(conn) = &connection {
    let mut others: Vec<String> = conn
        .conn
        .db
        .player()
        .iter()
        .filter(|p| p.is_online && Some(p.identity) != local_identity)
        .map(|p| p.username)
        .collect();
    others.sort();

    if others.is_empty() {
        "No other players online".to_string()
    } else {
        let mut s = String::from("Online players:");
        for name in others {
            s.push_str("\n- ");
            s.push_str(&name);
        }
        s
    }
} else {
    String::new()
};

for mut text in status_query.iter_mut() {
    text.0 = status.clone();
}
for mut text in online_query.iter_mut() {
    text.0 = online_list.clone();
}
}

```

Now every multiplayer session now goes through these stages:



We have written the pieces now: fetch the seed, show the Join button, handle the click, and update the status text. Now we need to register these systems in the network plugin.

Update `src/network/mod.rs` to export `PendingWorldSeed` and register everything:

```
src/network/mod.rs

mod connection;

use bevy::prelude::*;

use crate::module_bindings::DbConnection;
use crate::state::{in_multiplayer, GameState};

use connection::{
    cleanup_network, connect_to_spacetimedb, despawn_multiplayer_screen,
+   fetch_world_seed, handle_join_button, handle_join_button_hover,
    handle_multiplayer_back, process_spacetimedb_messages,
-   spawn_multiplayer_screen, update_multiplayer_screen,
+   spawn_multiplayer_screen, update_join_button, update_multiplayer_screen,
};
    Add the system that shows and hides the Join button.

};

+pub use connection::PendingWorldSeed;
    Re-exported so cleanup code in other modules can remove this resource.

#[derive(Resource)]
pub struct SpacetimeConnection {
    pub conn: DbConnection,
}

pub struct NetworkPlugin;

impl Plugin for NetworkPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(
            OnEnter(GameState::Loading),
```

```

        (connect_to_spacetimedb, spawn_multiplayer_screen).run_if(in_multiplayer),
    )
    .add_systems(
        Update,
        (
            process_spacetimedb_messages
                .run_if(resource_exists::<SpacetimeConnection>),
+         fetch_world_seed
+         .run_if(in_state(GameState::Loading))
+         .run_if(resource_exists::<SpacetimeConnection>)
+         .run_if(not(resource_exists::<PendingWorldSeed>)),
        |
        | Stop checking after the seed has been found.
        |
        update_multiplayer_screen.run_if(in_state(GameState::Loading)),
+       update_join_button.run_if(in_state(GameState::Loading)),
+       handle_join_button.run_if(in_state(GameState::Loading)),
+       handle_join_button_hover.run_if(in_state(GameState::Loading)),
        handle_multiplayer_back.run_if(in_state(GameState::Loading)),
    )
        .run_if(in_multiplayer),
    )
    .add_systems(
        OnExit(GameState::Loading),
        despawn_multiplayer_screen,
    )
    .add_systems(
        OnEnter(GameState::MainMenu),
        cleanup_network.run_if(resource_exists::<SpacetimeConnection>),
    );
}
}

```

`fetch_world_seed` has `not(resource_exists::<PendingWorldSeed>)` as a run condition. Without that, it would re-insert `PendingWorldSeed` on every single frame after the first successful read.

That handles the seed while the player is on the multiplayer screen. We also need to clean it up when the player leaves the current game and starts over.

That reset already happens in `cleanup_game_world` in `src/state/game_over.rs`, which runs when the player restarts after game over. Add `WorldSeed` and `PendingWorldSeed` to the resources it removes:

```

src/state/game_over.rs

Update src/state/game_over.rs

+ use crate::map::seed::WorldSeed;
    |
    | The active seed that may have started map generation.
+ use crate::network::PendingWorldSeed;
    |
    | The server seed waiting for the player to click Join.

pub fn cleanup_game_world(
    mut commands: Commands,
    // ... existing parameters ...
) {
    // ... existing entity despawning ...

    commands.remove_resource::<MapReady>();
+   commands.remove_resource::<WorldSeed>();
    |
    | Clear the active seed before the next game starts.
+   commands.remove_resource::<PendingWorldSeed>();
    |
    | Clear any seed that was received but not joined yet.
}

```


Start the server and run two game instances:



```
spacetime start  # keep this running
```



```
cargo run      # first instance
```



```
cargo run --release  # second instance
```

Click **Multiplayer** on both. You'll see both connections go through the status stages. When **Ready to join** appears on one instance, click **Join** and watch the world being generated.

Both maps will be identical. A player who joins three hours later will generate the same world as the players who were there from the start.

Players now share a world. But we still have a problem: the server doesn't know anything about the map. It doesn't know where walls are, so it can't tell if a player's movement is physically possible. Next chapter we'll build a collision grid on the server side so it can actually verify movement before accepting it.

The maps match! We're standing
in the exact same world!



Only took the whole chapter.



Same lake, same shore, same
everything. You know what that
means?



...Explore together? Some
fishing?



Push you in the lake. That's the dream.



Stay tuned for more chapters, including Multiplayer, AI-driven NPCs, ...